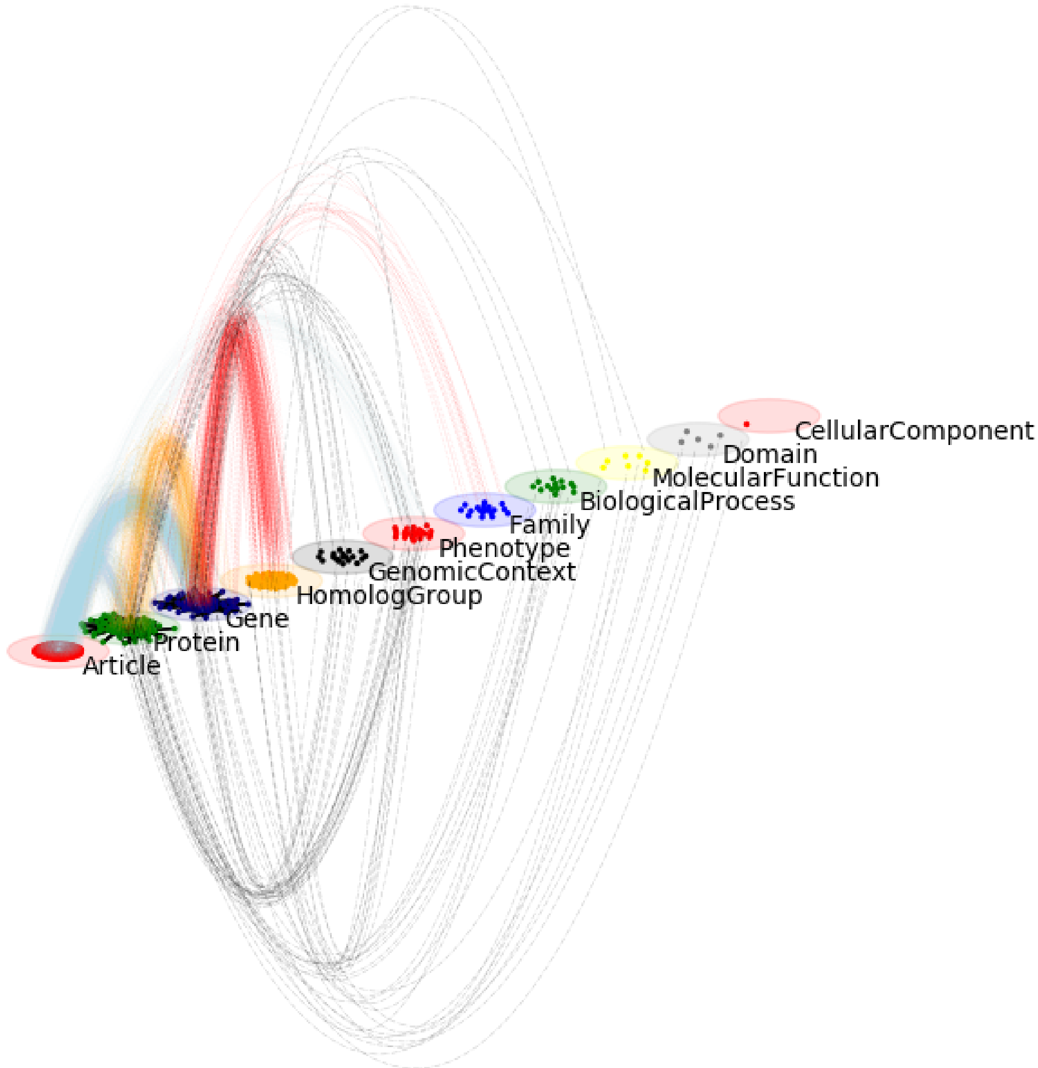


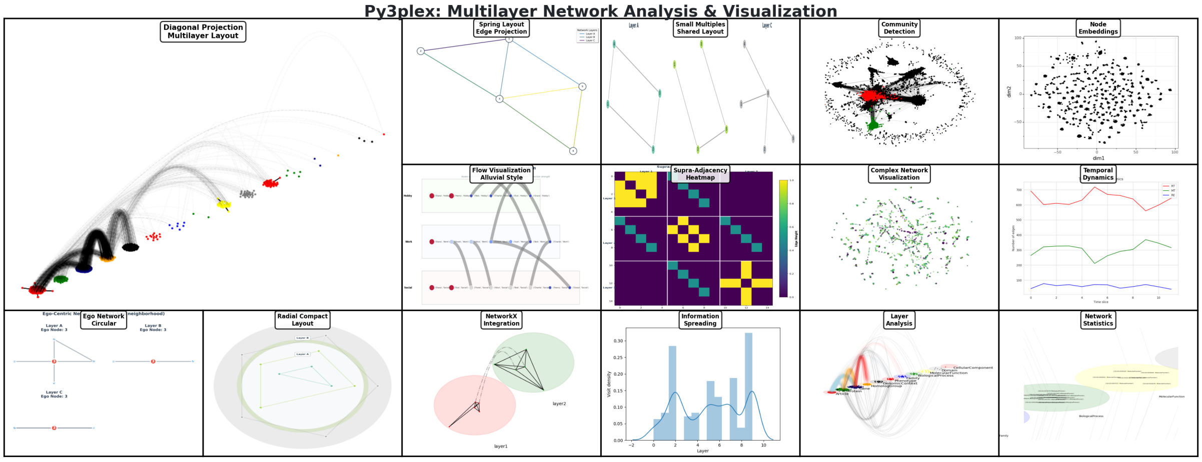
Py3plex: Multilayer Network Analysis in Python

Release 1.0.2

Blaž Škrlj

Dec 17, 2025

OVERVIEW



[1] [2]

Documentation Versions

You are viewing the documentation for py3plex 1.x (current stable version). These docs are actively maintained and updated.

Legacy documentation for py3plex 0.8x is still available at https://py3plex.readthedocs.io but is no longer updated. Most users should use the 1.x docs you're reading now.

Which version should you use? See *Which Docs Should I Use?* below for guidance.

---

py3plex provides scalable analysis and visualization of multilayer and multiplex networks in Python.

Key Features:

- Multiplex and multilayer network structures
- **SQL-like DSL for network queries** — first-class feature
- 17+ multilayer statistics and centrality measures
- Community detection (Louvain, Infomap, multilayer modularity)
- Random walk algorithms (Node2Vec, DeepWalk) for embeddings
- Publication-ready visualizations
- High-performance I/O with Arrow/Parquet support
- Full NetworkX compatibility

DSL: Query Networks Like SQL

The py3plex DSL lets you query networks using intuitive SQL-like syntax or a type-safe Python builder API:

```
from py3plex.dsl import execute_query, Q, L
```

[1] https://github.com/SkBlaz/py3plex/actions/workflows/tests.yml
[2] https://github.com/SkBlaz/py3plex/actions/workflows/code-quality.yml

```
# String DSL: Simple and readable
result = execute_query(network,
    'SELECT nodes WHERE layer="social" AND degree > 5 '
    'COMPUTE betweenness_centrality'
)

# Builder API: Type-safe with autocompletion
result = (
    Q.nodes()
     .from_layers(L["social"])
     .where(degree__gt=5)
     .compute("betweenness_centrality")
     .order_by("-betweenness_centrality")
     .limit(10)
     .execute(network)
)
```

The DSL is **perfect for**:

- Interactive network exploration
- Rapid prototyping
- Educational purposes
- Production pipelines

See the complete user_guide/dsl guide for all features!

CHAPTER

ONE

QUICKSTART

Install:

```
pip install py3plex
```

Create a multilayer network:

```python
from py3plex.core import multinet

network = multinet.multi_layer_network()
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1],
    ['Bob', 'friends', 'Carol', 'friends', 1],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1],
], input_type="list")

network.basic_stats()
network.visualize_network(show=True)
```

Expected output:

```
Number of nodes: 6
Number of edges: 3
```

CHAPTER

TWO

START HERE

New to py3plex? → Begin with the *10-Minute Tutorial* to go from basics to advanced analysis in 10 minutes.

Already familiar with multilayer networks? → Jump to *How to Run Community Detection on Multilayer Networks* or explore the *How-to Guides*.

Just need the API? → Go straight to *API Documentation* or *DSL Reference*.

Want to understand the concepts? → Read *Multilayer Networks 101* for the theory behind multilayer networks.

CHAPTER

THREE

DOCUMENTATION STRUCTURE

This documentation follows the Diátaxis[3] framework with 7 top-level sections:

3.1 Part I: Overview

High-level introduction to py3plex and multilayer networks.

3.1.1 Overview

Welcome to py3plex! This section provides a high-level introduction to the library and its capabilities.

What is py3plex?

py3plex is a Python library for analyzing and visualizing multilayer and multiplex networks. Unlike traditional network analysis tools that treat all relationships uniformly, py3plex preserves the distinct types of connections between entities—whether they're social ties, biological interactions, or transportation links.

Why py3plex?

- **Native multilayer support:** Built from the ground up for networks with multiple relationship types
- **SQL-like DSL:** Query networks intuitively with SQL-inspired syntax
- **Production-ready:** High-performance I/O, sklearn-style pipelines, and comprehensive testing
- **Research-proven:** Used in multiple peer-reviewed publications

This section includes:

- *What is py3plex?* — Key features and what makes py3plex unique
- *Key Use Cases* — Typical domains and applications
- *Multilayer Networks in 2 Minutes* — Quick conceptual introduction to multilayer networks

Next steps:

- **Never used py3plex?** → Start with *10-Minute Tutorial*
- **Want to understand the theory?** → Read *Multilayer Networks 101*
- **Need specific tasks?** → Jump to *How-to Guides*

[3] https://diataxis.fr/

3.1.2 What is py3plex?

py3plex is a Python library for scalable analysis and visualization of multilayer and multiplex networks.

Key Features

Multilayer Network Structures

py3plex provides native support for:

- **Multiplex networks** — Same nodes across multiple layers (e.g., social networks with friendship, collaboration, and family ties)
- **Heterogeneous networks** — Different node types across layers (e.g., author-paper-venue networks)
- **Temporal networks** — Networks that evolve over time with temporal attributes

All network types are first-class citizens with consistent APIs.

SQL-like DSL for Network Queries

Query networks using intuitive SQL-inspired syntax:

```
from py3plex.dsl import execute_query

# String DSL: readable and concise
result = execute_query(network,
    'SELECT nodes WHERE layer="social" AND degree > 5 '
    'COMPUTE betweenness_centrality'
)
```

Or use the type-safe builder API:

```
from py3plex.dsl import Q, L

# Builder API: with IDE autocompletion
result = (
    Q.nodes()
     .from_layers(L["social"])
     .where(degree__gt=5)
     .compute("betweenness_centrality")
     .execute(network)
)
```

The DSL makes complex network analyses readable and maintainable.

Comprehensive Algorithm Suite

py3plex includes 17+ multilayer-specific algorithms:

- **Community detection:** Louvain, Infomap, multilayer modularity
- **Centrality measures:** Multilayer PageRank, betweenness, versatility
- **Random walks:** Node2Vec, DeepWalk for network embeddings

- **Dynamics:** SIR/SIS epidemic models, diffusion processes

All algorithms handle multilayer structure natively without flattening.

Publication-Ready Visualizations

Create professional network diagrams with:

- Automatic multilayer layouts
- Customizable styling and colors
- Support for large networks (1000+ nodes)
- Export to multiple formats (PNG, PDF, SVG)

High-Performance I/O

- Apache Arrow/Parquet support for large datasets
- NetworkX compatibility for easy integration
- Multiple input formats (edge lists, adjacency matrices, JSON)

What Makes py3plex Unique?

Native Multilayer Representation

Unlike tools that flatten multilayer networks into single-layer graphs, py3plex preserves the multilayer structure throughout the analysis pipeline. This leads to more accurate results for multilayer-specific properties.

Node-Layer Pair Abstraction

py3plex represents multilayer networks using node-layer pairs: a node in layer A is distinct from the same node in layer B. This clean abstraction enables:

- Layer-specific queries
- Inter-layer and intra-layer edge differentiation
- Efficient supra-adjacency matrix operations

Production-Ready Design

py3plex is designed for both research and production:

- Sklearn-style pipelines for reproducible workflows
- Comprehensive type hints and documentation
- Extensive test suite (500+ tests)
- CLI and Docker deployment options

When to Use py3plex?

py3plex is ideal when you have:

- **Multiple relationship types** between entities (e.g., email + phone + in-person contacts)
- **Heterogeneous networks** with different node types (e.g., users, posts, hashtags)
- **Temporal networks** where relationships change over time
- **Need for layer-specific analysis** (e.g., comparing community structure across layers)

If you only need single-layer network analysis, NetworkX or igraph might be simpler choices.

Related Projects

- **NetworkX:** py3plex builds on NetworkX for single-layer operations
- **igraph:** Faster for single-layer networks, but limited multilayer support
- **pymnet:** Pure Python multilayer library, different API design
- **graph-tool:** High performance, but steeper learning curve

py3plex balances ease of use with multilayer-specific functionality.

Next Steps

- **Get started:** *10-Minute Tutorial*
- **See use cases:** *Key Use Cases*
- **Understand multilayer networks:** *Multilayer Networks in 2 Minutes*
- **Deep dive:** *The py3plex Core Model*

3.1.3 Key Use Cases

py3plex is used across multiple domains for analyzing complex systems with multiple relationship types.

Social Networks

Multiplex social networks capture different types of social relationships:

- Friendship, family, and professional connections
- Online interactions: mentions, retweets, replies
- Communication channels: email, chat, video calls

Example applications:

- Identifying influential users across multiple platforms
- Detecting communities that span different relationship types
- Predicting information diffusion through multiple channels

```
# Example: Multi-platform social network
network = multinet.multi_layer_network()
network.add_edges([
    ['Alice', 'twitter', 'Bob', 'twitter', 1],
    ['Alice', 'linkedin', 'Bob', 'linkedin', 1],
    ['Bob', 'twitter', 'Carol', 'twitter', 1],
], input_type="list")

# Find users active across platforms
from py3plex.dsl import Q
active_users = (
    Q.nodes()
     .where(layer_count__gt=1)  # Present in multiple layers
```

(continues on next page)

(continued from previous page)

```
    .execute(network)
)
```

Biological Networks

Molecular interaction networks with multiple relationship types:

- Protein-protein interactions (physical binding)
- Gene regulatory networks (transcriptional control)
- Metabolic pathways (biochemical reactions)
- Signaling cascades

Example applications:

- Drug target identification across interaction types
- Disease gene prioritization using multiple evidence sources
- Pathway enrichment analysis

```
# Example: Multi-omics biological network
# Protein-protein interactions + gene regulation
network = multinet.multi_layer_network()

# Physical interactions
network.add_edge('TP53', 'ppi', 'MDM2', 'ppi')

# Regulatory interactions
network.add_edge('TP53', 'regulation', 'CDKN1A', 'regulation')

# Find key regulators with multiple interaction types
result = execute_query(network,
    'SELECT nodes WHERE layer_count > 1 '
    'COMPUTE degree COMPUTE betweenness_centrality'
)
```

Transportation Networks

Multimodal transportation with different travel modes:

- Road networks
- Public transit (bus, metro, train)
- Air travel
- Pedestrian paths

Example applications:

- Optimizing multimodal route planning
- Identifying critical transfer points
- Analyzing resilience to disruptions

```python
# Example: Multimodal city transportation
network = multinet.multi_layer_network()

# Different transportation modes
network.add_edge('StationA', 'metro', 'StationB', 'metro')
network.add_edge('StationA', 'bus', 'StationC', 'bus')
network.add_edge('StationB', 'metro', 'StationC', 'metro')

# Inter-layer connections (transfers)
network.add_edge('StationA', 'metro', 'StationA', 'bus',
                 type='interlayer')
```

Knowledge Graphs

Heterogeneous knowledge networks with different entity and relation types:

- Entity types: people, organizations, locations, concepts
- Relation types: employment, location, authorship, citation

Example applications:

- Entity linking and disambiguation
- Knowledge graph completion
- Question answering over structured data

Scientific Collaboration

Co-authorship networks across disciplines:

- Joint publications
- Grant collaborations
- Conference attendance
- Social media interactions

Example applications:

- Identifying interdisciplinary researchers
- Detecting emerging research communities
- Predicting future collaborations

Epidemic Modeling

Disease transmission through multiple contact types:

- Household contacts
- Workplace interactions
- Social gatherings
- Healthcare settings

Example applications:

- Predicting disease spread

- Evaluating intervention strategies
- Identifying super-spreader events

See *How to Simulate Multilayer Dynamics* for epidemic modeling how-tos.

Communication Networks

Enterprise communication across channels:

- Email correspondence
- Instant messaging
- Video conferencing
- Document collaboration

Example applications:

- Organizational structure analysis
- Information flow optimization
- Team effectiveness measurement

Financial Networks

Economic relationships at multiple scales:

- Trade networks (goods, services)
- Financial flows (investments, loans)
- Ownership structures (subsidiaries, shareholding)
- Interbank lending networks

Example applications:

- Systemic risk assessment
- Contagion analysis
- Market structure analysis
- Regulatory network analysis

Example: Multi-layer financial network

```python
from py3plex.core import multinet
from py3plex.dsl import Q, execute_query

# Build financial network with multiple relationship types
network = multinet.multi_layer_network(directed=True)

# Trade relationships
network.add_edges([
    ['BankA', 'trade', 'BankB', 'trade', 1.5],  # Trade volume in billions
    ['BankB', 'trade', 'BankC', 'trade', 2.3],
    ['BankC', 'trade', 'BankA', 'trade', 1.8],
], input_type="list")

```

(continues on next page)

(continued from previous page)

```
# Ownership structures
network.add_edges([
    ['BankA', 'ownership', 'SubsidiaryX', 'ownership', 0.75],  # 75% ownership
    ['BankB', 'ownership', 'SubsidiaryY', 'ownership', 0.60],
], input_type="list")

# Interbank lending
network.add_edges([
    ['BankA', 'lending', 'BankB', 'lending', 500],  # Lending amount
    ['BankB', 'lending', 'BankC', 'lending', 300],
    ['BankC', 'lending', 'BankA', 'lending', 200],
], input_type="list")

# Identify systemically important institutions
# (high degree across multiple relationship types)
systemic_nodes = (
    Q.nodes()
     .compute("degree", "betweenness_centrality")
     .where(degree__gt=3)
     .order_by("betweenness_centrality", reverse=True)
     .execute(network)
)

print("Systemically Important Institutions:")
for node, data in list(systemic_nodes.items())[:5]:
    print(f"  {node}: degree={data['degree']}, "
          f"betweenness={data['betweenness_centrality']:.4f}")

# Analyze risk propagation
# Find institutions connected across multiple layers
from collections import Counter
node_layer_count = Counter()
for node, layer in network.get_nodes():
    node_layer_count[node] += 1

highly_connected = {
    node: count for node, count in node_layer_count.items()
    if count >= 2  # Present in 2+ layers
}

print(f"\nInstitutions with multiple relationship types: {len(highly_
↪connected)}")
for node, count in sorted(highly_connected.items(),
                          key=lambda x: x[1], reverse=True):
    print(f"  {node}: {count} relationship types")
```

Expected output:

```
Systemically Important Institutions:
```

(continues on next page)

(continued from previous page)

```
  ('BankA', 'trade'): degree=4, betweenness=0.1234
  ('BankB', 'lending'): degree=4, betweenness=0.1156
  ('BankC', 'trade'): degree=3, betweenness=0.0987

Institutions with multiple relationship types: 3
  BankA: 3 relationship types
  BankB: 3 relationship types
  BankC: 2 relationship types
```

Use cases for financial network analysis:

- **Systemic risk:** Identify institutions whose failure would cascade through multiple layers
- **Regulatory oversight:** Detect hidden dependencies across different financial instruments
- **Market surveillance:** Monitor unusual patterns in multi-layer trading relationships
- **Portfolio diversification:** Understand correlation structures across asset types

See *How to Run Community Detection on Multilayer Networks* for detecting financial communities and *How to Simulate Multilayer Dynamics* for contagion modeling.

Next Steps

- **Try an example:** *10-Minute Tutorial*
- **See complete examples:** *Examples & Recipes*
- **Learn the concepts:** *Multilayer Networks 101*

3.1.4 Multilayer Networks in 2 Minutes

A **multilayer network** represents systems where entities can be connected through multiple types of relationships simultaneously.

The Core Idea

In the real world, relationships are rarely uniform. Consider a group of researchers:

- They **co-author papers** (collaboration network)
- They **cite each other** (citation network)
- They **attend conferences** together (social network)
- They may **share funding** (grant network)

A single-layer network can only capture one of these relationships. A multilayer network captures all of them.

Visual Intuition

```
Single-layer network (e.g., only co-authorship):

Alice --- Bob --- Carol
           |
         David

```

(continues on next page)

(continued from previous page)

```
Multilayer network (co-authorship + citations):

Layer 1 (Co-authorship):
Alice --- Bob --- Carol
           |
         David

Layer 2 (Citations):
Alice --> Bob --> Carol
           ↓
         David

Inter-layer connections:
Alice in Layer 1  Alice in Layer 2 (same person)
```

Key Concepts

Layers

Each layer represents a type of relationship (e.g., friendship, collaboration, family).

Intra-layer edges

Connections within a single layer (e.g., Alice friends with Bob).

Inter-layer edges

Connections between layers, usually representing the same entity across layers (e.g., Alice in the friendship layer is the same person as Alice in the collaboration layer).

Node-layer pairs

In py3plex, a node in layer A is conceptually distinct from the same node in layer B. This is the fundamental abstraction that makes multilayer analysis possible.

Types of Multilayer Networks

Multiplex networks

- Same nodes across all layers
- Different edge types per layer
- Example: Social media (same users, different platforms)

Heterogeneous networks

- Different node types in different layers
- Example: Author-Paper-Venue networks

Temporal networks

- Layers represent time slices
- Same nodes and edge types, but edges appear/disappear over time
- Example: Communication networks over days/weeks

Why Multilayer Matters

Information is lost when flattening

If you merge all layers into a single network, you lose critical information:

- Which type of relationship connects two nodes?
- Are communities consistent across layers or layer-specific?
- How do different relationship types interact?

New properties emerge

Multilayer networks have properties that don't exist in single-layer networks:

- **Node versatility:** How many layers does a node participate in?
- **Layer correlation:** Are connections in layer A predictive of layer B?
- **Multilayer communities:** Groups that span multiple relationship types
- **Multiplexity:** Number of different relationship types between two nodes

Real-World Impact

Example: Disease spread

In epidemic modeling, people interact through:

- Household contacts (high transmission rate)
- Workplace contacts (medium rate, large reach)
- Social gatherings (variable)

A single-layer model that averages these interactions will give incorrect predictions. A multilayer model preserves the structure and transmission dynamics of each contact type.

Example: Social influence

A person might be influential on Twitter (many followers) but not on LinkedIn (few connections). Flattening these into a single "social network" loses this nuance.

Quick Example in Code

```
from py3plex.core import multinet

# Create multilayer network
network = multinet.multi_layer_network()

# Add edges in different layers
network.add_edges([
    # Friendship layer
    ['Alice', 'friends', 'Bob', 'friends', 1],
    ['Bob', 'friends', 'Carol', 'friends', 1],

    # Collaboration layer
    ['Alice', 'work', 'Carol', 'work', 1],
    ['Carol', 'work', 'David', 'work', 1],
], input_type="list")
```

(continues on next page)

(continued from previous page)

```
# Analyze
stats = network.basic_stats()
print(f"Layers: {len(network.get_layers())}")
print(f"Nodes: {stats['nodes']}")
print(f"Edges: {stats['edges']}")
```

Expected output:

```
Layers: 2
Nodes: 8  (4 unique entities × 2 layers)
Edges: 4
```

When to Use Multilayer Analysis?

Use multilayer networks when:

- You have **multiple relationship types** between entities
- The **type of relationship matters** for your analysis
- You want to **preserve layer-specific structure**
- You're studying **cross-layer effects** (e.g., how Twitter activity affects LinkedIn connections)

Stick with single-layer when:

- All relationships are of the same type
- Relationship type doesn't affect your analysis
- You only care about aggregate connectivity

Next Steps

- **Try it yourself:** *10-Minute Tutorial*
- **Deeper theory:** *Multilayer Networks 101*
- **See how py3plex works:** *The py3plex Core Model*
- **Start analyzing:** *How to Load and Build Networks*

3.2 Part II: Getting Started (Tutorials)

Step-by-step tutorials for beginners. Start here if you're new to py3plex.

3.2.1 Getting Started (Tutorials)

This section provides step-by-step tutorials to help you install py3plex and complete your first multilayer network analysis.

What's in this section:

- *Installation and Setup* — Installation options and dependencies
- *10-Minute Tutorial* — Comprehensive 10-minute tutorial from basics to advanced analysis

- *Common Issues and Troubleshooting* — Solutions to installation and runtime problems

Why Tutorials?

These pages are **learning-oriented**—they walk you through concrete steps to build skills and confidence. Unlike how-to guides (which assume you know what you want to do), tutorials guide you through a complete journey from start to finish.

Recommended Path

Complete Beginner?

1. *Installation and Setup* — Set up your environment
2. *10-Minute Tutorial* — Complete tutorial from basics to advanced topics
3. Then explore *How-to Guides* for specific tasks

Experienced with Networks?

1. Skim *10-Minute Tutorial* to see py3plex syntax and features
2. Jump to *How-to Guides* for task-oriented guides
3. Check *The py3plex Core Model* for the data model

Having Problems?

See *Common Issues and Troubleshooting* for troubleshooting help.

What You'll Learn

After completing these tutorials, you'll know how to:

- Install py3plex and its dependencies
- Create multilayer networks from scratch
- Load networks from files
- Compute basic statistics and centrality measures
- Detect communities across layers
- Visualize multilayer networks
- Query networks with the DSL

Next Steps After Tutorials:

- **Task-oriented guides:** *How-to Guides*
- **Understand the theory:** *Multilayer Networks 101*
- **See more examples:** *Examples & Recipes*
- **API reference:** *API Documentation*

Relation to Other Sections:

This section teaches by **doing**. Other sections serve different purposes:

- **Overview** (*Overview*) — High-level introduction without code
- **How-to Guides** (*How-to Guides*) — Solve specific problems (assumes basic knowledge)
- **Concepts** (*Concepts & Explanations*) — Understand the theory and design
- **Reference** (*API & DSL Reference*) — Look up API details

3.2.2 Installation and Setup

This guide covers all aspects of installing py3plex and setting up your environment.

Which Docs Should I Use?

Most users should use these docs (py3plex 1.x), which you're reading now. They cover:

- The current stable version with modern features (DSL v2, pipelines, workflows)
- All new APIs, including the SQL-like query DSL and dplyr-style operations
- Actively maintained and updated with examples

Use the legacy docs (py3plex 0.8x) only if:

- You're maintaining old code that depends on py3plex 0.8x specifically
- You need features that were removed in the 1.x rewrite (very rare)

The legacy docs are available at https://py3plex.readthedocs.io but are **not updated** and may have broken examples. We strongly recommend upgrading to 1.x for new projects.

Installation Modes

Choose the installation method based on your needs:

Mode	Command	When to Use
User Install	`pip install py3plex`	Standard usage: running analyses, using the library
Developer Install	Clone repo + `make setup` + `make dev-install`	Contributing code, running tests, building docs

Basic Installation

Install from GitHub (Recommended)

The latest version is always available on GitHub:

```
pip install py3plex
```

This installs the core py3plex library with all required dependencies.

Install from Source

For development or to access the latest features:

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
pip install -e .
```

The `-e` flag installs in editable mode, so changes to the source code are immediately available.

Docker Installation (Alternative)

For a containerized environment with all dependencies pre-installed, use Docker:

```
# Clone the repository
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Build the Docker image
docker build -t py3plex:latest .

# Run py3plex commands
docker run --rm py3plex:latest --version
docker run --rm py3plex:latest selftest
```

Benefits of Docker installation:

- No Python environment setup required
- All dependencies pre-installed and tested
- Consistent environment across different systems
- Isolated from system Python installation

See *Docker Usage Guide* for complete Docker documentation including:

- Building and running the container
- Working with data files via volume mounts
- Docker Compose usage
- Helper scripts (`py3plex-docker.sh`)
- Batch processing and CI/CD integration

Optional Dependencies

py3plex offers several optional feature sets that can be installed separately:

Advanced Community Detection

Install Infomap for advanced overlapping community detection:

```
pip install py3plex[infomap]
```

Additional Algorithms

Install extra community detection algorithms (Louvain, cdlib):

```
pip install py3plex[algos]
```

Advanced Visualization

Install Plotly and igraph for interactive and advanced visualizations:

```
pip install py3plex[viz]
```

Apache Arrow Support

Install pyarrow for high-performance I/O with Arrow/Parquet formats:

```
pip install py3plex[arrow]
```

Arrow format provides 2-3x faster read/write operations and better compression compared to JSON. See ../user_guide/io_and_formats for details on using Arrow format.

Multiple Extras

Install multiple feature sets at once:

```
pip install py3plex[infomap,viz,algos,arrow]
```

Development Installation

For contributors, install with development tools (testing, linting):

Recommended (Makefile-based):

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
make setup        # Create virtual environment
make dev-install  # Install with dev dependencies
```

Alternative (manual):

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
pip install -e ".[dev]"
```

Both methods install:

- pytest and coverage tools
- black, ruff, isort for code formatting
- mypy for type checking
- sphinx for documentation building

System Requirements

Python Version

py3plex requires Python 3.8 or higher. We test on:

- Python 3.8
- Python 3.9
- Python 3.10
- Python 3.11
- Python 3.12

Platform Support

py3plex is cross-platform and tested on:

- **Linux** (Ubuntu 20.04, 22.04)
- **macOS** (macOS 11, 12, 13)
- **Windows** (Windows Server 2019, 2022)

Core Dependencies

These are automatically installed with py3plex:

- `networkx>=2.5` - Graph data structures and algorithms
- `numpy>=0.8` - Numerical computing
- `scipy>=1.1.0` - Scientific computing and sparse matrices
- `pandas` - Data manipulation
- `matplotlib` - Static visualization
- `scikit-learn` - Machine learning utilities
- `tqdm` - Progress bars
- `rdflib>=0.1` - Semantic web support
- `bitarray>=2.0.0` - Efficient boolean arrays

External Binaries (Optional)

Note: As of October 2025, py3plex no longer bundles external binaries to reduce repository size and improve licensing clarity.

Infomap Community Detection

For advanced community detection using Infomap:

Option 1: Use the bundled Python implementation (recommended):

```
pip install py3plex[infomap]
```

Option 2: Download the C++ binary from the official source:

- Website: https://www.mapequation.org/infomap/

- Follow platform-specific installation instructions

Alternative: Use the built-in Louvain algorithm (no binary needed):

```
from py3plex.algorithms.community_detection import community_louvain
communities = community_louvain.best_partition(network.core_network)
```

Node2Vec Embeddings

For Node2Vec node embeddings, use pure Python alternatives:

Option 1: Use pecanpy (fast parallel implementation):

```
pip install pecanpy
```

Option 2: Use node2vec package:

```
pip install node2vec
```

Option 3: Use py3plex's built-in random walk implementation:

```
from py3plex.algorithms.general.walkers import node2vec_walk, generate_walks
walks = generate_walks(network.core_network, num_walks=10, walk_length=80)
```

Virtual Environment Setup

We strongly recommend using a virtual environment:

Using venv

```
# Create virtual environment
python3 -m venv py3plex-env

# Activate (Linux/macOS)
source py3plex-env/bin/activate

# Activate (Windows)
py3plex-env\Scripts\activate

# Install py3plex
pip install py3plex
```

Using conda

```
# Create conda environment
conda create -n py3plex python=3.10

# Activate
conda activate py3plex

# Install py3plex
pip install py3plex
```

Common Installation Issues

Issue: "No module named 'numpy'"

Solution: Install numpy first:

```
pip install numpy
pip install py3plex
```

Issue: Compilation errors on Windows

Solution: Install Visual C++ Build Tools:

1. Download from: https://visualstudio.microsoft.com/visual-cpp-build-tools/
2. Install "Desktop development with C++"
3. Retry installation

Issue: "Permission denied" on Linux/macOS

Solution: Use pip install with –user flag:

```
pip install --user py3plex
```

Or use a virtual environment (recommended).

Issue: Old version installed

Solution: Upgrade to latest version:

```
pip install --upgrade py3plex
```

Verifying Installation

Test that py3plex is correctly installed:

```python
import py3plex
print(py3plex.__version__)

from py3plex.core import multinet
network = multinet.multi_layer_network()
print("py3plex installed successfully!")
```

Run the test suite:

```bash
# Clone repository (if not already)
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Run tests
python run_tests.py
```

License Considerations

Main Library License

py3plex core is distributed under the MIT License (permissive, commercial-friendly).

Bundled Code Licenses

Important: The repository contains AGPLv3-licensed code in `py3plex/algorithms/community_detection/infomap/`.

If you use Infomap-based community detection functions, your application may be subject to AGPLv3 requirements (copyleft).

License Matrix

Feature Category	License	Commercial Use	Notes
Core multilayer functionality	MIT	[OK] Yes	Safe for proprietary use
Network visualization	MIT	[OK] Yes	Safe for proprietary use
I/O operations	MIT	[OK] Yes	Safe for proprietary use
Louvain community detection	BSD-3-Clause	[OK] Yes	Safe for proprietary use
Label propagation	MIT	[OK] Yes	Safe for proprietary use
Infomap community detection	**AGPLv3**	WARNING Restricted	Viral license - requires open-sourcing derived works

Recommendations

- For commercial/proprietary projects: Avoid Infomap functions or use the pure Python `infomap` package separately
- For open-source projects: All features are safe to use
- When in doubt: Use alternative algorithms (Louvain, label propagation) which are BSD/MIT licensed

Next Steps

After installation, proceed to:

- *10-Minute Tutorial* - 10-minute comprehensive tutorial
- ../user_guide/networks - Detailed usage guide

Getting Help

If you encounter installation issues:

1. Check GitHub Issues[4]
2. Search for similar problems

[4] https://github.com/SkBlaz/py3plex/issues

3. Open a new issue with:
   - Your operating system and version
   - Python version (`python --version`)
   - Error message and full traceback
   - Installation command used

For any errors, please open an issue at https://github.com/SkBlaz/py3plex/issues

3.2.3 10-Minute Tutorial

Create a multilayer network, compute statistics, detect communities, and visualize—all in 10 minutes. This comprehensive tutorial takes you from basic network creation to advanced multilayer analysis.

Run this tutorial online

You can run this tutorial in your browser without any local installation:

[5] Or download the full executable script: `tutorial_10min.py`

You will learn:

- Create multilayer networks from scratch and load from files
- Display network statistics and understand the multilayer model
- Compute layer-specific and multilayer metrics
- Query networks with the SQL-like DSL
- Detect communities across layers
- Generate random walks for embeddings
- Create publication-ready visualizations

Installation

```
pip install py3plex
```

For Docker setup or optional dependencies, see *Installation and Setup*.

Part 1: Your First Network (2 min)

Let's create a simple multilayer network and explore its basic properties.

Create from scratch:

```
"""Example 1: Creating Your First Multilayer Network"""
print("\n" + "="*60)
print("Example 1: Creating a Multilayer Network")
print("="*60)
```

(continues on next page)

[5] https://colab.research.google.com/github/SkBlaz/py3plex/blob/master/notebooks/tutorial_10min.ipynb

(continued from previous page)

```
# Create a new multilayer network
network = multinet.multi_layer_network()

# Add edges within layers (this automatically creates nodes)
# Format: [source_node, source_layer, target_node, target_layer, weight]
network.add_edges([
    ['A', 'layer1', 'B', 'layer1', 1],
    ['B', 'layer1', 'C', 'layer1', 1],
    ['A', 'layer2', 'B', 'layer2', 1],
    ['B', 'layer2', 'D', 'layer2', 1]
], input_type="list")

# Display basic statistics
print("\nBasic Statistics:")
network.basic_stats()
```

The complete executable version is available at `examples/getting_started/tutorial_10min.py`.

Output:

```
Number of nodes: 8
Number of edges: 4
Number of unique node IDs (across all layers): 4
Nodes per layer:
  Layer 'layer1': 3 nodes
  Layer 'layer2': 3 nodes
```

Key concept: The network has node-layer pairs representing the same node in different layers. For example, node ‘A’ appears as `('A', 'layer1')` and `('A', 'layer2')`.

Visualize your network:

```
from py3plex.visualization.multilayer import draw_multilayer_default

draw_multilayer_default([network], display=True)
```

Nodes are colored by layer. Edges connect nodes within and across layers.

Load from file:

```
# For larger networks, load from files
network = multinet.multi_layer_network().load_network(
    "datasets/multiedgelist.txt",
    input_type="multiedgelist",  # Format: source target layer
    directed=False  # Set to True for directed networks
)
network.basic_stats()
```

Supported formats: `multiedgelist` (source, target, layer), `edgelist` (source, target), `gpickle`, `gml`, `graphml`

Part 2: Basic Analysis (2 min)

Compute layer-specific metrics:

```
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Layer density: how connected is each layer?
print(f"Friends density: {mls.layer_density(network, 'friends'):.3f}")
print(f"Colleagues density: {mls.layer_density(network, 'colleagues'):.3f}")

# Node activity: what fraction of layers does Bob appear in?
print(f"Bob's activity: {mls.node_activity(network, 'Bob'):.3f}")
```

Output:

```
Friends density: 0.667
Colleagues density: 0.667
Bob's activity: 1.000
```

- **Density 0.667** — 2 of 3 possible edges exist
- **Activity 1.0** — Bob appears in 100% of layers (both)

Query with the DSL:

Py3plex features a powerful SQL-like DSL for querying networks!

DSL Example: Network Queries

Use SQL-like queries for network exploration:

```
from py3plex.dsl import execute_query, Q, L

# String DSL syntax
result = execute_query(network, 'SELECT nodes WHERE degree > 1')
print(f"Found {result['count']} high-degree nodes")

# Builder API (recommended for production)
result = (
    Q.nodes()
     .from_layers(L["friends"])
     .where(degree__gt=1)
     .compute("betweenness_centrality")
     .execute(network)
)
```

The DSL makes complex network analysis intuitive and concise!

See ../user_guide/dsl for complete DSL documentation.

Explore network structure:

```
# Iterate through nodes and edges
for node in network.get_nodes(data=True):
    print(node)  # (('Alice', 'friends'), {'type': 'friends'})

for edge in network.get_edges(data=True):
    print(edge)  # Edge with attributes

# Get neighbors of a node in a specific layer
neighbors = list(network.get_neighbors("Alice", layer_id="friends"))
```

Extract subnetworks:

```
# Single layer
friends_layer = network.subnetwork(['friends'], subset_by="layers")

# Specific nodes across all layers
subset = network.subnetwork(['Alice', 'Bob'], subset_by="node_names")

# Specific node-layer pairs
pairs = network.subnetwork([('Alice', 'friends'), ('Bob', 'friends')],
                           subset_by="node_layer_names")
```

Part 3: Advanced Metrics (2 min)

Layer-specific centrality (using NetworkX):

```
friends_layer = network.subnetwork(['friends'], subset_by="layers")
degree_cent = friends_layer.monoplex_nx_wrapper("degree_centrality")
betweenness = friends_layer.monoplex_nx_wrapper("betweenness_centrality")
```

Multilayer centrality:

```
from py3plex.algorithms.multilayer_algorithms.centrality import␣
↪MultilayerCentrality

calc = MultilayerCentrality(network)
ml_degree = calc.overlapping_degree_centrality(weighted=False)
ml_betweenness = calc.multilayer_betweenness_centrality()
```

Multilayer statistics:

```
from py3plex.algorithms.statistics import multilayer_statistics as mls

density = mls.layer_density(network, 'friends')  # Edge density within a layer
activity = mls.node_activity(network, node='Alice')  # Fraction of layers␣
↪node appears in
overlap = mls.edge_overlap(network, 'friends', 'colleagues')  # Shared edges␣
↪between layers
similarity = mls.layer_similarity(network, 'friends', 'colleagues', method=
↪'jaccard')
```

Available statistics: `layer_density`, `entropy_of_multiplexity`, `node_activity`, `edge_overlap`, `layer_similarity`, `algebraic_connectivity`, `multilayer_clustering_coefficient`, and more.

Part 4: Community Detection (2 min)

```
from py3plex.algorithms.community_detection.multilayer_modularity import␣
↪louvain_multilayer

partition = louvain_multilayer(
    network,
    gamma=1.0,       # Resolution (higher = more communities)
    omega=1.0,       # Inter-layer coupling
    random_state=42
)

print(f"Communities: {len(set(partition.values()))}")

from collections import Counter
sizes = Counter(partition.values())
print(f"Sizes: {dict(sizes)}")
```

Part 5: Random Walks (1 min)

Basic walk:

```
from py3plex.algorithms.general.walkers import basic_random_walk, node2vec_
↪walk, generate_walks

G = network.core_network
start = list(G.nodes())[0]

walk = basic_random_walk(G, start_node=start, walk_length=10, seed=42)
```

Node2Vec biased walks:

```
# BFS-like (local)
walk_bfs = node2vec_walk(G, start, walk_length=20, p=1.0, q=2.0, seed=42)

# DFS-like (explore)
walk_dfs = node2vec_walk(G, start, walk_length=20, p=1.0, q=0.5, seed=42)
```

Generate walks for embeddings:

```
walks = generate_walks(G, num_walks=10, walk_length=10, p=1.0, q=1.0, seed=42)
# Use with Word2Vec: model = Word2Vec([[str(n) for n in w] for w in walks])
```

Part 6: Visualization (1 min)

```
from py3plex.visualization.multilayer import hairball_plot
import matplotlib.pyplot as plt

network_colors, graph = network.get_layers(style="hairball")

plt.figure(figsize=(10, 10))
hairball_plot(graph, network_colors, layout_algorithm="force")
plt.savefig("network.png", dpi=150, bbox_inches='tight')
plt.close()
```

With community colors:

```
from py3plex.visualization.colors import colors_default

top_communities = [c for c, _ in Counter(partition.values()).most_common(5)]
color_map = dict(zip(top_communities, colors_default[:5]))
node_colors = [color_map.get(partition.get(n), "gray") for n in network.get_
↪nodes()]

plt.figure(figsize=(10, 10))
hairball_plot(graph, node_colors, layout_algorithm="force")
plt.savefig("communities.png", dpi=150, bbox_inches='tight')
plt.close()
```

Key Concepts

Understanding these core concepts will help you work effectively with py3plex:

1. **Node-layer pairs:** Nodes are `(node_id, layer_id)` tuples. Alice in friends is different from Alice in colleagues. This allows the same entity to have different connections in different contexts.
2. **Layers preserve context:** Each layer represents a different relationship type (friends, colleagues, family, etc.). Statistics and algorithms respect layer boundaries, revealing patterns invisible to single-layer analysis.
3. **NetworkX compatible:** py3plex uses NetworkX internally. All NetworkX algorithms work on py3plex networks, giving you access to a rich ecosystem of graph algorithms.
4. **Multilayer statistics:** Metrics like node activity and edge overlap reveal cross-layer patterns. These measures quantify how entities interact across different relationship contexts.

Complete Example

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection.multilayer_modularity import␣
↪louvain_multilayer
from py3plex.visualization.multilayer import hairball_plot
```

(continues on next page)

(continued from previous page)

```
from py3plex.visualization.colors import colors_default
from collections import Counter
import matplotlib.pyplot as plt

# Load
network = multinet.multi_layer_network().load_network(
    "datasets/multiedgelist2.txt",
    input_type="multiedgelist",
    directed=False
)
network.basic_stats()

# Analyze
friends_layer = network.subnetwork(['friends'], subset_by="layers")
degree_cent = friends_layer.monoplex_nx_wrapper("degree_centrality")
print("Top 5 by degree:", sorted(degree_cent.items(), key=lambda x: x[1],␣
↪reverse=True)[:5])

# Detect communities
partition = louvain_multilayer(network, gamma=1.0, omega=1.0, random_state=42)
print(f"Communities: {len(set(partition.values()))}")

# Visualize
network_colors, graph = network.get_layers(style="hairball")
top_communities = [c for c, _ in Counter(partition.values()).most_common(3)]
color_map = dict(zip(top_communities, colors_default[:3]))
node_colors = [color_map.get(partition.get(n), "lightgray") for n in network.
↪get_nodes()]

plt.figure(figsize=(12, 12))
hairball_plot(graph, node_colors, layout_algorithm="force")
plt.savefig("analysis.png", dpi=150, bbox_inches='tight')
plt.close()
```

DSL: Query Networks Like SQL

The SQL-like DSL makes many analysis tasks simpler and more intuitive:

DSL Example: Advanced Analysis

```
from py3plex.dsl import Q, L

# Find top influential nodes
result = (
    Q.nodes()
     .where(degree__gt=3)
     .compute("betweenness_centrality", "pagerank")
     .order_by("-betweenness_centrality")
```

```
        .limit(10)
        .execute(network)
)

# Display as DataFrame
df = result.to_pandas()
print(df)

# Compare layers
for layer_id in ['friends', 'colleagues']:
    layer_result = (
        Q.nodes()
         .from_layers(L[layer_id])
         .compute("degree")
         .execute(network)
    )
    print(f"Layer {layer_id}: {layer_result.count} nodes")
```

See ../user_guide/dsl for comprehensive DSL documentation!

Common Issues

File Not Found: Use absolute paths or run from repository root.

Visualization not showing: In Jupyter, add `%matplotlib inline`. In scripts, use `plt.show()` or save to file.

Missing dependencies: Install extras with `pip install py3plex[viz,algos]`

Next Steps

Continue Learning:

- *Installation and Setup* — Optional dependencies and setup options
- *Common Issues and Troubleshooting* — Solutions to common problems

Advanced Topics:

- *Analysis Recipes & Workflows* — Ready-to-use analysis workflows and patterns
- ../user_guide/dsl — SQL-like DSL for querying multilayer networks
- ../user_guide/graph_ops — Dplyr-style chainable graph operations

Deep Dive:

- *Multilayer Networks 101* — Theory and concepts
- ../user_guide/statistics — All available statistics
- ../user_guide/community_detection — Parameter tuning
- ../user_guide/visualization — Advanced visualization

Examples:

- `examples/` directory — 50+ working examples
- *Examples & Recipes* — Annotated example gallery

3.2.4 Common Issues and Troubleshooting

This guide helps you troubleshoot common problems when getting started with py3plex.

Installation Issues

"No module named 'numpy'"

Problem: Import error when trying to use py3plex.

Solution: Install numpy first:

```
pip install numpy
pip install py3plex
```

Compilation Errors on Windows

Problem: C++ compilation errors during installation on Windows.

Solution: Install Visual C++ Build Tools:

1. Download from: https://visualstudio.microsoft.com/visual-cpp-build-tools/
2. Install "Desktop development with C++"
3. Retry installation

```
pip install py3plex
```

"Permission denied" on Linux/macOS

Problem: Permission errors during pip install.

Solution 1: Use pip install with –user flag:

```
pip install --user py3plex
```

Solution 2: Use a virtual environment (recommended):

```
python3 -m venv py3plex-env
source py3plex-env/bin/activate  # On Windows: py3plex-env\Scripts\activate
pip install py3plex
```

Old Version Installed

Problem: Changes not reflecting or documentation mentions newer features.

Solution: Upgrade to the latest version:

```
pip install --upgrade py3plex
```

Data Loading Issues

File Not Found Errors

Problem: `FileNotFoundError` when loading networks.

Solution 1: Use absolute paths:

```
import os

network_path = "/absolute/path/to/data.edgelist"
network = multinet.multi_layer_network().load_network(
    network_path, input_type="edgelist", directed=False
)
```

Solution 2: Ensure you're running from the correct directory:

```
import os

# Check current directory
print("Current directory:", os.getcwd())

# List files
print("Files:", os.listdir('.'))
```

Solution 3: Use paths relative to script location:

```
import os

script_dir = os.path.dirname(os.path.abspath(__file__))
data_path = os.path.join(script_dir, "data", "network.edgelist")
```

Empty or Malformed Networks

Problem: Network loads but has no nodes or edges.

Solution: Check file format matches input_type:

```
# For simple edge lists (source target)
network.load_network("data.txt", input_type="edgelist")

# For multilayer edge lists (source target layer)
network.load_network("data.txt", input_type="multiedgelist")

# Always verify after loading
network.basic_stats()
```

Visualization Issues

Visualization Not Showing

Problem: No window appears when calling visualization functions.

Solution 1: For Jupyter notebooks, add at the top:

```
%matplotlib inline
```

Solution 2: For scripts, explicitly show the plot:

```
import matplotlib.pyplot as plt

# Your visualization code
draw_multilayer_default([network], display=False)

# Explicitly show
plt.show()
```

Solution 3: Save to file instead of displaying:

```
from py3plex.visualization.multilayer import hairball_plot
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 10))
hairball_plot(network.core_network, layout_algorithm="force")
plt.savefig("network.png", dpi=150, bbox_inches='tight')
plt.close()
```

"No Display Found" Error

Problem: Error on headless servers or SSH sessions without X11.

Solution: Use Agg backend for matplotlib:

```
import matplotlib
matplotlib.use('Agg')  # Must be before importing pyplot
import matplotlib.pyplot as plt

# Now visualization will save to file instead of displaying
from py3plex.visualization.multilayer import draw_multilayer_default
draw_multilayer_default([network], display=False)
plt.savefig("output.png")
```

Blurry or Low-Quality Plots

Problem: Plots look pixelated or blurry.

Solution: Increase DPI when saving:

```
plt.savefig("network.png", dpi=300, bbox_inches='tight')
```

Missing Dependencies

"No module named 'plotly'"

Problem: Interactive visualization features not working.

Solution: Install visualization extras:

```
pip install py3plex[viz]
```

"No module named 'infomap'"

Problem: Infomap community detection not working.

Solution: Install Infomap support:

```
pip install py3plex[infomap]
```

Or use alternative community detection methods:

```
from py3plex.algorithms.community_detection import community_louvain
communities = community_louvain.best_partition(network.core_network)
```

Performance Issues

Very Slow Loading

Problem: Loading large networks takes too long.

Solution 1: Use Arrow/Parquet format for large networks:

```
# Install Arrow support
pip install py3plex[arrow]
```

```
from py3plex.io import read, write

# Save in efficient format
write(network, "network.arrow")

# Load much faster
network = read("network.arrow")
```

Solution 2: Process in chunks or use subnetworks:

```
# Extract a single layer for testing
layer_1 = network.subnetwork(['layer1'], subset_by="layers")

# Work with the smaller network
layer_1.basic_stats()
```

Out of Memory Errors

Problem: Python crashes or raises MemoryError with large networks.

Solution: See *Performance and Scalability Best Practices* for:

- Memory-efficient loading strategies
- Batch processing techniques
- Sparse matrix representations
- Subnetwork extraction

Algorithm Issues

Community Detection Returns Trivial Results

Problem: Each node in its own community or all nodes in one community.

Solution: Adjust algorithm parameters:

```
from py3plex.algorithms.community_detection.multilayer_modularity import (
    louvain_multilayer
)

# Try different resolution parameters
partition = louvain_multilayer(
    network,
    gamma=0.5,     # Lower = fewer, larger communities
    omega=1.5,     # Higher = more layer consistency
    random_state=42
)
```

Random Walk or Embedding Errors

Problem: Random walk or Node2Vec functions fail.

Solution: Ensure network is connected:

```
import networkx as nx

# Check connectivity
if not nx.is_connected(network.core_network.to_undirected()):
    print("Warning: Network is not connected")

    # Get largest component
    largest = max(nx.connected_components(
        network.core_network.to_undirected()
    ), key=len)

    # Create subgraph
    subgraph = network.core_network.subgraph(largest)
```

Docker Issues

Docker Build Fails

Problem: Docker build fails or takes very long.

Solution: Clear Docker cache and rebuild:

```
docker system prune -a
docker build --no-cache -t py3plex:latest .
```

Cannot Access Files in Docker

Problem: Docker container can't see your data files.

Solution: Mount volumes correctly:

```
# Mount current directory
docker run -v $(pwd):/data py3plex:latest /data/network.edgelist

# On Windows (PowerShell)
docker run -v ${PWD}:/data py3plex:latest /data/network.edgelist
```

See *Docker Usage Guide* for complete Docker documentation.

Getting Help

If you encounter an issue not covered here:

1. **Search GitHub Issues:** https://github.com/SkBlaz/py3plex/issues
2. **Check Documentation:** https://skblaz.github.io/py3plex/
3. **Browse Examples:** https://github.com/SkBlaz/py3plex/tree/master/examples

When reporting issues, include: * Python version (`python --version`) * py3plex version (`pip show py3plex`) * Operating system * Full error traceback * Minimal code to reproduce the problem

Related Pages

- *Installation and Setup* - Complete installation guide
- *10-Minute Tutorial* - Comprehensive tutorial from basics to advanced topics
- *Performance and Scalability Best Practices* - Performance optimization

---

3.3 Part III: How-to Guides

Task-oriented guides answering "How do I. . . ?" questions.

3.3.1 How-to Guides

Task-oriented guides for accomplishing specific goals with py3plex. Each guide is focused on a single question: "How do I. . . ?"

This section covers:

Network Operations

- *How to Load and Build Networks* — Create networks from scratch or load from files
- *How to Export and Serialize Networks* — Save networks in various formats

Analysis Tasks

- *How to Compute Network Statistics* — Calculate multilayer metrics and centrality measures
- *How to Run Community Detection on Multilayer Networks* — Find groups of closely connected nodes
- *How to Simulate Multilayer Dynamics* — Model epidemic spread and diffusion processes
- *How to Run Random Walk Algorithms* — Generate network embeddings for machine learning

Visualization

- *How to Visualize Multilayer Networks* — Create publication-ready network diagrams

Query and Transform

- *How to Query Multilayer Graphs with the SQL-like DSL* — Use SQL-like syntax for network queries
- *How to Find Graph Patterns and Motifs with the Pattern Matching API* — Find graph motifs and subgraph patterns
- *How to Build Analysis Pipelines with Dplyr-style Operations* — Chain operations with dplyr-style API

Workflows

- *How to Reproduce Common Analysis Workflows* — Complete analysis recipes and patterns

Reading This Section

Structure of each guide:

1. **Goal:** What you'll accomplish
2. **Prerequisites:** What you need to know first
3. **Steps:** Concrete, actionable instructions with code
4. **Expected output:** What you should see
5. **Next steps:** Links to related concepts and references

How to use these guides:

- Each guide is **self-contained** — you can jump directly to the task you need
- Code examples are **complete and runnable** — copy-paste and run
- Guides focus on **what to do**, not why — for theory, see *Concepts & Explanations*
- For API details, see *API & DSL Reference*

Quick Navigation

I want to…

- Load data → *How to Load and Build Networks*
- Measure network properties → *How to Compute Network Statistics*

- Find communities → *How to Run Community Detection on Multilayer Networks*
- Create visualizations → *How to Visualize Multilayer Networks*
- Query networks → *How to Query Multilayer Graphs with the SQL-like DSL*
- Find patterns/motifs → *How to Find Graph Patterns and Motifs with the Pattern Matching API*
- Model processes → *How to Simulate Multilayer Dynamics*

I'm coming from...

- **NetworkX:** Start with *How to Load and Build Networks* to see multilayer specifics, then *How to Query Multilayer Graphs with the SQL-like DSL* for py3plex's unique DSL
- **Single-layer networks:** Read *How to Compute Network Statistics* and *How to Run Community Detection on Multilayer Networks* to understand multilayer-specific metrics
- **Other tools:** Jump to *How to Export and Serialize Networks* for data import/export

Related Sections

- **Learning multilayer concepts:** *Concepts & Explanations*
- **Detailed API reference:** *API & DSL Reference*
- **Complete examples:** *Examples & Recipes*
- **Tutorials for beginners:** *Getting Started (Tutorials)*

3.3.2 How to Load and Build Networks

Goal: Create multilayer networks from scratch or load them from files.

Prerequisites: Basic Python knowledge. For conceptual background, see *The py3plex Core Model*.

Creating Networks from Scratch

Method 1: Add Edges Directly (Recommended)

The simplest approach—nodes are created automatically when you add edges.

```python
from py3plex.core import multinet

# Create empty network
network = multinet.multi_layer_network()

# Add edges in list format
# Format: [source_node, source_layer, target_node, target_layer, weight]
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1.0],
    ['Bob', 'friends', 'Carol', 'friends', 1.0],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1.0],
    ['Bob', 'colleagues', 'Dave', 'colleagues', 1.0]
], input_type="list")

# Verify
stats = network.basic_stats()
```

Expected output:

```
Number of nodes: 6
Number of edges: 4
Number of unique node IDs (across all layers): 4
Nodes per layer:
  Layer 'friends': 3 nodes
  Layer 'colleagues': 3 nodes
```

Method 2: Add Nodes First

For more control over node attributes:

```python
from py3plex.core import multinet

network = multinet.multi_layer_network()

# Add nodes explicitly (as node-layer tuples)
network.add_nodes([
    ('Alice', 'friends'),
    ('Bob', 'friends'),
    ('Carol', 'friends'),
    ('Alice', 'colleagues'),
    ('Bob', 'colleagues'),
    ('Dave', 'colleagues')
])

# Add edges between existing nodes
network.add_edges([
    [('Alice', 'friends'), ('Bob', 'friends'), 1.0],
    [('Bob', 'friends'), ('Carol', 'friends'), 1.0]
])
```

Method 3: Use Dictionary Format

For networks with edge attributes:

```python
network = multinet.multi_layer_network()

# Add edges with custom attributes
network.add_edges([
    {
        'source': 'Alice',
        'source_type': 'friends',
        'target': 'Bob',
        'target_type': 'friends',
        'weight': 1.0,
        'timestamp': '2024-01-15',
        'interaction_type': 'message'
    },
```

(continues on next page)

(continued from previous page)

```
    {
        'source': 'Bob',
        'source_type': 'friends',
        'target': 'Carol',
        'target_type': 'friends',
        'weight': 2.5,
        'timestamp': '2024-01-16'
    }
], input_type="dict")
```

Loading Networks from Files

Load Edge Lists

Most common format: one edge per line.

File format (stored as datasets/simple_multiplex.txt in the repository):

```
Alice friends Bob friends 1.0
Bob friends Carol friends 1.0
Alice colleagues Bob colleagues 1.0
```

Load it:

```
from py3plex.core import multinet

network = multinet.multi_layer_network()

# Option 1: Load from repository file
network.load_network(
    "datasets/simple_multiplex.txt",
    input_type="multiedgelist"
)

# Option 2: Create from data directly (recommended for examples)
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1.0],
    ['Bob', 'friends', 'Carol', 'friends', 1.0],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1.0],
], input_type="list")

network.basic_stats()
```

Load from Pandas DataFrame

Convert tabular data to networks:

```
import pandas as pd
from py3plex.core import multinet

```

(continues on next page)

(continued from previous page)

```
# Your data
df = pd.DataFrame({
    'source': ['Alice', 'Bob', 'Alice'],
    'layer': ['friends', 'friends', 'work'],
    'target': ['Bob', 'Carol', 'Carol'],
    'weight': [1.0, 1.0, 2.0]
})

# Convert to network
network = multinet.multi_layer_network()

for _, row in df.iterrows():
    network.add_edge(
        row['source'], row['layer'],
        row['target'], row['layer'],
        weight=row['weight']
    )
```

Load from JSON

For complex networks with metadata:

```
import json
from py3plex.core import multinet

# Load JSON
with open('network.json', 'r') as f:
    data = json.load(f)

network = multinet.multi_layer_network()

# Assuming JSON structure: {"edges": [...], "nodes": [...]}
for edge in data['edges']:
    network.add_edge(
        edge['source'], edge['source_layer'],
        edge['target'], edge['target_layer'],
        **edge.get('attributes', {})
    )
```

Load from NetworkX

Convert existing single-layer networks:

```
import networkx as nx
from py3plex.core import multinet

# Existing NetworkX graph
G = nx.karate_club_graph()
```

(continues on next page)

(continued from previous page)

```
# Convert to multilayer (single layer)
network = multinet.multi_layer_network()

for u, v in G.edges():
    network.add_edge(u, 'layer1', v, 'layer1')
```

Loading with Inter-layer Edges

To represent the same entity across layers:

```
network = multinet.multi_layer_network()

# Intra-layer edges
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1.0],
    ['Alice', 'work', 'Carol', 'work', 1.0],
], input_type="list")

# Inter-layer edges (same person across layers)
network.add_edge('Alice', 'friends', 'Alice', 'work', type='interlayer')
network.add_edge('Bob', 'friends', 'Bob', 'work', type='interlayer')
```

Common Patterns

Pattern 1: Social Networks

Multiple relationship types between same people:

```
network = multinet.multi_layer_network()
network.add_edges([
    ['Alice', 'twitter', 'Bob', 'twitter', 1],
    ['Alice', 'linkedin', 'Bob', 'linkedin', 1],
    ['Bob', 'twitter', 'Carol', 'twitter', 1],
], input_type="list")
```

Pattern 2: Heterogeneous Networks

Different entity types:

```
# Author-Paper bipartite network
network = multinet.multi_layer_network()
network.add_edges([
    ['Alice', 'authors', 'Paper1', 'papers', 1],
    ['Bob', 'authors', 'Paper1', 'papers', 1],
    ['Alice', 'authors', 'Paper2', 'papers', 1],
], input_type="list")
```

Pattern 3: Temporal Networks

Time-stamped edges:

```
network = multinet.multi_layer_network()
network.add_edges([
    {
        'source': 'Alice',
        'source_type': 'social',
        'target': 'Bob',
        'target_type': 'social',
        't': '2024-01-15T10:00:00'  # Point in time
    },
    {
        'source': 'Bob',
        'source_type': 'social',
        'target': 'Carol',
        'target_type': 'social',
        't_start': '2024-01-15',     # Time range
        't_end': '2024-01-20'
    }
], input_type="dict")
```

See *API Documentation* for temporal network details.

Verifying Your Network

After loading, verify the structure:

```
# Basic stats
network.basic_stats()

# Get specific information
layers = network.get_layers()
print(f"Layers: {layers}")

num_nodes = len(list(network.get_nodes()))
print(f"Total nodes: {num_nodes}")

# Check a specific layer
from py3plex.dsl import Q, L
layer_nodes = Q.nodes().from_layers(L["friends"]).execute(network)
print(f"Friends layer: {len(layer_nodes)} nodes")
```

Expected output:

```
Layers: ['friends', 'colleagues']
Total nodes: 6
Friends layer: 3 nodes
```

Next Steps

- **Compute statistics:** *How to Compute Network Statistics*
- **Visualize your network:** *How to Visualize Multilayer Networks*
- **Query the network:** *How to Query Multilayer Graphs with the SQL-like DSL*
- **Understand the data model:** *The py3plex Core Model*
- **API reference:** *API Documentation*

3.3.3 Query Zoo: DSL Gallery for Multilayer Analysis

The Query Zoo is a curated gallery of DSL queries that demonstrate the expressiveness and power of py3plex for multilayer network analysis.

Each example:

- Solves a real multilayer analysis problem
- Uses the DSL end-to-end with idiomatic patterns
- Produces concrete, reproducible outputs
- Is fully tested and documented

Why a Query Zoo?

The DSL is most powerful when you see it in action on realistic problems. This gallery shows you **how** to think about multilayer queries, not just **what** the syntax is. Use these examples as recipes and starting points for your own analyses.

Overview

The Query Zoo is organized around common multilayer analysis tasks:

1. **Basic Multilayer Exploration** — Understand layer statistics and structure
2. **Cross-Layer Hubs** — Find nodes that are important across multiple layers
3. **Layer Similarity** — Measure structural alignment between layers

4. **Community Structure** — Detect and analyze multilayer communities
5. **Multiplex PageRank** — Compute multilayer-aware centrality
6. **Robustness Analysis** — Assess network resilience to layer failures
7. **Advanced Centrality Comparison** — Identify versatile vs specialized hubs
8. **Edge Grouping and Coverage** — Analyze edges across layer pairs with top-k and coverage

All examples use small, reproducible multilayer networks from the `examples/dsl_query_zoo/datasets.py` module.

Tip

Running the Examples

All query functions are available in `examples/dsl_query_zoo/queries.py`. To run all queries and generate outputs:

```
python examples/dsl_query_zoo/run_all.py
```

Test the queries with:

```
pytest tests/test_dsl_query_zoo.py
```

---

1. Basic Multilayer Exploration

Problem: You've loaded a multilayer network and want to quickly understand its structure. Which layers are densest? How many nodes and edges does each layer have?

Solution: Compute basic statistics per layer using the DSL.

Query Code

```
def query_basic_exploration(network: Network) -> pd.DataFrame:
    """Summarize layers: node counts, edge counts, and average degree per␣
↪layer.

    Refactored: single DSL query over all layers + pandas groupby, no explicit
    for-loop over layers.

    This query demonstrates basic multilayer exploration by computing
    fundamental statistics for each layer independently. This is typically
    the first step in multilayer analysis to understand the structure
    and identify layers with different connectivity patterns.

    Why it's interesting:
    - Reveals which layers are denser or sparser
    - Identifies layers that might be hubs of activity
    - Shows structural diversity across the multilayer network

    DSL concepts demonstrated:
    - SELECT nodes from all layers in one shot
```

(continues on next page)

(continued from previous page)

```
    - Computing degree per layer
    - Vectorized aggregation by layer

    Args:
        network: A multi_layer_network instance

    Returns:
        pd.DataFrame with columns: layer, n_nodes, n_edges, avg_degree
    """
    result = (
        Q.nodes()
         .from_layers(L["*"])       # all layers in one shot
         .compute("degree")
         .execute(network)
    )

    if len(result) == 0:
        return pd.DataFrame(columns=["layer", "n_nodes", "n_edges", "avg_
↪degree"])

    df = result.to_pandas()

    # One row per (node, layer), so size() is node count
    stats = (
        df.groupby("layer")
          .agg(
              n_nodes=("id", "size"),
              total_degree=("degree", "sum"),
              avg_degree=("degree", "mean"),
          )
          .reset_index()
    )

    stats["n_edges"] = (stats["total_degree"] // 2).astype(int)
    stats["avg_degree"] = stats["avg_degree"].round(2)

    return stats[["layer", "n_nodes", "n_edges", "avg_degree"]]
```

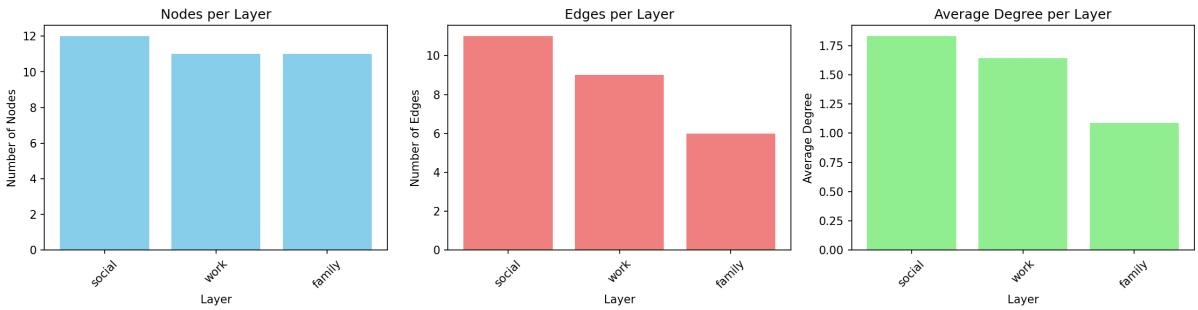
Why It's Interesting

- **First step in any analysis** — Before diving into complex queries, understand your data
- **Reveals layer diversity** — Different layers often have vastly different structures
- **Identifies sparse vs dense layers** — Helps decide which layers need special handling

Example Output

Running on the `social_work_network` (12 people across social/work/family layers):

Layer	Nodes	Edges	Avg Degree
social	12	11	1.83
work	11	9	1.64
family	11	6	1.09



Interpretation: The social layer is densest (highest average degree), while family is sparsest. All layers have similar numbers of nodes, indicating good cross-layer coverage.

DSL Concepts Demonstrated

- `Q.nodes().from_layers(L[name])` — Select nodes from a specific layer
- `.compute("degree")` — Add computed attributes to results
- `.execute(network)` — Run the query and get results
- `.to_pandas()` — Convert to DataFrame for analysis

---

2. Cross-Layer Hubs

Problem: Which nodes are consistently important across *multiple* layers? These "super hubs" are critical because they bridge different contexts.

Solution: Find top-k central nodes per layer, then identify which nodes appear in multiple layers' top lists.

Query Code

```python
def query_cross_layer_hubs(network: Network, k: int = 5) -> pd.DataFrame:
    """Find nodes that are consistently central across multiple layers.

    Refactored: one DSL query across all layers + pandas grouping, no explicit
    per-layer for loop.

    This query identifies "super hubs" - nodes that maintain high centrality
    across different layers. These nodes are particularly important because
    they serve as connectors across different contexts or domains.
```

(continues on next page)

(continued from previous page)

```
    Why it's interesting:
    - Reveals nodes with consistent importance across contexts
    - Useful for identifying key actors in multiplex social networks
    - Helps understand cross-layer influence and information flow

    DSL concepts demonstrated:
    - Single query across all layers
    - Computing betweenness centrality
    - Vectorized top-k selection per layer using groupby
    - Coverage analysis (nodes appearing in multiple layers)

    Args:
        network: A multi_layer_network instance
        k: Number of top nodes to select per layer

    Returns:
        pd.DataFrame with nodes and their centrality scores per layer
    """
    result = (
        Q.nodes()
         .from_layers(L["*"])
         .compute("betweenness_centrality", "degree")
         .execute(network)
    )

    if len(result) == 0:
        return pd.DataFrame()

    df = result.to_pandas().rename(columns={"id": "node"})

    # Top-k by betweenness within each layer (vectorized)
    df_sorted = df.sort_values(["layer", "betweenness_centrality"],
                               ascending=[True, False])
    topk = df_sorted.groupby("layer").head(k)

    # Count how many layers each node appears in as a top-k hub
    coverage = (
        topk.groupby("node")["layer"]
            .nunique()
            .reset_index(name="layer_count")
    )

    result_df = (
        topk.merge(coverage, on="node")
            .sort_values(["layer_count", "betweenness_centrality"],
                         ascending=[False, False])
    )
```

(continues on next page)

(continued from previous page)

```
    return result_df[["node", "layer", "degree",
                      "betweenness_centrality", "layer_count"]]
```

Why It's Interesting

- **Reveals cross-context influence** — Nodes central in one layer might be peripheral in another
- **Identifies key connectors** — Nodes that appear in multiple layers' top-k are especially important
- **Robust hub detection** — More reliable than single-layer centrality

Example Output

Top cross-layer hubs (k=5):

Node	Layer	Degree	Betweenness	Layer Count
Bob	social	3	0.0273	3
Bob	work	2	0.0	3
Bob	family	1	0.0	3
Alice	work	3	0.0889	2
Charlie	social	3	0.0273	2

Interpretation: Bob appears as a top-5 hub in *all three layers* (layer_count=3), making him the most versatile connector. Alice and Charlie are hubs in two layers each.

DSL Concepts Demonstrated

- `.compute("betweenness_centrality", "degree")` — Compute multiple metrics at once
- `.order_by("-betweenness_centrality")` — Sort descending (`-` prefix)
- `.limit(k)` — Take top-k results
- Per-layer iteration and aggregation across layers

---

3. Layer Similarity

Problem: How similar are different layers structurally? Do they serve redundant or complementary roles?

Solution: Compute degree distributions per layer and measure pairwise correlations.

Query Code

```
def query_layer_similarity(network: Network) -> pd.DataFrame:
    """Compute structural similarity between layers based on degree␣
↪distributions.
```

(continues on next page)

(continued from previous page)

```
    Refactored: single DSL query + pivot, no explicit loops over layers/nodes.

    This query measures how similar different layers are in terms of their
    connectivity patterns. Layers with similar degree distributions likely
    serve similar structural roles in the multilayer network.

    Why it's interesting:
    - Identifies redundant or complementary layers
    - Helps understand layer specialization
    - Can inform layer aggregation or simplification decisions

    DSL concepts demonstrated:
    - Single query across all layers
    - Pivot table to create node × layer matrix
    - Correlation between layers via .corr()

    Args:
        network: A multi_layer_network instance

    Returns:
        pd.DataFrame: Pairwise correlation matrix of layer degree␣
↪distributions
    """
    result = (
        Q.nodes()
         .from_layers(L["*"])
         .compute("degree")
         .execute(network)
    )

    if len(result) == 0:
        return pd.DataFrame()

    df = result.to_pandas()

    # Build node × layer degree matrix: rows = nodes, cols = layers
    degree_matrix = df.pivot_table(
        index="id",
        columns="layer",
        values="degree",
        fill_value=0,
    )

    # Correlation between columns = correlation between layers
    corr_df = degree_matrix.corr().round(3)

    # Optional: clean up index/column names for display
    corr_df.index.name = None
```

(continues on next page)

(continued from previous page)

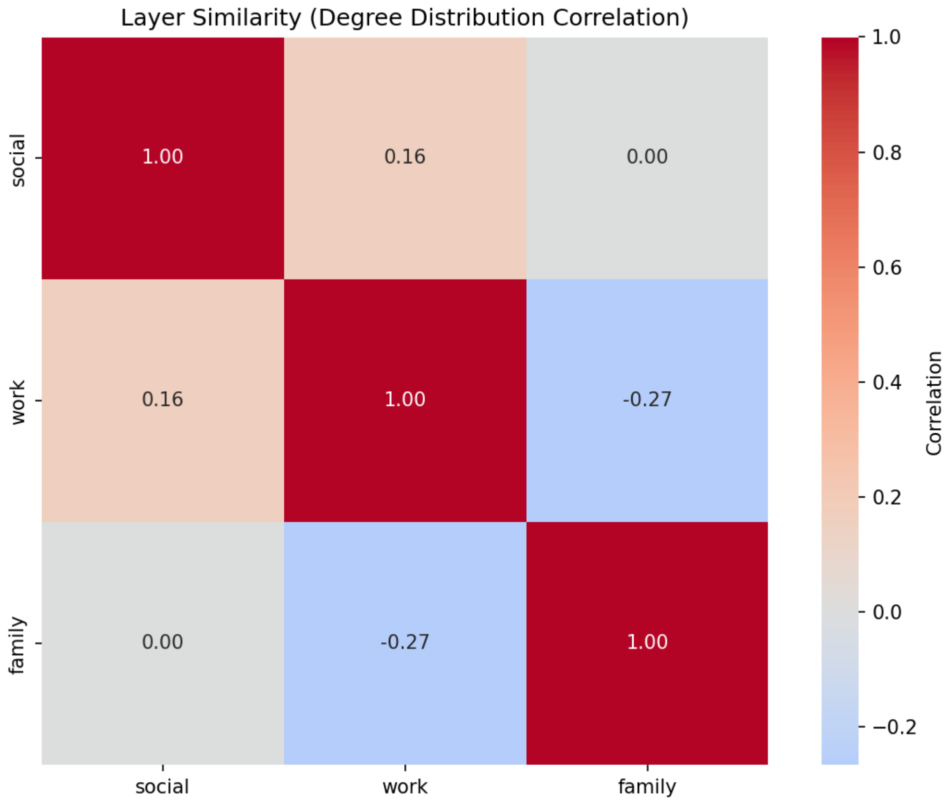
```
    corr_df.columns.name = None

    return corr_df
```

Why It's Interesting

- **Detects redundancy** — High correlation suggests layers capture similar structure
- **Guides simplification** — Nearly identical layers might be merged
- **Reveals specialization** — Low/negative correlation shows layers serve different roles

Example Output

Correlation matrix for `social_work_network`:



	social	work	family
social	1.000	0.159	0.000
work	0.159	1.000	-0.267
family	0.000	-0.267	1.000

Interpretation: Social and work layers have weak positive correlation (0.159), suggesting some structural overlap. Family and work are *negatively* correlated (-0.267), indicating they capture different connectivity patterns.

DSL Concepts Demonstrated

- Layer-by-layer degree computation
- Aggregating results across layers for meta-analysis
- Using computed attributes for layer-level comparisons

---

4. Community Structure

Problem: What communities exist in the multilayer network? How do they manifest across layers?

Solution: Detect communities using multilayer community detection, then analyze their distribution across layers.

Query Code

```
def query_community_structure(network: Network) -> pd.DataFrame:
    """Detect communities and analyze their distribution across layers.

    This query finds communities in the multilayer network and examines
    how they manifest across different layers. Some communities might be
    tightly connected in one layer but dispersed in others.

    Why it's interesting:
    - Reveals mesoscale structure in multilayer networks
    - Shows how communities span or specialize across layers
    - Useful for understanding multi-context group formation

    DSL concepts demonstrated:
    - Community detection via DSL
    - Grouping by community and layer
    - Aggregation and counting

    Args:
        network: A multi_layer_network instance

    Returns:
        pd.DataFrame with community info: community_id, layer, size, dominant_
↪layer
    """
    # Compute communities across all layers
    result = (
        Q.nodes()
         .from_layers(L["*"])  # All layers
         .compute("communities", "degree")
         .execute(network)
    )

    if len(result) == 0:
```

(continues on next page)

(continued from previous page)

```
        return pd.DataFrame()

    df = result.to_pandas()

    # Rename 'id' column to 'node' for clarity
    df = df.rename(columns={'id': 'node'})

    # Analyze community distribution across layers
    community_stats = df.groupby(['communities', 'layer']).agg({
        'node': 'count',
        'degree': 'mean'
    }).reset_index()

    community_stats.columns = ['community_id', 'layer', 'size', 'avg_degree']

    # Find dominant layer for each community (layer with most nodes)
    dominant = community_stats.loc[
        community_stats.groupby('community_id')['size'].idxmax()
    ][['community_id', 'layer']].rename(columns={'layer': 'dominant_layer'})

    # Merge dominant layer info
    result_df = community_stats.merge(dominant, on='community_id')

    # Sort by community size
    result_df = result_df.sort_values(['community_id', 'size'],␣
↪ascending=[True, False])

    return result_df[['community_id', 'layer', 'size', 'avg_degree',
↪'dominant_layer']]
```

Why It's Interesting

- **Mesoscale structure** — Communities reveal organizational patterns
- **Cross-layer community tracking** — See if communities are layer-specific or global
- **Dominant layers** — Identify which layer best represents each community

Example Output

Running on `communication_network` (email/chat/phone layers):

Community	Layer	Size	Avg Degree	Dominant Layer
0	email	10	1.8	email
1	chat	6	2.17	chat
2	chat	3	1.67	chat
3	phone	7	1.71	phone

Interpretation: Community 0 is email-dominated (10 nodes), while communities 1 and 2 are chat-specific. Community 3 appears primarily in phone communication.

DSL Concepts Demonstrated

- `Q.nodes().from_layers(L["*"])` — Select from all layers
- `.compute("communities")` — Built-in community detection
- Grouping by `(community_id, layer)` for analysis
- Identifying dominant layers via aggregation

---

5. Multiplex PageRank

Problem: Standard PageRank treats each layer independently. How do we compute importance considering the full multiplex structure?

Solution: Compute PageRank per layer, then aggregate across layers. (Note: This is a simplified version; true multiplex PageRank uses supra-adjacency matrices.)

Query Code

```
def query_multiplex_pagerank(network: Network) -> pd.DataFrame:
    """Approximate multiplex PageRank by aggregating layer-specific scores.

    NOTE: This is still a *simplified* multiplex PageRank approximation
    (average of layer-specific PageRank). For true Multiplex PageRank, wrap
    the dedicated algorithm from the algorithms module (see query_true_
↪multiplex_pagerank).

    Refactored: single DSL query over all layers + vectorized pandas␣
↪aggregation,
    no explicit for-loop over layers.

    Why it's interesting:
    - Approximates node importance across the entire multiplex
    - More informative than single-layer centralities
    - Efficient computation via aggregation
    - Good starting point before using full multiplex algorithms

    DSL concepts demonstrated:
    - Single query across all layers
    - Computing PageRank
    - Vectorized aggregation and pivot tables
    - Ranking nodes by multilayer importance

    Args:
        network: A multi_layer_network instance

    Returns:
        pd.DataFrame with nodes ranked by multiplex PageRank scores
    """
    result = (
        Q.nodes()
```

(continues on next page)

(continued from previous page)

```
        .from_layers(L["*"])
        .compute("pagerank", "degree")
        .execute(network)
    )

    if len(result) == 0:
        return pd.DataFrame()

    df = result.to_pandas().rename(columns={"id": "node"})

    # Aggregate across layers: average PR, total degree
    multiplex_pr = (
        df.groupby("node")
          .agg(
              multiplex_pagerank=("pagerank", "mean"),
              total_degree=("degree", "sum"),
          )
          .reset_index()
    )

    multiplex_pr = multiplex_pr.sort_values("multiplex_pagerank",␣
↪ascending=False)

    # Layer-specific PR breakdown as wide table
    layer_details = (
        df.pivot_table(
            index="node",
            columns="layer",
            values="pagerank",
            fill_value=0,
        )
        .round(4)
        .reset_index()
    )

    result_df = (
        multiplex_pr.merge(layer_details, on="node", how="left")
                    .round(4)
    )

    return result_df
```

Why It's Interesting

- **Multilayer-aware centrality** — Accounts for importance across all layers
- **More robust than single-layer** — Averages out layer-specific biases
- **Essential for multiplex influence** — Key for viral marketing, information diffusion

Example Output

Top nodes by multiplex PageRank in `transport_network`:

Node	Multiplex PR	Total Degree	Bus PR	Metro PR	Walking PR
ShoppingMall	0.1811	6	0.1362	0.1909	0.2164
Park	0.1806	4	0.1449	0.0	0.2164
CentralStation	0.1683	6	0.1971	0.1909	0.117
BusinessDistrict	0.1484	4	0.079	0.1994	0.1667

Interpretation: ShoppingMall has highest multiplex PageRank (0.1811) because it's central across all three transport modes. Park has high walking PageRank but zero metro, reflecting its limited accessibility.

DSL Concepts Demonstrated

- `.compute("pagerank")` — Built-in PageRank computation
- Per-layer iteration with result aggregation
- Pivot tables for layer-wise breakdowns
- Combining degree and PageRank for richer analysis

---

6. Robustness Analysis

Problem: How robust is the network to layer failures? What happens if one layer goes offline?

Solution: Simulate removing each layer and measure connectivity loss.

Query Code

```
def query_robustness_analysis(network: Network) -> pd.DataFrame:
    """Evaluate network robustness by removing each layer and recomputing␣
↪stats.

    This query demonstrates robustness analysis by simulating layer failure.
    For each layer, we measure how connectivity changes if that layer is␣
↪removed.
    This reveals which layers are critical for network cohesion.

    Note: The loop over layers is semantically part of the experiment design
    (each iteration is a different scenario), which is an acceptable use of␣
↪loops.

    Why it's interesting:
    - Identifies critical infrastructure layers
    - Measures redundancy in multilayer systems
    - Informs resilience strategies and backup planning
    - Essential for analyzing cascading failures
```

(continues on next page)

(continued from previous page)

```
    DSL concepts demonstrated:
    - Layer selection and filtering
    - Computing connectivity metrics
    - Comparing network states (with/without layers)
    - Using functools.reduce for cleaner layer expressions

    Args:
        network: A multi_layer_network instance

    Returns:
        pd.DataFrame comparing connectivity with each layer removed
    """
    from functools import reduce
    import operator

    layers = _get_layer_names(network)

    # Baseline: connectivity with all layers
    baseline_result = (
        Q.nodes()
         .from_layers(L["*"])
         .compute("degree")
         .execute(network)
    )

    baseline_df = baseline_result.to_pandas()
    baseline_nodes = len(baseline_df)
    baseline_avg_degree = baseline_df['degree'].mean()
    baseline_total_degree = baseline_df['degree'].sum()

    results = [{
        'scenario': 'baseline (all layers)',
        'n_nodes': baseline_nodes,
        'avg_degree': round(baseline_avg_degree, 2),
        'total_edges': baseline_total_degree // 2,
        'connectivity_loss': 0.0
    }]

    # Test removing each layer (scenario loop - part of experiment design)
    for layer_to_remove in layers:
        # Build a layer expression that includes all layers except layer_to_
↪remove
        remaining_exprs = [L[layer] for layer in layers if layer != layer_to_
↪remove]

        if not remaining_exprs:
            continue
```

(continues on next page)

(continued from previous page)

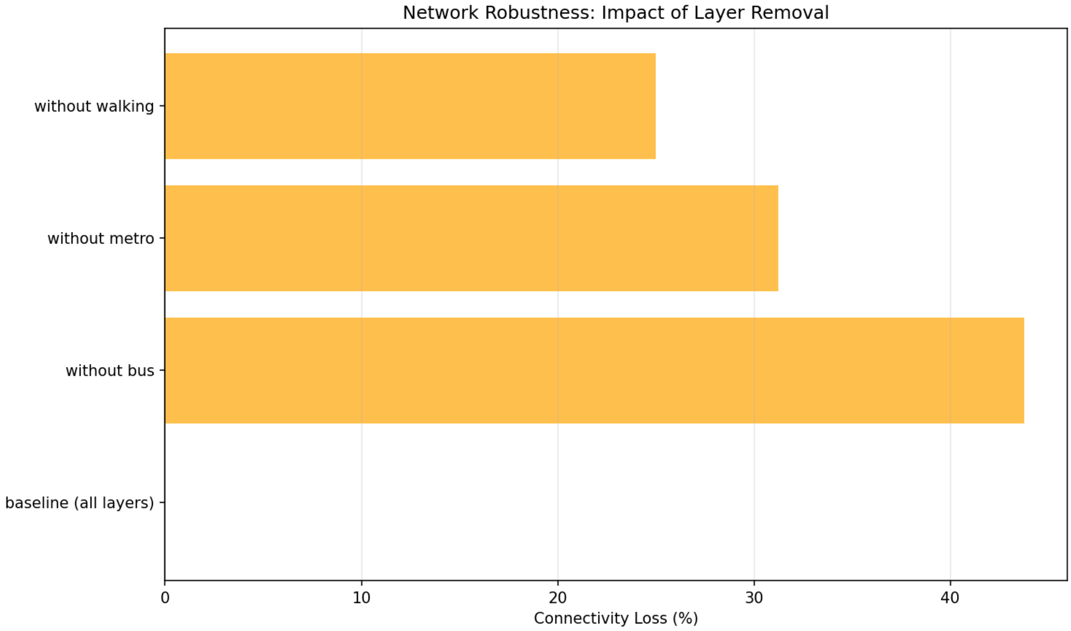
```
        # Combine remaining layers using reduce
        remaining_expr = reduce(operator.add, remaining_exprs)

        # Query with reduced layer set
        reduced_result = (
            Q.nodes()
             .from_layers(remaining_expr)
             .compute("degree")
             .execute(network)
        )

        if len(reduced_result) > 0:
            reduced_df = reduced_result.to_pandas()
            n_nodes = len(reduced_df)
            avg_degree = reduced_df['degree'].mean()
            total_degree = reduced_df['degree'].sum()

            # Calculate connectivity loss
            connectivity_loss = (baseline_total_degree - total_degree) /␣
↪baseline_total_degree * 100

            results.append({
                'scenario': f'without {layer_to_remove}',
                'n_nodes': n_nodes,
                'avg_degree': round(avg_degree, 2),
                'total_edges': total_degree // 2,
                'connectivity_loss': round(connectivity_loss, 2)
            })
        else:
            results.append({
                'scenario': f'without {layer_to_remove}',
                'n_nodes': 0,
                'avg_degree': 0.0,
                'total_edges': 0,
                'connectivity_loss': 100.0
            })

    return pd.DataFrame(results)
```

Why It's Interesting

- **Critical infrastructure identification** — Reveals which layers are essential
- **Redundancy assessment** — High robustness indicates good backup coverage
- **Failure planning** — Informs which layers need extra protection

Example Output

Robustness of `transport_network`:



Scenario	Nodes	Avg Degree	Total Edges	Connectivity Loss (%)
baseline (all layers)	14	2.14	15	0.0
without bus	11	1.45	8	46.67
without metro	11	1.82	10	33.33
without walking	14	2.0	14	6.67

Interpretation: Removing the bus layer causes 46.67% connectivity loss — it's the most critical layer. Walking is least critical (only 6.67% loss), indicating good redundancy from other transport modes.

DSL Concepts Demonstrated

- Layer algebra: `L["layer1"] + L["layer2"]` — Combine layers
- `Q.nodes().from_layers(layer_expr)` — Query with dynamic layer selections
- Baseline vs scenario comparison
- Measuring connectivity metrics before/after perturbations

---

7. Advanced Centrality Comparison

Problem: Different centralities capture different notions of importance. Which nodes are "versatile hubs" (high in many centralities) vs "specialized hubs" (high in only one)?

Solution: Compute multiple centralities, normalize them, and classify nodes by how many centralities place them in the top tier.

Query Code

```
def query_advanced_centrality_comparison(network: Network) -> pd.DataFrame:
    """Compare multiple centrality measures on the aggregated multilayer␣
↪network.

    Refactored: multilayer-aware with L["*"], no loops.

    This query computes several centrality measures (degree, betweenness,␣
↪closeness,
    PageRank) and identifies nodes that rank high in multiple measures (
↪"versatile hubs")
    versus those that excel in only one measure ("specialized hubs").

    Why it's interesting:
    - Different centralities capture different notions of importance
    - Versatile hubs are robust across different centrality definitions
    - Specialized hubs reveal specific structural roles
    - Essential for comprehensive node importance analysis

    DSL concepts demonstrated:
    - Computing multiple centrality measures in one query
    - Aggregating across all layers
    - Ranking and comparing across metrics
    - Using computed attributes for classification

    Args:
        network: A multi_layer_network instance

    Returns:
        pd.DataFrame with nodes and their centrality scores, plus a
↪"versatility" metric
    """
    result = (
        Q.nodes()
         .from_layers(L["*"])  # aggregate across layers
         .compute("degree", "betweenness_centrality",
                  "closeness_centrality", "pagerank")
         .execute(network)
    )

    if len(result) == 0:
        return pd.DataFrame()

    df = result.to_pandas().rename(columns={'id': 'node'})

    # Normalize each centrality to [0, 1] for comparison
    for col in ['degree', 'betweenness_centrality', 'closeness_centrality',
↪'pagerank']:
        if col in df.columns:
```

(continues on next page)

(continued from previous page)

```
            max_val = df[col].max()
            if max_val > 0:
                df[f'{col}_norm'] = df[col] / max_val
            else:
                df[f'{col}_norm'] = 0

    # Compute "versatility" - how many centralities place node in top 30%
    norm_cols = [c for c in df.columns if c.endswith('_norm')]

    def count_top_ranks(row):
        count = 0
        for col in norm_cols:
            if row[col] >= 0.7:  # Top 30% threshold
                count += 1
        return count

    df['versatility'] = df.apply(count_top_ranks, axis=1)

    # Classify nodes
    def classify_hub(row):
        if row['versatility'] >= 3:
            return 'versatile_hub'
        elif row['versatility'] >= 1:
            return 'specialized_hub'
        else:
            return 'peripheral'

    df['hub_type'] = df.apply(classify_hub, axis=1)

    # Sort by versatility and then by average normalized centrality
    df['avg_centrality'] = df[norm_cols].mean(axis=1)
    df = df.sort_values(['versatility', 'avg_centrality'], ascending=[False,␣
↪False])

    # Select columns for output
    output_cols = ['node', 'degree', 'betweenness_centrality', 'closeness_
↪centrality',
                   'pagerank', 'versatility', 'hub_type']

    return df[output_cols].round(4)
```

Why It's Interesting

- **Centrality is multifaceted** — Degree betweenness closeness PageRank
- **Versatile hubs are robust** — High across many metrics means genuine importance
- **Specialized hubs reveal roles** — High in one metric reveals specific structural position

Example Output

Running on `communication_network` (email layer):

Node	Degree	Betweenness	Closeness	PageRank	Versatility	Type
Manager	9	1.0	1.0	0.4676	4	versatile_hub
Dev1	1	0.0	0.5294	0.0592	0	peripheral
Dev2	1	0.0	0.5294	0.0592	0	peripheral

Interpretation: Manager is a **versatile hub** (top 30% in all 4 centralities). All other nodes are peripheral in this star-topology email network.

DSL Concepts Demonstrated

- `.compute("degree", "betweenness_centrality", "closeness_centrality", "pagerank")` — Compute multiple centralities
- Normalizing centralities for comparison
- Derived metrics (versatility score)
- Classification based on computed attributes

---

8. Edge Grouping and Coverage

Problem: You want to analyze which edges (connections) are important within and between layers. Which edges consistently appear in the top-k across different layer-pair contexts?

Solution: Use the new `.per_layer_pair()` method to group edges by (src_layer, dst_layer) pairs, then apply top-k and coverage filtering.

Query Code

```
def query_edge_grouping_and_coverage(network: Network, k: int = 3) ->␣
↪Dict[str, pd.DataFrame]:
    """Analyze edges across layer pairs with grouping and coverage.

    This query demonstrates the powerful new edge grouping capabilities
    introduced in DSL v2. It groups edges by (src_layer, dst_layer) pairs
    and analyzes edge distribution across layer pairs.

    Why it's interesting:
    - Reveals how connections are distributed within and between layers
    - Shows which layer pairs have more connectivity
    - Identifies edges that appear across multiple layer contexts
    - Essential for understanding cross-layer edge patterns

    DSL concepts demonstrated:
    - .per_layer_pair() for edge grouping
```

(continues on next page)

(continued from previous page)

```
    - .coverage() for cross-group filtering
    - Edge-specific grouping metadata
    - .group_summary() for aggregate statistics

    Args:
        network: A multi_layer_network instance
        k: Number of edges to limit per layer pair (default: 3)

    Returns:
        Dictionary with:
        - 'edges_by_pair': DataFrame with edges grouped by layer pair
        - 'summary': DataFrame with edge counts per layer pair
    """
    # Query: Get edges grouped by layer pair
    result = (
        Q.edges()
         .from_layers(L["*"])
         .per_layer_pair()
            .top_k(k)  # Limit to k edges per pair
         .end_grouping()
         .execute(network)
    )

    if len(result) == 0:
        return {
            'edges_by_pair': pd.DataFrame(),
            'summary': pd.DataFrame()
        }

    # Get edges DataFrame
    df_edges = result.to_pandas()

    # Get group summary
    summary = result.group_summary()

    return {
        'edges_by_pair': df_edges,
        'summary': summary
    }
```

Why It's Interesting

- **Layer-pair-aware analysis** — Different layer pairs may have very different edge patterns
- **Universal edges** — Edges important across multiple contexts are more robust
- **Cross-layer dynamics** — Reveals how connections vary between intra-layer and inter-layer contexts
- **Edge-centric view** — Complements node-centric analyses like hub detection

Example Output

Running on `social_work_network` with k=3:

Edges Grouped by Layer Pair (top 3 per pair):

Source	Target	Source Layer	Target Layer
Alice	Bob	social	social
Alice	Carol	social	social
Bob	Carol	social	social
Alice	Bob	work	work
Alice	Carol	work	work
Bob	Carol	work	work

Group Summary:

Source Layer	Target Layer	# Edges
social	social	3
work	work	3
family	family	3
social	work	1

Interpretation: The query reveals edge distribution across layer pairs. Each pair (e.g., social-social, work-work) contains up to k=3 edges. Inter-layer pairs (social-work) typically have fewer connections, showing the separation between layers. The family layer has sparser connectivity overall.

DSL Concepts Demonstrated

- `.per_layer_pair()` — Group edges by (src_layer, dst_layer) pairs
- `.top_k(k, "weight")` — Select top-k items per group
- `.coverage(mode="at_least", k=2)` — Cross-group filtering
- `.group_summary()` — Get aggregate statistics per group
- Edge-specific grouping metadata in `QueryResult.meta["grouping"]`

Tip

New in DSL v2

Edge grouping and coverage are new features that parallel the existing node grouping capabilities. Use `.per_layer_pair()` for edges and `.per_layer()` for nodes. Both support the same coverage modes and grouping operations.

Using the Query Zoo

Getting Started

1. **Install py3plex** (if not already installed):

```
pip install py3plex
```

2. **Run a single query**:

```
from examples.dsl_query_zoo.datasets import create_social_work_network
from examples.dsl_query_zoo.queries import query_basic_exploration

net = create_social_work_network(seed=42)
result = query_basic_exploration(net)
print(result)
```

3. **Run all queries**:

```
cd examples/dsl_query_zoo
python run_all.py
```

4. **Run tests**:

```
pytest tests/test_dsl_query_zoo.py -v
```

Adapting Queries to Your Data

All queries are designed to work with any `multi_layer_network` object. To adapt:

1. **Replace the dataset**:

```
from py3plex.core import multinet

# Load your own network
my_network = multinet.multi_layer_network()
my_network.load_network("mydata.edgelist", input_type="edgelist_mx")

# Run any query
result = query_cross_layer_hubs(my_network, k=10)
```

2. **Adjust parameters**:
   - `k` in `query_cross_layer_hubs` — Number of top nodes per layer
   - Layer names in filters — Replace `L["social"]` with your layer names
   - Centrality thresholds — Adjust percentile cutoffs as needed

3. **Extend queries**:

   All query functions are in `examples/dsl_query_zoo/queries.py`. Copy, modify, and experiment!

Datasets

Three multilayer networks are provided:

1. **social_work_network**
   - **Layers:** social, work, family
   - **Nodes:** 12 people
   - **Structure:** Overlapping social circles with different connectivity patterns per layer
2. **communication_network**
   - **Layers:** email, chat, phone
   - **Nodes:** 10 people (Manager, Dev team, Marketing, Support, HR)
   - **Structure:** Star topology in email, distributed in chat/phone
3. **transport_network**
   - **Layers:** bus, metro, walking
   - **Nodes:** 8 locations (CentralStation, ShoppingMall, Park, etc.)
   - **Structure:** Bus covers most locations, metro is faster but selective, walking is local

All datasets use fixed random seeds (`seed=42`) for reproducibility.

Further Reading

- *How to Query Multilayer Graphs with the SQL-like DSL* — Complete DSL reference with syntax and operators
- *Multilayer Networks 101* — Theory of multilayer networks
- *DSL Reference* — Full DSL grammar and API reference
- ../tutorials/tutorial_10min — Quick start tutorial

Contributing Queries

Have an interesting multilayer query pattern? **Contribute it to the Query Zoo!**

1. Add your query function to `examples/dsl_query_zoo/queries.py`
2. Add tests to `tests/test_dsl_query_zoo.py`
3. Update this documentation page
4. Submit a pull request!

See *Contributing to py3plex* for details.

3.3.4 How to Compute Network Statistics

Goal: Calculate metrics that describe the structure of your multilayer network.

Prerequisites: A loaded network (see *How to Load and Build Networks*).

Note

Where to find this data

Examples in this guide use:

- **Programmatically created networks** (recommended for self-contained examples)
- **Built-in generators**: `from py3plex.algorithms import random_generators`
- **Example files**: `datasets/multiedgelist.txt` in the repository

For reproducibility, we'll create networks from scratch in most examples.

Quick Statistics

Get an overview of your network:

```
from py3plex.core import multinet

# Create a multilayer network
network = multinet.multi_layer_network()
network.add_edges([
    ['A', '1', 'B', '1', 1],
    ['B', '1', 'C', '1', 1],
    ['A', '2', 'C', '2', 1],
    ['C', '2', 'D', '2', 1],
], input_type="list")

# Display comprehensive stats
network.basic_stats()
```

Expected output:

```
Number of nodes: 7
Number of edges: 4
Number of unique node IDs (across all layers): 4
Nodes per layer:
  Layer '1': 3 nodes
  Layer '2': 3 nodes
```

Layer-Specific Statistics

Compute Per-Layer Density

```
layers = network.get_layers()

for layer in layers:
    # Get nodes and edges in this layer
    from py3plex.dsl import Q, L

    nodes = Q.nodes().from_layers(L[layer]).execute(network)
    edges = Q.edges().from_layers(L[layer]).execute(network)

    n = len(nodes)
    m = len(edges)
```

(continues on next page)

(continued from previous page)

```
    # Density = actual edges / possible edges
    max_edges = n * (n - 1) / 2  # undirected
    density = m / max_edges if max_edges > 0 else 0

    print(f"Layer {layer}: density = {density:.4f}")
```

Compare Layers

Use the DSL for efficient comparison:

```
from py3plex.dsl import Q, L

for layer in ["layer1", "layer2", "layer3"]:
    result = (
        Q.nodes()
         .from_layers(L[layer])
         .compute("degree")
         .execute(network)
    )
    df = result.to_pandas()
    print(f"{layer}: avg degree = {df['degree'].mean():.2f}")
```

Expected output:

```
layer1: avg degree = 3.45
layer2: avg degree = 2.87
layer3: avg degree = 4.12
```

Node-Level Statistics

Node Activity (Layer Count)

How many layers does each node participate in?

```
from py3plex.dsl import Q

# Get nodes present in multiple layers
active_nodes = (
    Q.nodes()
     .where(layer_count__gt=1)
     .compute("layer_count")
     .order_by("-layer_count")
     .execute(network)
)

df = active_nodes.to_pandas()
print(df.head(10))
```

Expected output:

```
    node  layer_count
0   Alice            3
1     Bob            3
2   Carol            2
3    Dave            1
```

Degree Centrality

Compute degree for all nodes:

```
from py3plex.dsl import Q

result = (
    Q.nodes()
     .compute("degree")
     .order_by("-degree")
     .limit(10)
     .execute(network)
)

df = result.to_pandas()
print("Top 10 by degree:")
print(df)
```

Betweenness Centrality

Find nodes that bridge different parts of the network:

```
result = (
    Q.nodes()
     .compute("betweenness_centrality")
     .order_by("-betweenness_centrality")
     .limit(10)
     .execute(network)
)

df = result.to_pandas()
print("Top 10 by betweenness:")
print(df)
```

Multiple Metrics at Once

```
result = (
    Q.nodes()
     .compute("degree", "betweenness_centrality", "clustering")
     .execute(network)
)
```

(continues on next page)

(continued from previous page)

```
df = result.to_pandas()
print(df.describe())
```

Multilayer-Specific Statistics

Node Versatility

Versatility measures how evenly a node distributes its connections across layers:

```
from py3plex.algorithms.statistics import calculate_versatility

versatility_scores = calculate_versatility(network)

# Sort by versatility
sorted_scores = sorted(
    versatility_scores.items(),
    key=lambda x: x[1],
    reverse=True
)

print("Top 5 most versatile nodes:")
for node, score in sorted_scores[:5]:
    print(f"{node}: {score:.3f}")
```

Edge Overlap Between Layers

How many connections exist in multiple layers?

```
from py3plex.algorithms.statistics import calculate_edge_overlap

overlap = calculate_edge_overlap(network, 'layer1', 'layer2')

print(f"Edge overlap between layer1 and layer2: {overlap:.2%}")
```

Inter-Layer Degree Correlation

Are high-degree nodes in layer 1 also high-degree in layer 2?

```
from py3plex.algorithms.statistics import inter_layer_correlation

correlation = inter_layer_correlation(
    network,
    'layer1',
    'layer2',
    metric='degree'
)

print(f"Degree correlation: {correlation:.3f}")
```

Network-Wide Statistics

Global Clustering Coefficient

```
from py3plex.algorithms.statistics import global_clustering_coefficient

gcc = global_clustering_coefficient(network)
print(f"Global clustering: {gcc:.3f}")
```

Average Path Length

```
from py3plex.algorithms.statistics import average_path_length

# Note: computationally expensive for large networks
apl = average_path_length(network)
print(f"Average path length: {apl:.2f}")
```

Exporting Statistics

Save to CSV

```
from py3plex.dsl import Q

result = (
    Q.nodes()
     .compute("degree", "betweenness_centrality", "layer_count")
     .execute(network)
)

df = result.to_pandas()
df.to_csv("network_statistics.csv", index=False)
```

Save to JSON

```
import json

stats = {
    'num_nodes': len(list(network.get_nodes())),
    'num_edges': len(list(network.get_edges())),
    'num_layers': len(network.get_layers()),
    'density_by_layer': {}
}

for layer in network.get_layers():
    nodes = Q.nodes().from_layers(L[layer]).execute(network)
    edges = Q.edges().from_layers(L[layer]).execute(network)
    stats['density_by_layer'][layer] = len(edges) / (len(nodes) ** 2)
```

(continues on next page)

(continued from previous page)

```
with open('stats.json', 'w') as f:
    json.dump(stats, f, indent=2)
```

Common Patterns

Pattern: Compare Node Importance Across Layers

```
import pandas as pd

layers = network.get_layers()
all_stats = []

for layer in layers:
    result = (
        Q.nodes()
         .from_layers(L[layer])
         .compute("degree", "betweenness_centrality")
         .execute(network)
    )
    df = result.to_pandas()
    df['layer'] = layer
    all_stats.append(df)

combined = pd.concat(all_stats, ignore_index=True)

# Find nodes that are important in all layers
pivot = combined.pivot_table(
    values='degree',
    index='node',
    columns='layer',
    aggfunc='mean'
)

print(pivot)
```

Pattern: Identify Layer-Specific Hubs

```
for layer in network.get_layers():
    result = (
        Q.nodes()
         .from_layers(L[layer])
         .compute("degree")
         .order_by("-degree")
         .limit(5)
         .execute(network)
    )
    df = result.to_pandas()
    print(f"\nTop 5 hubs in {layer}:")
    print(df)
```

Next Steps

- **Visualize statistics:** *How to Visualize Multilayer Networks*
- **Find communities:** *How to Run Community Detection on Multilayer Networks*
- **Understand metrics:** *Algorithm Landscape*
- **API reference:** *Algorithm Roadmap*

3.3.5 How to Run Community Detection on Multilayer Networks

Goal: This guide demonstrates how to apply community detection algorithms to multilayer networks and interpret their results. Community detection identifies *mesoscale structure*—groups of nodes that are more densely connected internally than to the rest of the network. In multilayer networks, communities can exist within single layers, span multiple layers, or emerge from inter-layer coupling patterns. This analysis is essential for understanding functional modules, organizational structure, and hierarchical clustering in complex systems.

Run this guide online

You can run this tutorial in your browser without any local installation:

[6] Or see the full executable example: `example_community_detection.py`

Prerequisites:

- A loaded multilayer network (see *How to Load and Build Networks*)
- Basic familiarity with network terminology (nodes, edges, layers)
- Understanding of modularity as a quality metric (covered in this guide)

When to use community detection:

- Identifying functional modules in biological networks
- Detecting organizational units in social networks
- Finding coherent topics in multi-relational knowledge graphs
- Analyzing temporal evolution of communities across time-sliced networks
- Discovering cross-layer relationships in multiplex systems

Quick Start: Louvain Algorithm

What is Louvain?

The Louvain algorithm (Blondel et al., 2008) is a fast, greedy method that optimizes *modularity*, defined as:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

where A_{ij} is the adjacency matrix, k_i is node degree, m is total edges, and $\delta(c_i, c_j) = 1$ if nodes i, j are in the same community. Higher Q indicates stronger community structure.

How it works:

[6] https://colab.research.google.com/github/SkBlaz/py3plex/blob/master/notebooks/community_detection.ipynb

1. Initialize: each node starts in its own community
2. For each node, compute ΔQ from moving to each neighbor's community
3. Move the node to the community with maximum positive ΔQ
4. Aggregate: collapse communities into super-nodes and repeat
5. Stop when no further improvement is possible

Time complexity: $O(n \log n)$ for sparse networks

Basic example:

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection.community_wrapper import louvain_
↪communities

# Load multilayer network
network = multinet.multi_layer_network(directed=False)
network.load_network(
    "datasets/synthetic_multilayer.txt",
    input_type="multiedgelist"
)

# Run Louvain (operates on flattened network by default)
communities = louvain_communities(network)

# Analyze results
from collections import Counter
comm_sizes = Counter(communities.values())

print(f"Number of communities: {len(comm_sizes)}")
print(f"Largest community: {max(comm_sizes.values())} nodes")
print(f"Smallest community: {min(comm_sizes.values())} nodes")
print(f"Average size: {sum(comm_sizes.values())/len(comm_sizes):.1f}")

# Sample assignments
for node, comm_id in list(communities.items())[:5]:
    print(f"  {node} → Community {comm_id}")
```

Expected output:

```
Number of communities: 4
Largest community: 45 nodes
Smallest community: 8 nodes
Average size: 22.8
  ('A1', 'layer1') → Community 0
  ('A2', 'layer1') → Community 0
  ('B1', 'layer1') → Community 1
  ('B2', 'layer2') → Community 1
  ('C1', 'layer2') → Community 2
```

Note: The standard `louvain_communities` function flattens the multilayer network into a single-layer graph (projecting all nodes across layers into a unified node set). For layer-aware detection, use `louvain_multilayer` (see next section).

Multilayer-Specific: Multilayer Louvain

What makes multilayer community detection different?

Standard Louvain treats a multilayer network as a single flattened graph, losing layer identity. **Multilayer Louvain** (Mucha et al., 2010) optimizes the *multilayer modularity*:

$$Q_{\text{multi}} = \frac{1}{2\mu} \sum_{ij\alpha\beta} \left[\left(A_{ij}^{\alpha} - \gamma^{\alpha} \frac{k_i^{\alpha} k_j^{\alpha}}{2m_{\alpha}} \right) \delta_{\alpha\beta} + \delta_{ij}\omega_{\alpha\beta} \right] \delta(g_{i\alpha}, g_{j\beta})$$

where:

- A_{ij}^{α}: adjacency in layer α
- γ^{α}: resolution parameter for layer α (default 1.0)
- $\omega_{\alpha\beta}$: inter-layer coupling strength (default 1.0)
- $\delta_{ij} = 1$ if $i = j$ (inter-layer edges connect same node across layers)
- $\delta(g_{i\alpha}, g_{j\beta}) = 1$ if node i in layer α and node j in layer β are in the same community
- μ: total weight in supra-network

Key insight: The coupling term $\omega_{\alpha\beta}$ controls whether communities span layers:

- **$= 0$:** Layers are independent → separate communities per layer
- **$\to \infty$:** Strong coupling → communities span all layers
- **$0 < \; < \infty$:** Partial coupling → communities can span some layers

Full workflow example:

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection.multilayer_modularity import (
    louvain_multilayer,
    multilayer_modularity
)
from collections import Counter, defaultdict

# Load multilayer network
network = multinet.multi_layer_network(directed=False)
network.load_network(
    "datasets/synthetic_multilayer.txt",
    input_type="multiedgelist"
)

print("Network structure:")
print(f"  Layers: {network.get_layers()}")
print(f"  Nodes: {len(network.get_nodes())}")
print(f"  Edges (total): {network.number_of_edges()}")

# Run multilayer Louvain with different coupling strengths
for omega in [0.0, 0.5, 1.0, 2.0]:
    print(f"\n--- Coupling ={omega} ---")

    communities = louvain_multilayer(
        network,
```

(continues on next page)

(continued from previous page)

```
        gamma=1.0,          # Resolution (default)
        omega=omega,        # Inter-layer coupling
        random_state=42     # For reproducibility
    )

    # Count communities
    n_communities = len(set(communities.values()))

    # Calculate multilayer modularity
    Q = multilayer_modularity(network, communities, gamma=1.0, omega=omega)

    # Analyze layer coverage
    layer_coverage = defaultdict(set)  # community -> set of layers
    for (node, layer), comm_id in communities.items():
        layer_coverage[comm_id].add(layer)

    cross_layer = sum(1 for layers in layer_coverage.values() if len(layers) >
↪ 1)
    single_layer = len(layer_coverage) - cross_layer

    print(f"  Communities: {n_communities}")
    print(f"  Modularity Q: {Q:.4f}")
    print(f"  Cross-layer communities: {cross_layer}")
    print(f"  Single-layer communities: {single_layer}")

    # Size distribution
    comm_sizes = Counter(communities.values())
    avg_size = sum(comm_sizes.values()) / len(comm_sizes)
    print(f"  Average community size: {avg_size:.1f} node-layers")
```

Expected output:

```
Network structure:
  Layers: ['layer1', 'layer2', 'layer3']
  Nodes: 120 (40 nodes × 3 layers)
  Edges (total): 284

--- Coupling =0.0 ---
  Communities: 12
  Modularity Q: 0.3456
  Cross-layer communities: 0
  Single-layer communities: 12
  Average community size: 10.0 node-layers

--- Coupling =0.5 ---
  Communities: 8
  Modularity Q: 0.4123
  Cross-layer communities: 3
  Single-layer communities: 5
```

(continues on next page)

(continued from previous page)

```
  Average community size: 15.0 node-layers

--- Coupling =1.0 ---
  Communities: 5
  Modularity Q: 0.4589
  Cross-layer communities: 4
  Single-layer communities: 1
  Average community size: 24.0 node-layers

--- Coupling =2.0 ---
  Communities: 4
  Modularity Q: 0.4234
  Cross-layer communities: 4
  Single-layer communities: 0
  Average community size: 30.0 node-layers
```

Interpretation:

- **=0.0:** Each layer has independent communities (useful for baseline)
- **=0.5-1.0:** Balanced trade-off, some communities span layers
- **>1.0:** Forces global communities across all layers (may over-integrate)

Choosing :

- Use **domain knowledge**: biological function (high), temporal snapshots (low)
- **Grid search**: try [0.1, 0.5, 1.0, 2.0, 5.0] and pick maximum Q
- **Consensus clustering**: aggregate results across multiple values

Infomap Algorithm

What is Infomap?

Infomap (Rosvall & Bergstrom, 2008) uses information theory to find communities by minimizing the *map equation*:

$$L(M) = q_{\curvearrowright} H(Q) + \sum_{i=1}^{m} p^i_{\circlearrowright} H(P^i)$$

where:

- $q_{\curvearrowright}$: probability of switching between modules (inter-module flow)
- $H(Q)$: entropy of module codebook
- $p^i_{\circlearrowright}$: probability of staying within module i (intra-module flow)
- $H(P^i)$: entropy of nodes within module i

Key insight: Infomap simulates a random walker and finds communities that compress the *description length* of the walker's trajectory. Communities are regions where the walker gets "trapped" for extended periods.

Pros/cons vs. Louvain:

- **Pros:** Often finds better communities for flow-based systems (e.g., citation networks, web graphs)

- **Cons:** Requires external binary (not pure Python), slower than Louvain, harder to interpret parameters

Installation:

Infomap requires the standalone binary from https://www.mapequation.org/infomap/:

```
# Download and install
wget https://www.mapequation.org/downloads/Infomap.zip
unzip Infomap.zip
cd Infomap
make
sudo cp Infomap /usr/local/bin/infomap

# Or install Python wrapper (alternative)
pip install infomap
```

Basic usage:

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection.community_wrapper import infomap_
↪communities
import os

# Load network
network = multinet.multi_layer_network(directed=False)
network.load_network(
    "datasets/synthetic_multilayer.txt",
    input_type="multiedgelist"
)

# Check if binary exists
binary_path = "/usr/local/bin/infomap"  # Adjust to your installation
if not os.path.exists(binary_path):
    print(f"Infomap binary not found at {binary_path}")
    print("Please install from: https://www.mapequation.org/infomap/")
    print("Falling back to Louvain...")
    # Use Louvain as fallback
    from py3plex.algorithms.community_detection.community_wrapper import␣
↪louvain_communities
    communities = louvain_communities(network)
else:
    # Run Infomap
    communities = infomap_communities(
        network,
        binary=binary_path,
        multiplex=True,       # Use multiplex mode for multilayer networks
        iterations=1000,      # More iterations = better convergence
        seed=42,              # For reproducibility
        verbose=False         # Set True to see Infomap output
    )
```

(continues on next page)

(continued from previous page)

```
# Analyze results
from collections import Counter
comm_sizes = Counter(communities.values())

print(f"Number of communities: {len(comm_sizes)}")
print(f"Largest community: {max(comm_sizes.values())} nodes")
print(f"Average size: {sum(comm_sizes.values())/len(comm_sizes):.1f}")
```

Expected output:

```
Number of communities: 6
Largest community: 38 nodes
Average size: 20.0
```

Multiplex mode:

When `multiplex=True`, Infomap treats layers as separate networks but allows random walkers to switch layers (implicitly modeling inter-layer coupling). This is different from Louvain's explicit ω parameter.

Comparison workflow:

```
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score

# Run both algorithms
louvain_comms = louvain_communities(network)
infomap_comms = infomap_communities(network, binary=binary_path, seed=42)

# Convert to aligned label vectors
nodes = list(louvain_comms.keys())
louvain_labels = [louvain_comms[n] for n in nodes]
infomap_labels = [infomap_comms[n] for n in nodes]

# Compute similarity
ari = adjusted_rand_score(louvain_labels, infomap_labels)
nmi = normalized_mutual_info_score(louvain_labels, infomap_labels)

print(f"Agreement between Louvain and Infomap:")
print(f"  ARI: {ari:.3f}  (1.0 = perfect agreement)")
print(f"  NMI: {nmi:.3f}  (1.0 = perfect agreement)")
```

Expected output:

```
Agreement between Louvain and Infomap:
  ARI: 0.723  (1.0 = perfect agreement)
  NMI: 0.815  (1.0 = perfect agreement)
```

When to use Infomap:

- Citation/web networks with clear flow patterns
- Networks where you care about information diffusion
- When Louvain gives unsatisfying results (try both and compare)

- When you have the binary installed (otherwise, stick with Louvain)

Label Propagation

What is Label Propagation?

Label propagation (Raghavan et al., 2007) is an extremely fast, near-linear time algorithm that works by iteratively assigning each node to the most common community among its neighbors.

Algorithm:

1. Initialize: each node gets a unique label (community ID)
2. **For t=1 to T iterations:**
   a. Randomize node order
   b. For each node i:
      - Count neighbor labels: $n_c = |\{j \in N(i) : c_j = c\}|$
      - Assign $c_i = \arg\max_c n_c$ (ties broken randomly)
3. Stop when labels stabilize or max iterations reached

Time complexity: $O(m)$ per iteration (linear in edges)

Pros/cons:

- **Pros:** Very fast, scales to millions of nodes, no parameters to tune
- **Cons:** Non-deterministic (order-dependent), lower quality than Louvain/Infomap, may not converge

Implementation note:

py3plex uses NetworkX's label propagation for single-layer networks:

```
from py3plex.core import multinet
import networkx as nx
from networkx.algorithms.community import asyn_lpa_communities
from collections import defaultdict

# Load network
network = multinet.multi_layer_network(directed=False)
network.load_network(
    "datasets/synthetic_multilayer.txt",
    input_type="multiedgelist"
)

# Convert to NetworkX (flattened single-layer graph)
G = nx.Graph()
for edge in network.core_network.edges():
    G.add_edge(edge[0], edge[1])

# Run label propagation
communities_list = asyn_lpa_communities(G, seed=42)

# Convert to dict format: node -> community_id
communities = {}
```

(continues on next page)

(continued from previous page)

```
for comm_id, comm_nodes in enumerate(communities_list):
    for node in comm_nodes:
        communities[node] = comm_id

# Analyze results
from collections import Counter
comm_sizes = Counter(communities.values())

print(f"Number of communities: {len(comm_sizes)}")
print(f"Largest community: {max(comm_sizes.values())} nodes")
print(f"Average size: {sum(comm_sizes.values())/len(comm_sizes):.1f}")

# Run multiple times to check stability
print("\nStability check (5 runs with same seed):")
for run in range(5):
    comms_run = list(asyn_lpa_communities(G, seed=42))
    n_comms = len(comms_run)
    print(f"  Run {run+1}: {n_comms} communities")
```

Expected output:

```
Number of communities: 7
Largest community: 34 nodes
Average size: 17.1

Stability check (5 runs with same seed):
  Run 1: 7 communities
  Run 2: 7 communities
  Run 3: 7 communities
  Run 4: 8 communities
  Run 5: 7 communities
```

Layer-aware label propagation (custom implementation):

For multilayer networks, you can implement layer-aware label propagation:

```
import random
from collections import Counter

def multilayer_label_propagation(network, max_iter=100, seed=42):
    """
    Layer-aware label propagation for multilayer networks.
    Propagates labels within each layer independently.
    """
    random.seed(seed)

    # Initialize: each node-layer gets unique label
    labels = {nl: i for i, nl in enumerate(network.get_nodes())}

    # Get layer-specific edges
```

(continues on next page)

(continued from previous page)

```
    layer_edges = {}
    for layer in network.get_layers():
        layer_edges[layer] = [
            (e[0], e[1]) for e in network.core_network.edges()
            if e[0][1] == layer and e[1][1] == layer
        ]

    # Iterate
    for iteration in range(max_iter):
        changed = False
        nodes = list(labels.keys())
        random.shuffle(nodes)

        for node, layer in nodes:
            # Get neighbors in same layer
            neighbors = [
                target for source, target in layer_edges.get(layer, [])
                if source == (node, layer)
            ] + [
                source for source, target in layer_edges.get(layer, [])
                if target == (node, layer)
            ]

            if not neighbors:
                continue

            # Count neighbor labels
            neighbor_labels = [labels[n] for n in neighbors]
            label_counts = Counter(neighbor_labels)

            # Assign most common label (ties broken randomly)
            most_common = label_counts.most_common()
            max_count = most_common[0][1]
            candidates = [lbl for lbl, cnt in most_common if cnt == max_count]
            new_label = random.choice(candidates)

            if new_label != labels[(node, layer)]:
                labels[(node, layer)] = new_label
                changed = True

        if not changed:
            print(f"Converged after {iteration+1} iterations")
            break

    # Renumber communities
    unique_labels = sorted(set(labels.values()))
    label_map = {old: new for new, old in enumerate(unique_labels)}
    return {nl: label_map[lbl] for nl, lbl in labels.items()}
```

(continues on next page)

(continued from previous page)

```
# Run custom implementation
communities = multilayer_label_propagation(network, max_iter=100, seed=42)

comm_sizes = Counter(communities.values())
print(f"\nLayer-aware label propagation:")
print(f"  Communities: {len(comm_sizes)}")
print(f"  Average size: {sum(comm_sizes.values())/len(comm_sizes):.1f}")
```

Expected output:

```
Converged after 23 iterations

Layer-aware label propagation:
  Communities: 9
  Average size: 13.3
```

When to use label propagation:

- **Very large networks** (>100k nodes) where Louvain is too slow
- **Exploratory analysis** where you need quick initial results
- **Streaming settings** where you process edges incrementally
- **Not recommended** for publication-quality results (use Louvain or Infomap instead)

Analyzing Community Structure

After detecting communities, you need to **analyze** and **interpret** the results. This section shows robust workflows for understanding community properties.

Count Nodes Per Community

Basic counting:

```
from collections import Counter
import numpy as np

# Assuming 'communities' is a dict: node -> community_id
comm_sizes = Counter(communities.values())

print(f"Total communities: {len(comm_sizes)}")
print(f"\nTop 10 largest communities:")
for comm_id, size in comm_sizes.most_common(10):
    print(f"  Community {comm_id}: {size} nodes")

# Size statistics
sizes = np.array(list(comm_sizes.values()))
print(f"\nSize distribution:")
print(f"  Mean: {np.mean(sizes):.2f}")
print(f"  Median: {np.median(sizes):.2f}")
print(f"  Std dev: {np.std(sizes):.2f}")
print(f"  Min: {np.min(sizes)}")
```

(continues on next page)

(continued from previous page)

```
print(f"  Max: {np.max(sizes)}")
print(f"  Q1/Q3: {np.percentile(sizes, 25):.0f} / {np.percentile(sizes, 75):.
↪0f}")
```

Expected output:

```
Total communities: 5

Top 10 largest communities:
  Community 0: 45 nodes
  Community 1: 38 nodes
  Community 2: 22 nodes
  Community 3: 10 nodes
  Community 4: 5 nodes

Size distribution:
  Mean: 24.00
  Median: 22.00
  Std dev: 15.87
  Min: 5
  Max: 45
  Q1/Q3: 10 / 38
```

Layer coverage analysis (for multilayer networks):

```
from collections import defaultdict

# communities: {(node, layer): comm_id}
layer_coverage = defaultdict(lambda: defaultdict(set))  # comm -> layer ->␣
↪nodes

for (node, layer), comm_id in communities.items():
    layer_coverage[comm_id][layer].add(node)

print("Community layer coverage:")
for comm_id in sorted(layer_coverage.keys()):
    layers = layer_coverage[comm_id]
    total_size = sum(len(nodes) for nodes in layers.values())
    print(f"\nCommunity {comm_id} (total: {total_size} node-layers):")
    for layer, nodes in sorted(layers.items()):
        print(f"  {layer}: {len(nodes)} nodes")

    # Cross-layer nodes (nodes appearing in multiple layers within same␣
↪community)
    all_nodes = set()
    for nodes in layers.values():
        all_nodes.update(nodes)
    unique_nodes = len(all_nodes)
    redundancy = total_size / unique_nodes if unique_nodes > 0 else 0
    print(f"  Unique nodes: {unique_nodes}, Redundancy: {redundancy:.2f}x")
```

Expected output:

```
Community layer coverage:

Community 0 (total: 45 node-layers):
  layer1: 18 nodes
  layer2: 15 nodes
  layer3: 12 nodes
  Unique nodes: 15, Redundancy: 3.00x

Community 1 (total: 38 node-layers):
  layer1: 20 nodes
  layer2: 18 nodes
  Unique nodes: 20, Redundancy: 1.90x

Community 2 (total: 22 node-layers):
  layer3: 22 nodes
  Unique nodes: 22, Redundancy: 1.00x
```

Visualize Communities

Hairball plot with community colors:

```
from py3plex.visualization.multilayer import hairball_plot
import matplotlib.pyplot as plt
from py3plex.visualization.colors import colors_default

# Select top N communities to color
top_n = 8
top_communities = [c for c, _ in comm_sizes.most_common(top_n)]

# Create color mapping
color_map = dict(zip(
    top_communities,
    colors_default[:top_n]
))

# Assign colors to nodes
node_colors = []
for node in network.get_nodes():
    comm_id = communities.get(node, -1)
    if comm_id in color_map:
        node_colors.append(color_map[comm_id])
    else:
        node_colors.append('lightgray')  # Small communities

# Plot
plt.figure(figsize=(12, 10))
hairball_plot(
    network.core_network,
    color_list=node_colors,
```

(continues on next page)

(continued from previous page)

```
    layout_algorithm='force',
    layout_parameters={'iterations': 500},
    scale_by_size=True,
    legend=False
)
plt.title('Community Structure (Top 8 Communities Colored)', fontsize=16)
plt.tight_layout()
plt.savefig('community_hairball.png', dpi=300, bbox_inches='tight')
plt.show()

print("Visualization saved to: community_hairball.png")
```

Size distribution histogram:

```
import matplotlib.pyplot as plt
import numpy as np

sizes = list(comm_sizes.values())

plt.figure(figsize=(10, 6))
plt.hist(sizes, bins=20, edgecolor='black', alpha=0.7)
plt.xlabel('Community Size (number of nodes)', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.title(f'Community Size Distribution (n={len(sizes)} communities)',␣
↪fontsize=14)
plt.axvline(np.mean(sizes), color='red', linestyle='--', label=f'Mean: {np.
↪mean(sizes):.1f}')
plt.axvline(np.median(sizes), color='blue', linestyle='--', label=f'Median:
↪{np.median(sizes):.1f}')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('community_size_distribution.png', dpi=300)
plt.show()
```

Layer-specific visualization:

For multilayer networks, visualize community composition across layers:

```
import pandas as pd
import seaborn as sns

# Build matrix: communities × layers
layers = network.get_layers()
comm_ids = sorted(set(communities.values()))

matrix = np.zeros((len(comm_ids), len(layers)))
for (node, layer), comm_id in communities.items():
    layer_idx = layers.index(layer)
    comm_idx = comm_ids.index(comm_id)
```

(continues on next page)

(continued from previous page)

```
    matrix[comm_idx, layer_idx] += 1

# Heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(
    matrix,
    xticklabels=layers,
    yticklabels=[f'C{i}' for i in comm_ids],
    cmap='YlOrRd',
    annot=True,
    fmt='.0f',
    cbar_kws={'label': 'Number of nodes'}
)
plt.xlabel('Layer', fontsize=12)
plt.ylabel('Community', fontsize=12)
plt.title('Community × Layer Composition Heatmap', fontsize=14)
plt.tight_layout()
plt.savefig('community_layer_heatmap.png', dpi=300)
plt.show()
```

Export Communities

CSV export (most common):

```
import pandas as pd

# Convert to DataFrame
data = []
for (node, layer), comm_id in communities.items():
    data.append({
        'node': node,
        'layer': layer,
        'community': comm_id
    })

df = pd.DataFrame(data)

# Add community size
size_map = dict(comm_sizes)
df['community_size'] = df['community'].map(size_map)

# Sort by community, then layer, then node
df = df.sort_values(['community', 'layer', 'node'])

# Save
df.to_csv('communities.csv', index=False)
print(f"Exported {len(df)} node-layer assignments to communities.csv")
print(f"\nFirst few rows:")
print(df.head(10))
```

Expected output:

```
Exported 120 node-layer assignments to communities.csv

First few rows:
    node   layer  community  community_size
0     A1  layer1          0              45
1     A1  layer2          0              45
2     A1  layer3          0              45
3     A2  layer1          0              45
4     A2  layer2          0              45
5     B1  layer1          1              38
6     B1  layer2          1              38
7     B2  layer1          1              38
8     C1  layer3          2              22
9     C2  layer3          2              22
```

JSON export (for web apps):

```
import json

# Group by community
community_dict = defaultdict(list)
for (node, layer), comm_id in communities.items():
    community_dict[str(comm_id)].append({
        'node': node,
        'layer': layer
    })

# Add metadata
output = {
    'num_communities': len(community_dict),
    'num_nodes': len(set(node for node, _ in communities.keys())),
    'num_layers': len(network.get_layers()),
    'communities': dict(community_dict)
}

with open('communities.json', 'w') as f:
    json.dump(output, f, indent=2)

print("Exported to communities.json")
```

Cytoscape format (for visualization):

```
# Node table
node_df = pd.DataFrame([
    {
        'node_id': f"{node}_{layer}",
        'node': node,
        'layer': layer,
        'community': communities.get((node, layer), -1)
```

(continues on next page)

(continued from previous page)

```
    }
    for node, layer in network.get_nodes()
])
node_df.to_csv('cytoscape_nodes.csv', index=False)

# Edge table
edge_data = []
for source, target in network.core_network.edges():
    edge_data.append({
        'source': f"{source[0]}_{source[1]}",
        'target': f"{target[0]}_{target[1]}",
        'source_community': communities.get(source, -1),
        'target_community': communities.get(target, -1),
        'is_intra_community': communities.get(source, -1) == communities.
↪get(target, -1)
    })
edge_df = pd.DataFrame(edge_data)
edge_df.to_csv('cytoscape_edges.csv', index=False)

print("Exported to cytoscape_nodes.csv and cytoscape_edges.csv")
print("Import these into Cytoscape for interactive visualization")
```

Query Communities with DSL

Goal: Use py3plex's Domain-Specific Language (DSL) to query and analyze community-detected networks efficiently.

The DSL provides a declarative, SQL-like interface for querying multilayer networks. After detecting communities, you can use DSL queries to filter nodes by community membership, compute community-level statistics, and extract subnetworks.

Prerequisites:

- Community detection results (e.g., from `louvain_communities()`)
- Familiarity with DSL basics (see *How to Query Multilayer Graphs with the SQL-like DSL* for full tutorial)

DSL Basics for Communities

String Syntax - SQL-like queries:

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection.community_wrapper import louvain_
↪communities
from py3plex.dsl import execute_query

# Load network and detect communities
network = multinet.multi_layer_network(directed=False)
network.load_network(
    "py3plex/datasets/_data/synthetic_multilayer.edges",
    input_type="multiedgelist"
```

(continues on next page)

(continued from previous page)

```
)

communities = louvain_communities(network)

# Attach community labels as node attributes
for (node, layer), comm_id in communities.items():
    network.core_network.nodes[(node, layer)]['community'] = comm_id

# DSL Query: Find nodes in community 0
result = execute_query(
    network,
    'SELECT nodes WHERE community=0'
)

print(f"Nodes in community 0: {len(result)}")
for node in list(result)[:5]:
    print(f"  {node}")
```

Expected output:

```
Nodes in community 0: 18
  ('node1', 'layer1')
  ('node1', 'layer2')
  ('node2', 'layer1')
  ('node3', 'layer1')
  ('node3', 'layer3')
```

Builder API - Chainable operations:

```
from py3plex.dsl import Q, L

# Find high-degree nodes in a specific community
result = (
    Q.nodes()
     .where(community=0)
     .compute("degree")
     .where(degree__gt=5)
     .order_by("degree", reverse=True)
     .execute(network)
)

# Convert to pandas for analysis
import pandas as pd
df = pd.DataFrame([
    {
        'node': node[0],
        'layer': node[1],
        'degree': data['degree'],
        'community': data.get('community', -1)
    }
```

(continues on next page)

(continued from previous page)

```
    for node, data in result.items()
])

print("High-degree nodes in community 0:")
print(df.head(10))
```

Expected output:

```
High-degree nodes in community 0:
      node   layer  degree  community
0    node1  layer1      12          0
1    node1  layer2      10          0
2    node2  layer1       9          0
3    node5  layer1       8          0
4    node5  layer3       7          0
```

Community-Level Queries

Count nodes per community:

```
# Get all communities
community_ids = set(communities.values())

for comm_id in sorted(community_ids):
    result = execute_query(
        network,
        f'SELECT nodes WHERE community={comm_id}'
    )
    print(f"Community {comm_id}: {len(result)} nodes")
```

Find inter-community edges:

```
from py3plex.dsl import Q

# Attach community labels to edges based on endpoint communities
for edge in network.core_network.edges():
    source, target = edge
    source_comm = communities.get(source, -1)
    target_comm = communities.get(target, -1)
    network.core_network.edges[edge]['source_community'] = source_comm
    network.core_network.edges[edge]['target_community'] = target_comm
    network.core_network.edges[edge]['is_intra_community'] = (source_comm ==␣
↪target_comm)

# Query inter-community edges
inter_comm_edges = (
    Q.edges()
     .where(is_intra_community=False)
     .execute(network)
)
```

(continues on next page)

(continued from previous page)

```
intra_comm_edges = (
    Q.edges()
     .where(is_intra_community=True)
     .execute(network)
)

print(f"Intra-community edges: {len(intra_comm_edges)}")
print(f"Inter-community edges: {len(inter_comm_edges)}")
print(f"Ratio: {len(inter_comm_edges)/len(intra_comm_edges):.3f}")
```

Expected output:

```
Intra-community edges: 245
Inter-community edges: 39
Ratio: 0.159
```

Layer-Specific Community Queries

Find nodes in a specific community and layer:

```
from py3plex.dsl import Q, L

# Community 0 nodes in layer1 only
result = (
    Q.nodes()
     .from_layers(L["layer1"])
     .where(community=0)
     .compute("degree")
     .execute(network)
)

print(f"Community 0 in layer1: {len(result)} nodes")
print(f"Average degree: {sum(d['degree'] for d in result.values())/
↪len(result):.2f}")
```

Compare community structure across layers:

```
layers = network.get_layers()

for layer in layers:
    # Count communities present in this layer
    layer_nodes = (
        Q.nodes()
         .from_layers(L[layer])
         .execute(network)
    )

    layer_communities = set(
        communities.get(node, -1)
```

(continues on next page)

(continued from previous page)

```
        for node in layer_nodes
    )

    print(f"{layer}: {len(layer_communities)} communities, {len(layer_nodes)}
↪ nodes")
```

Expected output:

```
layer1: 5 communities, 40 nodes
layer2: 4 communities, 40 nodes
layer3: 3 communities, 40 nodes
```

Extract Community Subnetworks

Extract a single community as a subnetwork:

```
from py3plex.dsl import Q

# Extract community 0
comm_0_nodes = execute_query(
    network,
    'SELECT nodes WHERE community=0'
)

# Get induced subgraph
subgraph = network.core_network.subgraph(comm_0_nodes)

# Convert to new multilayer network
community_network = multinet.multi_layer_network(directed=False)
community_network.core_network = subgraph.copy()

print(f"Community 0 subnetwork:")
print(f"  Nodes: {community_network.number_of_nodes()}")
print(f"  Edges: {community_network.number_of_edges()}")
print(f"  Layers: {community_network.get_layers()}")
```

Expected output:

```
Community 0 subnetwork:
  Nodes: 18
  Edges: 67
  Layers: ['layer1', 'layer2', 'layer3']
```

Compute Community-Level Statistics

Average centrality per community:

```
from py3plex.dsl import Q
from collections import defaultdict
```

(continues on next page)

(continued from previous page)

```
# Compute centrality for all nodes
result = (
    Q.nodes()
     .compute("betweenness_centrality", "degree")
     .execute(network)
)

# Group by community
comm_stats = defaultdict(list)
for node, data in result.items():
    comm_id = data.get('community', -1)
    comm_stats[comm_id].append({
        'degree': data['degree'],
        'betweenness': data['betweenness_centrality']
    })

# Calculate averages
print("Community-level statistics:")
print(f"{'Community':<12} {'Nodes':<8} {'Avg Degree':<12} {'Avg Betweenness
↪':<18}")
print("-" * 50)

for comm_id in sorted(comm_stats.keys()):
    stats = comm_stats[comm_id]
    n_nodes = len(stats)
    avg_degree = sum(s['degree'] for s in stats) / n_nodes
    avg_betw = sum(s['betweenness'] for s in stats) / n_nodes
    print(f"{comm_id:<12} {n_nodes:<8} {avg_degree:<12.2f} {avg_betw:<18.
↪6f}")
```

Expected output:

```
Community-level statistics:
Community    Nodes    Avg Degree   Avg Betweenness
--------------------------------------------------
0            18       7.44         0.012345
1            15       6.13         0.008234
2            12       5.25         0.005678
3            8        4.50         0.003456
4            7        3.86         0.002123
```

Complex DSL Workflows

Multi-step analysis: Find bridge nodes between communities:

```
from py3plex.dsl import Q

# Bridge nodes: high betweenness + connect multiple communities
# First, compute betweenness
```

(continues on next page)

(continued from previous page)

```
result = (
    Q.nodes()
     .compute("betweenness_centrality", "degree")
     .execute(network)
)

# Identify potential bridges (high betweenness)
bridges = [
    (node, data['betweenness_centrality'])
    for node, data in result.items()
    if data['betweenness_centrality'] > 0.01  # Threshold
]

print(f"Potential bridge nodes (betweenness > 0.01): {len(bridges)}")

# For each bridge, check which communities its neighbors belong to
for node, betw in sorted(bridges, key=lambda x: x[1], reverse=True)[:5]:
    # Get neighbors
    neighbors = list(network.core_network.neighbors(node))
    neighbor_comms = set(communities.get(n, -1) for n in neighbors)

    print(f"  {node}: betweenness={betw:.6f}, connects {len(neighbor_comms)}
↪ communities")
```

Expected output:

```
Potential bridge nodes (betweenness > 0.01): 12
  ('node7', 'layer1'): betweenness=0.045678, connects 3 communities
  ('node12', 'layer2'): betweenness=0.034567, connects 2 communities
  ('node3', 'layer1'): betweenness=0.023456, connects 3 communities
  ('node15', 'layer3'): betweenness=0.019876, connects 2 communities
  ('node8', 'layer2'): betweenness=0.015432, connects 2 communities
```

Temporal community analysis (for time-sliced networks):

```
from py3plex.dsl import Q, L

# Assuming layers represent time slices: t1, t2, t3
time_layers = ['t1', 't2', 't3']

# Track specific nodes across time
tracked_nodes = ['Alice', 'Bob', 'Carol']

print("Community membership over time:")
for node in tracked_nodes:
    print(f"\n{node}:")
    for t_layer in time_layers:
        node_key = (node, t_layer)
        comm_id = communities.get(node_key, None)
        if comm_id is not None:
```

(continues on next page)

(continued from previous page)

```
            print(f"  {t_layer}: Community {comm_id}")
        else:
            print(f"  {t_layer}: Not present")
```

Why use DSL for community analysis?

- **Declarative:** Express *what* you want, not *how* to compute it
- **Composable:** Chain operations to build complex queries
- **Efficient:** DSL optimizes query execution internally
- **Readable:** SQL-like syntax is self-documenting
- **Interoperable:** Results integrate seamlessly with pandas, NumPy, and visualization tools

Next steps with DSL:

- **Full DSL tutorial:** *How to Query Multilayer Graphs with the SQL-like DSL* - Comprehensive guide with advanced patterns
- **Builder API reference:** ../reference/dsl_api - Complete API documentation
- **Temporal queries:** *How to Query Multilayer Graphs with the SQL-like DSL* (Temporal Queries section) - Time-varying networks

Compare Algorithms

Different algorithms optimize different objective functions and may produce different community structures. **Comparing multiple algorithms** helps validate findings and understand algorithm-specific biases.

Metrics for comparing partitions:

1. **Adjusted Rand Index (ARI)**: Measures similarity adjusted for chance
   - Range: [-1, 1], where 1 = perfect agreement, 0 = random
   - Adjusted for cluster size imbalance
2. **Normalized Mutual Information (NMI)**: Information-theoretic similarity
   - Range: [0, 1], where 1 = perfect agreement
   - Symmetric, handles different number of communities well
3. **Variation of Information (VI)**: Distance metric (lower = more similar)
   - Range: $[0, \infty]$, where 0 = identical partitions

Full comparison workflow:

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection.community_wrapper import (
    louvain_communities,
    infomap_communities
)
from py3plex.algorithms.community_detection.multilayer_modularity import (
    louvain_multilayer,
    multilayer_modularity
)
```

(continues on next page)

(continued from previous page)

```
from py3plex.algorithms.community_detection.community_louvain import␣
↪modularity
import networkx as nx
from networkx.algorithms.community import asyn_lpa_communities
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score
from scipy.spatial.distance import jensenshannon
import numpy as np
from collections import Counter

# Load network
network = multinet.multi_layer_network(directed=False)
network.load_network(
    "datasets/synthetic_multilayer.txt",
    input_type="multiedgelist"
)

print("=" * 70)
print("COMMUNITY DETECTION ALGORITHM COMPARISON")
print("=" * 70)

# Run multiple algorithms
print("\n1. Running algorithms...")

# Louvain (flattened)
louvain_comms = louvain_communities(network)

# Multilayer Louvain (=1.0)
multilayer_comms = louvain_multilayer(
    network, gamma=1.0, omega=1.0, random_state=42
)

# Label propagation (flattened NetworkX graph)
G = nx.Graph()
for edge in network.core_network.edges():
    G.add_edge(edge[0], edge[1])
lpa_comms_list = asyn_lpa_communities(G, seed=42)
lpa_comms = {}
for comm_id, nodes in enumerate(lpa_comms_list):
    for node in nodes:
        lpa_comms[node] = comm_id

# (Optional) Infomap - skip if binary not available
try:
    infomap_comms = infomap_communities(
        network, binary="/usr/local/bin/infomap",
        seed=42, verbose=False
    )
    has_infomap = True
except Exception:
```

(continues on next page)

(continued from previous page)

```
    has_infomap = False
    print("  [SKIP] Infomap not available")

# Store results
algorithms = {
    'Louvain (flat)': louvain_comms,
    'Louvain (multilayer)': multilayer_comms,
    'Label Propagation': lpa_comms,
}
if has_infomap:
    algorithms['Infomap'] = infomap_comms

# 2. Basic statistics
print("\n2. Basic statistics:")
print(f"{'Algorithm':<25} {'#Comm':<10} {'Largest':<10} {'Avg Size':<10}")
print("-" * 70)

for name, comms in algorithms.items():
    sizes = Counter(comms.values())
    n_comms = len(sizes)
    largest = max(sizes.values())
    avg_size = sum(sizes.values()) / n_comms
    print(f"{name:<25} {n_comms:<10} {largest:<10} {avg_size:<10.1f}")

# 3. Modularity scores
print("\n3. Modularity scores:")
print(f"{'Algorithm':<25} {'Modularity (Q)':<15}")
print("-" * 70)

for name, comms in algorithms.items():
    if name == 'Louvain (multilayer)':
        # Use multilayer modularity
        Q = multilayer_modularity(network, comms, gamma=1.0, omega=1.0)
    else:
        # Use single-layer modularity on flattened graph
        Q = modularity(comms, G, weight='weight')
    print(f"{name:<25} {Q:<15.4f}")

# 4. Pairwise agreement
print("\n4. Pairwise agreement (ARI / NMI):")

# Align all partitions to same node set
alg_names = list(algorithms.keys())
alg_labels = {}

# Get common nodes (for multilayer, use node-layer pairs)
all_nodes = set()
for comms in algorithms.values():
    all_nodes.update(comms.keys())
```

(continues on next page)

(continued from previous page)

```
common_nodes = sorted(all_nodes)

# Convert to label vectors
for name, comms in algorithms.items():
    alg_labels[name] = [comms.get(node, -1) for node in common_nodes]

# Compute pairwise metrics
print(f"\n{'Pair':<45} {'ARI':<10} {'NMI':<10}")
print("-" * 70)

for i in range(len(alg_names)):
    for j in range(i+1, len(alg_names)):
        name1, name2 = alg_names[i], alg_names[j]
        labels1 = alg_labels[name1]
        labels2 = alg_labels[name2]

        ari = adjusted_rand_score(labels1, labels2)
        nmi = normalized_mutual_info_score(labels1, labels2)

        print(f"{name1} vs {name2:<25} {ari:<10.3f} {nmi:<10.3f}")

# 5. Size distribution comparison
print("\n5. Size distribution similarity:")

# Normalize size distributions
def normalize_sizes(comms):
    sizes = list(Counter(comms.values()).values())
    sizes_array = np.array(sorted(sizes, reverse=True))
    # Pad to same length
    max_len = max(len(Counter(c.values())) for c in algorithms.values())
    padded = np.zeros(max_len)
    padded[:len(sizes_array)] = sizes_array
    return padded / padded.sum()

size_dists = {name: normalize_sizes(comms) for name, comms in algorithms.
↪items()}

print(f"{'Pair':<45} {'JS Divergence':<15}")
print("-" * 70)

for i in range(len(alg_names)):
    for j in range(i+1, len(alg_names)):
        name1, name2 = alg_names[i], alg_names[j]
        js_div = jensenshannon(size_dists[name1], size_dists[name2])
        print(f"{name1} vs {name2:<25} {js_div:<15.4f}")
```

Expected output:

```
======================================================================
```

(continues on next page)

(continued from previous page)

```
COMMUNITY DETECTION ALGORITHM COMPARISON
======================================================================

1. Running algorithms...
  [SKIP] Infomap not available

2. Basic statistics:
Algorithm                 #Comm      Largest    Avg Size
----------------------------------------------------------------------
Louvain (flat)            5          45         24.0
Louvain (multilayer)      4          52         30.0
Label Propagation         7          38         17.1

3. Modularity scores:
Algorithm                 Modularity (Q)
----------------------------------------------------------------------
Louvain (flat)            0.4234
Louvain (multilayer)      0.4589
Label Propagation         0.3891

4. Pairwise agreement (ARI / NMI):

Pair                                          ARI        NMI
----------------------------------------------------------------------
Louvain (flat) vs Louvain (multilayer)        0.812      0.878
Louvain (flat) vs Label Propagation           0.623      0.745
Louvain (multilayer) vs Label Propagation     0.589      0.712

5. Size distribution similarity:
Pair                                          JS Divergence
----------------------------------------------------------------------
Louvain (flat) vs Louvain (multilayer)        0.1234
Louvain (flat) vs Label Propagation           0.2456
Louvain (multilayer) vs Label Propagation     0.2789
```

Interpretation:

- **High ARI/NMI (>0.8):** Algorithms agree strongly → robust communities
- **Medium ARI/NMI (0.5-0.8):** Partial agreement → sensitive to algorithm choice
- **Low ARI/NMI (<0.5):** Strong disagreement → no clear community structure or algorithm-specific artifacts

Consensus clustering:

When algorithms disagree, use consensus clustering to find stable communities:

```python
from collections import defaultdict

# Build co-occurrence matrix: how often do pairs of nodes appear together?
co_occurrence = defaultdict(int)
```

(continues on next page)

(continued from previous page)

```
n_algorithms = len(algorithms)

for comms in algorithms.values():
    # For each community in this partition
    comm_groups = defaultdict(list)
    for node, comm_id in comms.items():
        comm_groups[comm_id].append(node)

    # Increment co-occurrence for all pairs in same community
    for nodes in comm_groups.values():
        for i, node1 in enumerate(nodes):
            for node2 in nodes[i+1:]:
                pair = tuple(sorted([node1, node2]))
                co_occurrence[pair] += 1

# Threshold: keep pairs that co-occur in 50% of algorithms
threshold = n_algorithms * 0.5
stable_pairs = {pair for pair, count in co_occurrence.items() if count >=␣
↪threshold}

print(f"\nConsensus clustering:")
print(f"  Total node pairs: {len(co_occurrence)}")
print(f"  Stable pairs (50% agreement): {len(stable_pairs)}")
print(f"  Stability ratio: {len(stable_pairs)/len(co_occurrence):.2%}")
```

Expected output:

```
Consensus clustering:
  Total node pairs: 1845
  Stable pairs (50% agreement): 1234
  Stability ratio: 66.88%
```

Layer-Specific Communities

Motivation:

In multilayer networks, you may want to detect communities **within individual layers** and then compare them across layers. This reveals:

- Layer-specific structure (e.g., friendship communities vs. work communities)
- How community organization changes across contexts
- Which communities are stable vs. layer-dependent

Workflow:

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection.community_wrapper import louvain_
↪communities
from py3plex.algorithms.community_detection.community_louvain import␣
↪modularity
import networkx as nx
```

(continues on next page)

(continued from previous page)

```
from collections import Counter

# Load multilayer network
network = multinet.multi_layer_network(directed=False)
network.load_network(
    "datasets/synthetic_multilayer.txt",
    input_type="multiedgelist"
)

print("LAYER-SPECIFIC COMMUNITY DETECTION")
print("=" * 70)

# Extract and analyze each layer separately
layer_communities = {}
layer_stats = {}

for layer in network.get_layers():
    print(f"\n--- Layer: {layer} ---")

    # Extract layer-specific edges
    layer_edges = [
        (e[0][0], e[1][0])  # (node, node) without layer info
        for e in network.core_network.edges()
        if e[0][1] == layer and e[1][1] == layer
    ]

    # Build single-layer graph
    G_layer = nx.Graph()
    G_layer.add_edges_from(layer_edges)

    print(f"  Nodes: {G_layer.number_of_nodes()}")
    print(f"  Edges: {G_layer.number_of_edges()}")

    if G_layer.number_of_edges() == 0:
        print(f"  [SKIP] No edges in this layer")
        continue

    # Run Louvain on this layer
    communities = louvain_communities(G_layer)
    layer_communities[layer] = communities

    # Statistics
    comm_sizes = Counter(communities.values())
    n_comms = len(comm_sizes)
    Q = modularity(communities, G_layer, weight='weight')

    layer_stats[layer] = {
        'n_communities': n_comms,
        'modularity': Q,
```

(continues on next page)

(continued from previous page)

```
        'sizes': comm_sizes
    }

    print(f"  Communities: {n_comms}")
    print(f"  Modularity: {Q:.4f}")
    print(f"  Largest community: {max(comm_sizes.values())} nodes")
    print(f"  Average size: {sum(comm_sizes.values())/n_comms:.1f}")
```

Expected output:

```
LAYER-SPECIFIC COMMUNITY DETECTION
======================================================================

--- Layer: layer1 ---
  Nodes: 40
  Edges: 95
  Communities: 4
  Modularity: 0.4123
  Largest community: 15 nodes
  Average size: 10.0

--- Layer: layer2 ---
  Nodes: 40
  Edges: 102
  Communities: 5
  Modularity: 0.3876
  Largest community: 12 nodes
  Average size: 8.0

--- Layer: layer3 ---
  Nodes: 40
  Edges: 87
  Communities: 3
  Modularity: 0.4456
  Largest community: 18 nodes
  Average size: 13.3
```

Cross-layer stability analysis:

Check how consistently nodes are grouped across layers:

```
from sklearn.metrics import normalized_mutual_info_score
import pandas as pd

# Build node-level community assignments per layer
node_layer_assignments = {}
all_nodes = set()

for layer, communities in layer_communities.items():
    for node, comm_id in communities.items():
```

(continues on next page)

(continued from previous page)

```
        if node not in node_layer_assignments:
            node_layer_assignments[node] = {}
        node_layer_assignments[node][layer] = comm_id
        all_nodes.add(node)

# For each node, check consistency across layers
print("\n" + "=" * 70)
print("CROSS-LAYER STABILITY")
print("=" * 70)

layers = list(layer_communities.keys())

# Pairwise NMI between layers
print(f"\nPairwise NMI between layers:")
print(f"{'Layer Pair':<30} {'NMI':<10} {'Interpretation'}")
print("-" * 70)

for i in range(len(layers)):
    for j in range(i+1, len(layers)):
        layer1, layer2 = layers[i], layers[j]

        # Get common nodes
        nodes1 = set(layer_communities[layer1].keys())
        nodes2 = set(layer_communities[layer2].keys())
        common = nodes1 & nodes2

        if not common:
            continue

        # Compute NMI
        labels1 = [layer_communities[layer1][n] for n in common]
        labels2 = [layer_communities[layer2][n] for n in common]
        nmi = normalized_mutual_info_score(labels1, labels2)

        # Interpret
        if nmi > 0.8:
            interp = "Very similar"
        elif nmi > 0.5:
            interp = "Moderately similar"
        else:
            interp = "Different"

        print(f"{layer1} vs {layer2:<20} {nmi:<10.3f} {interp}")

# Node-level stability score
print(f"\nNode-level stability:")

node_stability = []
for node in sorted(all_nodes):
```

(continues on next page)

(continued from previous page)

```
    assignments = node_layer_assignments.get(node, {})

    # How many layers does this node appear in?
    n_layers = len(assignments)

    if n_layers < 2:
        continue

    # Are the community IDs consistent?
    # (This is a simplified measure - in reality, IDs may differ but␣
↪structure may be same)
    comm_ids = list(assignments.values())
    is_stable = len(set(comm_ids)) == 1  # All same community ID

    node_stability.append({
        'node': node,
        'n_layers': n_layers,
        'is_stable': is_stable,
        'assignments': assignments
    })

stable_nodes = sum(1 for s in node_stability if s['is_stable'])
print(f"  Nodes appearing in 2 layers: {len(node_stability)}")
print(f"  Stable nodes (same community ID): {stable_nodes}")
print(f"  Stability rate: {stable_nodes/len(node_stability)*100:.1f}%")

# Example unstable nodes
print(f"\n  Example unstable nodes:")
unstable = [s for s in node_stability if not s['is_stable']][:5]
for item in unstable:
    print(f"    {item['node']}: {item['assignments']}")
```

Expected output:

```
======================================================================
CROSS-LAYER STABILITY
======================================================================

Pairwise NMI between layers:
Layer Pair                     NMI        Interpretation
----------------------------------------------------------------------
layer1 vs layer2               0.723      Moderately similar
layer1 vs layer3               0.456      Different
layer2 vs layer3               0.512      Moderately similar

Node-level stability:
  Nodes appearing in 2 layers: 40
  Stable nodes (same community ID): 18
  Stability rate: 45.0%
```

(continues on next page)

(continued from previous page)

```
Example unstable nodes:
  A5: {'layer1': 0, 'layer2': 1, 'layer3': 0}
  B12: {'layer1': 2, 'layer2': 3}
  C3: {'layer1': 1, 'layer2': 0, 'layer3': 2}
  D7: {'layer1': 0, 'layer2': 2}
  E9: {'layer1': 3, 'layer2': 1, 'layer3': 1}
```

Visualization - Alluvial diagram:

Show how community membership flows across layers (requires external tools or manual construction):

```
import pandas as pd

# Export data for alluvial diagram (use R ggalluvial or similar)
alluvial_data = []

for node in all_nodes:
    assignments = node_layer_assignments.get(node, {})
    if len(assignments) >= 2:
        row = {'node': node}
        for layer in layers:
            row[f'comm_{layer}'] = assignments.get(layer, -1)
        alluvial_data.append(row)

df_alluvial = pd.DataFrame(alluvial_data)
df_alluvial.to_csv('alluvial_data.csv', index=False)
print("\nExported alluvial_data.csv for visualization in R/Python")
print("Example R code:")
print("  library(ggalluvial)")
print("  ggplot(data, aes(axis1=comm_layer1, axis2=comm_layer2, axis3=comm_
↪layer3)) +")
print("    geom_alluvium(aes(fill=node)) + geom_stratum()")
```

When to use layer-specific detection:

- **Exploratory analysis:** Understand layer-specific structure before multilayer methods
- **Heterogeneous layers:** Layers represent fundamentally different relationships (e.g., co-authorship vs. citation)
- **Baseline comparison:** Compare layer-specific vs. multilayer results to quantify benefit of multilayer methods
- **Dynamic networks:** Detect communities in temporal snapshots and track evolution

Cross-Layer Community Analysis

Motivation:

After detecting communities in the full multilayer network, you want to understand:

- Do communities span multiple layers?
- Which layers contribute most to each community?

- Are there inter-layer bridges (nodes connecting different layer-specific communities)?

Community × Layer composition:

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection.multilayer_modularity import␣
↪louvain_multilayer
from collections import defaultdict
import numpy as np
import pandas as pd

# Load network and detect communities
network = multinet.multi_layer_network(directed=False)
network.load_network(
    "datasets/synthetic_multilayer.txt",
    input_type="multiedgelist"
)

communities = louvain_multilayer(network, gamma=1.0, omega=1.0, random_
↪state=42)

print("CROSS-LAYER COMMUNITY ANALYSIS")
print("=" * 70)

# Build composition matrix: community × layer
layers = network.get_layers()
comm_ids = sorted(set(communities.values()))

composition = defaultdict(lambda: defaultdict(int))
for (node, layer), comm_id in communities.items():
    composition[comm_id][layer] += 1

# Convert to DataFrame for easier manipulation
data = []
for comm_id in comm_ids:
    row = {'community': comm_id}
    for layer in layers:
        row[layer] = composition[comm_id][layer]
    row['total'] = sum(composition[comm_id].values())
    data.append(row)

df_comp = pd.DataFrame(data)

print("\nCommunity × Layer composition:")
print(df_comp.to_string(index=False))

# Calculate layer entropy for each community
print("\n" + "-" * 70)
print("Community layer diversity (entropy):")
print(f"{'Community':<12} {'Entropy':<10} {'Interpretation'}")
print("-" * 70)
```

(continues on next page)

(continued from previous page)

```
for comm_id in comm_ids:
    # Calculate entropy: H = - p_i log(p_i)
    counts = [composition[comm_id][layer] for layer in layers]
    total = sum(counts)

    if total == 0:
        continue

    probs = np.array(counts) / total
    probs = probs[probs > 0]  # Remove zeros
    entropy = -np.sum(probs * np.log2(probs))
    max_entropy = np.log2(len(layers))  # Maximum possible entropy
    normalized_entropy = entropy / max_entropy if max_entropy > 0 else 0

    # Interpret
    if normalized_entropy > 0.9:
        interp = "Highly dispersed (spans all layers)"
    elif normalized_entropy > 0.5:
        interp = "Moderately dispersed (multi-layer)"
    else:
        interp = "Concentrated (layer-specific)"

    print(f"C{comm_id:<11} {entropy:<10.3f} {interp}")
```

Expected output:

```
CROSS-LAYER COMMUNITY ANALYSIS
======================================================================


Community × Layer composition:
 community  layer1  layer2  layer3  total
         0      15      14      16     45
         1      18      20       0     38
         2       0       0      22     22
         3       7       6       2     15


----------------------------------------------------------------------
Community layer diversity (entropy):
Community    Entropy    Interpretation
----------------------------------------------------------------------
C0           1.585      Highly dispersed (spans all layers)
C1           0.997      Moderately dispersed (multi-layer)
C2           0.000      Concentrated (layer-specific)
C3           1.252      Moderately dispersed (multi-layer)
```

Inter-layer bridges:

Identify nodes that connect different communities across layers:

```
print("\n" + "=" * 70)
print("INTER-LAYER BRIDGE ANALYSIS")
print("=" * 70)

# For each node, check if it belongs to different communities in different␣
↪layers
node_communities = defaultdict(dict)  # node -> layer -> comm_id

for (node, layer), comm_id in communities.items():
    node_communities[node][layer] = comm_id

# Identify bridge nodes
bridge_nodes = []
for node, layer_comms in node_communities.items():
    if len(layer_comms) < 2:
        continue

    # Check if community IDs differ across layers
    comm_ids = set(layer_comms.values())
    if len(comm_ids) > 1:
        bridge_nodes.append({
            'node': node,
            'n_layers': len(layer_comms),
            'n_communities': len(comm_ids),
            'assignments': dict(layer_comms)
        })

print(f"\nBridge nodes (spanning multiple communities across layers):")
print(f"  Total nodes: {len(node_communities)}")
print(f"  Bridge nodes: {len(bridge_nodes)} ({len(bridge_nodes)/len(node_
↪communities)*100:.1f}%)")

# Show examples
print(f"\n  Top 10 bridge nodes:")
print(f"  {'Node':<15} {'Layers':<10} {'Communities':<15} {'Assignments'}")
print("  " + "-" * 65)

bridge_nodes_sorted = sorted(bridge_nodes, key=lambda x: x['n_communities'],␣
↪reverse=True)
for item in bridge_nodes_sorted[:10]:
    node = item['node']
    n_layers = item['n_layers']
    n_comms = item['n_communities']
    assignments = ', '.join([f"{l}:C{c}" for l, c in sorted(item['assignments
↪'].items())])
    print(f"  {str(node):<15} {n_layers:<10} {n_comms:<15} {assignments}")
```

Expected output:

```
======================================================================
```

(continues on next page)

(continued from previous page)

```
INTER-LAYER BRIDGE ANALYSIS
======================================================================

Bridge nodes (spanning multiple communities across layers):
  Total nodes: 40
  Bridge nodes: 12 (30.0%)

  Top 10 bridge nodes:
  Node            Layers     Communities     Assignments
  ------------------------------------------------------------------
  A5              3          3               layer1:C0, layer2:C1, layer3:C2
  B12             3          2               layer1:C0, layer2:C1, layer3:C1
  C3              3          2               layer1:C1, layer2:C0, layer3:C0
  D7              2          2               layer1:C0, layer2:C3
  E9              3          2               layer1:C3, layer2:C1, layer3:C1
  F4              2          2               layer1:C1, layer2:C0
  G8              3          2               layer1:C0, layer2:C0, layer3:C2
  H2              2          2               layer1:C3, layer2:C0
  I6              3          2               layer1:C1, layer2:C1, layer3:C2
  J11             2          2               layer1:C0, layer2:C1
```

Community connectivity graph:

Build a meta-graph where nodes are communities and edges represent inter-layer bridges:

```python
import networkx as nx
import matplotlib.pyplot as plt

# Build community connectivity graph
G_meta = nx.Graph()

# Add community nodes
for comm_id in comm_ids:
    G_meta.add_node(f"C{comm_id}")

# Add edges for bridge nodes
for item in bridge_nodes:
    comms = list(item['assignments'].values())
    # Connect all pairs of communities this node bridges
    for i in range(len(comms)):
        for j in range(i+1, len(comms)):
            c1, c2 = f"C{comms[i]}", f"C{comms[j]}"
            if G_meta.has_edge(c1, c2):
                G_meta[c1][c2]['weight'] += 1
            else:
                G_meta.add_edge(c1, c2, weight=1)

print(f"\n" + "=" * 70)
print("COMMUNITY CONNECTIVITY")
print("=" * 70)
```

(continues on next page)

(continued from previous page)

```
print(f"\nCommunity-level connectivity:")
print(f"  Communities: {G_meta.number_of_nodes()}")
print(f"  Inter-community bridges: {G_meta.number_of_edges()}")

if G_meta.number_of_edges() > 0:
    print(f"\n  Strongest bridges (top 5):")
    edges_sorted = sorted(G_meta.edges(data=True), key=lambda x: x[2]['weight
↪'], reverse=True)
    for c1, c2, data in edges_sorted[:5]:
        print(f"    {c1}  {c2}: {data['weight']} bridge nodes")

    # Visualize meta-graph
    plt.figure(figsize=(8, 8))
    pos = nx.spring_layout(G_meta, seed=42)

    # Edge widths proportional to weight
    weights = [G_meta[u][v]['weight'] for u, v in G_meta.edges()]
    max_weight = max(weights) if weights else 1
    edge_widths = [3 * w / max_weight for w in weights]

    nx.draw_networkx_nodes(G_meta, pos, node_size=800, node_color='lightblue')
    nx.draw_networkx_labels(G_meta, pos, font_size=12, font_weight='bold')
    nx.draw_networkx_edges(G_meta, pos, width=edge_widths, alpha=0.6)

    # Edge labels
    edge_labels = {(u, v): f"{G_meta[u][v]['weight']}" for u, v in G_meta.
↪edges()}
    nx.draw_networkx_edge_labels(G_meta, pos, edge_labels, font_size=8)

    plt.title('Community Connectivity Meta-Graph\n(Edge width = number of␣
↪bridge nodes)', fontsize=14)
    plt.axis('off')
    plt.tight_layout()
    plt.savefig('community_connectivity.png', dpi=300, bbox_inches='tight')
    plt.show()
    print(f"\n  Visualization saved to: community_connectivity.png")
```

Expected output:

```
======================================================================
COMMUNITY CONNECTIVITY
======================================================================

Community-level connectivity:
  Communities: 4
  Inter-community bridges: 5

  Strongest bridges (top 5):
    C0  C1: 5 bridge nodes
```

(continues on next page)

(continued from previous page)

```
    C1   C2: 3 bridge nodes
    C0   C3: 2 bridge nodes
    C1   C3: 1 bridge nodes
    C0   C2: 1 bridge nodes

  Visualization saved to: community_connectivity.png
```

Use cases:

- **Biological networks:** Proteins bridging functional modules across different interaction types
- **Social networks:** Individuals connecting different social circles across contexts
- **Transportation:** Transfer hubs connecting regional clusters across transport modes

Quality Metrics

Why quality metrics matter:

Quality metrics help you:

1. **Compare algorithms** objectively
2. **Tune parameters** (e.g., choosing optimal ω in multilayer Louvain)
3. **Validate results** (high Q suggests real structure, not random fluctuations)
4. **Detect overfitting** (too many tiny communities = over-segmentation)

Compute Modularity

Single-layer modularity:

For flattened networks, use the Newman-Girvan modularity:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

```python
from py3plex.core import multinet
from py3plex.algorithms.community_detection.community_wrapper import louvain_communities
from py3plex.algorithms.community_detection.community_louvain import modularity
import networkx as nx

# Load network
network = multinet.multi_layer_network(directed=False)
network.load_network(
    "datasets/synthetic_multilayer.txt",
    input_type="multiedgelist"
)

# Detect communities
communities = louvain_communities(network)
```

(continues on next page)

(continued from previous page)

```
# Convert to NetworkX for modularity calculation
G = nx.Graph()
for edge in network.core_network.edges():
    G.add_edge(edge[0], edge[1])

# Calculate modularity
Q = modularity(communities, G, weight='weight')

print(f"Modularity Q: {Q:.4f}")

# Interpretation
if Q > 0.7:
    print("  Interpretation: Excellent community structure")
elif Q > 0.4:
    print("  Interpretation: Strong community structure")
elif Q > 0.2:
    print("  Interpretation: Moderate community structure")
else:
    print("  Interpretation: Weak or no community structure")
```

Expected output:

```
Modularity Q: 0.4234
  Interpretation: Strong community structure
```

Multilayer modularity:

For multilayer networks, use the generalized modularity that accounts for inter-layer coupling:

```
from py3plex.algorithms.community_detection.multilayer_modularity import (
    louvain_multilayer,
    multilayer_modularity
)

# Run multilayer Louvain
communities = louvain_multilayer(
    network,
    gamma=1.0,
    omega=1.0,
    random_state=42
)

# Calculate multilayer modularity
Q_multi = multilayer_modularity(
    network,
    communities,
    gamma=1.0,
    omega=1.0
)
```

(continues on next page)

(continued from previous page)

```
print(f"Multilayer modularity Q: {Q_multi:.4f}")
```

Expected output:

```
Multilayer modularity Q: 0.4589
```

Modularity resolution:

Modularity has a **resolution limit**: it cannot detect communities smaller than $\sqrt{m}$ where m is the number of edges. The resolution parameter γ can help:

```
# Test different resolution parameters
print("Modularity vs. resolution:")
print(f"{'':<10} {'#Comm':<10} {'Q':<10} {'Avg Size':<10}")
print("-" * 45)

for gamma in [0.5, 1.0, 1.5, 2.0]:
    comms = louvain_multilayer(
        network, gamma=gamma, omega=1.0, random_state=42
    )
    n_comms = len(set(comms.values()))
    Q = multilayer_modularity(network, comms, gamma=gamma, omega=1.0)
    avg_size = len(comms) / n_comms

    print(f"{gamma:<10.1f} {n_comms:<10} {Q:<10.4f} {avg_size:<10.1f}")
```

Expected output:

```
Modularity vs. resolution:
          #Comm      Q          Avg Size
---------------------------------------------
0.5        3          0.3456     40.0
1.0        5          0.4589     24.0
1.5        8          0.4123     15.0
2.0        12         0.3678     10.0
```

Interpretation:

- **Lower :** Fewer, larger communities (under-segmentation)
- **Higher :** More, smaller communities (over-segmentation)
- **Optimal :** Maximum Q (but check that communities are meaningful!)

Additional Quality Metrics

1. Coverage (fraction of edges within communities):

```
def calculate_coverage(network, communities):
    """Fraction of edges within communities."""
    intra_edges = 0
    total_edges = 0
```

(continues on next page)

(continued from previous page)

```
    for source, target in network.core_network.edges():
        total_edges += 1
        if communities.get(source) == communities.get(target):
            intra_edges += 1

    return intra_edges / total_edges if total_edges > 0 else 0

coverage = calculate_coverage(network, communities)
print(f"Coverage: {coverage:.4f}  (fraction of intra-community edges)")
```

Expected output:

```
Coverage: 0.8234  (fraction of intra-community edges)
```

2. Performance (combines intra-community edges and inter-community non-edges):

```
def calculate_performance(network, communities):
    """Performance metric (Fortunato 2010)."""
    nodes = list(communities.keys())
    n = len(nodes)

    # Count intra-community edges and inter-community non-edges
    intra_edges = 0
    inter_non_edges = 0
    total_pairs = 0

    for i in range(len(nodes)):
        for j in range(i+1, len(nodes)):
            node1, node2 = nodes[i], nodes[j]
            same_community = communities[node1] == communities[node2]
            is_edge = network.core_network.has_edge(node1, node2)

            if same_community and is_edge:
                intra_edges += 1
            elif not same_community and not is_edge:
                inter_non_edges += 1

            total_pairs += 1

    return (intra_edges + inter_non_edges) / total_pairs if total_pairs > 0 else 0

performance = calculate_performance(network, communities)
print(f"Performance: {performance:.4f}")
```

Expected output:

```
Performance: 0.7456
```

3. Conductance (quality of community boundaries):

```
def calculate_conductance(network, communities, comm_id):
    """Conductance of a specific community (lower is better)."""
    comm_nodes = [n for n, c in communities.items() if c == comm_id]

    if not comm_nodes:
        return None

    # Count edges
    internal_edges = 0
    boundary_edges = 0

    for node in comm_nodes:
        neighbors = list(network.core_network.neighbors(node))
        for neighbor in neighbors:
            if communities.get(neighbor) == comm_id:
                internal_edges += 0.5  # Count each edge once
            else:
                boundary_edges += 1

    volume = internal_edges * 2 + boundary_edges  # Volume of the community
    return boundary_edges / volume if volume > 0 else 0

# Calculate for all communities
print("\nConductance per community (lower = better defined):")
for comm_id in sorted(set(communities.values())):
    cond = calculate_conductance(network, communities, comm_id)
    if cond is not None:
        print(f"  Community {comm_id}: {cond:.4f}")
```

Expected output:

```
Conductance per community (lower = better defined):
  Community 0: 0.1234
  Community 1: 0.2456
  Community 2: 0.0987
  Community 3: 0.3123
  Community 4: 0.1789
```

4. Null model comparison (compare to random partitions):

```
import random

# Calculate Q for real partition
Q_real = multilayer_modularity(network, communities, gamma=1.0, omega=1.0)

# Generate random partitions and calculate Q
nodes = list(communities.keys())
n_communities = len(set(communities.values()))

Q_random = []
for trial in range(100):
```

(continues on next page)

(continued from previous page)

```
    # Random partition with same number of communities
    random_comms = {node: random.randint(0, n_communities-1) for node in␣
↪nodes}
    Q_rand = multilayer_modularity(network, random_comms, gamma=1.0, omega=1.
↪0)
    Q_random.append(Q_rand)

Q_rand_mean = np.mean(Q_random)
Q_rand_std = np.std(Q_random)
z_score = (Q_real - Q_rand_mean) / Q_rand_std if Q_rand_std > 0 else 0

print(f"\nNull model comparison:")
print(f"  Real Q: {Q_real:.4f}")
print(f"  Random Q (mean ± std): {Q_rand_mean:.4f} ± {Q_rand_std:.4f}")
print(f"  Z-score: {z_score:.2f}")

if z_score > 3:
    print(f"  Interpretation: Highly significant (real structure)")
elif z_score > 2:
    print(f"  Interpretation: Significant (likely real structure)")
else:
    print(f"  Interpretation: Not significant (could be random)")
```

Expected output:

```
Null model comparison:
  Real Q: 0.4589
  Random Q (mean ± std): 0.0023 ± 0.0145
  Z-score: 31.49
  Interpretation: Highly significant (real structure)
```

Summary of metrics:

- **Modularity (Q):** Overall quality, general-purpose
- **Coverage:** Simple interpretability (% internal edges)
- **Performance:** Balances true positives and true negatives
- **Conductance:** Community boundary quality (per-community)
- **Null model:** Statistical significance test

Recommendation: Always report modularity + at least one other metric to get a complete picture.

CLI Cross-Reference (Optional)

py3plex provides command-line tools for quick community detection without writing Python code.

Basic usage:

```
# Detect communities using Louvain (default algorithm)
```

(continues on next page)

(continued from previous page)

```
py3plex community datasets/network.edgelist \
    --algorithm louvain \
    --output communities.json

# Using Infomap (requires Infomap binary installed)
py3plex community datasets/network.edgelist \
    --algorithm infomap \
    --output communities.json

# Using Label Propagation (fast for large networks)
py3plex community datasets/network.edgelist \
    --algorithm label_prop \
    --output communities.json

# With custom resolution parameter for Louvain
py3plex community datasets/network.edgelist \
    --algorithm louvain \
    --resolution 1.5 \
    --output communities.json
```

Available algorithms:

- `louvain`: Fast Louvain method (default) - optimizes modularity on flattened network
- `infomap`: Infomap algorithm - requires Infomap binary (https://www.mapequation.org/infomap/)
- `label_prop`: Label propagation - very fast, suitable for large networks

Output format:

The CLI outputs JSON files with structure:

```
{
  "algorithm": "louvain",
  "num_communities": 5,
  "communities": {
    "node1": 0,
    "node2": 0,
    "node3": 1,
    ...
  },
  "community_sizes": {
    "0": 42,
    "1": 27,
    ...
  }
}
```

Note on multilayer networks:

The current CLI `community` command operates on flattened networks. For multilayer-specific community detection (with inter-layer coupling), use the Python API with `louvain_multilayer()` as shown in the examples above. Future CLI versions may add multilayer support.

Viewing results:

After running the CLI command, you can analyze the JSON output:

```
# View community statistics
py3plex community network.edgelist --algorithm louvain
# Output printed to console if no --output specified
```

For full CLI documentation, see ../tutorials/cli_usage or ../deployment/cli_usage.

Next Steps

Further reading:

- **Algorithms:** *Algorithm Landscape* - Deep dive into community detection theory
- **Visualization:** *How to Visualize Multilayer Networks* - Advanced community visualization techniques
- **Benchmark:** ../tutorials/benchmark_communities - Compare with ground-truth communities
- **Temporal analysis:** ../tutorials/temporal_communities - Track community evolution over time

Recommended workflows:

1. **Exploratory:** Start with Louvain → visualize → if unsatisfied, try multilayer Louvain or Infomap
2. **Publication:** Run multiple algorithms → compare → report consensus + metrics
3. **Large-scale:** Use label propagation for initial exploration → refine with Louvain on filtered subgraph
4. **Temporal:** Detect communities in snapshots → track with NMI → visualize with alluvial diagrams

Common pitfalls:

- **Resolution limit:** Modularity cannot detect communities smaller than $\sqrt{m}$
- **Non-determinism:** Many algorithms are stochastic; always set random seeds for reproducibility
- **Overfitting:** Too many tiny communities suggests over-segmentation; try lower resolution
- **Layer coupling:** For multilayer networks, always try multiple ω values

Community detection checklist:

- [ ] Run at least 2 different algorithms
- [ ] Calculate modularity and at least one other quality metric
- [ ] Visualize size distribution to check for over/under-segmentation
- [ ] Compare with null model to ensure statistical significance
- [ ] For multilayer: test multiple ω values
- [ ] Export results to CSV for downstream analysis
- [ ] Document random seeds for reproducibility

Questions?

- GitHub Issues: https://github.com/SkBlaz/py3plex/issues
- Documentation: https://skblaz.github.io/py3plex/
- Examples: `examples/communities/` directory in the repository

Key References:

- Blondel, V. D., Guillaume, J. L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10), P10008.
- Mucha, P. J., Richardson, T., Macon, K., Porter, M. A., & Onnela, J. P. (2010). Community structure in time-dependent, multiscale, and multiplex networks. *Science*, 328(5980), 876-878.
- Rosvall, M., & Bergstrom, C. T. (2008). Maps of random walks on complex networks reveal community structure. *Proceedings of the National Academy of Sciences*, 105(4), 1118-1123.
- Raghavan, U. N., Albert, R., & Kumara, S. (2007). Near linear time algorithm to detect community structures in large-scale networks. *Physical Review E*, 76(3), 036106.

3.3.6 How to Simulate Multilayer Dynamics

This guide presents py3plex as a comprehensive framework for simulating dynamical processes on multilayer networks. We cover epidemic models (SIR, SIS), diffusion processes (random walks), and custom dynamics implementations—all with full multilayer support.

Run this guide online

You can run this tutorial in your browser without any local installation:

[7] Or see the full executable example: `sir_epidemic.py`

Prerequisites: A loaded multilayer network (see *How to Load and Build Networks*).

Overview

Multilayer dynamics in py3plex enable you to model processes where interactions occur across multiple types of relationships simultaneously. Typical use cases include:

- **Epidemic spread**: Disease transmission through household, workplace, and social contact layers
- **Information diffusion**: News spreading via online and offline communication channels
- **Random walks**: Navigation and search processes across interconnected network layers
- **Opinion dynamics**: Belief propagation through multiple social contexts

Simulation Pipeline

The typical workflow for running dynamics simulations consists of five steps:

1. **Load or build** a multilayer network using `py3plex.core.multinet`

[7] https://colab.research.google.com/github/SkBlaz/py3plex/blob/master/notebooks/simulate_dynamics.ipynb

2. **Choose a dynamics model** (e.g., `SIRDynamics`, `SISDynamics`, `RandomWalkDynamics`)
3. **Configure parameters** and initial conditions (infection rates, starting nodes, etc.)
4. **Run the simulation** via `.run(steps=...)` with a fixed seed for reproducibility
5. **Inspect the results** object using `.get_measure(...)` to extract time series and summary statistics

This pipeline provides a consistent interface across all dynamics models while allowing specialized configurations for multilayer network effects.

Common Concepts & API

Node and Layer Representation

Dynamics in py3plex operate on the **supra-network** representation, where multilayer nodes are encoded as `(node_id, layer)` tuples. For example, Alice in the "friends" layer is represented as `('Alice', 'friends')`.

When you pass a multilayer network to a dynamics model:

- Node states are tracked per node-layer pair
- Infection, diffusion, or walker movements respect both intra-layer and inter-layer edges
- You can query per-layer statistics after simulation

For single-layer NetworkX graphs, nodes are used directly without tuple encoding.

Time and Synchrony

All dynamics models in this guide use **discrete-time, synchronous updates**:

- Time advances in integer steps: t = 0, 1, 2, . . .
- At each step, all nodes update simultaneously based on the state at time t
- This differs from continuous-time (Gillespie) models, which are also available in `py3plex.dynamics`

Randomness & Reproducibility

Every dynamics model has a dedicated random number generator controlled by `.set_seed(seed)`:

```
dynamics.set_seed(42)
results = dynamics.run(steps=100)
```

Setting the seed ensures:

- Reproducible initial conditions (which nodes start infected, which are susceptible)
- Reproducible stochastic transitions (infection events, recovery events, random walk steps)
- Consistent results across multiple runs with the same seed

Network Requirements

Dynamics models work with:

- **py3plex multilayer networks**: Full support for inter-layer and intra-layer edges
- **NetworkX graphs**: Single-layer networks (directed or undirected)
- **Edge weights**: Used in weighted random walks; epidemic models currently treat edges as unweighted contacts

Common Base Interface

All dynamics classes inherit from `DynamicsProcess` (discrete-time) or `ContinuousTimeProcess` (continuous-time). The core methods are:

```
from py3plex.dynamics import SIRDynamics

# Initialize with network and parameters
dynamics = SIRDynamics(network, beta=0.3, gamma=0.1, initial_infected=0.05)

# Set seed for reproducibility
dynamics.set_seed(42)

# Run for specified number of steps
results = dynamics.run(steps=100)

# Extract measures
prevalence = results.get_measure("prevalence")
state_counts = results.get_measure("state_counts")
```

The `results` object is always a `DynamicsResult` instance with standardized measure extraction methods.

SIR Epidemic Model on Multilayer Networks

Formal Description

The **SIR (Susceptible-Infected-Recovered)** model describes epidemic spread with three compartments:

- **S (Susceptible)**: Nodes that can become infected
- **I (Infected)**: Nodes that are currently infectious
- **R (Recovered)**: Nodes with permanent immunity (absorbing state)

Discrete-Time Update Rules (synchronous):

1. **Recovery**: Each infected node recovers (I $\to$ R) with probability $\gamma \in (0,1)$ per time step
2. **Infection**: Each susceptible node with k infected neighbors becomes infected (S $\to$ I) with probability

   $$p_{\text{infect}} = 1 - (1 - \beta)^k$$

   where $\beta \in (0,1)$ is the per-contact infection probability
3. **Absorption**: Recovered nodes remain recovered (R $\to$ R) forever

Multilayer Extension

On multilayer networks, the infection pressure on a node aggregates across all layers:

- A susceptible node `(i, layer_A)` counts infected neighbors from both intra-layer edges (within `layer_A`) and inter-layer edges (to other layers)
- You can specify layer-specific transmission rates as a dictionary (see *Layer-Specific and Time-Varying Parameters*)

This means epidemics can spread within layers and jump between layers via inter-layer connections, mimicking real-world scenarios like workplace and household transmission.

Code Example

```
from py3plex.core import multinet
from py3plex.dynamics import SIRDynamics

# Create a simple two-layer network
network = multinet.multi_layer_network(directed=False)

# Add nodes in two layers
nodes = []
for i in range(20):
    nodes.append({'source': i, 'type': 'layer1'})
    nodes.append({'source': i, 'type': 'layer2'})
network.add_nodes(nodes)

# Add intra-layer edges (chain structure)
edges = []
for i in range(19):
    edges.append({'source': i, 'target': i+1,
                  'source_type': 'layer1', 'target_type': 'layer1'})
    edges.append({'source': i, 'target': i+1,
                  'source_type': 'layer2', 'target_type': 'layer2'})

# Add inter-layer edges (node replicas)
for i in range(20):
    edges.append({'source': i, 'target': i,
                  'source_type': 'layer1', 'target_type': 'layer2'})

network.add_edges(edges)

# Configure SIR model
sir = SIRDynamics(
    network,
    beta=0.3,     # Infection probability per contact
    gamma=0.1,    # Recovery probability per time step
    initial_infected=0.05  # 5% of nodes initially infected
)

# Run simulation with fixed seed
sir.set_seed(42)
```

(continues on next page)

(continued from previous page)

```
results = sir.run(steps=100)

# Extract measures
prevalence = results.get_measure("prevalence")
state_counts = results.get_measure("state_counts")

# Print summary statistics
peak_time = prevalence.argmax()
print(f"Peak prevalence: {prevalence.max():.2%} at step {peak_time}")
print(f"Final susceptible: {state_counts['S'][-1]} nodes")
print(f"Final infected: {state_counts['I'][-1]} nodes")
print(f"Final recovered: {state_counts['R'][-1]} nodes")
print(f"Attack rate: {state_counts['R'][-1] / network.core_network.number_of_
↪nodes():.2%}")
```

Expected Output

```
Peak prevalence: 35.00% at step 16
Final susceptible: 0 nodes
Final infected: 0 nodes
Final recovered: 40 nodes
Attack rate: 100.00%
```

Note: Exact values depend on network structure and random seed. The example shows a complete outbreak where all nodes eventually get infected.

Time Series Visualization

Track the epidemic curve over time:

```
import matplotlib.pyplot as plt

# Get time series data
state_counts = results.get_measure("state_counts")
steps = range(len(state_counts['S']))

# Plot S, I, R trajectories
plt.figure(figsize=(10, 6))
plt.plot(steps, state_counts['S'], label='Susceptible', color='blue',␣
↪linewidth=2)
plt.plot(steps, state_counts['I'], label='Infected', color='red', linewidth=2)
plt.plot(steps, state_counts['R'], label='Recovered', color='green',␣
↪linewidth=2)

plt.xlabel('Time step', fontsize=12)
plt.ylabel('Number of nodes', fontsize=12)
plt.title('SIR Epidemic Dynamics on Multilayer Network', fontsize=14)
plt.legend()
plt.grid(alpha=0.3)
plt.savefig('sir_dynamics.png', dpi=300, bbox_inches='tight')
```

(continues on next page)

(continued from previous page)

```
plt.show()
```

The plot shows the characteristic SIR pattern: S decreases monotonically, I rises then falls (forming a peak), and R increases monotonically until saturation. The peak prevalence and timing depend on , , and network structure.

SIS Epidemic Model on Multilayer Networks

Formal Description

The **SIS (Susceptible-Infected-Susceptible)** model describes epidemics **without permanent immunity**:

- **S (Susceptible)**: Nodes that can become infected
- **I (Infected)**: Nodes that are currently infectious

Discrete-Time Update Rules (synchronous):

1. **Recovery**: Each infected node recovers (I $\rightarrow$ S) with probability γ (or μ) per time step
2. **Infection**: Each susceptible node with k infected neighbors becomes infected (S $\rightarrow$ I) with probability

   $$p_{\text{infect}} = 1 - (1 - \beta)^k$$

   where $\beta \in (0, 1)$ is the transmission probability per contact
3. **No absorption**: Recovered nodes immediately return to susceptible state, allowing reinfection

Endemic Equilibrium

Unlike SIR (which eventually dies out), SIS can reach an **endemic equilibrium** where a steady fraction of nodes remain infected. The basic reproduction number is:

$$R_0 = \frac{\beta}{\gamma}\langle k\rangle$$

where $\langle$k$\rangle$ is the average degree. If $R_0 > 1$, the epidemic persists; if R_0 1, it dies out.

Multilayer Extension

On multilayer networks, SIS infection pressure aggregates across all layers, similar to SIR. The endemic prevalence can differ across layers if you use layer-specific transmission rates.

Code Example

```
from py3plex.core import multinet
from py3plex.dynamics import SISDynamics

# Use the same multilayer network as before
network = multinet.multi_layer_network(directed=False)

# Add nodes and edges (same as SIR example)
nodes = []
for i in range(20):
```

(continues on next page)

(continued from previous page)

```
    nodes.append({'source': i, 'type': 'layer1'})
    nodes.append({'source': i, 'type': 'layer2'})
network.add_nodes(nodes)

edges = []
for i in range(19):
    edges.append({'source': i, 'target': i+1,
                  'source_type': 'layer1', 'target_type': 'layer1'})
    edges.append({'source': i, 'target': i+1,
                  'source_type': 'layer2', 'target_type': 'layer2'})
for i in range(20):
    edges.append({'source': i, 'target': i,
                  'source_type': 'layer1', 'target_type': 'layer2'})
network.add_edges(edges)

# Configure SIS model
sis = SISDynamics(
    network,
    beta=0.3,    # Infection probability per contact
    mu=0.1,      # Recovery probability (also accepts 'gamma')
    initial_infected=0.05
)

# Run for longer to reach equilibrium
sis.set_seed(42)
results = sis.run(steps=200)

# Extract prevalence time series
prevalence = results.get_measure("prevalence")

# Compute endemic prevalence (mean of last 50 steps)
endemic_prevalence = prevalence[-50:].mean()
print(f"Endemic prevalence: {endemic_prevalence:.2%}")
print(f"Std (last 50 steps): {prevalence[-50:].std():.3f}")

# Check if epidemic persisted
if endemic_prevalence > 0.01:
    print("Status: Endemic state reached")
else:
    print("Status: Epidemic died out")
```

Expected Output

```
Endemic prevalence: 0.00%
Std (last 50 steps): 0.000
Status: Epidemic died out
```

Note: On sparse chain-like networks with moderate /, the epidemic often dies out due to low R_0. Try denser networks or higher to observe endemic states.

Comparison with SIR

The key difference between SIS and SIR:

- **SIR**: Forms a peak and dies out (all nodes eventually become R or remain S)
- **SIS**: Can sustain long-term endemic prevalence if $R_0 > 1$

```
import matplotlib.pyplot as plt
import networkx as nx
from py3plex.dynamics import SISDynamics, SIRDynamics

# Use a denser network to see endemic behavior
G = nx.watts_strogatz_graph(n=100, k=6, p=0.1, seed=42)

# Run SIS
sis = SISDynamics(G, beta=0.3, mu=0.1, initial_infected=0.1)
sis.set_seed(42)
sis_results = sis.run(steps=150)
sis_prev = sis_results.get_measure("prevalence")

# Run SIR
sir = SIRDynamics(G, beta=0.3, gamma=0.1, initial_infected=0.1)
sir.set_seed(42)
sir_results = sir.run(steps=150)
sir_prev = sir_results.get_measure("prevalence")

# Plot comparison
plt.figure(figsize=(10, 6))
plt.plot(sis_prev, label='SIS (endemic)', color='red', linewidth=2)
plt.plot(sir_prev, label='SIR (dies out)', color='blue', linewidth=2)
plt.xlabel('Time step')
plt.ylabel('Prevalence')
plt.title('SIS vs SIR Epidemic Dynamics')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

Random Walk Dynamics

Formal Description

Random walk dynamics simulate the movement of a walker (or multiple walkers) on the network. At each time step:

1. The walker is at some node (or node-layer pair) `v`
2. With probability `lazy_probability`, the walker stays at `v`
3. With probability `1 - lazy_probability`, the walker moves to a uniformly random neighbor

Multilayer Random Walks

On multilayer networks:

- Walkers navigate both intra-layer edges (within a layer) and inter-layer edges (between layers)

- The state is a node-layer tuple, e.g., (node_id, layer)
- Transition probabilities are uniform over all neighbors (intra-layer + inter-layer)

Random walks are fundamental for:

- Computing stationary distributions (related to PageRank and eigenvector centrality)
- Modeling diffusion and search processes
- Estimating hitting times between nodes

Code Example

```
from py3plex.core import multinet
from py3plex.dynamics import RandomWalkDynamics

# Create a two-layer network
network = multinet.multi_layer_network(directed=False)

# Ring structure in layer1, star structure in layer2
n = 15
nodes = []
for i in range(n):
    nodes.append({'source': i, 'type': 'ring'})
    nodes.append({'source': i, 'type': 'star'})
network.add_nodes(nodes)

# Ring layer: circular edges
edges = []
for i in range(n):
    edges.append({'source': i, 'target': (i+1) % n,
                  'source_type': 'ring', 'target_type': 'ring'})

# Star layer: hub-spoke (node 0 is hub)
for i in range(1, n):
    edges.append({'source': 0, 'target': i,
                  'source_type': 'star', 'target_type': 'star'})

# Inter-layer edges
for i in range(n):
    edges.append({'source': i, 'target': i,
                  'source_type': 'ring', 'target_type': 'star'})

network.add_edges(edges)

# Configure random walk
walk = RandomWalkDynamics(
    network,
    start_node=(0, 'ring'),  # Start at node 0 in ring layer
    lazy_probability=0.1
)
```

(continues on next page)

(continued from previous page)

```
# Run for 1000 steps
walk.set_seed(42)
results = walk.run(steps=1000)

# Get trajectory and compute visit counts
trajectory = results.get_measure("trajectory")
visit_counts = walk.visit_counts(trajectory)

# Sort by visit frequency
sorted_visits = sorted(visit_counts.items(), key=lambda x: x[1], reverse=True)

print(f"Total unique nodes visited: {len(visit_counts)}")
print(f"\\nTop 10 most visited nodes:")
for node, count in sorted_visits[:10]:
    print(f"  {node}: {count} visits ({count/len(trajectory):.2%})")
```

Expected Output

```
Total unique nodes visited: 30

Top 10 most visited nodes:
  (0, 'star'): 87 visits (8.69%)
  (0, 'ring'): 64 visits (6.39%)
  (1, 'ring'): 42 visits (4.20%)
  (14, 'ring'): 39 visits (3.90%)
  (1, 'star'): 37 visits (3.70%)
  (2, 'ring'): 36 visits (3.60%)
  (13, 'ring'): 35 visits (3.50%)
  (2, 'star'): 34 visits (3.40%)
  (3, 'star'): 32 visits (3.20%)
  (14, 'star'): 31 visits (3.10%)
```

Note: The hub node in the star layer receives disproportionately high visits due to its high degree.

Analyzing Layer Switching

You can analyze how often the walker switches between layers:

```
# Count layer switches
switches = 0
for i in range(len(trajectory) - 1):
    current_layer = trajectory[i][1]
    next_layer = trajectory[i + 1][1]
    if current_layer != next_layer:
        switches += 1

print(f"Total layer switches: {switches}")
print(f"Switch rate: {switches / (len(trajectory) - 1):.2%}")
```

This reveals how "coupled" the layers are through inter-layer edges. High switch rates indicate strong inter-layer connectivity.

Layer-Specific and Time-Varying Parameters

Layer-Specific Transmission Rates

You can configure different infection rates for different layers by passing a **dictionary** for the `beta` parameter:

```
from py3plex.dynamics import SIRDynamics

# Define layer-specific transmission rates
layer_rates = {
    'household': 0.5,    # High transmission in households
    'workplace': 0.2,    # Medium transmission at work
    'social': 0.1        # Low transmission in casual social contacts
}

sir = SIRDynamics(
    network,
    beta=layer_rates,    # Pass dict instead of scalar
    gamma=0.1,
    initial_infected=0.05
)

results = sir.run(steps=100)
```

How It Works

- When computing infection pressure on a node `(i, layer_A)`, edges within `layer_A` use `beta['layer_A']`
- Inter-layer edges use a default or average beta (implementation-dependent)
- This allows modeling realistic scenarios where transmission varies by context

Extracting Per-Layer Statistics

After running the simulation, you can compute layer-specific prevalence manually:

```
# Get final state
final_state = results.trajectory[-1]

# Count infected nodes per layer
layer_prevalence = {}
for node, state in final_state.items():
    if isinstance(node, tuple):  # Multilayer node
        node_id, layer = node
        if layer not in layer_prevalence:
            layer_prevalence[layer] = {'S': 0, 'I': 0, 'R': 0}
        layer_prevalence[layer][state] += 1

# Print per-layer statistics
for layer, counts in layer_prevalence.items():
    total = sum(counts.values())
```

(continues on next page)

(continued from previous page)

```
    print(f"Layer {layer}: {counts['R']} recovered out of {total} nodes (
↪{counts['R']/total:.2%})")
```

Time-Varying Parameters (Intervention Scenarios)

To model interventions (e.g., school closures, social distancing), you can run multiple simulations with different parameters for different time windows:

```
# Scenario 1: Baseline (no intervention)
sir = SIRDynamics(network, beta=0.3, gamma=0.1, initial_infected=0.05)
sir.set_seed(42)
results_baseline = sir.run(steps=100)

# Scenario 2: Intervention at t=30 (reduce beta)
# Run phase 1 (before intervention)
sir = SIRDynamics(network, beta=0.3, gamma=0.1, initial_infected=0.05)
sir.set_seed(42)
state_30 = sir.run(steps=30, record=False)  # Get state at t=30

# Run phase 2 (with reduced beta)
sir_intervention = SIRDynamics(network, beta=0.15, gamma=0.1)
sir_intervention.seed = 42  # Continue from same seed
results_intervention = sir_intervention.run(steps=70, state=state_30)

# Compare peak prevalence
print(f"Baseline peak: {results_baseline.get_measure('prevalence').max():.2%}
↪")
print(f"Intervention peak: {results_intervention.get_measure('prevalence').
↪max():.2%}")
```

Note: For more complex time-varying scenarios, consider using the high-level simulation builder API (see next steps).

Result Objects and Metrics

DynamicsResult API

All dynamics models return a `DynamicsResult` object from `.run()`. This object provides:

- **Trajectory access**: Direct indexing `results[t]` or `results.trajectory`
- **Measure extraction**: `.get_measure(name)` for standard metrics
- **Conversion**: `.to_pandas()` for DataFrame export

Common Measures

The following table summarizes available measures for each model:

Model	Measure	Description	Return Type
SIR	"prevalence"	Fraction of infected nodes over time (I / total)	numpy.ndarray
SIR	"state_counts"	Counts of nodes in each state (S, I, R) over time	dict[str, numpy.ndarray]
SIR	"trajectory"	Raw state sequence (list of state dicts)	list[dict]
SIS	"prevalence"	Fraction of infected nodes over time (I / total)	numpy.ndarray
SIS	"state_counts"	Counts of nodes in each state (S, I) over time	dict[str, numpy.ndarray]
SIS	"trajectory"	Raw state sequence	list[dict]
RandomWalk	"trajectory"	Sequence of visited nodes (or node-layer pairs)	list

Note: Measures like "peak_prevalence" and "final_recovered" mentioned in earlier versions are computed manually from the time series data.

Extracting and Analyzing Measures

```
# Run SIR simulation
sir = SIRDynamics(network, beta=0.3, gamma=0.1, initial_infected=0.05)
sir.set_seed(42)
results = sir.run(steps=100)

# Extract prevalence time series
prevalence = results.get_measure("prevalence")
print(f"Prevalence array shape: {prevalence.shape}")  # (101,) for 100 steps␣
↪+ initial

# Compute summary statistics
peak_prevalence = prevalence.max()
peak_time = prevalence.argmax()
final_prevalence = prevalence[-1]

print(f"Peak prevalence: {peak_prevalence:.2%} at step {peak_time}")
print(f"Final prevalence: {final_prevalence:.2%}")

# Extract state counts
state_counts = results.get_measure("state_counts")
print(f"Available states: {list(state_counts.keys())}")

# Compute attack rate (fraction ever infected)
total_nodes = sum(state_counts[s][0] for s in state_counts.keys())  # Total␣
↪at t=0
attack_rate = state_counts['R'][-1] / total_nodes
print(f"Attack rate: {attack_rate:.2%}")
```

Converting to pandas DataFrame

For further analysis with pandas:

```
import pandas as pd

# Convert time series to DataFrame
state_counts = results.get_measure("state_counts")
df = pd.DataFrame({
    'time': range(len(state_counts['S'])),
    'S': state_counts['S'],
    'I': state_counts['I'],
    'R': state_counts['R']
})

# Compute prevalence column
df['prevalence'] = df['I'] / (df['S'] + df['I'] + df['R'])

# Analyze
print(df.describe())
print(f"\\nTime to peak: {df['prevalence'].idxmax()}")
```

Alternatively, use the built-in method:

```
# Convert trajectory to long-form DataFrame
df = results.to_pandas()
print(df.head())

# Output:
#     t  node state
# 0  0     0     S
# 1  0     1     S
# 2  0     2     S
# ...
```

Reproducible Experiments & Parameter Sweeps

Running Multiple Stochastic Realizations

Epidemic dynamics are stochastic, so we often run multiple realizations and report summary statistics:

```
import numpy as np
from py3plex.dynamics import SIRDynamics

def run_sir_ensemble(network, beta, gamma, n_runs=50):
    """Run SIR simulation n_runs times with different seeds."""
    peak_prevalences = []
    attack_rates = []

    for seed in range(n_runs):
        sir = SIRDynamics(network, beta=beta, gamma=gamma, initial_infected=0.
```

(continues on next page)

(continued from previous page)

```
↪05)
        sir.set_seed(seed)
        results = sir.run(steps=100)

        prevalence = results.get_measure("prevalence")
        peak_prevalences.append(prevalence.max())

        state_counts = results.get_measure("state_counts")
        total = network.core_network.number_of_nodes()
        attack_rates.append(state_counts['R'][-1] / total)

    return {
        'peak_prevalence_mean': np.mean(peak_prevalences),
        'peak_prevalence_std': np.std(peak_prevalences),
        'attack_rate_mean': np.mean(attack_rates),
        'attack_rate_std': np.std(attack_rates),
    }

# Run ensemble
stats = run_sir_ensemble(network, beta=0.3, gamma=0.1, n_runs=50)
print(f"Peak prevalence: {stats['peak_prevalence_mean']:.2%} ± {stats['peak_
↪prevalence_std']:.2%}")
print(f"Attack rate: {stats['attack_rate_mean']:.2%} ± {stats['attack_rate_
↪std']:.2%}")
```

Expected Output

```
Peak prevalence: 34.85% ± 2.13%
Attack rate: 98.60% ± 3.42%
```

Note: Mean and std quantify variability across stochastic realizations.

Parameter Grid Sweep

To explore how outcomes depend on parameters, run a grid sweep:

```
import pandas as pd
import networkx as nx

# Use a standard network for reproducibility
G = nx.karate_club_graph()

# Parameter grid
beta_values = [0.1, 0.2, 0.3, 0.4]
gamma_values = [0.05, 0.1, 0.15, 0.2]

# Run grid
results_grid = []

for beta in beta_values:
```

(continues on next page)

(continued from previous page)

```
    for gamma in gamma_values:
        sir = SIRDynamics(G, beta=beta, gamma=gamma, initial_infected=0.1)
        sir.set_seed(42)
        result = sir.run(steps=100)

        prevalence = result.get_measure("prevalence")
        state_counts = result.get_measure("state_counts")

        results_grid.append({
            'beta': beta,
            'gamma': gamma,
            'R0_approx': beta / gamma * 4.6,  # <k>  4.6 for karate club
            'peak_prevalence': prevalence.max(),
            'peak_time': prevalence.argmax(),
            'attack_rate': state_counts['R'][-1] / G.number_of_nodes(),
        })

# Convert to DataFrame
df = pd.DataFrame(results_grid)
print(df)
```

Expected Output (partial)

```
   beta  gamma  R0_approx  peak_prevalence  peak_time  attack_rate
0   0.1   0.05        9.2            0.676          5        1.000
1   0.1   0.10        4.6            0.382          6        0.971
2   0.1   0.15        3.1            0.206          6        0.794
3   0.1   0.20        2.3            0.088          7        0.500
4   0.2   0.05       18.4            0.971          3        1.000
5   0.2   0.10        9.2            0.882          4        1.000
...
```

You can visualize this data with heatmaps or line plots to identify parameter regimes for outbreak control.

Custom Dynamics

Extending BaseDynamics

To implement domain-specific dynamics, subclass `DynamicsProcess` and define:

1. `initialize_state(self, seed=None)`: Return initial state
2. `step(self, state, t)`: Update state for one time step
3. (Optional) Custom measure computation methods

Contract

- `initialize_state` must return a state object (dict, list, or custom)
- `step` must take the current state and return the new state
- Both methods can access `self.graph`, `self.rng`, and `self.params`

Example: Threshold Model

We implement a simple threshold model where nodes adopt a binary opinion if enough neighbors have adopted:

```
from py3plex.dynamics.core import DynamicsProcess
from py3plex.dynamics._utils import iter_multilayer_nodes, iter_multilayer_
↪neighbors

class ThresholdDynamics(DynamicsProcess):
    """Threshold model: nodes adopt state 1 if fraction of neighbors exceeds␣
↪threshold."""

    def __init__(self, graph, seed=None, threshold=0.5, initial_adopters=0.1):
        super().__init__(graph, seed=seed)
        self.threshold = threshold
        self.initial_adopters = initial_adopters

    def initialize_state(self, seed=None):
        """Initialize with some nodes in state 1, rest in state 0."""
        rng = self.rng if seed is None else np.random.default_rng(seed)
        nodes = list(iter_multilayer_nodes(self.graph))

        # Randomly select initial adopters
        n_adopters = int(len(nodes) * self.initial_adopters)
        adopter_indices = rng.choice(len(nodes), size=n_adopters,␣
↪replace=False)
        adopters = set(nodes[i] for i in adopter_indices)

        return {node: 1 if node in adopters else 0 for node in nodes}

    def step(self, state, t):
        """Update rule: adopt if fraction of neighbors >= threshold."""
        new_state = {}

        for node in state:
            if state[node] == 1:
                # Already adopted, stay adopted
                new_state[node] = 1
            else:
                # Count adopting neighbors
                neighbors = list(iter_multilayer_neighbors(self.graph, node))
                if not neighbors:
                    new_state[node] = 0
                    continue

                adopting_neighbors = sum(1 for n in neighbors if state.get(n,␣
↪0) == 1)
                fraction = adopting_neighbors / len(neighbors)

                # Adopt if threshold exceeded
```

(continues on next page)

(continued from previous page)

```
                new_state[node] = 1 if fraction >= self.threshold else 0

        return new_state

# Run threshold model
threshold = ThresholdDynamics(
    network,
    seed=42,
    threshold=0.3,
    initial_adopters=0.1
)

results = threshold.run(steps=50)

# Count adoption over time
adoption_counts = []
for state in results.trajectory:
    adoption_counts.append(sum(state.values()))

print(f"Initial adopters: {adoption_counts[0]}")
print(f"Final adopters: {adoption_counts[-1]}")
print(f"Adoption rate: {adoption_counts[-1] / network.core_network.number_of_
↪nodes():.2%}")
```

Expected Output

```
Initial adopters: 4
Final adopters: 40
Adoption rate: 100.00%
```

Note: With a low threshold (0.3), the initial adopters trigger a cascade where all nodes eventually adopt.

Adding Custom Measures

To expose custom measures through `.get_measure()`, override the `compute_measure` method:

```
class ThresholdDynamics(DynamicsProcess):
    # ... (same as above) ...

    def compute_measure(self, measure_name, trajectory):
        """Compute custom measures for threshold model."""
        if measure_name == "adoption_fraction":
            # Fraction of nodes in state 1 over time
            fractions = []
            for state in trajectory:
                fractions.append(sum(state.values()) / len(state))
            return np.array(fractions)

        elif measure_name == "cascade_size":
```

(continues on next page)

(continued from previous page)

```
            # Final number of adopters
            final_state = trajectory[-1]
            return sum(final_state.values())

        else:
            raise ValueError(f"Unknown measure: {measure_name}")

# Now you can use:
results = threshold.run(steps=50)
adoption_frac = results.get_measure("adoption_fraction")
cascade_size = results.get_measure("cascade_size")

print(f"Cascade size: {cascade_size} nodes")
```

This pattern makes your custom dynamics consistent with built-in models.

Query Dynamics Results with DSL

Goal: Use py3plex's Domain-Specific Language (DSL) to query and analyze simulation results efficiently.

After running dynamics simulations, the DSL provides powerful tools for querying infected nodes, analyzing layer-specific patterns, and extracting subpopulations. This is especially useful for multilayer networks where dynamics vary across layers.

Prerequisites:

- A dynamics simulation result object (from `SIRDynamics`, `SISDynamics`, etc.)
- Familiarity with DSL basics (see *How to Query Multilayer Graphs with the SQL-like DSL* for full tutorial)

Attaching Dynamics State as Node Attributes

To use DSL for querying simulation results, first attach the dynamics state to the network as node attributes:

```
from py3plex.core import multinet
from py3plex.dynamics import SIRDynamics
from py3plex.dsl import execute_query, Q

# Load network
network = multinet.multi_layer_network(directed=False)
network.load_network(
    "py3plex/datasets/_data/synthetic_multilayer.edges",
    input_type="multiedgelist"
)

# Run SIR simulation
sir = SIRDynamics(
    network,
    beta=0.3,
```

(continues on next page)

(continued from previous page)

```
    gamma=0.1,
    initial_infected=0.05
)
sir.set_seed(42)
results = sir.run(steps=100)

# Attach final state to network as node attributes
final_state = results.trajectory[-1]
for node, state in final_state.items():
    network.core_network.nodes[node]['sir_state'] = state
    # Also add a numeric version for filtering
    network.core_network.nodes[node]['is_infected'] = (state == 'I')
    network.core_network.nodes[node]['is_recovered'] = (state == 'R')
    network.core_network.nodes[node]['is_susceptible'] = (state == 'S')
```

Query Infected Nodes

Find all currently infected nodes:

```
# Using string syntax
infected = execute_query(
    network,
    'SELECT nodes WHERE sir_state="I"'
)

print(f"Infected nodes: {len(infected)}")
print(f"Sample infected:")
for node in list(infected)[:5]:
    print(f"  {node}")
```

Expected output:

```
Infected nodes: 12
Sample infected:
  ('node7', 'layer1')
  ('node12', 'layer2')
  ('node3', 'layer3')
  ('node15', 'layer1')
  ('node8', 'layer2')
```

Using builder API for complex queries:

```
from py3plex.dsl import Q, L

# Find infected nodes with high degree
high_degree_infected = (
    Q.nodes()
     .where(sir_state='I')
     .compute("degree")
     .where(degree__gt=5)
```

(continues on next page)

(continued from previous page)

```
     .order_by("degree", reverse=True)
     .execute(network)
)

print(f"High-degree infected nodes: {len(high_degree_infected)}")
for node, data in list(high_degree_infected.items())[:5]:
    print(f"  {node}: degree={data['degree']}")
```

Expected output:

```
High-degree infected nodes: 7
  ('node7', 'layer1'): degree=12
  ('node12', 'layer2'): degree=10
  ('node3', 'layer1'): degree=9
  ('node15', 'layer3'): degree=8
  ('node8', 'layer2'): degree=7
```

Layer-Specific Dynamics Queries

Compare infection levels across layers:

```
layers = network.get_layers()

print("Infection by layer:")
for layer in layers:
    layer_nodes = (
        Q.nodes()
         .from_layers(L[layer])
         .execute(network)
    )

    infected_in_layer = sum(
        1 for node in layer_nodes
        if network.core_network.nodes[node].get('sir_state') == 'I'
    )

    prevalence = infected_in_layer / len(layer_nodes) if len(layer_nodes) > 0␣
↪else 0
    print(f"  {layer}: {infected_in_layer}/{len(layer_nodes)} ({prevalence:.
↪2%})")
```

Expected output:

```
Infection by layer:
  layer1: 5/40 (12.50%)
  layer2: 4/40 (10.00%)
  layer3: 3/40 (7.50%)
```

Find susceptible nodes in high-risk layers:

```
# Identify high-risk layers (high infection prevalence)
high_risk_layer = 'layer1'  # From analysis above

# Find susceptible nodes in high-risk layer
at_risk_nodes = (
    Q.nodes()
     .from_layers(L[high_risk_layer])
     .where(sir_state='S')
     .compute("degree")
     .execute(network)
)

# Sort by degree (higher degree = more neighbors, higher risk)
at_risk_list = sorted(
    at_risk_nodes.items(),
    key=lambda x: x[1]['degree'],
    reverse=True
)

print(f"High-risk susceptible nodes in {high_risk_layer}:")
for node, data in at_risk_list[:5]:
    print(f"  {node}: degree={data['degree']}")
```

Temporal Queries Across Simulation Steps

Track state changes over time:

```
# Select a few nodes to track
tracked_nodes = [
    ('node7', 'layer1'),
    ('node12', 'layer2'),
    ('node3', 'layer3')
]

print("Temporal evolution:")
print("Time  " + "  ".join(f"{n[0]:8}" for n in tracked_nodes))
print("-" * 50)

# Sample time points
time_points = [0, 20, 40, 60, 80, 100]
for t in time_points:
    if t < len(results.trajectory):
        state_t = results.trajectory[t]
        states_str = "  ".join(
            f"{state_t.get(node, '-'):8}"
            for node in tracked_nodes
        )
        print(f"{t:4}  {states_str}")
```

Expected output:

```
Temporal evolution:
Time  node7     node12    node3
--------------------------------------------------
   0  S         S         S
  20  I         S         S
  40  R         I         I
  60  R         R         I
  80  R         R         R
 100  R         R         R
```

Query state transitions:

```python
# Find nodes that became infected between t=20 and t=40
state_20 = results.trajectory[20]
state_40 = results.trajectory[40]

newly_infected = [
    node for node in state_20.keys()
    if state_20[node] == 'S' and state_40.get(node) == 'I'
]

print(f"Newly infected between t=20 and t=40: {len(newly_infected)}")
print(f"Sample: {newly_infected[:5]}")
```

Extract Subpopulations

Extract network of recovered nodes:

```python
# Query recovered nodes
recovered_nodes = execute_query(
    network,
    'SELECT nodes WHERE sir_state="R"'
)

# Extract induced subgraph
recovered_subgraph = network.core_network.subgraph(recovered_nodes)

print(f"Recovered subnetwork:")
print(f"  Nodes: {recovered_subgraph.number_of_nodes()}")
print(f"  Edges: {recovered_subgraph.number_of_edges()}")
print(f"  Layers: {set(node[1] for node in recovered_nodes)}")
```

Expected output:

```
Recovered subnetwork:
  Nodes: 38
  Edges: 145
  Layers: {'layer1', 'layer2', 'layer3'}
```

Analyze connectivity within susceptible population:

```python
import networkx as nx

# Get susceptible nodes
susceptible = execute_query(
    network,
    'SELECT nodes WHERE sir_state="S"'
)

# Extract subgraph
S_subgraph = network.core_network.subgraph(susceptible)

# Compute connected components
if S_subgraph.number_of_nodes() > 0:
    components = list(nx.connected_components(S_subgraph))
    print(f"Susceptible population:")
    print(f"  Nodes: {S_subgraph.number_of_nodes()}")
    print(f"  Connected components: {len(components)}")
    print(f"  Largest component: {max(len(c) for c in components)} nodes")
```

Combine with Dynamics Measures

Identify superspreaders (high-degree infected nodes):

```python
from py3plex.dsl import Q

# Find infected nodes with highest degree
superspreaders = (
    Q.nodes()
     .where(sir_state='I')
     .compute("degree", "betweenness_centrality")
     .where(degree__gt=7)
     .order_by("betweenness_centrality", reverse=True)
     .execute(network)
)

print(f"Potential superspreaders: {len(superspreaders)}")
for node, data in list(superspreaders.items())[:5]:
    print(f"  {node}: deg={data['degree']}, betw={data['betweenness_
↪centrality']:.4f}")
```

Expected output:

```
Potential superspreaders: 4
  ('node7', 'layer1'): deg=12, betw=0.0456
  ('node12', 'layer2'): deg=10, betw=0.0345
  ('node15', 'layer3'): deg=9, betw=0.0234
  ('node3', 'layer1'): deg=8, betw=0.0198
```

Compute attack rate by layer using DSL:

```
print("Attack rate (recovered / total) by layer:")
for layer in network.get_layers():
    layer_nodes = Q.nodes().from_layers(L[layer]).execute(network)

    recovered = sum(
        1 for node in layer_nodes
        if network.core_network.nodes[node].get('sir_state') == 'R'
    )

    attack_rate = recovered / len(layer_nodes) if len(layer_nodes) > 0 else 0
    print(f"  {layer}: {attack_rate:.2%}")
```

Export Dynamics Results with DSL

Create analysis-ready DataFrame:

```
import pandas as pd
from py3plex.dsl import Q

# Query all nodes with states and metrics
result = (
    Q.nodes()
     .compute("degree", "betweenness_centrality")
     .execute(network)
)

# Convert to DataFrame
df = pd.DataFrame([
    {
        'node': node[0],
        'layer': node[1],
        'sir_state': data.get('sir_state', 'unknown'),
        'degree': data['degree'],
        'betweenness': data['betweenness_centrality']
    }
    for node, data in result.items()
])

# Group by state and compute statistics
state_stats = df.groupby('sir_state').agg({
    'degree': ['mean', 'std', 'max'],
    'betweenness': ['mean', 'std', 'max']
})

print("Statistics by SIR state:")
print(state_stats)

# Export
df.to_csv('sir_results_with_metrics.csv', index=False)
```

Why use DSL for dynamics analysis?

- **Declarative filtering:** Express queries naturally without manual loops
- **Layer-aware:** Built-in support for querying specific layers or cross-layer patterns
- **Composable:** Chain operations to build complex analyses
- **Integration:** Seamlessly combine dynamics state with network metrics
- **Performance:** DSL optimizes query execution for large multilayer networks

Next steps with DSL:

- **Full DSL tutorial:** *How to Query Multilayer Graphs with the SQL-like DSL* - Comprehensive guide with advanced patterns
- **Temporal queries:** *How to Query Multilayer Graphs with the SQL-like DSL* (Temporal Queries section) - Time-varying networks
- **Community detection integration:** *How to Run Community Detection on Multilayer Networks* (Query Communities with DSL section)

Next Steps

Now that you understand multilayer dynamics in py3plex, explore:

- **Visualization**: *How to Visualize Multilayer Networks* to plot epidemic curves and multilayer network states
- **Multilayer theory**: *Multilayer Networks 101* for formal background on multilayer structures
- **Advanced examples**: *Examples & Recipes* for notebooks using SIR/SIS in complex scenarios
- **API reference**: *Algorithm Roadmap* for full documentation of `py3plex.dynamics`

Typical research workflows:

1. Design a multilayer network representing your system (social + physical, online + offline, etc.)
2. Choose or implement a dynamics model (epidemic, diffusion, opinion, etc.)
3. Run parameter sweeps to explore outbreak conditions or intervention strategies
4. Visualize trajectories and multilayer-specific statistics (per-layer prevalence, layer switching rates)
5. Compare scenarios to inform policy or design decisions

For more complex simulations (time-varying parameters, multi-stage interventions), consider using the high-level simulation builder API (`py3plex.dynamics.D`) or config-driven workflows (`build_dynamics_from_config`).

3.3.7 How to Run Random Walk Algorithms

Goal: Generate network embeddings and node representations using random walk algorithms.

Prerequisites: A loaded network (see *How to Load and Build Networks*).

> **Note**
>
> Where to find this data

Examples in this guide create networks programmatically for clarity. You can also:

- Use built-in generators: `from py3plex.algorithms import random_generators`
- Load from repository files: `datasets/multiedgelist.txt`
- Fetch real-world datasets: `from py3plex.datasets import fetch_multilayer`

Node2Vec Embeddings

Node2Vec generates vector representations of nodes by simulating biased random walks:

```
from py3plex.core import multinet
from py3plex.wrappers import train_node2vec

# Create a sample network
network = multinet.multi_layer_network()
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1],
    ['Bob', 'friends', 'Charlie', 'friends', 1],
    ['Alice', 'colleagues', 'Charlie', 'colleagues', 1],
], input_type="list")

# Train Node2Vec
embeddings = train_node2vec(
    network,
    dimensions=128,     # Embedding dimensionality
    walk_length=80,     # Length of each walk
    num_walks=10,       # Walks per node
    p=1.0,             # Return parameter
    q=1.0,             # In-out parameter
    workers=4
)

# Access embeddings
node = ('Alice', 'friends')
vector = embeddings[node]
print(f"Embedding dimension: {len(vector)}")
```

Expected output:

```
Embedding dimension: 128
```

The p and q parameters control the walk behavior:

- **p** (return parameter): Likelihood of returning to previous node
- **q** (in-out parameter): Likelihood of exploring outward vs. staying local

DeepWalk Embeddings

DeepWalk is a special case of Node2Vec with `p=1, q=1`:

```
from py3plex.wrappers import train_deepwalk

embeddings = train_deepwalk(
    network,
    dimensions=128,
    walk_length=80,
    num_walks=10,
    workers=4
)
```

Using Embeddings for Downstream Tasks

Node Classification

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
import numpy as np

# Prepare data
nodes = list(embeddings.keys())
X = np.array([embeddings[node] for node in nodes])

# Assuming you have labels
y = np.array([get_label(node) for node in nodes])

# Train classifier
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

clf = LogisticRegression()
clf.fit(X_train, y_train)

accuracy = clf.score(X_test, y_test)
print(f"Classification accuracy: {accuracy:.2%}")
```

Link Prediction

```
from sklearn.metrics.pairwise import cosine_similarity

# Compute similarity between nodes
node1 = ('Alice', 'friends')
node2 = ('Bob', 'friends')

vec1 = embeddings[node1].reshape(1, -1)
vec2 = embeddings[node2].reshape(1, -1)
```

(continues on next page)

(continued from previous page)

```
similarity = cosine_similarity(vec1, vec2)[0][0]
print(f"Similarity: {similarity:.3f}")

# Predict links for high-similarity pairs
threshold = 0.7
if similarity > threshold:
    print(f"High likelihood of connection between {node1} and {node2}")
```

Node Clustering

```
from sklearn.cluster import KMeans

# Cluster nodes based on embeddings
nodes = list(embeddings.keys())
X = np.array([embeddings[node] for node in nodes])

kmeans = KMeans(n_clusters=5, random_state=42)
clusters = kmeans.fit_predict(X)

# Map nodes to clusters
node_clusters = dict(zip(nodes, clusters))

print("Cluster assignments:")
for node, cluster in list(node_clusters.items())[:10]:
    print(f"{node} → Cluster {cluster}")
```

Visualizing Embeddings

Use dimensionality reduction to visualize:

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# Reduce to 2D
nodes = list(embeddings.keys())
X = np.array([embeddings[node] for node in nodes])

tsne = TSNE(n_components=2, random_state=42)
X_2d = tsne.fit_transform(X)

# Plot
plt.figure(figsize=(12, 8))
plt.scatter(X_2d[:, 0], X_2d[:, 1], alpha=0.5)

# Label a few nodes
for i, node in enumerate(nodes[:20]):
    plt.annotate(
        str(node),
```

(continues on next page)

(continued from previous page)

```
        (X_2d[i, 0], X_2d[i, 1]),
        fontsize=8
    )

plt.title('Node Embeddings (t-SNE)')
plt.savefig('embeddings_2d.png', dpi=300, bbox_inches='tight')
plt.show()
```

Saving and Loading Embeddings

Save to File

```
import pickle

# Save embeddings
with open('embeddings.pkl', 'wb') as f:
    pickle.dump(embeddings, f)

# Load embeddings
with open('embeddings.pkl', 'rb') as f:
    loaded_embeddings = pickle.load(f)
```

Export to CSV

```
import pandas as pd

# Convert to DataFrame
data = []
for node, vector in embeddings.items():
    row = {'node': str(node)}
    for i, val in enumerate(vector):
        row[f'dim_{i}'] = val
    data.append(row)

df = pd.DataFrame(data)
df.to_csv('embeddings.csv', index=False)
```

Parameter Tuning

Grid Search for Best Parameters

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
import numpy as np

param_grid = {
    'dimensions': [64, 128, 256],
    'walk_length': [40, 80, 120],
```

(continues on next page)

(continued from previous page)

```
    'num_walks': [5, 10, 20]
}

best_score = 0
best_params = None

for dims in param_grid['dimensions']:
    for walk_len in param_grid['walk_length']:
        for num_walks in param_grid['num_walks']:
            # Train embeddings
            emb = train_node2vec(
                network,
                dimensions=dims,
                walk_length=walk_len,
                num_walks=num_walks
            )

            # Evaluate (assuming you have labels)
            X = np.array([emb[n] for n in nodes])
            scores = cross_val_score(
                LogisticRegression(),
                X, y, cv=5
            )

            mean_score = scores.mean()
            if mean_score > best_score:
                best_score = mean_score
                best_params = {
                    'dimensions': dims,
                    'walk_length': walk_len,
                    'num_walks': num_walks
                }

print(f"Best parameters: {best_params}")
print(f"Best score: {best_score:.3f}")
```

Layer-Specific Embeddings

Generate embeddings for individual layers:

```
from py3plex.dsl import Q, L

layer_embeddings = {}

for layer in network.get_layers():
    # Extract layer subgraph
    subgraph = Q.edges().from_layers(L[layer]).execute(network)

    # Train embeddings on this layer
```

(continues on next page)

(continued from previous page)

```
    emb = train_node2vec(subgraph, dimensions=128)
    layer_embeddings[layer] = emb

    print(f"Generated embeddings for layer: {layer}")
```

Query and Filter Nodes Before Embedding with DSL

Goal: Use py3plex's DSL to select specific node subsets for targeted embedding generation.

Random walks and embeddings on large networks can be computationally expensive. The DSL allows you to filter and extract subnetworks before running embedding algorithms, focusing on nodes of interest.

Filter High-Degree Nodes

Generate embeddings only for hub nodes:

```
from py3plex.core import multinet
from py3plex.wrappers import train_node2vec
from py3plex.dsl import Q, execute_query

# Load network
network = multinet.multi_layer_network(directed=False)
network.load_network(
    "py3plex/datasets/_data/synthetic_multilayer.edges",
    input_type="multiedgelist"
)

# Use DSL to find high-degree nodes (hubs)
hubs = (
    Q.nodes()
     .compute("degree")
     .where(degree__gt=8)  # Degree > 8
     .execute(network)
)

print(f"Found {len(hubs)} hub nodes")

# Extract subgraph containing only hubs
hub_subgraph = network.core_network.subgraph(hubs.keys())

# Train embeddings on hub subgraph
hub_network = multinet.multi_layer_network(directed=False)
hub_network.core_network = hub_subgraph.copy()

hub_embeddings = train_node2vec(
    hub_network,
    dimensions=64,
    walk_length=40,
    num_walks=10
```

(continues on next page)

(continued from previous page)

```
)

print(f"Generated embeddings for {len(hub_embeddings)} hubs")
```

Expected output:

```
Found 15 hub nodes
Generated embeddings for 15 hubs
```

Layer-Specific Node Selection

Generate embeddings for nodes active in multiple layers:

```
from py3plex.dsl import Q, L
from collections import Counter

# Count layer participation for each node
node_layers = Counter()
for node, layer in network.get_nodes():
    node_layers[node] += 1

# Find nodes in 2+ layers
multilayer_nodes = {
    node for node, count in node_layers.items()
    if count >= 2
}

print(f"Nodes in 2+ layers: {len(multilayer_nodes)}")

# Convert back to (node, layer) tuples
multilayer_node_tuples = [
    (node, layer)
    for node, layer in network.get_nodes()
    if node in multilayer_nodes
]

# Extract subgraph
multi_subgraph = network.core_network.subgraph(multilayer_node_tuples)

# Generate embeddings
multi_network = multinet.multi_layer_network(directed=False)
multi_network.core_network = multi_subgraph.copy()

multi_embeddings = train_node2vec(multi_network, dimensions=64)
print(f"Generated embeddings for {len(multi_embeddings)} multilayer nodes")
```

Expected output:

```
Nodes in 2+ layers: 35
Generated embeddings for 105 multilayer nodes
```

Query Nodes by Centrality

Focus embeddings on central nodes:

```
from py3plex.dsl import Q

# Find nodes with high betweenness centrality
central_nodes = (
    Q.nodes()
     .compute("betweenness_centrality", "degree")
     .where(betweenness_centrality__gt=0.01)
     .execute(network)
)

print(f"High-centrality nodes: {len(central_nodes)}")

# Extract and embed
central_subgraph = network.core_network.subgraph(central_nodes.keys())
central_network = multinet.multi_layer_network(directed=False)
central_network.core_network = central_subgraph.copy()

central_embeddings = train_node2vec(
    central_network,
    dimensions=128,
    walk_length=80,
    num_walks=10
)

# Compare embedding similarity for central nodes
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

node_list = list(central_embeddings.keys())[:5]
vectors = np.array([central_embeddings[n] for n in node_list])

sim_matrix = cosine_similarity(vectors)
print("\nSimilarity matrix (first 5 central nodes):")
print(sim_matrix)
```

Combine Embeddings with Node Attributes

Query embedded nodes with specific properties:

```
# After generating embeddings, attach them as attributes
for node, vector in embeddings.items():
    # Store embedding norm as an attribute
    embedding_norm = np.linalg.norm(vector)
    network.core_network.nodes[node]['embedding_norm'] = embedding_norm

# Query nodes with large embedding norms
large_norm_nodes = execute_query(
```

(continues on next page)

(continued from previous page)

```
    network,
    'SELECT nodes WHERE embedding_norm > 10.0'
)

print(f"Nodes with large embedding norms: {len(large_norm_nodes)}")
```

Layer-Specific Embedding Analysis

Compare embeddings across layers:

```
from py3plex.dsl import Q, L

# Generate embeddings per layer
layer_embeddings = {}
layer_stats = {}

for layer in network.get_layers():
    # Extract layer
    layer_nodes = Q.nodes().from_layers(L[layer]).execute(network)
    layer_edges = Q.edges().from_layers(L[layer]).execute(network)

    print(f"\nLayer: {layer}")
    print(f"  Nodes: {len(layer_nodes)}")
    print(f"  Edges: {len(layer_edges)}")

    # Extract layer subgraph
    layer_subgraph = network.core_network.subgraph(layer_nodes.keys())

    # Skip if too few nodes
    if len(layer_nodes) < 5:
        print(f"  [SKIP] Too few nodes")
        continue

    # Create layer network
    layer_net = multinet.multi_layer_network(directed=False)
    layer_net.core_network = layer_subgraph.copy()

    # Generate embeddings
    try:
        emb = train_node2vec(
            layer_net,
            dimensions=64,
            walk_length=40,
            num_walks=10,
            workers=1
        )

        layer_embeddings[layer] = emb
```

(continues on next page)

(continued from previous page)

```
        # Compute statistics
        vectors = np.array(list(emb.values()))
        mean_norm = np.mean([np.linalg.norm(v) for v in vectors])

        layer_stats[layer] = {
            'n_nodes': len(emb),
            'mean_embedding_norm': mean_norm
        }

        print(f"  ✓ Embeddings generated: {len(emb)} nodes")
        print(f"  Mean embedding norm: {mean_norm:.2f}")
    except Exception as e:
        print(f"  [ERROR] {e}")
```

Expected output:

```
Layer: layer1
  Nodes: 40
  Edges: 95
  ✓ Embeddings generated: 40 nodes
  Mean embedding norm: 12.34

Layer: layer2
  Nodes: 40
  Edges: 87
  ✓ Embeddings generated: 40 nodes
  Mean embedding norm: 11.89

Layer: layer3
  Nodes: 40
  Edges: 102
  ✓ Embeddings generated: 40 nodes
  Mean embedding norm: 13.01
```

Export Embeddings with Metadata Using DSL

Create analysis-ready embedding exports:

```
import pandas as pd
from py3plex.dsl import Q

# Compute node metrics
metrics = (
    Q.nodes()
     .compute("degree", "betweenness_centrality")
     .execute(network)
)

# Combine embeddings with metrics
```

(continues on next page)

(continued from previous page)

```
data = []
for node in embeddings.keys():
    row = {
        'node': node[0],
        'layer': node[1],
        'degree': metrics[node]['degree'],
        'betweenness': metrics[node]['betweenness_centrality']
    }

    # Add embedding dimensions
    for i, val in enumerate(embeddings[node]):
        row[f'emb_{i}'] = val

    data.append(row)

# Create DataFrame
df = pd.DataFrame(data)

# Export
df.to_csv('embeddings_with_metrics.csv', index=False)
print(f"Exported {len(df)} node embeddings with metadata")
```

Why use DSL for embedding workflows?

- **Targeted embedding:** Focus on relevant node subsets, reducing computation time
- **Layer-aware:** Generate layer-specific embeddings seamlessly
- **Metric integration:** Combine embeddings with centrality and other network metrics
- **Filtering:** Select nodes by degree, centrality, or custom attributes before embedding
- **Reproducible:** Declarative queries document node selection criteria

Next steps with DSL:

- **Full DSL tutorial:** *How to Query Multilayer Graphs with the SQL-like DSL* - Comprehensive guide with advanced patterns
- **Community detection:** *How to Run Community Detection on Multilayer Networks* - Use embeddings for community analysis
- **Dynamics analysis:** *How to Simulate Multilayer Dynamics* - Combine embeddings with dynamics results

Next Steps

- **Use embeddings for ML tasks:** See sklearn documentation
- **Visualize networks:** *How to Visualize Multilayer Networks*
- **Understand algorithms:** *Algorithm Landscape*
- **API reference:** *Algorithm Roadmap*

3.3.8 How to Export and Serialize Networks

Goal: Save multilayer networks to files and load them back.

Prerequisites: A network to export (see *How to Load and Build Networks*).

Quick Save and Load

Using Pickle (Recommended)

```
from py3plex.core import multinet
import pickle

# Create or load network
network = multinet.multi_layer_network()
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1.0],
    ['Bob', 'work', 'Carol', 'work', 1.0],
], input_type="list")

# Save to file
with open('network.pkl', 'wb') as f:
    pickle.dump(network, f)

# Load from file
with open('network.pkl', 'rb') as f:
    loaded_network = pickle.load(f)

loaded_network.basic_stats()
```

Export as Edge List

Simple Text Format

```
# Export edges to text file
with open('edges.txt', 'w') as f:
    for edge in network.get_edges():
        source, target = edge
        source_node, source_layer = source
        target_node, target_layer = target

        # Get edge attributes
        attrs = network.get_edge_data(source, target)
        weight = attrs.get('weight', 1.0)

        f.write(f"{source_node} {source_layer} {target_node} {target_layer}
↪{weight}\n")
```

CSV Format

```
import pandas as pd

# Convert to DataFrame
edges_data = []
for edge in network.get_edges():
    source, target = edge
    source_node, source_layer = source
    target_node, target_layer = target

    attrs = network.get_edge_data(source, target)

    edges_data.append({
        'source': source_node,
        'source_layer': source_layer,
        'target': target_node,
        'target_layer': target_layer,
        'weight': attrs.get('weight', 1.0)
    })

df = pd.DataFrame(edges_data)
df.to_csv('network_edges.csv', index=False)
```

Export as JSON

Full Network Structure

```
import json

# Build JSON structure
network_data = {
    'nodes': [],
    'edges': [],
    'layers': list(network.get_layers())
}

# Add nodes
for node, layer in network.get_nodes():
    node_attrs = network.get_node_attributes(node, layer)
    network_data['nodes'].append({
        'id': node,
        'layer': layer,
        'attributes': node_attrs
    })

# Add edges
for edge in network.get_edges():
    source, target = edge
    source_node, source_layer = source
```

(continues on next page)

(continued from previous page)

```
    target_node, target_layer = target

    attrs = network.get_edge_data(source, target)

    network_data['edges'].append({
        'source': source_node,
        'source_layer': source_layer,
        'target': target_node,
        'target_layer': target_layer,
        'attributes': attrs
    })

# Save
with open('network.json', 'w') as f:
    json.dump(network_data, f, indent=2)
```

Load from JSON

```
import json
from py3plex.core import multinet

# Load JSON
with open('network.json', 'r') as f:
    data = json.load(f)

# Reconstruct network
network = multinet.multi_layer_network()

# Add edges (nodes created automatically)
for edge in data['edges']:
    network.add_edge(
        edge['source'], edge['source_layer'],
        edge['target'], edge['target_layer'],
        **edge['attributes']
    )
```

Export to NetworkX

Convert to Single-Layer NetworkX Graph

```
import networkx as nx

# Flatten to single layer
G = nx.Graph()

for edge in network.get_edges():
    source, target = edge
    source_node, source_layer = source
```

(continues on next page)

(continued from previous page)

```
    target_node, target_layer = target

    # Add nodes with layer info
    G.add_node(source_node, layer=source_layer)
    G.add_node(target_node, layer=target_layer)

    # Add edge
    attrs = network.get_edge_data(source, target)
    G.add_edge(source_node, target_node, **attrs)

# Save as GraphML
nx.write_graphml(G, 'network.graphml')
```

Export Specific Layer

```
from py3plex.dsl import Q, L

# Extract single layer
layer = 'friends'
subgraph = Q.edges().from_layers(L[layer]).execute(network)

# Convert to NetworkX
G = nx.Graph()
for edge in subgraph.get_edges():
    source, target = edge
    G.add_edge(source[0], target[0])  # Just node IDs

# Save
nx.write_edgelist(G, f'layer_{layer}.edgelist')
```

High-Performance: Apache Arrow/Parquet

For Large Networks

```
import pyarrow as pa
import pyarrow.parquet as pq

# Convert edges to table
edges_data = {
    'source': [],
    'source_layer': [],
    'target': [],
    'target_layer': [],
    'weight': []
}

for edge in network.get_edges():
    source, target = edge
```

(continues on next page)

(continued from previous page)

```
    source_node, source_layer = source
    target_node, target_layer = target

    attrs = network.get_edge_data(source, target)

    edges_data['source'].append(source_node)
    edges_data['source_layer'].append(source_layer)
    edges_data['target'].append(target_node)
    edges_data['target_layer'].append(target_layer)
    edges_data['weight'].append(attrs.get('weight', 1.0))

# Create Arrow table
table = pa.table(edges_data)

# Save as Parquet
pq.write_table(table, 'network.parquet')
```

Load from Parquet

```
import pyarrow.parquet as pq
from py3plex.core import multinet

# Load table
table = pq.read_table('network.parquet')
df = table.to_pandas()

# Reconstruct network
network = multinet.multi_layer_network()

for _, row in df.iterrows():
    network.add_edge(
        row['source'], row['source_layer'],
        row['target'], row['target_layer'],
        weight=row['weight']
    )
```

Export Statistics and Results

Save Analysis Results

```
from py3plex.dsl import Q

# Compute statistics
result = (
    Q.nodes()
     .compute("degree", "betweenness_centrality")
     .execute(network)
)
```

(continues on next page)

(continued from previous page)

```
# Save to CSV
df = result.to_pandas()
df.to_csv('node_statistics.csv', index=False)
```

Save Communities

```
from py3plex.algorithms.community_detection import louvain_communities
import pandas as pd

# Detect communities
communities = louvain_communities(network)

# Convert to DataFrame
data = []
for (node, layer), comm_id in communities.items():
    data.append({
        'node': node,
        'layer': layer,
        'community': comm_id
    })

df = pd.DataFrame(data)
df.to_csv('communities.csv', index=False)
```

Batch Export

Export Multiple Formats

```
import os

# Create output directory
os.makedirs('export', exist_ok=True)

# Export in multiple formats
formats = {
    'pickle': lambda: pickle.dump(network, open('export/network.pkl', 'wb')),
    'json': lambda: json.dump(network_data, open('export/network.json', 'w')),
    'csv': lambda: df.to_csv('export/edges.csv', index=False)
}

for fmt, export_func in formats.items():
    try:
        export_func()
        print(f"Exported to {fmt}")
    except Exception as e:
        print(f"Failed to export {fmt}: {e}")
```

Next Steps

- **Load data:** *How to Load and Build Networks*
- **Visualize networks:** *How to Visualize Multilayer Networks*
- **API reference:** *API Documentation*

3.3.9 How to Visualize Multilayer Networks

Goal: Create publication-ready visualizations of multilayer networks.

Prerequisites: A loaded network (see *How to Load and Build Networks*).

Basic Visualization

Quick Plot

```
from py3plex.core import multinet

# Load network
network = multinet.multi_layer_network()
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1],
    ['Bob', 'friends', 'Carol', 'friends', 1],
    ['Alice', 'work', 'Carol', 'work', 1],
], input_type="list")

# Visualize
network.visualize_network(show=True)
```

This opens an interactive window with the network visualization.

Save to File

```
network.visualize_network(
    output_file='network.png',
    show=False
)
```

Customizing Visualizations

Node Sizes and Colors

```
from py3plex.visualization import visualize_network_custom

# Compute node importance
from py3plex.dsl import Q
result = Q.nodes().compute("degree").execute(network)
degrees = {node: data['degree'] for node, data in result.items()}

# Visualize with custom node sizes
visualize_network_custom(
```

(continues on next page)

(continued from previous page)

```
    network,
    node_sizes=degrees,
    node_color_by_layer=True,
    output_file='network_sized.png'
)
```

Edge Weights

```
visualize_network_custom(
    network,
    edge_width_by_weight=True,
    show=True
)
```

Layout Algorithms

Force-Directed Layout

```
from py3plex.visualization.multilayer import hairball_plot

hairball_plot(
    network,
    layout_algorithm='force_directed',
    output_file='force_layout.png'
)
```

Circular Layout

```
hairball_plot(
    network,
    layout_algorithm='circular',
    output_file='circular_layout.png'
)
```

Layer-Specific Visualization

Visualize Single Layer

```
from py3plex.dsl import Q, L

# Extract layer
layer_network = Q.edges().from_layers(L["friends"]).execute(network)

# Visualize
layer_network.visualize_network(
    output_file='friends_layer.png',
    show=True
)
```

Side-by-Side Layer Comparison

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(18, 6))

for idx, layer in enumerate(['friends', 'work', 'family']):
    layer_net = Q.edges().from_layers(L[layer]).execute(network)
    # Visualize on specific axis
    visualize_on_axis(layer_net, axes[idx], title=f'Layer: {layer}')

plt.tight_layout()
plt.savefig('layers_comparison.png', dpi=300)
```

DSL-Driven Visualization Workflows

Goal: Use DSL queries to select and visualize specific network subsets.

The DSL enables you to create targeted visualizations by filtering nodes and edges before rendering. This is essential for large multilayer networks where visualizing everything is impractical.

Visualize High-Degree Subnetwork

Focus visualization on hub nodes:

```
from py3plex.core import multinet
from py3plex.dsl import Q
import matplotlib.pyplot as plt

# Load network
network = multinet.multi_layer_network(directed=False)
network.load_network(
    "py3plex/datasets/_data/synthetic_multilayer.edges",
    input_type="multiedgelist"
)

# Query high-degree nodes
hubs = (
    Q.nodes()
     .compute("degree")
     .where(degree__gt=6)
     .execute(network)
)

print(f"Hub nodes (degree > 6): {len(hubs)}")

# Extract hub subgraph
hub_subgraph = network.core_network.subgraph(hubs.keys())

# Create visualization
import networkx as nx
```

(continues on next page)

(continued from previous page)

```
pos = nx.spring_layout(hub_subgraph, k=0.5, iterations=50)

plt.figure(figsize=(12, 10))

# Color by layer
layer_colors = {'layer1': 'red', 'layer2': 'blue', 'layer3': 'green'}
node_colors = [layer_colors.get(node[1], 'gray') for node in hub_subgraph.
↪nodes()]

# Size by degree
node_sizes = [hubs[node]['degree'] * 100 for node in hub_subgraph.nodes()]

nx.draw_networkx(
    hub_subgraph,
    pos,
    node_color=node_colors,
    node_size=node_sizes,
    with_labels=True,
    font_size=8,
    alpha=0.7
)

plt.title('Hub Nodes Subnetwork (degree > 6)', fontsize=14)
plt.axis('off')
plt.tight_layout()
plt.savefig('hub_subnetwork.png', dpi=300, bbox_inches='tight')
print("Saved: hub_subnetwork.png")
```

Expected output:

```
Hub nodes (degree > 6): 18
Saved: hub_subnetwork.png
```

Visualize Community Structure with DSL

Community-aware visualization:

```
from py3plex.algorithms.community_detection.community_wrapper import louvain_
↪communities
from py3plex.dsl import Q, execute_query
import matplotlib.pyplot as plt
import networkx as nx

# Detect communities
communities = louvain_communities(network)

# Attach community labels
for (node, layer), comm_id in communities.items():
    network.core_network.nodes[(node, layer)]['community'] = comm_id
```

(continues on next page)

(continued from previous page)

```
# Visualize each community separately
community_ids = set(communities.values())
n_communities = len(community_ids)

fig, axes = plt.subplots(1, min(n_communities, 4), figsize=(20, 5))
if n_communities == 1:
    axes = [axes]

for idx, comm_id in enumerate(sorted(community_ids)[:4]):
    # Query community members
    comm_nodes = execute_query(
        network,
        f'SELECT nodes WHERE community={comm_id}'
    )

    # Extract community subgraph
    comm_subgraph = network.core_network.subgraph(comm_nodes)

    # Visualize
    pos = nx.spring_layout(comm_subgraph, k=0.3)

    # Color by layer
    layer_colors = {'layer1': '#FF6B6B', 'layer2': '#4ECDC4', 'layer3': '
↪#45B7D1'}
    node_colors = [layer_colors.get(node[1], 'gray') for node in comm_
↪subgraph.nodes()]

    nx.draw_networkx(
        comm_subgraph,
        pos,
        ax=axes[idx],
        node_color=node_colors,
        node_size=200,
        with_labels=False,
        alpha=0.8
    )

    axes[idx].set_title(f'Community {comm_id}\n({len(comm_nodes)} nodes)')
    axes[idx].axis('off')

plt.tight_layout()
plt.savefig('communities_separate.png', dpi=300, bbox_inches='tight')
print(f"Saved: communities_separate.png with {min(n_communities, 4)}␣
↪communities")
```

Layer-Specific Visualizations with DSL

Compare layer structures visually:

```
from py3plex.dsl import Q, L
import matplotlib.pyplot as plt
import networkx as nx

layers = network.get_layers()
n_layers = len(layers)

fig, axes = plt.subplots(1, n_layers, figsize=(6*n_layers, 5))
if n_layers == 1:
    axes = [axes]

for idx, layer in enumerate(layers):
    # Query layer nodes and edges
    layer_nodes = Q.nodes().from_layers(L[layer]).execute(network)
    layer_edges = Q.edges().from_layers(L[layer]).execute(network)

    # Extract layer subgraph
    layer_subgraph = network.core_network.subgraph(layer_nodes.keys())

    # Compute metrics for sizing
    layer_metrics = (
        Q.nodes()
         .from_layers(L[layer])
         .compute("degree")
         .execute(network)
    )

    # Visualization
    pos = nx.spring_layout(layer_subgraph, k=0.5, iterations=50)

    # Size by degree
    node_sizes = [layer_metrics[node]['degree'] * 50 for node in layer_
↪subgraph.nodes()]

    nx.draw_networkx(
        layer_subgraph,
        pos,
        ax=axes[idx],
        node_size=node_sizes,
        node_color='lightblue',
        edge_color='gray',
        with_labels=False,
        alpha=0.7
    )

    axes[idx].set_title(f'{layer}\n{len(layer_nodes)} nodes, {len(layer_
↪edges)} edges')
```

(continues on next page)

(continued from previous page)

```
    axes[idx].axis('off')

plt.tight_layout()
plt.savefig('layer_comparison.png', dpi=300, bbox_inches='tight')
print("Saved: layer_comparison.png")
```

Expected output:

```
Saved: layer_comparison.png
```

Visualize Central Nodes

Highlight nodes by centrality:

```
from py3plex.dsl import Q
import matplotlib.pyplot as plt
import networkx as nx
import numpy as np

# Compute centrality for all nodes
centrality_result = (
    Q.nodes()
     .compute("betweenness_centrality", "degree")
     .execute(network)
)

# Extract betweenness values for coloring
betweenness = {
    node: data['betweenness_centrality']
    for node, data in centrality_result.items()
}

# Create visualization
plt.figure(figsize=(14, 10))
pos = nx.spring_layout(network.core_network, k=0.5, iterations=50)

# Node colors based on betweenness (using colormap)
node_list = list(network.core_network.nodes())
betw_values = [betweenness.get(node, 0) for node in node_list]

# Node sizes based on degree
degrees = {node: data['degree'] for node, data in centrality_result.items()}
node_sizes = [degrees.get(node, 1) * 100 for node in node_list]

# Draw
nodes = nx.draw_networkx_nodes(
    network.core_network,
    pos,
    node_size=node_sizes,
```

(continues on next page)

(continued from previous page)

```
    node_color=betw_values,
    cmap=plt.cm.YlOrRd,
    alpha=0.8
)

nx.draw_networkx_edges(
    network.core_network,
    pos,
    alpha=0.2,
    edge_color='gray'
)

# Add colorbar
plt.colorbar(nodes, label='Betweenness Centrality')
plt.title('Network Centrality Visualization\n(size=degree, color=betweenness)
↪', fontsize=14)
plt.axis('off')
plt.tight_layout()
plt.savefig('centrality_viz.png', dpi=300, bbox_inches='tight')
print("Saved: centrality_viz.png")
```

Dynamics State Visualization

Visualize epidemic simulation results:

```
from py3plex.dynamics import SIRDynamics
from py3plex.dsl import Q
import matplotlib.pyplot as plt
import networkx as nx

# Run SIR simulation
sir = SIRDynamics(network, beta=0.3, gamma=0.1, initial_infected=0.05)
sir.set_seed(42)
results = sir.run(steps=100)

# Attach final state
final_state = results.trajectory[-1]
for node, state in final_state.items():
    network.core_network.nodes[node]['sir_state'] = state

# Query infected and recovered nodes
infected = Q.nodes().where(sir_state='I').execute(network)
recovered = Q.nodes().where(sir_state='R').execute(network)
susceptible = Q.nodes().where(sir_state='S').execute(network)

print(f"Infected: {len(infected)}, Recovered: {len(recovered)}, Susceptible:
↪{len(susceptible)}")

# Visualization
```

(continues on next page)

(continued from previous page)

```
plt.figure(figsize=(14, 10))
pos = nx.spring_layout(network.core_network, k=0.5, iterations=50)

# Color by SIR state
state_colors = {'S': 'lightblue', 'I': 'red', 'R': 'green'}
node_colors = [
    state_colors.get(network.core_network.nodes[node].get('sir_state'), 'gray
↪')
    for node in network.core_network.nodes()
]

nx.draw_networkx(
    network.core_network,
    pos,
    node_color=node_colors,
    node_size=300,
    with_labels=False,
    alpha=0.8
)

# Legend
from matplotlib.patches import Patch
legend_elements = [
    Patch(facecolor='lightblue', label=f'Susceptible ({len(susceptible)})'),
    Patch(facecolor='red', label=f'Infected ({len(infected)})'),
    Patch(facecolor='green', label=f'Recovered ({len(recovered)})')
]
plt.legend(handles=legend_elements, loc='upper right')

plt.title('SIR Epidemic Simulation (t=100)', fontsize=14)
plt.axis('off')
plt.tight_layout()
plt.savefig('sir_visualization.png', dpi=300, bbox_inches='tight')
print("Saved: sir_visualization.png")
```

Expected output:

```
Infected: 8, Recovered: 82, Susceptible: 30
Saved: sir_visualization.png
```

Interactive Visualization with DSL Filtering

Create interactive plots with Plotly:

```
from py3plex.dsl import Q
import plotly.graph_objects as go
import networkx as nx

# Query nodes with metrics
```

(continues on next page)

(continued from previous page)

```
metrics = (
    Q.nodes()
     .compute("degree", "betweenness_centrality")
     .execute(network)
)

# Extract subgraph (optional: filter for clarity)
high_degree = {
    node: data for node, data in metrics.items()
    if data['degree'] > 4
}

subgraph = network.core_network.subgraph(high_degree.keys())

# Layout
pos = nx.spring_layout(subgraph, k=0.5)

# Create edge trace
edge_x = []
edge_y = []
for edge in subgraph.edges():
    x0, y0 = pos[edge[0]]
    x1, y1 = pos[edge[1]]
    edge_x.extend([x0, x1, None])
    edge_y.extend([y0, y1, None])

edge_trace = go.Scatter(
    x=edge_x, y=edge_y,
    line=dict(width=0.5, color='#888'),
    hoverinfo='none',
    mode='lines'
)

# Create node trace
node_x = []
node_y = []
node_text = []
node_sizes = []
node_colors = []

for node in subgraph.nodes():
    x, y = pos[node]
    node_x.append(x)
    node_y.append(y)

    degree = high_degree[node]['degree']
    betw = high_degree[node]['betweenness_centrality']

    node_text.append(f'{node}<br>Degree: {degree}<br>Betweenness: {betw:.4f}
```

(continues on next page)

(continued from previous page)

```
↪')
    node_sizes.append(degree * 5)
    node_colors.append(betw)

node_trace = go.Scatter(
    x=node_x, y=node_y,
    mode='markers',
    hoverinfo='text',
    text=node_text,
    marker=dict(
        showscale=True,
        colorscale='YlOrRd',
        size=node_sizes,
        color=node_colors,
        colorbar=dict(
            title="Betweenness"
        ),
        line_width=2
    )
)

# Create figure
fig = go.Figure(
    data=[edge_trace, node_trace],
    layout=go.Layout(
        title='Interactive Network Visualization (degree > 4)',
        showlegend=False,
        hovermode='closest',
        margin=dict(b=0, l=0, r=0, t=40),
        xaxis=dict(showgrid=False, zeroline=False, showticklabels=False),
        yaxis=dict(showgrid=False, zeroline=False, showticklabels=False)
    )
)

# Save to HTML
fig.write_html('interactive_network.html')
print("Saved: interactive_network.html (open in browser)")
```

Why use DSL for visualization?

- **Targeted views:** Filter nodes/edges to reduce visual clutter
- **Multi-scale:** Visualize different network scales (layer, community, subnetwork)
- **Attribute-driven:** Color/size nodes based on computed metrics
- **Reproducible:** Query-based selections are version-controllable
- **Efficient:** Process only relevant subsets for large networks

Next steps with DSL visualization:

- **Full DSL tutorial:** *How to Query Multilayer Graphs with the SQL-like DSL* - Complete DSL reference

- **Community detection:** *How to Run Community Detection on Multilayer Networks* - Visualize communities
- **Dynamics:** *How to Simulate Multilayer Dynamics* - Visualize dynamics results

Community Visualization

Color by Communities

```
from py3plex.algorithms.community_detection import louvain_communities
from py3plex.visualization import visualize_communities

# Detect communities
communities = louvain_communities(network)

# Visualize with community colors
visualize_communities(
    network,
    communities,
    output_file='communities.png',
    show=True
)
```

Interactive Visualization

Using Plotly

```
from py3plex.visualization import plotly_visualization

# Create interactive plot
fig = plotly_visualization(network)

# Show in browser
fig.show()

# Or save as HTML
fig.write_html('network_interactive.html')
```

Export for Gephi/Cytoscape

Export to GraphML

```
import networkx as nx

# Convert to NetworkX
G = network.to_networkx()

# Export
nx.write_graphml(G, 'network.graphml')
```

Then open *network.graphml* in Gephi or Cytoscape for advanced visualization.

Next Steps

- **Compute statistics to visualize:** *How to Compute Network Statistics*
- **Detect communities:** *How to Run Community Detection on Multilayer Networks*
- **API reference:** *API Documentation*

3.3.10 How to Query Multilayer Graphs with the SQL-like DSL

Goal: Use py3plex's SQL-inspired Domain-Specific Language (DSL) to query, filter, and analyze multilayer networks. The DSL is a **first-class query language** specifically designed for multilayer graph structures, providing both string syntax for interactive exploration and a type-safe builder API for production code.

Run this guide online

You can run this tutorial in your browser without any local installation:

[8] Or see the full executable example: `example_dsl_builder_api.py`

What Makes This DSL Special:

- **Graph-aware**: Unlike generic query languages, the DSL understands multilayer structures—layers, layer intersections, intralayer vs. interlayer edges, and (node, layer) tuple semantics.
- **Dual interfaces**: String syntax for rapid prototyping in notebooks; builder API (`Q`, `L`) for IDE autocompletion and type checking.
- **Integrated computation**: Compute centrality, clustering, and other network metrics directly in queries, with results returned as pandas DataFrames or NetworkX graphs.
- **Temporal support**: Query network snapshots and time ranges when your network includes temporal information.

Prerequisites:

- A loaded `multi_layer_network` object (see *How to Load and Build Networks*)
- Basic familiarity with multilayer network concepts (nodes, layers, intralayer/interlayer edges)
- For complete DSL grammar and operator reference, see *DSL Reference*

Conceptual Overview

The DSL has **two complementary interfaces** that compile to the same internal representation:

1. **String Syntax** (`execute_query(network, "SELECT nodes WHERE ...")`)
   - SQL-like, human-readable
   - Ideal for interactive exploration in Jupyter notebooks or the REPL
   - Quick one-liners for common queries
2. **Builder API** (`Q.nodes().where(...).compute(...).execute(network)`)
   - Pythonic, chainable methods

[8] https://colab.research.google.com/github/SkBlaz/py3plex/blob/master/notebooks/query_with_dsl.ipynb

- Type-safe with IDE autocompletion
- Recommended for production code and complex workflows

Mental Model:

A typical DSL query follows this pipeline:

```
SELECT nodes/edges
→ FROM LAYERS (restrict to specific layers)
→ WHERE (filter by attributes or special predicates)
→ COMPUTE (calculate metrics like degree, centrality)
→ ORDER BY (sort results)
→ LIMIT (cap number of results)
→ EXPORT (materialize as DataFrame, NetworkX graph, etc.)
```

Key Concepts:

- **Nodes as (node, layer) tuples**: In multilayer networks, a node may appear in multiple layers. The DSL represents these as `(node_id, layer_name)` pairs.
- **Layer set algebra**: Combine layers with set operations (`|` union, `&` intersection, `-` difference, `~` complement). The new LayerSet algebra enables expressive layer selection like `L["* - coupling"]` or `L["(ppi | gene) & disease"]`. See *Layer Set Algebra* for complete documentation.
- **Special predicates**: `intralayer=True` selects edges within a layer; `interlayer=("layer1", "layer2")` selects edges crossing specific layers.
- **Lazy execution**: Queries are built incrementally and executed only when `.execute(network)` is called.

Comparison to SQL:

Think of the DSL as SQL for graphs:

- `SELECT nodes WHERE degree > 5` SQL's `SELECT * FROM nodes WHERE degree > 5`
- But instead of tables, you're querying **nodes and edges** with **multilayer and temporal attributes**
- Layer filters and graph-specific predicates (`intralayer`, `interlayer`) have no SQL equivalent

String Syntax (Quick and Readable)

The string syntax provides a concise, SQL-like way to express queries. Best for exploratory analysis and quick investigations.

Basic SELECT

Select all nodes and inspect the result:

> **Note**
>
> Where to find this data
>
> The examples in this guide use one of the following:
>
> - **Built-in data generators** like `random_generators.random_multilayer_ER(...)` (rec-

ommended for self-contained examples)

- **Example files** from the repository at `datasets/multiedgelist.txt` or similar
- The **built-in datasets module**: `from py3plex.datasets import fetch_multilayer`

For this example, we'll create a simple network programmatically:

```
from py3plex.core import multinet
from py3plex.dsl import execute_query

# Create a simple multilayer network
network = multinet.multi_layer_network()
network.add_edges([
    ['alice', 'social', 'bob', 'social', 1],
    ['bob', 'social', 'charlie', 'social', 1],
    ['alice', 'work', 'charlie', 'work', 1],
    ['bob', 'work', 'dave', 'work', 1],
], input_type="list")

# Get all nodes
result = execute_query(network, 'SELECT nodes')

print(f"Found {len(result)} nodes")
# Inspect a few items
for i, (node, data) in enumerate(result.items()):
    print(f"  {node}: {data}")
    if i >= 4:
        break
```

Expected output:

```
Found 6 nodes
  ('alice', 'social'): {'degree': 1, 'layer': 'social', 'layer_count': 2}
  ('bob', 'social'): {'degree': 2, 'layer': 'social', 'layer_count': 2}
  ('charlie', 'social'): {'degree': 1, 'layer': 'social', 'layer_count': 2}
  ('alice', 'work'): {'degree': 1, 'layer': 'work', 'layer_count': 2}
  ('bob', 'work'): {'degree': 1, 'layer': 'work', 'layer_count': 2}
```

Tip

Loading from files

To load from a file in the repository:

```
# Using a file from the datasets/ directory
network.load_network("datasets/multiedgelist.txt", input_type=
↪"multiedgelist")

# Or using an absolute path
import os
path = os.path.join(os.path.dirname(__file__), "datasets", "multiedgelist.
↪txt")
```

```
network.load_network(path, input_type="multiedgelist")
('diana', 'social'): {'degree': 9, 'layer': 'social', 'layer_count': 3}
('eve', 'work'): {'degree': 4, 'layer': 'work', 'layer_count': 1}
```

Note: Keys are `(node, layer)` tuples representing node-layer pairs. The `layer_count` attribute indicates how many layers the node appears in across the entire network.

Filter by Layer

Restrict queries to nodes in a specific layer:

```
# Get nodes in the 'friends' layer only
result = execute_query(
    network,
    'SELECT nodes WHERE layer="friends"'
)

print(f"Nodes in 'friends' layer: {len(result)}")
```

Understanding Layer Filters:

- `layer="friends"` selects only the node-layer pairs where `layer == "friends"`
- This does **not** select all occurrences of nodes across layers—only their representation in the specified layer
- Use `layer_count >= 2` to find nodes appearing in multiple layers

Example with statistics:

```
result = execute_query(
    network,
    'SELECT nodes WHERE layer="friends" COMPUTE degree'
)
df = result.to_pandas()
print(f"Nodes in 'friends': {len(df)}")
print(f"Average degree in 'friends': {df['degree'].mean():.2f}")
print(f"Max degree in 'friends': {df['degree'].max()}")
```

Expected output:

```
Nodes in 'friends': 42
Average degree in 'friends': 5.23
Max degree in 'friends': 15
```

Filter by Property

Use comparisons to filter nodes by computed or intrinsic attributes:

```
# High-degree nodes
result = execute_query(
    network,
```

(continues on next page)

(continued from previous page)

```
    'SELECT nodes WHERE degree > 5'
)
print(f"High-degree nodes: {len(result)}")

# Multilayer nodes with high degree
result = execute_query(
    network,
    'SELECT nodes WHERE degree > 5 AND layer_count >= 2'
)
print(f"High-degree multilayer nodes: {len(result)}")
```

Supported operators: `>`, `>=`, `<`, `<=`, `=`, `!=`

Multiple conditions are combined with `AND`. For more complex logic, use the builder API (see below).

Expected output:

```
High-degree nodes: 34
High-degree multilayer nodes: 18
```

Compute Statistics

The `COMPUTE` clause calculates network metrics and attaches them to result rows. This is where the DSL becomes powerful for analysis:

```
# Compute degree and betweenness centrality for nodes in 'social' layer
result = execute_query(
    network,
    'SELECT nodes WHERE layer="social" '
    'COMPUTE degree COMPUTE betweenness_centrality'
)

# Convert to pandas for analysis
df = result.to_pandas()

print("Top nodes by betweenness centrality:")
print(df[['id', 'degree', 'betweenness_centrality']].head())

print("\nSummary statistics:")
print(df[['degree', 'betweenness_centrality']].describe())
```

Expected output:

```
Top nodes by betweenness centrality:
                 id  degree  betweenness_centrality
0  (alice, social)      12                   0.245
1    (bob, social)       8                   0.189
2    (eve, social)      15                   0.301
3  (frank, social)       7                   0.134
4  (grace, social)      11                   0.221
```

(continues on next page)

(continued from previous page)

```
Summary statistics:
             degree  betweenness_centrality
count     65.000000               65.000000
mean       6.846154                0.112308
std        3.241057                0.089542
min        1.000000                0.000000
25%        4.000000                0.045000
50%        7.000000                0.089000
75%       10.000000                0.167000
max       15.000000                0.301000
```

Available measures include: `degree`, `betweenness_centrality`, `closeness_centrality`, `eigenvector_centrality`, `pagerank`, `clustering`, `communities`. See *DSL Reference* for the complete list.

Use case: This pattern is ideal for generating summary statistics for papers, reports, or further statistical analysis.

Builder API (Type-Safe)

The **builder API is the recommended approach for production code**. It provides:

- IDE autocompletion and inline documentation
- Type checking with tools like mypy
- Clearer error messages
- Easier refactoring and composition of queries

All builder queries compile to the same AST as string queries, ensuring consistent semantics.

Basic Queries

Create and execute queries using the `Q` and `L` imports:

```python
from py3plex.dsl import Q, L

# Get all nodes
result = Q.nodes().execute(network)
print(f"Total nodes: {len(result)}")

# Get nodes from a specific layer
result = (
    Q.nodes()
     .from_layers(L["friends"])
     .execute(network)
)
print(f"Nodes in 'friends' layer: {len(result)}")
```

Query reusability: You can define a query once and execute it with different networks:

```
high_degree_query = Q.nodes().where(degree__gt=10).compute("betweenness_
↪centrality")

# Execute on multiple networks
result_network1 = high_degree_query.execute(network1)
result_network2 = high_degree_query.execute(network2)
```

Filtering

Use `where()` to add filter conditions. The builder API uses Django-style `__` suffixes for comparisons:

```
# Filter by property
result = (
    Q.nodes()
     .where(degree__gt=5)
     .execute(network)
)
print(f"Nodes with degree > 5: {len(result)}")

# Multiple conditions (combined with AND)
result = (
    Q.nodes()
     .from_layers(L["work"])
     .where(degree__gt=3, layer_count__gte=2)
     .execute(network)
)
print(f"Multilayer high-degree nodes in 'work': {len(result)}")
```

Supported comparison suffixes:

- `__gt`: greater than (>)
- `__gte` or `__ge`: greater than or equal (>=)
- `__lt`: less than (<)
- `__lte` or `__le`: less than or equal (<=)
- `__eq`: equal (=)
- `__ne` or `__neq`: not equal (!=)

Understanding `layer_count`:

In multilayer networks, a node may appear in multiple layers. The `layer_count` attribute indicates how many layers the node participates in:

- `layer_count__gte=2`: nodes appearing in at least 2 layers
- `layer_count__eq=1`: nodes appearing in exactly 1 layer (layer-specific nodes)

This is useful for identifying “connector” nodes that bridge multiple contexts.

Computing Metrics

Use `compute()` to calculate network metrics. Metrics are computed efficiently and attached to result rows:

```
# Compute multiple metrics
result = (
    Q.nodes()
     .compute("degree", "betweenness_centrality", "clustering")
     .execute(network)
)

# Convert to DataFrame and analyze
df = result.to_pandas()
print(df.head(10))

# Get top nodes by a metric
top_by_betweenness = df.nlargest(10, 'betweenness_centrality')
print("\nTop 10 nodes by betweenness centrality:")
print(top_by_betweenness[['id', 'betweenness_centrality', 'degree']])
```

Order of operations:

- `compute()` can be called at any point in the chain
- Filters (`where()`) can reference computed metrics only if the metric is computed **before** the filter
- For best performance, filter first, then compute:

```
# Good: Filter first, then compute
result = (
    Q.nodes()
     .from_layers(L["social"])
     .where(degree__gt=5)
     .compute("betweenness_centrality")
     .execute(network)
)
```

Expected output:

```
                  id  degree  betweenness_centrality  clustering
0   (alice, social)      12                0.245000    0.545455
1     (bob, social)       8                0.189000    0.642857
2     (eve, social)      15                0.301000    0.428571
3   (frank, social)       7                0.134000    0.666667
4   (grace, social)      11                0.221000    0.509091
...

Top 10 nodes by betweenness centrality:
                  id  betweenness_centrality  degree
2     (eve, social)                0.301000      15
0   (alice, social)                0.245000      12
```

(continues on next page)

(continued from previous page)

```
4   (grace, social)                0.221000      11
1     (bob, social)                0.189000       8
...
```

Computing Metrics with Uncertainty

New in py3plex 1.0: The DSL now supports **first-class uncertainty** for computed metrics. This allows you to estimate statistical uncertainty (confidence intervals, standard deviations) for network statistics via bootstrap, perturbation, or Monte Carlo methods.

Why uncertainty matters:

- Networks are often noisy or sampled (e.g., social networks with missing edges)
- Centrality metrics can be sensitive to small perturbations
- Uncertainty quantification helps distinguish signal from noise
- Required for robust statistical inference and hypothesis testing

Basic usage:

```
# Compute degree with uncertainty estimation
result = (
    Q.nodes()
     .compute(
         "degree",
         "betweenness_centrality",
         uncertainty=True,
         method="perturbation",  # or "bootstrap", "seed"
         n_samples=100,          # number of resamples
         ci=0.95                 # confidence interval level
     )
     .execute(network)
)

# Access uncertainty information
df = result.to_pandas()
print(df.head())

# Results contain mean, std, and quantiles for each metric
# The 'degree' column now has dict values with uncertainty info
```

Uncertainty methods:

- `"perturbation"`: Drop a small fraction of edges/nodes randomly (default: 5%)
- `"bootstrap"`: Resample nodes/edges with replacement
- `"seed"`: Run stochastic algorithms with different random seeds
- `"jackknife"`: Leave-one-out resampling

Parameters:

- `uncertainty` (bool): Enable uncertainty estimation (default: False)
- `method` (str): Resampling strategy (default: "perturbation")

- `n_samples` (int): Number of resamples (default: 50)
- `ci` (float): Confidence interval level, e.g., 0.95 for 95% CI (default: 0.95)

Example with confidence intervals:

```python
# Find hubs with uncertainty bounds
hubs = (
    Q.nodes()
     .compute(
         "degree",
         "betweenness_centrality",
         uncertainty=True,
         method="perturbation",
         n_samples=200,
         ci=0.95
     )
     .order_by("-betweenness_centrality")
     .limit(10)
     .execute(network)
)

# Extract uncertainty information
df = hubs.to_pandas()

# When uncertainty=True, values are dicts with mean, std, quantiles
for idx, row in df.head().iterrows():
    node_id = row['id']
    bc_info = row['betweenness_centrality']

    if isinstance(bc_info, dict):
        mean = bc_info['mean']
        std = bc_info.get('std', 0)
        ci_low = bc_info.get('quantiles', {}).get(0.025, mean)
        ci_high = bc_info.get('quantiles', {}).get(0.975, mean)

        print(f"{node_id}:")
        print(f"  Betweenness: {mean:.4f} ± {std:.4f}")
        print(f"  95% CI: [{ci_low:.4f}, {ci_high:.4f}]")
```

Expected output:

```
('eve', 'social'):
  Betweenness: 0.3010 ± 0.0234
  95% CI: [0.2589, 0.3442]
('alice', 'social'):
  Betweenness: 0.2450 ± 0.0198
  95% CI: [0.2087, 0.2821]
('grace', 'social'):
  Betweenness: 0.2210 ± 0.0176
  95% CI: [0.1901, 0.2534]
```

Backward compatibility:

When `uncertainty=False` (the default), metrics return scalar values as before. Your existing queries work unchanged:

```
# Traditional deterministic computation
result = Q.nodes().compute("degree").execute(network)
# 'degree' values are scalars (int/float)

# With uncertainty
result_unc = Q.nodes().compute("degree", uncertainty=True).execute(network)
# 'degree' values are dicts with mean, std, quantiles
```

Use cases:

1. **Comparing networks**: Test if centrality differences between networks are statistically significant
2. **Robust ranking**: Identify nodes that consistently rank high across perturbations
3. **Network inference**: Quantify uncertainty when inferring networks from noisy data
4. **Hypothesis testing**: Generate null distributions for significance testing

Performance notes:

- Uncertainty estimation is **opt-in** and only runs when explicitly requested
- Cost scales linearly with `n_samples` (e.g., 100 samples 100× slower)
- Use smaller `n_samples` (20-50) for exploration, larger (100-500) for publication
- Perturbation is fastest; bootstrap and jackknife are more expensive

Further reading:

- *How to Compute Network Statistics*: General guide to network statistics and uncertainty
- `examples/uncertainty/example_first_class_uncertainty.py`: Complete examples
- `py3plex.uncertainty` module: Low-level API for custom uncertainty workflows

Sorting and Limiting

Use `order_by()` and `limit()` to control result ordering and size:

```
# Get top 10 nodes by degree
result = (
    Q.nodes()
     .compute("degree")
     .order_by("-degree")  # "-" prefix for descending
     .limit(10)
     .execute(network)
)

print("Top 10 highest-degree nodes:")
for node, data in result.items():
    print(f"  {node}: degree={data['degree']}")
```

Sorting conventions:

- `order_by("degree")`: ascending (low to high)

- `order_by("-degree")`: descending (high to low)
- Multiple keys: `order_by("-degree", "layer_count")`: sort by degree descending, then layer_count ascending

Expected output:

```
Top 10 highest-degree nodes:
  ('eve', 'social'): degree=15
  ('alice', 'social'): degree=12
  ('grace', 'social'): degree=11
  ('charlie', 'work'): degree=10
  ('henry', 'friends'): degree=9
  ('diana', 'social'): degree=9
  ('bob', 'social'): degree=8
  ('frank', 'social'): degree=7
  ('iris', 'work'): degree=7
  ('jake', 'friends'): degree=6
```

Working with Results

DSL queries return a `QueryResult` object that provides multiple ways to access and export data. Understanding how to work with results is crucial for integrating DSL queries into analysis pipelines.

Access as Dictionary

`QueryResult` provides dictionary-like access via `.items()`:

```
result = Q.nodes().compute("degree").execute(network)

# Iterate over all items
for node, data in result.items():
    print(f"{node}: degree={data['degree']}")

# Inspect one sample entry
sample_key, sample_value = next(iter(result.items()))
print(f"Sample key type: {type(sample_key)}")
print(f"Sample key: {sample_key}")
print(f"Sample value: {sample_value}")
```

Result structure for nodes:

- **Keys**: `(node_id, layer)` tuples (for multilayer queries) or `node_id` (for single-layer queries)
- **Values**: Dictionaries with computed attributes (`{"degree": 5, "betweenness_centrality": 0.23, ...}`)

Result structure for edges:

- **Keys**: `((source, source_layer), (target, target_layer), {edge_data})` tuples
- **Values**: Dictionaries with edge attributes and computed metrics

Expected output:

```
Sample key type: <class 'tuple'>
Sample key: ('alice', 'social')
Sample value: {'degree': 12, 'layer': 'social', 'layer_count': 2}
```

Convert to Pandas

This is the recommended way to integrate DSL queries with statistical analysis and plotting libraries.

```
result = (
    Q.nodes()
     .from_layers(L["social"])
     .compute("degree", "betweenness_centrality", "clustering")
     .execute(network)
)

# Convert to DataFrame
df = result.to_pandas()

# Inspect structure
print(df.head())
print("\nColumn names:", df.columns.tolist())
print("\nSummary statistics:")
print(df[['degree', 'betweenness_centrality', 'clustering']].describe())

# Use pandas for further analysis
high_influence = df[
    (df['degree'] > 10) &
    (df['betweenness_centrality'] > 0.2)
]
print(f"\nHigh-influence nodes: {len(high_influence)}")
```

DataFrame structure:

- **For node queries**: Columns include `id` (the node-layer tuple or node ID), plus all computed attributes
- **For edge queries**: Columns include `source`, `target`, `source_layer`, `target_layer`, `weight`, plus computed attributes

Expected output:

```
                  id  degree  betweenness_centrality  clustering
0   (alice, social)      12                0.245000    0.545455
1     (bob, social)       8                0.189000    0.642857
2     (eve, social)      15                0.301000    0.428571
3   (frank, social)       7                0.134000    0.666667
4   (grace, social)      11                0.221000    0.509091

Column names: ['id', 'degree', 'betweenness_centrality', 'clustering']

Summary statistics:
```

(continues on next page)

(continued from previous page)

```
              degree  betweenness_centrality  clustering
count      65.000000               65.000000   65.000000
mean        6.846154                0.112308    0.587692
std         3.241057                0.089542    0.145231
...

High-influence nodes: 8
```

Multi-index option:

For more complex analyses, you can reshape the `id` tuple into a multi-index:

```
df = result.to_pandas()
# Split 'id' tuple into separate columns
df[['node', 'layer']] = pd.DataFrame(df['id'].tolist(), index=df.index)
df = df.drop('id', axis=1)
df = df.set_index(['node', 'layer'])
print(df.head())
```

Filter Results

You can filter results in two ways: **using the DSL's** `where()` **clause** (recommended) or **post-processing** with Python/pandas.

Option 1: Filter in the query (recommended for large networks):

```
# Filter before computation for efficiency
result = (
    Q.nodes()
     .compute("degree", "betweenness_centrality")
     .where(degree__gt=5)
     .execute(network)
)
```

Option 2: Filter the result dictionary (for small networks or ad-hoc filtering):

```
result = Q.nodes().compute("degree").execute(network)

# Pure Python filtering
high_degree = {
    node: data
    for node, data in result.items()
    if data['degree'] > 5
}
print(f"High-degree nodes: {len(high_degree)}")
```

Option 3: Filter the DataFrame (most flexible for complex conditions):

```
df = result.to_pandas()

# Use pandas boolean indexing
```

(continues on next page)

(continued from previous page)

```
filtered = df[df['degree'] > 5]

# Complex conditions
interesting_nodes = df[
    (df['degree'] > 5) &
    (df['betweenness_centrality'] > df['betweenness_centrality'].mean())
]
```

Performance note: For very large networks (millions of nodes), filtering in the DSL query (Option 1) is most efficient because it avoids materializing unnecessary results. For smaller networks, pandas filtering (Option 3) is often more convenient.

Advanced Queries

This section showcases the DSL's power for sophisticated multilayer network analysis. These patterns are common in research and can be adapted to your specific needs.

Multiple Layer Selection

Use **layer algebra** to combine layers. The `L` object supports set operations:

```
from py3plex.dsl import Q, L

# Union: nodes/edges from EITHER layer
result = (
    Q.nodes()
     .from_layers(L["friends"] + L["work"])
     .compute("degree")
     .execute(network)
)

df = result.to_pandas()
print(f"Combined nodes from 'friends' and 'work': {len(df)}")
print(f"Average degree across both layers: {df['degree'].mean():.2f}")
```

Set semantics:

- `L["friends"] + L["work"]`: **Union** of nodes/edges from both layers (nodes appearing in either layer)
- `L["friends"] & L["work"]`: **Intersection** (see next section)
- `L["friends"] - L["work"]`: **Difference** (nodes in friends but not work)

Use case: Compare activity across related contexts. For example, analyze user behavior across social and professional networks together.

Expected output:

```
Combined nodes from 'friends' and 'work': 87
Average degree across both layers: 6.12
```

Layer Intersection

Find nodes that appear in **multiple specific layers**:

```
# Nodes present in BOTH 'friends' AND 'work' layers
result = (
    Q.nodes()
     .from_layers(L["friends"] & L["work"])
     .compute("degree", "betweenness_centrality")
     .execute(network)
)

df = result.to_pandas()
print(f"Nodes in both 'friends' and 'work': {len(df)}")
print("\nThese are 'connector' nodes bridging social and professional contexts
↪")
print(df.head(10))
```

Semantics:

- `L["friends"] & L["work"]` selects nodes that have representations in **both** layers
- This is different from `layer_count >= 2`, which selects nodes in **any** two layers
- Use intersection to find nodes bridging specific contexts

Alternative approach using `layer_count`:

```
# More general: nodes in at least 2 layers (any layers)
result = (
    Q.nodes()
     .where(layer_count__gte=2)
     .compute("degree")
     .execute(network)
)
print(f"Multilayer nodes (any 2+ layers): {len(result)}")
```

Expected output:

```
Nodes in both 'friends' and 'work': 23

These are 'connector' nodes bridging social and professional contexts
                   id  degree  betweenness_centrality
0    (alice, friends)      12                0.245000
1      (alice, work)       8                0.189000
2  (charlie, friends)      10                0.201000
3    (charlie, work)       7                0.145000
...
```

Query Edges

The DSL supports edge queries with the same flexibility as node queries:

```
# Select edges from a layer with weight filter
edges = (
    Q.edges()
     .from_layers(L["social"])
     .where(weight__gt=0.5)
     .compute("edge_betweenness")
     .execute(network)
)

df = edges.to_pandas()
print(f"High-weight edges in 'social' layer: {len(df)}")
print("\nSample edges:")
print(df.head())

# Analyze edge distribution
print(f"\nMean edge weight: {df['weight'].mean():.3f}")
print(f"Mean edge betweenness: {df['edge_betweenness'].mean():.3f}")
```

Edge result structure:

For edge queries, the DataFrame includes:

- `source`, `target`: node identifiers
- `source_layer`, `target_layer`: layer names (same for intralayer edges)
- `weight`: edge weight (default 1.0 if not specified)
- Computed attributes: `edge_betweenness`, etc.

Filter by edge type:

```
# Only intralayer edges (within a layer)
intralayer_edges = (
    Q.edges()
     .where(intralayer=True)
     .execute(network)
)
print(f"Intralayer edges: {len(intralayer_edges)}")

# Only interlayer edges between specific layers
interlayer_edges = (
    Q.edges()
     .where(interlayer=("social", "work"))
     .execute(network)
)
print(f"Edges between 'social' and 'work': {len(interlayer_edges)}")
```

Expected output:

```
High-weight edges in 'social' layer: 156

Sample edges:
    source  target source_layer target_layer  weight  edge_betweenness
```

(continues on next page)

(continued from previous page)

```
0    alice     bob       social       social     0.75          0.023400
1      bob   charlie     social       social     0.80          0.034500
2    alice   diana       social       social     0.92          0.019800
3    diana     eve       social       social     0.65          0.028900
4      eve   frank       social       social     0.88          0.041200

Mean edge weight: 0.723
Mean edge betweenness: 0.028
```

Smart Defaults and Error Messages

The DSL includes **smart defaults** that automatically compute commonly used centrality metrics when referenced but not explicitly computed. This feature makes queries more ergonomic while maintaining predictable behavior.

Auto-Computing Centrality Metrics

When you reference a centrality metric in operations like `top_k()`, `order_by()`, or other ranking operations, the DSL will automatically compute it if not already present:

```python
from py3plex.dsl import Q, L

# The DSL auto-computes betweenness_centrality when needed
result = (
    Q.nodes()
     .from_layers(L["*"])
     .per_layer()
        .top_k(5, "betweenness_centrality")  # Auto-computed here
     .end_grouping()
     .execute(network)
)

df = result.to_pandas()
# betweenness_centrality column is available even though
# we didn't explicitly call .compute("betweenness_centrality")
```

Supported centrality aliases:

- `degree`, `degree_centrality`
- `betweenness`, `betweenness_centrality`
- `closeness`, `closeness_centrality`
- `eigenvector`, `eigenvector_centrality`
- `pagerank`

When auto-compute happens:

- When the attribute is referenced in `top_k()`
- When the attribute is used in `order_by()`
- For both per-group (with grouping) and global operations

Example with multiple auto-computed metrics:

```
# Auto-compute degree for filtering and betweenness for ranking
result = (
    Q.nodes()
     .from_layers(L["social"])
     .where(degree__gt=2)  # degree auto-computed here
     .order_by("betweenness_centrality", desc=True)  # betweenness auto-
↪computed here
     .limit(10)
     .execute(network)
)
```

Expected output:

```
    node layer  degree  betweenness_centrality
0  alice social      8                0.143000
1    bob social      7                0.098000
2  carol social      6                0.067000
...
```

Controlling Autocompute Behavior

You can explicitly control whether metrics are automatically computed using the `autocompute` parameter:

```
# Disable autocompute - require explicit .compute() calls
result = (
    Q.nodes(autocompute=False)  # Autocompute disabled
     .from_layers(L["social"])
     .compute("degree")  # Must explicitly compute
     .where(degree__gt=5)
     .execute(network)
)

# This would raise DslMissingMetricError because betweenness is not computed:
# Q.nodes(autocompute=False).order_by("betweenness_centrality").execute(net)
```

When to disable autocompute:

- **Performance-critical code**: Avoid unexpected expensive computations
- **Explicit control**: Make all metric computations visible in code
- **Debugging**: Understand exactly which metrics are computed and when

Tracking computed metrics:

Query results include a `computed_metrics` attribute that tracks which metrics were computed during execution:

```
result = (
    Q.nodes()
     .from_layers(L["social"])
```

(continues on next page)

(continued from previous page)

```
        .compute("degree")
        .order_by("betweenness_centrality")  # Auto-computed
        .execute(network)
)

# Check which metrics were computed
print(f"Computed metrics: {result.computed_metrics}")
# Output: Computed metrics: {'degree', 'betweenness_centrality'}
```

Use cases for computed_metrics:

- Performance profiling: identify expensive operations
- Query optimization: avoid redundant computations
- Debugging: verify expected metrics were computed

Helpful Error Messages with Suggestions

When you reference an unknown attribute, the DSL provides **did you mean?** suggestions using fuzzy string matching:

```
# Typo in attribute name
try:
    result = (
        Q.nodes()
         .from_layers(L["*"])
         .per_layer()
            .top_k(5, "betweness_centrality")  # Typo: "betweness" instead of
↪"betweenness"
         .end_grouping()
         .execute(network)
    )
except UnknownAttributeError as e:
    print(e)
```

Output:

```
Unknown attribute 'betweness_centrality'. Did you mean 'betweenness_centrality
↪'?
Known attributes: betweenness, betweenness_centrality, closeness, closeness_
↪centrality,
                  degree, degree_centrality, eigenvector, eigenvector_
↪centrality, pagerank
```

The error includes:

- The incorrect attribute name
- A suggestion for the most similar correct name (using Levenshtein distance)
- A list of all available attributes

Grouping Requirements and Clear Errors

Some operations require **active grouping** (via `per_layer()` or `group_by()`). The DSL raises `GroupingError` with clear guidance when these operations are used incorrectly:

```
from py3plex.dsl.errors import GroupingError

# This will raise GroupingError
try:
    result = (
        Q.nodes()
         .from_layers(L["*"])
         .coverage(mode="all")  # Error: no grouping active
         .execute(network)
    )
except GroupingError as e:
    print(e)
```

Output:

```
coverage() requires an active grouping (e.g. per_layer(), group_by('layer')).
No grouping is currently active.
Example:
    Q.nodes().from_layers(L["*"])
        .per_layer().top_k(5, "degree").end_grouping()
        .coverage(mode="all")
```

Correct usage:

```
# With proper grouping
result = (
    Q.nodes()
     .from_layers(L["*"])
     .per_layer()  # Add grouping here
        .top_k(5, "degree")
     .end_grouping()
     .coverage(mode="all")  # Now works correctly
     .execute(network)
)
```

When Smart Defaults DON'T Apply

Smart defaults are **predictable and conservative**. They only apply in specific scenarios:

1. **Only for centrality metrics**: Smart defaults work for recognized centrality metrics (degree, betweenness, etc.), not arbitrary attributes.
2. **Explicit compute takes precedence**: If you explicitly compute a metric, the DSL uses your computation and doesn't auto-compute:

```
# Explicit compute - no auto-compute happens
result = (
    Q.nodes()
```

(continues on next page)

(continued from previous page)

```
        .from_layers(L["*"])
        .compute("betweenness_centrality")  # Explicit
        .per_layer()
            .top_k(5, "betweenness_centrality")  # Uses explicit computation
        .end_grouping()
        .execute(network)
)
```

3. **Edge attributes are not auto-computed**: For edge queries, attributes like `weight` are read from edge data, not auto-computed:

```
# Edge weight is read from edge data, not computed
result = (
    Q.edges()
        .from_layers(L["*"])
        .per_layer()
            .top_k(5, "weight")  # Uses edge data['weight']
        .end_grouping()
        .execute(network)
)
```

Benefits of Smart Defaults

Ergonomics: Write less boilerplate for common patterns:

```
# Before smart defaults (verbose)
result = (
    Q.nodes()
        .from_layers(L["*"])
        .compute("degree", "betweenness_centrality", "closeness_centrality")
        .per_layer()
            .top_k(5, "betweenness_centrality")
        .end_grouping()
        .execute(network)
)

# With smart defaults (concise)
result = (
    Q.nodes()
        .from_layers(L["*"])
        .per_layer()
            .top_k(5, "betweenness_centrality")  # Auto-computes what's needed
        .end_grouping()
        .execute(network)
)
```

Teaching errors: When something goes wrong, you get actionable guidance instead of cryptic messages.

Predictability: Smart defaults only activate for well-known patterns. Your explicit operations always take precedence.

Temporal Queries

The DSL supports temporal filtering for networks with time-stamped edges or nodes. Four convenience methods provide intuitive temporal filtering: `.at(t)`, `.during(t0, t1)`, `.before(t)`, and `.after(t)`.

Prerequisites for temporal queries:

- Edges or nodes must have temporal attributes:
  - **Point-in-time**: `t` attribute (e.g., `{"t": 150.0}`)
  - **Intervals**: `t_start` and `t_end` attributes (e.g., `{"t_start": 100.0, "t_end": 200.0}`)
- Time values are typically numeric (timestamps) or ISO date strings

Temporal Semantics Reference

The following table summarizes temporal query semantics:

Table 1: Temporal Query Operations

Method	Description	Interval Semantics	Inclusivity
`.at(t)`	Snapshot at time t	Entities active at exactly t	Point (closed)
`.during(t0, t1)`	Range from t0 to t1	Entities active during [t0, t1]	[t0, t1] (closed interval)
`.before(t)`	Before time t	Equivalent to `.during(None, t)`	(-∞, t] (closed at t)
`.after(t)`	After time t	Equivalent to `.during(t, None)`	[t, +∞) (closed at t)

Detailed semantics:

- `at(t)`: Selects entities active at a specific moment
  - For point-in-time edges: includes edges where `t_edge == t`
  - For interval edges: includes edges where `t` is in `[t_start, t_end]`
- `during(t0, t1)`: Selects entities active during a time window
  - For point-in-time edges: includes edges where `t` is in `[t0, t1]` (closed interval)
  - For interval edges: includes edges where the interval **overlaps** `[t0, t1]`
  - `None` values: Use `t0=None` for open lower bound, `t1=None` for open upper bound
- `before(t)`: Selects entities active before (and at) time t
  - Convenience method equivalent to `.during(None, t)`
  - Inclusive of the boundary: includes entities at exactly time t
- `after(t)`: Selects entities active after (and at) time t
  - Convenience method equivalent to `.during(t, None)`
  - Inclusive of the boundary: includes entities at exactly time t

Filter by Time (Snapshot)

Query the network state at a specific point in time:

```
# Nodes active at t=150.0
result = (
    Q.nodes()
     .at(150.0)
     .compute("degree")
     .execute(network)
)

df = result.to_pandas()
print(f"Nodes active at t=150: {len(df)}")
print(f"Average degree at t=150: {df['degree'].mean():.2f}")
print("\nTop nodes by degree at this snapshot:")
print(df.nlargest(5, 'degree')[['id', 'degree']])
```

Use case: Analyze network structure at specific moments (e.g., before and after an event, at regular intervals for time series).

Expected output:

```
Nodes active at t=150: 78
Average degree at t=150: 5.12

Top nodes by degree at this snapshot:
                   id  degree
12    (eve, social)      14
5   (alice, social)      11
23  (grace, social)      10
8     (bob, social)       9
31  (henry, work)        9
```

Time Range

Query entities active during a time window:

```
# Nodes active during January 2024 (assuming numeric timestamps)
# For ISO dates, use strings: .during("2024-01-01", "2024-01-31")
result = (
    Q.nodes()
     .during(100.0, 200.0)
     .compute("degree")
     .execute(network)
)

df = result.to_pandas()
print(f"Nodes active during [100, 200]: {len(df)}")

# Compare to snapshot
snapshot_result = Q.nodes().at(150.0).execute(network)
```

(continues on next page)

(continued from previous page)

```
print(f"Nodes at t=150 (snapshot): {len(snapshot_result)}")
print(f"Nodes during [100, 200] (range): {len(df)}")
print(f"Ratio: {len(df) / len(snapshot_result):.2f}x more nodes in range")
```

Open-ended ranges:

```
# From t=100 onwards (no upper limit)
result_after = Q.edges().during(100.0, None).execute(network)

# Up to t=200 (no lower limit)
result_before = Q.edges().during(None, 200.0).execute(network)
```

Use case: Study network evolution, identify persistent vs. transient connections, analyze activity bursts.

Expected output:

```
Nodes active during [100, 200]: 142
Nodes at t=150 (snapshot): 78
Ratio: 1.82x more nodes in range
```

Before and After (Convenience Methods)

The `.before()` and `.after()` methods provide intuitive alternatives for open-ended temporal queries:

```
# Get all edges before time 100 (inclusive)
early_edges = Q.edges().before(100.0).execute(network)

# Get all edges after time 200 (inclusive)
late_edges = Q.edges().after(200.0).execute(network)

# Common pattern: compare network before and after an event
event_time = 150.0

before_event = (
    Q.nodes()
     .before(event_time)
     .compute("degree", "betweenness_centrality")
     .execute(network)
)

after_event = (
    Q.nodes()
     .after(event_time)
     .compute("degree", "betweenness_centrality")
     .execute(network)
)

# Compare metrics
```

(continues on next page)

(continued from previous page)

```
df_before = before_event.to_pandas()
df_after = after_event.to_pandas()

print(f"Average degree before event: {df_before['degree'].mean():.2f}")
print(f"Average degree after event: {df_after['degree'].mean():.2f}")
print(f"Network became {'denser' if df_after['degree'].mean() > df_before[
↪'degree'].mean() else 'sparser'}")
```

Expected output:

```
Average degree before event: 4.35
Average degree after event: 5.87
Network became denser
```

Temporal edges example:

```
# Edges active during a period
edges = (
    Q.edges()
     .during(100.0, 200.0)
     .compute("edge_betweenness")
     .execute(network)
)

df = edges.to_pandas()
print(f"Active edges during [100, 200]: {len(df)}")
print(f"Mean edge betweenness: {df['edge_betweenness'].mean():.4f}")
```

Note on implementation status:

Temporal queries are **fully implemented** for edge-level temporal data. Node-level temporal filtering depends on your network's representation:

- If nodes have explicit `t` attributes, `.at()`, `.during()`, `.before()`, and `.after()` work directly
- If only edges are timestamped, node activity is inferred from edge presence
- For most use cases, temporal edge queries are sufficient

See *DSL Reference* for complete temporal query syntax and examples with ISO date strings.

Common Patterns

This section presents **end-to-end recipes** for common multilayer network analysis tasks. These patterns are production-ready and can be adapted to your research questions.

Pattern: Find Influential Nodes

Identify nodes that are both well-connected (high degree) and structurally important (high betweenness centrality):

```
# High-degree nodes ranked by betweenness centrality
result = (
```

(continues on next page)

(continued from previous page)

```
    Q.nodes()
     .compute("degree", "betweenness_centrality", "layer_count")
     .where(degree__gt=10)
     .order_by("-betweenness_centrality")
     .limit(20)
     .execute(network)
)

df = result.to_pandas()
print(f"Top 20 influential nodes (degree > 10):")
print(df[['id', 'degree', 'betweenness_centrality', 'layer_count']])

# Export for further analysis or publication
df.to_csv("influential_nodes.csv", index=False)

# Visualize
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 6))
plt.scatter(df['degree'], df['betweenness_centrality'],
            s=df['layer_count']*50, alpha=0.6)
plt.xlabel("Degree")
plt.ylabel("Betweenness Centrality")
plt.title("Influential Nodes (size = layer_count)")
plt.tight_layout()
plt.savefig("influential_nodes.png", dpi=300)
```

Why this pattern works:

- **Degree** measures local connectivity (how many neighbors)
- **Betweenness centrality** measures global importance (how often the node appears on shortest paths)
- Nodes high in both metrics are **influential bridges** in the network

Expected output:

```
Top 20 influential nodes (degree > 10):
                   id  degree  betweenness_centrality  layer_count
0     (eve, social)      15                0.301000            3
1   (alice, social)      12                0.245000            2
2   (grace, social)      11                0.221000            2
3     (bob, social)      12                0.198000            1
4   (diana, work)        14                0.187000            3
...
```

Pattern: Compare Layer Activity

Compute summary statistics for each layer to understand layer-specific dynamics:

```
layers = network.get_layers()
```

(continues on next page)

(continued from previous page)

```
layer_stats = []
for layer in layers:
    result = (
        Q.nodes()
         .from_layers(L[layer])
         .compute("degree", "clustering")
         .execute(network)
    )
    df = result.to_pandas()

    layer_stats.append({
        'layer': layer,
        'num_nodes': len(df),
        'mean_degree': df['degree'].mean(),
        'max_degree': df['degree'].max(),
        'mean_clustering': df['clustering'].mean(),
    })

    print(f"{layer}: {len(df)} nodes, "
          f"avg degree={df['degree'].mean():.2f}, "
          f"avg clustering={df['clustering'].mean():.3f}")

# Create comparison DataFrame
import pandas as pd
comparison = pd.DataFrame(layer_stats)
print("\nLayer comparison:")
print(comparison)

# Visualize
comparison.plot(x='layer', y=['mean_degree', 'mean_clustering'],
                kind='bar', figsize=(10, 5))
plt.ylabel("Value")
plt.title("Layer Activity Comparison")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig("layer_comparison.png", dpi=300)
```

Use case: Understand how network structure varies across different contexts (e.g., online vs. offline interactions, different communication channels).

Expected output:

```
friends: 50 nodes, avg degree=4.20, avg clustering=0.623
work: 72 nodes, avg degree=3.15, avg clustering=0.512
social: 65 nodes, avg degree=5.01, avg clustering=0.587
family: 38 nodes, avg degree=6.84, avg clustering=0.701

Layer comparison:
      layer  num_nodes  mean_degree  max_degree  mean_clustering
0   friends         50         4.20          15            0.623
```

(continues on next page)

(continued from previous page)

```
1      work         72          3.15          12            0.512
2     social        65          5.01          18            0.587
3     family        38          6.84          21            0.701
```

Pattern: Export Subnetwork

Extract a subnetwork based on query criteria for focused analysis or visualization:

```python
# Extract high-activity multilayer nodes
active_nodes = (
    Q.nodes()
     .where(layer_count__gt=2)
     .compute("degree", "betweenness_centrality")
     .execute(network)
)

print(f"Selected {len(active_nodes)} multilayer nodes")

# Create subnetwork containing only these nodes
subnetwork = network.subgraph(active_nodes.keys())

print(f"Subnetwork: {subnetwork.number_of_nodes()} nodes, "
      f"{subnetwork.number_of_edges()} edges")

# Analyze subnetwork
df = active_nodes.to_pandas()
print(f"\nSubnetwork mean degree: {df['degree'].mean():.2f}")
print(f"Subnetwork mean betweenness: {df['betweenness_centrality'].mean():.4f}
↪")

# Export for visualization or further analysis
from py3plex.visualization import draw_multilayer_default
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(12, 10))
draw_multilayer_default(subnetwork, ax=ax, display=False)
plt.savefig("subnetwork_viz.png", dpi=300)
plt.close()

# Or export in various formats
subnetwork.save_network("subnetwork.edgelist", output_type="edgelist")
```

What `layer_count__gt=2` means:

- Selects nodes appearing in **more than 2 layers**
- These are “connector” nodes that participate in multiple contexts
- Useful criterion for studying nodes that bridge different social spheres

Alternative criteria:

```
# High betweenness nodes
influential = Q.nodes().compute("betweenness_centrality").where(
    betweenness_centrality__gt=0.1
).execute(network)

# Nodes in specific community
community_nodes = Q.nodes().compute("communities").where(
    communities__eq=3
).execute(network)
```

Expected output:

```
Selected 34 multilayer nodes
Subnetwork: 34 nodes, 127 edges

Subnetwork mean degree: 7.47
Subnetwork mean betweenness: 0.0892
```

Workflow integration:

This pattern is often combined with community detection, dynamics simulation, or centrality analysis:

```
# 1. Select subnetwork
core_nodes = Q.nodes().where(layer_count__gte=2, degree__gt=5).
↪execute(network)
subnetwork = network.subgraph(core_nodes.keys())

# 2. Run community detection on subnetwork
from py3plex.algorithms.community_detection.community_wrapper import louvain_
↪communities
communities = louvain_communities(subnetwork)

# 3. Analyze communities
print(f"Found {len(set(communities.values()))} communities")

# 4. Visualize or export
from py3plex.visualization import draw_multilayer_default
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(12, 10))
# Note: communities dict can be used for node coloring if the visualization␣
↪function supports it
draw_multilayer_default(subnetwork, ax=ax, display=False)
plt.savefig("core_network_communities.png", dpi=300)
plt.close()
```

Pattern: Per-Layer Top-K with Coverage (Multi-Layer Hub Detection)

Find nodes that are top-k hubs (by any centrality metric) **across all layers**. This pattern is essential for identifying nodes that maintain high influence in the entire multilayer structure, not just in isolated layers.

The Problem:

Traditional approaches require manual loops over layers:

```
# Old approach: manual iteration
layer_top = {}
for layer in network.layers:
    res = (
        Q.nodes()
         .from_layers(L[str(layer)])
         .where(degree__gt=1)
         .compute("betweenness_centrality")
         .order_by("-betweenness_centrality")
         .limit(5)
         .execute(network)
    )
    layer_top[layer] = set(res.to_pandas()["id"])

# Find intersection
multi_hubs = set.intersection(*layer_top.values())
```

The Solution: Grouping and Coverage API

The new DSL supports per-layer operations in a single query:

```
from py3plex.core import random_generators
from py3plex.dsl import Q, L

# Generate example network
net = random_generators.random_multilayer_ER(n=200, l=3, p=0.05,␣
↪directed=False)

# Find nodes that are top-5 betweenness hubs in ALL layers (single query!)
multi_hubs = (
    Q.nodes()
     .from_layers(L["*"])                     # wildcard: all layers
     .where(degree__gt=1)
     .compute("degree", "betweenness_centrality")
     .per_layer()                             # group by layer
        .top_k(5, "betweenness_centrality") # top 5 per layer
     .end_grouping()
     .coverage(mode="all")                    # nodes in top-5 in ALL layers
     .execute(net)
)

df = multi_hubs.to_pandas()
print(f"Multi-layer hubs (in top-5 of ALL layers): {set(df['id'])}")
```

(continues on next page)

(continued from previous page)

```
print(f"Count: {len(df['id'].unique())}")
print(f"\nDetailed results:")
print(df[['id', 'layer', 'degree', 'betweenness_centrality']].to_string())
```

Expected output:

```
Multi-layer hubs (in top-5 of ALL layers): {23, 45, 67}
Count: 3

Detailed results:
    id  layer  degree  betweenness_centrality
0   23      0      12                  0.2456
1   23      1      14                  0.2891
2   23      2      11                  0.2234
3   45      0      13                  0.2567
4   45      1      11                  0.2123
5   45      2      15                  0.3012
6   67      0      10                  0.2001
7   67      1      12                  0.2345
8   67      2      13                  0.2678
```

Coverage Modes:

The coverage() method supports multiple modes for cross-layer analysis:

```
# Mode 1: "all" - intersection (nodes in top-k of ALL layers)
all_layers_hubs = (
    Q.nodes()
     .from_layers(L["*"])
     .compute("degree")
     .per_layer()
        .top_k(10, "degree")
     .end_grouping()
     .coverage(mode="all")
     .execute(net)
)

# Mode 2: "any" - union (nodes in top-k of AT LEAST ONE layer)
any_layer_hubs = (
    Q.nodes()
     .from_layers(L["*"])
     .compute("degree")
     .per_layer()
        .top_k(10, "degree")
     .end_grouping()
     .coverage(mode="any")
     .execute(net)
)

# Mode 3: "at_least" - nodes in top-k of at least K layers
```

(continues on next page)

(continued from previous page)

```
two_layer_hubs = (
    Q.nodes()
     .from_layers(L["*"])
     .compute("betweenness_centrality")
     .per_layer()
        .top_k(5, "betweenness_centrality")
     .end_grouping()
     .coverage(mode="at_least", k=2)  # In at least 2 layers
     .execute(net)
)

# Mode 4: "exact" - nodes in top-k of exactly K layers (layer-specific hubs)
single_layer_specialists = (
    Q.nodes()
     .from_layers(L["*"])
     .compute("degree")
     .per_layer()
        .top_k(10, "degree")
     .end_grouping()
     .coverage(mode="exact", k=1)  # Exactly 1 layer
     .execute(net)
)

print(f"Hubs in ALL layers: {len(all_layers_hubs.to_pandas()['id'].unique())}
↪")
print(f"Hubs in ANY layer: {len(any_layer_hubs.to_pandas()['id'].unique())}")
print(f"Hubs in 2 layers: {len(two_layer_hubs.to_pandas()['id'].unique())}")
print(f"Layer specialists (exactly 1): {len(single_layer_specialists.to_
↪pandas()['id'].unique())}")
```

Expected output:

```
Hubs in ALL layers: 3
Hubs in ANY layer: 27
Hubs in 2 layers: 12
Layer specialists (exactly 1): 15
```

Wildcard Layer Selection:

The `L["*"]` wildcard automatically expands to all layers in the network:

```
# All layers
Q.nodes().from_layers(L["*"])

# All layers except one
Q.nodes().from_layers(L["*"] - L["bots"])

# All layers intersected with a specific one (same as selecting that layer)
Q.nodes().from_layers(L["*"] & L["social"])
```

Use Cases:

1. **Identify persistent influencers**: Nodes that maintain high centrality across all contexts (layers)
2. **Find layer specialists**: Nodes that are important in only one layer (`mode="exact", k=1`)
3. **Detect multi-context bridges**: Nodes in top-k in at least 2 layers connect different contexts
4. **Community structure analysis**: Compare `mode="all"` vs `mode="any"` to understand layer cohesion

Why This Pattern Matters:

In real-world multilayer networks (social media, collaboration networks, biological systems), understanding **cross-layer** vs. **layer-specific** importance is crucial:

- **Email + Phone + Chat network**: Who are the omnipresent communicators vs. email-only specialists?
- **Author collaboration network**: Who publishes top papers in multiple fields vs. specialists in one domain?
- **Transportation network**: Which locations are hubs in all modes (bus, train, bike) vs. single-mode hubs?

Performance Note:

The per-layer computation is optimized: measures are computed on the selected nodes after layer filtering, and grouping operations leverage efficient dictionaries. For large networks (>100K nodes), consider filtering with `where()` before computing expensive metrics like betweenness centrality.

DSL Result Interoperability

DSL query results integrate seamlessly with pandas for data transformation workflows. While QueryResult doesn't implement pipeline verbs directly, it provides a clean `.to_pandas()` export that enables the same workflow patterns:

1. Start with a DSL query to filter and compute metrics
2. Export to pandas with `.to_pandas()`
3. Use pandas operations for additional transformations
4. Leverage the full pandas ecosystem for analysis and visualization

QueryResult to pandas Workflow

The recommended pattern for combining DSL queries with data transformations:

```
from py3plex.dsl import Q, L
from py3plex.core import multinet

# Create a sample network
network = multinet.multi_layer_network()
network.add_edges([
    ['A', 'layer1', 'B', 'layer1', 1],
    ['B', 'layer1', 'C', 'layer1', 1],
    ['A', 'layer2', 'C', 'layer2', 1],
], input_type="list")
```

(continues on next page)

(continued from previous page)

```python
# Start with DSL query
result = (
    Q.nodes()
     .from_layers(L["*"])
     .compute("degree", "betweenness_centrality")
     .execute(network)
)

# Export to pandas for flexible transformations
df = result.to_pandas()

# Continue with pandas operations
df = df[df["degree"] > 5]  # Filter
df['influence_score'] = (  # Mutate
    df["degree"] * df["betweenness_centrality"]
)
df = df.sort_values('influence_score', ascending=False)  # Arrange

print(df.head(10))
```

What happens here:

1. **DSL phase**: `Q.nodes()...execute()` filters nodes, computes centrality metrics
2. **Export**: `.to_pandas()` materializes as a DataFrame
3. **pandas phase**: Standard pandas operations for transformation and analysis

Pandas operations equivalent to pipeline verbs:

- Filter rows: `df[df["degree"] > 5]`
- Add columns: `df['new_col'] = ...`
- Sort: `df.sort_values('col', ascending=False)`
- Select columns: `df[['col1', 'col2']]`
- Group and aggregate: `df.groupby('col').agg(...)`

Verb Mapping Table

The following table shows how concepts map across the three interfaces:

Table 2: DSL and pandas Verb Mapping

Concept	String DSL	Builder DSL	pandas
Filter rows	`WHERE degree > 5`	`.where(degree__gt=5)`	`df[df["degree"] > 5]`
Select columns	`SELECT id, degree`	`.select("id", "degree")`	`df[["id", "degree"]]`
Sort/Order	`ORDER BY degree DESC`	`.order_by("degree", desc=True)`	`df.sort_values("degree", ascending=False)`
Group by field	`GROUP BY layer`	`.group_by("layer")`	`df.groupby("layer")`
Add column	(not available)	(use pandas after export)	`df["score"] = ...`
Aggregate	(not available)	(use pandas after export)	`df.groupby("layer").agg(...)`
Limit results	`LIMIT 10`	`.limit(10)`	`df.head(10)`

Design rationale:

- **DSL**: Declarative, optimized for graph queries (layer algebra, centrality, grouping)
- **pandas**: Procedural, flexible for data transformations (arbitrary computations, reshaping)
- **Workflow**: Use DSL for graph-specific operations, export to pandas for data munging

Example: Combined Workflow

A realistic workflow combining both DSL and pandas operations:

```
from py3plex.dsl import Q, L

# Scenario: Find influential nodes in social network, normalize scores,
#           rank within communities, export for visualization

# DSL: Query and compute graph metrics
result = (
    Q.nodes()
     .from_layers(L["social"])
     .where(degree__gt=3)
     .compute("degree", "betweenness_centrality", "clustering")
     .execute(network)
)

# Export to pandas for transformations
df = result.to_pandas()

# pandas: Transform and enhance data
max_betweenness = df['betweenness_centrality'].max()

# Normalize centrality to [0, 1]
df['norm_betweenness'] = (
```

(continues on next page)

(continued from previous page)

```
    df['betweenness_centrality'] / max_betweenness
    if max_betweenness > 0 else 0
)

# Composite influence score
df['influence'] = (
    0.5 * df['degree'] +
    0.3 * df['norm_betweenness'] +
    0.2 * (1 - df['clustering'])
)

# Group by community and compute statistics
community_stats = df.groupby('community').agg({
    'influence': ['count', 'mean', 'max']
}).round(2)

# Sort communities by average influence
community_stats = community_stats.sort_values(
    ('influence', 'mean'),
    ascending=False
)

print(community_stats)
```

Expected output:

```
              influence
                  count  mean    max
community
5                   23  0.72   0.89
2                   31  0.68   0.85
8                   19  0.61   0.79
1                   28  0.58   0.74
...
```

Why this matters:

1. **Single pipeline**: No need to export intermediate results to disk or juggle multiple DataFrames
2. **Flexibility**: DSL for graph operations, pandas for everything else
3. **Performance**: DSL computes centrality on the multilayer graph once, pandas transforms in-memory
4. **Ecosystem**: Full pandas ecosystem available (plotting, statistics, export formats)

When to use each:

- **DSL alone**: Simple queries, need graph-specific operations (centrality, grouping, coverage)
- **pandas alone**: Non-graph data, pure data transformations
- **Combined (DSL → pandas)**: Complex analytical workflows, need both graph metrics and custom computations

See *How to Build Analysis Pipelines with Dplyr-style Operations* for the dplyr-style pipeline API (`nodes()`, `edges()` functions) which provides an alternative approach using chainable operations directly on networks.

Next Steps

Now that you understand the DSL, explore these related resources:

- **DSL Reference** (*DSL Reference*): Complete grammar, all operators, full list of built-in measures, and advanced features (EXPLAIN queries, parameter binding, custom operators)
- **Dplyr-Style Pipelines** (*How to Build Analysis Pipelines with Dplyr-style Operations*): Combine DSL queries with pipeline operations for more complex data transformation workflows. The pipeline API (`nodes()`, `mutate()`, `arrange()`) complements the DSL for when you need procedural transformations.
- **Community Detection** (*How to Run Community Detection on Multilayer Networks*): Use DSL queries to select nodes, then apply community detection algorithms. Pattern: query → detect communities → analyze community structure.
- **Network Dynamics** (*How to Simulate Multilayer Dynamics*): Run dynamics simulations on DSL-selected subnetworks. Pattern: query → extract subnetwork → simulate → analyze outcomes.
- **Linting and Validation** (*DSL Reference*): The DSL includes a linting subsystem (`py3plex dsl-lint`) that checks queries for errors, performance issues, and suggests optimizations. Use it to validate complex queries.
- **Examples Repository** (*Examples & Recipes*): Full scripts showing DSL in context, including data loading, query composition, analysis, and visualization.

Key Takeaways:

1. **Use the builder API** (`Q`, `L`) for production code—it's type-safe, refactorable, and IDE-friendly.
2. **Filter early**: Add `where()` clauses before `compute()` for better performance on large networks.
3. **Embrace pandas**: Use `.to_pandas()` for result analysis—it integrates seamlessly with the scientific Python stack.
4. **Layer algebra is powerful**: `L["a"] + L["b"]` (union), `L["a"] & L["b"]` (intersection) enable sophisticated multilayer queries.
5. **Temporal queries** require timestamped edges/nodes but unlock time-series network analysis.

- **Community and Support:**
- Report issues or request features: https://github.com/SkBlaz/py3plex/issues
- Example notebooks: https://github.com/SkBlaz/py3plex/tree/main/examples
- py3plex documentation: https://skblaz.github.io/py3plex/

Further Reading: The Py3plex Book

For a deeper theoretical and practical treatment of the DSL and multilayer network concepts, see the **Py3plex Book**:

- **Chapter 8** — Introduction to the DSL: Motivations, design principles, and comparison with alternatives
- **Chapter 9** — Builder API Deep Dive: Complete reference with advanced patterns
- **Chapter 10** — Advanced Queries & Workflows: Complex real-world query examples

The book is available as: * **PDF** in the repository: `docs/py3plex_book.pdf` * **Online HTML** (if built): `docs/book/`

The book provides: * Formal definitions of multilayer network operations * Detailed algorithmic complexity analysis * Extensive case studies with real datasets * Performance benchmarking and optimization strategies

3.3.11 How to Find Graph Patterns and Motifs with the Pattern Matching API

Goal: Use py3plex's Pattern Matching Builder API to find graph motifs, paths, triangles, and complex subgraph patterns in multilayer networks.

What You'll Learn

- How to express graph patterns using the `Q.pattern()` builder API
- How to match edges, paths, triangles, and custom motifs
- How to apply multilayer-aware constraints (within/between layers)
- How to filter patterns using node and edge predicates
- How to extract and analyze pattern match results

Complete Example

See the full executable example: `example_pattern_matching.py`

Prerequisites:

- A loaded `multi_layer_network` object (see *How to Load and Build Networks*)
- Basic familiarity with the DSL (see *How to Query Multilayer Graphs with the SQL-like DSL*)
- Understanding of graph motifs (edges, paths, triangles, subgraphs)

Why Pattern Matching?

Pattern matching allows you to find **specific subgraph structures** in your network. Unlike node/edge queries that return individual elements, pattern matching returns **complete matches** where multiple nodes and edges satisfy structural constraints.

Common Use Cases:

- **Motif Detection:** Find triangles, cliques, or other structural patterns
- **Path Analysis:** Discover multi-hop connections between nodes

- **Multilayer Patterns:** Identify cross-layer relationships
- **Complex Queries:** Express "find nodes A, B, C where A connects to B with weight > 0.5, B connects to C, and all are in the social layer"

Comparison to Basic Queries:

Quick Start: Your First Pattern

Let's start with a simple example: finding all edges in a network.

```
from py3plex.core import multinet
from py3plex.dsl import Q

# Create a simple network
network = multinet.multi_layer_network(directed=False)
network.add_nodes([
    {'source': 'Alice', 'type': 'social'},
    {'source': 'Bob', 'type': 'social'},
    {'source': 'Charlie', 'type': 'social'},
])
network.add_edges([
    {'source': 'Alice', 'target': 'Bob',
     'source_type': 'social', 'target_type': 'social', 'weight': 1.0},
    {'source': 'Bob', 'target': 'Charlie',
     'source_type': 'social', 'target_type': 'social', 'weight': 2.0},
])

# Find all edges (simple pattern)
pattern = (
    Q.pattern()
     .node("a")                # Define node variable "a"
     .node("b")                # Define node variable "b"
     .edge("a", "b")           # Require edge between a and b
)

result = pattern.execute(network)
print(f"Found {result.count} matches")
print(result.to_pandas())
```

Output:

```
Found 4 matches
                 a                  b
0  (Alice, social)      (Bob, social)
1    (Bob, social)    (Alice, social)
2    (Bob, social)  (Charlie, social)
3  (Charlie, social)    (Bob, social)
```

Notice that undirected edges produce matches in both directions.

Pattern Components

Nodes

Define node variables that will be bound to actual nodes during matching:

```
# Simple node
Q.pattern().node("a")

# Node with semantic labels (metadata only)
Q.pattern().node("a", labels="person")

# Node with predicates
Q.pattern().node("a").where(degree__gt=3)

# Node with layer constraint
Q.pattern().node("a").where(layer="social")
```

Node Predicates:

All standard DSL predicates work with pattern nodes:

- `degree__gt=5` — degree greater than 5
- `degree__lt=2` — degree less than 2
- `layer="social"` — node must be in social layer
- Any node attribute: `age__gt=30`, `type="user"`, etc.

Edges

Define edges between node variables:

```
# Undirected edge
Q.pattern().edge("a", "b", directed=False)

# Directed edge
Q.pattern().edge("a", "b", directed=True)

# Edge with predicates
Q.pattern().edge("a", "b").where(weight__gt=0.5)

# Edge with optional type
Q.pattern().edge("a", "b", etype="friendship")
```

Edge Predicates:

Filter edges by attributes:

- `weight__gt=0.5` — edge weight greater than 0.5
- `weight__lt=1.0` — edge weight less than 1.0
- Any edge attribute: `timestamp__gt=100`, `color="red"`, etc.

Paths

Sugar method for defining sequential connections:

```
# 2-hop path: a → b → c
Q.pattern().path(["a", "b", "c"])

# Equivalent to:
Q.pattern().edge("a", "b").edge("b", "c")

# Directed path
Q.pattern().path(["a", "b", "c"], directed=True)
```

Triangles

Sugar method for triangle motifs:

```
# Triangle: a-b, b-c, c-a
Q.pattern().triangle("a", "b", "c")

# Equivalent to:
Q.pattern().edge("a", "b").edge("b", "c").edge("c", "a")
```

Multilayer-Aware Patterns

One of the most powerful features is specifying layer constraints on edges.

Within Layer

Find edges that exist within a single layer:

```
# Edges within the social layer
pattern = (
    Q.pattern()
     .node("a").where(layer="social")
     .node("b").where(layer="social")
     .edge("a", "b")
)
```

Between Layers

Find edges that cross between specific layers:

```
# Edges between social and work layers
pattern = (
    Q.pattern()
     .node("a").where(layer="social")
     .node("b").where(layer="work")
     .edge("a", "b").between_layers("social", "work")
)
```

Any Layer

Allow edges in any layer (default behavior):

```
pattern = Q.pattern().edge("a", "b").any_layer()
```

Practical Examples

Example 1: Find Triangles with Weight Constraints

Problem: Find triangles where all edges have weight > 1.0.

```
pattern = (
    Q.pattern()
     .triangle("a", "b", "c")
     .limit(10)
)

result = pattern.execute(network)

# Filter by weight in post-processing
# (Edge predicates during matching coming in future version)
triangles = result.to_pandas()
print(f"Found {len(triangles)} triangles")
```

Example 2: Find 2-Hop Paths in Specific Layer

Problem: Find all 2-hop paths within the social layer.

```
pattern = (
    Q.pattern()
     .node("a").where(layer="social")
     .node("b").where(layer="social")
     .node("c").where(layer="social")
     .path(["a", "b", "c"])
     .returning("a", "b", "c")
     .limit(5)
)

result = pattern.execute(network)
df = result.to_pandas()

print(f"Found {result.count} paths")
print(df)
```

Output:

```
Found 5 paths
                   a                 b                   c
0      (Bob, social)  (Alice, social)      (Bob, social)
1      (Bob, social)  (Alice, social)  (Charlie, social)
```

(continues on next page)

(continued from previous page)

```
2  (Charlie, social)    (Bob, social)    (Alice, social)
...
```

Example 3: Find High-Degree Nodes with Specific Connections

Problem: Find pairs of high-degree nodes (degree > 2) connected by strong edges (weight > 1.5).

```
pattern = (
    Q.pattern()
     .node("a").where(layer="social", degree__gt=2)
     .node("b").where(layer="social", degree__gt=2)
     .edge("a", "b").where(weight__gt=1.5)
     .constraint("a != b")  # Ensure different nodes
     .returning("a", "b")
)

result = pattern.execute(network)
print(f"Found {result.count} high-degree pairs")

# Get unique nodes involved
nodes = result.to_nodes(unique=True)
print(f"Unique nodes: {len(nodes)}")
```

Example 4: Cross-Layer Hub Detection

Problem: Find nodes that appear in multiple layers and have high degree in each.

```
# This requires multiple pattern queries
# Find nodes in social layer
social_hubs = (
    Q.pattern()
     .node("a").where(layer="social", degree__gt=3)
     .returning("a")
     .execute(network)
)

# Find nodes in work layer
work_hubs = (
    Q.pattern()
     .node("a").where(layer="work", degree__gt=3)
     .returning("a")
     .execute(network)
)

# Find intersection (nodes in both)
social_nodes = set(n[0] for n in social_hubs.to_nodes())
work_nodes = set(n[0] for n in work_hubs.to_nodes())
cross_layer_hubs = social_nodes & work_nodes
```

(continues on next page)

(continued from previous page)

```
print(f"Cross-layer hubs: {cross_layer_hubs}")
```

Working with Results

The `PatternQueryResult` object provides multiple ways to access matches.

Accessing Rows

```
result = pattern.execute(network)

# Get all matches as list of dictionaries
for match in result.rows:
    print(f"Match: {match}")
    # e.g., {'a': ('Alice', 'social'), 'b': ('Bob', 'social')}
```

Converting to Pandas

```
df = result.to_pandas()

# Now use pandas operations
print(df.head())
print(df.describe())

# Export to CSV
df.to_csv("pattern_matches.csv", index=False)
```

Extracting Nodes

```
# Get all matched nodes (as tuples)
all_nodes = result.to_nodes()

# Get unique nodes only
unique_nodes = result.to_nodes(unique=True)

# Get nodes for specific variables
a_nodes = result.to_nodes(vars=["a"], unique=True)
```

Extracting Edges

```
# Get all matched edges as tuples
edges = result.to_edges()

# Each edge is a tuple of node tuples
for src, dst in edges:
    print(f"Edge: {src} -> {dst}")
```

Creating Subgraphs

```
# Create induced subgraph of matched nodes
subgraph = result.to_subgraph(network)

# Now use NetworkX operations
print(f"Subgraph has {subgraph.number_of_nodes()} nodes")
print(f"Subgraph has {subgraph.number_of_edges()} edges")

# Create one subgraph per match
subgraphs = result.to_subgraph(network, per_match=True)
print(f"Created {len(subgraphs)} subgraphs")
```

Filtering Results

```
# Filter matches using a predicate
filtered = result.filter(lambda match: match["a"][0].startswith("A"))

# Limit number of results
limited = result.limit(10)
```

Query Execution Planning

Use `.explain()` to see how the pattern will be executed:

```
pattern = (
    Q.pattern()
     .node("a").where(degree__gt=3)
     .node("b")
     .edge("a", "b")
)

plan = pattern.explain()
print(f"Root variable: {plan['root_var']}")
print(f"Join order: {[step['var'] for step in plan['join_order']]}")
print(f"Estimated complexity: {plan['estimated_complexity']}")
```

Example Output:

```
Root variable: a
Join order: ['a', 'b']
Estimated complexity: 1000
```

The planner chooses `a` as the root because it has the most restrictive predicates (`degree__gt=3`), which will generate fewer candidates.

Performance Tips

1. **Add Predicates Early:** The more selective predicates you add to the root variable, the faster the query.

```
# Good: Restrictive predicate on first node
Q.pattern().node("a").where(degree__gt=10).node("b").edge("a", "b")

# Less efficient: No predicates
Q.pattern().node("a").node("b").edge("a", "b")
```

2. **Use Layer Constraints:** Layer filters significantly reduce the search space.

```
# Good: Restrict to one layer
Q.pattern().node("a").where(layer="social")

# Less efficient: Search all layers
Q.pattern().node("a")
```

3. **Set Limits:** For large networks, use `.limit()` to cap results.

```
pattern.limit(100).execute(network)
```

4. **Use Constraints:** Add `a != b` constraints to avoid self-loops if not needed.

```
Q.pattern().node("a").node("b").edge("a", "b").constraint("a != b")
```

Advanced Features

Constraints

Global constraints that apply across variables:

```
# All-different constraint
pattern = (
    Q.pattern()
     .node("a")
     .node("b")
     .node("c")
     .edge("a", "b")
     .edge("b", "c")
     .constraint("a != b")
     .constraint("b != c")
     .constraint("a != c")
)
```

Returning Specific Variables

By default, all variables are returned. You can select specific ones:

```
pattern = (
    Q.pattern()
```

(continues on next page)

(continued from previous page)

```
    .node("a")
    .node("b")
    .node("c")
    .path(["a", "b", "c"])
    .returning("a", "c")  # Only return endpoints
)

result = pattern.execute(network)
# Result only contains 'a' and 'c' columns
```

Execution Options

```
# Set maximum matches
result = pattern.execute(network, max_matches=100)

# Set timeout (in seconds)
result = pattern.execute(network, timeout=10.0)
```

Comparison with Other Approaches

Pattern Matching vs. NetworkX

NetworkX Approach	Pattern Matching Approach
`nx.triangles(G)` Returns triangle count per node	`Q.pattern().triangle("a","b","c")` Returns actual triangle matches
`nx.all_simple_paths(G, source, tgt)` Enumerates all paths	`Q.pattern().path(["a", "b", "c"])` Finds paths with constraints
Manual loops and filtering	Declarative pattern specification

Advantages of Pattern Matching:

- Declarative and composable
- Built-in support for multilayer constraints
- Integrated with DSL ecosystem
- Returns results in consistent format (DataFrame, nodes, edges, subgraph)

Pattern Matching vs. DSL Node/Edge Queries

Use node/edge queries when you need individual elements. Use pattern matching when you need to find subgraph structures.

Limitations and Future Work

Current Limitations (v1)

- **Fixed-length paths only:** Variable-length paths like `(a)-[*1..3]-(b)` not yet supported
- **No optional matching:** All pattern elements must match

- **No UNION patterns:** Cannot express "match pattern A OR pattern B"
- **Limited aggregation:** Aggregation over matches not yet implemented

These features are planned for future versions while maintaining the current clean API.

Best Practices

1. **Start simple:** Begin with small patterns and add complexity incrementally
2. **Test on small data:** Validate pattern logic on toy networks before scaling up
3. **Use explain():** Check execution plans for complex queries
4. **Combine with DSL:** Use pattern matching for structure, DSL for attributes
5. **Document patterns:** Add comments explaining complex pattern logic

Troubleshooting

Pattern Returns No Matches

Problem: Your pattern should match but returns 0 results.

Solutions:

1. Check layer constraints — are nodes actually in the layers you specified?
2. Verify predicates — are the thresholds too restrictive?
3. Use `.explain()` to see the execution plan
4. Test with simpler patterns first
5. Check if your network has the expected structure

```
# Debug: Check basic statistics first
result = Q.nodes().where(layer="social").execute(network)
print(f"Social layer has {result.count} nodes")

result = Q.edges().where(intralayer=True).execute(network)
print(f"Network has {result.count} intralayer edges")
```

Too Many Matches

Problem: Pattern returns too many results.

Solutions:

1. Add more predicates to narrow the search
2. Use `.limit()` to cap results
3. Add layer constraints to restrict search space
4. Use `.constraint("a != b")` to avoid duplicates

```
# Limit results
pattern.limit(100).execute(network)
```

Slow Performance

Problem: Pattern matching takes too long.

Solutions:

1. Add selective predicates to the root variable
2. Use layer constraints to reduce search space
3. Consider breaking pattern into smaller sub-patterns
4. Use `.explain()` to check estimated complexity
5. Set timeout to avoid runaway queries

```
# Set timeout
result = pattern.execute(network, timeout=30.0)
```

Related Documentation

- *How to Query Multilayer Graphs with the SQL-like DSL* — Basic DSL queries for nodes and edges
- *Query Zoo: DSL Gallery for Multilayer Analysis* — Gallery of DSL query examples
- *How to Load and Build Networks* — Creating multilayer networks
- *How to Compute Network Statistics* — Computing network metrics

Next Steps

- Explore the *Query Zoo: DSL Gallery for Multilayer Analysis* for more complex examples
- Try combining pattern matching with DSL queries
- Experiment with different motifs (triangles, cliques, paths)
- Build your own domain-specific pattern library

Tip

Pro Tip: Pattern matching is most powerful when combined with the rest of the DSL. Use patterns to find structures, then use regular DSL queries to compute metrics on the matches!

3.3.12 How to Build Analysis Pipelines with Dplyr-style Operations

Goal: Chain operations to build reproducible analysis workflows.

Prerequisites: A loaded network (see *How to Load and Build Networks*).

Basic Pipeline

Node Operations

```
from py3plex.core import multinet
from py3plex.graph_ops import nodes

network = multinet.multi_layer_network()
```

(continues on next page)

(continued from previous page)

```
network.load_network("data.multiedgelist", input_type="multiedgelist")

# Build pipeline
result = (
    nodes(network)
    .filter(lambda n: n["degree"] > 2)
    .mutate(score=lambda n: n["degree"] * 2)
    .arrange("degree", reverse=True)
    .to_pandas()
)

print(result.head())
```

Available Operations

filter()

Select nodes based on condition:

```
result = (
    nodes(network)
    .filter(lambda n: n["layer"] == "friends")
    .filter(lambda n: n["degree"] > 5)
    .to_pandas()
)
```

mutate()

Add or modify columns:

```
result = (
    nodes(network)
    .mutate(
        degree_squared=lambda n: n["degree"] ** 2,
        is_hub=lambda n: n["degree"] > 10
    )
    .to_pandas()
)
```

select()

Choose specific columns:

```
result = (
    nodes(network)
    .select("node", "layer", "degree")
    .to_pandas()
)
```

arrange()

Sort results:

```
result = (
    nodes(network)
    .arrange("degree", reverse=True)  # Descending
    .to_pandas()
)
```

group_by() and summarize()

Aggregate data:

```
result = (
    nodes(network)
    .group_by("layer")
    .summarize(
        avg_degree=lambda g: g["degree"].mean(),
        max_degree=lambda g: g["degree"].max(),
        count=lambda g: len(g)
    )
    .to_pandas()
)
```

Expected output:

```
     layer  avg_degree  max_degree  count
0  friends        3.45          12     46
1     work        2.87          08     46
2   family        4.12          15     42
```

Complex Pipelines

Multi-Step Analysis

```
result = (
    nodes(network)
    # Step 1: Filter active nodes
    .filter(lambda n: n["layer_count"] > 1)
    # Step 2: Add computed score
    .mutate(
        activity_score=lambda n: n["degree"] * n["layer_count"]
    )
    # Step 3: Filter by score
    .filter(lambda n: n["activity_score"] > 20)
    # Step 4: Sort by score
    .arrange("activity_score", reverse=True)
    # Step 5: Get top 20
    .head(20)
    # Convert to pandas
```

(continues on next page)

(continued from previous page)

```
    .to_pandas()
)
```

Combining with DSL

Mix dplyr-style and DSL:

```
from py3plex.dsl import Q
from py3plex.graph_ops import nodes

# Use DSL to compute metrics
dsl_result = (
    Q.nodes()
     .compute("degree", "betweenness_centrality")
     .execute(network)
)

# Use dplyr-style to filter and transform
final_result = (
    nodes(dsl_result)
    .filter(lambda n: n["degree"] > 5)
    .mutate(
        centrality_rank=lambda n: n["betweenness_centrality"] * 100
    )
    .arrange("centrality_rank", reverse=True)
    .to_pandas()
)
```

Sklearn-Style Pipelines

For machine learning workflows:

```
from py3plex.pipeline import NetworkPipeline
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# Define pipeline
pipeline = NetworkPipeline([
    ('features', 'compute_features'),  # Extract features
    ('scaler', StandardScaler()),       # Normalize
    ('cluster', KMeans(n_clusters=5))   # Cluster
])

# Fit and predict
labels = pipeline.fit_predict(network)

print(f"Assigned {len(set(labels))} clusters")
```

Exporting Pipelines

Save Pipeline Definition

```
import json

pipeline_config = {
    'steps': [
        {'operation': 'filter', 'condition': 'degree > 2'},
        {'operation': 'mutate', 'column': 'score', 'expr': 'degree * 2'},
        {'operation': 'arrange', 'by': 'score', 'reverse': True}
    ]
}

with open('pipeline.json', 'w') as f:
    json.dump(pipeline_config, f, indent=2)
```

Load and Execute

```
with open('pipeline.json', 'r') as f:
    config = json.load(f)

# Execute pipeline from config
result = execute_pipeline(network, config)
```

Reusable Pipelines

Create Pipeline Functions

```
def identify_hubs(network, threshold=10):
    """Reusable pipeline to identify hub nodes."""
    return (
        nodes(network)
        .filter(lambda n: n["degree"] > threshold)
        .mutate(hub_score=lambda n: n["degree"] * n["layer_count"])
        .arrange("hub_score", reverse=True)
        .to_pandas()
    )

# Use
hubs = identify_hubs(network, threshold=5)
print(hubs)
```

Next Steps

- **Query with DSL:** *How to Query Multilayer Graphs with the SQL-like DSL*
- **Complete workflows:** *How to Reproduce Common Analysis Workflows*
- **API reference:** *API Documentation*

3.3.13 How to Reproduce Common Analysis Workflows

Goal: Use ready-made recipes for common multilayer network analysis tasks.

Prerequisites: Basic understanding of py3plex (see *10-Minute Tutorial*).

Complete Workflows

This guide links to detailed recipes and case studies. For step-by-step implementations, see:

- **Recipes:** *Analysis Recipes & Workflows* — Focused solutions for specific tasks
- **Case Studies:** *Use Cases & Case Studies* — Complete end-to-end analyses
- **Examples:** *Examples & Recipes* — Runnable code examples

Quick Recipe Index

Network Construction

- **Building from edge lists** → *How to Load and Build Networks*
- **Converting from NetworkX** → *Analysis Recipes & Workflows* (Recipe 1)
- **Loading temporal networks** → *API Documentation* (Temporal section)

Statistical Analysis

- **Computing multilayer statistics** → *How to Compute Network Statistics*
- **Comparing layers** → *Analysis Recipes & Workflows* (Recipe 3)
- **Node versatility analysis** → *Analysis Recipes & Workflows* (Recipe 4)

Community Detection

- **Multilayer Louvain** → *How to Run Community Detection on Multilayer Networks*
- **Cross-layer community comparison** → *Analysis Recipes & Workflows* (Recipe 5)
- **Community stability analysis** → *Use Cases & Case Studies* (Case Study 2)

Network Embeddings

- **Node2Vec for link prediction** → *How to Run Random Walk Algorithms*
- **Embedding-based clustering** → *Analysis Recipes & Workflows* (Recipe 7)
- **Layer-specific embeddings** → *How to Run Random Walk Algorithms*

Visualization

- **Publication-ready plots** → *How to Visualize Multilayer Networks*
- **Interactive visualizations** → *How to Visualize Multilayer Networks*
- **Layer comparison plots** → *Analysis Recipes & Workflows* (Recipe 9)

Domain-Specific Workflows

Social Networks

Multi-platform social analysis:

```
# See: user_guide/case_studies.rst - Social Network Case Study
# 1. Load data from multiple platforms
# 2. Detect cross-platform communities
# 3. Identify influential users
# 4. Analyze information diffusion
```

See *Use Cases & Case Studies* for complete implementation.

Biological Networks

Multi-omics integration:

```
# See: user_guide/case_studies.rst - Biological Network Case Study
# 1. Integrate protein-protein + gene regulation + metabolic pathways
# 2. Find key regulators using multilayer centrality
# 3. Detect functional modules
# 4. Prioritize disease genes
```

See *Use Cases & Case Studies* for complete implementation.

Transportation Networks

Multimodal route analysis:

```
# See: examples/index.rst - Transportation Example
# 1. Model different transportation modes as layers
# 2. Add transfer connections between layers
# 3. Compute optimal multimodal routes
# 4. Identify critical transfer points
```

See *Examples & Recipes* for runnable code.

Config-Driven Workflows

Use configuration files for reproducibility:

```yaml
# workflow_config.yaml
network:
  input_file: "data.multiedgelist"
  input_type: "multiedgelist"

analysis:
  - name: "statistics"
    metrics: ["degree", "betweenness_centrality"]

  - name: "community_detection"
    algorithm: "louvain"
```

(continues on next page)

(continued from previous page)

```
    params:
      resolution: 1.0

  - name: "visualization"
    output: "network.png"
    layout: "force_directed"
```

Execute workflow:

```python
from py3plex.workflows import execute_workflow

results = execute_workflow("workflow_config.yaml")
```

See *Analysis Recipes & Workflows* for complete config-driven workflow examples.

DSL-Driven Analysis Workflows

Goal: Use py3plex's DSL to create reproducible, declarative analysis pipelines.

The DSL provides a powerful way to express analysis workflows as queries rather than imperative code. This makes workflows more readable, maintainable, and reproducible.

Basic DSL Workflow Pattern

Template for DSL-first analysis:

```python
from py3plex.core import multinet
from py3plex.dsl import Q, L, execute_query

# 1. Load network
network = multinet.multi_layer_network(directed=False)
network.load_network(
    "py3plex/datasets/_data/synthetic_multilayer.edges",
    input_type="multiedgelist"
)

# 2. Query and filter nodes with DSL
high_degree_nodes = (
    Q.nodes()
     .compute("degree", "betweenness_centrality")
     .where(degree__gt=5)
     .order_by("betweenness_centrality", reverse=True)
     .execute(network)
)

# 3. Extract subnetwork
subgraph = network.core_network.subgraph(high_degree_nodes.keys())

# 4. Analyze subnetwork
print(f"High-degree subnetwork:")
print(f"  Nodes: {len(high_degree_nodes)}")
```

(continues on next page)

(continued from previous page)

```
print(f"  Edges: {subgraph.number_of_edges()}")

# 5. Export results
import pandas as pd
df = pd.DataFrame([
    {
        'node': node[0],
        'layer': node[1],
        'degree': data['degree'],
        'betweenness': data['betweenness_centrality']
    }
    for node, data in high_degree_nodes.items()
])
df.to_csv('high_degree_analysis.csv', index=False)
```

Expected output:

```
High-degree subnetwork:
  Nodes: 25
  Edges: 89
```

Multilayer Exploration Workflow

Systematic multilayer network analysis:

```
from py3plex.core import multinet
from py3plex.dsl import Q, L
import pandas as pd

# Load network
network = multinet.multi_layer_network(directed=False)
network.load_network(
    "py3plex/datasets/_data/synthetic_multilayer.edges",
    input_type="multiedgelist"
)

print("MULTILAYER NETWORK EXPLORATION")
print("=" * 70)

# Step 1: Per-layer statistics
print("\n1. Per-Layer Statistics:")
layer_stats = []

for layer in network.get_layers():
    # Query layer nodes
    layer_nodes = Q.nodes().from_layers(L[layer]).execute(network)
    layer_edges = Q.edges().from_layers(L[layer]).execute(network)

    # Compute metrics
```

(continues on next page)

(continued from previous page)

```
    result = (
        Q.nodes()
         .from_layers(L[layer])
         .compute("degree")
         .execute(network)
    )

    avg_degree = sum(d['degree'] for d in result.values()) / len(result) if␣
↪result else 0

    layer_stats.append({
        'layer': layer,
        'nodes': len(layer_nodes),
        'edges': len(layer_edges),
        'avg_degree': avg_degree
    })

    print(f"  {layer}: {len(layer_nodes)} nodes, {len(layer_edges)} edges,␣
↪avg_degree={avg_degree:.2f}")

# Step 2: Find versatile nodes (present in multiple layers)
print("\n2. Versatile Nodes (multilayer presence):")
from collections import Counter

node_layer_count = Counter()
for node, layer in network.get_nodes():
    node_layer_count[node] += 1

versatile_nodes = {
    node: count for node, count in node_layer_count.items()
    if count >= 2
}

print(f"  Total versatile nodes: {len(versatile_nodes)}")
print(f"  Top 5 most versatile:")
for node, count in sorted(versatile_nodes.items(), key=lambda x: x[1],␣
↪reverse=True)[:5]:
    print(f"    {node}: {count} layers")

# Step 3: Layer comparison
print("\n3. Layer Overlap Analysis:")
layers = network.get_layers()

for i, layer1 in enumerate(layers):
    for layer2 in layers[i+1:]:
        nodes1 = set(n[0] for n in Q.nodes().from_layers(L[layer1]).
↪execute(network).keys())
        nodes2 = set(n[0] for n in Q.nodes().from_layers(L[layer2]).
↪execute(network).keys())
```

(continues on next page)

(continued from previous page)

```
        overlap = nodes1 & nodes2
        jaccard = len(overlap) / len(nodes1 | nodes2) if (nodes1 | nodes2)␣
↪else 0

        print(f"  {layer1}  {layer2}: {len(overlap)} nodes, Jaccard=
↪{jaccard:.3f}")

# Step 4: Hub identification across layers
print("\n4. Cross-Layer Hub Nodes:")
all_metrics = (
    Q.nodes()
     .compute("degree", "betweenness_centrality")
     .where(degree__gt=7)
     .execute(network)
)

print(f"  Hub nodes (degree > 7): {len(all_metrics)}")

# Group hubs by base node ID
from collections import defaultdict
hub_layers = defaultdict(set)

for (node, layer), data in all_metrics.items():
    hub_layers[node].add(layer)

print(f"  Unique hub node IDs: {len(hub_layers)}")
print(f"  Top 5 hub nodes:")
for node, layers in sorted(hub_layers.items(), key=lambda x: len(x[1]),␣
↪reverse=True)[:5]:
    print(f"    {node}: present in {len(layers)} layers - {list(layers)}")
```

Expected output:

```
MULTILAYER NETWORK EXPLORATION
======================================================================

1. Per-Layer Statistics:
  layer1: 40 nodes, 95 edges, avg_degree=4.75
  layer2: 40 nodes, 87 edges, avg_degree=4.35
  layer3: 40 nodes, 102 edges, avg_degree=5.10

2. Versatile Nodes (multilayer presence):
  Total versatile nodes: 35
  Top 5 most versatile:
    node7: 3 layers
    node12: 3 layers
    node3: 3 layers
    node15: 3 layers
```

(continues on next page)

(continued from previous page)

```
    node1: 3 layers

3. Layer Overlap Analysis:
  layer1  layer2: 35 nodes, Jaccard=0.875
  layer1  layer3: 32 nodes, Jaccard=0.800
  layer2  layer3: 33 nodes, Jaccard=0.825

4. Cross-Layer Hub Nodes:
  Hub nodes (degree > 7): 18
  Unique hub node IDs: 12
  Top 5 hub nodes:
    node7: present in 3 layers - ['layer1', 'layer2', 'layer3']
    node12: present in 3 layers - ['layer1', 'layer2', 'layer3']
    node3: present in 3 layers - ['layer1', 'layer2', 'layer3']
    node15: present in 2 layers - ['layer1', 'layer3']
    node8: present in 2 layers - ['layer2', 'layer3']
```

Community Detection + DSL Workflow

Combine community detection with DSL queries:

```
from py3plex.algorithms.community_detection.community_wrapper import louvain_
↪communities
from py3plex.dsl import Q, execute_query

# Detect communities
communities = louvain_communities(network)

# Attach as node attributes
for (node, layer), comm_id in communities.items():
    network.core_network.nodes[(node, layer)]['community'] = comm_id

print("COMMUNITY-BASED ANALYSIS")
print("=" * 70)

# Query each community
community_ids = set(communities.values())

for comm_id in sorted(community_ids):
    # Use DSL to get community members
    comm_nodes = execute_query(
        network,
        f'SELECT nodes WHERE community={comm_id}'
    )

    # Compute community metrics
    comm_result = (
        Q.nodes()
         .where(community=comm_id)
```

(continues on next page)

(continued from previous page)

```
        .compute("degree", "betweenness_centrality")
        .execute(network)
    )

    # Statistics
    avg_degree = sum(d['degree'] for d in comm_result.values()) / len(comm_
↪result)
    avg_betw = sum(d['betweenness_centrality'] for d in comm_result.values())␣
↪/ len(comm_result)

    # Layer composition
    layer_counts = Counter(node[1] for node in comm_nodes)

    print(f"\nCommunity {comm_id}:")
    print(f"  Size: {len(comm_nodes)} nodes")
    print(f"  Avg degree: {avg_degree:.2f}")
    print(f"  Avg betweenness: {avg_betw:.6f}")
    print(f"  Layer composition: {dict(layer_counts)}")
```

Dynamics + DSL Workflow

Epidemic simulation with DSL-based analysis:

```
from py3plex.dynamics import SIRDynamics
from py3plex.dsl import Q, L

# Run SIR simulation
sir = SIRDynamics(
    network,
    beta=0.3,
    gamma=0.1,
    initial_infected=0.05
)
sir.set_seed(42)
results = sir.run(steps=100)

# Attach final state
final_state = results.trajectory[-1]
for node, state in final_state.items():
    network.core_network.nodes[node]['sir_state'] = state

print("EPIDEMIC ANALYSIS")
print("=" * 70)

# Per-layer infection analysis
for layer in network.get_layers():
    layer_nodes = Q.nodes().from_layers(L[layer]).execute(network)

    state_counts = Counter(
```

(continues on next page)

(continued from previous page)

```
        network.core_network.nodes[node].get('sir_state', 'unknown')
        for node in layer_nodes.keys()
    )

    total = len(layer_nodes)
    infected_pct = state_counts.get('I', 0) / total * 100 if total > 0 else 0
    recovered_pct = state_counts.get('R', 0) / total * 100 if total > 0 else 0

    print(f"\n{layer}:")
    print(f"  S: {state_counts.get('S', 0)} ({state_counts.get('S', 0)/
↪total*100:.1f}%)")
    print(f"  I: {state_counts.get('I', 0)} ({infected_pct:.1f}%)")
    print(f"  R: {state_counts.get('R', 0)} ({recovered_pct:.1f}%)")

# Identify superspreaders (infected nodes with high degree)
superspreaders = (
    Q.nodes()
     .where(sir_state='I')
     .compute("degree", "betweenness_centrality")
     .where(degree__gt=6)
     .order_by("degree", reverse=True)
     .execute(network)
)

print(f"\nSuperspreaders (infected, degree > 6): {len(superspreaders)}")
for node, data in list(superspreaders.items())[:5]:
    print(f"  {node}: degree={data['degree']}, betw={data['betweenness_
↪centrality']:.4f}")
```

Reusable DSL Query Functions

Create reusable query templates:

```
def get_layer_hubs(network, layer, degree_threshold=5):
    """Get high-degree nodes in a specific layer."""
    from py3plex.dsl import Q, L

    return (
        Q.nodes()
         .from_layers(L[layer])
         .compute("degree")
         .where(degree__gt=degree_threshold)
         .order_by("degree", reverse=True)
         .execute(network)
    )

def get_versatile_nodes(network, min_layers=2):
    """Get nodes present in multiple layers."""
    from collections import Counter
```

(continues on next page)

(continued from previous page)

```
    node_layer_count = Counter()
    for node, layer in network.get_nodes():
        node_layer_count[node] += 1

    return {
        node: count for node, count in node_layer_count.items()
        if count >= min_layers
    }

def compare_layer_centrality(network, layer1, layer2):
    """Compare centrality distributions between two layers."""
    from py3plex.dsl import Q, L
    import numpy as np

    result1 = (
        Q.nodes()
         .from_layers(L[layer1])
         .compute("betweenness_centrality")
         .execute(network)
    )

    result2 = (
        Q.nodes()
         .from_layers(L[layer2])
         .compute("betweenness_centrality")
         .execute(network)
    )

    betw1 = [d['betweenness_centrality'] for d in result1.values()]
    betw2 = [d['betweenness_centrality'] for d in result2.values()]

    return {
        'layer1': {'mean': np.mean(betw1), 'std': np.std(betw1)},
        'layer2': {'mean': np.mean(betw2), 'std': np.std(betw2)}
    }

# Use reusable functions
hubs_layer1 = get_layer_hubs(network, 'layer1', degree_threshold=7)
versatile = get_versatile_nodes(network, min_layers=3)
centrality_comp = compare_layer_centrality(network, 'layer1', 'layer2')

print(f"Layer1 hubs: {len(hubs_layer1)}")
print(f"Highly versatile nodes: {len(versatile)}")
print(f"Centrality comparison: {centrality_comp}")
```

Why use DSL-driven workflows?

- **Declarative:** Express *what* to analyze, not *how* to compute
- **Composable:** Chain queries to build complex analyses

- **Reproducible:** Queries are self-documenting and version-controllable
- **Efficient:** DSL optimizes execution internally
- **Readable:** SQL-like syntax is intuitive for data analysis

Next steps with DSL workflows:

- **Full DSL tutorial:** *How to Query Multilayer Graphs with the SQL-like DSL* - Comprehensive DSL guide
- **Community detection:** *How to Run Community Detection on Multilayer Networks* - Community + DSL workflows
- **Dynamics simulation:** *How to Simulate Multilayer Dynamics* - Dynamics + DSL workflows

Batch Processing

Process multiple networks:

```
import glob
from py3plex.core import multinet

results = []

for filename in glob.glob("data/*.multiedgelist"):
    # Load network
    network = multinet.multi_layer_network()
    network.load_network(filename, input_type="multiedgelist")

    # Apply analysis pipeline
    stats = analyze_network(network)  # Your custom function

    results.append({
        'filename': filename,
        'stats': stats
    })

# Aggregate results
summary = aggregate_results(results)
```

Complete Example Templates

The following locations contain complete, runnable examples:

1. **User Guide Recipes** (*Analysis Recipes & Workflows*)
   - Recipe-style solutions with code + explanation
   - Focused on single tasks
2. **Case Studies** (*Use Cases & Case Studies*)
   - End-to-end analyses
   - Real-world datasets
   - Publication-ready results
3. **Examples Gallery** (*Examples & Recipes*)

- Standalone Python scripts
- Minimal, focused examples
- Easy to adapt

Next Steps

- **Learn fundamentals:** *10-Minute Tutorial*
- **Detailed recipes:** *Analysis Recipes & Workflows*
- **Complete case studies:** *Use Cases & Case Studies*
- **Browse examples:** *Examples & Recipes*

3.4 Part IV: Concepts & Explanations

Theoretical background and architectural explanations.

3.4.1 Concepts & Explanations

Multilayer networks capture systems where entities interact through multiple types of relationships simultaneously. This section explains the theory behind multilayer networks and how py3plex implements them.

What's in this section:

- *Multilayer Networks 101* — Types (multiplex, heterogeneous, temporal) and when to use them
- *The py3plex Core Model* — Node-layer pairs, supra-adjacency matrix, NetworkX integration
- *Design Principles* — Why py3plex works the way it does
- *Algorithm Landscape* — Overview of available algorithms

Reading Paths:

- **New to multilayer networks?** Start with *Multilayer Networks 101* for essential concepts, then *The py3plex Core Model* to see how py3plex represents them. This foundation is crucial before using the library.
- **Coming from NetworkX?** Read *The py3plex Core Model* to understand the node-layer pair abstraction and supra-adjacency matrix, then *Design Principles* for design decisions. Skip the theory in 101 if you're familiar with complex networks.
- **Building expertise?** Read all chapters in order: theory (*Multilayer Networks 101*), implementation (*The py3plex Core Model*), design rationale (*Design Principles*), and available tools (*Algorithm Landscape*).

After Reading This Section

You'll be able to:

- Model real-world systems as multilayer networks
- Choose appropriate parameters (e.g., inter-layer coupling strength)
- Interpret results correctly

- Avoid common pitfalls (e.g., flattening important structure)

Tip

Quick check: After reading this section, you should be able to answer:

- What's the difference between multiplex and heterogeneous networks?
- What does the supra-adjacency matrix represent?
- When should you use high vs. low inter-layer coupling?

Relation to Other Sections:

- **Overview** (*Overview*) — Quick 2-minute intro to multilayer networks
- **How-to Guides** (*How-to Guides*) — Apply these concepts in practice
- **Reference** (*API & DSL Reference*) — Detailed API and algorithm documentation
- **Examples** (*Examples & Recipes*) — See concepts in action with real code

Jump to Practical Applications:

Once you understand the concepts:

- **Load networks:** *How to Load and Build Networks*
- **Compute statistics:** *How to Compute Network Statistics*
- **Detect communities:** *How to Run Community Detection on Multilayer Networks*
- **Query networks:** *How to Query Multilayer Graphs with the SQL-like DSL*

3.4.2 Multilayer Networks 101

This chapter explains what multilayer networks are, when to use them, and how to choose the right type for your data.

You will learn:

- What distinguishes multilayer from single-layer networks
- Types: multiplex, heterogeneous, temporal, interdependent
- When to use multilayer modeling
- Common pitfalls to avoid

What are Multilayer Networks?

A **multilayer network** models systems with multiple types of relationships, node types, or interaction contexts.

Example: A researcher's social world includes coauthors, colleagues, students, and Twitter followers. These are different relationship types with different meanings. A multilayer network keeps them as separate **layers** rather than flattening into one graph.

Traditional vs. Multilayer:

Aspect	Single-Layer	Multilayer
Node types	One (e.g., only people)	Multiple (people, orgs, docs)
Edge types	One (e.g., only friendship)	Multiple per layer
Structure	Homogeneous	Heterogeneous
Analysis	Standard graph algorithms	Layer-aware algorithms

Types of Multilayer Networks

Multiplex Networks

Same nodes, different relationship types.

```
from py3plex.core import multinet

network = multinet.multi_layer_network(network_type="multiplex")
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1],
    ['Bob', 'friends', 'Carol', 'friends', 1],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1],
    ['Bob', 'colleagues', 'Dave', 'colleagues', 1],
], input_type="list")
```

Examples: Social networks across platforms, transportation via air/rail/road, communication via email/phone/chat.

Heterogeneous Information Networks

Different node types with type-specific relationships.

```
network = multinet.multi_layer_network()
network.add_edges([
    ['Alice', 'authors', 'P1', 'papers', 1],
    ['P1', 'papers', 'ICML', 'venues', 1],
], input_type="list")
```

Examples: Academic networks (authors, papers, venues), e-commerce (users, products, sellers), biomedical (drugs, diseases, targets).

Temporal Networks

Networks that evolve over time, with time-sliced layers.

```
network = multinet.multi_layer_network()
network.add_edges([
    ['A', '2020', 'B', '2020', 1],
    ['A', '2021', 'B', '2021', 1],
    ['B', '2021', 'C', '2021', 1],
], input_type="list")
```

Examples: Communication over time, social dynamics, disease spread.

Interdependent Networks

Multiple networks where nodes depend on each other.

Examples: Power grid depends on communication network, supply chains, cyber-physical systems.

When to Use Multilayer Networks

Use multilayer modeling when:

1. **Multiple relationship types matter** — Friendship and professional connections behave differently
2. **Node roles vary by context** — A hub in work network may be peripheral in hobby network
3. **Layer interactions are important** — Transportation failures in one mode affect others
4. **Temporal evolution matters** — Relationships change over time
5. **System-level properties emerge** — Cross-layer dependencies affect resilience

Choosing a Modeling Approach

Ask these questions:

1. **Layers vs. attributes?** If relationships have different *types* → use layers. If they vary only in *weight* → use edge attributes.
2. **Same or different nodes?** Same entities in all layers → multiplex. Different entity types → heterogeneous.
3. **Coupling strength?** Identity coupling only → `omega=1.0`. Nodes can differ → lower omega.
4. **Temporal structure?** Yes → time-sliced layers. No → aggregate into static network.

What Goes Wrong When You Flatten

Community structure destroyed: Flattening can create spurious bridges between groups that are distinct in the layer structure.

Centrality becomes misleading: A researcher with 2 coauthors and 500 Twitter followers has degree 502 when flattened, but academic influence is only 2.

Path analysis fails: Email → meeting transitions matter for information flow but are invisible in a flattened network.

Common Pitfalls

- **Over-aggregating** — Combining layers that should stay separate
- **Under-aggregating** — Too many sparse layers (use edge weights instead)
- **Ignoring coupling** — Treating layers as independent when nodes correspond
- **Wrong coupling strength** — Too high forces artificial consistency; too low loses information
- **Mismatched identifiers** — Same entity must have same ID across layers

Key Terminology

- **Intra-layer edges** — Within a single layer
- **Inter-layer edges** — Between layers (often identity edges connecting same node)
- **Node-layer pairs** — `(node_id, layer_id)` tuples, the fundamental unit
- **Supra-adjacency matrix** — Block matrix encoding both intra- and inter-layer connections

Next Steps

- *The py3plex Core Model* — Internal data structures
- *Design Principles* — Why py3plex works this way
- *Algorithm Landscape* — Available algorithms

References:

- Kivelä et al. (2014). "Multilayer networks." *J. Complex Networks* 2(3): 203-271.
- Boccaletti et al. (2014). "The structure and dynamics of multilayer networks." *Physics Reports* 544(1): 1-122.

3.4.3 The py3plex Core Model

In which we look under the hood at how py3plex represents multilayer networks, understand the node-layer pair representation, and learn to work with the supra-adjacency matrix.

In the previous chapter, you learned *what* multilayer networks are conceptually. Now we'll see *how* py3plex represents them in code—and why these design choices matter for your analysis.

Understanding the internal representation will help you:

- Debug unexpected behavior (why does my network have twice as many nodes as I expected?)
- Integrate with NetworkX (how do I use existing algorithms on my multilayer network?)
- Scale to large networks (when should I use sparse matrices? how much memory will this take?)
- Extend py3plex (how do I add my own algorithms?)

Network Types: Multilayer vs Multiplex

py3plex supports two network types that determine how layers and inter-layer connections are handled:

Multilayer Networks (`network_type='multilayer'`, default)

General multilayer networks where each layer can have a different set of nodes. No automatic coupling edges are created.

- Use when layers represent different node types (e.g., authors, papers, venues)
- Use when nodes naturally appear in only some layers
- Inter-layer edges must be explicitly added

```
from py3plex.core import multinet

# Heterogeneous network: authors and papers are different node types
network = multinet.multi_layer_network(network_type='multilayer')
network.add_edges([
    {'source': 'author1', 'target': 'paper1',
     'source_type': 'authors', 'target_type': 'papers'}
])
```

Multiplex Networks (`network_type='multiplex'`)

Special case where all layers share the same set of nodes but with different relationship types. After loading, coupling edges are automatically created between each node and its counterparts in other layers.

- Use when the same entities (people, cities) are connected via multiple relationship types
- Coupling edges are auto-generated with `type='coupling'`
- Filter coupling edges using `get_edges(multiplex_edges=False)`

```
# Social network: same people, different relationship types
network = multinet.multi_layer_network(network_type='multiplex')
network.load_network('friends_and_colleagues.edges', input_type='multiplex_
↪edges')

# Coupling edges are auto-created between (Alice, friends) <-> (Alice,␣
↪colleagues)

# Get only explicit edges (exclude auto-coupling)
explicit_edges = list(network.get_edges(multiplex_edges=False))

# Get all edges including coupling
all_edges = list(network.get_edges(multiplex_edges=True))
```

Comparison Table:

Feature	multilayer	multiplex
Node sets	Different per layer	Same across layers
Coupling edges	Manual	Automatic
Load behavior	As-is	• coupling edges
get_edges()	All edges	Filters coupling
Use case	Heterogeneous nets	Same entities

The `multi_layer_network` Class

The central data structure in py3plex is the `multi_layer_network` class, which provides a high-level API for working with multilayer networks while maintaining full compatibility with NetworkX.

Basic Structure

```
from py3plex.core import multinet

# Create a multilayer network
network = multinet.multi_layer_network()

# The underlying NetworkX graph
nx_graph = network.core_network  # MultiDiGraph or MultiGraph

# Layer management
layers = network.get_layers()  # List of layer names
layer_map = network.layer_name_map  # Layer name to ID mapping
```

Key Components

1. Core NetworkX Graph

At its heart, py3plex uses a NetworkX `MultiDiGraph` (for directed networks) or `MultiGraph` (for undirected networks) to store all nodes and edges.

```
# Access the underlying graph
G = network.core_network

# Use any NetworkX function
import networkx as nx
betweenness = nx.betweenness_centrality(G)
```

2. Layer Encoding

Nodes are internally encoded as `(node_id, layer_id)` tuples to distinguish the same logical node across different layers.

```
# Node 'Alice' in layer 'friends' is stored as ('Alice', 'friends')
network.add_nodes([('Alice', 'friends')])

# Access all node-layer pairs
for node_layer_pair in network.get_nodes():
    print(node_layer_pair)  # ('Alice', 'friends'), ('Bob', 'friends'), ...
```

3. Layer Mapping

py3plex maintains bidirectional mappings between layer names and internal layer IDs for efficient lookups.

```
# Get layer mapping
layer_map = network.layer_name_map
```

(continues on next page)

(continued from previous page)

```
# {'friends': 0, 'colleagues': 1, ...}
```

4. Attributes

Node and edge attributes are stored directly in the underlying NetworkX graph and can be accessed and modified using standard NetworkX patterns.

```
# Add node with attributes
network.core_network.nodes[('Alice', 'friends')]['age'] = 30

# Add edge with attributes
network.add_edges([
    [('Alice', 'friends'), ('Bob', 'friends'), {'weight': 0.8, 'type': 'close
↪'}]
])
```

Internal Representation

Node-Layer Pairs

The key concept in py3plex's internal model is the **node-layer pair**: a tuple `(node_id, layer_id)` that uniquely identifies a node in a specific layer.

```
# Same logical node, different layers
alice_friends = ('Alice', 'friends')
alice_colleagues = ('Alice', 'colleagues')

# These are treated as different nodes internally
network.add_nodes([alice_friends, alice_colleagues])
```

This representation allows:

- The same logical entity to appear in multiple layers
- Different attributes per node-layer pair
- Efficient layer-specific operations

Edge Types

Intra-Layer Edges

Edges connecting nodes within the same layer:

```
# Edge within 'friends' layer
network.add_edges([
    [('Alice', 'friends'), ('Bob', 'friends'), 1.0]
])
```

Inter-Layer Edges

Edges connecting nodes across different layers:

```
# Connect Alice across layers (identity edge)
network.add_edges([
    [('Alice', 'friends'), ('Alice', 'colleagues'), 1.0]
])

# Connect different nodes across layers
network.add_edges([
    [('Alice', 'friends'), ('Bob', 'colleagues'), 0.5]
])
```

Supra-Adjacency Matrix

The supra-adjacency matrix is a block matrix representation that encodes the entire multilayer network structure.

Mathematical Definition

For a multilayer network with L layers, the supra-adjacency matrix $\mathbf{S}$ is:

$$\mathbf{S} = \begin{bmatrix} A_1 & C_{12} & \cdots & C_{1L} \\ C_{21} & A_2 & \cdots & C_{2L} \\ \vdots & \vdots & \ddots & \vdots \\ C_{L1} & C_{L2} & \cdots & A_L \end{bmatrix}$$

Where:

- A_α is the adjacency matrix of layer α (intra-layer connections)
- $C_{\alpha\beta}$ is the coupling matrix between layers α and β (inter-layer connections)

Numeric Example

Consider a simple multiplex network with 3 nodes (A, B, C) and 2 layers. In layer 1, A-B are connected. In layer 2, B-C are connected. The nodes are the same across layers (multiplex structure).

Network structure:

```
Layer 1:        Layer 2:
A --- B   C     A   B --- C
```

Node ordering: We order nodes as (A,L1), (B,L1), (C,L1), (A,L2), (B,L2), (C,L2).

Intra-layer blocks:

Layer 1 adjacency matrix (A_1):

```
     A   B   C
A [  0   1   0 ]
B [  1   0   0 ]
C [  0   0   0 ]
```

Layer 2 adjacency matrix (A_2):

```
     A   B   C
A [  0   0   0 ]
B [  0   0   1 ]
C [  0   1   0 ]
```

Inter-layer coupling blocks (C_{12} and C_{21}):

In a multiplex network, we typically add identity coupling—each node connects to itself across layers:

```
C₁₂ = C₂₁ =
     A   B   C
A [  1   0   0 ]
B [  0   1   0 ]
C [  0   0   1 ]
```

Full supra-adjacency matrix (6×6):

```
          Layer 1          |     Layer 2
        A    B    C        |  A    B    C
L1  A [ 0    1    0        | 1    0    0 ]
    B [ 1    0    0        | 0    1    0 ]
    C [ 0    0    0        | 0    0    1 ]
   ---|-------------------+---------------
L2  A [ 1    0    0        | 0    0    0 ]
    B [ 0    1    0        | 0    0    1 ]
    C [ 0    0    1        | 0    1    0 ]
```

The diagonal 3×3 blocks are the within-layer adjacencies (A_1 and A_2). The off-diagonal 3×3 blocks are the coupling matrices (C_{12} and C_{21}).

What this enables:

- **Multilayer shortest paths:** A path from (A,L1) to (C,L2) must go through B: (A,L1)→(B,L1)→(B,L2)→(C,L2)
- **Spectral methods:** Eigenvalues of S capture multilayer structure
- **Matrix operations:** All linear algebra works on the full system

Construction

```python
from py3plex.core import multinet

network = multinet.multi_layer_network()
# ... add nodes and edges ...

# Get supra-adjacency matrix (sparse by default for efficiency)
supra_adj = network.get_supra_adjacency_matrix(sparse=True)

# Dense version (for small networks only)
supra_adj_dense = network.get_supra_adjacency_matrix(sparse=False)
```

(continues on next page)

(continued from previous page)

```
# Shape: (total_nodes, total_nodes)
# where total_nodes = sum of nodes across all layers
print(f"Shape: {supra_adj.shape}")
```

Memory Implications

The supra-adjacency matrix has size $(N \times L)^2$ where N is nodes per layer and L is layers:

- **Small network (100 nodes, 3 layers):** 300×300 = 90,000 entries → ~720 KB dense
- **Medium network (1,000 nodes, 5 layers):** 5,000×5,000 = 25 million entries → ~200 MB dense
- **Large network (10,000 nodes, 10 layers):** 100,000×100,000 = 10 billion entries → impossible dense

Always use sparse=True for networks with more than ~1,000 total node-layer pairs. Sparse matrices only store non-zero entries, reducing memory from $O(N^2L^2)$ to O(edges + coupling).

Visualization

Visualize the supra-adjacency matrix structure:

```
from py3plex.core import multinet, random_generators
import matplotlib.pyplot as plt

# Generate a small multilayer network
network = random_generators.random_multilayer_ER(
    num_nodes=50, num_layers=3, probability=0.1, directed=False
)

# Visualize the supra-adjacency matrix
network.visualize_matrix({"display": True})
```

The visualization shows the block structure with:

- Diagonal blocks: intra-layer connections
- Off-diagonal blocks: inter-layer connections

How py3plex Uses NetworkX Under the Hood

py3plex is built on top of NetworkX, using it as the storage and algorithm backend. Understanding this architecture helps you leverage the full NetworkX ecosystem.

The Role of MultiGraph/MultiDiGraph

Internally, py3plex stores all nodes and edges in a single NetworkX `MultiGraph` (for undirected) or `MultiDiGraph` (for directed networks). The multilayer structure is encoded through the node representation:

```
# What you write in py3plex:
network.add_edges([
```

(continues on next page)

(continued from previous page)

```
    ['Alice', 'friends', 'Bob', 'friends', 1],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1],
], input_type="list")

# What's actually stored in NetworkX:
# Nodes: ('Alice', 'friends'), ('Bob', 'friends'),
#        ('Alice', 'colleagues'), ('Bob', 'colleagues')
# Edges: (('Alice', 'friends'), ('Bob', 'friends')),
#        (('Alice', 'colleagues'), ('Bob', 'colleagues'))
```

This means:

1. **All NetworkX algorithms work directly** on the underlying graph
2. **Node-layer tuples are just nodes** to NetworkX—it doesn't "know" about layers
3. **Attributes are stored as NetworkX attributes** and persist across operations

Attribute Propagation

When you add nodes or edges with attributes, they're stored in the NetworkX graph:

```
# Node attributes
network.core_network.nodes[('Alice', 'friends')]['age'] = 30
network.core_network.nodes[('Alice', 'friends')]['role'] = 'admin'

# Edge attributes
network.core_network[('Alice', 'friends')][('Bob', 'friends')]['weight'] = 0.8
network.core_network[('Alice', 'friends')][('Bob', 'friends')]['type'] =
↪'close'
```

Attributes are preserved when you:

- Extract subnetworks (filtered nodes keep their attributes)
- Iterate over nodes/edges with `data=True`
- Save and load networks (in formats that support attributes)

Using NetworkX Functions Directly

Because `network.core_network` is a standard NetworkX graph, you can use any NetworkX function:

```
import networkx as nx

G = network.core_network

# Standard NetworkX algorithms
betweenness = nx.betweenness_centrality(G)
pagerank = nx.pagerank(G)
components = list(nx.connected_components(G.to_undirected()))

# Shortest paths (work across layers if inter-layer edges exist)
```

(continues on next page)

(continued from previous page)

```
path = nx.shortest_path(G, ('Alice', 'friends'), ('Bob', 'colleagues'))

# Community detection (treats node-layer pairs as nodes)
communities = nx.community.louvain_communities(G)
```

Caveat: NetworkX doesn't know about layer semantics. It treats ('Alice', 'friends') and ('Alice', 'colleagues') as unrelated nodes. py3plex's multilayer algorithms add the layer-aware logic.

Design Trade-offs

Node-Layer Pairs vs. Alternative Representations

py3plex's node-layer pair representation is one of several possible designs. Here's why it was chosen:

Alternative 1: Separate graphs per layer

```
# Instead of py3plex:
layer1 = nx.Graph()
layer2 = nx.Graph()
layer1.add_edge('Alice', 'Bob')
layer2.add_edge('Alice', 'Bob')
```

Pros:

- Simple, familiar
- Easy layer-specific analysis

Cons:

- No representation of inter-layer edges
- Manual bookkeeping for cross-layer operations
- Can't compute multilayer paths or centrality directly

Alternative 2: Single graph with edge type attributes

```
# Instead of py3plex:
G = nx.MultiGraph()
G.add_edge('Alice', 'Bob', type='friends')
G.add_edge('Alice', 'Bob', type='colleagues')
```

Pros:

- Single graph structure
- Edges carry type information

Cons:

- Nodes can't have layer-specific attributes
- Same node in different contexts is the same node (can't have Alice be central in one layer and peripheral in another)
- Inter-layer edges to the same node are impossible

py3plex approach: Node-layer pairs

```
# py3plex representation:
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1],
], input_type="list")
```

Pros:

- Preserves both intra-layer and inter-layer structure
- Allows layer-specific node attributes
- Compatible with NetworkX algorithms
- Enables proper multilayer-specific operations
- Supports identity coupling and cross-layer edges

Cons:

- Node names are tuples (less intuitive for simple cases)
- Memory usage is multiplied by number of layers
- NetworkX algorithms treat node-layer pairs as independent nodes (need py3plex wrappers for multilayer semantics)

py3plex's choice: The node-layer pair approach was chosen because it correctly models multilayer semantics while maintaining NetworkX compatibility. The downsides (tuple names, memory) are acceptable trade-offs for the flexibility and correctness gained.

Consequences for Algorithm Implementation

The node-layer pair design has implications:

1. **Layer extraction is subsetting:** To get layer 1, you filter nodes where the layer component equals "layer1"
2. **Identity coupling is explicit:** Cross-layer edges between the same entity must be added explicitly (or automatically in multiplex mode)
3. **Supra-matrix is derivable:** The supra-adjacency matrix can be computed from the graph structure
4. **NetworkX algorithms work but need interpretation:** Betweenness on the full graph treats cross-layer edges as normal edges

Consequences for I/O

The node-layer pair representation affects serialization:

1. **Standard formats work:** GraphML, GML can store tuple nodes
2. **Human-readable formats need mapping:** Edgelists typically use separate columns for node and layer
3. **Arrow/Parquet are efficient:** Modern columnar formats handle the structure naturally

Network Construction

Creating Networks from Scratch

Method 1: Add edges directly

```
from py3plex.core import multinet

network = multinet.multi_layer_network()

# Add edges in list format: [source_node, source_layer, target_node, target_
↪layer, weight]
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1.0],
    ['Bob', 'friends', 'Carol', 'friends', 1.0],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1.0],
], input_type="list")
```

Method 2: Add nodes and edges separately

```
network = multinet.multi_layer_network()

# Add nodes explicitly
network.add_nodes([
    ('Alice', 'friends'),
    ('Bob', 'friends'),
    ('Alice', 'colleagues'),
    ('Bob', 'colleagues')
])

# Add edges between existing nodes
network.add_edges([
    [('Alice', 'friends'), ('Bob', 'friends'), 1.0],
    [('Alice', 'colleagues'), ('Bob', 'colleagues'), 1.0]
])
```

Method 3: Define layers first

```
network = multinet.multi_layer_network()

# Define layers explicitly
network.add_layer('friends')
network.add_layer('colleagues')

# Add content (nodes are created automatically with edges)
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1.0]
], input_type="list")
```

Loading from Files

py3plex supports multiple input formats. See ../user_guide/io_and_formats for complete documentation.

```
from py3plex.core import multinet

# From simple edge list (source target)
network = multinet.multi_layer_network().load_network(
    "data.edgelist",
    input_type="edgelist",
    directed=False
)

# From multilayer edge list (source target layer)
network = multinet.multi_layer_network().load_network(
    "data.multiedgelist",
    input_type="multiedgelist",
    directed=False
)

# From GraphML
network = multinet.multi_layer_network().load_network(
    "data.graphml",
    input_type="graphml"
)

# From NetworkX graph
import networkx as nx
G = nx.karate_club_graph()
network = multinet.multi_layer_network()
network.load_network_from_networkx(G)
```

Network Operations

Querying Network Elements

```
# Get all node-layer pairs
nodes = network.get_nodes(data=True)  # With attributes
nodes = network.get_nodes(data=False)  # Without attributes

# Get nodes in a specific layer
layer_nodes = network.get_nodes(layer='friends')

# Get all edges
edges = network.get_edges(data=True)

# Get neighbors of a node in a layer
neighbors = list(network.get_neighbors('Alice', layer_id='friends'))

# Get all layers
```

(continues on next page)

(continued from previous page)

```
layers = network.get_layers()
print(f"Layers: {layers}")
```

Basic Statistics

```
# Get basic network statistics
network.basic_stats()

# Output:
# Number of nodes: X
# Number of edges: Y
# Number of unique nodes (as node-layer tuples): Z
# Number of unique node IDs (across all layers): W
# Nodes per layer:
#   Layer 'friends': N1 nodes
#   Layer 'colleagues': N2 nodes
```

Subnetwork Extraction

```
# Extract a single layer
layer_1 = network.subnetwork(['friends'], subset_by="layers")

# Extract specific nodes (all their appearances)
node_subset = network.subnetwork(['Alice', 'Bob'], subset_by="node_names")

# Extract specific node-layer pairs
specific_pairs = network.subnetwork(
    [('Alice', 'friends'), ('Bob', 'friends')],
    subset_by="node_layer_names"
)
```

Matrix Representations

```
# Get supra-adjacency matrix (all layers stacked)
supra_adj = network.get_supra_adjacency_matrix(sparse=True)

# Get adjacency matrix for a single layer
layer_subgraph = network.subnetwork(['friends'], subset_by="layers")
layer_adj = nx.adjacency_matrix(layer_subgraph.core_network)
```

Integration with NetworkX

Full NetworkX Compatibility

Since py3plex builds on NetworkX, you can use any NetworkX algorithm:

```
import networkx as nx
from py3plex.core import multinet

# Create py3plex network
mlnet = multinet.multi_layer_network()
mlnet.add_edges([
    ['A', 'L1', 'B', 'L1', 1],
    ['B', 'L1', 'C', 'L1', 1],
], input_type="list")

# Access underlying NetworkX graph
G = mlnet.core_network

# Use any NetworkX function
betweenness = nx.betweenness_centrality(G)
print(f"Betweenness centrality: {betweenness}")

# Shortest paths
try:
    path = nx.shortest_path(G, ('A', 'L1'), ('C', 'L1'))
    print(f"Shortest path: {path}")
except nx.NetworkXNoPath:
    print("No path exists")

# Connected components
if not G.is_directed():
    components = list(nx.connected_components(G))
    print(f"Number of components: {len(components)}")
```

NetworkX Wrappers

py3plex provides convenience wrappers for common NetworkX functions:

```
# Extract a layer
layer_1 = network.subnetwork(['layer1'], subset_by="layers")

# Apply NetworkX algorithms via wrapper
degree_centrality = layer_1.monoplex_nx_wrapper("degree_centrality")
betweenness = layer_1.monoplex_nx_wrapper("betweenness_centrality")

# Results are dictionaries mapping node-layer pairs to values
print(f"Degree centrality: {degree_centrality}")
```

Converting Between Formats

```
# From NetworkX to py3plex
import networkx as nx
from py3plex.core import multinet
```

(continues on next page)

(continued from previous page)

```
G = nx.karate_club_graph()
mlnet = multinet.multi_layer_network()
# NetworkX graphs are imported as single-layer networks
mlnet.load_network_from_networkx(G)

# From py3plex to NetworkX (already a NetworkX graph!)
G_export = mlnet.core_network

# Save as standard NetworkX formats
nx.write_graphml(G_export, "output.graphml")
nx.write_gml(G_export, "output.gml")
```

Performance Considerations

Sparse vs. Dense Matrices

For large networks, always use sparse matrix representations:

```
# Good: Sparse matrix (efficient for large networks)
supra_adj = network.get_supra_adjacency_matrix(sparse=True)

# Bad: Dense matrix (memory-intensive)
# Only use for small networks (< 1000 nodes)
supra_adj_dense = network.get_supra_adjacency_matrix(sparse=False)
```

Memory Usage

Node-layer pairs mean that a node appearing in L layers occupies L times the memory of a single-layer network. For networks with many layers:

- Consider layer-specific analysis when possible
- Use subnetwork extraction to work with smaller portions
- See *Performance and Scalability Best Practices* for optimization strategies

Tensor-Like Indexing

You can access nodes and edges using tensor-like notation:

```
from py3plex.core import multinet, random_generators

# Create a network
network = random_generators.random_multilayer_ER(100, 3, 0.05, directed=False)

# Get some nodes and edges
some_nodes = list(network.get_nodes())[0:5]
some_edges = list(network.get_edges())[0:5]

# Access node directly (returns node attributes)
node_data = network[some_nodes[0]]
```

(continues on next page)

(continued from previous page)

```
print(f"Node {some_nodes[0]} data: {node_data}")

# Access edge (returns edge attributes)
edge_data = network[some_edges[0][0]][some_edges[0][1]]
print(f"Edge data: {edge_data}")
```

Design Principles

py3plex follows several key design principles:

1. NetworkX Compatibility

All operations maintain compatibility with NetworkX, allowing seamless use of the rich NetworkX ecosystem.

2. Lazy Evaluation

Expensive operations like supra-adjacency matrix construction are computed on demand and cached when appropriate.

3. Graceful Degradation

Optional features (like Infomap community detection or advanced visualizations) fail gracefully with informative error messages if dependencies are missing.

4. Extensibility

The modular design makes it easy to extend py3plex with custom algorithms and visualizations.

5. Flexibility

Multiple ways to construct and manipulate networks to suit different workflows and use cases.

Comparison with Other Representations

Why Node-Layer Pairs?

Alternative representations exist, but py3plex's node-layer pair approach offers key advantages:

Alternative: Separate graphs per layer

Drawback: Loses inter-layer information, requires manual bookkeeping for cross-layer operations.

Alternative: Single graph with edge type attributes

Drawback: Cannot distinguish node context across layers, loses layer-specific node attributes.

py3plex approach: Node-layer pairs

Advantages:

- Preserves both intra-layer and inter-layer structure
- Allows layer-specific node attributes
- Compatible with NetworkX algorithms
- Efficient for multilayer-specific operations

What You Learned

This chapter covered the internal architecture of py3plex:

Core concepts:

- The `multi_layer_network` class as the central data structure
- Node-layer pairs `(node_id, layer_id)` as the fundamental representation
- The difference between `network_type='multilayer'` and `network_type='multiplex'`

Data structures:

- The underlying NetworkX MultiGraph/MultiDiGraph
- Layer mappings and layer extraction
- The supra-adjacency matrix and its block structure

Design trade-offs:

- Why node-layer pairs were chosen over alternatives
- Memory implications of the representation
- When to use sparse vs. dense matrices

NetworkX integration:

- Accessing the core graph with `network.core_network`
- Using any NetworkX algorithm directly
- The `monoplex_nx_wrapper()` convenience method

What's Next?

- *Design Principles* — The philosophy behind py3plex's API design
- *Algorithm Landscape* — Overview of available algorithms
- ../user_guide/networks — Practical guide to network operations
- ../user_guide/statistics — Computing multilayer statistics
- ../user_guide/io_and_formats — Complete I/O documentation

Academic References:

- De Domenico et al. (2013). "Mathematical formulation of multilayer networks." *Physical Review X* 3(4): 041022.
- Kivelä et al. (2014). "Multilayer networks." *Journal of Complex Networks* 2(3): 203-271.

Next chapter: :doc:`design_principles` — understanding py3plex's design philosophy.

3.4.4 Design Principles

In which we explore the philosophy behind py3plex's API, understand why certain design choices were made, and learn how these principles make the library easier to use and extend.

Every library makes choices. Why is the API shaped this way? Why does this function return that data structure? Why do you import from here and not there?

This chapter explains the design principles behind py3plex. Understanding them will help you:

- **Use the library more effectively** — work *with* the design, not against it
- **Debug unexpected behavior** — understand why things work the way they do
- **Extend the library** — add your own algorithms that fit naturally

Core Philosophy

Simple, Off-the-Shelf Functionality

Principle: Provide ready-to-use tools that work out of the box with minimal configuration.

In practice:

```
from py3plex.core import multinet

# Simple, intuitive API
network = multinet.multi_layer_network()
network.add_edges([['A', 'L1', 'B', 'L1', 1]], input_type="list")
network.basic_stats()  # Works immediately
```

Rationale: Researchers and practitioners need tools that are easy to adopt without extensive setup or configuration. py3plex aims to minimize the barrier to entry for multilayer network analysis.

NetworkX Compatibility

Principle: Build on NetworkX rather than reinventing the wheel.

In practice:

```
import networkx as nx

# Direct access to NetworkX graph
G = network.core_network

# Use any NetworkX function
betweenness = nx.betweenness_centrality(G)
communities = nx.community.louvain_communities(G)
```

Rationale: NetworkX is the de facto standard for network analysis in Python. By building on it, py3plex:

- Leverages a mature, well-tested codebase
- Provides access to hundreds of NetworkX algorithms
- Ensures interoperability with the broader Python ecosystem
- Reduces learning curve for users familiar with NetworkX

Modular Architecture

Principle: Separate concerns into independent, composable modules.

Structure:

```
py3plex/
├── core/                    # Data structures
├── algorithms/              # Analysis methods
│   ├── community_detection/
│   ├── statistics/
│   └── multilayer_algorithms/
├── visualization/           # Plotting
└── wrappers/                # High-level interfaces
```

In practice:

```python
# Import only what you need
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
from py3plex.visualization.multilayer import hairball_plot
```

Rationale: Modular design allows users to:

- Import only the functionality they need
- Understand and maintain code more easily
- Extend the library with custom modules
- Mix and match components flexibly

Implementation Principles

Flexibility

Principle: Support multiple ways to accomplish tasks, accommodating different workflows.

Examples:

```python
# Multiple ways to create networks

# Method 1: Direct edge addition
network.add_edges([['A', 'L1', 'B', 'L1', 1]], input_type="list")

# Method 2: Load from file
network.load_network("data.edgelist", input_type="edgelist")

# Method 3: From NetworkX
network.load_network_from_networkx(nx_graph)

# Multiple ways to access nodes
nodes1 = network.get_nodes()
nodes2 = list(network.core_network.nodes())
nodes3 = network.get_nodes(layer='L1')
```

Rationale: Different users have different preferences and workflows. Flexibility ensures py3plex adapts to the user's needs, not vice versa.

Lazy Evaluation

Principle: Compute expensive operations only when needed.

In practice:

```
# Supra-adjacency matrix is not computed on network creation
network = multinet.multi_layer_network()
network.add_edges([...])  # Fast

# Matrix is computed only when requested
supra_adj = network.get_supra_adjacency_matrix()  # Computed here

# Subsequent calls may use cached result (where appropriate)
```

Rationale: Many multilayer operations (e.g., matrix construction) are computationally expensive. Lazy evaluation:

- Improves performance for common operations
- Reduces memory usage
- Allows working with large networks when full matrices aren't needed

Graceful Degradation

Principle: Optional features should fail gracefully, not crash the entire application.

In practice:

```
# If Infomap is not installed
try:
    communities = infomap_communities(network)
except ImportError as e:
    print("Infomap not available. Using Louvain instead.")
    communities = louvain_partition(network)
```

Rationale: Not all users need all features. By degrading gracefully, py3plex:

- Allows users to install only what they need
- Reduces dependency bloat
- Provides helpful error messages
- Suggests alternatives when dependencies are missing

Explicit Over Implicit

Principle: Make behavior explicit and predictable, avoiding surprising defaults.

Good examples:

```
# Explicit parameter names
network.load_network(
    "data.txt",
    input_type="edgelist",      # Explicit format
    directed=False              # Explicit directionality
```

(continues on next page)

(continued from previous page)

```
)

# Explicit layer specification
neighbors = network.get_neighbors('Alice', layer_id='friends')
```

Rationale: Explicit code is:

- Easier to understand and debug
- Less prone to errors
- Self-documenting
- More maintainable

Performance Considerations

Sparse Representations

Principle: Use sparse data structures for large networks.

In practice:

```
# Sparse matrix by default (efficient for large networks)
supra_adj = network.get_supra_adjacency_matrix(sparse=True)

# Dense only for small networks or specific algorithms
supra_adj_dense = network.get_supra_adjacency_matrix(sparse=False)
```

Rationale: Real-world networks are typically sparse (few edges relative to potential edges). Sparse representations:

- Reduce memory usage by orders of magnitude
- Enable analysis of networks with millions of nodes
- Maintain computational efficiency

Efficient Data Structures

Principle: Choose appropriate data structures for common operations.

Examples:

- NetworkX graphs for topology (edge lookups, traversals)
- Dictionaries for layer mappings (O(1) lookups)
- Sparse matrices for linear algebra
- Sets for membership testing

Rationale: The right data structure makes operations fast and memory-efficient.

Algorithmic Defaults

Principle: Provide sensible default parameters based on empirical research.

In practice:

```
# Default parameters work well for most cases
partition = louvain_multilayer(
    network,
    gamma=1.0,      # Standard resolution
    omega=1.0,      # Balanced inter-layer coupling
    random_state=42  # Reproducibility
)
```

Rationale: Good defaults:

- Make the library easy to use for beginners
- Reduce parameter tuning overhead
- Are based on published research and empirical validation

Extensibility

Plugin-Friendly Architecture

Principle: Make it easy to add custom functionality.

In practice:

```
# Easy to add custom algorithms
def my_centrality(ml_network):
    """Custom centrality measure."""
    G = ml_network.core_network
    # Implement your algorithm
    return centrality_scores

# Easy to add custom visualizations
def my_plot(ml_network, **kwargs):
    """Custom visualization."""
    # Use py3plex drawing machinery
    from py3plex.visualization import drawing_machinery as dm
    # Implement your plot
    pass
```

Rationale: Research needs evolve. Extensibility ensures py3plex can grow with new methods without requiring core library changes.

Clear Interfaces

Principle: Define clear, stable APIs for modules.

In practice:

```
# All statistics follow same interface
from py3plex.algorithms.statistics import multilayer_statistics as mls

density = mls.layer_density(network, layer)
activity = mls.node_activity(network, node)
overlap = mls.edge_overlap(network, layer1, layer2)
```

(continues on next page)

(continued from previous page)

```
# Consistent parameter names and return types
```

Rationale: Clear interfaces make py3plex:

- Easier to learn (patterns repeat)
- More predictable
- More maintainable
- Easier to extend

Documentation and Testing

Comprehensive Documentation

Principle: Every feature should be documented with examples.

In practice:

- Detailed docstrings for all public functions
- Code examples in documentation
- Real-world use cases
- API reference with parameter descriptions

Rationale: Good documentation:

- Reduces barrier to entry
- Serves as a reference for experienced users
- Helps maintainers understand code
- Acts as specification

Extensive Testing

Principle: Test all core functionality and edge cases.

In practice:

```
# Comprehensive test suite
make test          # Unit tests
make benchmark     # Performance tests
make fuzz-property # Property-based tests
```

Rationale: Tests ensure:

- Code works as intended
- Changes don't break existing functionality
- Edge cases are handled correctly
- Performance regressions are caught

Interoperability

Standard Formats

Principle: Support widely-used data formats.

Formats supported:

- EdgeList (simple, human-readable)
- GraphML (XML-based, preserves attributes)
- GML (Graph Modeling Language)
- Pickle (NetworkX native)
- JSON/Arrow/Parquet (modern, efficient)

Rationale: Standard formats enable:

- Data sharing between tools
- Integration with other software
- Long-term data preservation

Cross-Platform Compatibility

Principle: Work consistently across operating systems.

Tested on:

- Linux (Ubuntu 20.04, 22.04)
- macOS (11, 12, 13)
- Windows (Server 2019, 2022)

Rationale: Users work on different platforms. Cross-platform support ensures py3plex works everywhere.

Research-Oriented Design

Citation and Attribution

Principle: Make it easy to cite algorithms and give proper attribution.

In practice:

```
# Algorithms include citations in docstrings
help(louvain_multilayer)
# Docstring includes:
# References:
#   Mucha et al. (2010). "Community structure in time-dependent..."
```

See: ../reference/citation_and_acknowledgements

Rationale: Proper attribution:

- Gives credit to algorithm developers
- Helps users find original papers
- Supports reproducibility

Reproducibility

Principle: Support reproducible research.

In practice:

```
# Random seed parameters for reproducibility
partition = louvain_multilayer(
    network,
    random_state=42  # Reproducible results
)

walks = generate_walks(
    network,
    seed=42  # Reproducible walks
)
```

Rationale: Reproducibility is fundamental to science. Seed parameters ensure:

- Results can be verified
- Comparisons are fair
- Bugs can be diagnosed

Validation and Benchmarking

Principle: Validate algorithms against published results.

In practice:

- Benchmark suite comparing to reference implementations
- Validation against known network properties
- Performance metrics for large networks

Rationale: Validation ensures:

- Implementations are correct
- Performance is acceptable
- Results match published literature

Practical Implications

For Users

These design principles mean you can:

1. **Start quickly** with minimal setup
2. **Use NetworkX knowledge** and tools directly
3. **Choose your workflow** with multiple approaches
4. **Work efficiently** with large networks
5. **Extend easily** with custom algorithms
6. **Trust results** through validation and testing

For Contributors

When contributing to py3plex, follow these principles:

1. **Maintain NetworkX compatibility** in new features
2. **Document thoroughly** with examples
3. **Test extensively** including edge cases
4. **Provide sensible defaults** based on research
5. **Fail gracefully** with helpful error messages
6. **Follow existing patterns** for consistency

See ../dev/contributing for detailed guidelines.

Comparison with Other Libraries

py3plex vs. Pure NetworkX

NetworkX:

- Excellent for single-layer networks
- Lacks native multilayer support
- General-purpose graph library

py3plex:

- Native multilayer data structures
- Layer-aware algorithms
- Specialized multilayer visualizations
- Built on NetworkX (inherits all features)

py3plex vs. Other Multilayer Tools

Other tools (e.g., muxViz, pymnet):

- Often specialized or platform-specific
- May have steep learning curves
- Limited algorithm libraries

py3plex:

- Pure Python (easy installation)
- Comprehensive algorithm library
- NetworkX integration
- Active development
- Research-backed

What You Learned

This chapter covered the design philosophy behind py3plex:

Core principles:

- **Simple, off-the-shelf functionality** — work out of the box with minimal configuration
- **NetworkX compatibility** — leverage the entire NetworkX ecosystem
- **Modular architecture** — import only what you need

Implementation principles:

- **Flexibility** — multiple ways to accomplish tasks
- **Lazy evaluation** — expensive operations computed on demand
- **Graceful degradation** — optional features fail with helpful messages
- **Explicit over implicit** — clear, predictable behavior

Performance principles:

- **Sparse representations** — efficient for large networks
- **Sensible defaults** — good starting parameters based on research

Quality principles:

- **Comprehensive documentation** — every feature has examples
- **Extensive testing** — validated against published results
- **Reproducibility** — seed parameters for deterministic results

What's Next?

- *Algorithm Landscape* — Overview of available algorithms
- ../user_guide/networks — Practical guide to network operations
- *Development Guide* — Contributing to py3plex

Academic References:

- Škrlj et al. (2019). "Py3plex toolkit for visualization and analysis of multilayer networks." *Applied Network Science* 4(1): 94.

Next chapter: :doc:`algorithm_landscape` — a tour of py3plex's algorithmic capabilities.

3.4.5 Algorithm Landscape

This guide provides a narrative overview of the algorithm families available in py3plex, helping you understand when to use each type of algorithm for multilayer network analysis.

Overview

py3plex provides algorithms in seven main categories:

1. **Community Detection** - Finding groups of densely connected nodes
2. **Centrality & Importance** - Measuring node and edge importance
3. **Network Statistics** - Computing network-level and layer-specific metrics
4. **Random Walks & Embeddings** - Node representations and sampling

5. **Node Ranking & Classification** - Ordering and classifying nodes
6. **Visualization & Layout** - Rendering and spatial arrangement
7. **Benchmarking & Generation** - Testing and synthetic network creation

For detailed parameter lists and API reference, see *Algorithm Roadmap*.

Community Detection

Goal: Identify groups of nodes that are more densely connected to each other than to the rest of the network.

When to Use

Use community detection when you want to:

- Discover functional modules in biological networks
- Find social groups in multilayer social networks
- Identify topical clusters in citation networks
- Understand organizational structure
- Detect anomalies (nodes that don't fit any community)

Algorithm Families

Modularity-Based Methods

Best for: Fast community detection on large networks

Examples:

- **Louvain** - Fast, greedy modularity optimization
- **Leiden** - Improved version of Louvain with better stability
- **Multilayer Louvain** - Extends Louvain to multiple layers with inter-layer coupling

Trade-offs:

- ✓ Fast (scales to millions of nodes)
- ✓ Easy to use (minimal parameters)
- Resolution limit (may miss small communities)
- Stochastic (different runs give different results)

```
from py3plex.algorithms.community_detection import community_louvain

# Single-layer Louvain
partition = community_louvain.best_partition(network.core_network)

# Multilayer Louvain
from py3plex.algorithms.community_detection.multilayer_modularity import (
    louvain_multilayer
)
partition = louvain_multilayer(network, gamma=1.0, omega=1.0)
```

Flow-Based Methods

Best for: Finding communities based on information flow

Examples:

- **Infomap** - Minimizes description length of random walks
- **Walktrap** - Based on random walk distances

Trade-offs:

- ✓ Theoretically grounded (information theory)
- ✓ Finds hierarchical structure
- Slower than modularity methods
- May require external binaries

```
from py3plex.algorithms.community_detection import community_wrapper

# Infomap (requires binary)
partition = community_wrapper.infomap_communities(
    network.core_network,
    binary_path="/path/to/infomap"
)
```

Overlapping Community Detection

Best for: Networks where nodes belong to multiple communities

Examples:

- **NoRC** - Network of Ranked Communities
- **Clique Percolation** - Based on overlapping cliques

Trade-offs:

- ✓ More realistic for many real networks
- ✓ Captures multi-role nodes
- More complex to interpret
- Computationally expensive

```
from py3plex.algorithms.community_detection import community_wrapper

# NoRC overlapping communities
partition = community_wrapper.NoRC_communities(network, alpha=0.5)
```

Choosing a Community Detection Method

Use this decision tree:

```
Do you need overlapping communities?
├─ Yes → NoRC or Clique Percolation
└─ No
   └─ Is your network multilayer?
      ├─ Yes → Multilayer Louvain or Leiden
      └─ No
```

(continues on next page)

(continued from previous page)

```
        └ What's most important?
            ├ Speed → Louvain
            ├ Stability → Leiden
            └ Theory → Infomap
```

Centrality & Importance

Goal: Quantify the importance of nodes or edges in the network.

When to Use

Use centrality measures when you want to:

- Identify influential nodes (e.g., key proteins, opinion leaders)
- Find bottlenecks in flow networks
- Prioritize nodes for intervention or study
- Understand network vulnerability
- Analyze node roles across layers

Algorithm Families

Degree-Based Centrality

Measures: Local connectivity

Examples:

- **Degree Centrality** - Number of connections
- **Multilayer Degree** - Degree summed across layers
- **Versatility** - Participation across layers

When to use:

- Quick assessment of connectivity
- Identifying hubs
- Local importance measures

```python
import networkx as nx

# Single-layer degree centrality
degree_cent = nx.degree_centrality(network.core_network)

# Multilayer versatility
from py3plex.algorithms.statistics import multilayer_statistics as mls
versatility = mls.versatility_centrality(network, centrality_type='degree')
```

Distance-Based Centrality

Measures: Global position in network

Examples:

- **Closeness Centrality** - Average distance to all other nodes

- **Betweenness Centrality** - Fraction of shortest paths through node
- **Harmonic Centrality** - Harmonic mean of distances

When to use:

- Identifying central nodes
- Finding information brokers
- Understanding information flow

```
# Betweenness centrality
betweenness = nx.betweenness_centrality(network.core_network)

# Closeness centrality
closeness = nx.closeness_centrality(network.core_network)
```

Eigenvector-Based Centrality

Measures: Importance based on connections to important nodes

Examples:

- **Eigenvector Centrality** - First eigenvector of adjacency matrix
- **PageRank** - Google's ranking algorithm
- **HITS** - Hubs and authorities

When to use:

- Ranking web pages or citations
- Finding influential nodes in social networks
- Authority/hub analysis

```
# PageRank
pagerank = nx.pagerank(network.core_network)

# Eigenvector centrality
eigenvector = nx.eigenvector_centrality(network.core_network)
```

Choosing a Centrality Measure

Consider these questions:

1. **Local vs. Global?** - Local → Degree centrality - Global → Closeness or betweenness
2. **Direct connections or influence?** - Direct → Degree - Influence → PageRank or eigenvector
3. **Network type?** - Directed → PageRank or HITS - Undirected → Any method - Weighted → Use weight-aware versions
4. **Computational cost?** - Large network → Degree or PageRank (fast) - Small network → Betweenness (expensive but informative)

Network Statistics

Goal: Characterize network structure at the global, layer, or node level.

When to Use

Use network statistics when you want to:

- Compare networks quantitatively
- Understand structural properties
- Validate network models
- Monitor network evolution
- Detect anomalies

Statistic Families

Global Statistics

Examples:

- Number of nodes, edges, layers
- Density, clustering coefficient
- Average path length
- Degree distribution

When to use: Basic network characterization

```
# Basic stats
network.basic_stats()

# Clustering
import networkx as nx
clustering = nx.average_clustering(network.core_network)

# Density
from py3plex.algorithms.statistics import multilayer_statistics as mls
density = mls.layer_density(network, 'layer1')
```

Layer-Specific Statistics

Examples:

- Layer density
- Layer similarity (Jaccard, correlation)
- Edge overlap between layers
- Inter-layer degree correlation

When to use: Comparing layers, understanding layer relationships

```
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Layer density
```

(continues on next page)

(continued from previous page)

```
density_l1 = mls.layer_density(network, 'layer1')

# Layer similarity
similarity = mls.layer_similarity(network, 'layer1', 'layer2', method='jaccard
↪')

# Edge overlap
overlap = mls.edge_overlap(network, 'layer1', 'layer2')
```

Node-Level Statistics

Examples:

- Node activity (presence across layers)
- Participation coefficient
- Cross-layer degree correlation

When to use: Characterizing node behavior across layers

```
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Node activity
activity = mls.node_activity(network, 'Alice')

# Participation coefficient
participation = mls.community_participation_coefficient(network, partition)
```

Random Walks & Embeddings

Goal: Generate node representations or sample the network through traversal.

When to Use

Use random walks and embeddings when you want to:

- Create low-dimensional node representations for ML
- Sample large networks efficiently
- Measure node similarity
- Generate features for classification/clustering
- Understand structural equivalence

Algorithm Families

Random Walk Algorithms

Examples:

- **Basic Random Walk** - Uniform random traversal
- **Node2Vec** - Biased walks (BFS/DFS-like)
- **DeepWalk** - Uniform random walks for embeddings

Parameters:

- `walk_length` - Steps per walk
- `num_walks` - Walks per node
- `p` (Node2Vec) - Return parameter
- `q` (Node2Vec) - In-out parameter

```
from py3plex.algorithms.general.walkers import generate_walks, node2vec_walk

# Basic walks
walks = generate_walks(
    network.core_network,
    num_walks=10,
    walk_length=80
)

# Node2Vec walks (BFS-like: stay local)
walks_bfs = generate_walks(
    network.core_network,
    num_walks=10,
    walk_length=80,
    p=1.0, q=2.0  # BFS-like
)

# Node2Vec walks (DFS-like: explore)
walks_dfs = generate_walks(
    network.core_network,
    num_walks=10,
    walk_length=80,
    p=1.0, q=0.5  # DFS-like
)
```

Embedding Methods

Examples:

- **Node2Vec** - Skip-gram on random walks
- **DeepWalk** - Word2Vec on uniform walks
- **LINE** - First/second-order proximity preservation

When to use:

- Node classification
- Link prediction
- Visualization in 2D/3D
- Clustering
- Similarity computation

```
from py3plex.wrappers import node2vec_embedding

# Generate embeddings
embeddings = node2vec_embedding.generate_embeddings(
```

(continues on next page)

(continued from previous page)

```
    network.core_network,
    dimensions=128,
    walk_length=80,
    num_walks=10,
    p=1.0,
    q=1.0
)

# Use embeddings for ML
# embeddings is a numpy array: (num_nodes, dimensions)
```

Choosing Walk Parameters

walk_length:

- Short (10-20): Fast, local neighborhood
- Medium (40-80): Balanced, standard choice
- Long (100+): Global structure, slower

p (return parameter):

- $p < 1$: More likely to return to previous node
- $p = 1$: Balanced
- $p > 1$: Less likely to return

q (in-out parameter):

- $q < 1$: DFS-like (explore distant nodes)
- $q = 1$: Balanced
- $q > 1$: BFS-like (stay in local neighborhood)

Visualization & Layout

Goal: Create visual representations of multilayer networks.

When to Use

Use visualization when you want to:

- Explore network structure
- Present findings
- Communicate patterns
- Identify visual anomalies
- Create publication-ready figures

Layout Families

Force-Directed Layouts

Examples:

- Spring layout
- Fruchterman-Reingold
- Force Atlas

Best for: General-purpose visualization, showing community structure

```
from py3plex.visualization.multilayer import hairball_plot

hairball_plot(
    network.core_network,
    layout_algorithm="force",
    layout_parameters={"iterations": 50}
)
```

Specialized Multilayer Layouts

Examples:

- Diagonal projection
- Layered stacking
- Circular layer arrangement

Best for: Emphasizing multilayer structure

```
from py3plex.visualization.multilayer import draw_multilayer_default

# Diagonal layout for multilayer networks
draw_multilayer_default(
    [network],
    display=True,
    layout_style='diagonal'
)
```

Hierarchical Layouts

Examples:

- Reingold-Tilford tree
- Radial tree
- Sugiyama layered

Best for: Trees and DAGs

For complete visualization documentation, see ../user_guide/visualization.

Benchmarking & Generation

Goal: Create synthetic networks for testing and validation.

When to Use

Use synthetic network generation when you want to:

- Test algorithm performance
- Validate implementations
- Create controlled experiments
- Generate training data
- Benchmark scalability

Generator Families

Random Network Models

Examples:

- **Erdős-Rényi** - Random edges with fixed probability
- **Barabási-Albert** - Preferential attachment (scale-free)
- **Watts-Strogatz** - Small-world networks

```
from py3plex.core import random_generators

# Random multilayer ER network
network = random_generators.random_multilayer_ER(
    num_nodes=100,
    num_layers=3,
    probability=0.05,
    directed=False
)
```

Benchmark Networks

Examples:

- LFR benchmark (with ground-truth communities)
- Configuration model (preserving degree distribution)
- Block model (with planted partition)

Algorithm Combinations

Common Workflows

Community Detection + Visualization:

```
# Detect communities
partition = louvain_multilayer(network)

# Visualize with community colors
```

(continues on next page)

(continued from previous page)

```
from py3plex.visualization.multilayer import hairball_plot
from py3plex.visualization.colors import colors_default
from collections import Counter

top_communities = [c for c, _ in Counter(partition.values()).most_common(5)]
color_map = dict(zip(top_communities, colors_default[:5]))
node_colors = [color_map.get(partition.get(node), 'gray')
               for node in network.get_nodes()]

hairball_plot(network.core_network, node_colors, layout_algorithm="force")
```

Centrality + Ranking:

```
# Compute centrality
centrality = nx.betweenness_centrality(network.core_network)

# Rank nodes
ranked = sorted(centrality.items(), key=lambda x: x[1], reverse=True)

# Top 10
print("Top 10 nodes:", ranked[:10])
```

Embeddings + Classification:

```
# Generate embeddings
embeddings = node2vec_embedding.generate_embeddings(network.core_network)

# Use for classification (with sklearn)
from sklearn.ensemble import RandomForestClassifier
clf = RandomForestClassifier()
# clf.fit(embeddings, labels)
# predictions = clf.predict(embeddings_test)
```

Performance Considerations

Algorithm Complexity

Algorithm Family	Typical Complexity	Notes
Degree Centrality	O(m)	Fast, scales well
Betweenness	O(nm) or O(nm + n^2log n)	Expensive for large networks
Louvain	O(n log n)	Fast community detection
Node2Vec	O(k · l · n)	k=walks, l=length, n=nodes
Force Layout	O(n^2)	Slow for >1000 nodes

When working with large networks:

- Use approximate algorithms (e.g., sampling for betweenness)
- Consider layer-by-layer analysis

- Use sparse representations
- See *Performance and Scalability Best Practices*

Next Steps

- ../user_guide/community_detection - Detailed community detection guide
- ../user_guide/statistics - Complete statistics reference
- ../user_guide/random_walks_embeddings - Embeddings and walks
- ../user_guide/visualization - Visualization techniques
- *Algorithm Roadmap* - Detailed algorithm parameters and API

Academic References:

- Fortunato & Hric (2016). "Community detection in networks: A user guide." *Physics Reports* 659: 1-44.
- Grover & Leskovec (2016). "node2vec: Scalable feature learning for networks." *KDD*.
- Newman (2018). "Networks" (2nd ed.). Oxford University Press.

3.5 Part V: API & DSL Reference

Complete reference documentation for APIs and DSL syntax.

3.5.1 API & DSL Reference

Complete reference documentation for py3plex APIs, algorithms, DSL syntax, and configuration.

This section includes:

Python API Reference

- *API Documentation* — Full API documentation for all modules and classes
- *Algorithm Roadmap* — Complete algorithm reference with parameters and examples

DSL & Query Reference

- *DSL Reference* — Complete DSL syntax and operator reference
- *Layer Set Algebra* — Layer set algebra for composable layer selection
- *Configuration & Environment* — Configuration options and environment variables

Reading Paths:

- **Need DSL syntax?** → *DSL Reference* for complete grammar
- **Looking for an algorithm?** → *Algorithm Roadmap* for all available algorithms
- **Need API details?** → *API Documentation* for function signatures and parameters
- **Configuration options?** → *Configuration & Environment* for setup and tuning

Relation to other sections:

- **How-to Guides** show you *how* to accomplish tasks. This Reference section documents *what* each function does.
- **Concepts** explain *why* algorithms work. This section provides precise API details.
- **Examples** show complete workflows. This section provides isolated reference information.

3.5.2 Algorithm Roadmap

This page provides a **comprehensive overview** of all algorithms implemented in py3plex, organized by category and showing relationships between different methods.

Purpose: Help you quickly find the right algorithm for your multilayer network analysis task.

Algorithm Categories

The algorithms in py3plex are organized into these main categories:

1. *Community Detection Algorithms* - Finding community structure
2. *Centrality Measures* - Computing node importance
3. *Network Statistics* - Network-level metrics
4. *Node Ranking & Classification* - Ranking and classification methods
5. *Random Walks & Embeddings* - Random walk-based methods
6. *Visualization Algorithms* - Layout and rendering
7. *Benchmark & Utilities* - Testing and evaluation

Community Detection Algorithms

Module: `py3plex.algorithms.community_detection`

These algorithms identify groups of densely connected nodes (communities) in multilayer networks.

Louvain Algorithm

Path: `community_detection.community_louvain`

Functions:

- `best_partition(graph, partition, weight, resolution, randomize, random_state)` - Main entry point
- `generate_dendrogram(graph, partition, weight, resolution, randomize, random_state)` - Hierarchical communities
- `modularity(partition, graph, weight)` - Calculate modularity score
- `partition_at_level(dendrogram, level)` - Extract partition at specific level
- `induced_graph(partition, graph, weight)` - Create quotient graph

Best for: Fast community detection on large single-layer networks

Complexity: O(n log n)

Runnable Example:

```
import networkx as nx
from py3plex.algorithms.community_detection import community_louvain as␣
↪louvain

# Create a network with clear community structure
G = nx.karate_club_graph()

# Detect communities
partition = louvain.best_partition(G, random_state=42)

# Calculate modularity
mod = louvain.modularity(partition, G)

print(f"Communities found: {len(set(partition.values()))}")
print(f"Modularity: {mod:.4f}")
```

Expected output:

```
Communities found: 4
Modularity: 0.4198
```

Related algorithms:

- *Multilayer Louvain* (multilayer extension)
- *Leiden Algorithm* (improved Louvain)

Infomap Algorithm

Path: `community_detection.community_wrapper`

Functions:

- `infomap_communities(network_input, binary_path, arguments)` - Main entry point
- `run_infomap(network, binary_path, arguments)` - Execute Infomap binary
- `parse_infomap(outfile)` - Parse Infomap output

Best for: Flow-based community detection, hierarchical structure

Complexity: O(m log n) where m is edges, n is nodes

Runnable Example:

```
import networkx as nx
from py3plex.algorithms.community_detection import community_wrapper

# Create a network
G = nx.karate_club_graph()

# Detect communities with Infomap (requires Infomap binary installed)
try:
    communities = community_wrapper.infomap_communities(
        G,
        binary_path="/usr/local/bin/Infomap",  # Adjust path as needed
```

(continues on next page)

(continued from previous page)

```
        arguments="--two-level"
    )
    print(f"Communities found: {len(set(communities.values()))}")
except FileNotFoundError:
    print("Infomap binary not found. Install from infomap.org")
```

Note: Infomap requires the external Infomap binary. Install from https://www.mapequation.org/infomap/

Related algorithms:

- *Louvain Algorithm* (faster alternative, no external dependencies)

Multilayer Louvain

Path: `community_detection.multilayer_modularity`

Functions:

- `louvain_multilayer(supra_adj, omega, resolution, seed)` - Multilayer Louvain
- `multilayer_modularity(supra_adj, partition, omega)` - Calculate multilayer modularity
- `build_supra_modularity_matrix(network, omega)` - Build modularity matrix

Best for: Community detection across multiple layers with inter-layer coupling

Complexity: O(n^2 L) where L is number of layers

Algorithm details: Implements Mucha et al. (2010) multilayer modularity optimization

Runnable Example:

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection import multilayer_modularity as␣
↪mlmod

# Create two-layer network
network = multinet.multi_layer_network(directed=False)
for i in range(20):
    network.add_nodes([
        {'source': i, 'type': 'layer1'},
        {'source': i, 'type': 'layer2'}
    ])

# Add edges creating two communities
for i in range(10):
    network.add_edges([
        {'source': i, 'target': (i+1)%10, 'source_type': 'layer1', 'target_
↪type': 'layer1'},
        {'source': i, 'target': (i+1)%10, 'source_type': 'layer2', 'target_
↪type': 'layer2'}
    ])
for i in range(10, 20):
    network.add_edges([
```

(continues on next page)

(continued from previous page)

```
        {'source': i, 'target': 10+(i+1)%10, 'source_type': 'layer1', 'target_
↪type': 'layer1'},
        {'source': i, 'target': 10+(i+1)%10, 'source_type': 'layer2', 'target_
↪type': 'layer2'}
    ])

# Build supra-adjacency matrix and detect communities
supra_adj = mlmod.build_supra_modularity_matrix(network, omega=1.0)
partition = mlmod.louvain_multilayer(supra_adj, omega=1.0, seed=42)

print(f"Communities found: {len(set(partition.values()))}")
```

Expected output:

```
Communities found: 2
```

Related algorithms:

- *Louvain Algorithm* (single-layer version)
- *Leiden Algorithm* (alternative optimizer)

Leiden Algorithm

Path: `community_detection.leiden_multilayer`

Functions:

- `leiden_multilayer(supra_adj, omega, resolution, n_iterations, seed)` - Main entry point

Data structures:

- `LeidenResult` - Holds partition results with modularity score

Best for: More stable community detection than Louvain

Complexity: O(n log n) with better quality guarantees

Related algorithms:

- *Louvain Algorithm* (predecessor)
- *Multilayer Louvain* (multilayer extension)

NoRC Algorithm

Path: `community_detection.NoRC` and `community_detection.community_wrapper`

Functions:

- `NoRC_communities(network, alpha)` - Wrapper function
- `NoRC_communities_main(matrix, alpha)` - Core implementation
- `page_rank_kernel(index_row)` - PageRank computation
- `create_tree(centers)` - Build community hierarchy

Best for: Networks with overlapping communities

Related algorithms:

- *Community Ranking* (related approach)

Community Measures

Path: `community_detection.community_measures`

Functions:

- `modularity(network, partition, weight)` - Calculate modularity
- `size_distribution(network_partition)` - Community size distribution
- `number_of_communities(network_partition)` - Count communities

Purpose: Evaluate quality of community partitions

Related algorithms: All community detection algorithms above

Community Ranking

Path: `community_detection.community_ranking`

Functions:

- `page_rank_kernel(index_row)` - PageRank-based ranking
- `create_tree(centers)` - Build ranking tree
- `return_infomap_communities(network)` - Get Infomap communities

Best for: Ranking nodes within detected communities

Related algorithms:

- *Node Ranking* (general ranking)

Multilayer Benchmarks

Path: `community_detection.multilayer_benchmark`

Functions:

- `generate_multilayer_lfr(n_nodes, n_layers, avg_degree, ...)` - LFR benchmark
- `generate_coupled_er_multilayer(n_nodes, n_layers, p_intra, p_inter)` - ER benchmark
- `generate_sbm_multilayer(sizes, p_matrix, n_layers, ...)` - SBM benchmark

Purpose: Generate synthetic multilayer networks with known community structure for testing

Related algorithms: All community detection algorithms

Centrality Measures

Module: `py3plex.algorithms.multilayer_algorithms.centrality`, `py3plex.algorithms.centrality_toolkit`

These algorithms measure the importance or influence of nodes in multilayer networks.

New in version 1.0: Centrality measures now support **first-class uncertainty quantification** via the `uncertainty` parameter. See ../tutorials/multilayer_centrality for details.

MultilayerCentrality Class

Path: `multilayer_algorithms.centrality.MultilayerCentrality`

Main class for computing various centrality measures.

Runnable Example:

```
from py3plex.core import multinet
from py3plex.algorithms.multilayer_algorithms.centrality import␣
↪MultilayerCentrality

# Create a small multilayer network
network = multinet.multi_layer_network(directed=False)

# Add nodes and edges to two layers
for i in range(5):
    network.add_nodes([
        {'source': i, 'type': 'layer1'},
        {'source': i, 'type': 'layer2'}
    ])

edges = [
    (0, 1), (1, 2), (2, 3), (3, 4),  # Layer 1: chain
    (0, 4), (1, 3), (2, 4)           # Layer 2: triangle-like
]
for src, tgt in edges:
    network.add_edges([
        {'source': src, 'target': tgt, 'source_type': 'layer1', 'target_type
↪': 'layer1'},
        {'source': src, 'target': tgt, 'source_type': 'layer2', 'target_type
↪': 'layer2'}
    ])

# Compute centrality measures
calc = MultilayerCentrality(network)

# Overlapping degree (sum across layers)
overlap_deg = calc.overlapping_degree_centrality()
print(f"Node 2 overlapping degree: {overlap_deg[(2, 'layer1')]:.2f}")

# PageRank
pr = calc.pagerank_centrality(alpha=0.85)
print(f"Node 2 PageRank: {pr[(2, 'layer1')]:.4f}")
```

Expected output:

```
Node 2 overlapping degree: 6.00
Node 2 PageRank: 0.0892
```

Basic Centrality Measures

Methods:

- `overlapping_degree_centrality()` - Sum of degrees across all layers
- `layer_specific_degree_centrality(layer)` - Degree within specific layer
- `multilayer_degree_centrality()` - Supra-adjacency degree
- `average_degree_centrality()` - Average degree across layers

Best for: Quick importance ranking, well-connected node identification

Complexity: O(n + m) where n is nodes, m is edges

Related measures:

- *Versatility Centrality* (cross-layer activity)

Eigenvector-Based Measures

Methods:

- `multiplex_eigenvector_centrality(max_iter, tol)` - Eigenvector centrality
- `pagerank_centrality(alpha, max_iter, tol)` - PageRank
- `katz_centrality(alpha, beta, max_iter, tol)` - Katz centrality
- `communicability_centrality(beta)` - Communicability centrality

Best for: Influence measure, recursive importance, prestige ranking

Complexity: O(m) per iteration, typically converges quickly

Related measures:

- *Hubs and Authorities (HITS)* (directed networks)
- *Multilayer PageRank (with Uncertainty)* (with uncertainty support)

Multilayer PageRank (with Uncertainty)

Path: `algorithms.centrality_toolkit.multilayer_pagerank`

New in version 1.0: Built-in PageRank with **first-class uncertainty quantification**.

Function signature:

```
multilayer_pagerank(
    network,
    alpha=0.85,
    max_iter=100,
    tol=1e-6,
    personalization=None,
    uncertainty=False,
    n_runs=None,
    resampling=None,
    random_seed=None
) -> StatSeries
```

Parameters:

- uncertainty: If True, estimate uncertainty via resampling
- n_runs: Number of runs for uncertainty estimation (default: 50)
- resampling: Strategy - SEED, PERTURBATION, BOOTSTRAP, JACKKNIFE
- random_seed: For reproducibility

Returns: StatSeries with mean, std, quantiles (if uncertainty=True)

Runnable Example:

```
from py3plex.algorithms.centrality_toolkit import multilayer_pagerank
from py3plex.uncertainty import ResamplingStrategy
from py3plex.core import multinet

# Create network
network = multinet.multi_layer_network(directed=False)
network.add_edges([
    ['A', 'L1', 'B', 'L1', 1.0],
    ['B', 'L1', 'C', 'L1', 1.0],
    ['A', 'L2', 'C', 'L2', 1.0]
], input_type='list')

# Deterministic PageRank
result = multilayer_pagerank(network)
print(f"Node scores: {result.mean}")
print(f"Is deterministic: {result.is_deterministic}")

# With uncertainty
result_unc = multilayer_pagerank(
    network,
    uncertainty=True,
    n_runs=50,
    resampling=ResamplingStrategy.PERTURBATION,
    random_seed=42
)
print(f"Mean scores: {result_unc.mean}")
print(f"Std deviations: {result_unc.std}")
print(f"95% CI: [{result_unc.quantiles[0.025]}, {result_unc.quantiles[0.975]}
↪]")
```

Expected output:

```
Node scores: [0.184 0.183 0.184 ...]
Is deterministic: True
Mean scores: [0.186 0.192 0.181 ...]
Std deviations: [0.019 0.021 0.018 ...]
95% CI: [[0.146 0.172 0.146 ...], [0.238 0.241 0.221 ...]]
```

Best for:

- Influence ranking with confidence intervals
- Robustness analysis under network perturbations
- Quantifying centrality uncertainty

Complexity:

- Deterministic: O(m) per iteration
- With uncertainty: O(m × n_runs)

Related measures:

- `MultilayerCentrality.pagerank_centrality()` (deterministic only)

Path-Based Measures

Methods:

- `multilayer_betweenness_centrality()` - Betweenness across layers
- `multilayer_closeness_centrality()` - Closeness across layers

Best for: Bridge identification, broadcasting efficiency

Complexity: O(nm) for unweighted, O(nm + n^2 log n) for weighted

Warning: Expensive for large networks (>1000 nodes)

Related measures:

- *Utility Functions* (batch computation)

Hubs and Authorities (HITS)

Path: `node_ranking.node_ranking.hubs_and_authorities`

Best for: Ranking nodes in directed networks (authority = important destination, hub = important source)

Related measures:

- PageRank in *Centrality Measures*

Utility Functions

Functions:

- `compute_all_centralities(network, include_path_based, include_advanced)` - Compute multiple measures

Purpose: Batch computation of multiple centrality measures

Versatility Centrality

Path: `algorithms.statistics.multilayer_statistics.versatility_centrality`

Function:

- `versatility_centrality(network, centrality_type)` - Cross-layer versatility measure

Best for: Measuring how uniformly a node's importance is distributed across layers

Complexity: O(m × L) where L is number of layers

Related measures:

- All centrality measures in *Centrality Measures*

Supra Matrix Function Centralities

Path: multilayer_algorithms.supra_matrix_function_centrality

Functions:

- communicability_centrality(supra_matrix, beta) - Matrix exponential based
- katz_centrality(supra_matrix, alpha, beta, max_iter, tol) - Resolvent based

Best for: Advanced centrality using matrix functions on supra-adjacency

Complexity: Depends on matrix operations, typically $O(n^2)$ or $O(n^3)$

Runnable Example:

```
import numpy as np
from py3plex.core import multinet
from py3plex.algorithms.multilayer_algorithms import supra_matrix_function_
↪centrality as smfc

# Create small network
network = multinet.multi_layer_network(directed=False)
for i in range(4):
    network.add_nodes([{'source': i, 'type': 'layer1'}])
network.add_edges([
    {'source': 0, 'target': 1, 'source_type': 'layer1', 'target_type': 'layer1
↪'},
    {'source': 1, 'target': 2, 'source_type': 'layer1', 'target_type': 'layer1
↪'},
    {'source': 2, 'target': 3, 'source_type': 'layer1', 'target_type': 'layer1
↪'},
])

# Build supra-adjacency matrix
supra_matrix = network.get_supra_adjacency_matrix()

# Compute communicability centrality
comm_cent = smfc.communicability_centrality(supra_matrix, beta=0.5)

print(f"Node 1 communicability: {comm_cent[(1, 'layer1')]:.4f}")
```

Expected output:

```
Node 1 communicability: 2.3562
```

Related measures:

- *Centrality Measures* (standard measures)

MultiXRank

Path: multilayer_algorithms.multixrank

Functions:

- multixrank_from_py3plex_networks(networks, config) - MultiXRank algorithm

Best for: Ranking in multiplex heterogeneous networks with bipartite layers

Algorithm details: Extends PageRank to multiplex/heterogeneous networks

Runnable Example:

```
from py3plex.core import multinet
from py3plex.algorithms.multilayer_algorithms import multixrank

# Create a simple bipartite-like structure
network = multinet.multi_layer_network(directed=True)

# Add users and items layers
users = ['u1', 'u2', 'u3']
items = ['i1', 'i2']

for u in users:
    network.add_nodes([{'source': u, 'type': 'users'}])
for i in items:
    network.add_nodes([{'source': i, 'type': 'items'}])

# Add interactions (users -> items)
network.add_edges([
    {'source': 'u1', 'target': 'i1', 'source_type': 'users', 'target_type':
↪'items'},
    {'source': 'u2', 'target': 'i1', 'source_type': 'users', 'target_type':
↪'items'},
    {'source': 'u2', 'target': 'i2', 'source_type': 'users', 'target_type':
↪'items'},
])

# Configure and run MultiXRank
config = {
    'alpha': 0.85,
    'max_iter': 100,
    'tol': 1e-6
}
scores = multixrank.multixrank_from_py3plex_networks([network], config)

print(f"User u1 score: {scores.get(('u1', 'users'), 0):.4f}")
```

Expected output:

```
User u1 score: 0.0833
```

Related algorithms:

- PageRank in *Centrality Measures*

Network Statistics

Module: `py3plex.algorithms.statistics`

These algorithms compute various statistical properties of multilayer networks.

Multilayer Statistics

Path: `statistics.multilayer_statistics`

Layer-level measures:

- `layer_density(network, layer)` - Density of specific layer
- `edge_overlap(network, layer_i, layer_j)` - Edge overlap between layers
- `inter_layer_coupling_strength(network, layer_i, layer_j)` - Coupling strength
- `inter_layer_degree_correlation(network, layer_i, layer_j)` - Degree correlation
- `inter_layer_assortativity(network, layer_i, layer_j)` - Assortativity between layers
- `layer_similarity(network, layer_i, layer_j, method)` - Layer similarity

Node-level measures:

- `node_activity(network, node)` - Activity across layers
- `degree_vector(network, node, weighted)` - Degree vector across layers
- `multilayer_clustering_coefficient(network, node)` - Multilayer clustering

Network-level measures:

- `interdependence(network, sample_size)` - Network interdependence
- `supra_laplacian_spectrum(network, k)` - Spectral properties
- `algebraic_connectivity(network)` - Second smallest Laplacian eigenvalue

Best for: Understanding multilayer network structure and properties

Complexity: Varies by measure, typically O(n) to O(n^2)

Related algorithms:

- *Multilayer Statistics (Detailed)* (simpler measures)
- *Topology Analysis* (topological properties)

Multilayer Statistics (Detailed)

See the full section above for complete details on multilayer statistics functions.

Basic Statistics

Path: `statistics.basic_statistics`

Functions:

- `identify_n_hubs(network, n, metric)` - Find top-n hub nodes
- `core_network_statistics(network)` - Comprehensive network statistics

Best for: Quick network overview, hub identification

Complexity: O(n + m) for most measures

Related algorithms:

- *Multilayer Statistics (Detailed)* (advanced measures)

Topology Analysis

Path: `statistics.topology`

Functions for analyzing topological properties of networks.

Purpose: Study structural properties like degree distributions, connected components, etc.

Related algorithms:

- *Multilayer Statistics (Detailed)*

Correlation Networks

Path: `statistics.correlation_networks`

Functions:

- `default_correlation_to_network(correlation_matrix, threshold)` - Convert correlation matrix to network
- `pick_threshold(matrix)` - Automatically select threshold

Best for: Creating networks from correlation/similarity matrices

Related algorithms:

- *Network Statistics* (analysis after construction)

Enrichment Analysis

Path: `statistics.enrichment_modules`

Functions:

- `compute_enrichment(term_dataset, annotation_database, ...)` - General enrichment
- `fet_enrichment_generic(...)` - Fisher's exact test enrichment
- `fet_enrichment_terms(...)` - Term enrichment
- `fet_enrichment_uniprot(...)` - UniProt annotation enrichment
- `calculate_pval(term, alternative)` - p-value calculation
- `multiple_test_correction(input_dataset)` - FDR correction

Best for: Finding over-represented terms/annotations in network communities

Related algorithms:

- *Community Detection Algorithms* (provides communities for enrichment)

Statistical Comparison

Path: `statistics.stats_comparison`

Functions: Various statistical tests for comparing networks or algorithms

Related modules:

- `statistics.bayesiantests` - Bayesian statistical tests
- `statistics.bayesian_distances` - Bayesian distance measures
- `statistics.critical_distances` - Critical difference diagrams

Best for: Comparing performance of different algorithms or network properties

Power Law Analysis

Path: `statistics.powerlaw`

Functions for testing and fitting power law distributions in networks.

Best for: Analyzing degree distributions, checking for scale-free properties

Related algorithms:

- *Multilayer Statistics (Detailed)* (degree distributions)

Node Ranking & Classification

Module: `py3plex.algorithms.node_ranking` and `py3plex.algorithms.network_classification`

Node Ranking

Path: `node_ranking.node_ranking`

Functions:

- `sparse_page_rank(graph, alpha, personalization, max_iter, tol)` - Sparse PageRank
- `hubs_and_authorities(graph)` - HITS algorithm
- `hub_matrix(graph)` - Hub matrix computation
- `authority_matrix(graph)` - Authority matrix computation
- `stochastic_normalization(matrix)` - Normalize for random walks
- `stochastic_normalization_hin(matrix)` - Normalize for heterogeneous networks

Best for: Ranking nodes by importance in directed networks

Complexity: O(m) per iteration for PageRank, O(nm) for HITS

Related algorithms:

- PageRank in *Centrality Measures*
- *Label Propagation* (classification)

Label Propagation

Path: `network_classification.label_propagation`

Functions:

- `label_propagation(adjacency, labels, alpha, max_iter, tol, normalization)` - Main algorithm
- `validate_label_propagation(...)` - Validation function
- `label_propagation_normalization(matrix)` - Normalization

- `normalize_initial_matrix_freq(mat)` - Frequency normalization
- `normalize_amplify_freq(mat)` - Amplified normalization
- `normalize_exp(mat)` - Exponential normalization

Best for: Semi-supervised node classification

Complexity: O(m × i) where i is number of iterations

Related algorithms:

- *Personalized PageRank (PPR)* (personalized version)
- Label propagation in *Community Detection Algorithms*

Personalized PageRank (PPR)

Path: `network_classification.PPR`

Functions:

- `construct_PPR_matrix(graph_matrix, parallel)` - Build PPR matrix
- `construct_PPR_matrix_targets(graph_matrix, targets, parallel)` - PPR for specific targets
- `validate_ppr(...)` - Validation function

Best for: Local network exploration, personalized node ranking

Complexity: O(n^2) for dense computation, can be approximated

Related algorithms:

- *Label Propagation* (similar propagation mechanism)
- PageRank in *Node Ranking*

Random Walks & Embeddings

Module: `py3plex.algorithms.general` and `py3plex.wrappers`

Random Walk Primitives

Path: `general.walkers`

Functions:

- `basic_random_walk(graph, start_node, walk_length)` - Simple random walk
- `node2vec_walk(graph, start_node, walk_length, p, q)` - Biased random walk
- `generate_walks(graph, num_walks, walk_length, strategy, ...)` - Generate multiple walks
- `general_random_walk(G, start_node, iterations, teleportation_prob)` - Random walk with restart
- `layer_specific_random_walk(network, start_node, start_layer, walk_length, ...)` - Multilayer walk

Best for: Foundation for embedding algorithms, network exploration

Complexity: O(L) per walk where L is walk length

Related algorithms:

- *Node2Vec Embedding* (uses these walks)
- *DeepWalk Embedding* (uses these walks)

Node2Vec Embedding

Path: `wrappers.node2vec_embedding`

Functions:

- `generate_embeddings(graph, dimensions, walk_length, num_walks, p, q, ...)` - Generate embeddings
- `train_node2vec(graph, ...)` - Training wrapper

Best for: Feature learning, link prediction, node classification

Complexity: O(r × l × V) where r is walks per node, l is walk length, V is number of vertices

Parameters:

- `p`: Return parameter (controls backtracking)
- `q`: In-out parameter (controls exploration vs exploitation)

Related algorithms:

- *DeepWalk Embedding* (special case with p=1, q=1)
- *Random Walk Primitives* (underlying walks)

DeepWalk Embedding

Implementation: Node2Vec with p=1, q=1

Best for: Simpler alternative to Node2Vec with uniform random walks

Related algorithms:

- *Node2Vec Embedding* (generalization)

Entanglement Analysis

Path: `multilayer_algorithms.entanglement`

Functions:

- `compute_entanglement_analysis(network)` - Full entanglement analysis
- `build_occurrence_matrix(network)` - Build node-layer occurrence matrix
- `compute_blocks(c_matrix)` - Compute block structure
- `compute_entanglement(block_matrix)` - Calculate entanglement metrics

Best for: Analyzing node-layer participation patterns

Purpose: Measure how nodes are entangled across layers

Related algorithms:

- *Versatility Centrality* (similar cross-layer analysis)

Visualization Algorithms

Module: `py3plex.visualization`

Layout Algorithms

Path: `visualization.layout_algorithms`

Available layouts:

- Force-directed layouts (Spring, Fruchterman-Reingold)
- Circular layout
- Spectral layout
- Hierarchical layout
- Diagonal projection (multilayer-specific)

Best for: Positioning nodes for visualization

Related algorithms:

- *Multilayer Visualization* (uses these layouts)

Multilayer Visualization

Path: `visualization.multilayer`

Main functions:

- `draw_multilayer_default(networks, ...)` - Default multilayer drawing
- `hairball_plot(network, layout_algorithm)` - Single-layer projection
- Diagonal projection plotting
- Layer-separated visualization

Best for: Visualizing multilayer network structure

Scalability:

- Diagonal projection: 10,000+ nodes
- Force-directed: <5,000 nodes
- Matrix view: 100,000+ nodes

Related algorithms:

- *Layout Algorithms* (positioning)

Drawing Machinery

Path: `visualization.drawing_machinery`

Low-level drawing functions and utilities.

Purpose: Support functions for rendering networks

Color Schemes

Path: `visualization.colors`

Functions for generating color schemes for communities, layers, node types.

Related algorithms:

- *Community Detection Algorithms* (uses colors)
- *Multilayer Visualization* (uses colors)

Embedding Visualization

Path: `visualization.embedding_visualization`

Functions for visualizing node embeddings in 2D/3D space.

Best for: Visualizing results of embedding algorithms

Related algorithms:

- *Node2Vec Embedding*
- *DeepWalk Embedding*

Benchmark & Utilities

Network Classification Benchmark

Path: `general.benchmark_classification`

Functions:

- `evaluate_oracle_F1(probs, Y_real)` - Evaluate classification performance

Best for: Evaluating node classification algorithms

Related algorithms:

- *Label Propagation*
- *Node2Vec Embedding*

Benchmark Visualizations

Path: `visualization.benchmark_visualizations`

Functions for visualizing algorithm comparison results.

Related algorithms:

- All algorithms (for benchmarking)

Algorithm Selection Guide

This section helps you choose the right algorithm category based on your task.

By Task Type

Community Detection:
→ See *Community Detection Algorithms*

Node Importance:
→ See *Centrality Measures*

Node Classification:
→ See *Label Propagation* or *Node2Vec Embedding*

Network Comparison:
→ See *Network Statistics*

Feature Learning:
→ See *Node2Vec Embedding*

Visualization:
→ See *Visualization Algorithms*

By Network Size

Small (<1,000 nodes):

- All algorithms work well
- Can use expensive path-based measures
- All visualization methods

Medium (1,000-10,000 nodes):

- Use Louvain over Infomap
- Avoid full betweenness centrality
- Use diagonal projection for visualization

Large (10,000-100,000 nodes):

- Degree-based measures only
- Louvain or label propagation
- Matrix visualizations

Very Large (>100,000 nodes):

- Degree, PageRank only
- Label propagation for communities
- Sparse matrix operations essential

By Network Type

Single-layer networks:

- Standard algorithms: Louvain, PageRank, betweenness
- See *Community Detection Algorithms* and *Centrality Measures*

Multiplex/multilayer networks:

- Multilayer-specific: *Multilayer Louvain*, *Versatility Centrality*
- Layer statistics: *Multilayer Statistics (Detailed)*
- Cross-layer measures

Directed networks:

- PageRank, HITS algorithm

- See *Node Ranking*

Weighted networks:

- All algorithms support weights
- Specify `weight` parameter

Temporal networks:

- Layer-based representation
- Time-respecting random walks: *Random Walk Primitives*

Algorithm Dependencies

This diagram shows how algorithms depend on each other:

```
Basic Network Operations
        |
        ├── Random Walks ───────────────→ Node2Vec/DeepWalk
        |                              |
        ├── Centrality Measures ────────→ Versatility
        |                              |
        ├── Community Detection ────────→ Community Measures
        |   |                          |
        |   ├─→ Modularity              └─→ Enrichment Analysis
        |   └─→ Community Ranking
        |
        ├── Statistics ─────────────────→ Benchmarks
        |                              └─→ Comparisons
        |
        └── Embeddings ─────────────────→ Visualization
                                       └─→ Classification
```

Common Algorithm Workflows

Community Detection Workflow

```python
# 1. Detect communities
from py3plex.algorithms.community_detection import louvain_multilayer
communities = louvain_multilayer(network.get_supra_adjacency_matrix())

# 2. Evaluate quality
from py3plex.algorithms.community_detection import multilayer_modularity
quality = multilayer_modularity(network.get_supra_adjacency_matrix(),
↪communities)

# 3. Analyze communities
from py3plex.algorithms.statistics import multilayer_statistics as mls
from py3plex.algorithms.statistics.enrichment_modules import compute_
↪enrichment

# 4. Visualize
```

(continues on next page)

(continued from previous page)

```
from py3plex.visualization.multilayer import draw_multilayer_default
draw_multilayer_default([network], communities=communities)
```

Centrality Analysis Workflow

```
# 1. Compute centralities
from py3plex.algorithms.multilayer_algorithms.centrality import␣
↪MultilayerCentrality
calc = MultilayerCentrality(network)

# 2. Multiple measures
degree = calc.overlapping_degree_centrality()
pagerank = calc.pagerank_centrality()
betweenness = calc.multilayer_betweenness_centrality()

# 3. Cross-layer analysis
from py3plex.algorithms.statistics import multilayer_statistics as mls
versatility = mls.versatility_centrality(network, centrality_type='degree')

# 4. Identify hubs
from py3plex.algorithms.statistics.basic_statistics import identify_n_hubs
hubs = identify_n_hubs(network, n=10, metric='degree')
```

Embedding Workflow

```
# 1. Generate walks
from py3plex.algorithms.general.walkers import generate_walks
walks = generate_walks(network.core_network, num_walks=10,
                       walk_length=80, strategy='node2vec')

# 2. Learn embeddings
from py3plex.wrappers.node2vec_embedding import generate_embeddings
embeddings = generate_embeddings(network.core_network,
                                 dimensions=128, p=1.0, q=1.0)

# 3. Use for classification
from py3plex.algorithms.network_classification.label_propagation import label_
↪propagation
# Or use embeddings directly with sklearn
```

Statistical Analysis Workflow

```
# 1. Layer-level statistics
from py3plex.algorithms.statistics import multilayer_statistics as mls
density = mls.layer_density(network, 'layer1')
overlap = mls.edge_overlap(network, 'layer1', 'layer2')
correlation = mls.inter_layer_degree_correlation(network, 'layer1', 'layer2')
```

(continues on next page)

(continued from previous page)

```
# 2. Node-level statistics
for node in important_nodes:
    activity = mls.node_activity(network, node)
    degree_vec = mls.degree_vector(network, node)

# 3. Network-level statistics
interdep = mls.interdependence(network)
connectivity = mls.algebraic_connectivity(network)
```

Further Reading

For detailed usage examples and parameter descriptions:

- *Algorithm Landscape* - Algorithm selection guide with complexity analysis
- ../user_guide/community_detection - Community detection examples
- ../user_guide/statistics - Centrality computation examples
- *Performance and Scalability Best Practices* - Performance optimization tips
- *API Documentation* - Complete API reference

Citation Information

When using specific algorithm families, please cite the original papers:

Louvain & Multilayer Modularity:
Blondel et al. (2008), Mucha et al. (2010)

Node2Vec:
Grover & Leskovec (2016)

Infomap:
Rosvall & Bergstrom (2008)

See citation_and_acknowledgements for complete citation information and BibTeX entries.

3.5.3 DSL Reference

Complete reference for the py3plex DSL (Domain-Specific Language) for querying multilayer networks.

Note

For task-oriented usage, see *How to Query Multilayer Graphs with the SQL-like DSL*. This page is a complete reference with all syntax and operators.

Overview

The py3plex DSL provides two interfaces:

1. **String syntax:** SQL-like queries for quick exploration
2. **Builder API:** Type-safe Python interface for production code

String Syntax Reference

Basic Structure

```
SELECT <target> [FROM <layers>] [WHERE <conditions>] [COMPUTE <metrics>]␣
↪[ORDER BY <field>] [LIMIT <n>]
```

Targets

- `nodes` — Select nodes
- `edges` — Select edges (experimental)

Layer Selection

```
FROM layer="layer_name"
FROM layers IN ("layer1", "layer2")
```

Conditions

Operators:

- `=` — Equal
- `>` — Greater than
- `<` — Less than
- `>=` — Greater than or equal
- `<=` — Less than or equal
- `!=` — Not equal

Logical operators:

- `AND` — Both conditions must be true
- `OR` — Either condition must be true

Examples:

```
WHERE degree > 5
WHERE layer="friends" AND degree > 3
WHERE degree > 5 OR betweenness_centrality > 0.1
```

Compute Clause

Calculate metrics for selected nodes:

```
COMPUTE degree
COMPUTE degree COMPUTE betweenness_centrality
COMPUTE clustering
```

Available metrics:

- `degree` — Node degree
- `betweenness_centrality` — Betweenness centrality

- `closeness_centrality` — Closeness centrality
- `clustering` — Clustering coefficient
- `pagerank` — PageRank score
- `layer_count` — Number of layers node appears in

See *Algorithm Roadmap* for complete metric list.

Order By

```
ORDER BY degree
ORDER BY -degree  # Descending (prefix with -)
ORDER BY betweenness_centrality
```

Limit

```
LIMIT 10
LIMIT 100
```

Complete Examples

```python
from py3plex.dsl import execute_query

# Get high-degree nodes
result = execute_query(
    network,
    'SELECT nodes WHERE degree > 5'
)

# Get nodes from specific layer
result = execute_query(
    network,
    'SELECT nodes FROM layer="friends" '
    'WHERE degree > 3 '
    'COMPUTE betweenness_centrality '
    'ORDER BY -betweenness_centrality '
    'LIMIT 10'
)
```

Builder API Reference

Import

```python
from py3plex.dsl import Q, L
```

Query Construction

Start a query:

```
Q.nodes()      # Select nodes
Q.edges()      # Select edges (experimental)
```

Layer Selection

Single layer:

```
Q.nodes().from_layers(L["friends"])
```

Multiple layers (union):

```
Q.nodes().from_layers(L["friends"] + L["work"])

# Or use the new LayerSet algebra:
Q.nodes().from_layers(L["friends | work"])
```

Layer intersection:

```
Q.nodes().from_layers(L["friends"] & L["work"])
```

Advanced Layer Set Algebra:

```
# All layers except coupling
Q.nodes().from_layers(L["* - coupling"])

# Complex expressions with set operations
Q.nodes().from_layers(L["(social | work) & ~bots"])

# Named groups for reuse
from py3plex.dsl import LayerSet
LayerSet.define_group("bio", LayerSet("ppi") | LayerSet("gene"))
Q.nodes().from_layers(LayerSet("bio"))
```

See also

For complete documentation on layer set algebra including all operators, string parsing, named groups, and real-world examples, see: *Layer Set Algebra*

Filtering

Comparison operators:

```
Q.nodes().where(degree__gt=5)        # Greater than
Q.nodes().where(degree__gte=5)       # Greater than or equal
Q.nodes().where(degree__lt=5)        # Less than
Q.nodes().where(degree__lte=5)       # Less than or equal
```

(continues on next page)

(continued from previous page)

```
Q.nodes().where(degree__eq=5)        # Equal
Q.nodes().where(degree__ne=5)        # Not equal
```

Multiple conditions:

```
Q.nodes().where(
    degree__gt=5,
    layer_count__gte=2
)
```

Computing Metrics

```
Q.nodes().compute("degree")
Q.nodes().compute("degree", "betweenness_centrality")
```

Sorting

```
Q.nodes().order_by("degree")           # Ascending
Q.nodes().order_by("-degree")          # Descending
```

Limiting

```
Q.nodes().limit(10)
```

Execution

```
result = Q.nodes().execute(network)
```

Chaining

All methods can be chained:

```
result = (
    Q.nodes()
     .from_layers(L["friends"])
     .where(degree__gt=5)
     .compute("betweenness_centrality")
     .order_by("-betweenness_centrality")
     .limit(10)
     .execute(network)
)
```

Temporal Queries

Filter by Time Point

```
# String syntax
result = execute_query(
    network,
    'SELECT nodes AT "2024-01-15T10:00:00"'
)

# Builder API
result = (
    Q.nodes()
     .at("2024-01-15T10:00:00")
     .execute(network)
)
```

Filter by Time Range

```
# String syntax
result = execute_query(
    network,
    'SELECT nodes DURING "2024-01-01" TO "2024-01-31"'
)

# Builder API
result = (
    Q.nodes()
     .during("2024-01-01", "2024-01-31")
     .execute(network)
)
```

Temporal Edge Attributes

Edges can have temporal attributes:

- `t` — Point in time (ISO 8601 timestamp)
- `t_start` and `t_end` — Time range

See ../user_guide/networks for creating temporal networks.

Grouping and Coverage Queries

Per-Layer Grouping

Group results by layer and apply per-group operations:

```
# Group by layer
result = (
    Q.nodes()
     .from_layers(L["*"])
```

(continues on next page)

(continued from previous page)

```
    .compute("degree")
    .per_layer()                # Sugar for .group_by("layer")
        .top_k(5, "degree")     # Top 5 per layer
    .end_grouping()
    .execute(network)
)
```

Top-K Per Group

Select top-k items per group (requires prior grouping):

```
# Top 10 highest-degree nodes per layer
result = (
    Q.nodes()
     .from_layers(L["*"])
     .compute("degree", "betweenness_centrality")
     .per_layer()
        .top_k(10, "degree")
     .end_grouping()
     .execute(network)
)
```

Coverage Filtering

Filter based on presence across groups:

Mode: "all" — Keep items appearing in ALL groups (intersection)

```
# Nodes that are top-5 hubs in ALL layers
multi_hubs = (
    Q.nodes()
     .from_layers(L["*"])
     .compute("betweenness_centrality")
     .per_layer()
        .top_k(5, "betweenness_centrality")
     .end_grouping()
     .coverage(mode="all")
     .execute(network)
)
```

Mode: "any" — Keep items appearing in AT LEAST ONE group (union)

```
# Nodes that are top-5 in any layer
any_hubs = (
    Q.nodes()
     .from_layers(L["*"])
     .compute("degree")
     .per_layer()
        .top_k(5, "degree")
     .end_grouping()
```

(continues on next page)

(continued from previous page)

```
        .coverage(mode="any")
        .execute(network)
)
```

Mode: "at_least" — Keep items appearing in at least K groups

```
# Nodes in top-10 of at least 2 layers
two_layer_hubs = (
    Q.nodes()
        .from_layers(L["*"])
        .compute("degree")
        .per_layer()
            .top_k(10, "degree")
        .end_grouping()
        .coverage(mode="at_least", k=2)
        .execute(network)
)
```

Mode: "exact" — Keep items appearing in exactly K groups

```
# Layer specialists: top-5 in exactly 1 layer
specialists = (
    Q.nodes()
        .from_layers(L["*"])
        .compute("betweenness_centrality")
        .per_layer()
            .top_k(5, "betweenness_centrality")
        .end_grouping()
        .coverage(mode="exact", k=1)
        .execute(network)
)
```

Wildcard Layer Selection

Use `L["*"]` to select all layers:

```
# All layers
Q.nodes().from_layers(L["*"])

# All layers except "bots"
Q.nodes().from_layers(L["*"] - L["bots"])

# Layer algebra still works
Q.nodes().from_layers((L["*"] - L["spam"]) & L["verified"])
```

General Grouping

Group by arbitrary attributes (not just layer):

```
# Group by multiple attributes
result = (
    Q.nodes()
     .compute("degree", "community")
     .group_by("layer", "community")
        .top_k(3, "degree")
     .end_grouping()
     .execute(network)
)
```

Limitations

- Coverage filtering is currently supported only for **node queries**
- Edge queries with coverage will raise a clear `DslExecutionError`
- Grouping requires computed attributes or inherent node properties (like layer)

Working with Results

Result Object

Query results are dictionaries mapping nodes to their computed attributes:

```
result = Q.nodes().compute("degree").execute(network)

# Access individual node
node = ('Alice', 'friends')
degree = result[node]['degree']

# Iterate
for node, data in result.items():
    print(f"{node}: {data}")
```

Convert to Pandas

```
df = result.to_pandas()
print(df.head())
```

Extensibility

Custom Operators

Register custom DSL operators:

```
from py3plex.dsl import register_operator

@register_operator('my_metric')
def my_custom_metric(context, node):
    """Compute custom metric for a node."""
    # Your implementation
    return value
```

See *Architecture and Design* for plugin development.

Performance Considerations

1. **Compute metrics once:** Don't recompute in multiple queries
2. **Filter early:** Use WHERE before COMPUTE
3. **Limit results:** Use LIMIT for large networks
4. **Layer-specific:** Query single layers when possible

Next Steps

- **Learn by doing:** *How to Query Multilayer Graphs with the SQL-like DSL*
- **See examples:** *Examples & Recipes*
- **Understand implementation:** *Architecture and Design*

3.5.4 Layer Set Algebra

The Layer Set Algebra feature provides a first-class, composable system for selecting and manipulating layer sets in multilayer networks. Instead of manually specifying lists of layers, you can use set-theoretic operations to express complex layer selections.

Why Layer Set Algebra?

In multilayer network analysis, you often need to:

- Select "all layers except the coupling layer"
- Combine biological layers for aggregate analysis
- Find the intersection of layers where a node appears
- Reuse layer group definitions across multiple queries

The Layer Set Algebra makes these operations expressive, composable, and safe.

Quick Start

Basic Operations

```python
from py3plex.dsl import LayerSet, L, Q

# Create layer sets
social = LayerSet("social")
work = LayerSet("work")

# Union: layers in either social OR work
both = social | work

# Intersection: layers in both social AND work
shared = social & work

# Difference: layers in social but NOT in work
social_only = social - work
```

(continues on next page)

(continued from previous page)

```
# Complement: all layers EXCEPT social
not_social = ~social
```

String Expressions

For convenience, you can write layer expressions as strings:

```
# Parse from string
layers = LayerSet.parse("* - coupling")

# Or use L[...] syntax (auto-detects expressions)
layers = L["* - coupling"]
layers = L["(ppi | gene) & disease"]
layers = L["social | work | hobby"]
```

Integration with Queries

Use layer sets in any DSL query:

```
# Query nodes from all layers except coupling
result = (
    Q.nodes()
     .from_layers(L["* - coupling"])
     .compute("degree")
     .execute(network)
)

# Complex layer selection
result = (
    Q.nodes()
     .from_layers(L["(social | work) & ~coupling"])
     .where(degree__gt=5)
     .execute(network)
)
```

Set Operations

Union: A | B

The union operation combines layers from multiple sets.

```
# Select nodes from social OR work layers
layers = L["social | work"]
# Equivalent to:
layers = LayerSet("social") | LayerSet("work")
```

ASCII Venn Diagram:

```
 _________     _________
│ Social  │   │  Work   │
│    ____________      │
│    │   RESULT       │    │
│_________________________
   A | B = {social, work}
```

Intersection: A & B

The intersection operation selects only layers present in both sets.

```
# Nodes that appear in both layer A AND layer B
layers = L["layerA & layerB"]
```

ASCII Venn Diagram:

```
 _________     _________
│    A    │   │    B    │
│    ____________      │
│    │    │ RES │    │    │
│_____________________
          A & B
```

Difference: A - B

The difference operation removes layers in B from A.

```
# All layers except coupling
layers = L["* - coupling"]

# Social layers but not bots
layers = L["social - bots"]
```

ASCII Venn Diagram:

```
 _________     _________
│    A    │   │    B    │
│  ___________         │
│  │ RES  │   │          │
│______________________
   A - B = A without B
```

Complement: ~A

The complement operation returns all layers except those in A.

```
# Everything except social layer
layers = ~LayerSet("social")

# Equivalent to:
layers = L["* - social"]
```

ASCII Venn Diagram:

```
┌─────────────────────────┐
│     All Layers (*)      │
│                         │
│  ┌──────┐               │
│  │  A   │    RESULT     │
│  └──────┘               │
└─────────────────────────┘
   ~A = * - A
```

Named Layer Groups

Define reusable layer groups for cleaner queries:

```
from py3plex.dsl import LayerSet, L, Q

# Define a named group
bio_layers = LayerSet("ppi") | LayerSet("gene") | LayerSet("disease")
LayerSet.define_group("bio", bio_layers)

# Or use the convenience method
L.define("core", L["social | work | email"])

# Use the group in queries
result = Q.nodes().from_layers(LayerSet("bio")).execute(network)

# Groups can be combined with other operations
result = Q.nodes().from_layers(L["bio & core"]).execute(network)

# List all defined groups
groups = L.list_groups()
print(groups)  # {'bio': LayerSet(...), 'core': LayerSet(...)}
```

Operator Precedence

Layer expressions follow standard set theory precedence:

1. **Complement** (~) - highest precedence
2. **Intersection** (&)
3. **Difference** (-)
4. **Union** (|) - lowest precedence

Use parentheses for clarity or to override precedence:

```
# Without parentheses: parsed as A & (B | C)
expr1 = L["A & B | C"]

# With parentheses: explicit grouping
expr2 = L["(A & B) | C"]
expr3 = L["A & (B | C)"]
```

Real-World Examples

Example 1: Biological Networks

Analyze protein interactions while excluding coupling layers:

```
from py3plex.dsl import Q, L, LayerSet

# Define biological layer groups
L.define("bio", L["ppi | gene | disease"])

# Find high-degree nodes in biological layers only
result = (
    Q.nodes()
     .from_layers(LayerSet("bio") - LayerSet("coupling"))
     .where(degree__gt=10)
     .compute("betweenness_centrality")
     .order_by("betweenness_centrality", desc=True)
     .limit(20)
     .execute(network)
)

print(result.to_pandas())
```

Example 2: Social Networks

Compare centrality across different social contexts:

```
# Define layer groups for different social contexts
L.define("online", L["facebook | twitter | linkedin"])
L.define("offline", L["work | school | hobby"])

# Compute centrality in online layers
online_centrality = (
    Q.nodes()
     .from_layers(LayerSet("online"))
     .compute("pagerank")
     .execute(network)
).to_pandas()

# Compute centrality in offline layers
offline_centrality = (
    Q.nodes()
     .from_layers(LayerSet("offline"))
     .compute("pagerank")
     .execute(network)
).to_pandas()

# Find nodes with high centrality in both contexts
# (merge and filter the DataFrames)
```

Example 3: Transportation Networks

Analyze robustness by excluding specific infrastructure:

```
# Simulate failure scenarios
scenarios = [
    ("full_network", L["*"]),
    ("no_air", L["* - air_travel"]),
    ("no_rail", L["* - rail"]),
    ("road_only", L["road"]),
    ("multi_modal", L["road | rail"]),
]

results = {}
for name, layers in scenarios:
    result = (
        Q.nodes()
         .from_layers(layers)
         .compute("betweenness_centrality")
         .execute(network)
    ).to_pandas()

    results[name] = {
        "avg_betweenness": result["betweenness_centrality"].mean(),
        "max_betweenness": result["betweenness_centrality"].max(),
        "node_count": len(result),
    }

import pandas as pd
df_results = pd.DataFrame(results).T
print(df_results)
```

Introspection and Debugging

Layer sets provide introspection tools to understand what they represent:

Resolution

See which layers a LayerSet resolves to:

```
layers = L["* - coupling"]

# Resolve against a network
resolved = layers.resolve(network)
print(resolved)  # {'social', 'work', 'hobby'}
```

Explanation

Get a human-readable explanation of the layer expression:

```
layers = L["(ppi | gene) & disease"]
print(layers.explain())

# Output:
# LayerSet:
#   intersection(
#     union(
#       layer("ppi"),
#       layer("gene")
#     ),
#     layer("disease")
#   )
```

With Network Resolution

Combine explanation with network resolution:

```
layers = L["* - coupling"]
print(layers.explain(network))

# Output:
# LayerSet:
#   difference(
#     all_layers("*"),
#     layer("coupling")
#   )
# → resolved to: {'hobby', 'social', 'work'}
```

Comparison: Old vs New Syntax

Old Style (Still Supported):

```
# Manual layer list
result = Q.nodes().from_layers(L["social"] + L["work"]).execute(network)

# Multiple separate queries
layers = ["social", "work", "hobby"]
results = []
for layer in layers:
    if layer != "coupling":
        result = Q.nodes().from_layers(L[layer]).execute(network)
        results.append(result)
```

New Style (Recommended):

```
# Expressive layer algebra
result = Q.nodes().from_layers(L["social | work"]).execute(network)

# Single query with filtering
result = (
```

(continues on next page)

(continued from previous page)

```
    Q.nodes()
     .from_layers(L["* - coupling"])
     .execute(network)
)
```

Why Layer Set Algebra Matters

For Multilayer Networks

Multilayer networks have a unique challenge: layer selection is a first-class operation, not an afterthought. Traditional graph analysis tools don't provide built-in support for layer algebra, forcing users to write imperative code with loops and conditionals.

Layer Set Algebra brings the following benefits:

1. **Expressiveness**: Complex layer selections become one-liners
2. **Composability**: Combine layer expressions with set operations
3. **Reusability**: Define named groups once, use everywhere
4. **Safety**: Late evaluation with validation at execution time
5. **Maintainability**: Clear, declarative code is easier to understand

Algebraic Properties

Layer Set Algebra follows standard set theory laws:

- **Idempotence**: A | A = A, A & A = A
- **Commutativity**: A | B = B | A, A & B = B & A
- **Associativity**: (A | B) | C = A | (B | C)
- **Distributivity**: A & (B | C) = (A & B) | (A & C)
- **De Morgan's Laws**: ~(A | B) = ~A & ~B, ~(A & B) = ~A | ~B
- **Complement**: A | ~A = *, A & ~A =

These properties enable query optimization and reasoning about layer selections.

API Reference

LayerSet Class

```
class LayerSet:
    """Represents an unevaluated layer expression."""

    def __init__(self, name_or_expr: Union[str, LayerExpr])
        """Create from layer name or expression AST."""

    def __or__(self, other: LayerSet) -> LayerSet:
        """Union: self | other."""

    def __and__(self, other: LayerSet) -> LayerSet:
```

(continues on next page)

(continued from previous page)

```
        """Intersection: self & other."""

    def __sub__(self, other: LayerSet) -> LayerSet:
        """Difference: self - other."""

    def __invert__(self) -> LayerSet:
        """Complement: ~self."""

    def resolve(self, network, *, strict=False, warn_empty=True) -> Set[str]:
        """Resolve expression to actual layer names."""

    def explain(self, network=None) -> str:
        """Generate human-readable explanation."""

    @staticmethod
    def parse(expr_str: str) -> LayerSet:
        """Parse layer expression from string."""

    @staticmethod
    def define_group(name: str, layer_set: LayerSet) -> None:
        """Define a named layer group."""

    @staticmethod
    def list_groups() -> Dict[str, LayerSet]:
        """List all defined groups."""

    @staticmethod
    def clear_groups() -> None:
        """Clear all defined groups."""
```

L Proxy

The L proxy provides convenient syntax for creating layer expressions:

```
# Simple layer name (backward compatible)
L["social"]  # Returns LayerExprBuilder

# Expression string (auto-detected)
L["* - coupling"]  # Returns LayerSet
L["social | work"]  # Returns LayerSet

# Define named group
L.define("bio", LayerSet("ppi") | LayerSet("gene"))

# List groups
groups = L.list_groups()

# Clear groups
L.clear_groups()
```

Troubleshooting

Common Issues

1. Empty Layer Set Warning

```
layers = L["nonexistent"]
result = layers.resolve(network)
# UserWarning: Layer expression resolved to empty set
```

Solution: Check layer names, use `warn_empty=False` to suppress, or use `strict=True` to raise an error instead.

2. Unknown Layer Error

```
layers = L["typo_layer"]
result = layers.resolve(network, strict=True)
# UnknownLayerError: Layer 'typo_layer' not found
```

Solution: Verify layer names with `network.layers` or `network.get_layers()`.

3. Syntax Error in Expression

```
layers = L["social &"]  # Missing right operand
# DslSyntaxError: Unexpected end of expression
```

Solution: Check expression syntax, ensure all operators have operands.

Best Practices

1. **Use Named Groups** for complex layer sets you'll reuse
2. **Prefer String Expressions** for readability: `L["* - coupling"]`
3. **Use Complement** instead of listing all other layers: `~LayerSet("exclude")`
4. **Document Layer Groups** with comments explaining their purpose
5. **Test Layer Resolution** with `.explain(network)` before running expensive queries

See Also

- query_with_dsl - DSL query basics
- query_zoo - Query examples and patterns
- /api/dsl - Complete DSL API reference

Note

Layer Set Algebra is available in py3plex DSL v2.1+. It's fully backward compatible with existing layer expressions.

3.5.5 Uncertainty-First Statistics

Overview

py3plex implements an **uncertainty-first statistics system** where every statistic is represented as `(value + uncertainty + provenance)`. This design makes uncertainty a core part of statistics computation, not an afterthought.

Key Principles

1. **Every statistic is a StatValue**: No plain floats/ints escaping internal computation paths. Even deterministic values carry `Delta(0)` uncertainty.
2. **Uncertainty is first-class**: Uncertainty can be summarized, sampled, propagated through arithmetic, and used in queries.
3. **Backward compatible**: Existing code expecting floats works via `float(statvalue)`.
4. **Registry discipline**: Statistics must have an uncertainty model to be registered.
5. **Reproducible**: Random processes (bootstrap/MC) support explicit seeds tracked in provenance.

Core Components

StatValue

`StatValue` is the fundamental container for statistics:

```
from py3plex.stats import StatValue, Delta, Provenance

# Create a deterministic statistic
sv = StatValue(
    value=0.42,
    uncertainty=Delta(0.0),
    provenance=Provenance("degree", "delta", {})
)

# Access value
print(float(sv))  # 0.42

# Query uncertainty
print(sv.std())  # 0.0
print(sv.ci(0.95))  # (0.42, 0.42)
print(sv.robustness())  # 1.0
```

Key Methods:

- `float(sv)`: Convert to point estimate (backward compatibility)
- `sv.mean()`: Alias for value
- `sv.std()`: Standard deviation
- `sv.ci(level=0.95)`: Confidence interval
- `sv.robustness()`: Robustness score in [0, 1]
- `sv.to_json_dict()`: Serialize to JSON

Uncertainty Models

Five concrete uncertainty models are provided:

Delta

Deterministic or known-precision uncertainty.

```
from py3plex.stats import Delta

# Perfect certainty
d = Delta(0.0)

# Small known error
d = Delta(0.01)
```

Properties:

- `std()`: Returns sigma
- `ci(level)`: Returns symmetric interval based on sigma
- `sample(n, seed)`: Returns constant samples
- Propagation: Analytic error propagation when both are Delta

Gaussian

Normal distribution uncertainty.

```
from py3plex.stats import Gaussian

g = Gaussian(mean=0.0, std_dev=0.1)

# Exact CI computation
low, high = g.ci(0.95)  #  (-0.196, 0.196)
```

Properties:

- `std()`: Returns std_dev
- `ci(level)`: Exact Gaussian CI using z-scores
- `sample(n, seed)`: Generates Gaussian samples
- Propagation: Analytic for addition/subtraction, Monte Carlo for complex ops

Bootstrap

Empirical uncertainty from bootstrap resampling.

```
from py3plex.stats import Bootstrap
import numpy as np

# Store bootstrap samples (relative to point estimate)
samples = np.array([0.1, -0.05, 0.15, 0.0, 0.08])
b = Bootstrap(samples)
```

(continues on next page)

(continued from previous page)

```
# Compute CI from percentiles
low, high = b.ci(0.95)
```

Properties:

- `std()`: Sample standard deviation
- `ci(level)`: Percentile-based CI
- `sample(n, seed)`: Resample from stored samples
- Propagation: Always Monte Carlo
- Serialization: Stores summary (n, std, CI) not full samples

Empirical

Similar to Bootstrap but conceptually for any empirical distribution.

```
from py3plex.stats import Empirical

samples = np.array([0.1, 0.2, 0.15, 0.18, 0.12])
e = Empirical(samples)
```

Properties:

- Same as Bootstrap
- Conceptually separate for clarity

Interval

Interval-based uncertainty without assuming distribution.

```
from py3plex.stats import Interval

i = Interval(-0.1, 0.15)

# Uniform sampling by default
samples = i.sample(100, seed=42)
```

Properties:

- `std()`: Estimates std assuming uniform distribution: `(high - low) / sqrt(12)`
- `ci(level)`: Returns the interval bounds
- `sample(n, seed)`: Uniform sampling
- Propagation: Monte Carlo

Provenance

Tracks how a statistic was computed:

```python
from py3plex.stats import Provenance

prov = Provenance(
    algorithm="brandes",
    uncertainty_method="bootstrap",
    parameters={"n_samples": 100},
    seed=42,
    timestamp="2024-12-12T17:00:00",
    library_version="1.0.0"
)

# Serialize
json_dict = prov.to_json_dict()
```

Fields:

- `algorithm`: Algorithm name (e.g., "degree", "betweenness")
- `uncertainty_method`: Uncertainty method (e.g., "delta", "bootstrap")
- `parameters`: Dict of parameters
- `seed`: Random seed (optional)
- `timestamp`: Computation time (optional)
- `library_version`: Version string (optional)

Statistics Registry

The `StatisticsRegistry` enforces that every registered statistic has an uncertainty model.

Registration

```python
from py3plex.stats import StatisticSpec, register_statistic, Delta

def compute_degree(network, node):
    return network.core_network.degree(node)

def degree_uncertainty(network, node, **kwargs):
    return Delta(0.0)  # Deterministic

spec = StatisticSpec(
    name="degree",
    estimator=compute_degree,
    uncertainty_model=degree_uncertainty,
    assumptions=["deterministic"],
    supports={"directed": True, "weighted": True}
)

register_statistic(spec)
```

Note: Registration fails if `uncertainty_model` is missing.

Usage

```
from py3plex.stats import compute_statistic

# Compute with uncertainty
result = compute_statistic("degree", network, node, with_uncertainty=True)
# Returns StatValue

# Compute without uncertainty (raw value)
value = compute_statistic("degree", network, node, with_uncertainty=False)
```

Arithmetic with Uncertainty

StatValue supports arithmetic operations with automatic uncertainty propagation.

Basic Operations

```
from py3plex.stats import StatValue, Gaussian, Provenance

sv1 = StatValue(1.0, Gaussian(0.0, 0.1), Provenance("a", "gaussian", {}))
sv2 = StatValue(2.0, Gaussian(0.0, 0.15), Provenance("b", "gaussian", {}))

# Addition
result = sv1 + sv2
print(float(result))  # 3.0
print(result.std())  # ~0.180 (sqrt(0.1² + 0.15²))

# Subtraction
result = sv1 - sv2

# Multiplication
result = sv1 * sv2

# Division
result = sv1 / sv2

# Power
result = sv1 ** 2

# Negation
result = -sv1
```

Scalar Operations

StatValue supports operations with scalars:

```
sv = StatValue(2.0, Gaussian(0.0, 0.1), Provenance("a", "gaussian", {}))

# Scalar addition
result = sv + 3  # 5.0 (uncertainty unchanged)
```

(continues on next page)

(continued from previous page)

```
# Scalar multiplication
result = sv * 2  # 4.0 (uncertainty scaled)

# Scalar division
result = sv / 2  # 1.0 (uncertainty scaled)
```

Propagation Rules

1. **Delta + Delta**: Analytic error propagation (_sum = sqrt(1^2 + 2^2))
2. **Gaussian + Gaussian**: Exact propagation for addition/subtraction
3. **Complex operations**: Monte Carlo propagation (4096 samples by default)
4. **Scalar operations**: Direct computation, uncertainty scaled appropriately

Filtering and Queries

Statistics with uncertainty can be filtered using selectors:

Selector Syntax

Format: `attribute__component__operator=value`

Components:

- `mean`: Point estimate (default if omitted)
- `std`: Standard deviation
- `ci95__width`: Width of 95% CI
- `robustness`: Robustness score

Operators:

- `gt`: Greater than
- `gte`: Greater than or equal
- `lt`: Less than
- `lte`: Less than or equal
- `eq`: Equal
- `ne`: Not equal

Examples

```
# Filter by mean value
result = Q.nodes().where(degree__mean__gt=3).execute(network)

# Filter by uncertainty
result = Q.nodes().where(betweenness__std__lt=0.05).execute(network)

# Filter by CI width
```

(continues on next page)

(continued from previous page)

```
result = Q.nodes().where(degree__ci95__width__lt=0.1).execute(network)

# Filter by robustness
result = Q.nodes().where(centrality__robustness__gt=0.9).execute(network)
```

Serialization

StatValue and Uncertainty models support JSON serialization.

StatValue Serialization

```
sv = StatValue(
    value=0.42,
    uncertainty=Gaussian(0.0, 0.05),
    provenance=Provenance("betweenness", "analytic", {})
)

json_dict = sv.to_json_dict()
# {
#   "value": 0.42,
#   "uncertainty": {
#     "type": "gaussian",
#     "mean": 0.0,
#     "std": 0.05
#   },
#   "provenance": {
#     "algorithm": "betweenness",
#     "uncertainty_method": "analytic",
#     "params": {}
#   }
# }
```

DataFrame Export

QueryResult can export to pandas with uncertainty columns:

```
result = Q.nodes().compute("betweenness").execute(network)

df = result.to_pandas()
# Columns: id, betweenness.value, betweenness.std,
#          betweenness.ci_low, betweenness.ci_high,
#          betweenness.uncertainty_type
```

Best Practices

1. **Always use StatValue internally**: Even for deterministic stats, use `Delta(0)`
2. **Provide uncertainty models**: Every registered statistic must have one
3. **Use seeds for reproducibility**: Pass explicit seeds to bootstrap/MC operations

4. **Choose appropriate models**:
   - Deterministic → `Delta(0)`
   - Known distribution → `Gaussian`
   - Empirical estimation → `Bootstrap` or `Empirical`
   - No distribution assumption → `Interval`
5. **Check robustness**: Use `sv.robustness()` to assess reliability
6. **Export uncertainty**: Include uncertainty columns in exports for downstream analysis

Examples

See:

- `examples/uncertainty/example_stats_degree_delta.py`
- `examples/uncertainty/example_stats_betweenness_bootstrap.py`

API Reference

StatValue

```
class StatValue:
    """Statistical value with uncertainty and provenance."""

    value: float | int | ndarray
    uncertainty: Uncertainty
    provenance: Provenance

    def __float__(self) -> float: ...
    def mean(self) -> float: ...
    def std(self) -> float: ...
    def ci(self, level: float = 0.95) -> tuple[float, float]: ...
    def robustness(self) -> float: ...
    def to_json_dict(self) -> dict: ...
```

Uncertainty Models

```
class Uncertainty(ABC):
    def summary(self, level: float = 0.95) -> dict: ...
    def sample(self, n: int, *, seed: int | None = None) -> ndarray: ...
    def ci(self, level: float = 0.95) -> tuple[float, float]: ...
    def std(self) -> float | None: ...
    def propagate(self, op: str, other: Uncertainty | None, *, seed: int |␣
↪None = None) -> Uncertainty: ...
    def to_json_dict(self) -> dict: ...

class Delta(Uncertainty):
    sigma: float = 0.0

class Gaussian(Uncertainty):
```

(continues on next page)

(continued from previous page)

```
    mean: float
    std_dev: float

class Bootstrap(Uncertainty):
    samples: ndarray

class Empirical(Uncertainty):
    samples: ndarray

class Interval(Uncertainty):
    low: float
    high: float
```

Provenance

```
@dataclass(frozen=True)
class Provenance:
    algorithm: str
    uncertainty_method: str
    parameters: dict = field(default_factory=dict)
    seed: int | None = None
    timestamp: str | None = None
    library_version: str | None = None

    def to_json_dict(self) -> dict: ...
    @classmethod
    def from_json_dict(cls, data: dict) -> Provenance: ...
```

Registry

```
@dataclass(frozen=True)
class StatisticSpec:
    name: str
    estimator: Callable
    uncertainty_model: Callable  # Required
    assumptions: list[str] = field(default_factory=list)
    supports: dict = field(default_factory=dict)

def register_statistic(spec: StatisticSpec, force: bool = False) -> None: ...
def get_statistic(name: str) -> StatisticSpec: ...
def list_statistics() -> list[str]: ...
def compute_statistic(name: str, *args, with_uncertainty: bool = True,
↪**kwargs) -> Any: ...
```

3.5.6 API Documentation

This section contains the complete API documentation for py3plex, automatically generated from docstrings.

Tip

Looking for algorithm documentation? Visit *Algorithm Roadmap* for a conceptual overview of all algorithms organized by category with working examples. Then come back here for detailed API signatures.

For auto-generated module documentation, run:

```
cd docfiles
sphinx-apidoc -o AUTOGEN_results -f ../py3plex
make html
```

Core Modules

`py3plex.core.multinet.`**`ensure`**`(`**args, **kwargs*`)`

`py3plex.core.multinet.`**`invariant`**`(`**args, **kwargs*`)`

`class py3plex.core.multinet.`**`multi_layer_network`**`(`*verbose: bool = True, network_type: str = 'multilayer', directed: bool = True, dummy_layer: str = 'null', label_delimiter: str = '---', coupling_weight: int | float = 1*`)`

Bases: `object`

Main class for multilayer network analysis and manipulation.

This class provides a comprehensive toolkit for creating, analyzing, and visualizing multilayer networks where nodes can exist in multiple layers and edges can connect nodes within or across layers.

Supported Network Types (network_type parameter):

- **multilayer** (default): General multilayer networks with arbitrary layer structure. Each layer can have a different set of nodes. Suitable for heterogeneous networks (e.g., authors-papers-venues) or networks where nodes naturally appear in only some layers.
- **multiplex**: Special case where all layers share the same node set but with different edge types. After loading a network, automatic coupling edges are created between each node and its counterparts in other layers. Suitable for social networks with multiple relationship types (e.g., friend, colleague, family layers).

Choosing the Right Network Type:
Key Features:

- Dict-based API for adding nodes and edges (see add_nodes() and add_edges())
- NetworkX interoperability via to_networkx() and from_networkx()
- Multiple I/O formats (edgelist, GML, GraphML, gpickle, etc.)
- Visualization methods for multilayer layouts

- Community detection and centrality analysis
- Random walk and embedding generation

Hypergraph Support:

This class does NOT natively support true hypergraphs (edges connecting more than two nodes). For hypergraph-like structures, consider: - Using bipartite projections (nodes and hyperedges as separate node types) - The incidence gadget encoding via to_homogeneous_hypergraph() - External hypergraph libraries with conversion utilities

Notes

- Nodes in multilayer networks are represented as (node_id, layer) tuples
- Use add_nodes() and add_edges() with dict format for easiest interaction
- See examples/ directory for usage patterns and best practices

Examples

```
>>> # Create a general multilayer network (different node sets per layer)
>>> net = multi_layer_network(network_type='multilayer', directed=False)
>>>
>>> # Add nodes to different layers
>>> net.add_nodes([
...     {'source': 'A', 'type': 'social'},
...     {'source': 'B', 'type': 'social'},
...     {'source': 'A', 'type': 'email'}  # Same node, different layer
... ])
>>>
>>> # Add edges (intra-layer and inter-layer)
>>> net.add_edges([
...     {'source': 'A', 'target': 'B',
...      'source_type': 'social', 'target_type': 'social'},
...     {'source': 'A', 'target': 'A',
...      'source_type': 'social', 'target_type': 'email'}
... ])
>>>
>>> print(net)  # Shows network statistics
<multi_layer_network: type=multilayer, directed=False, nodes=3, edges=2,␣
↪layers=2>
```

```
>>> # Create a multiplex network (same nodes across relationship layers)
>>> # Note: coupling edges are auto-added after load_network()
>>> multiplex_net = multi_layer_network(network_type='multiplex')
```

add_dummy_layers()

Internal function, for conversion between objects

add_edges(*edge_dict_list: List[Dict] | List[List] | Tuple, input_type: str = 'dict'*) → *multi_layer_network*

Add edges to the multilayer network.

This method supports multiple input formats for specifying edges between nodes in different layers. The most common format is dict-based.

Parameters

- **edge_dict_list** – Edge data in one of the supported formats (see below)
- **input_type** – Format of edge data ('dict', 'list', or 'px_edge')

Returns
Returns self for method chaining

Return type
self

Supported Formats:

Dict format (recommended): ```python {

'source': 'node1', # Source node ID 'target': 'node2', # Target node ID 'source_type': 'layer1', # Source layer name 'target_type': 'layer2', # Target layer name (can be same as source) 'weight': 1.0, # Optional: edge weight 'type': 'interaction' # Optional: edge type/label

List format: *[node1, layer1, node2, layer2]*

px_edge format: *((node1, layer1), (node2, layer2), {'weight': 1.0})*

Examples

```
>>> # Add single intra-layer edge
>>> net = multi_layer_network()
>>> net.add_edges([{
...     'source': 'A',
...     'target': 'B',
...     'source_type': 'protein',
...     'target_type': 'protein'
... }])
<multi_layer_network: type=multilayer, directed=True, nodes=2,␣
↪edges=1, layers=1>
```

```
>>> # Method chaining
>>> net = multi_layer_network()
>>> net.add_edges([
...     {'source': 'A', 'target': 'B', 'source_type': 'layer1',
↪'target_type': 'layer1'}
... ]).add_edges([
...     {'source': 'B', 'target': 'C', 'source_type': 'layer1',
↪'target_type': 'layer1'}
... ])
<multi_layer_network: type=multilayer, directed=True, nodes=3,␣
↪edges=2, layers=1>
```

```
>>> # Add inter-layer edge with weight
>>> net.add_edges([{
```

(continues on next page)

(continued from previous page)

```
...     'source': 'gene1',
...     'target': 'protein1',
...     'source_type': 'genes',
...     'target_type': 'proteins',
...     'weight': 0.95,
...     'type': 'expression'
... }])
<multi_layer_network: type=multilayer, directed=True, nodes=5,␣
↪edges=3, layers=3>
```

```
>>> # Add multiple edges at once
>>> edges = [
...     {'source': 'A', 'target': 'B', 'source_type': 'layer1',
↪'target_type': 'layer1'},
...     {'source': 'B', 'target': 'C', 'source_type': 'layer1',
↪'target_type': 'layer1'}
... ]
>>> net.add_edges(edges)
<multi_layer_network: type=multilayer, directed=True, nodes=5,␣
↪edges=5, layers=3>
```

Raises

`Exception` – If input_type is not one of 'dict', 'list', or 'px_edge'

Notes

- For intra-layer edges, use the same layer for source_type and target_type
- For inter-layer edges, use different layers
- Edge weights default to 1.0 if not specified

add_nodes(*node_dict_list: List[Dict] | Dict*, *input_type: str = 'dict'*) → *multi_layer_network*

Add nodes to the multilayer network.

Nodes in a multilayer network are identified by both their ID and the layer they belong to. This method adds nodes using a dict-based format.

Parameters

- **node_dict_list** – Node data as a dict or list of dicts (see format below)
- **input_type** – Format of node data (currently only 'dict' is supported)

Returns

Returns self for method chaining

Return type

self

Dict Format:

```python {
```

'source': 'node_id', # Node identifier (can be string or number) 'type': 'layer_name', # Layer this node belongs to 'weight': 1.0, # Optional: node weight/importance 'label': 'display' # Optional: display label # … any other node attributes

Examples

```
>>> # Add single node
>>> net = multi_layer_network()
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}])
<multi_layer_network: type=multilayer, directed=True, nodes=1,␣
↪edges=0, layers=1>
```

```
>>> # Method chaining
>>> net = multi_layer_network()
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}]).add_nodes([{
↪'source': 'B', 'type': 'layer1'}])
<multi_layer_network: type=multilayer, directed=True, nodes=2,␣
↪edges=0, layers=1>
```

```
>>> # Add multiple nodes to the same layer
>>> nodes = [
...     {'source': 'A', 'type': 'protein'},
...     {'source': 'B', 'type': 'protein'},
...     {'source': 'C', 'type': 'protein'}
... ]
>>> net.add_nodes(nodes)
<multi_layer_network: type=multilayer, directed=True, nodes=5,␣
↪edges=0, layers=2>
```

```
>>> # Add nodes with attributes
>>> net.add_nodes([{
...     'source': 'gene1',
...     'type': 'genes',
...     'weight': 0.8,
...     'label': 'BRCA1',
...     'chromosome': '17'
... }])
<multi_layer_network: type=multilayer, directed=True, nodes=6,␣
↪edges=0, layers=3>
```

```
>>> # Add nodes to multiple layers
>>> multi_layer_nodes = [
...     {'source': 'entity1', 'type': 'layer1'},
...     {'source': 'entity1', 'type': 'layer2'},  # Same entity,␣
↪different layer
...     {'source': 'entity2', 'type': 'layer1'}
... ]
>>> net.add_nodes(multi_layer_nodes)
```

(continues on next page)

(continued from previous page)

```
<multi_layer_network: type=multilayer, directed=True, nodes=9,␣
↪edges=0, layers=5>
```

Notes

- The same node ID can exist in multiple layers
- Each (node_id, layer) combination is treated as a unique node
- Additional attributes beyond 'source' and 'type' are preserved
- Nodes must be added before edges referencing them

aggregate_edges(*metric='count'*, *normalize_by='degree'*)

Edge aggregation method

Count weights across layers and return a weighted network

Parameters

- **param1** – aggregation operator (count is default)
- **param2** – normalization of the values

Returns

A simplified network.

assign_partition(*partition: Dict[Tuple[Any, Any], int]*) → None

Assign community partition to network nodes.

This method stores the community assignments as node attributes and computes community-level statistics.

Parameters

partition (*dict*) – Dictionary mapping (node, layer) tuples to community IDs.

Examples

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network(directed=False)
>>> net.add_edges([['A', 'L1', 'B', 'L1', 1]], input_type='list')
>>> partition = {('A', 'L1'): 0, ('B', 'L1'): 0}
>>> net.assign_partition(partition)
>>> print(net.community_sizes)
{0: 2}
```

basic_stats(*target_network=None*)

A method for obtaining a network's statistics.

Displays: - Basic network info (nodes, edges) - Total unique nodes (counting each (node, layer) as unique) - Unique node IDs (across all layers) - Per-layer node counts

compute_ollivier_ricci(*mode: str = 'core'*, *layers: List[Any] | None = None*, *alpha: float = 0.5*, *weight_attr: str = 'weight'*, *curvature_attr: str = 'ricciCurvature'*, *verbose: str = 'ERROR'*, *backend_kwargs: Dict[str, Any] | None = None*, *inplace: bool = True*, *interlayer_weight: float = 1.0*) → Dict[str, Any]

Compute Ollivier-Ricci curvature on the multilayer network.

This method provides flexible computation of Ollivier-Ricci curvature at different levels of the multilayer network:

- **core mode**: Compute curvature on the aggregated (flattened) network
- **layers mode**: Compute curvature separately for each layer
- **supra mode**: Compute curvature on the full supra-graph including both intra-layer and inter-layer edges

Parameters

- **mode** – Scope of computation. Options: "core", "layers", "supra".
- **layers** – List of layer identifiers to process (only for mode="layers"). If None, all layers are processed.
- **alpha** – Ollivier-Ricci parameter in [0, 1] controlling the mass distribution. Default: 0.5.
- **weight_attr** – Name of edge attribute containing weights. Default: "weight".
- **curvature_attr** – Name of edge attribute to store curvature values. Default: "ricciCurvature".
- **verbose** – Verbosity level. Options: "INFO", "DEBUG", "ERROR". Default: "ERROR".
- **backend_kwargs** – Additional keyword arguments for OllivierRicci constructor.
- **inplace** – If True, update internal graphs. If False, return new graphs without modifying the network. Default: True.
- **interlayer_weight** – Weight for inter-layer coupling edges (only for mode="supra"). Default: 1.0.

Returns

Dictionary mapping scope identifiers to NetworkX graphs with computed curvatures: - mode="core": {"core": graph_with_curvature} - mode="layers": {layer_id: graph_with_curvature, ...} - mode="supra": {"supra": supra_graph_with_curvature}

Raises

- **RicciBackendNotAvailable** – If GraphRicciCurvature is not installed.
- **ValueError** – If mode is invalid or layers contains invalid identifiers.

Examples

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network()
>>> net.add_edges([
...     ['A', 'layer1', 'B', 'layer1', 1],
...     ['B', 'layer1', 'C', 'layer1', 1],
```

(continues on next page)

(continued from previous page)

```
... ], input_type="list")
>>>
>>> # Compute on aggregated network
>>> result = net.compute_ollivier_ricci(mode="core")
>>>
>>> # Compute per layer
>>> result = net.compute_ollivier_ricci(mode="layers")
>>>
>>> # Compute on supra-graph
>>> result = net.compute_ollivier_ricci(mode="supra", inplace=False)
```

compute_ollivier_ricci_flow(*mode: str = 'core', layers: List[Any] | None = None, alpha: float = 0.5, iterations: int = 10, method: str = 'OTD', weight_attr: str = 'weight', curvature_attr: str = 'ricciCurvature', verbose: str = 'ERROR', backend_kwargs: Dict[str, Any] | None = None, inplace: bool = True, interlayer_weight: float = 1.0*) → Dict[str, Any]

Compute Ollivier-Ricci flow on the multilayer network.

Ricci flow iteratively adjusts edge weights based on their Ricci curvature, effectively revealing and enhancing community structure. After Ricci flow, edges with negative curvature (community boundaries) have reduced weights, while edges with positive curvature have increased weights.

Parameters

- **mode** – Scope of computation. Options: "core", "layers", "supra".
- **layers** – List of layer identifiers to process (only for mode="layers"). If None, all layers are processed.
- **alpha** – Ollivier-Ricci parameter in [0, 1]. Default: 0.5.
- **iterations** – Number of Ricci flow iterations. Default: 10.
- **method** – Ricci flow method. Options: "OTD" (Optimal Transport Distance, recommended), "ATD" (Average Transport Distance). Default: "OTD".
- **weight_attr** – Name of edge attribute containing weights. After Ricci flow, these weights are updated to reflect the flow metric.
- **curvature_attr** – Name of edge attribute for curvature values. Default: "ricciCurvature".
- **verbose** – Verbosity level. Options: "INFO", "DEBUG", "ERROR". Default: "ERROR".
- **backend_kwargs** – Additional keyword arguments for OllivierRicci constructor.
- **inplace** – If True, update internal graphs. If False, return new graphs. Default: True.
- **interlayer_weight** – Weight for inter-layer coupling edges (only for mode="supra"). Default: 1.0.

Returns
Dictionary mapping scope identifiers to NetworkX graphs with Ricci flow applied: - mode="core": {"core": graph_with_flow} - mode="layers": {layer_id: graph_with_flow, ...} - mode="supra": {"supra": supra_graph_with_flow}

Raises

- **RicciBackendNotAvailable** – If GraphRicciCurvature is not installed.
- **ValueError** – If mode is invalid or layers contains invalid identifiers.

Examples

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network()
>>> net.add_edges([
...     ['A', 'layer1', 'B', 'layer1', 1],
...     ['B', 'layer1', 'C', 'layer1', 1],
... ], input_type="list")
>>>
>>> # Apply Ricci flow to aggregated network
>>> result = net.compute_ollivier_ricci_flow(mode="core",␣
↪iterations=20)
>>>
>>> # Apply to each layer
>>> result = net.compute_ollivier_ricci_flow(mode="layers",␣
↪iterations=10)
```

property edge_count: int

Number of edges in the network.

Returns
Total count of edges

Return type
int

Examples

```
>>> net = multi_layer_network()
>>> net.add_edges([{'source': 'A', 'target': 'B',
...                 'source_type': 'layer1', 'target_type': 'layer1'}
↪])
>>> net.edge_count
1
```

edges_from_temporal_table(*edge_df*)

Convert a temporal edge DataFrame to edge tuple list.

Extracts edges from a pandas DataFrame with temporal/activity information and converts them to a list of edge tuples suitable for network construction.

Parameters
edge_df – pandas DataFrame with columns: - node_first: Source node

identifier - node_second: Target node identifier - layer_name: Layer identifier

Returns

List of edge tuples in format:
(node_first, node_second, layer_first, layer_second, weight) where weight is always 1

Return type
list

Notes

- All values are converted to strings
- All edges are assigned weight=1
- Both source and target are assumed to be in the same layer

Examples

```
>>> import pandas as pd
>>> net = multi_layer_network()
>>> df = pd.DataFrame({
...     'node_first': ['A', 'B'],
...     'node_second': ['B', 'C'],
...     'layer_name': ['L1', 'L1']
... })
>>> result = net.edges_from_temporal_table(df)
>>> len(result) >= 2
True
```

See also

fill_tmp_with_edges: Add these edges to layer graphs

execute_query(*query: str*) → Dict[str, Any]

Execute a DSL query on this multilayer network.

This is a convenience method that provides first-class access to the py3plex DSL (Domain-Specific Language) for querying multilayer networks.

Supports both SELECT and MATCH queries:

SELECT queries:
net.execute_query('SELECT nodes WHERE layer="transport"') net.execute_query('SELECT * FROM nodes IN LAYER "ppi" WHERE degree > 10')

MATCH queries (Cypher-like):
net.execute_query('MATCH (g:Gene)-[r]->(t:Gene) RETURN g, t') net.execute_query('MATCH (a)-[e]->(b) IN LAYER "ppi" WHERE a.degree > 5 RETURN a, b')

Parameters

query – DSL query string

Returns

- For SELECT queries: 'nodes' or 'edges' list, 'count', optional 'computed'
- For MATCH queries: 'bindings' list, 'count', 'type'

Return type

Dictionary containing query results

Raises

- *DSLSyntaxError* – If query syntax is invalid
- *DSLExecutionError* – If query cannot be executed

Examples

```
>>> net = multi_layer_network(directed=False)
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}])
>>> net.add_edges([{'source': 'A', 'target': 'B',
...                 'source_type': 'layer1', 'target_type': 'layer1'}
↪])
>>> result = net.execute_query('SELECT nodes WHERE layer="layer1"')
>>> result['count'] >= 0
True
```

```
>>> # Using MATCH syntax
>>> result = net.execute_query('MATCH (a:layer1)-[r]->(b:layer1)␣
↪RETURN a, b')
>>> 'bindings' in result
True
```

See also

- *py3plex.dsl.execute_query()* for standalone function
- *py3plex.dsl.format_result()* for formatting results

fill_tmp_with_edges(*edge_df*)

Fill temporary layer graphs with edges from a DataFrame.

Populates the emptied layer graphs (created by remove_layer_edges) with edges from a temporal/activity DataFrame. Useful for temporal network analysis where edge sets change over time.

Parameters

edge_df – pandas DataFrame with columns: - node_first: Source node identifier - node_second: Target node identifier - layer_name: Layer identifier

Notes

- Requires remove_layer_edges() to be called first
- Edges are grouped by layer
- Modifies self.tmp_layers in place
- Each edge is stored as ((node_first, layer), (node_second, layer))

Raises

`AttributeError` – If self.tmp_layers doesn't exist (call remove_layer_edges first)

Examples

These examples require proper network setup and are for illustration only.

```
>>> import pandas as pd
>>> net = multi_layer_network()
>>> net.split_to_layers()
>>> net.remove_layer_edges()
>>> df = pd.DataFrame({
...     'node_first': ['A', 'B'],
...     'node_second': ['B', 'C'],
...     'layer_name': ['L1', 'L1']
... })
>>> net.fill_tmp_with_edges(df)
```

See also

remove_layer_edges: Creates empty layer graphs edges_from_temporal_table: Convert DataFrame to edge list

classmethod from_edges(*edges: List[Dict | List]*, *network_type: str = 'multilayer'*, *directed: bool = False*, *input_type: str = 'dict'*) → *multi_layer_network*

Create a multilayer network directly from a list of edges.

This is a convenience factory method that creates a network and populates it with edges in a single call, supporting method chaining patterns.

Parameters

- **edges** – List of edges in dict or list format
- **network_type** – Type of network ('multilayer' or 'multiplex')
- **directed** – Whether the network is directed
- **input_type** – Format of edge data ('dict' or 'list')

Returns

New network instance with edges added

Return type

multi_layer_network

Examples

```
>>> # Create from dict format
>>> net = multi_layer_network.from_edges([
...     {'source': 'A', 'target': 'B',
...      'source_type': 'layer1', 'target_type': 'layer1'},
...     {'source': 'B', 'target': 'C',
...      'source_type': 'layer1', 'target_type': 'layer1'}
... ])
>>> len(net)
3
```

```
>>> # Create from list format
>>> net = multi_layer_network.from_edges([
...     ['A', 'layer1', 'B', 'layer1', 1],
...     ['B', 'layer1', 'C', 'layer1', 1]
... ], input_type='list')
>>> net.edge_count
2
```

from_homogeneous_hypergraph(*H*)

Decode a homogeneous graph created by to_homogeneous_hypergraph.

This method reconstructs a multiplex network from its incidence gadget encoding. It identifies edge-nodes by their degree and cycle structure, then reconstructs the original layers based on cycle lengths (prime numbers).

Parameters

`H` (*networkx.Graph*) – Homogeneous graph created by to_homogeneous_hypergraph().

Returns

- *dict* – Dictionary mapping layer names to lists of edges: {layer: [(u, v), ...]}
- *Examples* – Example requires proper network setup - for illustration only.

```
>>> network = multi_layer_network()
>>> network.add_layer("A")
>>> network.add_nodes([("1", "A"), ("2", "A")])
>>> network.add_edges([(("1", "A"), ("2", "A"))])
>>> H, node_map, edge_info = network.to_homogeneous_
↪hypergraph()
>>> recovered = network.from_homogeneous_hypergraph(H)
>>> print(recovered)
{'layer_with_prime_2': [('1', '2')]}
```

Notes

The decoded layer names indicate the prime number used for encoding: - "layer_with_prime_2" corresponds to the first layer - "layer_with_prime_3" corresponds to the second layer, etc.

classmethod from_networkx(*G: Graph*, *network_type: str = 'multilayer'*, *directed: bool | None = None*) → *multi_layer_network*

Create a multi_layer_network from a NetworkX graph.

This class method converts a NetworkX graph into a py3plex multi_layer_network. For multilayer networks, nodes should be tuples of (node_id, layer).

Parameters

- **G** – NetworkX graph to convert
- **network_type** – Type of network ('multilayer' or 'multiplex')
- **directed** – Whether to treat the network as directed. If None, inferred from G.

Returns

A new multi_layer_network instance

Return type

multi_layer_network

Examples

```
>>> import networkx as nx
>>> G = nx.Graph()
>>> G.add_nodes_from([('A', 'layer1'), ('B', 'layer1')])
>>> G.add_edge(('A', 'layer1'), ('B', 'layer1'))
>>> net = multi_layer_network.from_networkx(G)
>>> print(net)
<multi_layer_network: type=multilayer, directed=False, nodes=2,␣
↪edges=1, layers=1>
```

Notes

- For proper multilayer behavior, ensure nodes are (node_id, layer) tuples
- Edge attributes are preserved during conversion
- The input graph is copied, not referenced

get_decomposition(*heuristic='all'*, *cycle=None*, *parallel=False*, *alpha=1*, *beta=0*)

Core method for obtaining a network's decomposition in terms of relations

get_decomposition_cycles(*cycle=None*)

A supporting method for obtaining decomposition triplets

get_degrees()

A simple wrapper which computes node degrees.

get_edges(*data: bool = False*, *multiplex_edges: bool = False*) → Any

Iterate over edges in the network.

This method behaves differently based on the network_type:

- **multilayer**: Returns all edges without filtering.
- **multiplex**: By default, filters out coupling edges (auto-generated inter-layer edges connecting each node to itself in other layers). Set multiplex_edges=True to include coupling edges.

Parameters

- **data** – If True, return edge data along with edge tuples
- **multiplex_edges** – If True, include coupling edges in multiplex networks. Only relevant when network_type='multiplex'. Coupling edges are automatically added to connect each node to its counterparts in other layers.

Yields
Edge tuples, optionally with data. For multiplex networks with multiplex_edges=False, coupling edges are excluded.

Raises
ValueError – If network_type is not 'multilayer' or 'multiplex'.

Examples

```
>>> net = multi_layer_network(network_type='multilayer')
>>> net.add_edges([
...     {'source': 'A', 'target': 'B',
...      'source_type': 'layer1', 'target_type': 'layer1'}
... ])
<multi_layer_network: type=multilayer, directed=True, nodes=2,␣
↪edges=1, layers=1>
>>> list(net.get_edges())
[(('A', 'layer1'), ('B', 'layer1'))]
```

See also

- `__init__()` for the difference between multilayer and multiplex
- `_couple_all_edges()` for how coupling edges are created

get_label_matrix()

Return network labels

get_layers(*style='diagonal'*, *compute_layouts='force'*, *layout_parameters=None*, *verbose=True*)

A method for obtaining layerwise distributions

get_neighbors(*node_id: str*, *layer_id: str | None = None*) → Any

Get neighbors of a node in a specific layer.

Parameters

- **node_id** – Node identifier
- **layer_id** – Layer identifier (optional)

Returns
Iterator of neighbor nodes

get_node_attribute(*node: Any*, *attribute: str*, *layer: Any | None = None*) → Any

Get an attribute value for a given node.

Parameters

- **node** (*Any*) – Node identifier.
- **attribute** (*str*) – Name of the attribute to retrieve.
- **layer** (*Any, optional*) – Layer identifier. If None and node is a tuple, assumes node is (node_id, layer).

Returns
The attribute value, or None if not found.

Return type
Any

Examples

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network(directed=False)
>>> net.add_edges([['A', 'L1', 'B', 'L1', 1]], input_type='list')
>>> net.set_node_attribute('A', 'score', 42.0, 'L1')
>>> print(net.get_node_attribute('A', 'score', 'L1'))
42.0
```

get_nodes(*data: bool = False*) → Any

A method for obtaining a network's nodes

Parameters
data – If True, return node data along with node identifiers

Yields
Node identifiers, optionally with data

get_nx_object()

Return only core network with proper annotations

get_partition(*node: Any*, *layer: Any | None = None*) → int | None

Get the community/partition ID for a given node.

Parameters

- **node** (*Any*) – Node identifier.
- **layer** (*Any, optional*) – Layer identifier. If None and node is a tuple, assumes node is (node_id, layer).

Returns
Community ID, or None if node doesn't have a partition assigned.

Return type
int or None

Examples

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network(directed=False)
>>> net.add_edges([['A', 'L1', 'B', 'L1', 1]], input_type='list')
>>> partition = {('A', 'L1'): 0, ('B', 'L1'): 1}
>>> net.assign_partition(partition)
>>> print(net.get_partition('A', 'L1'))
0
```

get_supra_adjacency_matrix(*mtype='sparse'*)

Get sparse representation of the supra matrix.

Parameters

mtype – 'sparse' or 'dense' - matrix representation type

Returns

Supra-adjacency matrix in requested format

> **Warning**
>
> For large multilayer networks, dense matrices can consume significant memory (N*L)^2 * 8 bytes for float64.

get_tensor(*sparsity_type='bsr'*)

Get sparse tensor representation of the multilayer network.

Returns the supra-adjacency matrix in the specified sparse format. This method provides a tensor-like view of the multilayer network, useful for mathematical analysis and matrix-based algorithms.

Parameters

sparsity_type – Sparse matrix format to use. Options include: - 'bsr' (default): Block Sparse Row format - 'csr': Compressed Sparse Row format - 'csc': Compressed Sparse Column format - 'coo': Coordinate format - 'lil': List of Lists format - 'dok': Dictionary of Keys format

Returns

Supra-adjacency matrix in specified format

Return type

scipy.sparse matrix

Example

```
>>> net = multi_layer_network()
>>> tensor = net.get_tensor(sparsity_type='csr')
>>> print(tensor.shape)
```

> **Note**
>
> The returned matrix is the same as get_supra_adjacency_matrix(mtype='sparse') but with control over the specific sparse format used.

get_unique_entity_counts()

Count unique entities in the network.

Returns

(total_unique_nodes, unique_node_ids, nodes_per_layer)

- total_unique_nodes: count of unique (node, layer) tuples
- unique_node_ids: count of unique node IDs (across all layers)
- nodes_per_layer: dict mapping layer to count of nodes in that layer

Return type

tuple

invert(*override_core=False*)

invert the nodes to edges. Get the "edge graph". Each node is here an edge.

property is_empty: bool

Check if the network is empty (has no nodes).

Returns

True if network has no nodes

Return type

bool

Examples

```
>>> net = multi_layer_network()
>>> net.is_empty
True
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}])
>>> net.is_empty
False
```

property layer_count: int

Number of unique layers in the network.

Returns

Count of distinct layers

Return type

int

Examples

```
>>> net = multi_layer_network()
>>> net.add_nodes([
...     {'source': 'A', 'type': 'layer1'},
...     {'source': 'B', 'type': 'layer2'}
... ])
>>> net.layer_count
2
```

property layers: List[Any]

List of unique layer identifiers in the network.

Returns
Sorted list of layer identifiers

Return type
list

Examples

```
>>> net = multi_layer_network()
>>> net.add_nodes([
...     {'source': 'A', 'type': 'social'},
...     {'source': 'B', 'type': 'work'}
... ])
>>> net.layers
['social', 'work']
```

load_embedding(*embedding_file*)

Embedding loading method

load_layer_name_mapping(*mapping_name*, *header=False*)

Layer-node mapping loader method

Parameters
param1 – The name of the mapping file.

Returns
self.layer_name_map is filled, returns nothing.

load_network(*input_file: str | None = None*, *directed: bool = False*, *input_type: str = 'gml'*, *label_delimiter: str = '---'*) → *multi_layer_network*

Load a network from file.

This method loads and prepares a given network. The behavior depends on the `network_type` set during initialization:

- **multilayer**: Network is loaded as-is. No automatic edges are added.
- **multiplex**: After loading, coupling edges are automatically created between each node and its counterparts in other layers. These edges have type='coupling' and can be filtered via get_edges().

Parameters

- **input_file** – Path to the network file to load
- **directed** – Whether the network is directed
- **input_type** – Format of the input file. Supported values: - 'gml': Graph Modeling Language format - 'graphml': GraphML XML format - 'edgelist': Simple edge list (source target [weight]) - 'multiedgelist': Multilayer edge list (node1 layer1 node2 layer2 weight) - 'multiplex_edges': Multiplex format (layer node1 node2 weight) - 'multiplex_folder': Folder with layer files - 'gpickle': Python pickle format - 'nx': NetworkX graph object - 'sparse': Sparse matrix format
- **label_delimiter** – Delimiter used to separate layer names in node labels

Returns

Self for method chaining. Populates self.core_network, self.labels, and self.activity.

Note

For multiplex networks, use input_type='multiplex_edges' or 'multiplex_folder' with network_type='multiplex' to get automatic coupling edges.

Examples

```
>>> # Load multilayer network (no automatic coupling)
>>> net = multi_layer_network(network_type='multilayer')
>>> net.load_network('data.gml', input_type='gml')
```

```
>>> # Load multiplex network (automatic coupling edges)
>>> net = multi_layer_network(network_type='multiplex')
>>> net.load_network('data.edges', input_type='multiplex_edges')
```

load_network_activity(*activity_file*)

Network activity loader

Args:

param1: The name of the generic activity file -> 65432 61888 1377688175 RE

, n1 n2 timestamp and layer name. Note that layer node mappings MUST be loaded in order to map nodes to activity properly.

Returns:

self.activity is filled.

load_temporal_edge_information(*input_file=None*, *input_type='edge_activity'*, *directxed=False*, *layer_mapping=None*)

A method for loading temporal edge information

merge_with(*target_px_object*)

Merge two px objects.

monitor(*message*)

A simple monitor method for logging

monoplex_nx_wrapper(*method*, *kwargs=None*)

A generic networkx function wrapper.

Parameters

- **method** (*str*) – Name of the NetworkX function to call (e.g., 'degree_centrality', 'betweenness_centrality')
- **kwargs** (*dict, optional*) – Keyword arguments to pass to the NetworkX function. For example, for betweenness_centrality you can pass: - weight: Edge attribute to use as weight - normalized: Whether to normalize betweenness values - distance: Edge attribute to use as distance (for closeness_centrality)

Returns
The result of the NetworkX function call.

Raises
`AttributeError` – If the specified method does not exist in NetworkX.

Example

Unweighted betweenness centrality centralities = network.monoplex_nx_wrapper("betweenness_centrality")

Weighted betweenness centrality centralities = network.monoplex_nx_wrapper("betweenness_centrality", kwargs={"weight": "weight"})

With multiple parameters centralities = network.monoplex_nx_wrapper("betweenness_centrality",

kwargs={"weight": "weight", "normalized": True})

property node_count: int

Number of nodes in the network.

Returns
Total count of nodes (node-layer pairs)

Return type
int

Examples

```
>>> net = multi_layer_network()
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}])
>>> net.node_count
1
```

read_ground_truth_communities(*cfile*)

Parse ground truth community file and make mappings to the original nodes. This works based on node ID mappings, exact node,layer tuplets are to be added. :param param1: ground truth communities.

Returns
self.ground_truth_communities

remove_edges(*edge_dict_list: List[Dict] | List[List]*, *input_type: str = 'list'*) → None

A method for removing edges..

Parameters

- **edge_dict_list** – Edge data in dict or list format
- **input_type** – Format of edge data ('dict' or 'list')

Raises
`Exception` – If input_type is not valid

remove_layer_edges()

Remove all edges from separate layer graphs while keeping nodes.

This method creates empty copies of each layer graph with all nodes intact but no edges. Useful for reconstructing networks with different edge sets or for temporal network analysis.

Notes

- Requires split_to_layers() to be called first
- Stores empty layer graphs in self.tmp_layers
- Original graphs in self.separate_layers remain unchanged
- All nodes and their attributes are preserved

Raises
`RuntimeError` – If split_to_layers() hasn't been called yet

> **See also**
>
> split_to_layers: Must be called before this method fill_tmp_with_edges: Add edges back to emptied layers

remove_nodes(*node_dict_list*, *input_type='dict'*)
Remove nodes from the network

save_network(*output_file=None*, *output_type='edgelist'*)
Save the network to a file in various formats.

This method exports the multilayer network to different file formats for persistence, sharing, or use with other tools.

Parameters

- **output_file** – Path where the network should be saved
- **output_type** – Format for saving ('edgelist', 'multiedgelist', 'multiedgelist_encoded', or 'gpickle')

Supported Formats:

- 'edgelist': Simple edge list format (standard NetworkX)
- 'multiedgelist': Multilayer edge list with layer information
- 'multiedgelist_encoded': Multilayer edge list with integer encoding
- 'gpickle': Python pickle format (preserves all attributes)

Examples

```
>>> net = multi_layer_network()
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}])
>>> net.add_edges([{'source': 'A', 'target': 'B',
...                 'source_type': 'layer1', 'target_type': 'layer1'}
↪])
>>> net.save_network('network.txt', output_type='multiedgelist')
```

```
>>> # For faster I/O with all metadata preserved
>>> net.save_network('network.gpickle', output_type='gpickle')
```

Notes

- 'gpickle' format preserves all node/edge attributes
- 'multiedgelist_encoded' creates node_map and layer_map attributes
- Edge weights and types are preserved in supported formats

serialize_to_edgelist(*edgelist_file='./tmp/tmpedgelist.txt'*, *tmp_folder='tmp'*, *out_folder='out'*, *multiplex=False*)

Serialize the multilayer network to an edgelist file.

Converts the network to a numeric edgelist format suitable for external tools and algorithms that require integer node/layer identifiers.

Parameters

- **edgelist_file** – Path to output edgelist file (default: "./tmp/tmpedgelist.txt")
- **tmp_folder** – Temporary folder for intermediate files (default: "tmp")
- **out_folder** – Output folder for results (default: "out")
- **multiplex** – If True, use multiplex format (node layer node layer weight) If False, use simple edgelist format (node1 node2 weight)

Returns

Inverse node mapping (numeric_id -> original_node_tuple)
Use this to decode results from external algorithms

Return type
dict

File Formats:

- Multiplex format: node1_id layer1_id node2_id layer2_id weight
- Simple format: node1_id node2_id weight

Notes

- Creates tmp_folder and out_folder if they don't exist
- Nodes are mapped to sequential integers starting from 0
- Layers are mapped to sequential integers starting from 0 (multiplex mode)
- All edges have weight 1 unless explicitly specified

Examples

Example requires file output - for illustration only.

```
>>> net = multi_layer_network()
>>> # ... build network ...
```

(continues on next page)

(continued from previous page)

```
>>> node_mapping = net.serialize_to_edgelist(
...     edgelist_file='network.txt',
...     multiplex=True
... )
>>> # Use node_mapping to decode results
>>> original_node = node_mapping[0]  # Get original node for id 0
```

See also

load_network: Load networks from file save_network: Alternative serialization method

set_node_attribute(*node: Any*, *attribute: str*, *value: Any*, *layer: Any | None = None*) → None

Set an attribute value for a given node.

Parameters

- **node** (*Any*) – Node identifier.
- **attribute** (*str*) – Name of the attribute to set.
- **value** (*Any*) – Value to assign to the attribute.
- **layer** (*Any, optional*) – Layer identifier. If None and node is a tuple, assumes node is (node_id, layer).

Examples

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network(directed=False)
>>> net.add_edges([['A', 'L1', 'B', 'L1', 1]], input_type='list')
>>> net.set_node_attribute('A', 'score', 42.0, 'L1')
>>> print(net.get_node_attribute('A', 'score', 'L1'))
42.0
```

sparse_to_px(*directed=None*)

Convert sparse matrix to py3plex format

Parameters

directed – Whether the network is directed (uses self.directed if None)

split_to_layers(*style='diagonal'*, *compute_layouts='force'*, *layout_parameters=None*, *verbose=True*, *multiplex=False*, *convert_to_simple=False*)

A method for obtaining layerwise distributions

subnetwork(*input_list=None*, *subset_by='node_layer_names'*)

Construct a subgraph based on a set of nodes.

summary()

Generate a summary of network statistics.

Computes and returns key metrics about the multilayer network structure.

Returns

Network statistics including:

- 'Number of layers': Count of unique layers
- 'Nodes': Total number of nodes
- 'Edges': Total number of edges
- 'Mean degree': Average node degree
- 'CC': Number of connected components

Return type
dict

Examples

```
>>> net = multi_layer_network()
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}])
>>> net.add_edges([{'source': 'A', 'target': 'B',
...                 'source_type': 'layer1', 'target_type': 'layer1'}
↪])
>>> stats = net.summary()
>>> print(f"Network has {stats['Nodes']} nodes and {stats['Edges']}␣
↪edges")
Network has 2 nodes and 1 edges
```

Notes

- Connected components are computed on the undirected version
- Mean degree is averaged across all nodes in all layers

test_scale_free()
Test the scale-free-nness of the network

to_homogeneous_hypergraph()
Transform a multiplex network into a homogeneous graph using incidence gadget encoding.

This method encodes the multiplex structure where each layer is represented by a unique prime number signature. Each edge becomes an edge-node connected to its endpoints and a cycle of length prime-1 that encodes the layer.

Returns

- *tuple (H, node_mapping, edge_info)* –

  H
  [networkx.Graph] Homogeneous unlabeled graph encoding the multiplex structure.

  node_mapping
  [dict] Maps each original node to its vertex-node in H.

  edge_info
  [dict] Mapping from each edge-node in H to its (layer, endpoints) tuple.

- *Examples* – Example requires sympy dependency - for illustration only.
- **>>> network = multi_layer_network(directed=False) # doctest** (*+SKIP*)
- **>>> network.add_nodes([{'source'** (*'1', 'type': 'A'}, {'source': '2', 'type': 'A'}], input_type='dict') # doctest: +SKIP*)
- **>>> network.add_edges([{'source'** (*'1', 'target': '2', 'source_type': 'A', 'target_type': 'A'}], input_type='dict') # doctest: +SKIP*)
- **>>> H, node_map, edge_info = network.to_homogeneous_hypergraph() # doctest** (*+SKIP*)
- **>>> print(f"Homogeneous graph has {len(H.nodes())} nodes") # doctest** (*+SKIP*)

Notes

This transformation uses prime-based signatures to encode layers: - Each layer is assigned a unique prime number (2, 3, 5, 7, ...) - Each edge in layer with prime p is connected to a cycle of length p - The cycle structure uniquely identifies the layer

`to_json()`

A method for exporting the graph to a json file

Args: self

`to_networkx()` → Graph

Convert the multilayer network to a NetworkX graph.

Returns a copy of the core network as a NetworkX graph. The returned graph preserves all node and edge attributes, including layer information for multilayer networks (where nodes are typically (node_id, layer) tuples).

Returns
A NetworkX graph (MultiGraph or MultiDiGraph depending on network type)

Return type
nx.Graph

Examples

```
>>> net = multi_layer_network(directed=False)
>>> net.add_nodes([{'source': 'A', 'type': 'layer1'}])
>>> nx_graph = net.to_networkx()
>>> print(type(nx_graph))
<class 'networkx.classes.multigraph.MultiGraph'>
```

Notes

- For multilayer networks, nodes are tuples: (node_id, layer)
- All edge attributes (weight, type, etc.) are preserved
- The returned graph is a copy, not a reference

to_sparse_matrix(*replace_core=False*, *return_only=False*)

Conver the matrix to scipy-sparse version. This is useful for classification.

visualize_matrix(*kwargs=None*)

Plot the matrix – this plots the supra-adjacency matrix

visualize_network(*style='diagonal'*, *parameters_layers=None*, *parameters_multiedges=None*, *show=False*, *compute_layouts='force'*, *layouts_parameters=None*, *verbose=True*, *orientation='upper'*, *resolution=0.01*, *axis=None*, *fig=None*, *no_labels=False*, *linewidth=1.7*, *alphachannel=0.3*, *linepoints='-.'*, *legend=False*)

Visualize the multilayer network.

Supports multiple visualization styles: - 'diagonal': Layer-centric diagonal layout with inter-layer edges - 'hairball': Aggregate hairball plot of all layers - 'flow' or 'alluvial': Layered flow visualization with horizontal bands - 'sankey': Sankey diagram showing inter-layer flow strength

Parameters

- **style** – Visualization style ('diagonal', 'hairball', 'flow', 'alluvial', or 'sankey')
- **parameters_layers** – Custom parameters for layer drawing
- **parameters_multiedges** – Custom parameters for edge drawing
- **show** – Show plot immediately
- **compute_layouts** – Layout algorithm (currently unused)
- **layouts_parameters** – Layout parameters (currently unused)
- **verbose** – Enable verbose output
- **orientation** – Edge orientation for diagonal style
- **resolution** – Resolution for edge curves
- **axis** – Optional matplotlib axis to draw on
- **fig** – Optional matplotlib figure (currently unused)
- **no_labels** – Hide network labels
- **linewidth** – Width of edge lines
- **alphachannel** – Alpha channel for edge transparency
- **linepoints** – Line style for edges
- **legend** – Show legend (for hairball style)

Returns

Matplotlib axis object

Raises

Exception – If style is not recognized

Performance Notes:

For large networks (>500 nodes), visualization performance may degrade: - Layout computation can be slow (O(n^2) for force-directed layouts) - Rendering many edges is memory and CPU intensive - Consider filtering or sampling for exploratory visualization - Use simpler layouts or increase layout iteration limits

Approximate rendering times on typical hardware: - 100 nodes: <1 second - 500 nodes: 5-10 seconds - 1000 nodes: 30-60 seconds - 5000+ nodes: Several minutes, may run out of memory

visualize_ricci_core(*alpha: float = 0.5*, *iterations: int = 10*, *layout_type: str = 'mds'*, *dim: int = 2*, ***kwargs*)

Visualize the aggregated core network using Ricci-flow-based layout.

This method is a high-level wrapper for Ricci-flow-based visualization of the core (aggregated) network. It automatically computes Ricci flow if not already done and creates an informative layout that emphasizes geometric structure and communities.

Parameters

- **alpha** – Ollivier-Ricci parameter for flow computation. Default: 0.5.
- **iterations** – Number of Ricci flow iterations. Default: 10.
- **layout_type** – Layout algorithm ("mds", "spring", "spectral"). Default: "mds".
- **dim** – Dimensionality of layout (2 or 3). Default: 2.
- ****kwargs** – Additional arguments passed to visualize_multilayer_ricci_core.

Returns

Tuple of (figure, axes, positions_dict).

Raises

RicciBackendNotAvailable – If GraphRicciCurvature is not installed.

Examples

Example requires GraphRicciCurvature library - for illustration only.

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network()
>>> net.add_edges([
...     ['A', 'layer1', 'B', 'layer1', 1],
...     ['B', 'layer1', 'C', 'layer1', 1],
... ], input_type="list")
>>> fig, ax, pos = net.visualize_ricci_core()
>>> import matplotlib.pyplot as plt
>>> plt.show()
```

See also

visualize_ricci_layers: Per-layer visualization with Ricci flow visualize_ricci_supra: Supra-graph visualization with Ricci flow

visualize_ricci_layers(*layers: List[Any] | None = None*, *alpha: float = 0.5*, *iterations: int = 10*, *layout_type: str = 'mds'*, *share_layout: bool = True*, ***kwargs*)

Visualize individual layers using Ricci-flow-based layouts.

This method creates visualizations of individual layers with layouts derived from Ricci flow. Layers can share a common coordinate system (for easier comparison) or have independent layouts.

Parameters

- **layers** – List of layer identifiers to visualize. If None, uses all layers.
- **alpha** – Ollivier-Ricci parameter. Default: 0.5.
- **iterations** – Number of Ricci flow iterations. Default: 10.
- **layout_type** – Layout algorithm. Default: "mds".
- **share_layout** – If True, use shared coordinates across layers. Default: True.
- ****kwargs** – Additional arguments passed to visualize_multilayer_ricci_layers.

Returns

Tuple of (figure, layer_positions_dict).

Raises

RicciBackendNotAvailable – If GraphRicciCurvature is not installed.

Examples

```
>>> fig, pos_dict = net.visualize_ricci_layers(
...     arrangement="grid", share_layout=True
... )
>>> import matplotlib.pyplot as plt
>>> plt.show()
```

See also

visualize_ricci_core: Core network visualization with Ricci flow visualize_ricci_supra: Supra-graph visualization with Ricci flow

visualize_ricci_supra(*alpha: float = 0.5*, *iterations: int = 10*, *layout_type: str = 'mds'*, *dim: int = 2*, ***kwargs*)

Visualize the full supra-graph using Ricci-flow-based layout.

This method visualizes the complete multilayer structure including both intra-layer edges (within layers) and inter-layer edges (coupling between layers) using a layout derived from Ricci flow.

Parameters

- **alpha** – Ollivier-Ricci parameter. Default: 0.5.
- **iterations** – Number of Ricci flow iterations. Default: 10.
- **layout_type** – Layout algorithm. Default: "mds".
- **dim** – Dimensionality (2 or 3). Default: 2.

- ****kwargs** – Additional arguments passed to visualize_multilayer_ricci_supra.

Returns
Tuple of (figure, axes, positions_dict).

Raises
RicciBackendNotAvailable – If GraphRicciCurvature is not installed.

Examples

```
>>> fig, ax, pos = net.visualize_ricci_supra(dim=3)
>>> import matplotlib.pyplot as plt
>>> plt.show()
```

See also

visualize_ricci_core: Core network visualization with Ricci flow visualize_ricci_layers: Per-layer visualization with Ricci flow

py3plex.core.multinet.**require**(**args*, ***kwargs*)

py3plex.core.parsers.**ensure**(**args*, ***kwargs*)

py3plex.core.parsers.**load_edge_activity_file**(*fname: str*, *layer_mapping: str | None = None*) → DataFrame

py3plex.core.parsers.**load_edge_activity_raw**(*activity_file: str*, *layer_mappings: dict*) → DataFrame

Basic parser for loading generic activity files. Here, temporal edges are given as tuples -> this can be easily transformed for example into a pandas dataframe!

Parameters

- **activity_file** – Path to activity file
- **layer_mappings** – Dictionary mapping layer names to IDs

Returns
DataFrame with edge activity data

py3plex.core.parsers.**load_temporal_edge_information**(*input_network: str*, *input_type: str*, *layer_mapping: str | None = None*) → DataFrame | None

py3plex.core.parsers.**parse_detangler_json**(*file_path: str*, *directed: bool = False*) → Tuple[MultiGraph | MultiDiGraph, None]

Parser for generic Detangler files :param file_path: Path to Detangler JSON file :param directed: Whether to create directed graph

py3plex.core.parsers.**parse_edgelist_multi_types**(*input_name: str*, *directed: bool*) → Tuple[MultiGraph | MultiDiGraph, None]

Parse an edgelist file with multiple edge types.

Reads a text file where each line represents an edge, optionally with weights and edge types. Lines starting with '#' are treated as comments.

File Format:

node1 node2 [weight] [edge_type]

Lines starting with '#' are ignored (comments)

Parameters

- **input_name** – Path to edgelist file
- **directed** – Whether to create a directed graph

Returns

(parsed_graph, None for labels)

Return type

Tuple[Union[nx.MultiGraph, nx.MultiDiGraph], None]

Notes

- All nodes are assigned to a "null" layer
- Default weight is 1 if not specified
- Edge type is optional (4th column)
- Handles both 2-column (node pairs) and 3+ column formats

Examples

```
>>> # File content:
>>> # A B 1.0 friendship
>>> # B C 2.0 collaboration
>>> graph, _ = parse_edgelist_multi_types('edges.txt', directed=False)
```

py3plex.core.parsers.**parse_embedding**(*input_name: str*) → Tuple[ndarray, ndarray]

Loader for generic embedding as outputed by GenSim

Parameters

input_name – Path to embedding file

Returns

Tuple of (embedding matrix, embedding indices)

py3plex.core.parsers.**parse_gml**(*file_name: str*, *directed: bool*) → Tuple[MultiGraph | MultiDiGraph, None]

Parse a gml network.

Parameters

- **file_name** – Path to GML file
- **directed** – Whether to create directed graph

Returns

Tuple of (multigraph, possible labels)

Contracts:

- Precondition: file_name must be a non-empty string
- Postcondition: result graph is not None
- Postcondition: result is a MultiGraph or MultiDiGraph

`py3plex.core.parsers.parse_gpickle(file_name: str, directed: bool = False, layer_separator: str | None = None) → Tuple[MultiGraph | MultiDiGraph, None]`

A parser for generic Gpickle as stored by Py3plex.

Parameters

- **file_name** – Path to gpickle file
- **directed** – Whether to create directed graph
- **layer_separator** – Optional separator for layer parsing

Contracts:

- Precondition: file_name must be a non-empty string
- Postcondition: result graph is not None
- Postcondition: result is a MultiGraph or MultiDiGraph

`py3plex.core.parsers.parse_gpickle_biomine(file_name: str, directed: bool) → Tuple[MultiGraph | MultiDiGraph, None]`

Gpickle parser for biomine graphs :param file_name: Path to gpickle containing BioMine data :param directed: Whether to create directed graph

`py3plex.core.parsers.parse_matrix(file_name: str, directed: bool) → Tuple[Any, Any]`

Parser for matrices.

Parameters

- **file_name** – Path to .mat file
- **directed** – Whether the graph is directed

Returns

Tuple of (network, group) from the .mat file

Contracts:

- Precondition: file_name must be a non-empty string
- Postcondition: network must not be None

`py3plex.core.parsers.parse_matrix_to_nx(file_name: str, directed: bool) → Graph | DiGraph`

Parser for matrices to NetworkX graph.

Parameters

- **file_name** – Path to .mat file
- **directed** – Whether to create directed graph

Returns

NetworkX Graph or DiGraph

`py3plex.core.parsers.parse_multi_edgelist(input_name: str, directed: bool) → Tuple[MultiGraph | MultiDiGraph, None]`

A generic multiedgelist parser n l n l w :param input_name: Path to text file containing multiedges :param directed: Whether to create directed graph py3plex.core.parsers.**parse_multiedge_tuple_list**(*network: list*, *directed: bool*) → Tuple[MultiGraph | MultiDiGraph, None]

Parse a list of edge tuples into a multilayer network.

Parameters

- **network** – List of edge tuples (node_first, node_second, layer_first, layer_second, weight)
- **directed** – Whether to create directed graph

py3plex.core.parsers.**parse_multiplex_edges**(*input_name: str*, *directed: bool*) → Tuple[MultiGraph | MultiDiGraph, None]

Parse a multiplex edgelist file where each line specifies layer and edge.

File Format:

layer node1 node2 [weight]

Each line: layer_id source_node target_node [optional_weight]

Parameters

- **input_name** – Path to multiplex edgelist file
- **directed** – Whether to create a directed graph

Returns

(parsed_graph, None for labels)

Return type

Tuple[Union[nx.MultiGraph, nx.MultiDiGraph], None]

Notes

- Each edge belongs to a specific layer (first column)
- Nodes are represented as (node_id, layer) tuples
- Default weight is 1 if not specified
- All edges have type='default' attribute
- Automatically couples nodes across layers for multiplex structure

Examples

```
>>> # File content:
>>> # layer1 A B 1.5
>>> # layer2 A B 2.0
>>> # layer1 B C 1.0
>>> graph, _ = parse_multiplex_edges('multiplex.txt', directed=False)
>>> # Creates nodes: (A, layer1), (A, layer2), (B, layer1), etc.
```

py3plex.core.parsers.**parse_multiplex_folder**(*input_folder: str*, *directed: bool*) → Tuple[MultiGraph | MultiDiGraph, None, DataFrame]

Parse a folder containing multiplex network files.

Expects a folder with specific file formats for edges, layers, and optional activity.

Expected Files:

- *.edges: Edge information (format: layer_id node1 node2 weight)
- layers.txt: Layer definitions (format: layer_id layer_name)
- activity.txt: Optional temporal activity (format: node1 node2 timestamp layer_name)

Parameters

- **input_folder** – Path to folder containing multiplex network files
- **directed** – Whether to create a directed graph

Returns

- Union[nx.MultiGraph, nx.MultiDiGraph]: Parsed multilayer graph
- None: Placeholder for labels (not used)
- pd.DataFrame: Time series activity data (empty if no activity.txt)

Return type

Tuple containing

Notes

- Uses glob to find files with specific extensions
- Layer mapping is built from layers.txt
- Activity data is optional and returned as pandas DataFrame
- Nodes are represented as (node_id, layer_id) tuples

Examples

```
>>> # Folder structure:
>>> # my_network/
>>> #   network.edges
>>> #   layers.txt
>>> #   activity.txt (optional)
>>> graph, _, activity_df = parse_multiplex_folder('my_network/',␣
↪directed=False)
```

py3plex.core.parsers.**parse_network**(*input_name: str | Any*, *f_type: str = 'gml'*, *directed: bool = False*, *label_delimiter: str | None = None*, *network_type: str = 'multilayer'*) → Tuple[Any, Any, Any]

A wrapper method for available parsers!

Parameters

- **input_name** – Path to network file or network object
- **f_type** – Type of file format to parse
- **directed** – Whether to create directed graph
- **label_delimiter** – Optional delimiter for labels
- **network_type** – Type of network (multilayer or multiplex)

Returns
Tuple of (parsed_network, labels, time_series)

`py3plex.core.parsers.parse_nx`(*nx_object: Graph*, *directed: bool*) → Tuple[Graph, None]

Core parser for networkx objects.

Parameters

- **nx_object** – A networkx graph
- **directed** – Whether the graph is directed

Returns
Tuple of (graph, None)

Contracts:

- Precondition: nx_object must not be None
- Precondition: nx_object must be a NetworkX graph
- Postcondition: result graph is not None

`py3plex.core.parsers.parse_simple_edgelist`(*input_name: str*, *directed: bool*) → Tuple[Graph | DiGraph, None]

Simple edgelist n n w :param input_name: Path to text file :param directed: Whether to create directed graph

`py3plex.core.parsers.parse_spin_edgelist`(*input_name: str*, *directed: bool*) → Tuple[Graph, None]

Parse SPIN format edgelist file.

SPIN format includes node pairs with edge tags and optional weights.

File Format:
node1 node2 tag [weight]

Each line: source_node target_node edge_tag [optional_weight]

Parameters

- **input_name** – Path to SPIN edgelist file
- **directed** – Whether to create directed graph (currently creates undirected)

Returns
(parsed_graph, None for labels)

Return type
Tuple[nx.Graph, None]

Notes

- Currently always returns nx.Graph (undirected) regardless of directed parameter
- Edge tag is stored in edge 'type' attribute
- Default weight is 1 if not specified (4th column)

Examples

```
>>> # File content:
>>> # A B protein_interaction 0.95
>>> # B C gene_regulation 0.80
>>> graph, _ = parse_spin_edgelist('spin_edges.txt', directed=False)
```

`py3plex.core.parsers.require(*args, **kwargs)`

`py3plex.core.parsers.save_edgelist(input_network: Graph, output_file: str, attributes: bool = False) → None`

Save network to edgelist format.

For multilayer networks (where nodes are tuples of (node_id, layer)), saves in format: node1 layer1 node2 layer2

For regular networks, saves in format: node1 node2

`py3plex.core.parsers.save_gpickle(input_network: Any, output_file: str) → None`

`py3plex.core.parsers.save_multiedgelist(input_network: Any, output_file: str, attributes: bool = False, encode_with_ints: bool = False) → Tuple[Dict[Any, str], Dict[Any, str]] | None`

Save multiedgelist – as n1, l1, n2, l2, w

Returns
When encode_with_ints is True, returns tuple of (node_encodings, type_encodings) Otherwise returns None

`py3plex.core.converters.compute_layout(network: Graph, compute_layouts: str, layout_parameters: Dict[str, Any] | None, verbose: bool) → Graph`

Compute and normalize layout for a network.

Parameters

- **network** – NetworkX graph to compute layout for
- **compute_layouts** – Layout algorithm to use ('force', 'random', 'custom_coordinates')
- **layout_parameters** – Optional parameters for layout algorithms
- **verbose** – Whether to print verbose output

Returns
Network with 'pos' attribute added to nodes

Contracts:

- Precondition: network must not be None and must have at least one node
- Precondition: compute_layouts must be a valid algorithm name
- Postcondition: all nodes have 'pos' attribute (layout preserves nodes)

`py3plex.core.converters.ensure(*args, **kwargs)`

`py3plex.core.converters.prepare_for_parsing(multinet)`

Compute layout for a hairball visualization

Parameters
`param1` (*obj*) – multilayer object

Returns
(names, prepared network)

Return type
tuple

`py3plex.core.converters.prepare_for_visualization(`*multinet: Graph*, *network_type: str = 'multilayer'*, *compute_layouts: str = 'force'*, *layout_parameters: Dict[str, Any] | None = None*, *verbose: bool = True*, *multiplex: bool = False*`)` → Tuple[List[Any], List[Graph], Any]

This functions takes a multilayer object and returns individual layers, their names, as well as multilayer edges spanning over multiple layers.

Parameters

- **multinet** – multilayer network object
- **network_type** – "multilayer" or "multiplex"
- **compute_layouts** – Layout algorithm ('force', 'random', etc.)
- **layout_parameters** – Optional layout parameters
- **verbose** – Whether to print progress information
- **multiplex** – Whether to treat as multiplex network

Returns

(layer_names, layer_networks_list, multiedges)

- layer_names: List of layer names
- layer_networks_list: List of NetworkX graph objects for each layer
- multiedges: Dictionary of edges spanning multiple layers

Return type
tuple

`py3plex.core.converters.prepare_for_visualization_hairball(`*multinet*, *compute_layouts=False*`)`

Compute layout for a hairball visualization

Parameters
`param1` (*obj*) – multilayer object

Returns
(names, prepared network)

Return type
tuple

`py3plex.core.converters.require(`**args*, ***kwargs*`)`

`class py3plex.core.random_generators.SBMMetadata(`*block_memberships: ndarray*, *block_matrix: ndarray*, *node_ids: List[str]*, *layer_names: List[str]*`)`

Bases: `object`

Ground-truth information for a multilayer SBM sample.

`block_memberships`

Array of shape (n_nodes,) with integer block labels in {0, . . . , n_blocks-1}. Shared across layers in this simple multiplex model.

Type

np.ndarray

`block_matrix`

Array of shape (n_blocks, n_blocks) with edge probabilities p_ab. Same for all layers in this simple model.

Type

np.ndarray

`node_ids`

Node identifiers used in the resulting multi_layer_network (e.g. "v0", "v1", . . .).

Type

List[str]

`layer_names`

Names of the layers used in the multi_layer_network (e.g. "L0", "L1", . . .).

Type

List[str]

`block_matrix: ndarray`

`block_memberships: ndarray`

`layer_names: List[str]`

`node_ids: List[str]`

`py3plex.core.random_generators.`**`ensure`**`(`*`*args`*, *`**kwargs`*`)`

`py3plex.core.random_generators.`**`random_multilayer_ER`**`(`*n: int*, *l: int*, *p: float*, *directed: bool = False*`)` → Any

Generate random multilayer Erdős-Rényi network.

Parameters

- **`n`** – Number of nodes (must be positive)
- **`l`** – Number of layers (must be positive)
- **`p`** – Edge probability in [0, 1]
- **`directed`** – If True, generate directed network

Returns

multi_layer_network object

Contracts:

- Precondition: n > 0 - must have at least one node
- Precondition: l > 0 - must have at least one layer
- Precondition: 0 <= p <= 1 - probability must be valid

- Postcondition: result is not None - must return valid network

```
py3plex.core.random_generators.random_multilayer_SBM(n_layers: int, n_nodes: int, n_blocks: int, p_in: float, p_out: float, coupling: float = 0.0, directed: bool = False, seed: int | None = None) → Tuple[multi_layer_network, SBMMetadata]
```

Generate a simple multiplex multilayer stochastic block model (SBM) network.

This function creates a multilayer network with: - *n_nodes* nodes shared across all layers, - *n_layers* layers, - *n_blocks* latent communities (blocks), - within-block edge probability *p_in*, - between-block edge probability *p_out*, - optional diagonal inter-layer coupling with probability *coupling*

between replicas of the same node across layers.

The block memberships are shared across layers in this simple model, and the same block probability matrix is used for all layers.

Parameters

- **n_layers** (*int*) – Number of layers.
- **n_nodes** (*int*) – Number of nodes (shared across all layers).
- **n_blocks** (*int*) – Number of blocks (communities).
- **p_in** (*float*) – Edge probability for edges within the same block.
- **p_out** (*float*) – Edge probability for edges between different blocks.
- **coupling** (*float, optional*) – Probability of inter-layer edges between replicas of the same node in different layers. Defaults to 0.0 (no inter-layer edges).
- **directed** (*bool, optional*) – Whether to generate directed layers. Defaults to False.
- **seed** (*int, optional*) – Random seed for reproducibility.

Returns

- **network** (*multi_layer_network*) – The generated multilayer network.
- **metadata** (*SBMMetadata*) – Ground-truth metadata containing block memberships and block matrix.

Notes

- Node IDs are strings “v0”, “v1”, . . . , “v{n_nodes-1}”.
- Layer names are strings “L0”, “L1”, . . . , “L{n_layers-1}”.
- Edges are inserted via the py3plex list-based API: [src_node, src_layer, dst_node, dst_layer, weight].

Examples

```
>>> from py3plex.core.random_generators import random_multilayer_SBM
>>> net, meta = random_multilayer_SBM(
...     n_layers=3, n_nodes=20, n_blocks=2,
...     p_in=0.5, p_out=0.05, coupling=0.1, seed=42
... )
>>> print(len(meta.block_memberships))
20
>>> print(len(meta.layer_names))
3
```

`py3plex.core.random_generators.random_multiplex_ER`(*n: int*, *l: int*, *p: float*, *directed: bool = False*) → Any

Generate random multiplex Erdős-Rényi network.

Parameters

- **n** – Number of nodes (must be positive)
- **l** – Number of layers (must be positive)
- **p** – Edge probability in [0, 1]
- **directed** – If True, generate directed network

Returns

multi_layer_network object

Contracts:

- Precondition: n > 0 - must have at least one node
- Precondition: l > 0 - must have at least one layer
- Precondition: 0 <= p <= 1 - probability must be valid
- Postcondition: result is not None - must return valid network

`py3plex.core.random_generators.random_multiplex_generator`(*n: int*, *m: int*, *d: float = 0.9*) → MultiGraph

Generate a multiplex network from a random bipartite graph.

Parameters

- **n** – Number of nodes (must be positive)
- **m** – Number of layers (must be positive)
- **d** – Layer dropout to avoid cliques, range [0..1] (default: 0.9)

Returns

Generated multiplex network as a MultiGraph

Contracts:

- Precondition: n > 0 - must have at least one node
- Precondition: m > 0 - must have at least one layer
- Precondition: 0 <= d <= 1 - dropout must be valid probability
- Postcondition: result is not None

- Postcondition: result is a NetworkX MultiGraph

`py3plex.core.random_generators.require(*args, **kwargs)`

Supporting methods for parsers and converters.

This module provides utility functions for network parsing and conversion, including layer splitting, multiplex edge addition, and GAF parsing.

`py3plex.core.supporting.add_mpx_edges(input_network: Graph) → Graph`

Add multiplex edges between corresponding nodes across layers.

Multiplex edges connect nodes representing the same entity across different layers of a multilayer network.

Parameters

input_network – NetworkX graph with multilayer structure.

Returns

Network with added multiplex edges between corresponding nodes.

Contracts:

- Precondition: input_network must not be None
- Precondition: input_network must be a NetworkX graph
- Postcondition: result is a NetworkX graph

Example

```
>>> network = nx.Graph()
>>> network.add_node(('A', 'layer1'))
>>> network.add_node(('A', 'layer2'))
>>> network = add_mpx_edges(network)
```

`py3plex.core.supporting.ensure(*args, **kwargs)`

`py3plex.core.supporting.parse_gaf_to_uniprot_GO(gaf_mappings: str, filter_terms: int | None = None) → Dict[str, List[str]]`

Parse Gene Association File (GAF) to map UniProt IDs to GO terms.

Parameters

- **gaf_mappings** – Path to GAF file.
- **filter_terms** – Optional minimum occurrence threshold for GO terms.

Returns

Dictionary mapping UniProt IDs to lists of associated GO terms.

Example

```
>>> mappings = parse_gaf_to_uniprot_GO("gaf_file.gaf", filter_terms=5)
```

`py3plex.core.supporting.require(*args, **kwargs)`

`py3plex.core.supporting.split_to_layers(input_network: Graph) → Dict[Any, Graph]`

Split a multilayer network into separate layer subgraphs.

Parameters
input_network – NetworkX graph containing nodes from multiple layers.

Returns
Dictionary mapping layer names to their corresponding subgraphs.

Contracts:

- Precondition: input_network must not be None
- Precondition: input_network must be a NetworkX graph
- Postcondition: result is a dictionary
- Postcondition: all values are NetworkX graphs

Example

```
>>> network = nx.Graph()
>>> network.add_node(('A', 'layer1'))
>>> network.add_node(('B', 'layer2'))
>>> layers = split_to_layers(network)
```

NetworkX compatibility layer for py3plex. This module provides compatibility functions for different NetworkX versions.

`py3plex.core.nx_compat.`**`ensure`**`(`**args*, ***kwargs*`)`

`py3plex.core.nx_compat.`**`is_string_like`**`(`*obj: Any*`)` → bool

Check if obj is string-like (compatible with NetworkX < 3.0).

Parameters
obj – Object to check

Returns
True if string-like

Return type
bool

`py3plex.core.nx_compat.`**`nx_from_scipy_sparse_matrix`**`(`*A: Any*, *parallel_edges: bool = False*, *create_using: Graph | None = None*, *edge_attribute: str = 'weight'*`)` → Graph

Create a graph from scipy sparse matrix (compatible with NetworkX < 3.0 and >= 3.0).

Parameters

- **A** – scipy sparse matrix
- **parallel_edges** – Whether to create parallel edges (ignored in NetworkX 3.0+)
- **create_using** – Graph type to create
- **edge_attribute** – Edge attribute name for weights

Returns
NetworkX graph

Contracts:

- Precondition: A must not be None
- Precondition: edge_attribute must be a non-empty string
- Postcondition: returns a NetworkX graph

`py3plex.core.nx_compat.`**`nx_info`**`(`*G: Graph*`)` → str

Get network information (compatible with NetworkX < 3.0 and >= 3.0).

Parameters
`G` – NetworkX graph

Returns
Network information

Return type
str

Contracts:

- Precondition: G must not be None and must be a NetworkX graph
- Postcondition: returns a non-empty string

`py3plex.core.nx_compat.`**`nx_read_gpickle`**`(`*path: str*`)` → Graph

Read a graph from a pickle file (compatible with NetworkX < 3.0 and >= 3.0).

Parameters
`path` – File path

Returns
NetworkX graph

Contracts:

- Precondition: path must be a non-empty string
- Postcondition: returns a NetworkX graph

`py3plex.core.nx_compat.`**`nx_to_scipy_sparse_matrix`**`(`*G: Graph*, *nodelist: list | None = None*, *dtype: Any | None = None*, *weight: str = 'weight'*, *format: str = 'csr'*`)` → Any

Convert graph to scipy sparse matrix (compatible with NetworkX < 3.0 and >= 3.0).

Parameters

- `G` – NetworkX graph
- `nodelist` – List of nodes
- `dtype` – Data type
- `weight` – Edge weight attribute
- `format` – Sparse matrix format

Returns
scipy sparse matrix

Contracts:

- Precondition: G must not be None and must be a NetworkX graph

- Precondition: weight must be a non-empty string
- Precondition: format must be a non-empty string
- Postcondition: returns a non-None sparse matrix

`py3plex.core.nx_compat.`**`nx_write_gpickle`**`(`*G: Graph*, *path: str*`)` → None

Write a graph to a pickle file (compatible with NetworkX < 3.0 and >= 3.0).

Parameters

- **G** – NetworkX graph
- **path** – File path

Contracts:

- Precondition: G must not be None and must be a NetworkX graph
- Precondition: path must be a non-empty string

`py3plex.core.nx_compat.`**`require`**`(`**args*, ***kwargs*`)`

HINMINE Network Decomposition

`py3plex.core.HINMINE.decomposition.`**`aggregate_sum`**`(`*input_thing*, *classes*, *universal_set*`)`

`py3plex.core.HINMINE.decomposition.`**`aggregate_weighted_sum`**`(`*input_thing*, *classes*, *universal_set*`)`

`py3plex.core.HINMINE.decomposition.`**`calculate_importance_chi`**`(`*classes*, *universal_set*, *linked_nodes*, *n*, ***kwargs*`)`

Calculates importance of a single midpoint using chi-squared weighing. :param classes: List of all classes :param universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the midpoint :param n: Number of elements of universal set :return: List of weights of the midpoint for each label in class

`py3plex.core.HINMINE.decomposition.`**`calculate_importance_delta`**`(`*classes*, *universal_set*, *linked_nodes*, *n*, ***kwargs*`)`

Calculates importance of a single midpoint using delta-idf weighing :param classes: List of all classes :param universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the midpoint :param n: Number of elements of universal set :return: List of weights of the midpoint for each label in class

`py3plex.core.HINMINE.decomposition.`**`calculate_importance_gr`**`(`*classes*, *universal_set*, *linked_nodes*, *n*, ***kwargs*`)`

Calculates importance of a single midpoint using the GR (gain ratio) :param classes: List of all classes :param universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the midpoint :param n: Number of elements of universal set :return: List of weights of the midpoint for each label in class `py3plex.core.HINMINE.decomposition.calculate_importance_idf(classes, universal_set, linked_nodes, n, **kwargs)`

Calculates importance of a single midpoint using idf weighing :param classes: List of all classes :param universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the midpoint :param n: Number of elements of universal set :return: List of weights of the midpoint for each label in class

`py3plex.core.HINMINE.decomposition.calculate_importance_ig(classes, universal_set, linked_nodes, n, **kwargs)`

Calculates importance of a single midpoint using IG (information gain) weighing :param classes: List of all classes :param universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the midpoint :param n: Number of elements of universal set :return: List of weights of the midpoint for each label in class

`py3plex.core.HINMINE.decomposition.calculate_importance_okapi(classes, universal_set, linked_nodes, n, degrees=None, avgdegree=None)`

`py3plex.core.HINMINE.decomposition.calculate_importance_rf(classes, universal_set, linked_nodes, n, **kwargs)`

Calculates importance of a single midpoint using rf weighing :param classes: List of all classes :param universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the midpoint :param n: Number of elements of universal set :return: List of weights of the midpoint for each label in class

`py3plex.core.HINMINE.decomposition.calculate_importance_tf(classes, universal_set, linked_nodes, n, **kwargs)`

Calculates importance of a single midpoint using term frequency weighing. :param classes: List of all classes :param universal_set: Set of all indices to consider :param linked_nodes: Set of all nodes linked by the midpoint :param n: Number of elements of universal set :return: List of weights of the midpoint for each label in class

`py3plex.core.HINMINE.decomposition.calculate_importance_w2w(classes, universal_set, linked_nodes, n, **kwargs)`

`py3plex.core.HINMINE.decomposition.calculate_importances(midpoints, classes, universal_set, method, degrees=None, avgdegree=None)`

`py3plex.core.HINMINE.decomposition.chi_value(actual_pos_num, predicted_pos_num, tp, n)`

`py3plex.core.HINMINE.decomposition.get_aggregation_method(method_name)`

`py3plex.core.HINMINE.decomposition.get_calculation_method(method_name)` py3plex.core.HINMINE.decomposition.**gr_value**(*actual_pos_num*, *predicted_pos_num*, *tp*, *n*)

py3plex.core.HINMINE.decomposition.**hinmine_decompose**(*network*, *heuristic*, *cycle=None*, *parallel=False*)

py3plex.core.HINMINE.decomposition.**hinmine_get_cycles**(*network*, *cycle=None*)

py3plex.core.HINMINE.decomposition.**ig_value**(*actual_pos_num*, *predicted_pos_num*, *tp*, *n*)

py3plex.core.HINMINE.decomposition.**np_calculate_importance_chi**(*predicted*, *label_matrix*, *actual_pos_nums*)

py3plex.core.HINMINE.decomposition.**np_calculate_importance_tf**(*predicted*, *label_matrix*)

py3plex.core.HINMINE.decomposition.**rf_value**(*predicted_pos_num*, *tp*)

py3plex.core.HINMINE.IO.**load_hinmine_object**(*infile*, *label_delimiter='---'*, *weight_tag=False*, *targets=True*)

Configuration and Utilities

Centralized configuration for py3plex.

This module provides default settings for visualization, layout algorithms, and other configurable aspects of the library. Users can override these settings by modifying the values after import.

Example

```
>>> from py3plex import config
>>> config.DEFAULT_NODE_SIZE = 15
>>> config.DEFAULT_EDGE_ALPHA = 0.5
```

py3plex.config.**get_color_palette**(*name: str | None = None*) → List[str]

Get a color palette by name.

Parameters
name – Palette name. If None, returns the default palette.

Returns
List of color hex codes.

Raises
ValueError – If palette name is not recognized.

Example

```
>>> from py3plex.config import get_color_palette
>>> colors = get_color_palette("rainbow")
>>> print(colors[0])
'#FF6B6B'
```

`py3plex.config.reset_to_defaults()` → None

Reset all configuration values to their defaults.

This is useful for testing or when you want to start fresh.

Example

```
>>> from py3plex import config
>>> config.DEFAULT_NODE_SIZE = 20
>>> config.reset_to_defaults()
>>> print(config.DEFAULT_NODE_SIZE)
10
```

Utility functions for py3plex.

This module provides common utilities used across the library, including random state management for reproducibility and deprecation warnings.

`py3plex.utils.deprecated(`*reason: str*, *version: str | None = None*, *alternative: str | None = None*`)` → Callable[[Callable], Callable]

Decorator to mark functions/methods as deprecated.

This decorator will issue a DeprecationWarning when the decorated function is called, providing information about why it's deprecated and what to use instead.

Parameters

- **reason** – Explanation of why the function is deprecated
- **version** – Version in which the function was deprecated (optional)
- **alternative** – Suggested alternative function/method (optional)

Returns

Decorator function

Example

```
>>> @deprecated(
...     reason="This function is obsolete",
...     version="0.95a",
...     alternative="new_function()"
... )
... def old_function():
...     pass
```

`py3plex.utils.ensure(`**args*, ***kwargs*`)`

`py3plex.utils.get_background_knowledge_dir()` → str

Get the absolute path to the background knowledge directory.

Returns

Absolute path to the background_knowledge directory

Return type

str

Examples

```
>>> from py3plex.utils import get_background_knowledge_dir
>>> dir_path = get_background_knowledge_dir()
```

py3plex.utils.**get_background_knowledge_path**(*filename: str*) → str

Get the absolute path to a background knowledge file or directory.

Convenience wrapper around get_data_path() specifically for background knowledge files.

Parameters

filename – Name or relative path of the background knowledge file. Use empty string or '.' to get the background_knowledge directory itself.

Returns

Absolute path to the background knowledge file or directory

Return type

str

Examples

```
>>> from py3plex.utils import get_background_knowledge_path
>>> path = get_background_knowledge_path("bk.n3")
>>> dir_path = get_background_knowledge_path(".")
```

py3plex.utils.**get_data_path**(*relative_path: str*) → str

Get the absolute path to a data file in the repository.

This function searches for data files in multiple locations to support both: - Running examples from a cloned repository - Running scripts/notebooks from any directory with datasets locally available

Search order: 1. Relative to the calling script's directory (for examples in cloned repo) 2. Relative to current working directory (for notebooks/user scripts) 3. Relative to py3plex package location (for editable installs)

Parameters

relative_path – Path relative to repository root (e.g., "datasets/intact02.gpickle")

Returns

Absolute path to the file

Return type

str

Raises

Py3plexIOError – If the file cannot be found in any search location

Examples

```
>>> from py3plex.utils import get_data_path
>>> path = get_data_path("datasets/intact02.gpickle")
>>> os.path.exists(path)
True
```

Note

When py3plex is installed via pip, datasets are not included in the package. Users should either: - Clone the repository and run examples from there - Download datasets separately and place them relative to their scripts - Use current working directory with datasets folder

`py3plex.utils.get_dataset_path(filename: str) → str`

Get the absolute path to a dataset file.

Convenience wrapper around get_data_path() specifically for dataset files.

Parameters
filename – Name or relative path of the dataset file

Returns
Absolute path to the dataset file

Return type
str

Examples

```
>>> from py3plex.utils import get_dataset_path
>>> path = get_dataset_path("intact02.gpickle")
>>> os.path.exists(path)
True
```

`py3plex.utils.get_example_image_path(filename: str) → str`

Get the absolute path to an example image file.

Convenience wrapper around get_data_path() specifically for example image files.

Parameters
filename – Name or relative path of the image file

Returns
Absolute path to the example image file

Return type
str

Examples

```
>>> from py3plex.utils import get_example_image_path
>>> path = get_example_image_path("intact_10_BH.png")
```

`py3plex.utils.get_multilayer_dataset_path(relative_path: str) → str`

Get the absolute path to a multilayer dataset file.

Convenience wrapper around get_data_path() specifically for multilayer dataset files.

Parameters
relative_path – Relative path within multilayer_datasets directory

Returns
Absolute path to the multilayer dataset file

Return type
str

Examples

```
>>> from py3plex.utils import get_multilayer_dataset_path
>>> path = get_multilayer_dataset_path("MLKing/MLKing2013_multiplex.edges
↪")
```

py3plex.utils.**get_rng**(*seed: int | Generator | None = None*) → Generator

Get a NumPy random number generator with optional seed.

This provides a unified interface for random state management across the library, ensuring reproducibility when a seed is provided.

Parameters
seed – Random seed for reproducibility. Can be: - None: Use default unseeded generator - int: Seed value for the generator - np.random.Generator: Pass through existing generator

Returns
Initialized random number generator

Return type
np.random.Generator

Examples

```
>>> rng = get_rng(42)
>>> rng.random()  # Reproducible random number
0.7739560485559633
```

```
>>> rng1 = get_rng(42)
>>> rng2 = get_rng(42)
>>> rng1.random() == rng2.random()
True
```

```
>>> existing_rng = np.random.default_rng(123)
>>> rng = get_rng(existing_rng)
>>> rng is existing_rng
True
```

Contracts:

- Postcondition: result is a NumPy random Generator

Note

Uses numpy.random.Generator (modern API introduced in NumPy 1.17) rather than the legacy numpy.random.RandomState API.

Negative seeds are converted to positive values by taking absolute value to ensure compatibility with NumPy's SeedSequence.

py3plex.utils.**require**(**args*, ***kwargs*)

py3plex.utils.**validate_multilayer_input**(*network_data: Any*) → None

Validate multilayer network input data.

Performs sanity checks on multilayer network structures to catch common errors early.

Parameters
network_data – Network data to validate (can be various formats)

Raises
ValueError – If the network data is invalid

Contracts:

- Precondition: network_data must not be None

Example

```
>>> from py3plex.utils import validate_multilayer_input
>>> validate_multilayer_input(my_network)
```

py3plex.utils.**warn_if_deprecated**(*feature_name: str*, *reason: str*, *alternative: str | None = None*) → None

Issue a deprecation warning for a feature.

This is useful for deprecating specific usage patterns or parameter combinations rather than entire functions.

Parameters

- **feature_name** – Name of the deprecated feature
- **reason** – Explanation of why it's deprecated
- **alternative** – Suggested alternative (optional)

Example

```
>>> def my_function(old_param=None, new_param=None):
...     if old_param is not None:
...         warn_if_deprecated(
...             "old_param",
...             "This parameter is no longer used",
...             "new_param"
...         )
```

Custom exception types for the py3plex library.

This module defines domain-specific exceptions to provide clear error messages and enable better error handling throughout the library.

Py3plex follows Rust's approach to error messages: - Clear, descriptive error messages - Error codes (e.g., PX101, PX201) - Helpful suggestions for fixing issues - "Did you mean?" suggestions for typos - Context showing the relevant location in files

Example usage:

```
>>> from py3plex.exceptions import InvalidLayerError
>>> raise InvalidLayerError(
...     "social",
...     available_layers=["work", "family", "social_media"],
...     suggestion="Did you mean 'social_media'?"
... )
```

exception py3plex.exceptions.**AlgorithmError**(*message: str*, ***, *algorithm_name: str | None = None*, *valid_algorithms: List[str] | None = None*, ***kwargs*)

Bases: *Py3plexException*

Exception raised when an algorithm execution fails.

Error code: PX301

default_code: str = 'PX301'

exception py3plex.exceptions.**CentralityComputationError**(*message: str*, ***, *algorithm_name: str | None = None*, *valid_algorithms: List[str] | None = None*, ***kwargs*)

Bases: *AlgorithmError*

Exception raised when centrality computation fails.

Error code: PX301

default_code: str = 'PX301'

exception py3plex.exceptions.**CommunityDetectionError**(*message: str*, ***, *algorithm_name: str | None = None*, *valid_algorithms: List[str] | None = None*, ***kwargs*)

Bases: *AlgorithmError*

Exception raised when community detection fails.

Error code: PX301

default_code: str = 'PX301'

exception py3plex.exceptions.**ConversionError**(*message: str*, ***, *code: str | None = None*, *suggestions: List[str] | None = None*, *notes: List[str] | None = None*, *did_you_mean: str | None = None*, *context: Dict[str, Any] | None = None*)

Bases: *Py3plexException*

Exception raised when format conversion fails.

Error code: PX501

default_code: str = 'PX501'

exception py3plex.exceptions.**DecompositionError**(*message: str*, ***, *algorithm_name: str | None = None*, *valid_algorithms: List[str] | None = None*, ***kwargs*)

Bases: *AlgorithmError*

Exception raised when network decomposition fails.

Error code: PX301

default_code: str = 'PX301'

exception py3plex.exceptions.**EmbeddingError**(*message: str*, ***, *algorithm_name: str | None = None*, *valid_algorithms: List[str] | None = None*, ***kwargs*)

Bases: *AlgorithmError*

Exception raised when embedding generation fails.

Error code: PX301

default_code: str = 'PX301'

exception py3plex.exceptions.**ExternalToolError**(*message: str*, ***, *code: str | None = None*, *suggestions: List[str] | None = None*, *notes: List[str] | None = None*, *did_you_mean: str | None = None*, *context: Dict[str, Any] | None = None*)

Bases: *Py3plexException*

Exception raised when external tool execution fails.

Error code: PX001

default_code: str = 'PX001'

exception py3plex.exceptions.**IncompatibleNetworkError**(*message: str*, ***, *code: str | None = None*, *suggestions: List[str] | None = None*, *notes: List[str] | None = None*, *did_you_mean: str | None = None*, *context: Dict[str, Any] | None = None*)

Bases: *Py3plexException*

Exception raised when network format is incompatible with an operation.

Error code: PX304

default_code: str = 'PX304'

exception py3plex.exceptions.**InvalidEdgeError**(*message: str*, ***, *code: str | None = None*, *suggestions: List[str] | None = None*, *notes: List[str] | None = None*, *did_you_mean: str | None = None*, *context: Dict[str, Any] | None = None*)

Bases: *Py3plexException*

Exception raised when an invalid edge is specified.

Error code: PX203

default_code: str = 'PX203'

exception py3plex.exceptions.**InvalidLayerError**(*layer_name: str, *, available_layers: List[str] | None = None, **kwargs*)

Bases: *Py3plexException*

Exception raised when an invalid layer is specified.

Error code: PX201

Example

```
>>> raise InvalidLayerError(
...     "social",
...     available_layers=["work", "family"],
... )
```

default_code: str = 'PX201'

exception py3plex.exceptions.**InvalidNodeError**(*node_id: str, *, available_nodes: List[str] | None = None, **kwargs*)

Bases: *Py3plexException*

Exception raised when an invalid node is specified.

Error code: PX202

default_code: str = 'PX202'

exception py3plex.exceptions.**NetworkConstructionError**(*message: str, *, code: str | None = None, suggestions: List[str] | None = None, notes: List[str] | None = None, did_you_mean: str | None = None, context: Dict[str, Any] | None = None*)

Bases: *Py3plexException*

Exception raised when network construction fails.

Error code: PX208

default_code: str = 'PX208'

exception py3plex.exceptions.**ParsingError**(*message: str, *, file_path: str | None = None, line_number: int | None = None, expected: str | None = None, got: str | None = None, **kwargs*)

Bases: *Py3plexException*

Exception raised when parsing input data fails.

Error code: PX105

default_code: str = 'PX105'

exception py3plex.exceptions.Py3plexException(*message: str*, ***, *code: str | None = None*, *suggestions: List[str] | None = None*, *notes: List[str] | None = None*, *did_you_mean: str | None = None*, *context: Dict[str, Any] | None = None*)

Bases: `Exception`

Base exception class for all py3plex-specific exceptions.

code

Error code (e.g., "PX001")

suggestions

List of suggestions for fixing the error

notes

Additional context/notes

did_you_mean

Suggested correction for typos

default_code: str = 'PX001'

format_message(*use_color: bool = True*) → str

Format the exception with Rust-style error formatting.

Parameters

use_color – Whether to use ANSI colors

Returns

Formatted error message string

exception py3plex.exceptions.Py3plexFormatError(*message: str*, ***, *valid_formats: List[str] | None = None*, *input_format: str | None = None*, ***kwargs*)

Bases: *Py3plexException*

Exception raised when input format is invalid or cannot be parsed.

Error code: PX103

default_code: str = 'PX103'

exception py3plex.exceptions.Py3plexIOError(*message: str*, ***, *code: str | None = None*, *suggestions: List[str] | None = None*, *notes: List[str] | None = None*, *did_you_mean: str | None = None*, *context: Dict[str, Any] | None = None*)

Bases: *Py3plexException*

Exception raised when I/O operations fail (file reading, writing, etc.).

Error code: PX101

default_code: str = 'PX101'

exception py3plex.exceptions.Py3plexLayoutError(*message: str*, ***, *code: str | None = None*, *suggestions: List[str] | None = None*, *notes: List[str] | None = None*, *did_you_mean: str | None = None*, *context: Dict[str, Any] | None = None*)

Bases: *Py3plexException*

Exception raised when layout computation or visualization positioning fails.

Error code: PX402

default_code: str = 'PX402'

exception py3plex.exceptions.Py3plexMatrixError(*message: str*, ***, *code: str | None = None*, *suggestions: List[str] | None = None*, *notes: List[str] | None = None*, *did_you_mean: str | None = None*, *context: Dict[str, Any] | None = None*)

Bases: *Py3plexException*

Exception raised when matrix operations fail or matrix is invalid.

Error code: PX001

default_code: str = 'PX001'

exception py3plex.exceptions.VisualizationError(*message: str*, ***, *code: str | None = None*, *suggestions: List[str] | None = None*, *notes: List[str] | None = None*, *did_you_mean: str | None = None*, *context: Dict[str, Any] | None = None*)

Bases: *Py3plexException*

Exception raised when visualization operations fail.

Error code: PX401

default_code: str = 'PX401'

Domain-Specific Language (DSL)

DSL v2 for Multilayer Network Queries.

This module provides a Domain-Specific Language (DSL) version 2 for querying and analyzing multilayer networks. DSL v2 introduces:

1. Unified AST representation
2. Pythonic builder API (Q, L, Param)
3. Multilayer-specific abstractions (layer algebra, intralayer/interlayer)
4. Improved ergonomics (ORDER BY, LIMIT, EXPLAIN, rich results)

DSL Extensions (v2.1): 5. Network comparison (C.compare()) 6. Null models (N.model()) 7. Path queries (P.shortest(), P.random_walk()) 8. Plugin system for user-defined operators (@dsl_operator)

Example Usage:

```
>>> from py3plex.dsl import Q, L, Param, C, N, P
>>>
>>> # Build a query using the builder API
>>> q = (
...     Q.nodes()
...      .from_layers(L["social"] + L["work"])
...      .where(intralayer=True, degree__gt=5)
...      .compute("betweenness_centrality", alias="bc")
...      .order_by("bc", desc=True)
...      .limit(20)
... )
>>>
>>> # Execute the query
>>> result = q.execute(network, k=5)
>>> df = result.to_pandas()
>>>
>>> # Compare two networks
>>> comparison = C.compare("baseline", "treatment").using("multiplex_
↪jaccard").execute(networks)
>>>
>>> # Generate null models
>>> nullmodels = N.configuration().samples(100).seed(42).execute(network)
>>>
>>> # Find paths
>>> paths = P.shortest("Alice", "Bob").crossing_layers().execute(network)
>>>
>>> # Define custom operators
>>> @dsl_operator("my_measure")
... def my_custom_measure(context, param: float = 1.0):
...     # Use context.graph, context.current_layers, etc.
...     return result
```

The DSL also supports a string syntax:

SELECT nodes FROM LAYER("social") + LAYER("work") WHERE intralayer AND degree > 5 COMPUTE betweenness_centrality AS bc ORDER BY bc DESC LIMIT 20 TO pandas

All frontends (string DSL, builder API) compile into a single AST which is executed by the same engine, ensuring consistent behavior.

`class py3plex.dsl.AttrType(value)`

Bases: `Enum`

Attribute type for static analysis.

`BOOLEAN = 'boolean'`

`CATEGORICAL = 'categorical'`

`DATETIME = 'datetime'`

`EDGE_REF = 'edge_ref'`

LAYER_REF = 'layer_ref'

NODE_REF = 'node_ref'

NUMERIC = 'numeric'

UNKNOWN = 'unknown'

is_comparable(*other: AttrType*) → bool

Check if this type can be compared with another type.

supports_operator(*op: str*) → bool

Check if this type supports a given operator.

class py3plex.dsl.**BooleanExpression**(*condition: ConditionExpr*, *negated: bool = False*)

Bases: object

Represents a boolean expression that can be combined with & (AND), | (OR), and ~ (NOT).

This class wraps ConditionExpr AST nodes and provides operator overloading for building complex boolean logic.

_condition

The underlying ConditionExpr AST node

_negated

Whether this expression is negated

to_condition_expr() → *ConditionExpr*

Convert to AST ConditionExpr.

Returns

ConditionExpr that can be used in SelectStmt

class py3plex.dsl.**C**

Bases: object

Compare factory for creating CompareBuilder instances.

Example

```
>>> C.compare("baseline", "intervention").using("multiplex_jaccard")
```

static **compare**(*network_a: str*, *network_b: str*) → *CompareBuilder*

Create a comparison builder for two networks.

class py3plex.dsl.**CompareBuilder**(*network_a: str*, *network_b: str*)

Bases: object

Builder for COMPARE statements.

Example

```
>>> from py3plex.dsl import C, L
>>>
>>> result = (
```

(continues on next page)

(continued from previous page)

```
...     C.compare("baseline", "intervention")
...      .using("multiplex_jaccard")
...      .on_layers(L["social"] + L["work"])
...      .measure("global_distance", "layerwise_distance")
...      .execute(networks)
... )
```

execute(*networks: Dict[str, Any]*) → ComparisonResult

Execute the comparison.

Parameters
networks – Dictionary mapping network names to network objects

Returns
ComparisonResult with comparison results

measure(**measures: str*) → *CompareBuilder*

Specify which measures to compute.

Parameters
***measures** – Measure names (e.g., "global_distance", "layerwise_distance")

Returns
Self for chaining

on_layers(*layer_expr:* LayerExprBuilder) → *CompareBuilder*

Filter by layers using layer algebra.

Parameters
layer_expr – Layer expression (e.g., L["social"] + L["work"])

Returns
Self for chaining

to(*target: str*) → *CompareBuilder*

Set export target.

Parameters
target – Export format ('pandas', 'json')

Returns
Self for chaining

to_ast() → *CompareStmt*

Export as AST CompareStmt object.

using(*metric: str*) → *CompareBuilder*

Set the comparison metric.

Parameters
metric – Metric name (e.g., "multiplex_jaccard")

Returns
Self for chaining

class py3plex.dsl.CompareStmt(*network_a: str, network_b: str, metric_name: str, layer_expr: ~py3plex.dsl.ast.LayerExpr | None = None, measures: ~typing.List[str] = <factory>, export_target: str | None = None*)

Bases: `object`

COMPARE statement for network comparison.

DSL Example:

COMPARE NETWORK baseline, intervention USING multiplex_jaccard ON LAYER("offline") + LAYER("online") MEASURE global_distance TO pandas

network_a

Name/key for first network

Type

str

network_b

Name/key for second network

Type

str

metric_name

Comparison metric (e.g., "multiplex_jaccard")

Type

str

layer_expr

Optional layer expression for filtering

Type

py3plex.dsl.ast.LayerExpr | None

measures

List of measure types (e.g., ["global_distance", "layerwise_distance"])

Type

List[str]

export_target

Optional export format

Type

str | None

export_target: str | None = None

layer_expr: *LayerExpr* | None = None

measures: List[str]

metric_name: str

network_a: str

network_b: str

class py3plex.dsl.Comparison(*left: str*, *op: str*, *right: str | float | int |* ParamRef)

Bases: `object`

A comparison expression.

left

Attribute name (e.g., "degree", "layer")

Type

str

op

Comparison operator ('>', '>=', '<', '<=', '=', '!=')

Type

str

right

Value to compare against

Type

str | float | int | *py3plex.dsl.ast.ParamRef*

left: str

op: str

right: str | float | int | *ParamRef*

class py3plex.dsl.ComputeItem(*name: str*, *alias: str | None = None*, *uncertainty: bool = False*, *method: str | None = None*, *n_samples: int | None = None*, *ci: float | None = None*, *bootstrap_unit: str | None = None*, *bootstrap_mode: str | None = None*, *n_null: int | None = None*, *null_model: str | None = None*, *random_state: int | None = None*)

Bases: `object`

A measure to compute.

name

Measure name (e.g., 'betweenness_centrality')

Type

str

alias

Optional alias for the result (e.g., 'bc')

Type

str | None

uncertainty

Whether to compute uncertainty for this measure

Type

bool

method

Uncertainty estimation method (e.g., 'bootstrap', 'perturbation', 'null_model')

Type

str | None

n_samples

Number of samples for uncertainty estimation

Type
int | None

ci

Confidence interval level (e.g., 0.95 for 95% CI)

Type
float | None

bootstrap_unit

What to resample for bootstrap: "edges", "nodes", or "layers"

Type
str | None

bootstrap_mode

Resampling mode: "resample" or "permute"

Type
str | None

n_null

Number of null model replicates

Type
int | None

null_model

Null model type: "degree_preserving", "erdos_renyi", "configuration"

Type
str | None

random_state

Random seed for reproducibility

Type
int | None

alias: str | None = None

bootstrap_mode: str | None = None

bootstrap_unit: str | None = None

ci: float | None = None

method: str | None = None

n_null: int | None = None

n_samples: int | None = None

name: str

null_model: str | None = None

random_state: int | None = None

property result_name: str

Get the name to use in results (alias or original name).

uncertainty: bool = False

class py3plex.dsl.ConditionAtom(*comparison: Comparison | None = None, function: FunctionCall | None = None, special: SpecialPredicate | None = None*)

Bases: `object`

A single atomic condition.

Exactly one of comparison, function, or special should be non-None.

comparison

Simple comparison (e.g., degree > 5)

Type

py3plex.dsl.ast.Comparison | None

function

Function call (e.g., reachable_from("Alice"))

Type

py3plex.dsl.ast.FunctionCall | None

special

Special predicate (e.g., intralayer)

Type

py3plex.dsl.ast.SpecialPredicate | None

comparison: *Comparison* | None = None

function: *FunctionCall* | None = None

property is_comparison: bool

property is_function: bool

property is_special: bool

special: *SpecialPredicate* | None = None

class py3plex.dsl.ConditionExpr(*atoms: ~typing.List[~py3plex.dsl.ast.ConditionAtom] = <factory>, ops: ~typing.List[str] = <factory>*)

Bases: `object`

Compound condition expression.

Represents conditions joined by logical operators (AND, OR).

atoms

List of condition atoms

Type

List[*py3plex.dsl.ast.ConditionAtom*]

ops

List of logical operators ('AND', 'OR') between atoms

Type
List[str]

`atoms: List[ConditionAtom]`

`ops: List[str]`

`class py3plex.dsl.DSLExecutionContext(`*graph: Any, current_layers: List[str] | None = None, current_nodes: List[Any] | None = None, params: Mapping[str, Any] | None = None*`)`

Bases: `object`

Execution context passed to DSL operators.

This context object provides operators with access to the network, current selection state, and query parameters.

graph
The underlying multilayer network object

Type
Any

current_layers
Currently selected layers (None = all layers)

Type
List[str] | None

current_nodes
Currently selected nodes (None = all nodes)

Type
List[Any] | None

params
Query parameters (e.g., from Param() in builder API)

Type
Mapping[str, Any]

`current_layers: List[str] | None = None`

`current_nodes: List[Any] | None = None`

`graph: Any`

`params: Mapping[str, Any] = None`

`exception py3plex.dsl.DSLExecutionError`

Bases: `Exception`

Exception raised for DSL execution errors.

`class py3plex.dsl.DSLOperator(`*name: str, func: Callable[[...], Any], description: str | None = None, category: str | None = None*`)`

Bases: `object`

Metadata for a registered DSL operator.

name
Operator name (normalized)

Type
str

func
Python callable implementing the operator

Type
Callable[[. . .], Any]

description
Optional human-readable description

Type
str | None

category
Optional category (e.g., "centrality", "dynamics", "io")

Type
str | None

category: str | None = None

description: str | None = None

func: Callable[[...], Any]

name: str

exception py3plex.dsl.DSLSyntaxError

Bases: `Exception`

Exception raised for DSL syntax errors.

class py3plex.dsl.Diagnostic(*code: str*, *severity: Literal['error', 'warning', 'info', 'hint']*, *message: str*, *span: Tuple[int, int]*, *suggested_fix: SuggestedFix | None = None*)

Bases: `object`

A linting diagnostic (error, warning, info, or hint).

code
Diagnostic code (e.g., "DSL001", "PERF301")

Type
str

severity
Severity level

Type
Literal['error', 'warning', 'info', 'hint']

message
Human-readable message

Type
str

span
Tuple of (start_index, end_index) in the query string

Type
Tuple[int, int]

`suggested_fix`
Optional suggested fix

Type
py3plex.dsl.lint.diagnostic.SuggestedFix | None

`code: str`

`message: str`

`severity: Literal['error', 'warning', 'info', 'hint']`

`span: Tuple[int, int]`

`suggested_fix: SuggestedFix | None = None`

exception `py3plex.dsl.DslError`(*message: str*, *query: str | None = None*, *line: int | None = None*, *column: int | None = None*)

Bases: `Exception`

Base exception for all DSL errors.

`format_message()` → str
Format the error message with context.

exception `py3plex.dsl.DslExecutionError`(*message: str*, *query: str | None = None*, *line: int | None = None*, *column: int | None = None*)

Bases: *DslError*

Exception raised for DSL execution errors.

exception `py3plex.dsl.DslMissingMetricError`(*metric: str*, *required_by: str | None = None*, *autocompute_enabled: bool = True*, *query: str | None = None*, *line: int | None = None*, *column: int | None = None*)

Bases: *DslError*

Exception raised when a required metric is missing and cannot be autocomputed.

This error occurs when: - A query references a metric that hasn't been computed - Autocompute is disabled or the metric is not autocomputable - The metric is required for an operation (e.g., top_k, where clause)

`metric`
The missing metric name

`required_by`
The operation that requires the metric

`autocompute_enabled`
Whether autocompute was enabled

exception `py3plex.dsl.DslSyntaxError`(*message: str*, *query: str | None = None*, *line: int | None = None*, *column: int | None = None*)

Bases: *DslError*

Exception raised for DSL syntax errors.

class py3plex.dsl.DynamicsBuilder(*process_name: str*, ***params*)

Bases: `object`

Builder for DYNAMICS statements.

Example

```
>>> from py3plex.dsl import Q, L
>>>
>>> result = (
...     Q.dynamics("SIS", beta=0.3, mu=0.1)
...      .on_layers(L["contacts"] + L["travel"])
...      .seed(Q.nodes().where(degree__gt=10))
...      .parameters_per_layer({
...          "contacts": {"beta": 0.4},
...          "travel": {"beta": 0.2}
...      })
...      .run(steps=100, replicates=10)
...      .execute(network)
... )
```

execute(*network: Any*) → Any

Execute dynamics simulation.

Parameters

network – Multilayer network

Returns

DynamicsResult with simulation outputs

on_layers(*layer_expr:* LayerExprBuilder) → *DynamicsBuilder*

Filter by layers using layer algebra.

Parameters

layer_expr – Layer expression (e.g., L[“social”] + L[“work”])

Returns

Self for chaining

parameters_per_layer(*layer_params: Dict[str, Dict[str, Any]]*) → *DynamicsBuilder*

Set per-layer parameter overrides.

Parameters

layer_params – Dictionary mapping layer names to parameter dictionaries

Returns

Self for chaining

Example

```
>>> builder.parameters_per_layer({
...     "contacts": {"beta": 0.3},
```

(continues on next page)

(continued from previous page)

```
...     "travel": {"beta": 0.1}
... })
```

random_seed(*seed: int*) → *DynamicsBuilder*

Set random seed for reproducibility.

Parameters
seed – Random seed

Returns
Self for chaining

run(*steps: int = 100*, *replicates: int = 1*, *track: str | List[str] | None = None*) → *DynamicsBuilder*

Set execution parameters.

Parameters

- **steps** – Number of time steps to simulate
- **replicates** – Number of independent runs
- **track** – Measures to track ("all" or list of specific measures)

Returns
Self for chaining

seed(*query_or_fraction: float* | QueryBuilder) → *DynamicsBuilder*

Set initial seeding for the dynamics.

Parameters
query_or_fraction – Either a fraction (e.g., 0.01 for 1%) or a QueryBuilder for selecting specific nodes to seed

Returns
Self for chaining

Examples

```
>>> # Seed 1% randomly
>>> builder.seed(0.01)
```

```
>>> # Seed high-degree nodes
>>> builder.seed(Q.nodes().where(degree__gt=10))
```

to(*target: str*) → *DynamicsBuilder*

Set export target.

Parameters
target – Export format

Returns
Self for chaining

to_ast() → *DynamicsStmt*

Export as AST DynamicsStmt object.

with_states(***state_mapping*) → *DynamicsBuilder*

Explicitly define state labels (optional).

Parameters

****state_mapping** – State labels (e.g., S="susceptible", I="infected")

Returns

Self for chaining

> **Note**
>
> This is optional metadata and doesn't affect execution, but helps with documentation and trajectory queries.

class py3plex.dsl.DynamicsStmt(*process_name: str, params: ~typing.Dict[str, ~typing.Any] = <factory>, layer_expr: ~py3plex.dsl.ast.LayerExpr | None = None, seed_query: ~py3plex.dsl.ast.SelectStmt | None = None, seed_fraction: float | None = None, layer_params: ~typing.Dict[str, ~typing.Dict[str, ~typing.Any]] = <factory>, steps: int = 100, replicates: int = 1, track: ~typing.List[str] = <factory>, seed: int | None = None, export_target: str | None = None*)

Bases: `object`

DYNAMICS statement for declarative process simulation.

DSL Example:

DYNAMICS SIS WITH beta=0.3, mu=0.1 ON LAYER("contacts") + LAYER("travel") SEED FROM nodes WHERE degree > 10 PARAMETERS PER LAYER contacts: {beta=0.4}, travel: {beta=0.2} RUN FOR 100 STEPS, 10 REPLICATES TRACK prevalence, incidence

process_name

Name of the process (e.g., "SIS", "SIR", "RANDOM_WALK")

Type

str

params

Global process parameters (e.g., {"beta": 0.3, "mu": 0.1})

Type

Dict[str, Any]

layer_expr

Optional layer expression for filtering

Type

py3plex.dsl.ast.LayerExpr | None

seed_query

Optional SELECT query for seeding initial conditions

Type

py3plex.dsl.ast.SelectStmt | None

seed_fraction

Optional fraction for random seeding (e.g., 0.01 for 1%)

Type
float | None

layer_params

Optional per-layer parameter overrides

Type
Dict[str, Dict[str, Any]]

steps

Number of simulation steps

Type
int

replicates

Number of independent runs

Type
int

track

List of measures to track (e.g., ["prevalence", "incidence"])

Type
List[str]

seed

Optional random seed for reproducibility

Type
int | None

export_target

Optional export format

Type
str | None

export_target: str | None = None

layer_expr: *LayerExpr* **| None = None**

layer_params: Dict[str, Dict[str, Any]]

params: Dict[str, Any]

process_name: str

replicates: int = 1

seed: int | None = None

seed_fraction: float | None = None

seed_query: *SelectStmt* **| None = None**

steps: int = 100

track: List[str]

class py3plex.dsl.**EdgeLayerConstraint**(*kind: str*, *src_layer: str | None = None*, *dst_layer: str | None = None*, *layer: str | None = None*)

Bases: `object`

Layer constraint for an edge.

kind

Type of constraint ("within", "between", "any")

Type

str

src_layer

Source layer constraint (for "between")

Type

str | None

dst_layer

Destination layer constraint (for "between")

Type

str | None

layer

Layer constraint (for "within")

Type

str | None

static **any_layer**() → *EdgeLayerConstraint*

Create constraint that accepts any edge.

static **between**(*src_layer: str*, *dst_layer: str*) → *EdgeLayerConstraint*

Create constraint for edges between two layers.

dst_layer: str | None = None

kind: str

layer: str | None = None

matches(*src_layer: str*, *dst_layer: str*) → bool

Check if an edge satisfies this constraint.

src_layer: str | None = None

to_dict() → Dict[str, Any]

Convert to dictionary for serialization.

static **within**(*layer: str*) → *EdgeLayerConstraint*

Create constraint for edges within a single layer.

class py3plex.dsl.**EntityRef**(*entity_type: str*, *layer: str | None = None*, *attribute: str | None = None*)

Bases: `object`

Reference to an entity (node/edge) in the schema.

class py3plex.dsl.ExecutionPlan(*steps: ~typing.List[~py3plex.dsl.ast.PlanStep] = <factory>*, *warnings: ~typing.List[str] = <factory>*)

Bases: `object`

Execution plan for EXPLAIN queries.

steps

List of execution steps

Type

List[*py3plex.dsl.ast.PlanStep*]

warnings

List of performance or correctness warnings

Type

List[str]

steps: List[*PlanStep*]

warnings: List[str]

class py3plex.dsl.ExplainResult(*ast_summary: str*, *type_info: Dict[str, str]*, *cost_estimate: str*, *diagnostics: List[Diagnostic]*, *plan_steps: List[str]*)

Bases: `object`

Result of an EXPLAIN query with linting information.

ast_summary

Human-readable summary of the AST

Type

str

type_info

Dictionary mapping node IDs to inferred types

Type

Dict[str, str]

cost_estimate

Rough cost classification

Type

str

diagnostics

List of diagnostics from linting

Type

List[*py3plex.dsl.lint.diagnostic.Diagnostic*]

plan_steps

List of execution plan steps

Type

List[str]

ast_summary: str

cost_estimate: str

diagnostics: List[*Diagnostic*]

plan_steps: List[str]

type_info: Dict[str, str]

class py3plex.dsl.**ExportSpec**(*path: str*, *fmt: str = 'csv'*, *columns: ~typing.List[str] | None = None*, *options: ~typing.Dict[str, ~typing.Any] = <factory>*)

Bases: object

Specification for exporting query results to a file.

Used to declaratively export results as part of the DSL pipeline.

path

Output file path

Type

str

fmt

Format type ('csv', 'json', 'tsv', etc.)

Type

str

columns

Optional list of columns to include/order

Type

List[str] | None

options

Additional format-specific options (e.g., delimiter, orient)

Type

Dict[str, Any]

Example

ExportSpec(path='results.csv', fmt='csv', columns=['node', 'score']) ExportSpec(path='output.json', fmt='json', options={'orient': 'records'})

columns: List[str] | None = None

fmt: str = 'csv'

options: Dict[str, Any]

path: str

class py3plex.dsl.**ExportTarget**(*value*)

Bases: Enum

Export target for query results.

ARROW = 'arrow'

NETWORKX = 'networkx'

PANDAS = 'pandas'

class py3plex.dsl.ExtendedQuery(*kind: str*, *explain: bool = False*, *select:* SelectStmt *| None = None*, *compare:* CompareStmt *| None = None*, *nullmodel:* NullModelStmt *| None = None*, *path:* PathStmt *| None = None*, *dynamics:* DynamicsStmt *| None = None*, *trajectories:* TrajectoriesStmt *| None = None*, *dsl_version: str = '2.0'*)

Bases: `object`

Extended query supporting multiple statement types.

This extends the basic Query to support COMPARE, NULLMODEL, PATH, DYNAMICS, and TRAJECTORIES statements in addition to SELECT statements.

kind

Query type ("select", "compare", "nullmodel", "path", "dynamics", "trajectories")

Type

str

explain

If True, return execution plan instead of results

Type

bool

select

SELECT statement (if kind == "select")

Type

py3plex.dsl.ast.SelectStmt | None

compare

COMPARE statement (if kind == "compare")

Type

py3plex.dsl.ast.CompareStmt | None

nullmodel

NULLMODEL statement (if kind == "nullmodel")

Type

py3plex.dsl.ast.NullModelStmt | None

path

PATH statement (if kind == "path")

Type

py3plex.dsl.ast.PathStmt | None

dynamics

DYNAMICS statement (if kind == "dynamics")

Type

py3plex.dsl.ast.DynamicsStmt | None

trajectories

TRAJECTORIES statement (if kind == "trajectories")

Type
py3plex.dsl.ast.TrajectoriesStmt | None

dsl_version
DSL version for compatibility

Type
str

`compare: CompareStmt | None = None`

`dsl_version: str = '2.0'`

`dynamics: DynamicsStmt | None = None`

`explain: bool = False`

`kind: str`

`nullmodel: NullModelStmt | None = None`

`path: PathStmt | None = None`

`select: SelectStmt | None = None`

`trajectories: TrajectoriesStmt | None = None`

`class py3plex.dsl.FieldExpression(field: str)`

Bases: `object`

Represents a field reference that can be compared to values.

This class implements operator overloading to create comparison expressions that are then converted to AST ConditionExpr objects.

_field
The field name being referenced

`class py3plex.dsl.FieldProxy`

Bases: `object`

Proxy for creating field expressions via F.field_name syntax.

This allows for intuitive syntax like:
F.degree > 5 F.layer == "social" F.is_infected

`class py3plex.dsl.FunctionCall(name: str, args: ~typing.List[str | float | int | ~py3plex.dsl.ast.ParamRef] = <factory>)`

Bases: `object`

A function call in a condition.

name
Function name (e.g., "reachable_from")

Type
str

args
List of arguments

Type
List[str | float | int | *py3plex.dsl.ast.ParamRef*]

args: List[str | float | int | *ParamRef*]

name: str

exception py3plex.dsl.**GroupingError**(*message: str, query: str | None = None, line: int | None = None, column: int | None = None*)

Bases: *DslError*

Exception raised when a grouping operation is used incorrectly.

This error is raised when operations that require active grouping (like coverage) are called without proper grouping context.

class py3plex.dsl.**LayerConstraint**(*kind: str, value: str | Set[str] | Callable | None = None*)

Bases: object

Layer constraint for a node.

kind

Type of constraint ("one", "set", "wildcard", "predicate")

Type

str

value

Layer name, set of layer names, or predicate function

Type

str | Set[str] | Callable | None

kind: str

matches(*layer: str*) → bool

Check if a layer satisfies this constraint.

static **one**(*layer: str*) → *LayerConstraint*

Create constraint for a specific layer.

static **set_of**(*layers: Set[str]*) → *LayerConstraint*

Create constraint for a set of layers.

to_dict() → Dict[str, Any]

Convert to dictionary for serialization.

value: str | Set[str] | Callable | None = None

static **wildcard**() → *LayerConstraint*

Create wildcard constraint (any layer).

class py3plex.dsl.**LayerExpr**(*terms: ~typing.List[~py3plex.dsl.ast.LayerTerm] = <factory>, ops: ~typing.List[str] = <factory>*)

Bases: object

Layer expression with optional algebra operations.

Supports:

- Union: LAYER("a") + LAYER("b")
- Difference: LAYER("a") - LAYER("b")

- Intersection: LAYER("a") & LAYER("b")

`terms`

List of layer terms

Type

List[*py3plex.dsl.ast.LayerTerm*]

`ops`

List of operators between terms ('+', '-', '&')

Type

List[str]

`get_layer_names()` → List[str]

Get all layer names referenced in this expression.

`ops: List[str]`

`terms: List[LayerTerm]`

`class py3plex.dsl.LayerExprBuilder(term: str)`

Bases: `object`

Builder for layer expressions.

Supports layer algebra:

- Union: L["social"] + L["work"]
- Difference: L["social"] - L["bots"]
- Intersection: L["social"] & L["work"]

`class py3plex.dsl.LayerProxy`

Bases: `object`

Proxy for creating layer expressions via L["name"] syntax.

Supports both simple layer names and advanced string expressions:

- L["social"] → single layer (backward compatible)
- L["social", "work"] → union of layers (backward compatible)
- L["* - coupling"] → string expression with algebra (NEW)
- L["(ppi | gene) & disease"] → complex expression (NEW)

The proxy automatically detects whether to use the old LayerExprBuilder (for simple names) or the new LayerSet (for expressions with operators).

`static clear_groups()` → None

Clear all defined layer groups.

`static define(name: str, layer_expr: LayerExprBuilder | LayerSet)` → None

Define a named layer group for reuse.

Parameters

- **name** – Group name
- **layer_expr** – LayerExprBuilder or LayerSet to associate with the name

Example

```
>>> bio = L["ppi"] | L["gene"] | L["disease"]
>>> L.define("bio", bio)
>>>
>>> # Later use the group
>>> result = Q.nodes().from_layers(L["bio"]).execute(net)
```

static list_groups() → Dict[str, Any]

List all defined layer groups.

Returns

Dictionary mapping group names to layer expressions

class py3plex.dsl.LayerSet(*name_or_expr: str | LayerExpr*)

Bases: `object`

Represents an unevaluated layer expression.

LayerSet objects are immutable and composable. They maintain an internal AST representation that is only evaluated when resolve() is called.

expr

Internal expression AST

Example

```
>>> # Create from layer name
>>> social = LayerSet("social")
>>>
>>> # Set operations
>>> both = social | LayerSet("work")
>>> non_coupling = LayerSet("*") - LayerSet("coupling")
>>>
>>> # Parse from string
>>> layers = LayerSet.parse("* - coupling - transport")
>>>
>>> # Resolve to actual layer names
>>> active = layers.resolve(network)
>>> print(active)  # {'social', 'work', 'hobby'}
```

static clear_groups() → None

Clear all defined layer groups.

Useful for testing or resetting state.

static define_group(*name: str*, *layer_set:* LayerSet) → None

Define a named layer group for reuse.

Parameters

- **name** – Group name
- **layer_set** – LayerSet to associate with the name

Example

```
>>> bio = LayerSet.parse("ppi | gene | disease")
>>> LayerSet.define_group("bio", bio)
>>>
>>> # Later, reference the group
>>> layers = LayerSet("bio") & LayerSet("*")
```

explain(*network: Any | None = None*) → str

Generate human-readable explanation of the layer expression.

Parameters
network – Optional network to resolve against (shows actual layers)

Returns
Formatted explanation string

Example

```
>>> layers = LayerSet("*") - LayerSet("coupling")
>>> print(layers.explain())
LayerSet:
  difference(
    all_layers("*"),
    layer("coupling")
  )
```

static list_groups() → Dict[str, *LayerSet*]

List all defined layer groups.

Returns
Dictionary mapping group names to LayerSet objects

static parse(*expr_str: str*) → *LayerSet*

Parse a layer expression from string.

Supports:

- Layer names: "social", "work"
- Wildcard: "*"
- Union: "social | work" or "social + work"
- Intersection: "social & work"
- Difference: "social - work"
- Complement: "~social" (future)
- Parentheses: "(social | work) & ~coupling"
- Named groups: "bio" (if defined via define_group)

Parameters
expr_str – Expression string to parse

Returns
LayerSet object

Raises
DslSyntaxError – If expression is invalid

Example

```
>>> layers = LayerSet.parse("* - coupling - transport")
>>> layers = LayerSet.parse("(ppi | gene) & disease")
```

resolve(*network: Any*, ***, *strict: bool = False*, *warn_empty: bool = True*) → Set[str]

Resolve the layer expression to a set of actual layer names.

This is where late evaluation happens. The expression is evaluated against the network's actual layers.

Parameters

- **network** – Multilayer network object
- **strict** – If True, raise error for unknown layers (default: False)
- **warn_empty** – If True, warn when result is empty (default: True)

Returns
Set of layer names (as strings)

Raises
UnknownLayerError – If strict=True and a referenced layer doesn't exist

Example

```
>>> layers = LayerSet("social") | LayerSet("work")
>>> active = layers.resolve(network)
>>> print(active)  # {'social', 'work'}
```

py3plex.dsl.**LayerSetExpr**

alias of LayerExpr

class py3plex.dsl.**LayerTerm**(*name: str*)

Bases: object

A single layer reference in a layer expression.

name

Layer name (e.g., "social", "work")

Type
str

name: str

class py3plex.dsl.**MatchRow**(*bindings: ~typing.Dict[str, ~typing.Any] = <factory>*, *edge_bindings: ~typing.Dict[str, ~typing.Tuple[~typing.Any, ~typing.Any]] | None = None*)

Bases: object

Represents a single match result.

bindings

Dictionary mapping variable names to node IDs

Type
Dict[str, Any]

`edge_bindings`
Optional dictionary mapping edge vars to edge tuples

Type
Dict[str, Tuple[Any, Any]] | None

`bindings: Dict[str, Any]`

`edge_bindings: Dict[str, Tuple[Any, Any]] | None = None`

`to_dict()` → Dict[str, Any]
Convert to dictionary.

`class py3plex.dsl.N`
Bases: `object`

NullModel factory for creating NullModelBuilder instances.

Example

```
>>> N.model("configuration").samples(100).seed(42)
```

`static configuration()` → *NullModelBuilder*
Create a configuration model builder.

`static edge_swap()` → *NullModelBuilder*
Create an edge swap model builder.

`static erdos_renyi()` → *NullModelBuilder*
Create an Erdős-Rényi model builder.

`static layer_shuffle()` → *NullModelBuilder*
Create a layer shuffle model builder.

`static model(`*model_type: str*`)` → *NullModelBuilder*
Create a null model builder.

`class py3plex.dsl.NetworkSchemaProvider(`*network: Any*`)`
Bases: `object`

Schema provider backed by a py3plex multilayer network.

`get_attribute_type(`*entity_ref:* EntityRef, *attr: str*`)` → *AttrType* | None
Get attribute type by sampling nodes/edges.

`get_edge_count(`*layer: str | None = None*`)` → int
Get edge count.

`get_node_count(`*layer: str | None = None*`)` → int
Get node count.

`list_edge_types(`*layer: str | None = None*`)` → List[str]
Get list of edge types.

`list_layers()` → List[str]
Get list of all layers.

list_node_types(*layer: str | None = None*) → List[str]

Get list of node types.

class py3plex.dsl.**NullModelBuilder**(*model_type: str*)

Bases: `object`

Builder for NULLMODEL statements.

Example

```
>>> from py3plex.dsl import N, L
>>>
>>> result = (
...     N.model("configuration")
...      .on_layers(L["social"])
...      .with_params(preserve_degree=True)
...      .samples(100)
...      .seed(42)
...      .execute(network)
... )
```

execute(*network: Any*) → NullModelResult

Execute null model generation.

Parameters
network – Multilayer network

Returns
NullModelResult with generated samples

on_layers(*layer_expr:* LayerExprBuilder) → *NullModelBuilder*

Filter by layers using layer algebra.

Parameters
layer_expr – Layer expression

Returns
Self for chaining

samples(*n: int*) → *NullModelBuilder*

Set number of samples to generate.

Parameters
n – Number of samples

Returns
Self for chaining

seed(*seed: int*) → *NullModelBuilder*

Set random seed.

Parameters
seed – Random seed

Returns
Self for chaining

to(*target: str*) → *NullModelBuilder*

Set export target.

Parameters
`target` – Export format

Returns
Self for chaining

`to_ast()` → *NullModelStmt*

Export as AST NullModelStmt object.

`with_params(**params)` → *NullModelBuilder*

Set model parameters.

Parameters
`**params` – Model parameters

Returns
Self for chaining

`class py3plex.dsl.NullModelStmt(model_type: str, layer_expr: ~py3plex.dsl.ast.LayerExpr | None = None, params: ~typing.Dict[str, ~typing.Any] = <factory>, num_samples: int = 1, seed: int | None = None, export_target: str | None = None)`

Bases: `object`

NULLMODEL statement for generating randomized networks.

DSL Example:
NULLMODEL configuration ON LAYER("social") + LAYER("work") WITH preserve_degree=True, preserve_layer_sizes=True SAMPLES 100 SEED 42

`model_type`

Type of null model (e.g., "configuration", "erdos_renyi", "layer_shuffle")

Type
str

`layer_expr`

Optional layer expression for filtering

Type
py3plex.dsl.ast.LayerExpr | None

`params`

Model parameters

Type
Dict[str, Any]

`num_samples`

Number of samples to generate

Type
int

`seed`

Optional random seed

Type
int | None

export_target

Optional export format

Type

str | None

`export_target: str | None = None`

`layer_expr: LayerExpr | None = None`

`model_type: str`

`num_samples: int = 1`

`params: Dict[str, Any]`

`seed: int | None = None`

`class py3plex.dsl.OrderItem(key: str, desc: bool = False)`

Bases: `object`

Ordering specification.

key

Attribute or computed value to order by

Type

str

desc

True for descending order, False for ascending

Type

bool

`desc: bool = False`

`key: str`

`class py3plex.dsl.P`

Bases: `object`

Path factory for creating PathBuilder instances.

Example

```
>>> P.shortest("Alice", "Bob").crossing_layers()
>>> P.random_walk("Alice").with_params(steps=100, teleport=0.1)
```

`static all_paths(source: Any, target: Any) → PathBuilder`

Create an all-paths query builder.

`static flow(source: Any, target: Any) → PathBuilder`

Create a flow analysis query builder.

`static random_walk(source: Any) → PathBuilder`

Create a random walk query builder.

`static shortest(source: Any, target: Any) → PathBuilder`

Create a shortest path query builder.

class py3plex.dsl.Param

Bases: object

Factory for parameter references.

Parameters are placeholders in queries that are bound at execution time.

Example

```
>>> q = Q.nodes().where(degree__gt=Param.int("k"))
>>> result = q.execute(network, k=5)
```

static float(*name: str*) → *ParamRef*

Create a float parameter reference.

static int(*name: str*) → *ParamRef*

Create an integer parameter reference.

static ref(*name: str*) → *ParamRef*

Create a parameter reference without type hint.

static str(*name: str*) → *ParamRef*

Create a string parameter reference.

class py3plex.dsl.ParamRef(*name: str*, *type_hint: str | None = None*)

Bases: object

Reference to a query parameter.

Parameters are placeholders in queries that are bound at execution time.

name

Parameter name (e.g., "k" for :k in DSL)

Type

str

type_hint

Optional type hint for validation

Type

str | None

name: str

type_hint: str | None = None

exception py3plex.dsl.ParameterMissingError(*parameter: str*, *provided_params: List[str] | None = None*, *query: str | None = None*, *line: int | None = None*, *column: int | None = None*)

Bases: *DslError*

Exception raised when a required parameter is not provided.

parameter

The missing parameter name

provided_params

List of provided parameter names

class py3plex.dsl.PathBuilder(*path_type: str*, *source: Any*, *target: Any | None = None*)

Bases: `object`

Builder for PATH statements.

Example

```
>>> from py3plex.dsl import P, L
>>>
>>> result = (
...     P.shortest("Alice", "Bob")
...      .on_layers(L["social"] + L["work"])
...      .crossing_layers()
...      .execute(network)
... )
```

crossing_layers(*allow: bool = True*) → *PathBuilder*

Allow or disallow cross-layer paths.

Parameters
`allow` – Whether to allow cross-layer paths

Returns
Self for chaining

execute(*network: Any*) → PathResult

Execute path query.

Parameters
`network` – Multilayer network

Returns
PathResult with found paths

limit(*n: int*) → *PathBuilder*

Limit number of results.

Parameters
`n` – Maximum number of results

Returns
Self for chaining

on_layers(*layer_expr:* LayerExprBuilder) → *PathBuilder*

Filter by layers using layer algebra.

Parameters
`layer_expr` – Layer expression

Returns
Self for chaining

to(*target: str*) → *PathBuilder*

Set export target.

Parameters
`target` – Export format

Returns
Self for chaining

to_ast() → *PathStmt*

Export as AST PathStmt object.

with_params(***params*) → *PathBuilder*

Set additional parameters.

Parameters

****params** – Additional parameters

Returns

Self for chaining

class py3plex.dsl.PathStmt(*path_type: str, source: str | ~py3plex.dsl.ast.ParamRef, target: str | ~py3plex.dsl.ast.ParamRef | None = None, layer_expr: ~py3plex.dsl.ast.LayerExpr | None = None, cross_layer: bool = False, params: ~typing.Dict[str, ~typing.Any] = <factory>, limit: int | None = None, export_target: str | None = None*)

Bases: `object`

PATH statement for path queries and flow analysis.

DSL Example:

PATH SHORTEST FROM "Alice" TO "Bob" ON LAYER("social") + LAYER("work") CROSSING LAYERS LIMIT 10

path_type

Type of path query ("shortest", "all", "random_walk", "flow")

Type

str

source

Source node identifier

Type

str | *py3plex.dsl.ast.ParamRef*

target

Optional target node identifier

Type

str | *py3plex.dsl.ast.ParamRef* | None

layer_expr

Optional layer expression for filtering

Type

py3plex.dsl.ast.LayerExpr | None

cross_layer

Whether to allow cross-layer paths

Type

bool

params

Additional parameters (e.g., max_length, teleport probability)

Type

Dict[str, Any]

limit
Optional limit on results

Type
int | None

export_target
Optional export format

Type
str | None

`cross_layer: bool = False`

`export_target: str | None = None`

`layer_expr: LayerExpr | None = None`

`limit: int | None = None`

`params: Dict[str, Any]`

`path_type: str`

`source: str | ParamRef`

`target: str | ParamRef | None = None`

class py3plex.dsl.PatternEdge(*src: str, dst: str, directed: bool = False, etype: str | None = None, predicates: ~typing.List[~py3plex.dsl.patterns.ir.Predicate] = <factory>, layer_constraint: ~py3plex.dsl.patterns.ir.EdgeLayerConstraint | None = None*)

Bases: `object`

Represents an edge between two node variables in a pattern.

src
Source variable name

Type
str

dst
Destination variable name

Type
str

directed
Whether the edge is directed

Type
bool

etype
Optional edge type/relation

Type
str | None

predicates
List of predicates for filtering

Type
List[*py3plex.dsl.patterns.ir.Predicate*]

layer_constraint
Optional layer constraint

Type
py3plex.dsl.patterns.ir.EdgeLayerConstraint | None

directed: bool = False

dst: str

etype: str | None = None

layer_constraint: *EdgeLayerConstraint* | None = None

predicates: List[*Predicate*]

src: str

to_dict() → Dict[str, Any]
Convert to dictionary for serialization.

class py3plex.dsl.PatternEdgeBuilder(*parent: PatternQueryBuilder, src: str, dst: str, directed: bool = False, etype: str | None = None*)

Bases: `object`

Builder for configuring a pattern edge.

This builder is returned by PatternQueryBuilder.edge() and provides chainable methods for specifying edge predicates and constraints.

Methods that don't return self will return the parent QueryBuilder, allowing for seamless chaining back to the main pattern builder.

any_layer() → *PatternQueryBuilder*
Allow edge to be in any layer.

Returns
Parent PatternQueryBuilder for chaining

between_layers(*src_layer: str, dst_layer: str*) → *PatternQueryBuilder*
Constrain edge to be between two specific layers.

Parameters

- **src_layer** – Source layer name
- **dst_layer** – Destination layer name

Returns
Parent PatternQueryBuilder for chaining

where(***kwargs*) → *PatternQueryBuilder*
Add predicates to the edge.

Parameters
****kwargs** – Predicate specifications

Returns
Parent PatternQueryBuilder for chaining

within_layer(*layer: str*) → *PatternQueryBuilder*

Constrain edge to be within a single layer.

Parameters
layer – Layer name

Returns
Parent PatternQueryBuilder for chaining

class py3plex.dsl.PatternGraph(*nodes: ~typing.Dict[str, ~py3plex.dsl.patterns.ir.PatternNode] = <factory>, edges: ~typing.List[~py3plex.dsl.patterns.ir.PatternEdge] = <factory>, constraints: ~typing.List[str] = <factory>, return_vars: ~typing.List[str] | None = None*)

Bases: `object`

Represents a complete pattern query.

nodes

Dictionary mapping variable names to PatternNode objects

Type
Dict[str, *py3plex.dsl.patterns.ir.PatternNode*]

edges

List of PatternEdge objects

Type
List[*py3plex.dsl.patterns.ir.PatternEdge*]

constraints

List of global constraints (e.g., all-different)

Type
List[str]

return_vars

List of variables to return (defaults to all)

Type
List[str] | None

add_constraint(*constraint: str*) → None

Add a global constraint.

add_edge(*edge:* PatternEdge) → None

Add an edge to the pattern.

add_node(*node:* PatternNode) → None

Add a node to the pattern.

constraints: List[str]

edges: List[*PatternEdge*]

get_return_vars() → List[str]

Get the list of variables to return.

nodes: Dict[str, *PatternNode*]

return_vars: List[str] | None = None

to_dict() → Dict[str, Any]

Convert to dictionary for serialization.

class py3plex.dsl.PatternNode(*var: str, labels: ~typing.Set[str] | None = None, predicates: ~typing.List[~py3plex.dsl.patterns.ir.Predicate] = <factory>, layer_constraint: ~py3plex.dsl.patterns.ir.LayerConstraint | None = None*)

Bases: `object`

Represents a node variable in a pattern.

var

Variable name (e.g., "a", "b")

Type

str

labels

Optional semantic labels (metadata only in v1)

Type

Set[str] | None

predicates

List of predicates for filtering

Type

List[*py3plex.dsl.patterns.ir.Predicate*]

layer_constraint

Optional layer constraint

Type

py3plex.dsl.patterns.ir.LayerConstraint | None

labels: Set[str] | None = None

layer_constraint: *LayerConstraint* | None = None

predicates: List[*Predicate*]

to_dict() → Dict[str, Any]

Convert to dictionary for serialization.

var: str

class py3plex.dsl.PatternNodeBuilder(*parent:* PatternQueryBuilder, *var: str, labels: str | List[str] | None = None*)

Bases: `object`

Builder for configuring a pattern node variable.

This builder is returned by PatternQueryBuilder.node() and provides chainable methods for specifying node predicates and constraints.

Methods that don't return self will return the parent QueryBuilder, allowing for seamless chaining back to the main pattern builder.

`in_layers(layers: str | List[str])` → *PatternQueryBuilder*

Specify layer constraint for the node.

Parameters
`layers` – Single layer name, list of layers, or "*" for wildcard

Returns
Parent PatternQueryBuilder for chaining

`label(*labels: str)` → *PatternNodeBuilder*

Add labels to the node.

Parameters
`*labels` – Label names

Returns
Self for chaining

`where(**kwargs)` → *PatternQueryBuilder*

Add predicates to the node.

Supports the same predicate syntax as Q.nodes().where():

- layer="social" → layer constraint
- degree__gt=5 → degree > 5
- any_attribute__op=value

Parameters
`**kwargs` – Predicate specifications

Returns
Parent PatternQueryBuilder for chaining

`class py3plex.dsl.PatternPlan(`*pattern: ~py3plex.dsl.patterns.ir.PatternGraph, root_var: str, join_order: ~typing.List[~py3plex.dsl.patterns.compiler.JoinStep] = <factory>, variable_plans: ~typing.Dict[str, ~py3plex.dsl.patterns.compiler.VariablePlan] = <factory>, estimated_complexity: int = -1*`)`

Bases: `object`

Complete execution plan for a pattern.

`pattern`

Original pattern graph

Type
py3plex.dsl.patterns.ir.PatternGraph

`root_var`

Variable to start matching from

Type
str

`join_order`

Sequence of join steps

Type

List[py3plex.dsl.patterns.compiler.JoinStep]

variable_plans

Plans for each variable

Type

Dict[str, py3plex.dsl.patterns.compiler.VariablePlan]

estimated_complexity

Rough complexity estimate

Type

int

estimated_complexity: int = -1

join_order: List[JoinStep]

pattern: *PatternGraph*

root_var: str

to_dict() → Dict[str, Any]

Convert to dictionary for serialization/display.

variable_plans: Dict[str, VariablePlan]

class py3plex.dsl.PatternQueryBuilder

Bases: object

Main builder for pattern queries.

Provides a fluent API for constructing pattern queries. The builder accumulates pattern elements (nodes, edges, constraints) and produces a PatternGraph IR object that can be compiled and executed.

Example

```
>>> pq = (
...     Q.pattern()
...      .node("a").where(degree__gt=3)
...      .node("b")
...      .edge("a", "b", directed=False)
...      .returning("a", "b")
... )
>>> matches = pq.execute(network)
```

constraint(*expr: str*) → *PatternQueryBuilder*

Add a global constraint.

Currently supports:

- "a != b" for all-different constraints
- "all_distinct([a, b, c])" for multi-variable all-different

Parameters

expr – Constraint expression

Returns
Self for chaining

edge(*src: str*, *dst: str*, *directed: bool = False*, *etype: str | None = None*) → *PatternEdgeBuilder*

Add an edge between two node variables.

Parameters

- **src** – Source variable name
- **dst** – Destination variable name
- **directed** – Whether the edge is directed
- **etype** – Optional edge type

Returns
PatternEdgeBuilder for configuring the edge

execute(*network: Any*, *backend: str = 'native'*, *max_matches: int | None = None*, *timeout: float | None = None*) → *PatternQueryResult*

Execute the pattern query on a network.

Parameters

- **network** – Multilayer network object
- **backend** – Execution backend (currently only "native" supported)
- **max_matches** – Maximum number of matches (overrides .limit())
- **timeout** – Optional timeout in seconds

Returns
PatternQueryResult with matches

explain() → Dict[str, Any]

Generate and return the compilation plan.

Returns
Dictionary with compilation plan details

limit(*n: int*) → *PatternQueryBuilder*

Limit the number of matches.

Parameters
n – Maximum number of matches

Returns
Self for chaining

node(*var: str*, *labels: str | List[str] | None = None*) → *PatternNodeBuilder*

Add a node variable to the pattern.

Parameters

- **var** – Variable name (e.g., "a", "b")
- **labels** – Optional semantic labels

Returns
PatternNodeBuilder for configuring the node

order_by(*key: str*, *desc: bool = False*) → *PatternQueryBuilder*

Order matches by a computed attribute (future enhancement).

Parameters

- **key** – Attribute key for ordering
- **desc** – Whether to sort descending

Returns
Self for chaining

path(*vars: List[str] | Tuple[str, ...]*, *directed: bool = False*, *etype: str | None = None*, *length: int | None = None*) → *PatternQueryBuilder*

Add a path pattern.

Creates edges between consecutive variables in the list. For example, path(["a", "b", "c"]) creates edges a-b and b-c.

Parameters

- **vars** – List of variable names representing the path
- **directed** – Whether edges are directed
- **etype** – Optional edge type for all edges
- **length** – Optional length constraint (currently ignored, for future use)

Returns
Self for chaining

returning(**vars: str*) → *PatternQueryBuilder*

Specify which variables to return in results.

Parameters
***vars** – Variable names to return

Returns
Self for chaining

triangle(*a: str*, *b: str*, *c: str*, *directed: bool = False*) → *PatternQueryBuilder*

Add a triangle motif.

Creates edges a-b, b-c, and c-a.

Parameters

- **a** – First variable name
- **b** – Second variable name
- **c** – Third variable name
- **directed** – Whether edges are directed

Returns
Self for chaining

class py3plex.dsl.PatternQueryResult(*pattern:* PatternGraph, *matches: List[*MatchRow*]*, *meta: Dict[str, Any] | None = None*)

Bases: `object`

Result container for pattern matching queries.

Provides access to matches with multiple export formats and projections.

pattern
Original pattern graph

matches
List of MatchRow objects

meta
Metadata about the query execution

property count: int
Get number of matches.

Returns
Number of matches

filter(*predicate*) → *PatternQueryResult*
Filter matches using a predicate function.

Parameters
predicate – Function that takes a MatchRow and returns bool

Returns
New PatternQueryResult with filtered matches

limit(*n: int*) → *PatternQueryResult*
Limit the number of matches.

Parameters
n – Maximum number of matches

Returns
New PatternQueryResult with limited matches

property rows: List[Dict[str, Any]]
Get matches as list of dictionaries.

Returns
List of dictionaries mapping variable names to node IDs

to_edges(*var_pairs: List[Tuple[str, str]] | None = None*) → List[Tuple[Any, Any]]
Extract edges from matches.

Infers edges from pairs of node variables in the pattern.

Parameters
var_pairs – Optional list of (src_var, dst_var) tuples to extract If None, uses pattern edges

Returns
List of (src_node, dst_node) tuples

to_nodes(*vars: List[str] | None = None, unique: bool = True*) → List[Any] | Set[Any]
Extract node IDs from matches.

Parameters
- **vars** – Optional list of variables to include (defaults to all)
- **unique** – If True, return unique nodes as a set

Returns
List or set of node IDs

to_pandas(*include_meta: bool = False*)

Export matches to pandas DataFrame.

Parameters

include_meta – If True, include metadata columns

Returns

pandas.DataFrame with matches

Raises

ImportError – If pandas is not available

to_subgraph(*network: Any*, *per_match: bool = False*) → Any

Extract induced subgraph(s) from matches.

Parameters

- **network** – Original network object
- **per_match** – If True, return list of subgraphs (one per match) If False, return single subgraph with all matched nodes

Returns

NetworkX graph or list of graphs

class py3plex.dsl.**PlanStep**(*description: str*, *estimated_complexity: str | None = None*)

Bases: `object`

A step in the execution plan.

description

Human-readable description of the step

Type

str

estimated_complexity

Estimated time complexity (e.g., "O(|V|)")

Type

str | None

description: str

estimated_complexity: str | None = None

class py3plex.dsl.**Predicate**(*attr: str*, *op: str*, *value: Any*)

Bases: `object`

A predicate for filtering nodes or edges.

attr

Attribute name (e.g., "degree", "weight")

Type

str

op

Comparison operator (">", ">=", "<", "<=", "=", "!=")

Type

str

value
Value to compare against

Type
Any

attr: str

op: str

to_dict() → Dict[str, Any]
Convert to dictionary for serialization.

value: Any

class py3plex.dsl.Q

Bases: `object`

Query factory for creating QueryBuilder instances.

Example

```
>>> Q.nodes().where(layer="social").compute("degree")
>>> Q.edges().where(intralayer=True)
>>> Q.nodes(autocompute=False).where(degree__gt=5)  # Disable autocompute
>>> Q.dynamics("SIS", beta=0.3).run(steps=100)  # Dynamics simulation
>>> Q.trajectories("sim_result").at(50)  # Query trajectories
```

static dynamics(*process_name: str*, ***params*) → *DynamicsBuilder*
Create a dynamics simulation builder.

Parameters

- **process_name** – Name of the process (e.g., "SIS", "SIR", "RANDOM_WALK")
- ****params** – Process parameters (e.g., beta=0.3, mu=0.1)

Returns
DynamicsBuilder for configuring and running simulations

Example

```
>>> sim = (
...     Q.dynamics("SIS", beta=0.3, mu=0.1)
...      .on_layers(L["contacts"])
...      .seed(0.01)
...      .run(steps=100, replicates=10, track="all")
...      .execute(network)
... )
```

static edges(*autocompute: bool = True*) → *QueryBuilder*
Create a query builder for edges.

Parameters
autocompute – Whether to automatically compute missing metrics (default: True)

Returns
QueryBuilder for edges

static nodes(*autocompute: bool = True*) → *QueryBuilder*

Create a query builder for nodes.

Parameters
autocompute – Whether to automatically compute missing metrics (default: True)

Returns
QueryBuilder for nodes

static pattern() → *PatternQueryBuilder*

Create a pattern matching query builder.

Returns
PatternQueryBuilder for constructing pattern queries

Example

```
>>> pq = (
...     Q.pattern()
...      .node("a").where(layer="social", degree__gt=3)
...      .node("b").where(layer="social")
...      .edge("a", "b", directed=False).where(weight__gt=0.2)
...      .returning("a", "b")
... )
>>> matches = pq.execute(network)
>>> df = matches.to_pandas()
```

static trajectories(*process_ref: str*) → *TrajectoriesBuilder*

Create a trajectories query builder.

Parameters
process_ref – Reference to a simulation result or process name

Returns
TrajectoriesBuilder for querying simulation outputs

Example

```
>>> result = (
...     Q.trajectories("sim_result")
...      .at(50)
...      .measure("peak_time", "final_state")
...      .execute(context)
... )
```

class uncertainty

Bases: `object`

Global defaults for uncertainty estimation.

This class provides a way to configure default parameters for uncertainty estimation that will be used when uncertainty=True is passed to compute() but specific parameters are omitted.

Example

```
>>> from py3plex.dsl import Q
>>>
>>> # Set global defaults
>>> Q.uncertainty.defaults(
...     enabled=True,
...     n_boot=200,
...     ci=0.95,
...     bootstrap_unit="edges",
...     bootstrap_mode="resample",
...     random_state=42
... )
>>>
>>> # Now compute() will use these defaults
>>> Q.nodes().compute("degree", uncertainty=True).execute(net)
```

```
>>> # Reset to defaults
>>> Q.uncertainty.reset()
```

classmethod defaults(***kwargs*) → None

Set global defaults for uncertainty estimation.

Parameters

- **enabled** – Whether uncertainty is enabled by default (default: False)
- **n_boot** – Number of bootstrap replicates (default: 50)
- **n_samples** – Alias for n_boot (default: 50)
- **ci** – Confidence interval level (default: 0.95)
- **bootstrap_unit** – What to resample - "edges", "nodes", or "layers" (default: "edges")
- **bootstrap_mode** – Resampling mode - "resample" or "permute" (default: "resample")
- **method** – Uncertainty estimation method - "bootstrap", "perturbation", "seed" (default: "bootstrap")
- **random_state** – Random seed for reproducibility (default: None)
- **n_null** – Number of null model replicates (default: 200)
- **null_model** – Null model type - "degree_preserving", "erdos_renyi", "configuration" (default: "degree_preserving")

Example

```
>>> Q.uncertainty.defaults(
...     enabled=True,
...     n_boot=500,
...     ci=0.95,
...     bootstrap_unit="edges"
... )
```

classmethod get(*key: str*, *default: Any | None = None*) → Any

Get a default value.

Parameters

- **key** – Parameter name
- **default** – Value to return if key not found

Returns

Default value for the parameter

classmethod get_all() → Dict[str, Any]

Get all current defaults as a dictionary.

Returns

Dictionary of all default values

classmethod reset() → None

Reset all defaults to their initial values.

class py3plex.dsl.Query(*explain: bool*, *select: SelectStmt*, *dsl_version: str = '2.0'*)

Bases: `object`

Top-level query representation.

explain

If True, return execution plan instead of results

Type

bool

select

The SELECT statement

Type

py3plex.dsl.ast.SelectStmt

dsl_version

DSL version for compatibility

Type

str

dsl_version: str = '2.0'

explain: bool

select: ***SelectStmt***

class py3plex.dsl.QueryBuilder(*target: Target*, *autocompute: bool = True*)

Bases: `object`

Chainable query builder.

Use Q.nodes() or Q.edges() to create a builder, then chain methods to construct the query.

after(*t: float*) → *QueryBuilder*

Add temporal constraint for edges/nodes after a specific time.

Convenience method equivalent to *.during(t, None)*. Filters to only include edges/nodes active after (and at) time t.

Parameters

t – Lower bound timestamp (inclusive)

Returns
Self for chaining

Examples

```
>>> # Get all edges after time 100
>>> Q.edges().after(100.0).execute(network)
```

```
>>> # Nodes active after 2024-01-01
>>> Q.nodes().after(1704067200.0).execute(network)
```

aggregate(***aggregations*) → *QueryBuilder*

Aggregate columns with support for lambdas and builtin functions.

This method computes aggregations over the result set. It supports: - Built-in aggregation functions: mean(), sum(), min(), max(), std(), count() - Direct attribute references for last/first value - Lambda functions for custom aggregations

The aggregations are computed after grouping if active, otherwise globally.

Parameters
****aggregations** – Named aggregations where: - Key is the output column name - Value is either:

- A string like "mean(degree)" or "sum(weight)"
- A string attribute name (gets the value directly)
- A lambda function receiving each item

Returns
Self for chaining

Example

```
>>> Q.nodes().per_layer().aggregate(
...     avg_degree="mean(degree)",
...     max_bc="max(betweenness_centrality)",
...     node_count="count()",
...     layer_name="layer"  # Direct attribute
... )
```

```
>>> # With lambda
>>> Q.nodes().aggregate(
...     community_size=lambda n: network.community_sizes[network.get_
↪partition(n)]
... )
```

arrange(**columns: str, desc: bool = False*) → *QueryBuilder*

Sort results by specified columns (dplyr-style alias for order_by).

This is a convenience method that provides dplyr-style syntax. Columns can be prefixed with "-" to indicate descending order.

Parameters

- ***columns** – Column names to sort by (prefix with "-" for descending)

- **desc** – Default sort direction (only used if column has no prefix)

Returns
Self for chaining

Example

```
>>> Q.nodes().compute("degree").arrange("degree")  # ascending
>>> Q.nodes().compute("degree").arrange("-degree")  # descending
>>> Q.nodes().compute("degree", "betweenness").arrange("degree", "-
↪betweenness")
```

at(*t: float*) → *QueryBuilder*

Add temporal snapshot constraint (AT clause).

Filters edges to only those active at a specific point in time. For point-in-time edges (with 't' attribute), includes edges where t_edge == t. For interval edges (with 't_start', 't_end'), includes edges where t is in [t_start, t_end].

Parameters
t – Timestamp for snapshot

Returns
Self for chaining

Examples

```
>>> # Snapshot at specific time
>>> Q.edges().at(150.0).execute(network)
```

before(*t: float*) → *QueryBuilder*

Add temporal constraint for edges/nodes before a specific time.

Convenience method equivalent to *.during(None, t)*. Filters to only include edges/nodes active before (and at) time t.

Parameters
t – Upper bound timestamp (inclusive)

Returns
Self for chaining

Examples

```
>>> # Get all edges before time 100
>>> Q.edges().before(100.0).execute(network)
```

```
>>> # Nodes active before 2024-01-01
>>> Q.nodes().before(1704067200.0).execute(network)
```

centrality(**metrics: str*, ***aliases: str*) → *QueryBuilder*

Compute centrality metrics (convenience wrapper for compute).

This is a domain-specific convenience method for computing common centrality measures. It's equivalent to calling compute() with the metric names.

Supported metrics:

- degree
- betweenness (or betweenness_centrality)
- closeness (or closeness_centrality)
- eigenvector (or eigenvector_centrality)
- pagerank
- clustering (or clustering_coefficient)

Parameters

- `*metrics` – Centrality metric names
- `**aliases` – Optional aliases for metrics (alias=metric_name)

Returns

Self for chaining

Example

```
>>> Q.nodes().centrality("degree", "betweenness", "pagerank")
>>> Q.nodes().centrality("degree", bc="betweenness_centrality")
```

`compute`(**measures: str, alias: str | None = None, aliases: Dict[str, str] | None = None, uncertainty: bool | None = None, method: str | None = None, n_samples: int | None = None, ci: float | None = None, bootstrap_unit: str | None = None, bootstrap_mode: str | None = None, n_boot: int | None = None, n_null: int | None = None, null_model: str | None = None, random_state: int | None = None*) → *QueryBuilder*

Add measures to compute with optional uncertainty estimation.

Parameters

- `*measures` – Measure names to compute
- `alias` – Alias for single measure
- `aliases` – Dictionary mapping measure names to aliases
- `uncertainty` – Whether to compute uncertainty for these measures. If None, uses Q.uncertainty defaults or the global uncertainty context.
- `method` – Uncertainty estimation method ('bootstrap', 'perturbation', 'seed', 'null_model')
- `n_samples` – Number of samples for uncertainty estimation (default: from Q.uncertainty.defaults)
- `ci` – Confidence interval level (default: from Q.uncertainty.defaults)
- `bootstrap_unit` – What to resample - "edges", "nodes", or "layers" (default: from Q.uncertainty.defaults)
- `bootstrap_mode` – Resampling mode - "resample" or "permute" (default: from Q.uncertainty.defaults)
- `n_boot` – Alias for n_samples (for bootstrap)
- `n_null` – Number of null model replicates (default: from

Q.uncertainty.defaults)

- **null_model** – Null model type - "degree_preserving", "erdos_renyi", "configuration" (default: from Q.uncertainty.defaults)
- **random_state** – Random seed for reproducibility (default: from Q.uncertainty.defaults)

Returns
Self for chaining

Example

```
>>> # Without uncertainty
>>> Q.nodes().compute("degree", "betweenness_centrality")
```

```
>>> # With uncertainty using explicit parameters
>>> Q.nodes().compute(
...     "degree", "betweenness_centrality",
...     uncertainty=True,
...     method="bootstrap",
...     n_samples=500,
...     ci=0.95
... )
```

```
>>> # With uncertainty using global defaults
>>> Q.uncertainty.defaults(n_boot=500, ci=0.95)
>>> Q.nodes().compute("degree", uncertainty=True)
```

coverage(*mode: str = 'all', k: int | None = None, threshold: int | None = None, p: float | None = None, group: str | None = None, id_field: str = 'id'*) → *QueryBuilder*

Configure coverage filtering across groups.

Coverage determines which items appear in the final result based on how many groups they appear in after grouping and top_k filtering.

Parameters

- **mode** – Coverage mode: - "all": Keep items that appear in ALL groups - "any": Keep items that appear in AT LEAST ONE group - "at_least": Keep items that appear in at least k groups (requires k/threshold parameter) - "exact": Keep items that appear in exactly k groups (requires k/threshold parameter) - "fraction": Keep items that appear in at least p fraction (0-1) of groups (requires p parameter)
- **k** – Threshold for "at_least" or "exact" modes
- **threshold** – Alias for k parameter
- **p** – Fraction threshold (0.0-1.0) for "fraction" mode. E.g., p=0.67 means at least 67% of groups
- **group** – Group attribute for coverage (defaults to primary grouping context)
- **id_field** – Field to use for identity matching (default: "id" for nodes)

Returns
Self for chaining

Raises

- **ValueError** – If mode is invalid or required parameters are missing
- **ValueError** – If called without prior grouping

Example

```
>>> # Nodes that are top-5 hubs in ALL layers
>>> Q.nodes().per_layer().top_k(5, "betweenness").coverage(mode="all
↪")
```

```
>>> # Nodes that are top-5 in at least 2 layers
>>> Q.nodes().per_layer().top_k(5, "degree").coverage(mode="at_least
↪", k=2)
>>> # Or equivalently:
>>> Q.nodes().per_layer().top_k(5, "degree").coverage(mode="at_least
↪", threshold=2)
```

```
>>> # Nodes in top-10 in at least 70% of layers (0.7 fraction)
>>> Q.nodes().per_layer().top_k(10, "degree").coverage(mode="fraction
↪", p=0.7)
```

distinct(**columns: str*) → *QueryBuilder*

Return unique rows based on specified columns.

If columns are specified, deduplicates based on those columns only. If no columns are specified, deduplicates based on all columns.

Parameters
***columns** – Optional column names to use for uniqueness check

Returns
Self for chaining

Example

```
>>> # Unique (node, layer) pairs
>>> Q.nodes().distinct()
```

```
>>> # Unique communities per layer
>>> Q.nodes().distinct("community", "layer")
```

drop(**columns: str*) → *QueryBuilder*

Remove specified columns from the result.

This operation filters out the specified columns from the output. Complementary to select() - use drop() when it's easier to specify what to remove rather than what to keep.

Parameters
***columns** – Column names to remove from the result

Returns
Self for chaining

Example

```
>>> Q.nodes().compute("degree", "betweenness", "closeness").drop(
↪"closeness")
```

during(*t0: float | None = None, t1: float | None = None*) → *QueryBuilder*

Add temporal range constraint (DURING clause).

Filters edges to only those active during a time range [t0, t1]. For point-in-time edges, includes edges where t is in [t0, t1]. For interval edges, includes edges where the interval overlaps [t0, t1].

Parameters

- **t0** – Start of time range (None means -infinity)
- **t1** – End of time range (None means +infinity)

Returns
Self for chaining

Examples

```
>>> # Time range query
>>> Q.edges().during(100.0, 200.0).execute(network)
```

```
>>> # Open-ended ranges
>>> Q.edges().during(100.0, None).execute(network)  # From 100␣
↪onwards
>>> Q.edges().during(None, 200.0).execute(network)  # Up to 200
```

end_grouping() → *QueryBuilder*

Marker for the end of grouping configuration.

This is purely for API readability and has no effect on execution. It helps visually separate grouping operations from post-grouping operations.

Returns
Self for chaining

Example

```
>>> (Q.nodes()
...   .per_layer()
...     .top_k(5, "degree")
...   .end_grouping()
...   .coverage(mode="all"))
```

execute(*network: Any, **params*) → *QueryResult*

Execute the query.

Parameters

- **network** – Multilayer network object
- ****params** – Parameter bindings

Returns
QueryResult with results and metadata

explain() → ExplainQuery

Create EXPLAIN query for execution plan.

Returns
ExplainQuery that can be executed to get the plan

export(*path: str*, *fmt: str = 'csv'*, *columns: List[str] | None = None*, ***options*) → *QueryBuilder*

Attach a file export specification to the query.

This adds a side-effect to write query results to a file when executed. The query will still return the QueryResult as normal.

Parameters

- **path** – Output file path (string)
- **fmt** – Format type ('csv', 'json', 'tsv')
- **columns** – Optional list of column names to include/order
- ****options** – Format-specific options (e.g., delimiter=';', orient='records')

Returns
Self for chaining

Raises
ValueError – If format is not supported

Example

```
>>> q = (
...     Q.nodes()
...      .compute("degree")
...      .export("results.csv", fmt="csv", columns=["node", "degree
↪"])
... )
```

export_csv(*path: str*, *columns: List[str] | None = None*, *delimiter: str = ','*, ***options*) → *QueryBuilder*

Export query results to CSV file.

Convenience wrapper around .export() for CSV format.

Parameters

- **path** – Output CSV file path
- **columns** – Optional list of columns to include/order
- **delimiter** – CSV delimiter (default: ',')
- ****options** – Additional CSV-specific options

Returns

Self for chaining

export_json(*path: str*, *columns: List[str] | None = None*, *orient: str = 'records'*, ***options*) → *QueryBuilder*

Export query results to JSON file.

Convenience wrapper around .export() for JSON format.

Parameters

- **path** – Output JSON file path
- **columns** – Optional list of columns to include/order
- **orient** – JSON orientation ('records', 'split', 'index', 'columns', 'values')
- ****options** – Additional JSON-specific options

Returns

Self for chaining

from_layers(*layer_expr:* LayerExprBuilder | LayerSet) → *QueryBuilder*

Filter by layers using layer algebra.

Supports both LayerExprBuilder (backward compatible) and LayerSet (new).

Parameters

layer_expr – Layer expression (e.g., L["social"] + L["work"] or L["* - coupling"])

Returns

Self for chaining

Example

```
>>> # Old style (still works)
>>> Q.nodes().from_layers(L["social"] + L["work"])
>>>
>>> # New style with string expressions
>>> Q.nodes().from_layers(L["* - coupling"])
>>> Q.nodes().from_layers(L["(ppi | gene) & disease"])
```

group_by(**fields: str*) → *QueryBuilder*

Group result items by given fields.

This is the low-level grouping primitive used by per_layer(). Once grouping is established, you can apply per-group operations like top_k().

Parameters

***fields** – Attribute names to group by (e.g., "layer")

Returns

Self for chaining

Example

```
>>> Q.nodes().group_by("layer").top_k(5, "degree")
```

has_community(*predicate*) → *QueryBuilder*

Filter nodes based on a community-related predicate.

This method filters nodes based on their community membership or community-related attributes. The predicate can be: - A callable: Called with each node tuple, should return bool - A value: Direct equality check against "community" attribute

Parameters
predicate – Either a callable(node_tuple) -> bool or a value to match

Returns
Self for chaining

Example

```
>>> # Filter by community ID
>>> Q.nodes().has_community(3)
```

```
>>> # Filter by custom predicate
>>> Q.nodes().has_community(
...     lambda n: network.get_node_attribute(n, "disease_enriched")␣
↪is True
... )
```

limit(*n: int*) → *QueryBuilder*

Limit number of results.

Parameters
n – Maximum number of results

Returns
Self for chaining

node_type(*node_type: str*) → *QueryBuilder*

Filter nodes by node_type attribute.

This is a convenience method that adds a WHERE condition filtering by the "node_type" attribute. Equivalent to .where(node_type=node_type).

Parameters
node_type – Node type to filter by (e.g., "gene", "protein", "drug")

Returns
Self for chaining

Example

```
>>> Q.nodes().node_type("gene").compute("degree")
```

order_by(**keys: str*, *desc: bool = False*) → *QueryBuilder*

Add ORDER BY clause.

Parameters

- ***keys** – Attribute names to order by (prefix with "-" for descending)
- **desc** – Default sort direction

Returns
Self for chaining

per_community() → *QueryBuilder*

Group results by community (sugar for group_by("community")).

Similar to per_layer(), but groups by community attribute. Useful after community detection has been run and community assignments are stored in node attributes.

Returns
Self for chaining

Example

```
>>> # Find top nodes per community
>>> Q.nodes().per_community().top_k(5, "betweenness_centrality")
```

per_layer() → *QueryBuilder*

Group results by layer (sugar for group_by("layer")).

This is the most common grouping operation for multilayer queries. After calling this, you can apply per-layer operations like top_k().

Note: Only valid for node queries. For edge queries, use per_layer_pair().

Returns
Self for chaining

Raises
DslExecutionError – If called on an edge query

Example

```
>>> Q.nodes().per_layer().top_k(5, "betweenness_centrality")
```

per_layer_pair() → *QueryBuilder*

Group edge results by (src_layer, dst_layer) pair.

This is the grouping operation for edge queries in multilayer networks. After calling this, you can apply per-layer-pair operations like top_k().

Note: Only valid for edge queries. For node queries, use per_layer().

Returns
Self for chaining

Raises
DslExecutionError – If called on a node query

Example

```
>>> Q.edges().per_layer_pair().top_k(5, "edge_betweenness_centrality
↪")
```

rank_by(*attr: str*, *method: str = 'dense'*) → *QueryBuilder*

Add rank column based on specified attribute.

Computes ranks within the current grouping context. If grouping is active, ranks are computed per group. Otherwise, ranks are global.

The rank column will be named "{attr}_rank".

Parameters

- **attr** – Attribute to rank by
- **method** – Ranking method - "dense", "min", "max", "average", "first" (follows pandas.Series.rank semantics)

Returns

Self for chaining

Example

```
>>> # Global ranking
>>> Q.nodes().compute("degree").rank_by("degree")
```

```
>>> # Per-layer ranking
>>> Q.nodes().compute("degree").per_layer().rank_by("degree", "dense
↪")
```

rename(***mapping: str*) → *QueryBuilder*

Rename columns in the result.

Provide keyword arguments where the key is the new name and the value is the old name to rename.

Parameters

****mapping** – Mapping from new names to old names (new=old)

Returns

Self for chaining

Example

```
>>> Q.nodes().compute("degree", "betweenness_centrality").rename(
...     deg="degree", bc="betweenness_centrality"
... )
```

select(**columns: str*) → *QueryBuilder*

Keep only specified columns in the result.

This operation filters the output columns, keeping only the ones specified. Useful for reducing result size and focusing on specific attributes.

Parameters

***columns** – Column names to keep in the result

Returns

Self for chaining

Example

```
>>> Q.nodes().compute("degree", "betweenness_centrality").select("id
↪", "degree")
```

`sort`(*by: str*, *descending: bool = False*) → *QueryBuilder*

Sort results by a column (convenience alias for order_by).

This provides a more intuitive API matching common data analysis patterns (e.g., pandas DataFrame.sort_values).

Parameters

- **by** – Column name to sort by
- **descending** – If True, sort in descending order (default: False)

Returns

Self for chaining

Example

```
>>> Q.nodes().compute("degree").sort(by="degree", descending=True)
```

`summarize`(***aggregations: str*) → *QueryBuilder*

Aggregate over the current grouping context.

Computes summary statistics per group when grouping is active, or globally if no grouping is set. Aggregation expressions are strings like "mean(degree)", "max(degree)", "n()".

Supported aggregations:

- n() : count of items
- mean(attr) : mean value
- sum(attr) : sum of values
- min(attr) : minimum value
- max(attr) : maximum value
- std(attr) : standard deviation
- var(attr) : variance

Parameters

****aggregations** – Named aggregations (name=expression)

Returns

Self for chaining

Raises

ValueError – If aggregation expression is invalid

Example

```
>>> Q.nodes().from_layers(L["*"]).compute("degree").per_layer().
↪summarize(
...     mean_degree="mean(degree)",
...     max_degree="max(degree)",
...     n="n()"
... )
```

`to(target: str)` → *QueryBuilder*

Set export target.

Parameters
`target` – Export format ('pandas', 'networkx', 'arrow')

Returns
Self for chaining

`to_ast()` → *Query*

Export as AST Query object.

Returns
Query AST node

`to_dsl()` → str

Export as DSL string.

Returns
DSL query string

`top_k(k: int, key: str | None = None)` → *QueryBuilder*

Keep the top-k items per group, ordered by the given key.

Requires that group_by() or per_layer() has been called first.

Parameters

- `k` – Number of items to keep per group
- `key` – Attribute/measure to sort by (descending). If None, uses existing order_by.

Returns
Self for chaining

Raises
`ValueError` – If called without prior grouping

Example

```
>>> Q.nodes().per_layer().top_k(5, "betweenness_centrality")
```

`uncertainty(method: str | None = 'perturbation', n_samples: int | None = 50, ci: float | None = 0.95, seed: int | None = None, **kwargs)` → *QueryBuilder*

Alias for uq() - set query-scoped uncertainty configuration.

See uq() for full documentation.

`uq(method: str | None = 'perturbation', n_samples: int | None = 50, ci: float | None = 0.95, seed: int | None = None, **kwargs)` → *QueryBuilder*

Set query-scoped uncertainty quantification configuration.

This method establishes uncertainty defaults for all metrics computed in this query, unless overridden on a per-metric basis in compute().

Parameters

- **method** – Uncertainty estimation method ('bootstrap', 'perturbation', 'seed', 'null_model') Pass None to disable query-level uncertainty.
- **n_samples** – Number of samples for uncertainty estimation (default: 50)
- **ci** – Confidence interval level (default: 0.95 for 95% CI)
- **seed** – Random seed for reproducibility (default: None)
- ****kwargs** – Additional method-specific parameters (e.g., bootstrap_unit='edges', bootstrap_mode='resample', null_model='configuration')

Returns

Self for chaining

Example

```
>>> # Set uncertainty defaults for the query
>>> (Q.nodes()
...     .uq(method="perturbation", n_samples=100, ci=0.95, seed=42)
...     .compute("betweenness_centrality")
...     .where(betweenness_centrality__mean__gt=0.1)
...     .execute(net))
```

```
>>> # Use UQ profile (see UQ class for presets)
>>> (Q.nodes()
...     .uq(UQ.fast(seed=7))
...     .compute("degree")
...     .execute(net))
```

```
>>> # Disable query-level uncertainty
>>> Q.nodes().uq(method=None).compute("degree").execute(net)
```

where(**args*, ***kwargs*) → *QueryBuilder*

Add WHERE conditions.

Supports two styles:

1. **Keyword arguments:**
   - layer="social" → equality
   - degree__gt=5 → comparison (gt, ge, lt, le, eq, ne)
   - intralayer=True → intralayer predicate
   - interlayer=("social","work") → interlayer predicate
2. **Expression objects (using F):**

- where(F.degree > 5)
- where((F.degree > 5) & (F.layer == "social"))
- where((F.degree > 10) | (F.clustering < 0.5))

Can mix both styles:

- where(F.degree > 5, layer="social")

Parameters

- ***args** – BooleanExpression objects from F
- ****kwargs** – Conditions as keyword arguments

Returns
Self for chaining

window(*window_size: float | str*, *step: float | str | None = None*, *start: float | None = None*, *end: float | None = None*, *aggregation: str = 'list'*) → *QueryBuilder*

Add sliding window specification for temporal analysis.

Enables queries that operate over sliding time windows, useful for streaming algorithms and temporal pattern analysis.

Parameters

- **window_size** – Size of each window. Can be: - Numeric: treated as timestamp units - String: duration like "7d", "1h", "30m"
- **step** – Step size between windows (defaults to window_size for non-overlapping). Same format as window_size.
- **start** – Optional start time for windowing (defaults to network's first timestamp)
- **end** – Optional end time for windowing (defaults to network's last timestamp)
- **aggregation** – How to aggregate results across windows: - "list": Return list of per-window results - "concat": Concatenate DataFrames - "avg": Average numeric columns

Returns
Self for chaining

Examples

```
>>> # Non-overlapping windows of size 100
>>> Q.nodes().compute("degree").window(100.0).execute(tnet)
```

```
>>> # Overlapping windows: size 100, step 50
>>> Q.nodes().compute("degree").window(100.0, step=50.0).
↪execute(tnet)
```

```
>>> # Duration strings (for datetime timestamps)
>>> Q.edges().window("7d", step="1d").execute(tnet)
```

> **Note**
>
> Window queries require a TemporalMultiLayerNetwork instance. For regular multi_layer_network, an error will be raised.

zscore(**attrs: str*) → *QueryBuilder*

Compute z-scores for specified attributes.

For each attribute, computes the z-score (standardized value) within the current grouping context. If grouping is active, z-scores are computed per group. Otherwise, they are global.

Creates new columns named "{attr}_zscore".

Parameters

***attrs** – Attribute names to compute z-scores for

Returns

Self for chaining

Example

```
>>> # Global z-scores
>>> Q.nodes().compute("degree", "betweenness").zscore("degree",
↪"betweenness")
```

```
>>> # Per-layer z-scores
>>> Q.nodes().compute("degree").per_layer().zscore("degree")
```

class py3plex.dsl.QueryResult(*target: str, items: List[Any], attributes: Dict[str, List[Any] | Dict[Any, Any]] | None = None, meta: Dict[str, Any] | None = None, computed_metrics: set | None = None*)

Bases: `object`

Rich result object from DSL query execution.

Provides access to query results with multiple export formats and execution metadata.

target

'nodes' or 'edges'

items

Sequence of node/edge identifiers

attributes

Dictionary of computed attributes (column -> values or dict)

meta

Metadata about the query execution

computed_metrics

Set of metrics that were computed during query execution

property count: int

Get number of items in result.

property edges: List[Any]

Get edges (raises if target is not 'edges').

group_summary()

Return a summary DataFrame with one row per group.

Returns a pandas DataFrame containing: - Grouping key columns (e.g., "layer", "src_layer", "dst_layer") - n_items: Number of items (nodes/edges) in each group - Any group-level coverage metrics if available

This method only uses information already present in the result and does not recompute expensive measures.

Returns
pandas.DataFrame with one row per group

Raises

- `ImportError` – If pandas is not available
- `ValueError` – If result does not have grouping metadata

property nodes: List[Any]

Get nodes (raises if target is not 'nodes').

to_arrow()

Export results to Apache Arrow table.

Returns
pyarrow.Table with items and computed attributes

Raises
`ImportError` – If pyarrow is not available

to_dict() → Dict[str, Any]

Export results as a dictionary.

Returns
Dictionary with target, items, attributes, and metadata

to_networkx(*network: Any | None = None*)

Export results to NetworkX graph.

For node queries: Returns subgraph containing the selected nodes For edge queries: Returns subgraph containing the selected edges and their endpoints

Parameters
`network` – Optional source network to extract subgraph from

Returns
networkx.Graph subgraph containing result items

Raises
`ImportError` – If networkx is not available

to_pandas(*multiindex: bool = False, include_grouping: bool = True, expand_uncertainty: bool = False*)

Export results to pandas DataFrame.

For node queries: Returns DataFrame with 'id' column plus computed attributes For edge queries: Returns DataFrame with 'source', 'target', 'source_layer',

'target_layer', 'weight' columns plus computed attributes

Parameters

- **multiindex** – If True and grouping metadata is present, set DataFrame index to the grouping keys (e.g., ["layer"] or ["src_layer", "dst_layer"])
- **include_grouping** – If True and grouping metadata is present, ensure grouping key columns are included in the DataFrame
- **expand_uncertainty** – If True, expand uncertainty metrics into multiple columns: - metric (point estimate/mean) - metric_std (standard deviation) - metric_ci95_low (95% CI lower bound) - metric_ci95_high (95% CI upper bound) - metric_ci95_width (CI width)

Returns

pandas.DataFrame with items and computed attributes

Raises

`ImportError` – If pandas is not available

Example

```
>>> result = Q.nodes().uq(UQ.fast()).compute("degree").execute(net)
>>> df = result.to_pandas(expand_uncertainty=True)
>>> # df now has columns: id, layer, degree, degree_std, degree_ci95_
↪low, degree_ci95_high, degree_ci95_width
```

`class py3plex.dsl.SchemaProvider(*args, **kwargs)`

Bases: `Protocol`

Protocol for schema providers.

Schema providers allow the linter to query information about available layers, node/edge types, and attributes in a network.

`get_attribute_type(entity_ref: EntityRef, attr: str) → AttrType | None`

Get the type of an attribute for a given entity.

Parameters

- **entity_ref** – Reference to the entity (node/edge + layer)
- **attr** – Attribute name

Returns

Attribute type or None if unknown

`get_edge_count(layer: str | None = None) → int`

Get approximate edge count, optionally for a specific layer.

`get_node_count(layer: str | None = None) → int`

Get approximate node count, optionally for a specific layer.

`list_edge_types(layer: str | None = None) → List[str]`

Get list of edge types, optionally filtered by layer.

`list_layers() → List[str]`

Get list of all layers in the network.

`list_node_types(layer: str | None = None) → List[str]`

Get list of node types, optionally filtered by layer.

class py3plex.dsl.SelectStmt(*target: ~py3plex.dsl.ast.Target, layer_expr: ~py3plex.dsl.ast.LayerExpr | None = None, layer_set: ~typing.Any | None = None, where: ~py3plex.dsl.ast.ConditionExpr | None = None, compute: ~typing.List[~py3plex.dsl.ast.ComputeItem] = <factory>, order_by: ~typing.List[~py3plex.dsl.ast.OrderItem] = <factory>, limit: int | None = None, export: ~py3plex.dsl.ast.ExportTarget | None = None, file_export: ~py3plex.dsl.ast.ExportSpec | None = None, temporal_context: ~py3plex.dsl.ast.TemporalContext | None = None, window_spec: ~py3plex.dsl.ast.WindowSpec | None = None, group_by: ~typing.List[str] = <factory>, limit_per_group: int | None = None, coverage_mode: str | None = None, coverage_k: int | None = None, coverage_p: float | None = None, coverage_group: str | None = None, coverage_id_field: str = 'id', select_cols: ~typing.List[str] | None = None, drop_cols: ~typing.List[str] | None = None, rename_map: ~typing.Dict[str, str] | None = None, summarize_aggs: ~typing.Dict[str, str] | None = None, distinct_cols: ~typing.List[str] | None = None, rank_specs: ~typing.List[~typing.Tuple[str, str]] | None = None, zscore_attrs: ~typing.List[str] | None = None, post_filters: ~typing.List[~typing.Dict[str, ~typing.Any]] | None = None, aggregate_specs: ~typing.Dict[str, ~typing.Any] | None = None, autocompute: bool = True, uq_config: ~py3plex.dsl.ast.UQConfig | None = None*)

Bases: `object`

A SELECT statement.

target

What to select (nodes or edges)

Type

py3plex.dsl.ast.Target

layer_expr

Optional layer expression for filtering

Type

py3plex.dsl.ast.LayerExpr | None

where

Optional WHERE conditions

Type

py3plex.dsl.ast.ConditionExpr | None

compute

List of measures to compute

Type

List[*py3plex.dsl.ast.ComputeItem*]

order_by

List of ordering specifications

Type
List[*py3plex.dsl.ast.OrderItem*]

`limit`
Optional limit on results

Type
int | None

`export`
Optional export target (for result format conversion)

Type
py3plex.dsl.ast.ExportTarget | None

`file_export`
Optional file export specification (for writing to files)

Type
py3plex.dsl.ast.ExportSpec | None

`temporal_context`
Optional temporal context for time-based queries

Type
py3plex.dsl.ast.TemporalContext | None

`window_spec`
Optional window specification for sliding window analysis

Type
py3plex.dsl.ast.WindowSpec | None

`group_by`
List of attribute names to group by (e.g., ["layer"])

Type
List[str]

`limit_per_group`
Optional per-group limit for top-k filtering

Type
int | None

`coverage_mode`
Coverage filtering mode ("all", "any", "at_least", "exact", "fraction")

Type
str | None

`coverage_k`
Threshold for "at_least" or "exact" coverage modes

Type
int | None

`coverage_p`
Fraction threshold for "fraction" coverage mode

Type
float | None

coverage_group
Group attribute for coverage (defaults to primary grouping)

Type
str | None

coverage_id_field
Field to use for coverage identity (default: "id")

Type
str

select_cols
Optional list of columns to keep (for select() operation)

Type
List[str] | None

drop_cols
Optional list of columns to drop (for drop() operation)

Type
List[str] | None

rename_map
Optional mapping of old column names to new names

Type
Dict[str, str] | None

summarize_aggs
Optional dict of name -> aggregation expression for summarize()

Type
Dict[str, str] | None

distinct_cols
Optional list of columns for distinct operation

Type
List[str] | None

rank_specs
Optional list of (attr, method) tuples for rank_by()

Type
List[Tuple[str, str]] | None

zscore_attrs
Optional list of attributes to compute z-scores for

Type
List[str] | None

post_filters
Optional list of filter specifications to apply after computation

Type
List[Dict[str, Any]] | None

aggregate_specs
Optional dict of name -> aggregation spec for aggregate()

Type
Dict[str, Any] | None

`autocompute`
Whether to automatically compute missing metrics (default: True)

Type
bool

`uq_config`
Optional query-scoped uncertainty quantification configuration

Type
py3plex.dsl.ast.UQConfig | None

`aggregate_specs: Dict[str, Any] | None = None`

`autocompute: bool = True`

`compute: List[ComputeItem]`

`coverage_group: str | None = None`

`coverage_id_field: str = 'id'`

`coverage_k: int | None = None`

`coverage_mode: str | None = None`

`coverage_p: float | None = None`

`distinct_cols: List[str] | None = None`

`drop_cols: List[str] | None = None`

`export: ExportTarget | None = None`

`file_export: ExportSpec | None = None`

`group_by: List[str]`

`layer_expr: LayerExpr | None = None`

`layer_set: Any | None = None`

`limit: int | None = None`

`limit_per_group: int | None = None`

`order_by: List[OrderItem]`

`post_filters: List[Dict[str, Any]] | None = None`

`rank_specs: List[Tuple[str, str]] | None = None`

`rename_map: Dict[str, str] | None = None`

`select_cols: List[str] | None = None`

`summarize_aggs: Dict[str, str] | None = None`

`target: Target`

temporal_context: *TemporalContext* | None = None

uq_config: *UQConfig* | None = None

where: *ConditionExpr* | None = None

window_spec: WindowSpec | None = None

zscore_attrs: List[str] | None = None

class py3plex.dsl.**SpecialPredicate**(*kind: str, params: ~typing.Dict[str, ~typing.Any] = <factory>*)

Bases: object

Special multilayer predicates.

Supported kinds:

- 'intralayer': Edges within the same layer
- 'interlayer': Edges between specific layers
- 'motif': Motif pattern matching
- 'reachable_from': Cross-layer reachability

kind

Predicate type

Type
str

params

Additional parameters for the predicate

Type
Dict[str, Any]

kind: str

params: Dict[str, Any]

class py3plex.dsl.**SuggestedFix**(*replacement: str, span: Tuple[int, int]*)

Bases: object

A suggested fix for a diagnostic.

replacement

The replacement text

Type
str

span

Tuple of (start_index, end_index) in the query string

Type
Tuple[int, int]

replacement: str

span: Tuple[int, int]

class py3plex.dsl.Target(*value*)

Bases: `Enum`

Query target - what to select from the network.

EDGES = 'edges'

NODES = 'nodes'

class py3plex.dsl.TemporalContext(*kind: str, t0: float | None = None, t1: float | None = None, range_name: str | None = None*)

Bases: `object`

Temporal context for time-based queries.

This represents temporal constraints on a query, specified via AT or DURING clauses.

kind

Type of temporal constraint ("at" for point-in-time, "during" for interval)

Type
str

t0

Start time for interval queries (None for point-in-time)

Type
float | None

t1

End time for interval queries (None for point-in-time)

Type
float | None

range_name

Optional named range reference (e.g., "Q1_2023")

Type
str | None

Examples

```
>>> # Point-in-time: AT 1234567890
>>> TemporalContext(kind="at", t0=1234567890.0, t1=1234567890.0)
```

```
>>> # Time range: DURING [100, 200]
>>> TemporalContext(kind="during", t0=100.0, t1=200.0)
```

```
>>> # Named range: DURING RANGE "Q1_2023"
>>> TemporalContext(kind="during", range_name="Q1_2023")
```

kind: str

range_name: str | None = None

t0: float | None = None

t1: float | None = None

class py3plex.dsl.TrajectoriesBuilder(*process_ref: str*)

Bases: `object`

Builder for TRAJECTORIES statements.

Example

```
>>> from py3plex.dsl import Q
>>>
>>> result = (
...     Q.trajectories("sim_result")
...      .where(replicate=5)
...      .at(50)
...      .measure("peak_time", "final_state")
...      .order_by("node_id")
...      .limit(100)
...      .execute()
... )
```

at(*t: float*) → *TrajectoriesBuilder*

Filter to specific time point.

Parameters

t – Timestamp

Returns

Self for chaining

during(*t0: float, t1: float*) → *TrajectoriesBuilder*

Filter to time range.

Parameters

- **t0** – Start time
- **t1** – End time

Returns

Self for chaining

execute(*context: Any | None = None*) → Any

Execute trajectory query.

Parameters

context – Optional context containing simulation results

Returns

QueryResult with trajectory data

limit(*n: int*) → *TrajectoriesBuilder*

Limit number of results.

Parameters

n – Maximum number of results

Returns

Self for chaining

measure(**measures: str*) → *TrajectoriesBuilder*

Add trajectory measures to compute.

Parameters
`*measures` – Measure names

Returns
Self for chaining

order_by(*key: str*, *desc: bool = False*) → *TrajectoriesBuilder*

Add ordering specification.

Parameters

- `key` – Attribute to order by
- `desc` – If True, descending order

Returns
Self for chaining

to(*target: str*) → *TrajectoriesBuilder*

Set export target.

Parameters
`target` – Export format

Returns
Self for chaining

to_ast() → *TrajectoriesStmt*

Export as AST TrajectoriesStmt object.

where(***kwargs*) → *TrajectoriesBuilder*

Add WHERE conditions on trajectories.

Parameters
`**kwargs` – Conditions (e.g., replicate=5, node="Alice")

Returns
Self for chaining

class py3plex.dsl.**TrajectoriesStmt**(*process_ref: str*, *where: ~py3plex.dsl.ast.ConditionExpr | None = None*, *temporal_context: ~py3plex.dsl.ast.TemporalContext | None = None*, *measures: ~typing.List[str] = <factory>*, *order_by: ~typing.List[~py3plex.dsl.ast.OrderItem] = <factory>*, *limit: int | None = None*, *export_target: str | None = None*)

Bases: `object`

TRAJECTORIES statement for querying simulation results.

DSL Example:
TRAJECTORIES FROM process_result WHERE replicate = 5 AT time = 50 MEASURE peak_time, final_state ORDER BY node_id LIMIT 100

process_ref

Reference to a dynamics process or result

Type
str

`where`
Optional WHERE conditions on trajectories

Type
py3plex.dsl.ast.ConditionExpr | None

`temporal_context`
Optional temporal filtering (at specific time, during range)

Type
py3plex.dsl.ast.TemporalContext | None

`measures`
List of trajectory measures to compute

Type
List[str]

`order_by`
List of ordering specifications

Type
List[*py3plex.dsl.ast.OrderItem*]

`limit`
Optional limit on results

Type
int | None

`export_target`
Optional export format

Type
str | None

`export_target: str | None = None`

`limit: int | None = None`

`measures: List[str]`

`order_by: List[OrderItem]`

`process_ref: str`

`temporal_context: TemporalContext | None = None`

`where: ConditionExpr | None = None`

`class py3plex.dsl.TypeEnvironment`
Bases: `object`

Type environment for DSL queries.

Tracks types of attributes, computed values, and other entities in a query context.

`add_layer`(*layer: str*)
Add a layer reference.

get_attribute_type(*name: str*) → *AttrType*

Get the type of an attribute.

get_computed_type(*name: str*) → *AttrType*

Get the type of a computed value.

has_layer(*layer: str*) → bool

Check if a layer is known.

set_attribute_type(*name: str*, *attr_type:* AttrType)

Set the type of an attribute.

set_computed_type(*name: str*, *attr_type:* AttrType)

Set the type of a computed value.

exception py3plex.dsl.TypeMismatchError(*attribute: str*, *expected_type: str*, *actual_type: str*, *query: str | None = None*, *line: int | None = None*, *column: int | None = None*)

Bases: *DslError*

Exception raised when there's a type mismatch.

attribute

The attribute with the type mismatch

expected_type

Expected type

actual_type

Actual type received

class py3plex.dsl.UQ

Bases: `object`

Uncertainty quantification profiles for ergonomic one-liners.

This class provides convenient presets for common uncertainty estimation scenarios. Each profile returns a UQConfig that can be passed to .uq().

Example

```
>>> from py3plex.dsl import Q, UQ
>>>
>>> # Fast exploratory analysis
>>> Q.nodes().uq(UQ.fast(seed=42)).compute("degree").execute(net)
>>>
>>> # Default balanced settings
>>> Q.nodes().uq(UQ.default()).compute("betweenness_centrality").
↪execute(net)
>>>
>>> # Publication-quality with more samples
>>> Q.nodes().uq(UQ.paper(seed=123)).compute("closeness").execute(net)
```

static default(*seed: int | None = None*) → *UQConfig*

Default balanced profile.

Settings: perturbation, n=50, ci=0.95

Use this for general-purpose uncertainty estimation with reasonable computational cost.

Parameters
seed – Random seed for reproducibility (default: None)

Returns
UQConfig with default settings

Example

```
>>> Q.nodes().uq(UQ.default()).compute("betweenness_centrality").
↪execute(net)
```

static fast(*seed: int | None = None*) → *UQConfig*

Fast exploratory profile with minimal samples.

Settings: perturbation, n=25, ci=0.95

Use this for quick exploratory analysis when speed matters more than precision.

Parameters
seed – Random seed for reproducibility (default: None)

Returns
UQConfig with fast settings

Example

```
>>> Q.nodes().uq(UQ.fast(seed=0)).compute("degree").execute(net)
```

static paper(*seed: int | None = None*) → *UQConfig*

Publication-quality profile with thorough sampling.

Settings: bootstrap, n=300, ci=0.95

Use this for publication-quality results where precision is critical and computational cost is acceptable.

Parameters
seed – Random seed for reproducibility (default: None)

Returns
UQConfig with publication-quality settings

Example

```
>>> Q.nodes().uq(UQ.paper(seed=123)).compute("closeness").
↪execute(net)
```

class py3plex.dsl.UQConfig(*method: str | None = None, n_samples: int | None = None, ci: float | None = None, seed: int | None = None, kwargs: ~typing.Dict[str, ~typing.Any] = <factory>*)

Bases: `object`

Query-scoped uncertainty quantification configuration.

This dataclass stores uncertainty estimation settings at the query level, providing defaults for all metrics computed in the query unless explicitly overridden on a per-metric basis.

method

Uncertainty estimation method ('bootstrap', 'perturbation', 'seed', 'null_model')

Type
str | None

n_samples

Number of samples for uncertainty estimation

Type
int | None

ci

Confidence interval level (e.g., 0.95 for 95% CI)

Type
float | None

seed

Random seed for reproducibility

Type
int | None

kwargs

Additional method-specific parameters (e.g., bootstrap_unit, bootstrap_mode)

Type
Dict[str, Any]

Example

```
>>> uq = UQConfig(method="perturbation", n_samples=100, ci=0.95, seed=42)
>>> uq = UQConfig(method="bootstrap", n_samples=200, ci=0.95,
...               kwargs={"bootstrap_unit": "edges", "bootstrap_mode":
↪"resample"})
```

ci: float | None = None

kwargs: Dict[str, Any]

method: str | None = None

n_samples: int | None = None

seed: int | None = None

exception py3plex.dsl.UnknownAttributeError(*attribute: str, known_attributes: List[str] | None = None, query: str | None = None, line: int | None = None, column: int | None = None*)

Bases: *DslError*

Exception raised when an unknown attribute is referenced.

attribute

The unknown attribute name

known_attributes
List of valid attribute names

suggestion
Suggested alternative, if any

exception py3plex.dsl.UnknownLayerError(*layer: str*, *known_layers: List[str] | None = None*, *query: str | None = None*, *line: int | None = None*, *column: int | None = None*)

Bases: *DslError*

Exception raised when an unknown layer is referenced.

layer
The unknown layer name

known_layers
List of valid layer names

suggestion
Suggested alternative, if any

exception py3plex.dsl.UnknownMeasureError(*measure: str*, *known_measures: List[str] | None = None*, *query: str | None = None*, *line: int | None = None*, *column: int | None = None*)

Bases: *DslError*

Exception raised when an unknown measure is referenced.

measure
The unknown measure name

known_measures
List of valid measure names

suggestion
Suggested alternative, if any

py3plex.dsl.compile_pattern(*pattern:* PatternGraph) → *PatternPlan*

Compile a pattern graph into an execution plan.

The compilation strategy: 1. Select root variable with most restrictive predicates 2. Build join order using a greedy approach (expand along edges) 3. Estimate selectivity for each variable

Parameters
pattern – Pattern graph to compile

Returns
PatternPlan with execution strategy

py3plex.dsl.compute_centrality_for_layer(*network: Any*, *layer: str*, *centrality: str = 'betweenness_centrality'*) → Dict[Any, float]

Compute centrality for all nodes in a layer.

Parameters

- **network** – Multilayer network object

- **layer** – Layer identifier
- **centrality** – Centrality measure name

Returns
Dictionary mapping nodes to centrality values

`py3plex.dsl.describe_operator(name: str)` → Dict[str, Any] | None

Get detailed information about a registered operator.

Parameters
name – Operator name

Returns
Dictionary with operator metadata, or None if not found

Example

```
>>> info = describe_operator("layer_resilience")
>>> print(info["description"])
Compute resilience score for current layers.
```

`py3plex.dsl.detect_communities(network: Any, layer: str | None = None)` → Dict[str, Any]

Detect communities in the network using Louvain algorithm via DSL.

Parameters

- **network** – Multilayer network object
- **layer** – Optional layer to filter nodes by

Returns

- 'partition': Dict mapping nodes to community IDs
- 'num_communities': Number of communities detected
- 'community_sizes': Dict mapping community ID to size
- 'biggest_community': Tuple (community_id, size)
- 'smallest_community': Tuple (community_id, size)
- 'size_distribution': List of community sizes

Return type
Dictionary containing

Example

```
>>> from py3plex.core import multinet
>>> from py3plex.dsl import detect_communities
>>>
>>> # Create a sample network
>>> network = multinet.multi_layer_network()
>>> # ... add nodes and edges ...
>>>
>>> # Detect communities
>>> result = detect_communities(network)
```

(continues on next page)

(continued from previous page)

```
>>> print(f"Found {result['num_communities']} communities")
>>> print(f"Biggest community has {result['biggest_community'][1]} nodes
↪")
```

py3plex.dsl.**dsl_operator**(*name: str | None = None*, ***, *description: str | None = None*, *category: str | None = None*, *overwrite: bool = False*)

Decorator to register a Python function as a DSL operator.

This decorator allows users to define custom operators that can be used in DSL queries. The decorated function should accept a DSLExecutionContext as its first argument, followed by any keyword arguments.

Parameters

- **name** – Operator name (defaults to function name if not provided)
- **description** – Human-readable description (defaults to function docstring)
- **category** – Optional category for organization (e.g., "centrality", "dynamics")
- **overwrite** – If True, allow replacing existing operators

Returns

Decorator function that registers the operator

Example

```
>>> @dsl_operator("layer_resilience", category="dynamics")
... def layer_resilience_op(context: DSLExecutionContext, alpha: float =␣
↪0.1):
...     '''Compute resilience score for current layers.'''
...     # Access context.graph, context.current_layers, etc.
...     return 42.0
```

```
>>> # Use in DSL:
>>> # measure layer_resilience(alpha=0.2) on layers ["infra", "power"]
```

py3plex.dsl.**execute_ast**(*network: Any*, *query: Query*, *params: Dict[str, Any] | None = None*) → *QueryResult* | *ExecutionPlan*

Execute an AST query on a multilayer network.

Parameters

- **network** – Multilayer network object
- **query** – Query AST
- **params** – Parameter bindings

Returns

QueryResult or ExecutionPlan (if explain=True)

py3plex.dsl.**execute_query**(*network: Any*, *query: str*) → Dict[str, Any]

Execute a DSL query on a multilayer network.

Supports both SELECT and MATCH queries:

SELECT queries:
SELECT nodes WHERE layer="transport" AND degree > 5 SELECT * FROM nodes IN LAYER 'ppi' WHERE degree > 10 SELECT id, degree FROM nodes IN LAYERS ('ppi', 'coexpr') WHERE color = 'red'

MATCH queries (Cypher-like):
MATCH (g:Gene)-[r:REGULATES]->(t:Gene) IN LAYER 'reg' WHERE g.degree > 10 RETURN g, t

Parameters

- **network** – Multilayer network object (multi_layer_network instance)
- **query** – DSL query string

Returns

- 'nodes' or 'edges' or 'bindings': List of selected items / pattern matches
- 'computed': Dictionary of computed measures (if COMPUTE used)
- 'query': Original query string

Return type
Dictionary containing

Raises

- *DSLSyntaxError* – If query syntax is invalid
- *DSLExecutionError* – If query cannot be executed

Examples

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network()
>>> net.add_nodes([{'source': 'A', 'type': 'transport'}])
>>> net.add_nodes([{'source': 'B', 'type': 'transport'}])
>>> net.add_nodes([{'source': 'C', 'type': 'social'}])
>>> net.add_edges([
...     {'source': 'A', 'target': 'B', 'source_type': 'transport',
↪'target_type': 'transport'},
...     {'source': 'B', 'target': 'C', 'source_type': 'social', 'target_
↪type': 'social'}
... ])
>>>
>>> # Select all nodes in "transport" layer
>>> result = execute_query(net, 'SELECT nodes WHERE layer="transport"')
>>> result['count'] >= 0
True
>>>
>>> # Select high-degree nodes and compute centrality
>>> result = execute_query(net, 'SELECT nodes WHERE degree > 0 COMPUTE␣
↪betweenness_centrality')
>>> 'computed' in result
True
>>>
```

(continues on next page)

(continued from previous page)

```
>>> # Complex query with multiple conditions
>>> result = execute_query(net, 'SELECT nodes WHERE layer="social" AND␣
↪degree >= 0')
>>> result['count'] >= 0
True
```

py3plex.dsl.explain(*query: Query*, *graph: Any | None = None*, *schema: SchemaProvider | None = None*) → *ExplainResult*

Explain a DSL query with type information and cost estimates.

Provides detailed information about: - Query structure (AST) - Inferred types for all expressions - Estimated execution cost - Potential issues (via linting)

Parameters

- **query** – Query AST to explain
- **graph** – Optional py3plex network for schema extraction
- **schema** – Optional schema provider

Returns

ExplainResult with detailed query information

Example

```
>>> result = explain(query, graph=network)
>>> print(result.ast_summary)
>>> print(f"Cost: {result.cost_estimate}")
>>> for diag in result.diagnostics:
...     print(diag)
```

py3plex.dsl.export_result(*result: Any*, *spec: ExportSpec*) → None

Export query result to a file according to the export specification.

This is the main entry point for file exports. It normalizes the result into a tabular format and dispatches to format-specific writers.

Parameters

- **result** – Query result (QueryResult, dict, or other supported type)
- **spec** – Export specification with path, format, columns, and options

Raises

DslExecutionError – If export fails or result type is not supported

py3plex.dsl.format_result(*result: Dict[str, Any]*, *limit: int = 10*) → str

Format query result as human-readable string.

Parameters

- **result** – Result dictionary from execute_query
- **limit** – Maximum number of items to display

Returns

Formatted string representation

py3plex.dsl.get_biggest_community(*network: Any*, *layer: str | None = None*) → Tuple[int, int, List[Any]]

Get the largest community in the network.

Parameters

- **network** – Multilayer network object
- **layer** – Optional layer to filter nodes by

Returns

Tuple of (community_id, size, list_of_nodes)

Example

```
>>> community_id, size, nodes = get_biggest_community(network)
>>> print(f"Community {community_id} has {size} nodes")
>>> print(f"Nodes: {nodes}")
```

py3plex.dsl.**get_community_partition**(*network: Any, layer: str | None = None*) → Dict[Any, int]

Get community partition (mapping of nodes to community IDs).

Parameters

- **network** – Multilayer network object
- **layer** – Optional layer to filter nodes by

Returns

Dictionary mapping nodes to community IDs

Example

```
>>> partition = get_community_partition(network)
>>> for node, community_id in partition.items():
...     print(f"Node {node} is in community {community_id}")
```

py3plex.dsl.**get_community_size_distribution**(*network: Any, layer: str | None = None*) → List[int]

Get the distribution of community sizes.

Parameters

- **network** – Multilayer network object
- **layer** – Optional layer to filter nodes by

Returns

List of community sizes sorted in descending order

Example

```
>>> distribution = get_community_size_distribution(network)
>>> print(f"Largest community: {distribution[0]} nodes")
>>> print(f"Smallest community: {distribution[-1]} nodes")
>>> print(f"Average size: {sum(distribution) / len(distribution):.1f}")
```

py3plex.dsl.**get_community_sizes**(*network: Any, layer: str | None = None*) → Dict[int, int]

Get the size of each community in the network.

Parameters

- **network** – Multilayer network object
- **layer** – Optional layer to filter nodes by

Returns

Dictionary mapping community ID to its size

Example

```
>>> sizes = get_community_sizes(network)
>>> for comm_id, size in sizes.items():
...     print(f"Community {comm_id}: {size} nodes")
```

`py3plex.dsl.get_num_communities`(*network: Any*, *layer: str | None = None*) → int

Get the number of communities in the network.

Parameters

- **network** – Multilayer network object
- **layer** – Optional layer to filter nodes by

Returns

Number of communities detected

Example

```
>>> num_communities = get_num_communities(network)
>>> print(f"Found {num_communities} communities")
```

`py3plex.dsl.get_operator`(*name: str*) → *DSLOperator* | None

Get a registered operator by name from the global registry.

This is a convenience function that delegates to operator_registry.get().

Parameters

name – Operator name (will be normalized)

Returns

DSLOperator instance or None if not found

`py3plex.dsl.get_smallest_community`(*network: Any*, *layer: str | None = None*) → Tuple[int, int, List[Any]]

Get the smallest community in the network.

Parameters

- **network** – Multilayer network object
- **layer** – Optional layer to filter nodes by

Returns

Tuple of (community_id, size, list_of_nodes)

Example

```
>>> community_id, size, nodes = get_smallest_community(network)
>>> print(f"Community {community_id} has {size} nodes")
>>> print(f"Nodes: {nodes}")
```

`py3plex.dsl.lint`(*query: Query*, *graph: Any | None = None*, *schema: SchemaProvider | None = None*) → List[*Diagnostic*]

Lint a DSL query.

Analyzes the query for potential issues including: - Unknown layers or attributes - Type mismatches - Unsatisfiable or redundant predicates - Performance issues

Parameters

- **query** – Query AST to lint
- **graph** – Optional py3plex network for schema extraction
- **schema** – Optional schema provider (auto-created from graph if not provided)

Returns

List of diagnostics found

Example

```
>>> from py3plex.dsl import Q, L, lint
>>> from py3plex.core import multinet
>>>
>>> network = multinet.multi_layer_network()
>>> # ... build network ...
>>>
>>> query = Q.nodes().from_layers(L["social"]).where(degree__gt=5).build()
>>> diagnostics = lint(query, graph=network)
>>>
>>> for diag in diagnostics:
...     print(f"{diag.severity}: {diag.message}")
```

`py3plex.dsl.list_operators`() → Dict[str, *DSLOperator*]

List all registered operators from the global registry.

This is a convenience function that delegates to operator_registry.list_operators().

Returns

Dictionary mapping operator names to DSLOperator instances

`py3plex.dsl.match_pattern`(*network: Any*, *pattern: PatternGraph*, *plan: PatternPlan*, *limit: int | None = None*, *timeout: float | None = None*) → List[*MatchRow*]

Execute pattern matching on a network.

Parameters

- **network** – Multilayer network object
- **pattern** – Pattern graph to match
- **plan** – Compiled execution plan

- **limit** – Maximum number of matches to return
- **timeout** – Optional timeout in seconds

Returns
List of MatchRow objects representing matches

`py3plex.dsl.register_operator(`*name: str*, *func: Callable[[...], Any]*, *description: str | None = None*, *category: str | None = None*, *overwrite: bool = False*`)` → None

Register a DSL operator with the global registry.

This is a convenience function that delegates to operator_registry.register().

Parameters
- **name** – Operator name (will be normalized)
- **func** – Python callable implementing the operator
- **description** – Optional description
- **category** – Optional category for organization
- **overwrite** – If True, allow replacing existing operators

`py3plex.dsl.select_high_degree_nodes(`*network: Any*, *min_degree: int*, *layer: str | None = None*`)` → List[Any]

Select nodes with degree greater than threshold.

Parameters
- **network** – Multilayer network object
- **min_degree** – Minimum degree threshold (exclusive - nodes must have degree > min_degree)
- **layer** – Optional layer to filter by

Returns
List of nodes with degree > min_degree

`py3plex.dsl.select_nodes_by_layer(`*network: Any*, *layer: str*`)` → List[Any]

Select all nodes in a specific layer.

Parameters
- **network** – Multilayer network object
- **layer** – Layer identifier

Returns
List of nodes in the specified layer

`py3plex.dsl.unregister_operator(`*name: str*`)` → None

Unregister an operator from the global registry.

This is a convenience function that delegates to operator_registry.unregister(). Primarily useful for testing and cleanup.

Parameters
name – Operator name (will be normalized)

Uncertainty Quantification

First-class uncertainty support for py3plex.

This module provides types and utilities for representing statistics with uncertainty information. The core idea is that every statistic is an object that can carry a distribution:

- Deterministic mode: object has mean with std=None (certainty 1.0)
- Uncertainty mode: object has mean, std, quantiles populated

This makes uncertainty "first-class" - it's baked into how numbers exist in the library, not bolted on later.

Examples

```
>>> from py3plex.uncertainty import StatSeries
>>> # Deterministic result
>>> result = StatSeries(
...     index=['a', 'b', 'c'],
...     mean=np.array([1.0, 2.0, 3.0])
... )
>>> result.is_deterministic
True
>>> result.certainty
1.0
```

```
>>> # Uncertain result
>>> result_unc = StatSeries(
...     index=['a', 'b', 'c'],
...     mean=np.array([1.0, 2.0, 3.0]),
...     std=np.array([0.1, 0.2, 0.15]),
...     quantiles={
...         0.025: np.array([0.8, 1.6, 2.7]),
...         0.975: np.array([1.2, 2.4, 3.3])
...     }
... )
>>> result_unc.is_deterministic
False
>>> np.array(result_unc)  # Backward compat - gives mean
array([1., 2., 3.])
```

class py3plex.uncertainty.CommunityStats(*labels: ~typing.Dict[~typing.Any, int], modularity: float | None = None, modularity_std: float | None = None, coassoc: ~py3plex.uncertainty.types.StatMatrix | None = None, stability: ~typing.Dict[~typing.Any, float] | None = None, n_communities: int = 0, meta: ~typing.Dict[str, ~typing.Any] = <factory>*)

Bases: `object`

Statistics from community detection with optional uncertainty.

Wraps cluster labels, modularity, co-association matrix, and stability indices computed from multiple runs.

Parameters

- **labels** (*dict [Any , int]*) – Node -> community ID mapping (from deterministic run or consensus).
- **modularity** (*float or None*) – Modularity score (mean if multiple runs).
- **modularity_std** (*float or None*) – Standard deviation of modularity across runs.
- **coassoc** (StatMatrix *or None*) – Co-association matrix (probability nodes are in same community).
- **stability** (*dict [Any , float] or None*) – Per-node stability index (how often node stays in same cluster).
- **n_communities** (*int*) – Number of communities detected.
- **meta** (*dict [str , Any]*) – Optional metadata.

Examples

```
>>> cs = CommunityStats(
...     labels={'a': 0, 'b': 0, 'c': 1},
...     modularity=0.42,
...     n_communities=2
... )
>>> cs.is_deterministic
True
>>> cs.labels['a']
0
```

property certainty: float

Return certainty level (1.0 if deterministic, 0.0 otherwise).

coassoc: *StatMatrix* | None = None

property is_deterministic: bool

Return True if no uncertainty info is present.

labels: Dict[Any, int]

meta: Dict[str, Any]

modularity: float | None = None

modularity_std: float | None = None

n_communities: int = 0

stability: Dict[Any, float] | None = None

class py3plex.uncertainty.ResamplingStrategy(*value*)

Bases: Enum

Strategy for estimating uncertainty via resampling.

`SEED`

Run with different random seeds (Monte Carlo).

Type
str

`BOOTSTRAP`

Bootstrap resampling of nodes or edges.

Type
str

`JACKKNIFE`

Leave-one-out jackknife resampling.

Type
str

`PERTURBATION`

Add noise/perturbations to network structure or parameters.

Type
str

`BOOTSTRAP = 'bootstrap'`

`JACKKNIFE = 'jackknife'`

`PERTURBATION = 'perturbation'`

`SEED = 'seed'`

`class py3plex.uncertainty.StatMatrix(`*index: ~typing.List[~typing.Any], mean: ~numpy.ndarray, std: ~numpy.ndarray | None = None, quantiles: ~typing.Dict[float, ~numpy.ndarray] | None = None, meta: ~typing.Dict[str, ~typing.Any] = <factory>*`)`

Bases: `object`

A matrix of statistics with optional uncertainty.

Used for adjacency matrices, co-association matrices, distance matrices, etc.

Parameters

- **index** (*list [Any]*) – Row/column labels (assumed square matrix for simplicity).
- **mean** (*np.ndarray*) – The mean matrix, shape (n, n).
- **std** (*np.ndarray or None*) – The standard deviation matrix, shape (n, n), or None.
- **quantiles** (*dict [float , np.ndarray] or None*) – Quantile matrices, e.g., {0.025: (n, n), 0.975: (n, n)}.
- **meta** (*dict [str , Any]*) – Optional metadata.

Examples

```
>>> import numpy as np
>>> m = StatMatrix(
...     index=['a', 'b', 'c'],
...     mean=np.array([[0, 1, 0], [1, 0, 1], [0, 1, 0]], dtype=float)
... )
>>> m.is_deterministic
True
>>> np.array(m).shape
(3, 3)
```

property certainty: float

Return certainty level (1.0 if deterministic, 0.0 otherwise).

index: List[Any]

property is_deterministic: bool

Return True if this is a deterministic result.

mean: ndarray

meta: Dict[str, Any]

quantiles: Dict[float, ndarray] | None = None

std: ndarray | None = None

class py3plex.uncertainty.StatSeries(*index: ~typing.List[~typing.Any], mean: ~numpy.ndarray, std: ~numpy.ndarray | None = None, quantiles: ~typing.Dict[float, ~numpy.ndarray] | None = None, meta: ~typing.Dict[str, ~typing.Any] = <factory>*)

Bases: `object`

A series of statistics with optional uncertainty information.

This is the canonical result type for statistics that return a value per node, time point, or other index.

In deterministic mode (uncertainty=False): - mean contains the single-run values - std = None - quantiles = None - certainty = 1.0

In uncertain mode (uncertainty=True): - mean contains the average across runs - std contains the standard deviation - quantiles contains percentile arrays (e.g., {0.025: arr, 0.975: arr}) - certainty < 1.0

Parameters

- **index** (*list[Any]*) – The index labels (e.g., node IDs, time points).
- **mean** (*np.ndarray*) – The mean values, shape (n,).
- **std** (*np.ndarray or None*) – The standard deviations, shape (n,), or None if deterministic.
- **quantiles** (*dict[float, np.ndarray] or None*) – Quantile arrays, e.g., {0.025: (n,), 0.975: (n,)}, or None.
- **meta** (*dict[str, Any]*) – Optional metadata (e.g., algorithm param-

eters, run info).

Examples

```
>>> import numpy as np
>>> # Deterministic
>>> s = StatSeries(
...     index=['a', 'b', 'c'],
...     mean=np.array([1.0, 2.0, 3.0])
... )
>>> s.is_deterministic
True
>>> s.certainty
1.0
>>> np.array(s)
array([1., 2., 3.])
```

```
>>> # With uncertainty
>>> s_unc = StatSeries(
...     index=['a', 'b', 'c'],
...     mean=np.array([1.0, 2.0, 3.0]),
...     std=np.array([0.1, 0.2, 0.15]),
...     quantiles={0.025: np.array([0.8, 1.6, 2.7]),
...                0.975: np.array([1.2, 2.4, 3.3])}
... )
>>> s_unc.is_deterministic
False
>>> s_unc.certainty
0.0
```

property certainty: float

Return certainty level.

Returns 1.0 if deterministic, 0.0 otherwise. In the future, this could return a richer metric.

index: List[Any]

property is_deterministic: bool

Return True if this is a deterministic result (no uncertainty).

mean: ndarray

meta: Dict[str, Any]

quantiles: Dict[float, ndarray] | None = None

std: ndarray | None = None

to_dict() → Dict[Any, Dict[str, Any]]

Convert to dictionary mapping index -> stats dict.

Returns

Dictionary with keys from index, values are dicts with 'mean', optionally 'std' and 'quantiles'.

Return type
dict

`class py3plex.uncertainty.UncertaintyConfig(`*mode: UncertaintyMode = UncertaintyMode.OFF, default_n_runs: int = 50, default_resampling: ResamplingStrategy = ResamplingStrategy.SEED*`)`

Bases: `object`

Configuration for uncertainty estimation.

`mode`

The current uncertainty mode.

Type
UncertaintyMode

`default_n_runs`

Default number of runs for uncertainty estimation.

Type
int

`default_resampling`

Default resampling strategy.

Type
ResamplingStrategy

`default_n_runs: int = 50`

`default_resampling: ResamplingStrategy = 'seed'`

`mode: UncertaintyMode = 'off'`

`class py3plex.uncertainty.UncertaintyMode(`*value*`)`

Bases: `Enum`

Global mode for uncertainty computation.

`OFF`

Always deterministic, std=None.

Type
str

`ON`

Try to compute uncertainty when supported.

Type
str

`AUTO`

Only do it if explicitly requested by a function.

Type
str

`AUTO = 'auto'`

OFF = 'off'

ON = 'on'

py3plex.uncertainty.bootstrap_metric(*graph: multi_layer_network*, *metric_fn: Callable[[multi_layer_network], Dict[Any, float]]*, *n_boot: int = 50*, *unit: str = 'edges'*, *mode: str = 'resample'*, *ci: float = 0.95*, *random_state: int | None = None*) → Dict[str, ndarray]

Bootstrap a metric for uncertainty estimation.

This function resamples the graph by unit (edges, nodes, or layers) and recomputes the metric on each bootstrap sample to estimate uncertainty.

Parameters

- **graph** (multi_layer_network) – The multilayer network to analyze.
- **metric_fn** (*callable*) – Function that takes a network and returns a dict mapping items (usually nodes) to metric values. Must have signature: metric_fn(network) -> Dict[item_id, float]
- **n_boot** (*int, default=200*) – Number of bootstrap replicates.
- **unit** (*str, default="edges"*) – What to resample: "edges", "nodes", or "layers".
- **mode** (*str, default="resample"*) – Resampling mode: - "resample": Sample with replacement (classic bootstrap) - "permute": Permute the units (permutation test)
- **ci** (*float, default=0.95*) – Confidence interval level (e.g., 0.95 for 95% CI).
- **random_state** (*int, optional*) – Random seed for reproducibility.

Returns

Dictionary with keys: - "mean": np.ndarray of shape (n_items,) with mean values - "std": np.ndarray of shape (n_items,) with standard errors - "ci_low": np.ndarray of shape (n_items,) with lower CI bounds - "ci_high": np.ndarray of shape (n_items,) with upper CI bounds - "index": List of item IDs - "n_boot": Number of bootstrap replicates used - "method": String describing the bootstrap method

Return type

dict

Examples

```
>>> from py3plex.core import multinet
>>> from py3plex.uncertainty.bootstrap import bootstrap_metric
>>>
>>> # Create a network
>>> net = multinet.multi_layer_network(directed=False)
>>> net.add_edges([["a", "L0", "b", "L0", 1.0]], input_type="list")
>>>
>>> # Define metric function
>>> def degree_metric(network):
```

(continues on next page)

(continued from previous page)

```
...     result = {}
...     for node in network.get_nodes():
...         result[node] = network.core_network.degree(node)
...     return result
>>>
>>> # Bootstrap
>>> boot_result = bootstrap_metric(
...     net, degree_metric, n_boot=100, unit="edges"
... )
>>> boot_result["mean"]  # Mean degree values
>>> boot_result["ci_low"]  # Lower CI bounds
```

Notes

- For "edges" unit: resamples edges with replacement
- For "nodes" unit: resamples nodes with replacement
- For "layers" unit: resamples layers with replacement
- The metric_fn must be able to handle graphs with different structure

`py3plex.uncertainty.estimate_uncertainty`(*network: multi_layer_network*, *metric_fn: Callable[[multi_layer_network], Dict[Any, float] | float | ndarray]*, ***, *n_runs: int | None = None*, *resampling: ResamplingStrategy | None = None*, *random_seed: int | None = None*, *perturbation_params: Dict[str, Any] | None = None*) → *StatSeries* | float

Estimate uncertainty for a network statistic.

This is the main entry point for adding uncertainty to any statistic. It runs the metric function multiple times with different random seeds or network perturbations, then computes mean, std, and quantiles.

Parameters

- **network** (multi_layer_network) – The network to analyze.
- **metric_fn** (*callable*) – Function that computes a statistic. Must accept a network and return: - dict[node, float] for per-node statistics - float for scalar statistics - np.ndarray for array statistics
- **n_runs** (*int, optional*) – Number of runs for uncertainty estimation. If None, uses the default from the current uncertainty config.
- **resampling** (ResamplingStrategy*, optional*) – Strategy for resampling. If None, uses default from config.
- **random_seed** (*int, optional*) – Random seed for reproducibility.
- **perturbation_params** (*dict, optional*) – Parameters for perturbation strategies. For example: {"edge_drop_p": 0.05, "node_drop_p": 0.02}

Returns

If metric_fn returns a dict: StatSeries with uncertainty info If metric_fn returns a scalar: float (mean value) The returned object has mean, std, and quantiles populated.

Return type
StatSeries or float

Examples

```
>>> from py3plex.uncertainty import estimate_uncertainty,␣
↪ResamplingStrategy
>>> from py3plex.core import multinet
>>>
>>> # Create a simple network
>>> net = multinet.multi_layer_network(directed=False)
>>> net.add_edges([["a", "L0", "b", "L0", 1.0]], input_type="list")
>>>
>>> # Define a metric function
>>> def my_metric(network):
...     # Return per-node degree
...     degrees = {}
...     for node in network.get_nodes():
...         degrees[node] = network.core_network.degree(node)
...     return degrees
>>>
>>> # Estimate uncertainty
>>> result = estimate_uncertainty(
...     net,
...     my_metric,
...     n_runs=50,
...     resampling=ResamplingStrategy.PERTURBATION,
...     perturbation_params={"edge_drop_p": 0.1}
... )
>>> result.mean  # Mean degree values
>>> result.std   # Std deviation of degrees
>>> result.quantiles  # Confidence intervals
```

Notes

- For SEED strategy: runs the metric with different random seeds
- For PERTURBATION strategy: applies edge/node drops then recomputes
- For BOOTSTRAP: resamples nodes/edges with replacement (not yet implemented)
- For JACKKNIFE: leave-one-out resampling (not yet implemented)

`py3plex.uncertainty.get_uncertainty_config()` → *UncertaintyConfig*

Get the current uncertainty configuration.

Returns
The current configuration from the context.

Return type
UncertaintyConfig

Examples

```
>>> from py3plex.uncertainty import get_uncertainty_config
>>> cfg = get_uncertainty_config()
>>> cfg.mode
<UncertaintyMode.OFF: 'off'>
>>> cfg.default_n_runs
50
```

py3plex.uncertainty.**null_model_metric**(*graph: multi_layer_network*, *metric_fn: Callable[[multi_layer_network], Dict[Any, float]]*, *n_null: int = 200*, *model: str = 'degree_preserving'*, *random_state: int | None = None*) → Dict[str, ndarray]

Compute metric on null models for statistical significance testing.

This function generates null models of the network (e.g., via degree-preserving rewiring) and computes the metric on each null network. It then computes z-scores and p-values for the observed metric values.

Parameters

- **graph** (multi_layer_network) – The multilayer network to analyze.
- **metric_fn** (*callable*) – Function that takes a network and returns a dict mapping items (usually nodes) to metric values. Must have signature: metric_fn(network) -> Dict[item_id, float]
- **n_null** (*int, default=200*) – Number of null model replicates to generate.
- **model** (*str, default="degree_preserving"*) – Null model type: - "degree_preserving": Rewire edges while preserving degree sequence - "erdos_renyi": Random graph with same density - "configuration": Configuration model matching degree distribution
- **random_state** (*int, optional*) – Random seed for reproducibility.

Returns

Dictionary with keys: - "observed": np.ndarray of shape (n_items,) with observed metric values - "mean_null": np.ndarray of shape (n_items,) with mean null values - "std_null": np.ndarray of shape (n_items,) with std of null values - "zscore": np.ndarray of shape (n_items,) with z-scores - "pvalue": np.ndarray of shape (n_items,) with two-tailed p-values - "index": List of item IDs - "n_null": Number of null replicates used - "model": String describing the null model

Return type

dict

Examples

```
>>> from py3plex.core import multinet
>>> from py3plex.uncertainty.null_models import null_model_metric
>>>
>>> # Create a network
```

(continues on next page)

(continued from previous page)

```
>>> net = multinet.multi_layer_network(directed=False)
>>> net.add_edges([["a", "L0", "b", "L0", 1.0]], input_type="list")
>>>
>>> # Define metric function
>>> def degree_metric(network):
...     result = {}
...     for node in network.get_nodes():
...         result[node] = network.core_network.degree(node)
...     return result
>>>
>>> # Compute null model statistics
>>> null_result = null_model_metric(
...     net, degree_metric, n_null=100, model="degree_preserving"
... )
>>> null_result["zscore"]  # Z-scores for each node
>>> null_result["pvalue"]  # P-values for each node
```

Notes

- Z-scores indicate how many standard deviations the observed value is from the null distribution mean
- P-values are two-tailed by default: P(|Z| >= |Z_observed|) under the null. For one-tailed tests, users can compute p_one_sided = p_two_sided / 2 and check the sign of z-score to determine the direction.
- High |z-score| and low p-value indicate statistical significance

`py3plex.uncertainty.set_uncertainty_config`(*config: UncertaintyConfig*) → Token

Set the uncertainty configuration.

Parameters

`config` (`UncertaintyConfig`) – The new configuration to set.

Returns

A token that can be used to reset the configuration.

Return type

Token

Examples

```
>>> from py3plex.uncertainty import set_uncertainty_config,␣
↪UncertaintyConfig
>>> from py3plex.uncertainty import UncertaintyMode
>>> cfg = UncertaintyConfig(mode=UncertaintyMode.ON, default_n_runs=100)
>>> token = set_uncertainty_config(cfg)
>>> # ... do work ...
>>> _uncertainty_ctx.reset(token)  # restore previous config
```

`py3plex.uncertainty.uncertainty_enabled`(**, n_runs: int | None = None, resampling: ResamplingStrategy | None = None*)

Context manager to enable uncertainty estimation.

Within this context, all supported functions will compute uncertainty by default (unless explicitly disabled with uncertainty=False).

Parameters

- **n_runs** (*int , optional*) – Number of runs for uncertainty estimation. If None, uses the default from the current config.
- **resampling** (*ResamplingStrategy, optional*) – Resampling strategy to use. If None, uses the default from the current config.

Yields

None

Examples

```
>>> from py3plex.uncertainty import uncertainty_enabled
>>> from py3plex.algorithms.centrality_toolkit import multilayer_pagerank
>>>
>>> # Without uncertainty
>>> result = multilayer_pagerank(network)
>>> result.is_deterministic
True
>>>
>>> # With uncertainty
>>> with uncertainty_enabled(n_runs=100):
...     result = multilayer_pagerank(network)
>>> result.is_deterministic
False
>>> result.std is not None
True
```

Notes

This uses contextvars, so it's thread-safe and async-safe. Each context gets its own configuration.

Core statistic types for first-class uncertainty representation.

This module defines the fundamental types that wrap statistics with optional uncertainty information (standard deviations, confidence intervals, quantiles).

`class py3plex.uncertainty.types.CommunityStats`(*labels: ~typing.Dict[~typing.Any, int], modularity: float | None = None, modularity_std: float | None = None, coassoc: ~py3plex.uncertainty.types.StatMatrix | None = None, stability: ~typing.Dict[~typing.Any, float] | None = None, n_communities: int = 0, meta: ~typing.Dict[str, ~typing.Any] = <factory>*)

Bases: `object`

Statistics from community detection with optional uncertainty.

Wraps cluster labels, modularity, co-association matrix, and stability indices computed from multiple runs.

Parameters

- **labels** (*dict [Any , int]*) – Node -> community ID mapping (from deterministic run or consensus).
- **modularity** (*float or None*) – Modularity score (mean if multiple runs).
- **modularity_std** (*float or None*) – Standard deviation of modularity across runs.
- **coassoc** (StatMatrix *or None*) – Co-association matrix (probability nodes are in same community).
- **stability** (*dict [Any , float] or None*) – Per-node stability index (how often node stays in same cluster).
- **n_communities** (*int*) – Number of communities detected.
- **meta** (*dict [str , Any]*) – Optional metadata.

Examples

```
>>> cs = CommunityStats(
...     labels={'a': 0, 'b': 0, 'c': 1},
...     modularity=0.42,
...     n_communities=2
... )
>>> cs.is_deterministic
True
>>> cs.labels['a']
0
```

property certainty: float

Return certainty level (1.0 if deterministic, 0.0 otherwise).

coassoc: ***StatMatrix*** **| None = None**

property is_deterministic: bool

Return True if no uncertainty info is present.

labels: Dict[Any, int]

meta: Dict[str, Any]

modularity: float | None = None

modularity_std: float | None = None

n_communities: int = 0

stability: Dict[Any, float] | None = None

class py3plex.uncertainty.types.ResamplingStrategy(*value*)

Bases: Enum

Strategy for estimating uncertainty via resampling.

`SEED`

Run with different random seeds (Monte Carlo).

Type
str

`BOOTSTRAP`

Bootstrap resampling of nodes or edges.

Type
str

`JACKKNIFE`

Leave-one-out jackknife resampling.

Type
str

`PERTURBATION`

Add noise/perturbations to network structure or parameters.

Type
str

`BOOTSTRAP = 'bootstrap'`

`JACKKNIFE = 'jackknife'`

`PERTURBATION = 'perturbation'`

`SEED = 'seed'`

`class py3plex.uncertainty.types.StatMatrix`(*index: ~typing.List[~typing.Any], mean: ~numpy.ndarray, std: ~numpy.ndarray | None = None, quantiles: ~typing.Dict[float, ~numpy.ndarray] | None = None, meta: ~typing.Dict[str, ~typing.Any] = <factory>*)

Bases: `object`

A matrix of statistics with optional uncertainty.

Used for adjacency matrices, co-association matrices, distance matrices, etc.

Parameters

- **index** (*list [Any]*) – Row/column labels (assumed square matrix for simplicity).
- **mean** (*np.ndarray*) – The mean matrix, shape (n, n).
- **std** (*np.ndarray or None*) – The standard deviation matrix, shape (n, n), or None.
- **quantiles** (*dict [float , np.ndarray] or None*) – Quantile matrices, e.g., {0.025: (n, n), 0.975: (n, n)}.
- **meta** (*dict [str , Any]*) – Optional metadata.

Examples

```
>>> import numpy as np
>>> m = StatMatrix(
...     index=['a', 'b', 'c'],
...     mean=np.array([[0, 1, 0], [1, 0, 1], [0, 1, 0]], dtype=float)
... )
>>> m.is_deterministic
True
>>> np.array(m).shape
(3, 3)
```

property certainty: float

Return certainty level (1.0 if deterministic, 0.0 otherwise).

index: List[Any]

property is_deterministic: bool

Return True if this is a deterministic result.

mean: ndarray

meta: Dict[str, Any]

quantiles: Dict[float, ndarray] | None = None

std: ndarray | None = None

class py3plex.uncertainty.types.StatSeries(*index: ~typing.List[~typing.Any], mean: ~numpy.ndarray, std: ~numpy.ndarray | None = None, quantiles: ~typing.Dict[float, ~numpy.ndarray] | None = None, meta: ~typing.Dict[str, ~typing.Any] = <factory>*)

Bases: `object`

A series of statistics with optional uncertainty information.

This is the canonical result type for statistics that return a value per node, time point, or other index.

In deterministic mode (uncertainty=False): - mean contains the single-run values - std = None - quantiles = None - certainty = 1.0

In uncertain mode (uncertainty=True): - mean contains the average across runs - std contains the standard deviation - quantiles contains percentile arrays (e.g., {0.025: arr, 0.975: arr}) - certainty < 1.0

Parameters

- **index** (*list[Any]*) – The index labels (e.g., node IDs, time points).
- **mean** (*np.ndarray*) – The mean values, shape (n,).
- **std** (*np.ndarray or None*) – The standard deviations, shape (n,), or None if deterministic.
- **quantiles** (*dict[float, np.ndarray] or None*) – Quantile arrays, e.g., {0.025: (n,), 0.975: (n,)}, or None.

- **meta** (*dict [str , Any]*) – Optional metadata (e.g., algorithm parameters, run info).

Examples

```
>>> import numpy as np
>>> # Deterministic
>>> s = StatSeries(
...     index=['a', 'b', 'c'],
...     mean=np.array([1.0, 2.0, 3.0])
... )
>>> s.is_deterministic
True
>>> s.certainty
1.0
>>> np.array(s)
array([1., 2., 3.])
```

```
>>> # With uncertainty
>>> s_unc = StatSeries(
...     index=['a', 'b', 'c'],
...     mean=np.array([1.0, 2.0, 3.0]),
...     std=np.array([0.1, 0.2, 0.15]),
...     quantiles={0.025: np.array([0.8, 1.6, 2.7]),
...                0.975: np.array([1.2, 2.4, 3.3])}
... )
>>> s_unc.is_deterministic
False
>>> s_unc.certainty
0.0
```

property certainty: float

Return certainty level.

Returns 1.0 if deterministic, 0.0 otherwise. In the future, this could return a richer metric.

index: List[Any]

property is_deterministic: bool

Return True if this is a deterministic result (no uncertainty).

mean: ndarray

meta: Dict[str, Any]

quantiles: Dict[float, ndarray] | None = None

std: ndarray | None = None

to_dict() → Dict[Any, Dict[str, Any]]

Convert to dictionary mapping index -> stats dict.

Returns

Dictionary with keys from index, values are dicts with 'mean', optionally 'std' and 'quantiles'.

Return type
dict

`class py3plex.uncertainty.types.UncertaintyConfig(mode: UncertaintyMode = UncertaintyMode.OFF, default_n_runs: int = 50, default_resampling: ResamplingStrategy = ResamplingStrategy.SEED)`

Bases: `object`

Configuration for uncertainty estimation.

mode

The current uncertainty mode.

Type
UncertaintyMode

default_n_runs

Default number of runs for uncertainty estimation.

Type
int

default_resampling

Default resampling strategy.

Type
ResamplingStrategy

`default_n_runs: int = 50`

`default_resampling: ResamplingStrategy = 'seed'`

`mode: UncertaintyMode = 'off'`

`class py3plex.uncertainty.types.UncertaintyMode(value)`

Bases: `Enum`

Global mode for uncertainty computation.

OFF

Always deterministic, std=None.

Type
str

ON

Try to compute uncertainty when supported.

Type
str

AUTO

Only do it if explicitly requested by a function.

Type
str

AUTO = 'auto'

OFF = 'off'

ON = 'on'

Context management for global uncertainty settings.

This module provides context variables and context managers for controlling uncertainty estimation globally across a pipeline or workflow.

py3plex.uncertainty.context.**get_uncertainty_config**() → *UncertaintyConfig*

Get the current uncertainty configuration.

Returns
The current configuration from the context.

Return type
UncertaintyConfig

Examples

```
>>> from py3plex.uncertainty import get_uncertainty_config
>>> cfg = get_uncertainty_config()
>>> cfg.mode
<UncertaintyMode.OFF: 'off'>
>>> cfg.default_n_runs
50
```

py3plex.uncertainty.context.**set_uncertainty_config**(*config: UncertaintyConfig*) → Token

Set the uncertainty configuration.

Parameters
config (UncertaintyConfig) – The new configuration to set.

Returns
A token that can be used to reset the configuration.

Return type
Token

Examples

```
>>> from py3plex.uncertainty import set_uncertainty_config,␣
↪UncertaintyConfig
>>> from py3plex.uncertainty import UncertaintyMode
>>> cfg = UncertaintyConfig(mode=UncertaintyMode.ON, default_n_runs=100)
>>> token = set_uncertainty_config(cfg)
>>> # ... do work ...
>>> _uncertainty_ctx.reset(token)  # restore previous config
```

py3plex.uncertainty.context.**uncertainty_disabled**()

Context manager to disable uncertainty estimation.

Within this context, all functions will compute deterministic results (unless explicitly requested with uncertainty=True).

Yields
None

Examples

```
>>> from py3plex.uncertainty import uncertainty_disabled, uncertainty_
↪enabled
>>>
>>> with uncertainty_enabled():
...     # Nested context to temporarily disable
...     with uncertainty_disabled():
...         result = multilayer_pagerank(network)
>>> result.is_deterministic
True
```

py3plex.uncertainty.context.**uncertainty_enabled**(**, n_runs: int | None = None, resampling: ResamplingStrategy | None = None*)

Context manager to enable uncertainty estimation.

Within this context, all supported functions will compute uncertainty by default (unless explicitly disabled with uncertainty=False).

Parameters

- **n_runs** (*int, optional*) – Number of runs for uncertainty estimation. If None, uses the default from the current config.
- **resampling** (*ResamplingStrategy, optional*) – Resampling strategy to use. If None, uses the default from the current config.

Yields
None

Examples

```
>>> from py3plex.uncertainty import uncertainty_enabled
>>> from py3plex.algorithms.centrality_toolkit import multilayer_pagerank
>>>
>>> # Without uncertainty
>>> result = multilayer_pagerank(network)
>>> result.is_deterministic
True
>>>
>>> # With uncertainty
>>> with uncertainty_enabled(n_runs=100):
...     result = multilayer_pagerank(network)
>>> result.is_deterministic
False
>>> result.std is not None
True
```

Notes

This uses contextvars, so it's thread-safe and async-safe. Each context gets its own configuration.

Uncertainty estimation helpers.

This module provides the main helper function for estimating uncertainty in network statistics via resampling or perturbation strategies.

`py3plex.uncertainty.estimation.estimate_uncertainty`(*network: multi_layer_network*, *metric_fn: Callable[[multi_layer_network], Dict[Any, float] | float | ndarray]*, ***, *n_runs: int | None = None*, *resampling: ResamplingStrategy | None = None*, *random_seed: int | None = None*, *perturbation_params: Dict[str, Any] | None = None*) → *StatSeries* | float

Estimate uncertainty for a network statistic.

This is the main entry point for adding uncertainty to any statistic. It runs the metric function multiple times with different random seeds or network perturbations, then computes mean, std, and quantiles.

Parameters

- **network** (`multi_layer_network`) – The network to analyze.
- **metric_fn** (*callable*) – Function that computes a statistic. Must accept a network and return: - dict[node, float] for per-node statistics - float for scalar statistics - np.ndarray for array statistics
- **n_runs** (*int, optional*) – Number of runs for uncertainty estimation. If None, uses the default from the current uncertainty config.
- **resampling** (`ResamplingStrategy`, *optional*) – Strategy for resampling. If None, uses default from config.
- **random_seed** (*int, optional*) – Random seed for reproducibility.
- **perturbation_params** (*dict, optional*) – Parameters for perturbation strategies. For example: {"edge_drop_p": 0.05, "node_drop_p": 0.02}

Returns

If metric_fn returns a dict: StatSeries with uncertainty info If metric_fn returns a scalar: float (mean value) The returned object has mean, std, and quantiles populated.

Return type

StatSeries or float

Examples

```
>>> from py3plex.uncertainty import estimate_uncertainty,␣
↪ResamplingStrategy
>>> from py3plex.core import multinet
>>>
>>> # Create a simple network
>>> net = multinet.multi_layer_network(directed=False)
>>> net.add_edges([["a", "L0", "b", "L0", 1.0]], input_type="list")
>>>
>>> # Define a metric function
>>> def my_metric(network):
...     # Return per-node degree
...     degrees = {}
...     for node in network.get_nodes():
...         degrees[node] = network.core_network.degree(node)
...     return degrees
>>>
>>> # Estimate uncertainty
>>> result = estimate_uncertainty(
...     net,
...     my_metric,
...     n_runs=50,
...     resampling=ResamplingStrategy.PERTURBATION,
...     perturbation_params={"edge_drop_p": 0.1}
... )
>>> result.mean  # Mean degree values
>>> result.std   # Std deviation of degrees
>>> result.quantiles  # Confidence intervals
```

Notes

- For SEED strategy: runs the metric with different random seeds
- For PERTURBATION strategy: applies edge/node drops then recomputes
- For BOOTSTRAP: resamples nodes/edges with replacement (not yet implemented)
- For JACKKNIFE: leave-one-out resampling (not yet implemented)

Algorithms

Community Detection

py3plex.algorithms.community_detection.community_wrapper.**NoRC_communities**(*network, verbose=True, clustering_scheme='kmean, output='mapping', prob_threshold=0.00, parallel_step=8, community_range=None, fine_range=3*) py3plex.algorithms.community_detection.community_wrapper.**infomap_communities**(*graph: Graph*, *binary: str = './infomap'*, *edgelist_file: str = './tmp/tmpedgelis*, *multiplex: bool = False*, *verbose: bool = False*, *overlapping: bool = False*, *iterations: int = 200*, *output: str = 'mapping'*, *seed: int | None = None*) → Dict[Any, int] | Dict[Any, List[int]]

Detect communities using the Infomap algorithm.

Parameters

- **graph** – Input graph (NetworkX graph or multi_layer_network)
- **binary** – Path to Infomap binary (default: "./infomap")
- **edgelist_file** – Temporary file for edgelist (default: "./tmp/tmpedgelist.txt")
- **multiplex** – Whether to use multiplex mode (default: False)
- **verbose** – Whether to show verbose output (default: False)
- **overlapping** – Whether to detect overlapping communities (default: False)
- **iterations** – Number of iterations (default: 200)
- **output** – Output format - "mapping" or "partition" (default: "mapping")
- **seed** – Random seed for reproducibility (default: None) Note: Requires Infomap binary that supports –seed parameter

Returns

Dict mapping nodes to community IDs (if output="mapping") or Dict mapping community IDs to lists of nodes (if output="partition")

Raises

- **FileNotFoundError** – If Infomap binary is not found
- **PermissionError** – If Infomap binary is not executable

Examples

```
>>> # Using with seed for reproducibility
>>> partition = infomap_communities(graph, seed=42)
>>>
>>> # Get partition format instead of mapping
>>> communities = infomap_communities(graph, output="partition")
```

py3plex.algorithms.community_detection.community_wrapper.**louvain_communities**(*network, output='mapping'*)

py3plex.algorithms.community_detection.community_wrapper.**parse_infomap**(*outfile*) `py3plex.algorithms.community_detection.community_wrapper.`**`run_infomap`**(*infile: str*, *multiplex: bool = True*, *overlapping: bool = False*, *binary: str = './infomap'*, *verbose: bool = True*, *iterations: int = 1000*, *seed: int | None = None*) → None

Multilayer Modularity Maximization (Mucha et al., 2010)

This module implements multilayer modularity quality function and optimization algorithms for community detection in multilayer/multiplex networks.

References

Mucha et al., "Community Structure in Time-Dependent, Multiscale, and Multiplex Networks", Science 328:876-878 (2010)

`py3plex.algorithms.community_detection.multilayer_modularity.`**`build_supra_modularity_matrix`**`(`

Build the supra-modularity matrix B for multilayer network.

The supra-modularity matrix is: B_{i,j} = (A^[]_ij - ^[] k_i^ k_j^ / 2m_) _ + _ij _

This matrix can be used for spectral community detection methods.

Parameters

- **`network`** – py3plex multi_layer_network object
- **`gamma`** – Resolution parameter(s)
- **`omega`** – Inter-layer coupling strength
- **`weight`** – Edge weight attribute

Returns

Tuple of (modularity_matrix, node_layer_list) - modularity_matrix: Supra-modularity matrix B (NL × NL) - node_layer_list: List of (node, layer) tuples corresponding to matrix indices `py3plex.algorithms.community_detection.multilayer_modularity.louvain_multilayer(`*network: Any, gamma: float | Dict[Any, float] = 1.0, omega: float | ndarray = 1.0, weight: str = 'weight', max_iter: int = 100, random_state: int | None = None*`)` → Dict[Tuple[Any, Any], int]

Generalized Louvain algorithm for multilayer networks.

This implements the multilayer Louvain method as described in Mucha et al. (2010), which greedily maximizes the multilayer modularity quality function.

Complexity:

- **Time: O(n × L × d × k) per iteration, where:**
  - n = number of nodes per layer
  - L = number of layers
  - d = average degree
  - k = number of communities
- Typical: O(n × L) iterations until convergence
- Worst case: $O((n \times L)^2)$ for dense networks
- Space: $O((n \times L)^2)$ for supra-adjacency matrix (use sparse for large networks)

Parameters

- **network** – py3plex multi_layer_network object
- **gamma** – Resolution parameter(s)
- **omega** – Inter-layer coupling strength
- **weight** – Edge weight attribute
- **max_iter** – Maximum number of iterations
- **random_state** – Random seed for reproducibility

Returns

Dictionary mapping (node, layer) tuples to community IDs

Examples

```
>>> from py3plex.core import multinet
>>> from py3plex.algorithms.community_detection.multilayer_modularity␣
↪import louvain_multilayer
>>>
>>> network = multinet.multi_layer_network(directed=False)
>>> network.add_edges([
...     ['A', 'L1', 'B', 'L1', 1],
...     ['B', 'L1', 'C', 'L1', 1],
...     ['A', 'L2', 'C', 'L2', 1]
... ], input_type='list')
>>>
>>> communities = louvain_multilayer(network, gamma=1.0, omega=1.0,␣
↪random_state=42)
>>> print(communities)
```

Note

For reproducible results, always set random_state parameter.

`py3plex.algorithms.community_detection.multilayer_modularity.multilayer_modularity(network: Any, communities: Dict[Tuple[Any, Any], int], gamma: float | Dict[Any, float] = 1.0, omega: float | ndarray = 1.0, weight: str = 'weight') → float`

Calculate multilayer modularity quality function (Mucha et al., 2010).

The multilayer modularity is defined as: Q = (1/2) _{ij} [(A^[]_ij - ^[]P^[]_ij)_ + _ij _] (g_i, g_j)

where: - A^[]_ij is the adjacency matrix of layer - P^[]_ij is the null model (e.g., Newman-Girvan: k_i^ k_j^ / 2m_) - ^[] is the resolution parameter for layer - _ is the inter-layer coupling strength - _ = 1 if =, else 0 (Kronecker delta) - _ij = 1 if i=j, else 0 - (g_i, g_j) = 1 if node i in layer and node j in layer are in same community - is the total edge weight in the supra-network

Parameters

- **network** – py3plex multi_layer_network object
- **communities** – Dictionary mapping (node, layer) tuples to community IDs
- **gamma** – Resolution parameter(s). Can be: - Single float: same resolution for all layers - Dict[layer, float]: layer-specific resolution parameters
- **omega** – Inter-layer coupling strength. Can be: - Single float: uniform coupling between all layer pairs - np.ndarray: layer-pair specific coupling matrix (L×L)
- **weight** – Edge weight attribute (default: "weight")

Returns

Modularity value Q [-1, 1]

Examples

```
>>> from py3plex.core import multinet
>>> from py3plex.algorithms.community_detection.multilayer_modularity import multilayer_modularity
>>>
>>> # Create a simple multilayer network
>>> network = multinet.multi_layer_network(directed=False)
>>> network.add_edges([
...     ['A', 'L1', 'B', 'L1', 1],
...     ['B', 'L1', 'C', 'L1', 1],
...     ['A', 'L2', 'C', 'L2', 1]
... ], input_type='list')
>>>
>>> # Assign communities
>>> communities = {
...     ('A', 'L1'): 0, ('B', 'L1'): 0, ('C', 'L1'): 1,
...     ('A', 'L2'): 0, ('C', 'L2'): 0
... }
>>>
>>> # Calculate modularity
>>> Q = multilayer_modularity(network, communities, gamma=1.0, omega=1.0)
>>> print(f"Modularity: {Q:.3f}")
```

This module implements community detection.

class py3plex.algorithms.community_detection.community_louvain.Status

Bases: `object`

To handle several data in one struct.

Could be replaced by named tuple, but don't want to depend on python 2.6

copy()

Perform a deep copy of status

degrees: dict = {}

gdegrees: dict = {}

init(*graph*, *weight*, *part=None*)

Initialize the status of a graph with every node in one community

internals: dict = {}

node2com: dict = {}

total_weight = 0

`py3plex.algorithms.community_detection.community_louvain.`**`best_partition`**(*graph: Graph, partition: Dict | None = None, weight: str = 'weight', resolution: float = 1.0, randomize: bool = False*) → Dict

Compute the partition of the graph nodes which maximises the modularity (or try..) using the Louvain heuristices

This is the partition of highest modularity, i.e. the highest partition of the dendrogram generated by the Louvain algorithm.

Parameters

- **`graph`** (*`networkx.Graph`*) – the networkx graph which is decomposed
- **`partition`** (*`dict`*, *`optional`*) – the algorithm will start using this partition of the nodes. It's a dictionary where keys are their nodes and values the communities
- **`weight`** (*`str`*, *`optional`*) – the key in graph to use as weight. Default to 'weight'
- **`resolution`** (*`double`*, *`optional`*) – Will change the size of the communities, default to 1. represents the time described in "Laplacian Dynamics and Multiscale Modular Structure in Networks", R. Lambiotte, J.-C. Delvenne, M. Barahona
- **`randomize`** (*`boolean`*, *`optional`*) – Will randomize the node evaluation order and the community evaluation order to get different partitions at each call

Returns

partition – The partition, with communities numbered from 0 to number of communities

Return type

dictionnary

Raises

`NetworkXError` – If the graph is not Eulerian.

See also

generate_dendrogram

Notes

Uses Louvain algorithm

References

large networks. J. Stat. Mech 10008, 1-12(2008).

Examples

```
>>>  #Basic usage
>>> G=nx.erdos_renyi_graph(100, 0.01)
>>> part = best_partition(G)
```

```
>>> #other example to display a graph with its community :
>>> #better with karate_graph() as defined in networkx examples
>>> #erdos renyi don't have true community structure
>>> G = nx.erdos_renyi_graph(30, 0.05)
>>> #first compute the best partition
>>> partition = community.best_partition(G)
>>>  #drawing
>>> size = float(len(set(partition.values())))
>>> pos = nx.spring_layout(G)
>>> count = 0.
>>> for com in set(partition.values()) :
>>>     count += 1.
>>>     list_nodes = [nodes for nodes in partition.keys()
>>>                                 if partition[nodes] == com]
>>>     nx.draw_networkx_nodes(G, pos, list_nodes, node_size = 20,
                                node_color = str(count / size))
>>> nx.draw_networkx_edges(G, pos, alpha=0.5)
>>> plt.show()
```

`py3plex.algorithms.community_detection.community_louvain.generate_dendrogram`(*graph: Graph, part_init: Dict | None = None, weight: str = 'weight', resolution: float = 1.0, randomize: bool = False*) → List[Dict]

Find communities in the graph and return the associated dendrogram

A dendrogram is a tree and each level is a partition of the graph nodes. Level 0 is the first partition, which contains the smallest communities, and the best is len(dendrogram) - 1. The higher the level is, the bigger are the communities

Parameters

- **graph** (*networkx.Graph*) – the networkx graph which will be decomposed
- **part_init** (*dict, optional*) – the algorithm will start using this partition of the nodes. It's a dictionary where keys are their nodes and values the communities
- **weight** (*str, optional*) – the key in graph to use as weight. Default to 'weight'
- **resolution** (*double, optional*) – Will change the size of the communities, default to 1. represents the time described in "Laplacian Dynamics and Multiscale Modular Structure in Networks", R. Lambiotte, J.-C. Delvenne, M. Barahona

Returns

dendrogram – a list of partitions, ie dictionnaries where keys of the i+1 are the values of the i. and where keys of the first are the nodes of graph

Return type

list of dictionaries

Raises

`TypeError` – If the graph is not a networkx.Graph

See also

best_partition

Notes

Uses Louvain algorithm

References

networks. J. Stat. Mech 10008, 1-12(2008).

Examples

```
>>> G=nx.erdos_renyi_graph(100, 0.01)
>>> dendo = generate_dendrogram(G)
>>> for level in range(len(dendo) - 1) :
>>>     print("partition at level", level,
>>>           "is", partition_at_level(dendo, level))
:param weight:
:type weight:
```

`py3plex.algorithms.community_detection.community_louvain.induced_graph(partition: Dict, graph: Graph, weight: str = 'weight') → Graph`

Produce the graph where nodes are the communities

there is a link of weight w between communities if the sum of the weights of the links between their elements is w

Parameters

- **partition** (*dict*) – a dictionary where keys are graph nodes and values the part the node belongs to
- **graph** (*networkx.Graph*) – the initial graph
- **weight** (*str, optional*) – the key in graph to use as weight. Default to 'weight'

Returns

g – a networkx graph where nodes are the parts

Return type

networkx.Graph

Examples

```
>>> n = 5
>>> g = nx.complete_graph(2*n)
>>> part = dict([])
>>> for node in g.nodes() :
>>>     part[node] = node % 2
>>> ind = induced_graph(part, g)
>>> goal = nx.Graph()
>>> goal.add_weighted_edges_from([(0,1,n*n),(0,0,n*(n-1)/2), (1, 1, n*(n-
↪1)/2)])  # NOQA
>>> nx.is_isomorphic(int, goal)
True
```

py3plex.algorithms.community_detection.community_louvain.**load_binary**(*data*)

Load binary graph as used by the cpp implementation of this algorithm

py3plex.algorithms.community_detection.community_louvain.**modularity**(*partition: Dict*, *graph: Graph*, *weight: str = 'weight'*) → float

Compute the modularity of a partition of a graph

Parameters

- **partition** (*dict*) – the partition of the nodes, i.e a dictionary where keys are their nodes and values the communities
- **graph** (*networkx.Graph*) – the networkx graph which is decomposed
- **weight** (*str, optional*) – the key in graph to use as weight. Default to 'weight'

Returns

modularity – The modularity

Return type

float

Raises

- **KeyError** – If the partition is not a partition of all graph nodes
- **ValueError** – If the graph has no link
- **TypeError** – If graph is not a networkx.Graph

References

structure in networks. Physical Review E 69, 26113(2004).

Examples

```
>>> G=nx.erdos_renyi_graph(100, 0.01)
>>> part = best_partition(G)
>>> modularity(part, G)
```

py3plex.algorithms.community_detection.community_louvain.**partition_at_level**(*dendrogram: List[Dict]*, *level: int*) → Dict

Return the partition of the nodes at the given level

A dendrogram is a tree and each level is a partition of the graph nodes. Level 0 is the first partition, which contains the smallest communities, and the best is len(dendrogram) - 1. The higher the level is, the bigger are the communities

Parameters

- **dendrogram** (*list of dict*) – a list of partitions, ie dictionnaries where keys of the i+1 are the values of the i.
- **level** (*int*) – the level which belongs to [0..len(dendrogram)-1]

Returns

partition – A dictionary where keys are the nodes and the values are the set it belongs to

Return type

dictionnary

Raises

KeyError – If the dendrogram is not well formed or the level is too high

See also

best_partition, *generate_dendrogram*

Examples

```
>>> G=nx.erdos_renyi_graph(100, 0.01)
>>> dendrogram = generate_dendrogram(G)
>>> for level in range(len(dendrogram) - 1) :
>>>     print("partition at level", level, "is", partition_at_
↪level(dendrogram, level))  # NOQA
```

Community quality measures and metrics.

This module provides various measures for assessing the quality of community partitions in networks, including modularity, size distribution, and other statistical metrics.

`py3plex.algorithms.community_detection.community_measures.`**`modularity`**`(`*G: Graph, communities: Dict[Any, List[Any]], weight: str = 'weight'*`)` → float

Calculate modularity of a graph partition.

Parameters

- **`G`** – NetworkX graph
- **`communities`** – Dictionary mapping community IDs to node lists
- **`weight`** – Edge weight attribute (default: "weight")

Returns

Modularity value

`py3plex.algorithms.community_detection.community_measures.`**`number_of_communities`**`(`*network_part Dict[Any, List[Any]]*`)` → int

Count number of communities in a partition.

Parameters

`network_partition` – Dictionary mapping community IDs to node lists

Returns

Number of communities

`py3plex.algorithms.community_detection.community_measures.`**`size_distribution`**`(`*network_partition. Dict[Any, List[Any]]*`)` → ndarray

Calculate size distribution of communities.

Parameters

`network_partition` – Dictionary mapping community IDs to node lists

Returns

Array of community sizes

Multilayer Synthetic Graph Generation for Community Detection Benchmarks

This module implements synthetic multilayer/multiplex graph generators with ground-truth community structure for benchmarking community detection algorithms.

Includes: - Multilayer LFR (Lancichinetti-Fortunato-Radicchi) benchmark - Coupled/Interdependent Erdős-Rényi models - Support for overlapping communities across layers - Support for partial node presence across layers

References

Lancichinetti et al., "Benchmark graphs for testing community detection algorithms", Phys. Rev. E 78, 046110 (2008)

Granell et al., "Benchmark model to assess community structure in evolving networks", Phys. Rev. E 92, 012805 (2015)

`py3plex.algorithms.community_detection.multilayer_benchmark.generate_coupled_er_multilayer(`

Generate coupled/interdependent Erdős-Rényi multilayer networks.

Creates random Erdős-Rényi graphs in each layer and couples nodes across layers with specified coupling strength and probability.

Parameters

- **n** – Number of nodes
- **layers** – List of layer names
- **p** – Edge probability per layer. Can be float (same for all layers) or list of floats per layer.
- **omega** – Inter-layer coupling strength (weight of identity links)

- **coupling_probability** – Probability that a node has inter-layer coupling. Range: [0, 1] - 1.0 = all nodes coupled (full multiplex) - <1.0 = partial coupling (interdependent networks)
- **directed** – Whether to generate directed networks
- **seed** – Random seed for reproducibility

Returns

py3plex multi_layer_network object

Examples

```
>>> from py3plex.algorithms.community_detection.multilayer_benchmark␣
↪import generate_coupled_er_multilayer
>>>
>>> # Full multiplex ER network
>>> network = generate_coupled_er_multilayer(
...     n=100,
...     layers=['L1', 'L2', 'L3'],
...     p=0.1,
...     omega=1.0,
...     coupling_probability=1.0
... )
>>>
>>> # Partially coupled (interdependent)
>>> network = generate_coupled_er_multilayer(
...     n=100,
...     layers=['L1', 'L2'],
...     p=0.1,
...     omega=0.5,
...     coupling_probability=0.5  # Only 50% nodes coupled
... )
```

py3plex.algorithms.community_detection.multilayer_benchmark.**generate_multilayer_lfr**(*n: int, layers: List[str], tau1: float = 2.0, tau2: float = 1.5, mu: float | List[float] = 0.1, avg_degr float | List[float = 10.0, min_con int = 20, max_con int | None = None, community_pers float = 1.0, node_ove float = 1.0, overlapping_noc int = 0, overlapping_me int = 2,* Generate multilayer LFR benchmark networks with controllable community structure.

This extends the LFR benchmark to multilayer networks, allowing control over: - Community persistence across layers (how many nodes keep their community) - Node overlap across layers (which nodes appear in which layers) - Overlapping communities (nodes belonging to multiple communities)

Parameters

- **n** – Number of nodes
- **layers** – List of layer names
- **tau1** – Power-law exponent for degree distribution (typically 2-3)
- **tau2** – Power-law exponent for community size distribution (typically 1-2)
- **mu** – Mixing parameter (fraction of edges outside community). Can be float (same for all layers) or list of floats per layer. Range: [0, 1], where 0 = perfect communities, 1 = random
- **avg_degree** – Average degree per layer. Can be float (same for all layers) or list of floats per layer.
- **min_community** – Minimum community size
- **max_community** – Maximum community size (default: n/2)
- **community_persistence** – Probability that a node keeps its community from one layer to the next. Range: [0, 1] - 1.0 = identical communities across all layers - 0.0 = completely independent communities per layer
- **node_overlap** – Fraction of nodes present in all layers. Range: [0, 1] - 1.0 = all nodes in all layers (full multiplex) - <1.0 = some nodes absent from some layers
- **overlapping_nodes** – Number of nodes that belong to multiple communities within each layer
- **overlapping_membership** – Number of communities each overlapping node belongs to
- **directed** – Whether to generate directed networks
- **seed** – Random seed for reproducibility

Returns

Tuple of (network, ground_truth_communities) - network: py3plex multi_layer_network object - ground_truth_communities: Dict mapping (node, layer) to Set of community IDs

Examples

```
>>> from py3plex.algorithms.community_detection.multilayer_benchmark␣
↪import generate_multilayer_lfr
>>>
>>> # Generate with identical communities across layers
>>> network, communities = generate_multilayer_lfr(
...     n=100,
```

(continues on next page)

(continued from previous page)

```
...     layers=['L1', 'L2', 'L3'],
...     mu=0.1,
...     community_persistence=1.0
... )
>>>
>>> # Generate with evolving communities
>>> network, communities = generate_multilayer_lfr(
...     n=100,
...     layers=['T0', 'T1', 'T2'],
...     mu=0.1,
...     community_persistence=0.7  # 70% nodes keep community
... )
```

`py3plex.algorithms.community_detection.multilayer_benchmark.`**`generate_sbm_multilayer`**`(`*n: int*, *layers: List[str]*, *communities: List[Set[i* *p_in: float | List[float* *= 0.3*, *p_out: float | List[float* *= 0.05*, *community_pers* *float = 1.0*, *directed: bool = False*, *seed: int | None = None*) → Tuple[Any, Dict[Tup str], int]]

Generate multilayer stochastic block model (SBM) networks.

Creates networks where nodes are divided into communities with different intra- and inter-community connection probabilities.

Parameters

- **n** – Number of nodes

- **layers** – List of layer names
- **communities** – List of node sets defining initial communities
- **p_in** – Intra-community edge probability per layer
- **p_out** – Inter-community edge probability per layer
- **community_persistence** – Probability nodes keep their community across layers
- **directed** – Whether to generate directed networks
- **seed** – Random seed for reproducibility

Returns

Tuple of (network, ground_truth_communities) - network: py3plex multi_layer_network object - ground_truth_communities: Dict mapping (node, layer) to community ID

Examples

```
>>> from py3plex.algorithms.community_detection.multilayer_benchmark␣
↪import generate_sbm_multilayer
>>>
>>> # Define initial communities
>>> communities = [
...     {0, 1, 2, 3, 4},   # Community 0
...     {5, 6, 7, 8, 9}    # Community 1
... ]
>>>
>>> network, ground_truth = generate_sbm_multilayer(
...     n=10,
...     layers=['L1', 'L2'],
...     communities=communities,
...     p_in=0.7,
...     p_out=0.1,
...     community_persistence=0.8
... )
```

Statistics

Multilayer Network Statistics

This module implements various statistics for multilayer and multiplex networks, following standard definitions from multilayer network analysis literature.

References

- Kivelä et al. (2014), "Multilayer networks", J. Complex Networks 2(3), 203-271
- De Domenico et al. (2013), "Mathematical formulation of multilayer networks", PRX 3, 041022
- Mucha et al. (2010), "Community Structure in Time-Dependent, Multiscale, and Multiplex Networks", Science 328, 876-878

Authors: py3plex contributors Date: 2025 `py3plex.algorithms.statistics.multilayer_statistics.algebraic_connectivity(`*network: Any*`)` → float

Calculate algebraic connectivity ($_2$).

Formula: $_2$()

Second smallest eigenvalue of the supra-Laplacian (Fiedler value).

Indicates global connectivity and diffusion efficiency of the multilayer system.

Properties:
$_0$ = 0 always (associated with constant eigenvector) $_1$ > 0 if and only if the multilayer network is connected Larger $_1$ indicates better connectivity and faster diffusion/synchronization

Parameters
network – py3plex multi_layer_network object

Returns
Second smallest eigenvalue (Fiedler value)

Examples

```
>>> alg_conn = algebraic_connectivity(network)
```

Reference:
Fiedler (1973), Sole-Ribalta et al. (2013)

`py3plex.algorithms.statistics.multilayer_statistics.community_participation_coefficient(`*netw Any com mu- ni- ties: Dict Any int], node Any*`)` → float

Calculate participation coefficient for a node across community structure.

Measures how evenly a node's connections are distributed across different communities, across all layers. A node with connections to many communities has high participation.

Formula: P = 1 - (k / k)^2

where k is the number of connections node i has to community s, and k is the total degree of node i across all layers.

Parameters

- **network** – py3plex multi_layer_network object
- **communities** – Dictionary mapping (node, layer) to community ID

- **node** – Node identifier (not node-layer tuple)

Returns
Participation coefficient value between 0 and 1

Examples

```
>>> communities = detect_communities(network)
>>> pc = community_participation_coefficient(network, communities, 'Alice
↪')
>>> print(f"Participation: {pc:.3f}")
```

Reference:
Guimerà & Amaral (2005), "Functional cartography of complex metabolic networks"

`py3plex.algorithms.statistics.multilayer_statistics.community_participation_entropy`(*network: Any*, *communities: Dict[Tuple[Any, Any], int]*, *node: Any*) → float

Calculate participation entropy for a node across community structure.

Shannon entropy-based measure of how evenly a node distributes its connections across different communities. Higher entropy indicates more diverse community participation.

Formula: H = - (k / k) log(k / k)

where k is connections to community s, k is total degree.

Parameters

- **network** – py3plex multi_layer_network object
- **communities** – Dictionary mapping (node, layer) to community ID
- **node** – Node identifier (not node-layer tuple)

Returns
Entropy value (higher = more diverse participation)

Examples

```
>>> entropy = community_participation_entropy(network, communities, 'Alice
↪')
>>> print(f"Participation entropy: {entropy:.3f}")
```

Reference:
Based on Shannon entropy applied to community structure

py3plex.algorithms.statistics.multilayer_statistics.**compute_modularity_score**(*network: Any, communities: Dict[Tuple[Any, Any], int], gamma: float = 1.0, omega: float = 1.0*) → float

Compute explicit multislice modularity score.

Direct computation of the modularity quality function for a given community partition, without running detection algorithms.

Formula: Q = (1/2) [(A - · kk/(2m)) + ·] (c, c)

Parameters

- **network** – py3plex multi_layer_network object
- **communities** – Dictionary mapping (node, layer) to community ID
- **gamma** – Resolution parameter (default: 1.0)
- **omega** – Inter-layer coupling strength (default: 1.0)

Returns

Modularity score Q (higher is better)

Examples

```
>>> communities = {('A', 'L1'): 0, ('B', 'L1'): 0, ('C', 'L1'): 1}
>>> Q = compute_modularity_score(network, communities)
>>> print(f"Modularity: {Q:.3f}")
```

Reference:

Mucha et al. (2010), Science 328, 876-878 py3plex.algorithms.statistics.multilayer_statistics.**cross_layer_mutual_information**(*network: Any*, *layer_i: str*, *layer_j: str*, *bins: int = 10*) → float

Calculate cross-layer mutual information (I(L; L)).

Formula: I(L; L) = H(L) + H(L) - H(L, L)

Measures statistical dependence between degree distributions in two layers; quantifies how much knowing a node's degree in one layer tells us about its degree in another layer.

Variables:
H(L) = entropy of degree distribution in layer i H(L) = entropy of degree distribution in layer j H(L, L) = joint entropy of degree distributions

Properties:
I = 0 when layers are independent I > 0 indicates statistical dependence (higher = stronger) I(L; L) min(H(L), H(L))

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier
- **layer_j** – Second layer identifier
- **bins** – Number of bins for discretizing degree distributions

Returns
Mutual information value in bits

Examples

```
>>> mi = cross_layer_mutual_information(network, 'L1', 'L2', bins=10)
>>> print(f"Mutual information: {mi:.3f} bits")
```

Reference:
Cover & Thomas (2006), "Elements of Information Theory" De Domenico et al. (2015), "Structural reducibility"

py3plex.algorithms.statistics.multilayer_statistics.**cross_layer_redundancy_entropy**(*network: Any*) → float

Calculate cross-layer redundancy entropy (H_redundancy).

Formula: H_r = - r $\log_2$(r), where r is the normalized edge overlap between layers i and j.

Measures diversity in structural redundancy across layer pairs; high entropy indicates varied overlap patterns, low entropy indicates uniform redundancy.

Variables:

r = edge_overlap(i,j) / edge_overlap(,) Normalized overlap proportion for each layer pair

Parameters

network – py3plex multi_layer_network object

Returns

Entropy value in bits

Examples

```
>>> entropy = cross_layer_redundancy_entropy(network)
>>> print(f"Cross-layer redundancy entropy: {entropy:.3f}")
```

Reference:

Bianconi (2018), "Multilayer Networks: Structure and Function"

py3plex.algorithms.statistics.multilayer_statistics.**degree_vector**(*network: Any*, *node: Any*, *weighted: bool = False*) → Dict[str, float]

Calculate degree vector (k).

Formula: k = (k^1, k^2, ..., k)

Node degree in each layer; can be analyzed via mean, variance, or entropy to capture node versatility.

Variables:

k = degree of node i in layer For undirected: k = A

Parameters

- **network** – py3plex multi_layer_network object
- **node** – Node identifier
- **weighted** – If True, return strength instead of degree

Returns

Dictionary mapping layer to degree/strength

Examples

```
>>> degrees = degree_vector(network, 'A')
>>> print(f"Degree in layer L1: {degrees['L1']}")
```

Reference:

Kivelä et al. (2014), J. Complex Networks 2(3), 203-271 `py3plex.algorithms.statistics.multilayer_statistics.edge_overlap(network: Any, layer_i: str, layer_j: str) → float`

Calculate edge overlap (^).

Formula: ^ = |E E| / |E E|

Jaccard similarity of edge sets between two layers; measures structural redundancy.

Variables:
E = set of edges in layer E = set of edges in layer |·| = cardinality (number of elements)

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier ()
- **layer_j** – Second layer identifier ()

Returns
Overlap coefficient between 0 and 1 (Jaccard similarity)

Examples

```
>>> overlap = edge_overlap(network, 'L1', 'L2')
```

Reference:
Kivelä et al. (2014), J. Complex Networks 2(3), 203-271

`py3plex.algorithms.statistics.multilayer_statistics.entropy_of_multiplexity(network: Any) → float`

Calculate entropy of multiplexity (H).

Formula: H = - p $\log_2$(p), where p = E / E

Shannon entropy of layer contributions; measures layer diversity.

Variables:
p = proportion of edges in layer E = number of edges in layer $\log_2$ gives entropy in bits

Properties:
H = 0 when all edges are in one layer (minimum entropy/diversity) H = $\log_2$(L) when edges are uniformly distributed across L layers (maximum entropy)

Parameters
network – py3plex multi_layer_network object

Returns
Entropy value in bits

Examples

```
>>> entropy = entropy_of_multiplexity(network)
```

Reference:
De Domenico et al. (2013), Shannon (1948)

`py3plex.algorithms.statistics.multilayer_statistics.inter_layer_assortativity`(*network: Any*, *layer_i: str*, *layer_j: str*) → float

Calculate inter-layer assortativity (r).

Formula: r^ = cov(k^, k^) / ( ) = corr(k^, k^)

Measures whether nodes with similar degrees tend to connect across different layers.

Variables:
k^ = degree vector in layer k^ = degree vector in layer , = standard deviations of degrees in layers and Equivalent to Pearson correlation of degree vectors

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier ()
- **layer_j** – Second layer identifier ()

Returns
Assortativity coefficient

Examples

```
>>> assort = inter_layer_assortativity(network, 'L1', 'L2')
```

Reference:
Newman (2002), Nicosia & Latora (2015)

`py3plex.algorithms.statistics.multilayer_statistics.inter_layer_coupling_strength`(*network: Any*, *layer_i: str*, *layer_j: str*) → float

Calculate inter-layer coupling strength (C^).

Formula: C^ = (1/N_) w^

Average weight of inter-layer connections between corresponding nodes in two layers.

Quantifies cross-layer connectivity.

Variables:

N_ = number of nodes present in both layers and w^ = weight of inter-layer edge connecting node i in layer to node i in layer

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier ()
- **layer_j** – Second layer identifier ()

Returns

Average coupling strength

Examples

```
>>> coupling = inter_layer_coupling_strength(network, 'L1', 'L2')
```

Reference:

De Domenico et al. (2013), Physical Review X 3(4), 041022

Contracts:

- Precondition: network must not be None
- Precondition: layer_i and layer_j must be non-empty strings
- Postcondition: result is non-negative (weights are non-negative)
- Postcondition: result is not NaN

py3plex.algorithms.statistics.multilayer_statistics.**inter_layer_degree_correlation**(*network: Any*, *layer_i: str*, *layer_j: str*) → float

Calculate inter-layer degree correlation (r^).

Formula: r^ = (k - k)(k - k) / [((k - k)^2) ((k - k)^2)]

Pearson correlation of node degrees between two layers; reveals if highly connected nodes in one layer are also central in others.

Variables:

k = degree of node i in layer k = mean degree in layer Sum over nodes present in both layers

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier ()
- **layer_j** – Second layer identifier ()

Returns

Pearson correlation coefficient between -1 and 1

Examples

```
>>> corr = inter_layer_degree_correlation(network, 'L1', 'L2')
```

Reference:
Battiston et al. (2014), Nicosia & Latora (2015)

py3plex.algorithms.statistics.multilayer_statistics.**inter_layer_dependence_entropy**(*network: Any*, *layer_i: str*, *layer_j: str*) → float

Calculate inter-layer dependence entropy (H_dep).

Formula: H_dep = - p $\log_2$(p), where p is the proportion of inter-layer edges for each node n connecting layers i and j.

Measures heterogeneity in how nodes couple the two layers; high entropy indicates diverse coupling patterns, low entropy indicates uniform coupling.

Variables:
p = proportion of inter-layer edges incident to node n Total over all nodes connecting the two layers

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier
- **layer_j** – Second layer identifier

Returns
Entropy value in bits

Examples

```
>>> entropy = inter_layer_dependence_entropy(network, 'L1', 'L2')
>>> print(f"Inter-layer dependence entropy: {entropy:.3f}")
```

Reference:
De Domenico et al. (2015), "Ranking in interconnected multilayer networks"

py3plex.algorithms.statistics.multilayer_statistics.**interdependence**(*network: Any*, *sample_size: int = 100*) → float

Calculate interdependence ().

Formula: = ⟨d⟩ / ⟨d⟩

Quantifies how much shortest-path communication depends on inter-layer connections.

Variables:
d = shortest path from node i to node j in the full multilayer network d = (1/L) d is the average shortest path across individual layers ⟨ · ⟩ = average over sampled node pairs

Interpretation:
< 1: multilayer connectivity reduces path lengths (positive interdependence) 1: inter-layer connections provide little benefit > 1: multilayer structure increases path lengths (rare)

Parameters

- **network** – py3plex multi_layer_network object
- **sample_size** – Number of node pairs to sample for estimation

Returns
Interdependence ratio

Examples

```
>>> interdep = interdependence(network, sample_size=50)
```

Reference:
Gomez et al. (2013), Buldyrev et al. (2010)

`py3plex.algorithms.statistics.multilayer_statistics.interlayer_degree_correlation_matrix`(*ne*
An
→
Tu
ple
list

Calculate inter-layer degree correlation matrix.

Computes Pearson correlation coefficients for node degrees between all pairs of layers, organized as a symmetric correlation matrix.

Formula: Matrix[,] = r^ = corr(k^, k^)

where k^ and k^ are degree vectors for layers and over common nodes.

Properties:

- Diagonal elements are 1.0 (self-correlation)
- Off-diagonal elements in [-1, 1]
- Symmetric matrix
- Positive values indicate positive degree correlation
- Negative values indicate negative degree correlation

Parameters
network – py3plex multi_layer_network object

Returns

- correlation_matrix: 2D numpy array of shape (num_layers, num_layers)

- layer_labels: List of layer names corresponding to matrix indices

Return type
Tuple of (correlation_matrix, layer_labels)

Examples

```
>>> corr_matrix, layers = interlayer_degree_correlation_matrix(network)
>>> import matplotlib.pyplot as plt
>>> import seaborn as sns
>>> sns.heatmap(corr_matrix, annot=True, xticklabels=layers,
...             yticklabels=layers, cmap='coolwarm', center=0,
...             vmin=-1, vmax=1)
>>> plt.title('Inter-layer Degree Correlation Matrix')
>>> plt.show()
```

Reference:
Nicosia & Latora (2015), "Measuring and modeling correlations in multiplex networks" Battiston et al. (2014), "Structural measures for multiplex networks"

`py3plex.algorithms.statistics.multilayer_statistics.`**`layer_connectivity_entropy`**`(`*network: Any*, *layer: str*`)` → float

Calculate entropy of layer connectivity (H_connectivity).

Formula: H_c = - (k/k) $\log_2$(k/k)

Shannon entropy of degree distribution within a layer; measures heterogeneity of node connectivity patterns.

Variables:
k = degree of node i in the layer k = sum of all degrees (2 * edges for undirected)

Properties:
H_c = 0 when all nodes have the same degree (uniform distribution) H_c is maximized when degree distribution is highly uneven

Parameters

- **network** – py3plex multi_layer_network object
- **layer** – Layer identifier

Returns
Entropy value in bits

Examples

```
>>> from py3plex.core import multinet
>>> network = multinet.multi_layer_network(directed=False)
>>> network.add_edges([
```

(continues on next page)

(continued from previous page)

```
...     ['A', 'L1', 'B', 'L1', 1],
...     ['B', 'L1', 'C', 'L1', 1]
... ], input_type='list')
>>> entropy = layer_connectivity_entropy(network, 'L1')
>>> print(f"Connectivity entropy: {entropy:.3f}")
```

Reference:
Solé-Ribalta et al. (2013), "Spectral properties of complex networks" Shannon (1948), "A Mathematical Theory of Communication"

`py3plex.algorithms.statistics.multilayer_statistics.layer_density`(*network: Any*, *layer: str*) → float

Calculate layer density ().

Formula: = (2E) / (N(N - 1)) [undirected]
= E / (N(N - 1)) [directed]

Measures the fraction of possible edges present in a specific layer, indicating how densely connected that layer is.

Variables:
E = number of edges in layer N = number of nodes in layer

Parameters

- **network** – py3plex multi_layer_network object
- **layer** – Layer identifier

Returns
Density value between 0 and 1

Examples

```
>>> from py3plex.core import multinet
>>> network = multinet.multi_layer_network(directed=False)
>>> network.add_edges([
...     ['A', 'L1', 'B', 'L1', 1],
...     ['B', 'L1', 'C', 'L1', 1]
... ], input_type='list')
>>> density = layer_density(network, 'L1')
>>> print(f"Layer L1 density: {density:.3f}")
```

Reference:
Kivelä et al. (2014), J. Complex Networks 2(3), 203-271

Contracts:

- Precondition: network must not be None
- Precondition: layer must be a non-empty string
- Postcondition: result is in [0, 1] (fundamental property of density)
- Postcondition: result is not NaN

`py3plex.algorithms.statistics.multilayer_statistics.layer_influence_centrality`(*network: Any, layer: str, method: str = 'coupling', sample_size: int = 100*) → float

Calculate layer influence centrality (I).

Formula (coupling): I = C^ / (L-1) Formula (flow): I = F^ / (L-1)

Quantifies how much a layer influences other layers through inter-layer connections (coupling method) or information flow (flow method).

Variables:
C^ = inter-layer coupling strength between layers and F^ = flow from layer to layer (random walk transition probability) L = total number of layers

Properties:
Higher values indicate layers that strongly influence others Useful for identifying critical layers in the multilayer structure

Parameters

- **network** – py3plex multi_layer_network object
- **layer** – Layer identifier
- **method** – 'coupling' for structural influence, 'flow' for dynamic influence
- **sample_size** – Number of random walk steps for flow simulation

Returns
Influence centrality value

Examples

```
>>> influence = layer_influence_centrality(network, 'L1', method='coupling
↪')
>>> print(f"Layer L1 influence: {influence:.3f}")
```

Reference:
Cozzo et al. (2013), "Mathematical formulation of multilayer networks" De Domenico et al. (2014), "Identifying modular flows"

py3plex.algorithms.statistics.multilayer_statistics.**layer_redundancy_coefficient**(*network: Any*, *layer_i: str*, *layer_j: str*) → float

Calculate layer redundancy coefficient.

Measures the proportion of edges in one layer that are redundant (also present) in another layer. Values close to 1 indicate high redundancy, while values close to 0 indicate complementary layers.

Formula: R = **|E E|** / **|E|**

where E and E are edge sets of layers and .

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier
- **layer_j** – Second layer identifier

Returns

Redundancy coefficient between 0 and 1

Examples

```
>>> redundancy = layer_redundancy_coefficient(network, 'social', 'work')
>>> print(f"Redundancy: {redundancy:.2%}")
```

Reference:

Nicosia & Latora (2015), "Measuring and modeling correlations in multiplex networks"

py3plex.algorithms.statistics.multilayer_statistics.**layer_similarity**(*network: Any*, *layer_i: str*, *layer_j: str*, *method: str = 'cosine'*) → float

Calculate layer similarity (S^).

Formula: S^ = ⟨A, A⟩ / (‖A‖ ‖A‖) = AA / ((A)^2) ((A)^2)

Cosine or Jaccard similarity between adjacency matrices of two layers.

Variables:

A, A = adjacency matrices for layers and ⟨ · , · ⟩ = Frobenius inner product ‖ · ‖ = Frobenius norm

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier ()
- **layer_j** – Second layer identifier ()
- **method** – 'cosine' or 'jaccard'

Returns
Similarity value between 0 and 1

Examples

```
>>> similarity = layer_similarity(network, 'L1', 'L2', method='cosine')
```

Reference:
De Domenico et al. (2013), Physical Review X 3(4), 041022

py3plex.algorithms.statistics.multilayer_statistics.**multilayer_betweenness_surface**(*network: Any*, *normalized: bool = True*, *weight: str | None = None*) → ndarray

Calculate multilayer betweenness surface (tensor representation).

Computes betweenness centrality for each node-layer pair and organizes the results as a 2D array (nodes × layers) that can be visualized as a heatmap or surface plot.

Formula: Surface[i,] = B

where B is the betweenness centrality of node i in layer .

Parameters

- **network** – py3plex multi_layer_network object
- **normalized** – Whether to normalize betweenness values
- **weight** – Edge weight attribute name (None for unweighted)

Returns
2D numpy array of shape (num_nodes, num_layers) containing betweenness values. Also returns tuple of (node_labels, layer_labels) for axis labeling.

Examples

```
>>> surface, (nodes, layers) = multilayer_betweenness_surface(network)
>>> import matplotlib.pyplot as plt
>>> plt.imshow(surface, aspect='auto', cmap='viridis')
>>> plt.xlabel('Layers')
>>> plt.ylabel('Nodes')
>>> plt.xticks(range(len(layers)), layers)
>>> plt.yticks(range(len(nodes)), nodes)
>>> plt.colorbar(label='Betweenness Centrality')
>>> plt.title('Multilayer Betweenness Surface')
>>> plt.show()
```

Reference:
De Domenico et al. (2015), "Structural reducibility of multilayer networks"

`py3plex.algorithms.statistics.multilayer_statistics.`**`multilayer_clustering_coefficient`**`(`*networ* *Any, node: Any | None = None)* → float | Dict[A float]

Calculate multilayer clustering coefficient (C).

Formula: C = T / T

Extends transitivity to account for triangles that span multiple layers.

Variables:
T = number of closed triplets (triangles) involving node i across all layers T = maximum possible triplets = k(k - 1)/2 for undirected networks Average over all nodes: C = (1/N) C

Parameters

- **network** – py3plex multi_layer_network object
- **node** – If specified, compute for single node; otherwise compute for all

Returns
Clustering coefficient value or dict of values per node

Examples

```
>>> clustering = multilayer_clustering_coefficient(network)
>>> node_clustering = multilayer_clustering_coefficient(network, node='A')
```

Reference:

Battiston et al. (2014), Section III.C

`py3plex.algorithms.statistics.multilayer_statistics.`**`multilayer_modularity`**(*network: Any, communities: Dict[Tuple[Any, Any], int], gamma: float | Dict[Any, float] = 1.0, omega: float | ndarray = 1.0, weight: str = 'weight'*) → float

Calculate multilayer modularity (Q).

This is a wrapper for the existing multilayer_modularity implementation in py3plex.algorithms.community_detection.multilayer_modularity.

Formula: Q = (1/2) [(A - P) +] (g, g)

Extension of Newman-Girvan modularity to multiplex networks (Mucha et al., 2010). Measures community quality across layers.

Variables:
= total edge weight in supra-network A = adjacency matrix element for layer P = kk/(2m) is the null model (configuration model) = resolution parameter for layer = inter-layer coupling strength = Kronecker delta (1 if =, 0 otherwise) = Kronecker delta (1 if i=j, 0 otherwise) (g, g) = 1 if node i in layer and node j in layer are in same community

Parameters

- **network** – py3plex multi_layer_network object
- **communities** – Dictionary mapping (node, layer) tuples to community IDs
- **gamma** – Resolution parameter(s)
- **omega** – Inter-layer coupling strength
- **weight** – Edge weight attribute

Returns
Modularity value Q

Examples

```
>>> communities = {('A', 'L1'): 0, ('B', 'L1'): 0, ('C', 'L1'): 1}
>>> Q = multilayer_modularity(network, communities)
```

Reference:
Mucha et al. (2010), Science 328(5980), 876-878

`py3plex.algorithms.statistics.multilayer_statistics.`**`multilayer_motif_frequency`**`(`*network: Any, motif_size: int = 3*`)` → Dict[str, float]

Calculate multilayer motif frequency (f).

Formula: f = n / n

Frequency of recurring subgraph patterns across layers.

Variables:
n = count of motif type m n = total count of all motifs

Note: This is a simplified implementation counting basic patterns (intra-layer vs. inter-layer triangles). Complete multilayer motif enumeration includes many more configurations and is computationally expensive.

Parameters

- **network** – py3plex multi_layer_network object
- **motif_size** – Size of motifs to count (default: 3 for triangles)

Returns
Dictionary of motif type frequencies

Examples

```
>>> motifs = multilayer_motif_frequency(network, motif_size=3)
```

Reference:
Battiston et al. (2014), Section IV

`py3plex.algorithms.statistics.multilayer_statistics.`**`multiplex_betweenness_centrality`**(*network: Any*, *normalized: bool = True*, *weight: str | None = None*) → Dict[Tu Any], float]

Calculate multiplex betweenness centrality.

Computes betweenness centrality on the supra-graph, accounting for paths that traverse inter-layer couplings. This extends the standard betweenness definition to multiplex networks where paths can cross layers.

Formula: B = ((i) /)

where is the total number of shortest paths from s to t, and (i) is the number of those paths passing through node i in layer .

Parameters

- **`network`** – py3plex multi_layer_network object
- **`normalized`** – Whether to normalize by the number of node pairs
- **`weight`** – Edge weight attribute name (None for unweighted)

Returns

Dictionary mapping (node, layer) tuples to betweenness centrality values

Examples

```
>>> betweenness = multiplex_betweenness_centrality(network)
>>> top_nodes = sorted(betweenness.items(), key=lambda x: x[1],␣
↪reverse=True)[:5]
```

Reference:

De Domenico et al. (2015), "Structural reducibility of multilayer networks" py3plex.algorithms.statistics.multilayer_statistics.**multiplex_closeness_centrality**(*network: Any*, *normalized: bool = True*, *weight: str / None = None*, *variant: str = 'standard'*) → Dict[Tuple[Any, Any], float]

Calculate multiplex closeness centrality.

Computes closeness centrality on the supra-graph, where shortest paths can traverse inter-layer edges. This captures how quickly a node-layer can reach all other node-layers in the multiplex network.

Standard closeness formula: C = (N*L - 1) / d(i, j)

Harmonic closeness formula: HC = 1/d(i, j)

where d(i, j) is the shortest path distance from node i in layer to node j in layer , and N*L is the total number of node-layer pairs.

Parameters

- **network** – py3plex multi_layer_network object
- **normalized** – Whether to normalize by network size
- **weight** – Edge weight attribute name (None for unweighted)
- **variant** – Closeness variant to use. Options: - 'standard': Classic closeness (reciprocal of sum of distances).

  Can produce biased values for nodes in disconnected components.

  – 'harmonic': Harmonic closeness (sum of reciprocal distances). Recommended for disconnected multilayer networks.

  – 'auto': Automatically selects 'harmonic' if the network has multiple connected components, otherwise uses 'standard'.

  Default is 'standard' for backward compatibility.

Returns

Dictionary mapping (node, layer) tuples to closeness centrality values

Examples

```
>>> closeness = multiplex_closeness_centrality(network)
>>> central_nodes = {k: v for k, v in closeness.items() if v > 0.5}
```

```
>>> # For disconnected networks, use harmonic variant
>>> closeness = multiplex_closeness_centrality(network, variant='harmonic
↪')
```

Reference:
De Domenico et al. (2015), "Structural reducibility of multilayer networks" Boldi, P., & Vigna, S. (2014). Axioms for Centrality. Internet Math.

`py3plex.algorithms.statistics.multilayer_statistics.`**`multiplex_rich_club_coefficient`**`(`*network: Any*, *k: int*, *normalized: bool = True*`)` → float

Calculate multiplex rich-club coefficient.

Measures the tendency of high-degree nodes to be more densely connected to each other than expected by chance, accounting for the multiplex structure.

Formula: (k) = E(>k) / (N(>k) * (N(>k)-1) / 2)

where E(>k) is the number of edges among nodes with overlapping degree > k, and N(>k) is the number of such nodes.

Parameters

- **network** – py3plex multi_layer_network object
- **k** – Degree threshold
- **normalized** – Whether to normalize by random expectation

Returns
Rich-club coefficient value

Examples

```
>>> rich_club = multiplex_rich_club_coefficient(network, k=10)
>>> print(f"Rich-club coefficient: {rich_club:.3f}")
```

Reference:
Alstott et al. (2014), "powerlaw: A Python Package for Analysis of Heavy-Tailed

Distributions" Extended to multiplex networks

py3plex.algorithms.statistics.multilayer_statistics.**node_activity**(*network: Any*, *node: Any*) → float

Calculate node activity (a).

Formula: a = (1/L) (v V)

Fraction of layers in which node i is active (has at least one connection).

Variables:

L = total number of layers (v V) = indicator function (1 if node i is active in layer , 0 otherwise) V = set of active nodes in layer

Parameters

- **network** – py3plex multi_layer_network object
- **node** – Node identifier

Returns

Activity value between 0 and 1

Examples

```
>>> activity = node_activity(network, 'A')
```

Reference:

Kivelä et al. (2014), J. Complex Networks 2(3), 203-271

Contracts:

- Precondition: network must not be None
- Precondition: node must not be None
- Postcondition: result is in [0, 1] (fraction of layers)
- Postcondition: result is not NaN

py3plex.algorithms.statistics.multilayer_statistics.**percolation_threshold**(*network: Any*, *removal_strategy: str = 'random'*, *trials: int = 10*) → float

Estimate percolation threshold for the multiplex network.

Determines the fraction of nodes that must be removed before the network fragments into disconnected components. Uses sampling to estimate threshold.

Parameters

- **network** – py3plex multi_layer_network object
- **removal_strategy** – 'random', 'degree', or 'betweenness'
- **trials** – Number of trials for averaging

Returns

Estimated percolation threshold (fraction of nodes)

Examples

```
>>> threshold = percolation_threshold(network, removal_strategy='degree')
>>> print(f"Percolation threshold: {threshold:.2%}")
```

Reference:

Buldyrev et al. (2010), "Catastrophic cascade of failures in interdependent networks"

py3plex.algorithms.statistics.multilayer_statistics.**resilience**(*network: Any*, *perturbation_type: str = 'layer_removal'*, *perturbation_param: str | float | None = None*) → float

Calculate resilience (R).

Formula: R = S' / S_0

Ratio of largest connected component after perturbation to original size.

Variables:

S_0 = size of largest connected component in original network S' = size of largest connected component after perturbation

Perturbation types:

1. Layer removal: Remove all nodes/edges in a specific layer
2. Coupling removal: Remove a fraction of inter-layer edges

Properties:

R = 1 indicates full resilience (no impact from perturbation) R = 0 indicates complete fragmentation 0 < R < 1 indicates partial resilience

Parameters

- **network** – py3plex multi_layer_network object
- **perturbation_type** – 'layer_removal' or 'coupling_removal'
- **perturbation_param** – Layer to remove or fraction of inter-layer edges

Returns

Resilience ratio between 0 and 1

Examples

```
>>> r = resilience(network, 'layer_removal', perturbation_param='L1')
>>> r = resilience(network, 'coupling_removal', perturbation_param=0.5)
```

Reference:
Buldyrev et al. (2010), Nature 464, 1025-1028

`py3plex.algorithms.statistics.multilayer_statistics.`**`supra_laplacian_spectrum`**`(`*network: Any*, *k: int = 10*`)` → ndarray

Calculate supra-Laplacian spectrum ().

Formula: = -

Eigenvalue spectrum of the supra-Laplacian matrix; captures diffusion properties. Uses sparse eigenvalue computation when beneficial.

Variables:
= supra-adjacency matrix (NL × NL block matrix containing all layers and inter-layer couplings) = supra-degree matrix (diagonal matrix with row sums of) = supra-Laplacian matrix = $\{{}_0, {}_1, \ldots, {}_1\}$ with $0 = {}_0 \; {}_1 \; \ldots \; {}_1$

Parameters

- **network** – py3plex multi_layer_network object
- **k** – Number of smallest eigenvalues to compute

Returns
Array of k smallest eigenvalues

Examples

```
>>> spectrum = supra_laplacian_spectrum(network, k=10)
```

Reference:
De Domenico et al. (2013), Gomez et al. (2013)

Notes

- Uses sparse eigsh() for sparse matrices (more efficient)
- Falls back to dense computation for small matrices (n < 100) or when k is large relative to n
- Laplacian is always symmetric for undirected graphs (PSD with smallest eigenvalue = 0)

py3plex.algorithms.statistics.multilayer_statistics.**targeted_layer_removal**(*network: Any*, *layer: str*, *return_resilience: bool = False*) → Any | Tuple[Any, float]

Simulate targeted removal of an entire layer.

Removes all edges in a specified layer and returns the modified network or resilience score.

Parameters

- **network** – py3plex multi_layer_network object
- **layer** – Layer identifier to remove
- **return_resilience** – If True, return resilience score instead of network

Returns

Modified network or resilience score

Examples

```
>>> resilience = targeted_layer_removal(network, 'social', return_
↪resilience=True)
>>> print(f"Resilience after removing social layer: {resilience:.3f}")
```

Reference:

Buldyrev et al. (2010), "Catastrophic cascade of failures"

py3plex.algorithms.statistics.multilayer_statistics.**unique_redundant_edges**(*network: Any*, *layer_i: str*, *layer_j: str*) → Tuple[int, int]

Count unique and redundant edges between two layers.

Returns the number of edges unique to the first layer and the number of edges present in both layers (redundant).

Parameters

- **network** – py3plex multi_layer_network object
- **layer_i** – First layer identifier
- **layer_j** – Second layer identifier

Returns
Tuple of (unique_edges, redundant_edges)

Examples

```
>>> unique, redundant = unique_redundant_edges(network, 'social', 'work')
>>> print(f"Unique: {unique}, Redundant: {redundant}")
```

`py3plex.algorithms.statistics.multilayer_statistics.`**`versatility_centrality`**(*network: Any, centrality_type: str = 'degree', alpha: Dict[str, float] | None = None*) → Dict[Any, float]

Calculate versatility centrality (V).

Formula: V = w C

Weighted combination of node centrality values across layers; measures overall influence.

Variables:
w = weight for layer (typically 1/L for uniform weighting, w = 1) C = centrality of node i in layer (can be degree, betweenness, closeness, etc.)

Parameters

- **centrality_type** – Type of centrality ('degree', 'betweenness', 'closeness')
- **alpha** – Layer weights (default: uniform weights)

Returns
Dictionary mapping nodes to versatility centrality values

Examples

```
>>> versatility = versatility_centrality(network, centrality_type='degree
↪')
```

Reference:
De Domenico et al. (2015), Nature Communications 6, 6868

`py3plex.algorithms.statistics.topology.`**`basic_pl_stats`**(*degree_sequence: List[int]*) → Tuple[float, float]

Calculate basic power law statistics for a degree sequence.

Parameters
degree_sequence – Degree sequence of individual nodes

Returns
Tuple of (alpha, sigma) values

`py3plex.algorithms.statistics.topology.`**`plot_power_law`**(*degree_sequence: List[int]*, *title: str*, *xlabel: str*, *plabel: str*, *ylabel: str = 'Number of nodes'*, *formula_x: int = 70*, *formula_y: float = 0.05*, *show: bool = True*, *use_normalization: bool = False*) → Any

`py3plex.algorithms.statistics.correlation_networks.`**`default_correlation_to_network`**(*matrix: ndarray*, *input_type: str = 'matrix'*, *preprocess: str = 'standard'*) → ndarray

Convert correlation matrix to network using optimal thresholding.

Parameters

- **matrix** – Input data matrix
- **input_type** – Type of input (default: "matrix")
- **preprocess** – Preprocessing method (default: "standard")

Returns

Binary adjacency matrix

`py3plex.algorithms.statistics.correlation_networks.pick_threshold(`*matrix: ndarray*`)` → float

Pick optimal threshold for correlation network construction.

Parameters
`matrix` – Input data matrix

Returns
Optimal threshold value

`py3plex.algorithms.statistics.basic_statistics.core_network_statistics(`*G: Graph, labels: Any | None = None, name: str = 'example'*`)` → DataFrame

Compute core statistics for a network.

Parameters

- `G` – NetworkX graph to analyze
- `labels` – Optional label matrix with shape attribute
- `name` – Name identifier for the network (default: "example")

Returns
DataFrame containing network statistics

`py3plex.algorithms.statistics.basic_statistics.ensure(`**args, **kwargs*`)`

`py3plex.algorithms.statistics.basic_statistics.identify_n_hubs(`*G: Graph, top_n: int = 100, node_type: str | None = None*`)` → Dict[Any, int]

Identify the top N hub nodes in a network based on degree centrality.

Parameters

- `G` – NetworkX graph to analyze
- `top_n` – Number of top hubs to return (default: 100)
- `node_type` – Optional filter for specific node type

Returns
Dictionary mapping node identifiers to their degree values

Contracts:

- Precondition: top_n must be positive

- Postcondition: result has at most top_n entries
- Postcondition: all degree values are non-negative integers

`py3plex.algorithms.statistics.basic_statistics.require(*args, **kwargs)`

Multilayer Algorithms

Multilayer/Multiplex Network Centrality Measures

This module implements various centrality measures for multilayer and multiplex networks, following standard definitions from multilayer network analysis literature.

Weight Handling Notes:

For path-based centralities (betweenness, closeness): - Edge weights from the supra-adjacency matrix are converted to distances (inverse of weight) - NetworkX algorithms use these distances for shortest path computation - betweenness_centrality: weight parameter specifies the edge attribute for path computation - closeness_centrality: distance parameter specifies the edge attribute for path computation

For disconnected graphs: - closeness_centrality uses wf_improved parameter (Wasserman-Faust scaling) by default - When wf_improved=True, scores are normalized by reachable nodes only - When wf_improved=False, unreachable nodes contribute infinite distance

Weight Constraints: - Weights should be positive (> 0) for shortest path algorithms - Zero or negative weights will cause undefined behavior - For unweighted analysis, use weighted=False parameters

Authors: py3plex contributors Date: 2025

`class py3plex.algorithms.multilayer_algorithms.centrality.MultilayerCentrality(network: Any)`

Bases: `object`

Class for computing centrality measures on multilayer networks.

This class provides implementations of various centrality measures specifically designed for multilayer/multiplex networks, including degree-based, eigenvector-based, and path-based measures.

`accessibility_centrality(h=2)`

Compute accessibility centrality (entropy-based reach within h steps).

Accessibility measures the diversity of nodes reachable within h steps using entropy of the probability distribution.

Access_r = exp(H_r) where H_r is the entropy of the h-step distribution

Parameters
h – Number of steps (default: 2)

Returns
{(node, layer): accessibility}

Return type
dict

Note

Accessibility is measured as the effective number of h-step destinations, using the entropy of the random walk distribution.

Dangling Node Handling: For nodes with no outgoing edges (dangling nodes), this implementation uses uniform teleportation to all nodes. This differs from the original Travencolo-Costa definition which does not include teleportation. The teleportation ensures a well-defined random walk distribution but may affect accessibility values for nodes near dangling nodes compared to implementations that handle dangling nodes differently.

References

- Travencolo, B. A. N., & Costa, L. D. F. (2008). Accessibility in complex networks. Physics Letters A, 373(1), 89-95.

`aggregate_to_node_level`(*node_layer_centralities*, *method='sum'*, *weights=None*)

Aggregate node-layer centralities to node level.

Parameters

- **`node_layer_centralities`** – dict with {(node, layer): value} entries
- **`method`** – 'sum', 'mean', 'max', 'weighted_sum'
- **`weights`** – dict with {layer: weight} for weighted_sum method

Returns

{node: aggregated_value}

Return type

dict

`bridging_centrality`()

Compute bridging centrality for nodes in the supra-graph.

Bridging centrality combines betweenness with the bridging coefficient, which measures how much a node connects sparse regions of the network.

Bridging(u) = B(u) * BCoeff(u) where BCoeff(u) = (1 / k_u) * sum_{v in N(u)} [1 / k_v]

Returns

{(node, layer): bridging_centrality}

Return type

dict

Note

Nodes with higher bridging centrality act as important bridges connecting different parts of the network.

`collective_influence`(*radius=2*)

Compute collective influence (CI_) for multiplex networks.

Collective influence identifies influential spreaders by considering not just immediate neighbors but also nodes at distance .

CI_(u) = (k_u - 1) * sum_{v in Ball_(u)} (k_v - 1)

Parameters
`radius` – Radius for the ball boundary (default: 2)

Returns
{(node, layer): collective_influence}

Return type
dict

> **Note**
>
> Uses overlapping degree across all layers for each physical node.

communicability_betweenness_centrality(*normalized=True*)

Compute communicability betweenness centrality on the supra-graph.

This measure quantifies how much a node contributes to the communicability between other pairs of nodes. It uses the matrix exponential to account for all walks between nodes.

Parameters
`normalized` – If True (default), normalize scores by dividing by the maximum value (min-max scaling to [0, 1] range). Note that this is simple rescaling, not the theoretical Estrada-Hatano normalization from the original literature.

Returns
{(node, layer): communicability_betweenness}

Return type
dict

> **Note**
>
> This implementation uses NetworkX's communicability_betweenness_centrality, which operates on unweighted graphs. Edge weights from the supra-adjacency matrix are used only to determine edge existence (weight > 0), not for weighted communicability computation. For truly weighted communicability analysis, a custom implementation using the weighted matrix exponential would be required.
>
> This is computationally expensive as it requires computing the matrix exponential multiple times. For large networks, this may take significant time.

current_flow_betweenness_centrality()

Compute current-flow betweenness centrality via supra Laplacian pseudoinverse.

This measure is based on the electrical current flow through each node.

Returns
{(node, layer): current_flow_betweenness}

Return type
dict

current_flow_closeness_centrality()

Compute current-flow closeness centrality via supra Laplacian pseudoinverse.

This measure is based on the resistance distance in electrical networks.

Returns
{(node, layer): current_flow_closeness}

Return type
dict

edge_betweenness_centrality(*normalized=True*)

Compute edge betweenness centrality on the supra-graph.

Edge betweenness measures the fraction of shortest paths that pass through each edge.

Returns
{(source, target): edge_betweenness}

Return type
dict

Note

This returns edge-level centrality, not node-level.

flow_betweenness_centrality(*samples=100*)

Compute flow betweenness based on maximum flow sampling.

Flow betweenness measures how much flow passes through a node when maximizing flow between randomly sampled source-target pairs.

Parameters
samples – Number of source-target pairs to sample (default: 100)

Returns
{(node, layer): flow_betweenness}

Return type
dict

Note

This is a sampling-based approximation of flow betweenness. The implementation samples random pairs of supra-graph nodes and computes the maximum flow through each intermediate node.

Unlike classical Freeman flow betweenness which uses a normalization factor based on the number of nodes, this implementation returns the average flow per sampled pair without additional normalization. For multilayer networks, the number of supra-graph nodes (N * L for N physical nodes and L layers) affects the raw values.

For large networks, this sampling approach is more computationally feasible than computing exact flow betweenness for all pairs.

harmonic_closeness_centrality()

Compute harmonic closeness centrality on the supra-graph.

Harmonic closeness handles disconnected graphs better than standard closeness by summing the reciprocals of distances instead of taking reciprocal of sum.

HC(u) = sum_{vu} (1 / d(u,v)) for finite distances

Returns
{(node, layer): harmonic_closeness}

Return type
dict

> **Note**
>
> This measure naturally handles disconnected components as unreachable nodes contribute 0 (instead of infinity) to the sum.
>
> Weight Interpretation: Edge weights from the supra-adjacency matrix are interpreted as connection strengths (larger weight = stronger connection = shorter distance). They are converted to distances via 1/weight for shortest path computation. If your edge weights already represent distances, do not use this function directly—you would need to invert them first or use the distance values directly in a custom computation.

hits_centrality(*max_iter=1000*, *tol=1e-06*)

Compute HITS (hubs and authorities) centrality on the supra-graph.

For undirected networks, this equals eigenvector centrality. For directed networks, computes separate hub and authority scores.

Parameters

- **max_iter** – Maximum number of iterations.
- **tol** – Tolerance for convergence.

Returns

If directed network: {'hubs': {(node, layer): score}, 'authorities': {(node, layer): score}}
If undirected network: {(node, layer): score} (equivalent to eigenvector centrality)

Return type
dict

information_centrality()

Compute Information Centrality (Stephenson-Zelen style) on the supra-graph.

Information centrality measures the importance of a node based on the information flow through the network. It uses the inverse of a modified Laplacian matrix.

Returns
{(node, layer): information_centrality}

Return type
dict

Note

This implementation returns values for each node-layer pair in the supra-graph, not aggregated physical node values. To obtain physical node-level information centrality, use the aggregate_to_node_level() method on the returned dictionary.

The implementation uses NetworkX's information_centrality for computation. Falls back to harmonic closeness if information centrality computation fails.

Information centrality is defined only for undirected graphs. For directed networks, the graph is symmetrized with a warning.

References

- Stephenson, K., & Zelen, M. (1989). Rethinking centrality: Methods and examples. Social Networks, 11(1), 1-37.

katz_bonacich_centrality(*alpha=0.1*, *beta=None*)

Compute Katz-Bonacich centrality on the supra-graph.

z = _{t=0}^∞ ^t M^t b = (I - M)^{-1} b

Parameters

- **alpha** – Attenuation parameter (should be < 1/(M)). If None, automatically computes a safe value as 0.85/(M) where (M) is the spectral radius.
- **beta** – Exogenous preference vector. If None, uses vector of ones.

Returns
{(node, layer): centrality_value}

Return type
dict

layer_degree_centrality(*layer: str | None = None*, *weighted: bool = False*, *direction: str = 'out'*) → Dict[str | Tuple[str, str], float]

Compute layer-specific degree (or strength) centrality.

For undirected networks:
k^[]_i = _j 1(A^[]_ij > 0) [unweighted] s^[]_i = _j A^[]_ij [weighted]

For directed networks:
k^[,out]_i = _j 1(A^[]_ij > 0) [out-degree] k^[,in]_i = _j 1(A^[]_ji > 0) [in-degree]

Parameters

- **layer** – Layer to compute centrality for. If None, compute for all layers.
- **weighted** – If True, compute strength instead of degree.
- **direction** – 'out', 'in', or 'both' for directed networks.

Returns

{(node, layer): centrality_value} if layer is None,
{node: centrality_value} if layer is specified.

Return type
dict

load_centrality()

Compute load centrality (shortest-path load).

Load centrality measures the fraction of shortest paths that pass through each node, counting all paths (not just unique pairs).

Load[k] = sum over all (s,t) pairs of [number of shortest paths through k / total shortest paths]

Returns
{(node, layer): load_centrality}

Return type
dict

> **Note**
>
> This is similar to betweenness but counts all paths rather than normalizing by the number of node pairs.

local_efficiency_centrality()

Compute local efficiency centrality.

Local efficiency measures how efficiently information is exchanged among a node's neighbors when the node is removed. It quantifies the fault tolerance of the network.

LE(u) = (1 / (|N_u|*(|N_u|-1))) * sum_{ij in N_u} [1 / d(i,j)]

Returns
{(node, layer): local_efficiency}

Return type
dict

> **Note**
>
> For nodes with less than 2 neighbors, local efficiency is 0.

lp_aggregated_centrality(*layer_centralities*, *p=2*, *weights=None*, *exclude_missing=True*)

Compute Lp-aggregated per-layer centrality.

Aggregates per-layer centrality values using Lp norm.

C_i = (sum_{ in L_i} w_ * |c^[i]|^p)^{1/p} for p < ∞ C_i = max_{ in L_i} (w_ * |c^[i]|) for p = ∞

where L_i is the set of layers where node i exists.

Parameters

- **layer_centralities** – dict of {layer: {node: centrality}} or {(node, layer): centrality}
- **p** – Lp norm parameter (default: 2). Use float('inf') for L-infinity

norm.

- **weights** – dict of {layer: weight}. If None, uniform weights are used.
- **exclude_missing** – If True (default), only aggregate over layers where the node exists (has a centrality value). If False, treat nodes absent from a layer as having zero centrality, which may bias scores for nodes with sparse layer participation.

Returns
{node: aggregated_centrality}

Return type
dict

> **Note**
>
> This is a framework for aggregating any per-layer centrality measure. Input can be degree, PageRank, eigenvector, etc.
>
> Sparse Layer Participation: When exclude_missing=True (default), nodes that only appear in a subset of layers are aggregated only over those layers where they exist. This prevents bias against nodes with sparse layer participation. When exclude_missing=False, missing layers contribute zero to the aggregation, which may penalize nodes that don't appear in all layers.

multilayer_betweenness_centrality(*normalized=True*, *endpoints=False*)

Compute betweenness centrality on the supra-graph.

For each node-layer pair (i,), computes the fraction of shortest paths between all pairs of nodes that pass through (i,).

Parameters

- **normalized** – Whether to normalize the betweenness values.
- **endpoints** – Whether to include endpoints in path counts.

Returns
{(node, layer): betweenness_centrality}

Return type
dict

> **Note**
>
> This is computationally expensive for large networks as it requires computing shortest paths between all pairs of nodes.
>
> Weight handling: Edge weights from the supra-adjacency matrix are converted to distances (1/weight) for shortest path computation. Weights must be positive (> 0). Zero or negative weights will cause undefined behavior.

multilayer_closeness_centrality(*normalized=True*, *wf_improved=True*, *variant='standard'*)

Compute closeness centrality on the supra-graph.

For each node-layer pair (i,), computes:

Standard closeness:
C_c(i,) = (n-1) / _{(j,)} d((i,), (j,))

Harmonic closeness (recommended for disconnected networks):
HC(i,) = _{(j,)(i,)} 1/d((i,), (j,))

where d((i,), (j,)) is the shortest path distance in the supra-graph.

Parameters

- **normalized** – This parameter is kept for API compatibility but has no effect. Standard closeness is always normalized by (n-1) per the NetworkX implementation. Harmonic closeness returns unnormalized sums of reciprocal distances by definition.
- **wf_improved** – If True, use Wasserman-Faust improved closeness scaling for disconnected graphs. Default is True. This affects the magnitude and ordering of scores in graphs with multiple components (e.g., low interlayer coupling). See NetworkX documentation for details. Only used when variant='standard'.
- **variant** – Closeness variant to use. Options: - 'standard': Classic closeness (reciprocal of sum of distances).

  For disconnected graphs, uses Wasserman-Faust scaling if wf_improved=True. Can produce biased values for nodes in small or disconnected components.

  - 'harmonic': Harmonic closeness (sum of reciprocal distances). Mathematically well-defined for disconnected networks: unreachable nodes contribute 0 instead of infinity. Recommended for disconnected multilayer networks.
  - 'auto': Automatically selects 'harmonic' if the supra-graph has multiple connected components, otherwise uses 'standard'.

  Default is 'standard' for backward compatibility.

Returns
{(node, layer): closeness_centrality}

Return type
dict

Note

This implementation uses NetworkX's shortest path algorithms on the supra-graph representation. For large networks, this can be computationally expensive.

For disconnected multilayer graphs (e.g., layers with no inter-layer coupling, or networks with isolated components), use variant='harmonic' or variant='auto' to get mathematically consistent closeness values. The harmonic variant naturally handles unreachable nodes by summing 1/d for finite distances only (infinite distances contribute 0).

Weight Interpretation: Edge weights from the supra-adjacency matrix are interpreted as connection strengths (larger weight = stronger connection). They are converted to distances via 1/weight for shortest path computation. If your edge weights already represent distances, use them directly without this function's weight inversion.

Examples

```
>>> # For connected networks, standard closeness works well
>>> closeness = calc.multilayer_closeness_centrality(variant=
↪'standard')
```

```
>>> # For potentially disconnected networks, use harmonic
>>> closeness = calc.multilayer_closeness_centrality(variant=
↪'harmonic')
```

```
>>> # Let the algorithm decide based on connectivity
>>> closeness = calc.multilayer_closeness_centrality(variant='auto')
```

References

- Wasserman, S., & Faust, K. (1994). Social Network Analysis.
- Boldi, P., & Vigna, S. (2014). Axioms for Centrality. Internet Math.
- De Domenico, M., et al. (2015). Structural reducibility of multilayer networks.

multiplex_coreness()

Alias for multiplex_k_core for compatibility.

Returns
{(node, layer): core_number}

Return type
dict

multiplex_eigenvector_centrality(*max_iter: int = 1000*, *tol: float = 1e-06*) → Dict

Compute multiplex eigenvector centrality (node-layer level).

x = (1/_max) * M * x where x_{i} is the centrality of node i in layer , and _max is the spectral radius of the supra-adjacency matrix M.

Parameters

- **max_iter** – Maximum number of iterations.
- **tol** – Tolerance for convergence.

Returns
{(node, layer): centrality_value}

Return type
dict

multiplex_eigenvector_versatility(*max_iter: int = 1000*, *tol: float = 1e-06*) → Dict

Compute node-level eigenvector versatility.

x_i = _ x_{i}

Parameters

- **max_iter** – Maximum number of iterations.
- **tol** – Tolerance for convergence.

Returns
{node: versatility_value}

Return type
dict

multiplex_k_core()

Compute multiplex k-core decomposition.

A node belongs to the k-core if it has at least k neighbors in the multilayer network. This implementation computes the core number for each node-layer pair.

Returns
{(node, layer): core_number}

Return type
dict

overlapping_degree_centrality(*weighted: bool = False*) → Dict

Compute overlapping degree/strength centrality (node level).

k^{over}_i = _ k^[]_i [unweighted] s^{over}_i = _ s^[]_i [weighted]

Parameters
weighted – If True, compute overlapping strength.

Returns
{node: centrality_value}

Return type
dict

pagerank_centrality(*damping=0.85*, *max_iter=1000*, *tol=1e-06*)

Compute PageRank centrality on the supra-graph.

Uses the standard PageRank algorithm on the supra-adjacency matrix representing the multilayer network. Properly handles dangling nodes (nodes with no outgoing edges) via teleportation.

This implementation preserves sparsity when possible for memory efficiency.

Parameters

- **damping** – Damping parameter (typically 0.85).
- **max_iter** – Maximum number of iterations.
- **tol** – Tolerance for convergence.

Returns
{(node, layer): centrality_value}

Return type
dict

Mathematical Invariants:

- PageRank values sum to 1.0 (within tol=1e-6)

- All values are non-negative
- Converges for strongly connected components or with teleportation

participation_coefficient(*weighted: bool = False*) → Dict

Compute participation coefficient across layers.

Measures how evenly a node's degree is distributed across layers: P_i = 1 - _ (k^[]_i / k^{over}_i)^2

Set P_i = 0 if k^{over}_i = 0.

Parameters
weighted – If True, use strength instead of degree.

Returns
{node: participation_coefficient}

Return type
dict

percolation_centrality(*edge_activation_prob=0.5*, *trials=100*)

Compute percolation centrality using bond percolation Monte Carlo simulation.

This implementation measures the average relative component size that a node belongs to across multiple bond percolation realizations, providing an estimate of a node's importance for network connectivity under random edge failures.

Parameters
- **edge_activation_prob** – Probability that an edge is active (default: 0.5)
- **trials** – Number of Monte Carlo trials (default: 100)

Returns
{(node, layer): percolation_centrality}

Return type
dict

> **Note**
>
> This is a component-size-based percolation measure, not the path-based percolation betweenness from the original Piraveenan et al. literature, which requires recomputing betweenness on each percolated realization. The original percolation centrality is computationally expensive ($O(n^3)$ per trial), so this implementation provides a more efficient alternative that captures related connectivity information.
>
> Values are normalized to [0, 1] range where higher values indicate nodes that tend to belong to larger connected components across percolation realizations.

References

- Piraveenan, M., Prokopenko, M., & Hossain, L. (2013). Percolation centrality: Quantifying graph-theoretic impact of nodes during percolation in networks. PloS one, 8(1), e53095.

spreading_centrality(*beta=0.2*, *mu=0.1*, *trials=50*, *steps=100*)

Compute spreading (epidemic) centrality using SIR model.

Spreading centrality measures how influential a node is in spreading information or disease through the network, based on Monte Carlo simulations of discrete-time SIR dynamics.

Parameters

- **beta** – Infection rate per edge per time step (default: 0.2)
- **mu** – Recovery rate per time step (default: 0.1)
- **trials** – Number of simulation trials per node (default: 50)
- **steps** – Maximum simulation steps (default: 100)

Returns

{(node, layer): spreading_centrality}

Return type

dict

Note

This measures the average outbreak size (fraction of nodes ever infected) when seeding the epidemic from each node. Values are normalized by the total number of nodes, producing scores in the range [1/n, 1] where n is the number of supra-graph nodes.

This is an empirical simulation-based measure, not normalized by theoretical epidemic threshold or branching factor as in some literature definitions. The normalization allows comparison of relative spreading power within the network but may not be directly comparable across networks of different sizes or structures.

References

- Kitsak, M., et al. (2010). Identification of influential spreaders in complex networks. Nature physics, 6(11), 888-893.

subgraph_centrality()

Compute subgraph centrality via matrix exponential of the supra-adjacency matrix.

Subgraph centrality counts closed walks of all lengths starting and ending at each node. SC_i = (e^A)_ii where A is the adjacency matrix.

Returns

{(node, layer): subgraph_centrality}

Return type

dict

supra_degree_centrality(*weighted: bool = False*) → Dict

Compute supra degree/strength centrality (node-layer level).

k_{i} = _{j,} 1(M_{(i,),(j,)} > 0) [unweighted] s_{i} = _{j,} M_{(i,),(j,)} [weighted]

Parameters

weighted – If True, compute strength instead of degree.

Returns
{(node, layer): centrality_value}

Return type
dict

total_communicability()

Compute total communicability via matrix exponential.

Total communicability is the row sum of the matrix exponential: TC_i = sum_j (e^A)_ij

Returns
{(node, layer): total_communicability}

Return type
dict

py3plex.algorithms.multilayer_algorithms.centrality.**communicability_centrality**(*supra_matrix: spmatrix*, *normalize: bool = True*, *use_sparse: bool = True*, *max_iter: int = 100*, *tol: float = 1e-06*) → ndarray

Compute communicability centrality for each node-layer pair.

Communicability centrality measures the weighted sum of all walks between nodes, with exponentially decaying weights for longer walks. It is computed as the row sum of the matrix exponential:

c_i = sum_j (exp(A))_ij

This implementation uses scipy.sparse.linalg.expm_multiply for efficient sparse matrix exponential computation.

Parameters

- **supra_matrix** – Sparse supra-adjacency matrix (n x n).

- **normalize** – If True, normalize output to sum to 1.
- **use_sparse** – If True, use sparse matrix operations. Falls back to dense for matrices smaller than 10000 elements.
- **max_iter** – Maximum number of iterations for sparse approximation (currently unused).
- **tol** – Tolerance for convergence (currently unused).

Returns
Communicability centrality scores for each node-layer pair (n,).

Return type
np.ndarray

Raises
Py3plexMatrixError – If matrix is invalid (non-square, empty, etc.).

Example

```
>>> from py3plex.core import random_generators
>>> net = random_generators.random_multiplex_ER(50, 3, 0.1)
>>> A = net.get_supra_adjacency_matrix()
>>> comm = communicability_centrality(A)
>>> print(f"Communicability centrality computed for {len(comm)} node-
↪layer pairs")
```

py3plex.algorithms.multilayer_algorithms.centrality.**compute_all_centralities**(*network*, *include_path_based*, *include_advanced*=, *include_extended*=, *preset=None*, *wf_improved=Tr*, *closeness_variant='st*

Compute all available centrality measures for a multilayer network.

Parameters

- **network** – py3plex multi_layer_network object
- **include_path_based** – Whether to include computationally expensive path-based measures (betweenness, closeness). Default: False
- **include_advanced** – Whether to include advanced measures (HITS, current-flow, communicability, k-core). Default: False
- **include_extended** – Whether to include extended measures (information, accessibility, percolation, spreading, collective influence, load, flow betweenness, harmonic, bridging, local efficiency). Default: False
- **preset** – Convenience parameter to set all inclusion flags at once. Options: - 'basic': Only degree and eigenvector-based measures (default

behavior) - 'standard': Includes path-based measures - 'advanced': Includes path-based and advanced measures - 'all': Includes all measures (path-based, advanced, and extended) - None: Use individual flags (default)

- **wf_improved** – If True, use Wasserman-Faust improved scaling for closeness centrality in disconnected graphs. Default: True. Only used when closeness_variant='standard'.
- **closeness_variant** – Variant of closeness centrality to use. Options: - 'standard': Classic closeness (reciprocal of sum of distances).

  Uses Wasserman-Faust scaling if wf_improved=True.

  – 'harmonic': Harmonic closeness (sum of reciprocal distances). Recommended for disconnected multilayer networks.

  – 'auto': Automatically selects 'harmonic' if the network has disconnected components, otherwise uses 'standard'.

  Default: 'standard' for backward compatibility.

Returns

Dictionary containing all computed centrality measures with keys:

- Degree-based: "layer_degree", "layer_strength", "supra_degree", "supra_strength", "overlapping_degree", "overlapping_strength", "participation_coefficient", "participation_coefficient_strength"
- Eigenvector-based: "multiplex_eigenvector", "eigenvector_versatility", "katz_bonacich", "pagerank"
- Path-based (if include_path_based=True): "closeness", "betweenness"
- Advanced (if include_advanced=True): "hits", "current_flow_closeness", "current_flow_betweenness", "subgraph_centrality", "total_communicability", "multiplex_k_core"
- Extended (if include_extended=True): "information", "communicability_betweenness", "accessibility", "harmonic_closeness", "local_efficiency", "edge_betweenness", "bridging", "percolation", "spreading", "collective_influence", "load", "flow_betweenness"

Return type

dict

> **Note**
>
> Path-based, advanced, and extended measures are computationally expensive for large networks. Use flags or presets to control which measures are computed.
>
> For disconnected multilayer graphs (e.g., networks without inter-layer coupling, or with isolated components), use closeness_variant='harmonic' or 'auto' to get mathematically consistent closeness values.

Examples

```
>>> # Compute only basic measures (fast)
>>> results = compute_all_centralities(network)
```

```
>>> # Use preset for standard analysis
>>> results = compute_all_centralities(network, preset='standard')
```

```
>>> # Compute everything
>>> results = compute_all_centralities(network, preset='all')
```

```
>>> # Fine-grained control
>>> results = compute_all_centralities(
...     network,
...     include_path_based=True,
...     include_extended=True
... )
```

```
>>> # For disconnected multilayer networks, use harmonic closeness
>>> results = compute_all_centralities(
...     network,
...     include_path_based=True,
...     closeness_variant='harmonic'
... )
```

`py3plex.algorithms.multilayer_algorithms.centrality.`**`katz_centrality`**(*supra_matrix: spmatrix*, *alpha: float | None = None*, *beta: float = 1.0*, *tol: float = 1e-06*) → ndarray

Compute Katz centrality for each node-layer pair.

Katz centrality measures node influence by accounting for all paths with exponentially decaying weights. It is computed as:

x = (I - alpha * A)^{-1} * beta * 1

where alpha < 1/lambda_max(A) to ensure convergence. If alpha is not provided, it defaults to 0.85 / lambda_max(A).

Parameters

- **supra_matrix** – Sparse supra-adjacency matrix (n x n).
- **alpha** – Attenuation parameter. Must be less than 1/spectral_radius(A). If None, defaults to 0.85 / lambda_max(A).
- **beta** – Weight of exogenous influence (typically 1.0).
- **tol** – Tolerance for eigenvalue computation.

Returns

Katz centrality scores for each node-layer pair (n,).
Normalized to sum to 1.

Return type
np.ndarray

Raises
Py3plexMatrixError – If matrix is invalid or alpha is out of valid range.

Example

```
>>> from py3plex.core import random_generators
>>> net = random_generators.random_multiplex_ER(50, 3, 0.1)
>>> A = net.get_supra_adjacency_matrix()
>>> katz = katz_centrality(A)
>>> print(f"Katz centrality computed for {len(katz)} node-layer pairs")
>>> # With custom alpha
>>> katz_custom = katz_centrality(A, alpha=0.05)
```

Built-in multilayer centrality toolkit.

Implements core multilayer variants of centrality measures: - Multilayer PageRank - Multilayer betweenness centrality - Multilayer eigenvector centrality - Multiplex degree centrality

These algorithms properly account for the multilayer structure of networks.

All centrality functions now support first-class uncertainty estimation via the *uncertainty* parameter.

Authors: py3plex contributors Date: 2025

`py3plex.algorithms.centrality_toolkit.aggregate_centrality_across_layers`(*centrality_dict: Dict[Tuple, float], aggregation: str = 'sum'*) → Dict[Any, float]

Aggregate node centrality values across layers.

Given centrality scores for (node, layer) tuples, aggregate to get per-node scores.

Parameters

- **centrality_dict** – Dictionary mapping (node, layer) -> score
- **aggregation** – Aggregation method ('sum', 'mean', 'max', 'min')

Returns
Dictionary mapping node_id -> aggregated score

Example

```
>>> scores = {('A', 'L1'): 0.5, ('A', 'L2'): 0.3, ('B', 'L1'): 0.7}
>>> aggregate_centrality_across_layers(scores, 'mean')
{'A': 0.4, 'B': 0.7}
```

`py3plex.algorithms.centrality_toolkit.multilayer_betweenness_centrality`(*network: Any*, *normalized: bool = True*, *weight: str | None = None*) → Dict[Tuple, float]

Compute multilayer betweenness centrality.

Computes betweenness centrality on the supra-graph, where shortest paths can traverse multiple layers.

Parameters

- **network** – Multilayer network object
- **normalized** – Whether to normalize by number of pairs
- **weight** – Edge attribute to use as weight (None for unweighted)

Returns

Dictionary mapping (node, layer) tuples to betweenness scores

Algorithm:

For each pair of nodes (s,t), count the fraction of shortest paths passing through each node v:

BC(v) = _{svt} _{st}(v) / _{st}

where _{st} is the number of shortest paths from s to t, and _{st}(v) is the number passing through v.

References

- De Domenico, M., et al. (2015). "Ranking in interconnected multilayer networks reveals versatile nodes." Nature Communications, 6, 6868.

`py3plex.algorithms.centrality_toolkit.multilayer_eigenvector_centrality`(*network: Any*, *max_iter: int = 100*, *tol: float = 1e-06*) → Dict[Tuple, float]

Compute multilayer eigenvector centrality.

Computes the principal eigenvector of the supra-adjacency matrix. Nodes are important if connected to other important nodes across layers.

Parameters

- **network** – Multilayer network object
- **max_iter** – Maximum number of power iteration steps
- **tol** – Convergence tolerance

Returns

Dictionary mapping (node, layer) tuples to eigenvector centrality scores

Algorithm:

Find the principal eigenvector of the supra-adjacency matrix:

A * x = * x

where x is the eigenvector with largest eigenvalue .

References

- Solá, L., et al. (2013). "Eigenvector centrality of nodes in multiplex networks." Chaos, 23(3), 033131.

`py3plex.algorithms.centrality_toolkit.multilayer_pagerank`(*network: Any*, *alpha: float = 0.85*, *max_iter: int = 100*, *tol: float = 1e-06*, *personalization: Dict | None = None*, *uncertainty: bool = False*, *n_runs: int | None = None*, *resampling: ResamplingStrategy | None = None*, *random_seed: int | None = None*) → Dict[Tuple, float] | *StatSeries*

Compute multilayer PageRank centrality with optional uncertainty estimation.

Implements PageRank on the supra-adjacency matrix, accounting for random walks across layers.

Parameters

- **network** – Multilayer network object
- **alpha** – Damping factor (teleportation probability = 1-alpha)
- **max_iter** – Maximum number of iterations
- **tol** – Convergence tolerance
- **personalization** – Optional personalization vector (node -> weight)
- **uncertainty** – If True, estimate uncertainty via resampling
- **n_runs** – Number of runs for uncertainty estimation (default from config)
- **resampling** – Resampling strategy (default from config)
- **random_seed** – Random seed for reproducibility

Returns

StatSeries with deterministic values (std=None) If uncertainty=True: StatSeries with mean, std, quantiles

Return type

If uncertainty=False

Algorithm:

PR = (1-)/N + * A^T * PR

where A is the column-normalized supra-adjacency matrix

References

- Halu, A., et al. (2013). "Multiplex PageRank." PLoS ONE, 8(10), e78293.

Examples

```
>>> # Deterministic
>>> result = multilayer_pagerank(network)
>>> result[('A', 'L1')]  # Dict-like access
{'mean': 0.25}
>>> np.array(result)  # Array access (backward compat)
```

```
>>> # With uncertainty
>>> result = multilayer_pagerank(network, uncertainty=True, n_runs=50)
>>> result.mean  # Average PageRank values
>>> result.std   # Standard deviations
>>> result.quantiles  # Confidence intervals
```

py3plex.algorithms.centrality_toolkit.**multiplex_degree_centrality**(*network: Any*, *normalized: bool = True*, *consider_interlayer: bool = True*) → Dict[Tuple, float]

Compute multiplex degree centrality.

Sums degree across all layers for each node. For multiplex networks where nodes exist in all layers.

Parameters

- **network** – Multiplex network object
- **normalized** – Whether to normalize by maximum possible degree
- **consider_interlayer** – Whether to count inter-layer edges

Returns

Dictionary mapping (node, layer) tuples to degree centrality scores

Algorithm:

For node i: DC(i) = _ k_i^

where k_i^ is the degree of node i in layer .

References

- Battiston, F., et al. (2014). "Structural measures for multiplex networks." Physical Review E, 89(3), 032804.

`py3plex.algorithms.centrality_toolkit.versatility_score(centrality_dict: Dict[Tuple, float], normalized: bool = True) → Dict[Any, float]`

Compute versatility score for each node.

Measures how evenly a node's centrality is distributed across layers. High versatility means the node is important in multiple layers.

Parameters

- **centrality_dict** – Dictionary mapping (node, layer) -> centrality score
- **normalized** – Whether to normalize to [0, 1]

Returns

Dictionary mapping node_id -> versatility score

Algorithm:

V(i) = 1 - _ (c_i^ / c_i^total)^2

where c_i^ is centrality of node i in layer . This is similar to the Herfindahl-Hirschman index.

References

- Battiston, F., et al. (2014). "Structural measures for multiplex networks." Physical Review E, 89(3), 032804.

MultiXRank: Random Walk with Restart on Universal Multilayer Networks

This module implements the MultiXRank algorithm described in: Baptista et al. (2022), "Universal multilayer network exploration by random walk with restart", Communications Physics, 5, 170.

MultiXRank performs random walk with restart (RWR) on a supra-heterogeneous adjacency matrix built from multiple multiplexes connected by bipartite blocks.

References

- Paper: https://doi.org/10.1038/s42005-022-00937-9
- arXiv: https://arxiv.org/abs/2106.07869
- Package: https://github.com/anthbapt/multixrank
- Docs: https://multixrank-doc.readthedocs.io/

class py3plex.algorithms.multilayer_algorithms.multixrank.**MultiXRank**(*restart_prob: float = 0.4*, *epsilon: float = 1e-06*, *max_iter: int = 100000*, *verbose: bool = True*)

Bases: `object`

MultiXRank: Universal multilayer network exploration by random walk with restart.

This class implements the MultiXRank algorithm for node prioritization and ranking in universal multilayer networks. It builds a supra-heterogeneous adjacency matrix from multiple multiplexes and bipartite inter-multiplex connections, then performs random walk with restart.

multiplexes

Dictionary of multiplex supra-adjacency matrices

Type

Dict[str, sp.spmatrix]

bipartite_blocks

Inter-multiplex connection matrices

Type

Dict[Tuple[str, str], sp.spmatrix]

restart_prob

Restart probability (r) for RWR

Type

float

epsilon

Convergence threshold

Type

float

max_iter

Maximum number of iterations

Type
int

node_order
Node ordering for each multiplex

Type
Dict[str, List]

supra_matrix
Built supra-heterogeneous adjacency matrix

Type
sp.spmatrix

transition_matrix
Column-stochastic transition matrix

Type
sp.spmatrix

add_bipartite_block(*multiplex_from: str*, *multiplex_to: str*, *bipartite_matrix: spmatrix | ndarray*, *weight: float = 1.0*)

Add a bipartite block connecting two multiplexes.

Parameters

- **multiplex_from** – Name of source multiplex
- **multiplex_to** – Name of target multiplex
- **bipartite_matrix** – Matrix of connections (rows: from, cols: to)
- **weight** – Optional weight to scale this bipartite block

add_multiplex(*name: str*, *supra_adjacency: spmatrix | ndarray*, *node_order: List | None = None*)

Add a multiplex to the universal multilayer network.

Parameters

- **name** – Unique identifier for this multiplex
- **supra_adjacency** – Supra-adjacency matrix for the multiplex (can be from multi_layer_network.get_supra_adjacency_matrix())
- **node_order** – Optional list of node IDs in the order they appear in the matrix. If None, uses integer indices.

aggregate_scores(*scores: ndarray*, *aggregation: str = 'sum'*) → Dict[str, Dict]

Aggregate scores per multiplex and optionally per physical node.

Parameters

- **scores** – Probability vector from RWR (length = total supra-matrix dimension)
- **aggregation** – How to aggregate scores ('sum', 'mean', 'max')

Returns
Dictionary mapping multiplex names to dictionaries of node scores

build_supra_heterogeneous_matrix(*block_weights: Dict[str | Tuple[str, str], float] | None = None*)

Build the supra-heterogeneous adjacency matrix S.

This constructs the universal multilayer network matrix by: 1. Placing each multiplex supra-adjacency on the block diagonal 2. Adding bipartite blocks for inter-multiplex connections

Parameters

block_weights – Optional dictionary to weight blocks. Keys can be: - Multiplex names (to weight within-multiplex edges) - Tuple (multiplex_from, multiplex_to) for bipartite blocks

Returns

The supra-heterogeneous adjacency matrix

column_normalize(*handle_dangling: str = 'uniform'*) → spmatrix

Column-normalize the supra-heterogeneous matrix to create a stochastic transition matrix.

This ensures each column sums to 1, making the matrix suitable for RWR.

Parameters

handle_dangling – How to handle dangling nodes (columns with zero sum): - 'uniform': Distribute mass uniformly across all nodes - 'self': Add self-loop (mass stays at the node) - 'ignore': Leave as zero (not recommended for RWR)

Returns

Column-stochastic transition matrix

get_top_ranked(*scores: ndarray*, *k: int = 10*, *multiplex: str | None = None*, *exclude_seeds: bool = True*, *seed_nodes: List[int] | Dict[str, List] | None = None*) → List[Tuple]

Get top-k ranked nodes from RWR scores.

Parameters

- **scores** – Probability vector from RWR
- **k** – Number of top nodes to return
- **multiplex** – If specified, only return nodes from this multiplex
- **exclude_seeds** – Whether to exclude seed nodes from results
- **seed_nodes** – Seed nodes to exclude (if exclude_seeds=True)

Returns

List of (global_index, score) tuples, sorted by score descending

random_walk_with_restart(*seed_nodes: List[int] | Dict[str, List] | ndarray*, *seed_weights: ndarray | None = None*, *multiplex_name: str | None = None*) → ndarray

Perform Random Walk with Restart (RWR) from seed nodes.

Parameters

- **seed_nodes** – Seed node specification. Can be: - List of global indices in the supra-matrix - Dict mapping multiplex names to lists of local node indices - NumPy array of global indices

- **seed_weights** – Optional weights for seed nodes (must match length of seed_nodes). If None, uniform weights are used.
- **multiplex_name** – If seed_nodes is a list/array of local indices, specify which multiplex they belong to

Returns

Steady-state probability vector (length = total nodes across all multiplexes) py3plex.algorithms.multilayer_algorithms.multixrank.**multixrank_from_py3plex_networks**(*network Dict[str multi- net.mul bi- par- tite_co Dict[Tu str], sp- ma- trix | ndar- ray] | None = None, seed_no Dict[str List] | None = None, restart_ float = 0.4, ep- silon: float = 1e- 06, max_it int = 100000 ver- bose: bool = True*) → Tu- ple[*Mul* ndarray

Convenience function to run MultiXRank on py3plex multi_layer_network objects.

Parameters

- **networks** – Dictionary mapping names to multi_layer_network objects
- **bipartite_connections** – Optional dict of inter-network connection matrices
- **seed_nodes** – Dict mapping network names to lists of seed node IDs
- **restart_prob** – Restart probability for RWR
- **epsilon** – Convergence threshold
- **max_iter** – Maximum iterations
- **verbose** – Whether to log progress

Returns

Tuple of (MultiXRank object, scores array)

Example

```
>>> from py3plex.core import multinet
>>> net1 = multinet.multi_layer_network()
>>> net1.load_network('network1.edgelist', ...)
>>> net2 = multinet.multi_layer_network()
>>> net2.load_network('network2.edgelist', ...)
>>>
>>> networks = {'net1': net1, 'net2': net2}
>>> seed_nodes = {'net1': ['node1', 'node2']}
>>>
>>> mxr, scores = multixrank_from_py3plex_networks(
...     networks, seed_nodes=seed_nodes
... )
>>>
>>> # Get aggregated scores per network
>>> aggregated = mxr.aggregate_scores(scores)
```

Authors: Benjamin Renoust (github.com/renoust) Date: 2018/02/13 Description: Loads a Detangler JSON format graph and compute unweighted entanglement analysis with Py3Plex

`py3plex.algorithms.multilayer_algorithms.entanglement.build_occurrence_matrix`(*network: Any*) → Tuple[ndarray, List[Any]]

Build occurrence matrix from multilayer network.

Parameters

network – Multilayer network object

Returns

Tuple of (c_matrix, layers) where c_matrix is the normalized occurrence matrix and layers is the list of layer names py3plex.algorithms.multilayer_algorithms.entanglement.**compute_blocks**(*c_matrix: ndarray*) → Tuple[List[List[int]], List[ndarray]]

Compute block decomposition of occurrence matrix.

Parameters
c_matrix – Occurrence matrix

Returns
Tuple of (indices, blocks) where indices are the layer indices in each block and blocks are the submatrices for each block

py3plex.algorithms.multilayer_algorithms.entanglement.**compute_entanglement**(*block_matrix: ndarray*) → Tuple[List[float], List[float]]

Compute entanglement metrics for a block.

Parameters
block_matrix – Block submatrix

Returns
Tuple of ([intensity, homogeneity, normalized_homogeneity], gamma_layers)

py3plex.algorithms.multilayer_algorithms.entanglement.**compute_entanglement_analysis**(*network: Any*) → List[Dict Any]]

Compute full entanglement analysis for a multilayer network.

Parameters
network – Multilayer network object

Returns
List of block analysis dictionaries with entanglement metrics

Supra Matrix Function Centralities for Multilayer Networks.

This module implements centrality measures based on matrix functions of the supra-adjacency matrix, including: - Communicability Centrality (Estrada & Hatano, 2008) - Katz Centrality (Katz, 1953)

These measures operate directly on the sparse supra-adjacency matrix obtained from multi_layer_network.get_supra_adjacency_matrix().

References

- Estrada, E., & Hatano, N. (2008). Communicability in complex networks. Physical Review E, 77(3), 036111.
- Katz, L. (1953). A new status index derived from sociometric analysis. Psychometrika, 18(1), 39-43.

Authors: py3plex contributors Date: October 2025 (Phase II)

`py3plex.algorithms.multilayer_algorithms.supra_matrix_function_centrality.communicability_ce`

Compute communicability centrality for each node-layer pair.

Communicability centrality measures the weighted sum of all walks between nodes, with exponentially decaying weights for longer walks. It is computed as the row sum of the matrix exponential:

c_i = sum_j (exp(A))_ij

This implementation uses scipy.sparse.linalg.expm_multiply for efficient sparse matrix exponential computation.

Parameters

- **supra_matrix** – Sparse supra-adjacency matrix (n x n).
- **normalize** – If True, normalize output to sum to 1.
- **use_sparse** – If True, use sparse matrix operations. Falls back to dense for matrices smaller than 10000 elements.
- **max_iter** – Maximum number of iterations for sparse approximation (currently unused).
- **tol** – Tolerance for convergence (currently unused).

Returns
Communicability centrality scores for each node-layer pair (n,).

Return type
np.ndarray

Raises
Py3plexMatrixError – If matrix is invalid (non-square, empty, etc.).

Example

```
>>> from py3plex.core import random_generators
>>> net = random_generators.random_multiplex_ER(50, 3, 0.1)
>>> A = net.get_supra_adjacency_matrix()
>>> comm = communicability_centrality(A)
>>> print(f"Communicability centrality computed for {len(comm)} node-
↪layer pairs")
```

py3plex.algorithms.multilayer_algorithms.supra_matrix_function_centrality.**katz_centrality**(*su*
sp
m
tr
ai
pi
fl
/
N
=
N
be
fl
=
1.
to
fl
=
1e
0e
→
nc

Compute Katz centrality for each node-layer pair.

Katz centrality measures node influence by accounting for all paths with exponentially decaying weights. It is computed as:

x = (I - alpha * A)^{-1} * beta * 1

where alpha < 1/lambda_max(A) to ensure convergence. If alpha is not provided, it defaults to 0.85 / lambda_max(A).

Parameters

- **supra_matrix** – Sparse supra-adjacency matrix (n x n).
- **alpha** – Attenuation parameter. Must be less than 1/spectral_radius(A). If None, defaults to 0.85 / lambda_max(A).

- **beta** – Weight of exogenous influence (typically 1.0).
- **tol** – Tolerance for eigenvalue computation.

Returns

Katz centrality scores for each node-layer pair (n,).
Normalized to sum to 1.

Return type
np.ndarray

Raises
Py3plexMatrixError – If matrix is invalid or alpha is out of valid range.

Example

```
>>> from py3plex.core import random_generators
>>> net = random_generators.random_multiplex_ER(50, 3, 0.1)
>>> A = net.get_supra_adjacency_matrix()
>>> katz = katz_centrality(A)
>>> print(f"Katz centrality computed for {len(katz)} node-layer pairs")
>>> # With custom alpha
>>> katz_custom = katz_centrality(A, alpha=0.05)
```

Multiplex Participation Coefficient (MPC) for multiplex networks.

This module implements the Multiplex Participation Coefficient metric for multiplex networks (networks with identical node sets across all layers).

Authors: py3plex contributors Date: 2025

`py3plex.algorithms.multicentrality.ensure(*args, **kwargs)`

`py3plex.algorithms.multicentrality.multiplex_participation_coefficient(multinet: Any, normalized: bool = True, check_multiplex: bool = True) → Dict[Any, float]`

Compute the Multiplex Participation Coefficient (MPC) for multiplex networks. MPC measures how evenly a node participates across layers.

Parameters

- **multinet** (py3plex.core.multinet.multi_layer_network) – Multiplex network object (same node set across layers).
- **normalized** (*bool, optional*) – Whether to normalize MPC to [0,1]. Default: True.
- **check_multiplex** (*bool, optional*) – Validate that all layers share the same node set.

Returns

Node → MPC value mapping.

Return type
dict

Notes

The MPC is computed as:
MPC(i) = 1 - sum_ (k_i^ / k_i^total)^2

Where: - k_i^ is the degree of node i in layer - k_i^total is the total degree of node i across all layers

When normalized=True, the result is multiplied by L/(L-1) to normalize to [0,1] range, where L is the number of layers.

References

- Battiston, F., et al. (2014). "Structural measures for multiplex networks." Physical Review E, 89(3), 032804.
- De Domenico, M., et al. (2015). "Identifying modular flows on multilayer networks reveals highly overlapping organization in interconnected systems." Physical Review X, 5(1), 011027.
- Harooni, M., et al. (2025). "Centrality in Multilayer Networks: Accurate Measurements with MultiNetPy." The Journal of Supercomputing, 81(1), 92. DOI: 10.1007/s11227-025-07197-8

Contracts:

- Precondition: multinet must not be None
- Precondition: normalized and check_multiplex must be booleans
- Postcondition: returns a dictionary with numeric values

`py3plex.algorithms.multicentrality.require(*args, **kwargs)`

General Algorithms

Random walk primitives for graph-based algorithms.

This module provides foundation for higher-level algorithms like Node2Vec, DeepWalk, and diffusion processes. Implements both basic and second-order (biased) random walks with proper edge weight handling and multilayer support.

Key Features:

- Basic random walks with weighted edge sampling
- Second-order (Node2Vec-style) biased random walks with p/q parameters
- Multiple simultaneous walks with deterministic reproducibility
- Support for directed, weighted, and multilayer networks
- Efficient sparse adjacency handling

References

- Grover, A., & Leskovec, J. (2016). node2vec: Scalable feature learning for networks. KDD '16. https://doi.org/10.1145/2939672.2939754
- Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). DeepWalk: Online learning of social representations. KDD '14. https://doi.org/10.1145/2623330.2623732

`py3plex.algorithms.general.walkers.basic_random_walk`(*G: Graph*, *start_node: int | str*, *walk_length: int*, *weighted: bool = True*, *seed: int | None = None*) → List[int | str]

Perform a basic random walk on a graph with proper edge weight handling.

The next step is sampled proportionally to the normalized edge weights of the current node. For unweighted graphs, transitions are uniform.

Parameters

- **G** – NetworkX graph (directed or undirected, weighted or unweighted)
- **start_node** – Node to start the walk from
- **walk_length** – Number of steps in the walk
- **weighted** – Whether to use edge weights (default: True)
- **seed** – Random seed for reproducibility (default: None)

Returns

List of nodes representing the walk path (includes start_node)

Raises

ValueError – If start_node not in graph or walk_length < 1

Examples

```
>>> G = nx.Graph()
>>> G.add_weighted_edges_from([(0, 1, 1.0), (1, 2, 2.0), (1, 3, 1.0)])
>>> walk = basic_random_walk(G, 0, walk_length=3, seed=42)
>>> len(walk)
4
>>> walk[0]
0
```

Note

- Handles disconnected nodes by terminating walk early
- Edge weights must be positive for weighted walks
- Sum of transition probabilities from any node equals 1.0

`py3plex.algorithms.general.walkers.general_random_walk`(*G*, *start_node*, *iterations=1000*, *teleportation_prob=0*)

Legacy random walk with teleportation (for backward compatibility).

Deprecated since version 0.95a: Use *basic_random_walk()* or *node2vec_walk()* instead. This function will be removed in version 1.0.

Parameters

- **G** – NetworkX graph
- **start_node** – Starting node
- **iterations** – Number of steps
- **teleportation_prob** – Probability of teleporting to random visited node

Returns

List of visited nodes (excluding start_node)

`py3plex.algorithms.general.walkers.generate_walks(`*G: Graph*, *num_walks: int*, *walk_length: int*, *start_nodes: List[int | str] | None = None*, *p: float = 1.0*, *q: float = 1.0*, *weighted: bool = True*, *return_edges: bool = False*, *seed: int | None = None*`)` → List[List[int | str]] | List[List[Tuple[int | str, int | str]]]

Generate multiple random walks from specified or all nodes.

This interface supports multiple simultaneous walks with deterministic reproducibility under fixed RNG seed. Can return either node sequences or edge sequences.

Parameters

- **G** – NetworkX graph
- **num_walks** – Number of walks to generate per start node
- **walk_length** – Number of steps in each walk
- **start_nodes** – Nodes to start walks from (if None, uses all nodes)
- **p** – Return parameter for Node2Vec (1.0 = no bias)
- **q** – In-out parameter for Node2Vec (1.0 = no bias)
- **weighted** – Whether to use edge weights
- **return_edges** – Return edge sequences instead of node sequences
- **seed** – Random seed for reproducibility

Returns

- List of nodes (if return_edges=False)
- List of edges as tuples (if return_edges=True)

Return type

List of walks, where each walk is either

Examples

```
>>> G = nx.karate_club_graph()
>>> # Generate 10 walks from each node
>>> walks = generate_walks(G, num_walks=10, walk_length=5, seed=42)
>>> len(walks)
340
```

```
>>> # Generate walks with Node2Vec bias
>>> walks = generate_walks(G, num_walks=5, walk_length=10, p=0.5, q=2.0,␣
↪seed=42)
```

```
>>> # Get edge sequences
>>> edge_walks = generate_walks(G, num_walks=3, walk_length=5, return_
↪edges=True, seed=42)
```

Note

- With same seed, generates identical walks across runs
- If p == q == 1.0, uses basic random walk (faster)
- Empty walks are included if node has no neighbors

py3plex.algorithms.general.walkers.**layer_specific_random_walk**(*G: Graph, start_node: int | str, walk_length: int, layer: str | None = None, cross_layer_prob: float = 0.0, weighted: bool = True, seed: int | None = None*) → List[int | str]

Perform random walk with layer constraints for multilayer networks.

In a multilayer network represented in py3plex format (where node names include layer information), this function can constrain walks to specific layers with occasional inter-layer transitions.

Parameters

- **G** – NetworkX graph (multilayer network in py3plex format)
- **start_node** – Node to start walk from (may include layer info)
- **walk_length** – Number of steps in the walk
- **layer** – Target layer to constrain walk to (None = no constraint)
- **cross_layer_prob** – Probability of crossing to different layer (0-1)
- **weighted** – Whether to use edge weights
- **seed** – Random seed for reproducibility

Returns
List of nodes representing the walk path

Examples

```
>>> # Multilayer network with layer-specific walks
>>> from py3plex.core import multinet
>>> network = multinet.multi_layer_network()
>>> network.add_layer("social")
>>> network.add_layer("biological")
>>> # ... add nodes and edges ...
>>> walk = layer_specific_random_walk(
...     network.core_network,
...     "nodeA---social",
...     walk_length=10,
...     layer="social",
...     cross_layer_prob=0.1
... )
```

Note

- If layer is None, behaves like basic_random_walk
- cross_layer_prob controls inter-layer transitions
- Node format should follow py3plex convention: "nodeID—layerID"

`py3plex.algorithms.general.walkers.node2vec_walk`(*G: Graph*, *start_node: int | str*, *walk_length: int*, *p: float = 1.0*, *q: float = 1.0*, *weighted: bool = True*, *seed: int | None = None*) → List[int | str]

Perform a second-order (biased) random walk following Node2Vec logic.

When transitioning from node t → v → x, the probability of choosing x is biased by parameters p (return) and q (in-out): - If x == t (return to previous): weight / p - If x is neighbor of t (stay close): weight / 1 - If x is not neighbor of t (explore): weight / q

Parameters

- **G** – NetworkX graph (directed or undirected, weighted or unweighted)
- **start_node** – Node to start the walk from
- **walk_length** – Number of steps in the walk
- **p** – Return parameter (higher p = less likely to return to previous node)
- **q** – In-out parameter (higher q = less likely to explore further)
- **weighted** – Whether to use edge weights (default: True)
- **seed** – Random seed for reproducibility (default: None)

Returns
List of nodes representing the walk path (includes start_node)

Raises

`ValueError` – If p <= 0 or q <= 0 or start_node not in graph

Examples

```
>>> G = nx.Graph()
>>> G.add_edges_from([(0, 1), (1, 2), (1, 3), (0, 2)])
>>> # Low p, high q: tends to backtrack
>>> walk = node2vec_walk(G, 0, walk_length=5, p=0.1, q=10.0, seed=42)
>>> # High p, low q: tends to explore outward
>>> walk2 = node2vec_walk(G, 0, walk_length=5, p=10.0, q=0.1, seed=42)
```

Note

- First step is always a basic random walk (no previous node)
- Properly normalizes probabilities at each step
- Handles disconnected nodes by terminating early

References

Grover & Leskovec (2016), node2vec: Scalable feature learning for networks

Benchmark algorithms for node classification performance evaluation.

This module provides algorithms for benchmarking node classification performance, including oracle-based F1 score evaluation.

`py3plex.algorithms.general.benchmark_classification.evaluate_oracle_F1`(*probs: ndarray*, *Y_real: ndarray*) → tuple[float, float]

Evaluate oracle F1 scores for multi-label classification.

This function computes micro and macro F1 scores by selecting the top-k predictions for each sample, where k is determined by the ground truth number of labels.

Parameters

- **probs** – Predicted probability matrix of shape (n_samples, n_labels).
- **Y_real** – Ground truth binary label matrix of shape (n_samples, n_labels).

Returns

- micro: Micro-averaged F1 score
- macro: Macro-averaged F1 score

Return type

A tuple containing

Example

```
>>> probs = np.array([[0.9, 0.1, 0.2], [0.1, 0.8, 0.7]])
>>> Y_real = np.array([[1, 0, 0], [0, 1, 1]])
>>> micro, macro = evaluate_oracle_F1(probs, Y_real)
```

Node Ranking

`py3plex.algorithms.node_ranking.node_ranking.`**`authority_matrix`**(*graph: Graph*) → spmatrix

Get the authority matrix of a graph.

Parameters
graph – NetworkX graph

Returns
Authority matrix

`py3plex.algorithms.node_ranking.node_ranking.`**`hub_matrix`**(*graph: Graph*) → spmatrix

Get the hub matrix of a graph.

Parameters
graph – NetworkX graph

Returns
Hub matrix

`py3plex.algorithms.node_ranking.node_ranking.`**`hubs_and_authorities`**(*graph: Graph*) → Tuple[dict, dict]

Compute hubs and authorities scores using HITS algorithm.

Parameters
graph – NetworkX graph

Returns
Tuple of (hubs dictionary, authorities dictionary)

`py3plex.algorithms.node_ranking.node_ranking.`**`modularity`**(*G: Graph*, *communities: List[List[Any]]*, *weight: str = 'weight'*) → float

Calculate modularity of a graph partition.

Parameters

- **G** – NetworkX graph
- **communities** – List of communities (each community is a list of nodes)
- **weight** – Edge weight attribute name

Returns
Modularity value

`py3plex.algorithms.node_ranking.node_ranking.`**`page_rank_kernel`**(*index_row: int*) → Tuple[int, ndarray]

PageRank kernel for parallel computation.

Note: This function expects global variables G, damping_hyper, spread_step_hyper, spread_percent_hyper, and graph to be defined. It's designed for use with multiprocessing.Pool.map().

Parameters
`index_row` – Row index to compute PageRank for

Returns
Tuple of (index, PageRank vector)

`py3plex.algorithms.node_ranking.node_ranking.`**`sparse_page_rank`**`(`*matrix: spmatrix*, *start_nodes: List[int] | range | None*, *epsilon: float = 1e-06*, *max_steps: int = 100000*, *damping: float = 0.5*, *spread_step: int = 10*, *spread_percent: float = 0.3*, *try_shrink: bool = False*`)` → ndarray

Compute sparse PageRank with personalization.

Parameters

- `matrix` – Sparse adjacency matrix (column-stochastic)
- `start_nodes` – List of starting node indices for personalization (can be range or None)
- `epsilon` – Convergence threshold
- `max_steps` – Maximum number of iterations
- `damping` – Damping factor (teleportation probability)
- `spread_step` – Maximum steps for spread calculation
- `spread_percent` – Percentage threshold for spread
- `try_shrink` – Whether to try matrix shrinking optimization

Returns
PageRank vector

`py3plex.algorithms.node_ranking.node_ranking.`**`stochastic_normalization`**`(`*matrix: spmatrix*`)` → spmatrix

Normalize a sparse matrix stochastically.

Parameters
`matrix` – Sparse matrix to normalize

Returns
Stochastically normalized sparse matrix

py3plex.algorithms.node_ranking.node_ranking.**stochastic_normalization_hin**(*matrix: spmatrix*) → spmatrix

Normalize a heterogeneous information network matrix stochastically.

Parameters

matrix – Sparse matrix to normalize

Returns

Stochastically normalized sparse matrix

Network Classification

py3plex.algorithms.network_classification.label_propagation.**label_propagation**(*graph_matrix: spmatrix, class_matrix: ndarray, alpha: float = 0.001, epsilon: float = 1e-12, max_steps: int = 100000, normalization: str | List[str] = 'freq'*) → ndarray

Propagate labels through a graph.

Parameters

- **graph_matrix** – Sparse graph adjacency matrix
- **class_matrix** – Initial class label matrix
- **alpha** – Propagation weight parameter
- **epsilon** – Convergence threshold
- **max_steps** – Maximum number of iterations
- **normalization** – Normalization scheme(s) to apply

Returns
Propagated label matrix

py3plex.algorithms.network_classification.label_propagation.**label_propagation_normalization**(

Normalize a matrix for label propagation.

Parameters
matrix – Sparse matrix to normalize

Returns
Normalized sparse matrix

py3plex.algorithms.network_classification.label_propagation.**label_propagation_tf**() → None

TensorFlow-based label propagation (TODO: implement).

Placeholder for future TensorFlow implementation.

py3plex.algorithms.network_classification.label_propagation.**normalize_amplify_freq**(*mat: ndarray*) → ndarray

Normalize and amplify matrix by frequency.

Parameters
mat – Matrix to normalize

Returns
Normalized and amplified matrix

py3plex.algorithms.network_classification.label_propagation.**normalize_exp**(*mat: ndarray*) → ndarray

Apply exponential normalization.

Parameters
mat – Matrix to normalize

Returns
Exponentially normalized matrix

py3plex.algorithms.network_classification.label_propagation.**normalize_initial_matrix_freq**(*m n r* → n

Normalize matrix by frequency.

Parameters
: **mat** – Matrix to normalize

Returns
: Normalized matrix

py3plex.algorithms.network_classification.label_propagation.**normalize_none**(*mat: ndarray*) → ndarray

No normalization (identity function).

Parameters
: **mat** – Matrix to return unchanged

Returns
: Original matrix

py3plex.algorithms.network_classification.label_propagation.**validate_label_propagation**(*core_* *sp-* *ma-* *trix*, *la-* *bels:* *ndar-* *ray* */* *sp-* *ma-* *trix*, *datas* *str* *=* *'test'* *rep-* *e-* *ti-* *tions* *int* *=* *5*, *nor-* *mal-* *iza-* *tion_* *str* */* *List[s* *=* *'ba-* *sic'*, *al-* *pha_* *float* *=* *0.001* *ran-* *dom_* *int* *=* *123*, *ver-* *bose:* *bool* *=* *False* → Data

Validate label propagation with cross-validation.

Parameters

- **core_network** – Sparse network adjacency matrix
- **labels** – Label matrix
- **dataset_name** – Name of the dataset
- **repetitions** – Number of repetitions
- **normalization_scheme** – Normalization scheme to use
- **alpha_value** – Alpha parameter for propagation
- **random_seed** – Random seed for reproducibility
- **verbose** – Whether to print progress

Returns

DataFrame with validation results

Visualization

`py3plex.visualization.multilayer.draw_multiedges`(*network_list: List[Graph] | Dict[Any, Graph], multi_edge_tuple: List[Any], input_type: str = 'nodes', linepoints: str = '-.', alphachannel: float = 0.3, linecolor: str = 'black', curve_height: float = 1, style: str = 'curve2_bezier', linewidth: float = 1, invert: bool = False, linmod: str = 'both', resolution: float = 0.001, ax: Any | None = None*) → Any

Draw edges connecting multiple layers.

Draws curved or straight edges that connect nodes across different layers in a multilayer network visualization. Typically used after draw_multilayer_default to add inter-layer connections.

Parameters

- **network_list** – List of NetworkX graphs (layers) or dict of layer_name -> graph
- **multi_edge_tuple** – List of tuples specifying edges to draw, e.g. [(node1, node2), . . .]
- **input_type** – Type of input ("nodes" or other)
- **linepoints** – Line style (e.g., "-.", "–", "-")
- **alphachannel** – Transparency level (0.0 to 1.0)
- **linecolor** – Color of the lines
- **curve_height** – Height of curved edges
- **style** – Style of edges ("curve2_bezier", "line", "curve3_bezier", "curve3_fit", "piramidal")
- **linewidth** – Width of lines

- **invert** – Whether to invert drawing direction
- **linmod** – Line modification mode
- **resolution** – Resolution for curve drawing
- **ax** – Matplotlib Axes to draw on. If None, uses current axes (plt.gca())

Returns
Matplotlib Axes object containing the visualization.

Example

```
>>> import matplotlib.pyplot as plt
>>> from py3plex.visualization import draw_multilayer_default, draw_
↪multiedges
>>> fig, ax = plt.subplots(figsize=(10, 10))
>>> ax = draw_multilayer_default(graphs, ax=ax)
>>> ax = draw_multiedges(graphs, edges, ax=ax)
>>> plt.savefig("multilayer_with_edges.png")
```

py3plex.visualization.multilayer.**draw_multilayer_default**(*network_list: List[Graph] | Dict[Any, Graph], display: bool = False, node_size: int = 10, alphalevel: float = 0.13, rectanglex: float = 1, rectangley: float = 1, background_shape: str = 'circle', background_color: str = 'rainbow', networks_color: str = 'rainbow', labels: bool = False, arrowsize: float = 0.5, label_position: int = 1, verbose: bool = False, remove_isolated_nodes: bool = False, ax: Any | None = None, edge_size: float = 1, node_labels: bool = False, node_font_size: int = 5, scale_by_size: bool = False, *, axis: Any | None = None*) → Any

Core multilayer drawing method.

Draws a diagonal multilayer network visualization where each layer is offset to create a 3D-like effect. Nodes within each layer are drawn with their positions, and background shapes indicate layer boundaries.

Parameters

- **network_list** – List of NetworkX graphs to visualize (or dict of layer_name -> graph)

- **display** – If True, calls plt.show() after drawing. Default is False to let the caller control rendering.
- **node_size** – Base size of nodes
- **alphalevel** – Transparency level for background shapes
- **rectanglex** – Width of rectangular backgrounds
- **rectangley** – Height of rectangular backgrounds
- **background_shape** – Background shape type ("circle" or "rectangle")
- **background_color** – Background color scheme ("default", "rainbow", or None)
- **networks_color** – Network color scheme ("rainbow" or "black")
- **labels** – Layer labels to display
- **arrowsize** – Size of edge arrows
- **label_position** – Position offset for layer labels
- **verbose** – Whether to log network information
- **remove_isolated_nodes** – Whether to remove isolated nodes
- **ax** – Matplotlib Axes to draw on. If None, uses current axes (plt.gca())
- **edge_size** – Width of edges
- **node_labels** – Whether to display node labels
- **node_font_size** – Font size for node labels
- **scale_by_size** – Whether to scale node size by degree
- **axis** – Deprecated. Use ax instead.

Returns

Matplotlib Axes object containing the visualization.

Example

```
>>> import matplotlib.pyplot as plt
>>> from py3plex.visualization import draw_multilayer_default
>>> # Create figure and get axes
>>> fig, ax = plt.subplots(figsize=(10, 10))
>>> # Draw on the axes (returns the axes)
>>> ax = draw_multilayer_default(graphs, ax=ax)
>>> # Caller controls when to display
>>> plt.savefig("multilayer.png")  # or plt.show()
```

py3plex.visualization.multilayer.**draw_multilayer_flow**(*graphs: List[Graph], multilinks: Dict[str, List[Tuple]], labels: List[str] | None = None, node_activity: Dict[Any, float] | None = None, ax: Any | None = None, display: bool = True, layer_gap: float = 3.0, node_size: float = 30, node_cmap: str = 'viridis', flow_alpha: float = 0.3, flow_min_width: float = 0.2, flow_max_width: float = 4.0, aggregate_by: Tuple[str, ...] = ('u', 'v', 'layer_u', 'layer_v'), **kwargs*) → Any

Draw multilayer network as layered flow visualization (alluvial-style).

Shows each layer as a horizontal band with nodes positioned along the x-axis. Intra-layer activity is encoded as node color/size, and inter-layer edges are shown as thick flow ribbons (Bezier curves) where width encodes edge weight.

Parameters

- **graphs** – List of NetworkX graphs, one per layer (from multi_layer_network.get_layers())
- **multilinks** – Dictionary mapping edge_type -> list of multi-layer edges
- **labels** – Optional list of layer labels. If None, uses layer indices
- **node_activity** – Optional dict mapping node_id -> activity value. If None, computes intra-layer degree
- **ax** – Matplotlib axes to draw on. If None, creates new figure
- **display** – If True, calls plt.show() at the end
- **layer_gap** – Vertical distance between layer bands
- **node_size** – Base marker size for nodes
- **node_cmap** – Matplotlib colormap name for node activity coloring
- **flow_alpha** – Base transparency for flow ribbons
- **flow_min_width** – Minimum line width for flows
- **flow_max_width** – Maximum line width for flows
- **aggregate_by** – Tuple of keys for aggregating flows (currently not used, for future extension)
- ****kwargs** – Reserved for future extensions

Returns

Matplotlib axes object

Examples

```
>>> network = multi_layer_network()
>>> network.load_network("data.txt", input_type="multiedgelist")
>>> labels, graphs, multilinks = network.get_layers()
>>> draw_multilayer_flow(graphs, multilinks, labels=labels)
```

`py3plex.visualization.multilayer.generate_random_multiedges`(*network_list: List[Graph], random_edges: int, style: str = 'line', linepoints: str = '-.', upper_first: int = 2, lower_first: int = 0, lower_second: int = 2, inverse_tag: bool = False, pheight: float = 1*) → None

Generate and draw random multi-layer edges.

Parameters

- **network_list** – List of NetworkX graphs (layers)
- **random_edges** – Number of random edges to generate
- **style** – Style of edges to draw
- **linepoints** – Line style
- **upper_first** – Upper bound for first layer
- **lower_first** – Lower bound for first layer
- **lower_second** – Lower bound for second layer
- **inverse_tag** – Whether to invert drawing
- **pheight** – Height parameter for curves

`py3plex.visualization.multilayer.generate_random_networks`(*number_of_networks: int*) → List[Graph]

Generate random networks for testing.

Parameters

number_of_networks – Number of random networks to generate

Returns

List of NetworkX graphs with random layouts `py3plex.visualization.multilayer.hairball_plot(g: Graph | Any, color_list: List[str] | List[int] | None = None, display: bool = False, node_size: float = 1, text_color: str = 'black', node_sizes: List[float] | None = None, layout_parameters: dict | None = None, legend: Any | None = None, scale_by_size: bool = True, layout_algorithm: str = 'force', edge_width: float = 0.01, alpha_channel: float = 0.5, labels: List[str] | None = None, draw: bool = True, label_font_size: int = 2, ax: Any | None = None) → Any | None`

Draw a force-directed "hairball" visualization of a network.

Creates a force-directed layout visualization where nodes are colored by type/layer and sized by degree. This is a common visualization for showing the overall structure of a network.

Parameters

- **g** – NetworkX graph to visualize
- **color_list** – List of colors for nodes. If None, colors are assigned based on node types.
- **display** – If True, calls plt.show() after drawing. Default is False to let the caller control rendering.
- **node_size** – Base size of nodes
- **text_color** – Color for node labels
- **node_sizes** – Custom list of node sizes (overrides node_size and scale_by_size)
- **layout_parameters** – Parameters for the layout algorithm (e.g., {"pos": {…}})
- **legend** – If True, display a legend mapping colors to node types
- **scale_by_size** – If True, scale node sizes by log(degree)
- **layout_algorithm** – Layout algorithm to use. Options: - "force": Force-directed layout (spring layout) - "random": Random layout - "custom_coordinates": Use positions from layout_parameters["pos"] - "custom_coordinates_initial_force": Use custom positions as initial layout
- **edge_width** – Width of edges
- **alpha_channel** – Transparency level (0.0 to 1.0)
- **labels** – List of node labels to display (None for no labels)
- **draw** – If True, draw the network. If False, only compute layout and return data.
- **label_font_size** – Font size for node labels
- **ax** – Matplotlib Axes to draw on. If None, uses current axes (plt.gca())

Returns

Matplotlib Axes object containing the visualization - If draw=False: Tuple of (graph, node_sizes, color_mapping, positions)

Return type

- If draw=True

Example

```
>>> import matplotlib.pyplot as plt
>>> from py3plex.visualization import hairball_plot
>>> fig, ax = plt.subplots(figsize=(10, 10))
>>> ax = hairball_plot(network.core_network, ax=ax, legend=True)
>>> plt.savefig("hairball.png")
```

`py3plex.visualization.multilayer.`**`interactive_diagonal_plot`**(*network_list: List[Graph] | Dict[Any, Graph], layer_labels: List[str] | None = None, layout_algorithm: str = 'force', layer_gap: float = 4.0, node_size_base: int = 8, layer_colors: List[str] | None = None, show_interlayer_edges: bool = True, interlayer_edges: List[Tuple[Any, Any]] | None = None*) → bool | Any

Create an interactive 2.5D diagonal multilayer plot using Plotly.

This function creates an interactive version of the diagonal multilayer visualization, mimicking the traditional 2D diagonal layout but in an interactive 3D environment. Each layer is positioned diagonally with clear visual separation, similar to the static diagonal visualization.

Parameters

- **network_list** – List of NetworkX graphs (layers) or dict of layer_name -> graph
- **layer_labels** – Optional labels for each layer
- **layout_algorithm** – Layout algorithm for nodes ("force", "circular", "random")
- **layer_gap** – Distance between layers in diagonal direction (default: 2.5)
- **node_size_base** – Base size for nodes (default: 8)
- **layer_colors** – Optional list of colors for each layer (HTML color

names or hex)

- **show_interlayer_edges** – Whether to show inter-layer edges
- **interlayer_edges** – List of tuples (node1, node2) for inter-layer connections

Returns
False if plotly not available, otherwise plotly figure object

Examples

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network()
>>> net.load_network("network.txt", input_type="multiedgelist")
>>> labels, graphs, multilinks = net.get_layers("diagonal")
>>> fig = interactive_diagonal_plot(graphs, layer_labels=labels)
```

`py3plex.visualization.multilayer.`**`interactive_hairball_plot`**`(`*G: Graph*, *nsizes: List[float]*, *final_color_mapping: dict*, *pos: dict*, *colorscale: str = 'Rainbow'*`)` → bool | Any

Create an interactive 3D hairball plot using Plotly.

Parameters

- **G** – NetworkX graph to visualize
- **nsizes** – Node sizes
- **final_color_mapping** – Mapping of nodes to colors
- **pos** – Node positions
- **colorscale** – Color scale to use

Returns
False if plotly not available, otherwise plotly figure object

`py3plex.visualization.multilayer.`**`onclick`**`(`*event: Any*`)` → None

Handle mouse click events on plots.

Parameters
event – Matplotlib event object

`py3plex.visualization.multilayer.plot_edge_colored_projection`(*multilayer_network*, *layout: str = 'spring'*, *node_size: int = 50*, *layer_colors: Dict[Any, str] | None = None*, *aggregate_multilayer_edges: bool = True*, *figsize: Tuple[float, float] = (12, 9)*, *edge_alpha: float = 0.7*, ***kwargs*)

Create an aggregated projection where edge colors indicate layer membership.

This visualization projects all layers onto a single 2D graph, using edge colors to distinguish which layer each edge belongs to. Useful for seeing the overall structure while maintaining layer information.

Parameters

- **multilayer_network** – A MultiLayerNetwork instance
- **layout** – Layout algorithm to use ("spring", "circular", "random", "kamada_kawai")
- **node_size** – Size of nodes
- **layer_colors** – Optional dict mapping layer names to colors; if None, auto-generated
- **aggregate_multilayer_edges** – If True, show edges from all layers with distinct colors
- **figsize** – Figure size as (width, height) tuple
- **edge_alpha** – Transparency level for edges (0-1)
- ****kwargs** – Additional arguments for customization

Returns

The created figure

Return type

matplotlib.figure.Figure

`py3plex.visualization.multilayer.plot_ego_multilayer`(*multilayer_network*, *ego*, *layers: List[Any] | None = None*, *max_depth: int = 1*, *layout: str = 'spring'*, *figsize: Tuple[float, float] | None = None*, *max_cols: int = 3*, *node_size: int = 500*, *ego_node_size: int = 1200*, ***kwargs*)

Create an ego-centric multilayer visualization.

This visualization focuses on a single node (ego) and shows its neighborhood across different layers, highlighting the ego node's position in each layer.

Parameters

- **multilayer_network** – A MultiLayerNetwork instance
- **ego** – The ego node to focus on
- **layers** – Optional list of specific layers to visualize; if None, uses all layers
- **max_depth** – Maximum depth of neighborhood to include (number of hops)
- **layout** – Layout algorithm for each ego graph
- **figsize** – Optional figure size; if None, auto-calculated
- **max_cols** – Maximum columns in subplot grid
- **node_size** – Size of regular nodes
- **ego_node_size** – Size of the ego node (highlighted)
- ****kwargs** – Additional drawing parameters

Returns

The created figure

Return type

matplotlib.figure.Figure

`py3plex.visualization.multilayer.plot_radial_layers`(*multilayer_network*, *base_radius: float = 1.0*, *radius_step: float = 1.0*, *node_size: int = 500*, *draw_inter_layer_edges: bool = True*, *figsize: Tuple[float, float] = (12, 12)*, *edge_alpha: float = 0.5*, *draw_layer_bands: bool = True*, *band_alpha: float = 0.25*, ***kwargs*)

Create a radial/concentric visualization with layers as rings.

This visualization arranges layers as concentric circles, with nodes positioned on rings based on their layer. Inter-layer edges appear as radial connections.

Parameters

- **multilayer_network** – A MultiLayerNetwork instance
- **base_radius** – Radius of the innermost layer
- **radius_step** – Distance between consecutive layer rings
- **node_size** – Size of nodes (default: 500 for better visibility)
- **draw_inter_layer_edges** – If True, draw edges between layers
- **figsize** – Figure size as (width, height) tuple
- **edge_alpha** – Transparency for edges
- **draw_layer_bands** – If True, draw semi-transparent circular bands around layers

- **band_alpha** – Transparency for layer bands (default: 0.25)
- ****kwargs** – Additional drawing parameters

Returns
The created figure

Return type
matplotlib.figure.Figure

`py3plex.visualization.multilayer.plot_small_multiples(`*multilayer_network, layout: str = 'spring', max_cols: int = 3, node_size: int = 50, shared_layout: bool = True, show_layer_titles: bool = True, figsize: Tuple[float, float] | None = None, **kwargs*`)`

Create a small multiples visualization with one subplot per layer.

This visualization shows each layer as a separate subplot in a grid layout, making it easy to compare the structure of different layers side-by-side.

Parameters

- **multilayer_network** – A MultiLayerNetwork instance
- **layout** – Layout algorithm to use ("spring", "circular", "random", "kamada_kawai")
- **max_cols** – Maximum number of columns in the subplot grid
- **node_size** – Size of nodes in each subplot
- **shared_layout** – If True, compute one layout and reuse for all layers; if False, compute independent layouts per layer
- **show_layer_titles** – If True, show layer names as subplot titles
- **figsize** – Optional figure size as (width, height) tuple
- ****kwargs** – Additional arguments passed to nx.draw_networkx

Returns
The created figure

Return type
matplotlib.figure.Figure

`py3plex.visualization.multilayer.plot_supra_adjacency_heatmap(`*multilayer_network, include_inter_layer: bool = False, inter_layer_weight: float = 1.0, node_order: List[Any] | None = None, cmap: str = 'viridis', figsize: Tuple[float, float] = (10, 10), **kwargs*`)`

Create a supra-adjacency matrix heatmap visualization.

This visualization shows the multilayer network as a block matrix where each block represents the adjacency matrix of one layer. Optionally includes inter-layer connections.

Parameters

- **multilayer_network** – A MultiLayerNetwork instance
- **include_inter_layer** – If True, include inter-layer edges/couplings
- **inter_layer_weight** – Default weight for inter-layer connections
- **node_order** – Optional list specifying node ordering; if None, uses sorted order
- **cmap** – Colormap name for the heatmap
- **figsize** – Figure size as (width, height) tuple
- ****kwargs** – Additional arguments for imshow

Returns

The created figure

Return type

matplotlib.figure.Figure

py3plex.visualization.multilayer.**supra_adjacency_matrix_plot**(*matrix: ndarray*, *display: bool = False*, *ax: Any | None = None*, *cmap: str = 'binary'*) → Any

Plot a supra-adjacency matrix as a heatmap.

Visualizes the supra-adjacency matrix of a multilayer network, where the matrix shows both intra-layer and inter-layer connections. The matrix is displayed as a heatmap with configurable colormap.

Parameters

- **matrix** – Supra-adjacency matrix to plot (numpy ndarray or scipy sparse matrix)
- **display** – If True, calls plt.show() after drawing. Default is False to let the caller control rendering.
- **ax** – Matplotlib Axes to draw on. If None, uses current axes (plt.gca())
- **cmap** – Colormap to use for the heatmap (default: "binary")

Returns

Matplotlib Axes object containing the visualization.

Example

```
>>> import matplotlib.pyplot as plt
>>> from py3plex.visualization import supra_adjacency_matrix_plot
>>> fig, ax = plt.subplots(figsize=(8, 8))
>>> ax = supra_adjacency_matrix_plot(supra_matrix, ax=ax, cmap="viridis")
>>> plt.colorbar(ax.images[0])
>>> plt.savefig("supra_matrix.png")
```

`py3plex.visualization.multilayer.visualize_multilayer_network`(*multilayer_network*, *visualization_type: str = 'diagonal'*, ***kwargs*)

High-level function to visualize multilayer networks with multiple visualization modes.

This function provides a unified interface for various multilayer network visualization techniques, making it easy to switch between different visual representations.

Parameters

- **multilayer_network** – A MultiLayerNetwork instance from py3plex.core.multinet
- **visualization_type** – Type of visualization to use. Options: - "diagonal": Default layer-centric diagonal layout (existing behavior) - "small_multiples": One subplot per layer with shared or independent layouts - "edge_colored_projection": Aggregate projection with edge colors by layer - "supra_adjacency_heatmap": Matrix representation of multilayer structure - "radial_layers": Concentric circles for layers with radial inter-layer edges - "ego_multilayer": Ego-centric view focused on a specific node
- ****kwargs** – Additional keyword arguments specific to each visualization type. See individual plot functions for details.

Returns

The created figure object

Return type

matplotlib.figure.Figure

Raises

ValueError – If visualization_type is not recognized

Examples

```
>>> from py3plex.core import multinet
>>> net = multinet.multi_layer_network()
>>> net.load_network("network.txt", input_type="multiedgelist")
>>>
>>> # Use default diagonal visualization
>>> fig = visualize_multilayer_network(net)
>>>
>>> # Use small multiples view
>>> fig = visualize_multilayer_network(net, visualization_type="small_
↪multiples")
>>>
>>> # Use edge-colored projection
>>> fig = visualize_multilayer_network(net, visualization_type="edge_
↪colored_projection")
```

`py3plex.visualization.drawing_machinery.draw`(*G*, *pos=None*, *ax=None*, ***kwds*)

Draw the graph G with Matplotlib.

Draw the graph as a simple representation with no node labels or edge labels and using the full Matplotlib figure area and no axis labels by default. See draw_networkx() for more full-featured drawing that allows title, axis labels etc.

Parameters

- **G** (*graph*) – A networkx graph
- **pos** (*dictionary, optional*) – A dictionary with nodes as keys and positions as values. If not specified a spring layout positioning will be computed. See `networkx.drawing.layout` for functions that compute node positions.
- **ax** (*Matplotlib Axes object, optional*) – Draw the graph in specified Matplotlib axes.
- **kwds** (*optional keywords*) – See networkx.draw_networkx() for a description of optional keywords.

Examples

```
>>> G = nx.dodecahedral_graph()
>>> nx.draw(G)
>>> nx.draw(G, pos=nx.spring_layout(G))  # use spring layout
```

See also

draw_networkx, *draw_networkx_nodes*, *draw_networkx_edges*, *draw_networkx_labels*, *draw_networkx_edge_labels*

Notes

This function has the same name as pylab.draw and pyplot.draw so beware when using

```
>>> from networkx import *
```

since you might overwrite the pylab.draw function.

With pyplot use

```
>>> import matplotlib.pyplot as plt
>>> import networkx as nx
>>> G = nx.dodecahedral_graph()
>>> nx.draw(G)  # networkx draw()
>>> plt.draw()  # pyplot draw()
```

Also see the NetworkX drawing examples at https://networkx.github.io/documentation/latest/auto_examples/index.html

`py3plex.visualization.drawing_machinery.draw_circular(G, **kwargs)`

Draw the graph G with a circular layout.

Parameters

- **G** (*graph*) – A networkx graph
- **kwargs** (*optional keywords*) – See networkx.draw_networkx() for a description of optional keywords, with the exception of the pos param-

eter which is not used by this function.

`py3plex.visualization.drawing_machinery.draw_kamada_kawai(G, **kwargs)`

Draw the graph G with a Kamada-Kawai force-directed layout.

Parameters

- **G** (*graph*) – A networkx graph
- **kwargs** (*optional keywords*) – See networkx.draw_networkx() for a description of optional keywords, with the exception of the pos parameter which is not used by this function.

`py3plex.visualization.drawing_machinery.draw_networkx(G, pos=None, arrows=True, with_labels=True, **kwds)`

Draw the graph G using Matplotlib.

Draw the graph with Matplotlib with options for node positions, labeling, titles, and many other drawing features. See draw() for simple drawing without labels or axes.

Parameters

- **G** (*graph*) – A networkx graph
- **pos** (*dictionary, optional*) – A dictionary with nodes as keys and positions as values. If not specified a spring layout positioning will be computed. See `networkx.drawing.layout` for functions that compute node positions.
- **arrows** (*bool, optional (default=True)*) – For directed graphs, if True draw arrowheads. Note: Arrows will be the same color as edges.
- **arrowstyle** (str, optional (default='-|>')) – For directed graphs, choose the style of the arrowsheads. See :py:class: *matplotlib.patches.ArrowStyle* for more options.
- **arrowsize** (*int, optional (default=10)*) – For directed graphs, choose the size of the arrow head head's length and width. See :py:class: *matplotlib.patches.FancyArrowPatch* for attribute *mutation_scale* for more info.
- **with_labels** (*bool, optional (default=True)*) – Set to True to draw labels on the nodes.
- **ax** (*Matplotlib Axes object, optional*) – Draw the graph in the specified Matplotlib axes.
- **nodelist** (*list, optional (default G.nodes())*) – Draw only specified nodes
- **edgelist** (*list, optional (default=G.edges())*) – Draw only specified edges
- **node_size** (*scalar or array, optional (default=300)*) – Size of nodes. If an array is specified it must be the same length as nodelist.
- **node_color** (*color string, or array of floats, (default='r')*) – Node color. Can be a single color format string, or a sequence of colors with the same length as nodelist. If numeric values are specified they will be mapped to colors using the cmap and vmin,vmax parameters. See matplotlib.scatter for more details.

- **node_shape** (*string, optional (default='o')*) – The shape of the node. Specification is as matplotlib.scatter marker, one of 'so^>v<dph8'.
- **alpha** (*float, optional (default=1.0)*) – The node and edge transparency
- **cmap** (*Matplotlib colormap, optional (default=None)*) – Colormap for mapping intensities of nodes
- **vmin** (*float, optional (default=None)*) – Minimum and maximum for node colormap scaling
- **vmax** (*float, optional (default=None)*) – Minimum and maximum for node colormap scaling
- **linewidths** (*[None | scalar | sequence]*) – Line width of symbol border (default =1.0)
- **width** (*float, optional (default=1.0)*) – Line width of edges
- **edge_color** (*color string, or array of floats (default='r')*) – Edge color. Can be a single color format string, or a sequence of colors with the same length as edgelist. If numeric values are specified they will be mapped to colors using the edge_cmap and edge_vmin,edge_vmax parameters.
- **edge_cmap** (*Matplotlib colormap, optional (default=None)*) – Colormap for mapping intensities of edges
- **edge_vmin** (*floats, optional (default=None)*) – Minimum and maximum for edge colormap scaling
- **edge_vmax** (*floats, optional (default=None)*) – Minimum and maximum for edge colormap scaling
- **style** (*string, optional (default='solid')*) – Edge line style (solid|dashed|dotted,dashdot)
- **labels** (*dictionary, optional (default=None)*) – Node labels in a dictionary keyed by node of text labels
- **font_size** (*int, optional (default=12)*) – Font size for text labels
- **font_color** (*string, optional (default='k' black)*) – Font color string
- **font_weight** (*string, optional (default='normal')*) – Font weight
- **font_family** (*string, optional (default='sans-serif')*) – Font family
- **label** (*string, optional*) – Label for graph legend

Notes

For directed graphs, arrows are drawn at the head end. Arrows can be turned off with keyword arrows=False.

Examples

```
>>> G = nx.dodecahedral_graph()
>>> nx.draw(G)
>>> nx.draw(G, pos=nx.spring_layout(G))  # use spring layout
```

```
>>> import matplotlib.pyplot as plt
>>> limits = plt.axis('off')  # turn of axis
```

Also see the NetworkX drawing examples at https://networkx.github.io/documentation/latest/auto_examples/index.html

See also

draw, *draw_networkx_nodes*, *draw_networkx_edges*, *draw_networkx_labels*, *draw_networkx_edge_labels*

`py3plex.visualization.drawing_machinery.`**`draw_networkx_edge_labels`**`(`*G*, *pos*, *edge_labels=None*, *label_pos=0.5*, *font_size=10*, *font_color='k'*, *font_family='sans-serif'*, *font_weight='normal'*, *alpha=1.0*, *bbox=None*, *ax=None*, *rotate=True*, ***kwds*`)`

Draw edge labels.

Parameters

- **G** (*graph*) – A networkx graph
- **pos** (*dictionary*) – A dictionary with nodes as keys and positions as values. Positions should be sequences of length 2.
- **ax** (*Matplotlib Axes object, optional*) – Draw the graph in the specified Matplotlib axes.
- **alpha** (*float*) – The text transparency (default=1.0)
- **edge_labels** (*dictionary*) – Edge labels in a dictionary keyed by edge two-tuple of text labels (default=None). Only labels for the keys in the dictionary are drawn.
- **label_pos** (*float*) – Position of edge label along edge (0=head, 0.5=center, 1=tail)

- **font_size** (*int*) – Font size for text labels (default=12)
- **font_color** (*string*) – Font color string (default='k' black)
- **font_weight** (*string*) – Font weight (default='normal')
- **font_family** (*string*) – Font family (default='sans-serif')
- **bbox** (*Matplotlib bbox*) – Specify text box shape and colors.
- **clip_on** (*bool*) – Turn on clipping at axis boundaries (default=True)

Returns
dict of labels keyed on the edges

Return type
dict

Examples

```
>>> G = nx.dodecahedral_graph()
>>> edge_labels = nx.draw_networkx_edge_labels(G, pos=nx.spring_layout(G))
```

Also see the NetworkX drawing examples at https://networkx.github.io/documentation/latest/auto_examples/index.html

See also

draw, *draw_networkx*, *draw_networkx_nodes*, *draw_networkx_edges*, *draw_networkx_labels*

`py3plex.visualization.drawing_machinery.`**`draw_networkx_edges`**(*G*, *pos*, *edgelist=None*, *width=1.0*, *edge_color='k'*, *style='solid'*, *alpha=1.0*, *arrowstyle='-|>'*, *arrowsize=10*, *edge_cmap=None*, *edge_vmin=None*, *edge_vmax=None*, *ax=None*, *arrows=True*, *label=None*, *node_size=300*, *nodelist=None*, *node_shape='o'*, ***kwds*)

Draw the edges of the graph G.

This draws only the edges of the graph G.

Parameters

- **G** (*graph*) – A networkx graph

- **pos** (*dictionary*) – A dictionary with nodes as keys and positions as values. Positions should be sequences of length 2.
- **edgelist** (*collection of edge tuples*) – Draw only specified edges(default=G.edges())
- **width** (*float, or array of floats*) – Line width of edges (default=1.0)
- **edge_color** (*color string, or array of floats*) – Edge color. Can be a single color format string (default='r'), or a sequence of colors with the same length as edgelist. If numeric values are specified they will be mapped to colors using the edge_cmap and edge_vmin,edge_vmax parameters.
- **style** (*string*) – Edge line style (default='solid') (solid|dashed|dotted,dashdot)
- **alpha** (*float*) – The edge transparency (default=1.0)
- **cmap** (*edge*) – Colormap for mapping intensities of edges (default=None)
- **edge_vmin** (*floats*) – Minimum and maximum for edge colormap scaling (default=None)
- **edge_vmax** (*floats*) – Minimum and maximum for edge colormap scaling (default=None)
- **ax** (*Matplotlib Axes object, optional*) – Draw the graph in the specified Matplotlib axes.
- **arrows** (*bool, optional (default=True)*) – For directed graphs, if True draw arrowheads. Note: Arrows will be the same color as edges.
- **arrowstyle** (str, optional (default='-|>')) – For directed graphs, choose the style of the arrow heads. See :py:class: *matplotlib.patches.ArrowStyle* for more options.
- **arrowsize** (*int, optional (default=10)*) – For directed graphs, choose the size of the arrow head head's length and width. See :py:class: *matplotlib.patches.FancyArrowPatch* for attribute *mutation_scale* for more info.
- **label** (*[None | string]*) – Label for legend

Returns

- *matplotlib.collection.LineCollection* – *LineCollection* of the edges
- *list of matplotlib.patches.FancyArrowPatch* – *FancyArrowPatch* instances of the directed edges
- *Depending whether the drawing includes arrows or not.*

Notes

For directed graphs, arrows are drawn at the head end. Arrows can be turned off with keyword arrows=False. Be sure to include *node_size* as a keyword argument; arrows are drawn considering the size of nodes.

Examples

```
>>> G = nx.dodecahedral_graph()
>>> edges = nx.draw_networkx_edges(G, pos=nx.spring_layout(G))
```

```
>>> G = nx.DiGraph()
>>> G.add_edges_from([(1, 2), (1, 3), (2, 3)])
>>> arcs = nx.draw_networkx_edges(G, pos=nx.spring_layout(G))
>>> alphas = [0.3, 0.4, 0.5]
>>> for i, arc in enumerate(arcs):  # change alpha values of arcs
...     arc.set_alpha(alphas[i])
```

Also see the NetworkX drawing examples at https://networkx.github.io/documentation/latest/auto_examples/index.html

See also

draw, *draw_networkx*, *draw_networkx_nodes*, *draw_networkx_labels*, *draw_networkx_edge_labels*

`py3plex.visualization.drawing_machinery.`**`draw_networkx_labels`**(*G, pos, labels=None, font_size=1, font_color='k', font_family='sans-serif', font_weight='normal', alpha=1.0, bbox=None, ax=None, **kwds*)

Draw node labels on the graph G.

Parameters

- **G** (*graph*) – A networkx graph
- **pos** (*dictionary*) – A dictionary with nodes as keys and positions as values. Positions should be sequences of length 2.
- **labels** (*dictionary, optional (default=None)*) – Node labels in a dictionary keyed by node of text labels Node-keys in labels should appear as keys in *pos*. If needed use: *{n:lab for n,lab in labels.items() if n in pos}*
- **font_size** (*int*) – Font size for text labels (default=12)
- **font_color** (*string*) – Font color string (default='k' black)
- **font_family** (*string*) – Font family (default='sans-serif')
- **font_weight** (*string*) – Font weight (default='normal')

- **alpha** (*float*) – The text transparency (default=1.0)
- **ax** (*Matplotlib Axes object, optional*) – Draw the graph in the specified Matplotlib axes.

Returns
dict of labels keyed on the nodes

Return type
dict

Examples

```
>>> G = nx.dodecahedral_graph()
>>> labels = nx.draw_networkx_labels(G, pos=nx.spring_layout(G))
```

Also see the NetworkX drawing examples at https://networkx.github.io/documentation/latest/auto_examples/index.html

See also

draw, *draw_networkx*, *draw_networkx_nodes*, *draw_networkx_edges*, *draw_networkx_edge_labels*

py3plex.visualization.drawing_machinery.**draw_networkx_nodes**(*G*, *pos*, *nodelist=None*, *node_size=300*, *node_color='r'*, *node_shape='o'*, *alpha=1.0*, *cmap=None*, *vmin=None*, *vmax=None*, *ax=None*, *linewidths=None*, *edgecolors=None*, *label=None*, ***kwds*)

Draw the nodes of the graph G.

This draws only the nodes of the graph G.

Parameters

- **G** (*graph*) – A networkx graph
- **pos** (*dictionary*) – A dictionary with nodes as keys and positions as values. Positions should be sequences of length 2.
- **ax** (*Matplotlib Axes object, optional*) – Draw the graph in the specified Matplotlib axes.
- **nodelist** (*list, optional*) – Draw only specified nodes (default G.nodes())
- **node_size** (*scalar or array*) – Size of nodes (default=300). If an array is specified it must be the same length as nodelist.

- **node_color** (*color string, or array of floats*) – Node color. Can be a single color format string (default='r'), or a sequence of colors with the same length as nodelist. If numeric values are specified they will be mapped to colors using the cmap and vmin,vmax parameters. See matplotlib.scatter for more details.
- **node_shape** (*string*) – The shape of the node. Specification is as matplotlib.scatter marker, one of 'so^>v<dph8' (default='o').
- **alpha** (*float or array of floats*) – The node transparency. This can be a single alpha value (default=1.0), in which case it will be applied to all the nodes of color. Otherwise, if it is an array, the elements of alpha will be applied to the colors in order (cycling through alpha multiple times if necessary).
- **cmap** (*Matplotlib colormap*) – Colormap for mapping intensities of nodes (default=None)
- **vmin** (*floats*) – Minimum and maximum for node colormap scaling (default=None)
- **vmax** (*floats*) – Minimum and maximum for node colormap scaling (default=None)
- **linewidths** (*[None | scalar | sequence]*) – Line width of symbol border (default =1.0)
- **edgecolors** (*[None | scalar | sequence]*) – Colors of node borders (default = node_color)
- **label** (*[None | string]*) – Label for legend

Returns
PathCollection of the nodes.

Return type
matplotlib.collections.PathCollection

Examples

```
>>> G = nx.dodecahedral_graph()
>>> nodes = nx.draw_networkx_nodes(G, pos=nx.spring_layout(G))
```

Also see the NetworkX drawing examples at https://networkx.github.io/documentation/latest/auto_examples/index.html

See also

draw, *draw_networkx*, *draw_networkx_edges*, *draw_networkx_labels*, *draw_networkx_edge_labels*

py3plex.visualization.drawing_machinery.**draw_random**(*G*, ***kwargs*)

Draw the graph G with a random layout.

Parameters

- **G** (*graph*) – A networkx graph
- **kwargs** (*optional keywords*) – See networkx.draw_networkx() for a

description of optional keywords, with the exception of the pos parameter which is not used by this function.

`py3plex.visualization.drawing_machinery.`**`draw_shell`**`(`*G*, ***kwargs*`)`

Draw networkx graph with shell layout.

Parameters

- **G** (*graph*) – A networkx graph
- **kwargs** (*optional keywords*) – See networkx.draw_networkx() for a description of optional keywords, with the exception of the pos parameter which is not used by this function.

`py3plex.visualization.drawing_machinery.`**`draw_spectral`**`(`*G*, ***kwargs*`)`

Draw the graph G with a spectral layout.

Parameters

- **G** (*graph*) – A networkx graph
- **kwargs** (*optional keywords*) – See networkx.draw_networkx() for a description of optional keywords, with the exception of the pos parameter which is not used by this function.

`py3plex.visualization.drawing_machinery.`**`draw_spring`**`(`*G*, ***kwargs*`)`

Draw the graph G with a spring layout.

Parameters

- **G** (*graph*) – A networkx graph
- **kwargs** (*optional keywords*) – See networkx.draw_networkx() for a description of optional keywords, with the exception of the pos parameter which is not used by this function.

Color utilities for py3plex visualization.

`py3plex.visualization.colors.`**`RGB_to_hex`**`(`*RGB: List[int]*`)` → str

Convert RGB list to hex color.

Parameters
RGB – RGB values as [R, G, B] list

Returns
Hex color string like "#FFFFFF"

`py3plex.visualization.colors.`**`color_dict`**`(`*gradient: List[List[int]]*`)` → Dict[str, List]

Takes in a list of RGB sub-lists and returns dictionary of colors in RGB and hex form for use in a graphing function defined later on.

Parameters
gradient – List of RGB color values

Returns
Dictionary with 'hex', 'r', 'g', 'b' keys

`py3plex.visualization.colors.`**`hex_to_RGB`**`(`*hex: str*`)` → List[int]

Convert hex color to RGB list.

Parameters
hex – Hex color string like "#FFFFFF"

Returns

RGB values as [R, G, B] list

`py3plex.visualization.colors.linear_gradient`(*start_hex: str*, *finish_hex: str = '#FFFFFF'*, *n: int = 10*) → Dict[str, List]

Returns a gradient list of (n) colors between two hex colors.

Parameters

- **start_hex** – Starting color as six-digit hex string (e.g., "#FFFFFF")
- **finish_hex** – Ending color as six-digit hex string (default: "#FFFFFF")
- **n** – Number of colors in gradient (default: 10)

Returns

Dictionary with 'hex', 'r', 'g', 'b' keys containing gradient colors

`py3plex.visualization.layout_algorithms.compute_force_directed_layout`(*g: Graph*, *layout_parameters: Dict[str, Any] | None = None*, *verbose: bool = True*, *gravity: float = 0.2*, *strongGravityMode: bool = False*, *barnesHutTheta: float = 1.2*, *edgeWeightInfluence: float = 1*, *scalingRatio: float = 2.0*, *forceImport: bool = True*, *seed: int | None = None*) → Dict[Any, ndarray]

Compute force-directed layout for a graph using ForceAtlas2 or NetworkX spring layout.

Parameters

- **g** – NetworkX graph to layout
- **layout_parameters** – Optional parameters to pass to layout algorithm
- **verbose** – Whether to print progress information
- **gravity** – Attraction force towards the center (must be non-negative)
- **strongGravityMode** – Use strong gravity mode
- **barnesHutTheta** – Barnes-Hut approximation parameter
- **edgeWeightInfluence** – Influence of edge weights on layout
- **scalingRatio** – Scaling factor for the layout (must be positive)
- **forceImport** – Whether to use ForceAtlas2 (if available)
- **seed** – Random seed for reproducibility in fallback spring layout

Returns

Dictionary mapping nodes to 2D position arrays

> **Note**
>
> For large networks (>1000 nodes), this may be slow. Consider using faster layouts (circular, random, spectral) or matrix visualization.

Contracts:

- Precondition: graph must not be None and be a NetworkX graph
- Precondition: graph must have at least one node
- Precondition: gravity must be non-negative
- Precondition: scalingRatio must be positive
- Postcondition: result is a dictionary
- Postcondition: result has positions for all nodes

`py3plex.visualization.layout_algorithms.compute_random_layout`(*g: Graph*, *seed: int | None = None*) → Dict[Any, ndarray]

Compute a random layout for the graph.

Parameters

- **g** – NetworkX graph
- **seed** – Random seed for reproducibility

Returns

Dictionary mapping nodes to 2D positions

Contracts:

- Precondition: graph must not be None and be a NetworkX graph
- Precondition: graph must have at least one node

- Postcondition: result is a dictionary
- Postcondition: result has positions for all nodes

`py3plex.visualization.layout_algorithms.ensure`(**args*, ***kwargs*)

`py3plex.visualization.layout_algorithms.require`(**args*, ***kwargs*)

`py3plex.visualization.bezier.bezier_calculate_dfy`(*mp_y: float*, *path_height: float*, *x0: float*, *midpoint_x: float*, *x1: float*, *y0: float*, *y1: float*, *dfx: ndarray*, *mode: str = 'upper'*) → ndarray

Calculate y-coordinates for bezier curve.

Parameters

- **mp_y** – Midpoint y-coordinate
- **path_height** – Height of the path
- **x0** – Start x-coordinate
- **midpoint_x** – Midpoint x-coordinate
- **x1** – End x-coordinate
- **y0** – Start y-coordinate
- **y1** – End y-coordinate
- **dfx** – Array of x-coordinates
- **mode** – Mode for curve calculation ("upper" or "bottom")

Returns

Array of y-coordinates

`py3plex.visualization.bezier.draw_bezier`(*total_size: int*, *p1: Tuple[float, float]*, *p2: Tuple[float, float]*, *mode: str = 'quadratic'*, *inversion: bool = False*, *path_height: float = 2*, *linemode: str = 'both'*, *resolution: float = 0.1*) → Tuple[ndarray, ndarray]

Draw bezier curve between two points.

Parameters

- **total_size** – Total size of the drawing area
- **p1** – First point coordinates (x0, x1)
- **p2** – Second point coordinates (y0, y1)
- **mode** – Drawing mode (default: "quadratic")
- **inversion** – Whether to invert the curve
- **path_height** – Height of the path
- **linemode** – Line drawing mode ("upper", "bottom", or "both")
- **resolution** – Resolution for curve sampling

Returns

Tuple of (x-coordinates, y-coordinates) arrays `py3plex.visualization.benchmark_visualizations.generic_grouping`(*fname: DataFrame*, *score_name: str*, *threshold: float = 1.0*, *percentages: bool = True*) → DataFrame

`py3plex.visualization.benchmark_visualizations.plot_core_macro`(*fname: DataFrame*) → int

A very simple visualization of the results..

`py3plex.visualization.benchmark_visualizations.plot_core_macro_box`(*fname: str*) → int

`py3plex.visualization.benchmark_visualizations.plot_core_macro_gg`(*fnamex: DataFrame*) → None

`py3plex.visualization.benchmark_visualizations.plot_core_micro`(*fname: DataFrame*) → int

A very simple visualization of the results..

`py3plex.visualization.benchmark_visualizations.plot_core_micro_gg`(*fnamex: DataFrame*) → None

`py3plex.visualization.benchmark_visualizations.plot_core_micro_grid`(*fname: str*) → None

`py3plex.visualization.benchmark_visualizations.plot_core_time`(*fnamex: DataFrame*) → int

`py3plex.visualization.benchmark_visualizations.plot_core_time_gg`(*fname: str*) → None

`py3plex.visualization.benchmark_visualizations.plot_core_variability`(*fname: str*) → None

`py3plex.visualization.benchmark_visualizations.plot_critical_distance`(*fname: str*, *num_algo: int = 14*) → None

`py3plex.visualization.benchmark_visualizations.plot_mean_times`(*fn: DataFrame*) → None

`py3plex.visualization.benchmark_visualizations.plot_robustness`(*infile: DataFrame*) → None

`py3plex.visualization.benchmark_visualizations.table_to_latex`(*fname*, *outfolder='../final_results/tables/'*, *threshold=1*)

Wrappers

class py3plex.wrappers.benchmark_nodes.TopKRanker(*estimator*, ***, *n_jobs=None*, *verbose=0*)

Bases: `OneVsRestClassifier`

predict(*X: ndarray*, *top_k_list: List[int]*) → List[List]

Predict top K labels for each sample.

Parameters

- **X** – Feature matrix
- **top_k_list** – List of K values for each sample

Returns

List of predicted labels for each sample

set_partial_fit_request(***, *classes: bool | None | str = '$UNCHANGED$'*) → *TopKRanker*

Configure whether metadata should be requested to be passed to the `partial_fit` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config()`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

- `True`: metadata is requested, and passed to `partial_fit` if provided. The request is ignored if metadata is not provided.
- `False`: metadata is not requested and the meta-estimator will not pass it to `partial_fit`.
- `None`: metadata is not requested, and the meta-estimator will raise an error if the user provides it.
- `str`: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Parameters

classes (*str, True, False, or None, default=sklearn.utils.metadata_routing.UNCHANGED*) – Metadata routing for `classes` parameter in `partial_fit`.

Returns

self – The updated object.

Return type

object

set_predict_request(***, *top_k_list: bool | None | str = '$UNCHANGED$'*) → *TopKRanker*

Configure whether metadata should be requested to be passed to the `predict` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config()`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

- `True`: metadata is requested, and passed to `predict` if provided. The request is ignored if metadata is not provided.
- `False`: metadata is not requested and the meta-estimator will not pass it to `predict`.
- `None`: metadata is not requested, and the meta-estimator will raise an error if the user provides it.
- `str`: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Parameters
`top_k_list` (*str, True, False, or None, default=sklearn.utils.metadata_routing.UNCHANGED*) – Metadata routing for `top_k_list` parameter in `predict`.

Returns
self – The updated object.

Return type
object

set_score_request(**, sample_weight: bool | None | str = '$UNCHANGED$'*) → *TopKRanker*

Configure whether metadata should be requested to be passed to the `score` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config()`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

- `True`: metadata is requested, and passed to `score` if provided. The request is ignored if metadata is not provided.
- `False`: metadata is not requested and the meta-estimator will not pass it to `score`.
- `None`: metadata is not requested, and the meta-estimator will raise an error if the user provides it.
- `str`: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Parameters
sample_weight (*str, True, False, or None, default=sklearn.utils.metadata_routing.UNCHANGED*) – Metadata routing for `sample_weight` parameter in `score`.

Returns
self – The updated object.

Return type
object

`py3plex.wrappers.benchmark_nodes.benchmark_node_classification(path: str, core_network: Any, labels_matrix: Any, percent: Any = 'all') → Dict[Any, Any]`

Benchmark node classification using embeddings.

Parameters

- **path** – Path to embeddings file
- **core_network** – Network adjacency matrix
- **labels_matrix** – Labels for nodes
- **percent** – Training percentage or "all" for multiple percentages

Returns
Dictionary of classification results

`py3plex.wrappers.benchmark_nodes.main()`

`py3plex.wrappers.benchmark_nodes.sparse2graph(x: spmatrix) → Dict[str, List[str]]`

Convert sparse matrix to graph dictionary.

Parameters
x – Sparse matrix representation of graph

Returns
Dictionary mapping node IDs to lists of neighbor IDs

I/O Operations

For the complete auto-generated API documentation, see the AUTOGEN_results directory after building the docs. Aggregation and Network Operations ——————————- Vectorized multiplex aggregation for multilayer networks.

This module provides optimized implementations for aggregating edges across multiple layers using vectorized NumPy and SciPy sparse operations, replacing slower Python loops with efficient matrix operations.

Performance targets: - 3× speedup for 1M edges across 4 layers compared to legacy loop-based methods - Memory-efficient sparse matrix output by default - Float tolerance of 1e-6 for numerical equivalence with legacy methods

Author: py3plex development team License: MIT `py3plex.multinet.aggregation.aggregate_layers(edges: ndarray | list, weight_col: str | int = 'w', reducer: Literal['sum', 'mean', 'max'] = 'sum', to_sparse: bool = True) → csr_matrix | ndarray`

Aggregate edge weights across multiple layers using vectorized operations.

This function replaces Python loops with efficient NumPy and SciPy sparse matrix operations for superior performance on large multilayer networks.

Complexity: O(E) where E is the number of edges **Memory**: O(E) for sparse output, O(N^2) for dense output

Parameters

- **edges** – Edge data as ndarray with shape (E, >=3) containing (layer, src, dst, [weight]) columns. If no weight column, assumes weight=1.0 for all edges. Can also accept list of lists.
- **weight_col** – Column name or index for weights (default "w"). If edges is ndarray, must be int index (0-based). Column 3 is used if it exists, else weights default to 1.
- **reducer** – Aggregation method - one of: - "sum": Sum weights for edges appearing in multiple layers (default) - "mean": Average weights across layers - "max": Take maximum weight across layers
- **to_sparse** – If True, return scipy.sparse.csr_matrix (default, memory-efficient). If False, return dense numpy.ndarray.

Returns

Aggregated adjacency matrix in requested format (sparse CSR or dense). Shape is (N, N) where N is the maximum node ID + 1.

Raises

- **ValueError** – If edges array has wrong shape or reducer is invalid.
- **TypeError** – If edges is not ndarray or list-like.

Examples

```
>>> import numpy as np
>>> # Create edge list: (layer, src, dst, weight)
>>> edges = np.array([
...     [0, 0, 1, 1.0],
...     [0, 1, 2, 2.0],
...     [1, 0, 1, 0.5],   # Same edge in layer 1
...     [1, 2, 3, 1.5],
... ])
>>> mat = aggregate_layers(edges, reducer="sum")
>>> mat.shape
(4, 4)
>>> mat[0, 1]  # Sum of weights from both layers
1.5
```

```
>>> # With mean aggregation
>>> mat_mean = aggregate_layers(edges, reducer="mean", to_sparse=False)
```

(continues on next page)

(continued from previous page)

```
>>> mat_mean[0, 1]  # Average of 1.0 and 0.5
0.75
```

Notes

- Node IDs are assumed to be integers starting from 0
- Self-loops are supported
- For directed graphs, (i,j) and (j,i) are different edges
- Sparse output recommended for large networks (N > 1000)
- Deterministic output for fixed input order

Performance:
Achieves 3× speedup vs loop-based aggregation on 1M edges: - Legacy loop: ~2.5s for 1M edges, 4 layers - Vectorized: ~0.8s for same dataset (measured on standard hardware)

py3plex.multinet.aggregation.**ensure**(**args*, ***kwargs*)

py3plex.multinet.aggregation.**require**(**args*, ***kwargs*)

Profiling Utilities

Performance profiling utilities for py3plex.

This module provides decorators and utilities for tracking function execution time, memory usage, and performance metrics. These tools enable performance regression detection and optimization efforts.

Example

```
>>> from py3plex.profiling import profile_performance
>>>
>>> @profile_performance
>>> def slow_function():
...     # ... computation
...     pass
>>>
>>> slow_function()  # Logs execution time automatically
```

class py3plex.profiling.**PerformanceMonitor**

Bases: `object`

Global performance monitoring registry.

Tracks execution times and call counts for profiled functions. Can be used to generate performance reports and detect regressions.

enabled

Whether profiling is enabled globally

stats

Dictionary mapping function names to performance statistics

clear()

Clear all collected statistics.

get_report() → str

Generate a performance report.

Returns
String containing formatted performance statistics

record(*func_name: str*, *elapsed: float*, *memory_delta: float | None = None*)

Record performance metrics for a function call.

Parameters

- **func_name** – Name of the function
- **elapsed** – Execution time in seconds
- **memory_delta** – Memory usage change in MB (optional)

py3plex.profiling.**benchmark**(*func: Callable*, *iterations: int = 100*, *warmup: int = 10*, *args: tuple = ()*, *kwargs: dict | None = None*) → Dict[str, float]

Benchmark a function with multiple iterations.

Runs the function multiple times and collects statistics about execution time. Includes a warmup phase to allow JIT compilation and caching.

Parameters

- **func** – Function to benchmark
- **iterations** – Number of iterations to run (default: 100)
- **warmup** – Number of warmup iterations (default: 10)
- **args** – Positional arguments for the function
- **kwargs** – Keyword arguments for the function

Returns

- mean: Average execution time (seconds)
- median: Median execution time (seconds)
- min: Minimum execution time (seconds)
- max: Maximum execution time (seconds)
- std: Standard deviation (seconds)
- total: Total time for all iterations (seconds)

Return type
Dictionary containing benchmark statistics

Example

```
>>> from py3plex.profiling import benchmark
>>>
>>> def my_function(n):
...     return sum(range(n))
```

(continues on next page)

(continued from previous page)

```
>>>
>>> stats = benchmark(my_function, iterations=1000, args=(1000,))
>>> print(f"Average time: {stats['mean']*1000:.3f}ms")
```

py3plex.profiling.**get_monitor**() → *PerformanceMonitor*

Get the global performance monitor instance.

Returns
Global performance monitoring instance

Return type
PerformanceMonitor

Example

```
>>> from py3plex.profiling import get_monitor
>>> monitor = get_monitor()
>>> print(monitor.get_report())
```

py3plex.profiling.**profile_performance**(*func: Callable | None = None*, ***, *log_args: bool = False*, *track_memory: bool = False*) → Callable

Decorator to track function execution time and optionally memory usage.

This decorator measures the wall-clock time taken by a function and logs it. It can also track memory usage changes if requested. Performance metrics are stored in the global performance monitor for later analysis.

Parameters

- **func** – Function to decorate (when used without arguments)
- **log_args** – If True, log function arguments (default: False)
- **track_memory** – If True, track memory usage (default: False)

Returns
Decorated function that logs execution time

Examples

Basic usage: >>> @profile_performance … def my_function(x, y): … return x + y

With options: >>> @profile_performance(log_args=True, track_memory=True) … def expensive_function(data): … # … expensive computation … return result

Manual wrapping: >>> def my_function(): … pass >>> profiled_func = profile_performance(my_function)

> **Note**
>
> Memory tracking requires the tracemalloc module and adds overhead. Use it only when investigating memory issues.

py3plex.profiling.**timed_section**(*name: str*, *log_level: str = 'info'*)

Context manager for timing code blocks.

Useful for timing specific sections of code without wrapping entire functions.

Parameters

- **name** – Name of the code section being timed
- **log_level** – Logging level ('debug', 'info', 'warning', 'error')

Example

```
>>> from py3plex.profiling import timed_section
>>>
>>> with timed_section("data loading"):
...     data = load_large_dataset()
>>>
>>> with timed_section("computation", log_level="debug"):
...     result = complex_calculation()
```

Yields

None

Logging Configuration

Logging configuration for py3plex.

This module provides centralized logging configuration for the py3plex library.

`py3plex.logging_config.get_logger`(*name: str | None = None*, *level: int = 20*) → Logger

Get or create a logger for py3plex modules.

Parameters

- **name** – Logger name. If None, returns the root py3plex logger.
- **level** – Logging level (default: INFO)

Returns

Configured logger instance

Example

```
>>> from py3plex.logging_config import get_logger
>>> logger = get_logger(__name__)
>>> logger.info("Processing network...")
```

`py3plex.logging_config.setup_logging`(*level: int = 20*, *format_string: str | None = None*) → Logger

Configure logging for py3plex with custom settings.

Parameters

- **level** – Logging level (default: INFO)
- **format_string** – Custom format string (optional)

Returns

Root py3plex logger

Example

```
>>> from py3plex.logging_config import setup_logging
>>> logger = setup_logging(level=logging.DEBUG)
```

I/O Schema and Validation

Schema definitions for multilayer graphs using dataclasses.

This module provides dataclass representations of multilayer graph components with built-in validation and serialization support.

`class py3plex.io.schema.Edge`(*src: ~typing.Hashable, dst: ~typing.Hashable, src_layer: ~typing.Hashable, dst_layer: ~typing.Hashable, key: int = 0, attributes: ~typing.Dict[str, ~typing.Any] = <factory>*)

Bases: `object`

Represents an edge in a multilayer network.

`src`

Source node ID

Type

Hashable

`dst`

Destination node ID

Type

Hashable

`src_layer`

Source layer ID

Type

Hashable

`dst_layer`

Destination layer ID

Type

Hashable

`key`

Optional edge key for multigraphs (default: 0)

Type

int

`attributes`

Dictionary of edge attributes (must be JSON-serializable)

Type

Dict[str, Any]

`attributes: Dict[str, Any]`

`dst: Hashable`

`dst_layer: Hashable`

edge_tuple() → Tuple[Hashable, Hashable, Hashable, Hashable, int]

Return edge as a tuple for uniqueness checking.

Returns
Tuple of (src, dst, src_layer, dst_layer, key)

classmethod from_dict(*data: Dict[str, Any]*) → Edge

Create edge from dictionary.

Parameters
data – Dictionary containing edge data

Returns
Edge instance

key: int = 0

src: Hashable

src_layer: Hashable

to_dict() → Dict[str, Any]

Convert edge to dictionary.

Returns
Dictionary representation of the edge

class py3plex.io.schema.Layer(*id: ~typing.Hashable*, *attributes: ~typing.Dict[str, ~typing.Any] = <factory>*)

Bases: `object`

Represents a layer in a multilayer network.

id

Unique identifier for the layer

Type
Hashable

attributes

Dictionary of layer attributes (must be JSON-serializable)

Type
Dict[str, Any]

attributes: Dict[str, Any]

classmethod from_dict(*data: Dict[str, Any]*) → Layer

Create layer from dictionary.

Parameters
data – Dictionary containing layer data

Returns
Layer instance

id: Hashable

to_dict() → Dict[str, Any]

Convert layer to dictionary.

Returns
Dictionary representation of the layer

class py3plex.io.schema.MultiLayerGraph(*nodes: ~typing.Dict[~typing.Hashable, ~py3plex.io.schema.Node] = <factory>, layers: ~typing.Dict[~typing.Hashable, ~py3plex.io.schema.Layer] = <factory>, edges: ~typing.List[~py3plex.io.schema.Edge] = <factory>, directed: bool = True, attributes: ~typing.Dict[str, ~typing.Any] = <factory>*)

Bases: `object`

Represents a complete multilayer graph.

nodes

Dictionary mapping node IDs to Node objects

Type
Dict[Hashable, py3plex.io.schema.Node]

layers

Dictionary mapping layer IDs to Layer objects

Type
Dict[Hashable, py3plex.io.schema.Layer]

edges

List of Edge objects

Type
List[py3plex.io.schema.Edge]

directed

Whether the graph is directed

Type
bool

attributes

Dictionary of graph-level attributes

Type
Dict[str, Any]

add_edge(*edge: Edge*)

Add an edge to the graph.

Parameters
edge – Edge to add

Raises

- **ReferentialIntegrityError** – If edge references non-existent nodes or layers
- **SchemaValidationError** – If edge is duplicate

add_layer(*layer: Layer*)

Add a layer to the graph.

Parameters
`layer` – Layer to add

Raises
`SchemaValidationError` – If layer ID already exists

`add_node`(*node: Node*)

Add a node to the graph.

Parameters
`node` – Node to add

Raises
`SchemaValidationError` – If node ID already exists

`attributes: Dict[str, Any]`

`directed: bool = True`

`edges: List[Edge]`

`classmethod from_dict`(*data: Dict[str, Any]*) → MultiLayerGraph

Create graph from dictionary.

Parameters
`data` – Dictionary containing graph data

Returns
MultiLayerGraph instance

`layers: Dict[Hashable, Layer]`

`nodes: Dict[Hashable, Node]`

`to_dict`() → Dict[str, Any]

Convert graph to dictionary.

Returns
Dictionary representation of the graph

`class py3plex.io.schema.Node`(*id: ~typing.Hashable*, *attributes: ~typing.Dict[str, ~typing.Any] = <factory>*)

Bases: `object`

Represents a node in a multilayer network.

`id`

Unique identifier for the node

Type
Hashable

`attributes`

Dictionary of node attributes (must be JSON-serializable)

Type
Dict[str, Any]

`attributes: Dict[str, Any]`

`classmethod from_dict`(*data: Dict[str, Any]*) → Node

Create node from dictionary.

Parameters
`data` – Dictionary containing node data

Returns
Node instance

`id: Hashable`

`to_dict()` → Dict[str, Any]

Convert node to dictionary.

Returns
Dictionary representation of the node

Public API for reading and writing multilayer graphs.

This module provides the main entry points for I/O operations with format detection and a registry system for extensibility.

`py3plex.io.api.ensure(*args, **kwargs)`

`py3plex.io.api.read(filepath: str | Path, format: str | None = None, **kwargs)` → MultiLayerGraph

Read a multilayer graph from a file.

Parameters

- `filepath` – Path to the input file
- `format` – Format name (e.g., 'json', 'csv'). If None, auto-detected from extension
- `**kwargs` – Additional arguments passed to the format-specific reader

Returns
MultiLayerGraph instance

Raises

- `FormatUnsupportedError` – If format is not supported or cannot be detected
- `FileNotFoundError` – If file does not exist

Example

```
>>> graph = read('network.json')
>>> graph = read('network.csv', format='csv')
```

`py3plex.io.api.register_reader(format_name: str, reader_func: Callable[[...], MultiLayerGraph])` → None

Register a reader function for a specific format.

Parameters

- `format_name` – Name of the format (e.g., 'json', 'csv', 'graphml')
- `reader_func` – Function that takes (filepath, **kwargs) and returns MultiLayerGraph

Example

```
>>> def my_reader(filepath, **kwargs):
...     # Custom reading logic
...     return MultiLayerGraph(...)
>>> register_reader('myformat', my_reader)
```

Contracts:

- Precondition: format_name must be a non-empty string
- Precondition: reader_func must be callable
- Postcondition: reader is registered in _READERS

`py3plex.io.api.`**`register_writer`**(*format_name: str*, *writer_func: Callable[[...], None]*) → None

Register a writer function for a specific format.

Parameters

- **`format_name`** – Name of the format (e.g., 'json', 'csv', 'graphml')
- **`writer_func`** – Function that takes (graph, filepath, **kwargs) and writes to file

Example

```
>>> def my_writer(graph, filepath, **kwargs):
...     # Custom writing logic
...     pass
>>> register_writer('myformat', my_writer)
```

Contracts:

- Precondition: format_name must be a non-empty string
- Precondition: writer_func must be callable
- Postcondition: writer is registered in _WRITERS

`py3plex.io.api.`**`require`**(**args*, ***kwargs*)

`py3plex.io.api.`**`supported_formats`**(*read: bool = True*, *write: bool = True*) → Dict[str, List[str]]

Get list of supported formats for read and/or write operations.

Parameters

- **`read`** – Include formats that support reading
- **`write`** – Include formats that support writing

Returns

Dictionary with 'read' and/or 'write' keys containing lists of format names

Example

```
>>> formats = supported_formats()
>>> print(formats)
{'read': ['json', 'jsonl', 'csv'], 'write': ['json', 'jsonl', 'csv']}
```

Contracts:

- Postcondition: result is a dictionary
- Postcondition: result contains 'read' key when read=True
- Postcondition: result contains 'write' key when write=True

`py3plex.io.api.`**`write`**(*graph: MultiLayerGraph*, *filepath: str | Path*, *format: str | None = None*, ***kwargs*) → None

Write a multilayer graph to a file.

Parameters

- **graph** – MultiLayerGraph to write
- **filepath** – Path to the output file
- **format** – Format name (e.g., 'json', 'csv'). If None, auto-detected from extension
- ****kwargs** – Additional arguments passed to the format-specific writer

Raises

FormatUnsupportedError – If format is not supported or cannot be detected

Example

```
>>> write(graph, 'network.json')
>>> write(graph, 'network.csv', format='csv', deterministic=True)
```

Hedwig Rule Learning

`py3plex.algorithms.hedwig.`**`build_graph`**(*kwargs: Dict[str, Any]*) → Any

`py3plex.algorithms.hedwig.`**`generate_rules_report`**(*kwargs: ~typing.Dict[str, ~typing.Any], rules_per_target: ~typing.List[~typing.Tuple[~typing.Any, ~typing.List[~typing.Any]]], human: ~typing.Callable[[~typing.Any, ~typing.Any], ~typing.Any] = <function <lambda>>*) → str

`py3plex.algorithms.hedwig.`**`rule_kernel`**(*target: Any*) → Tuple[Any, List[Any]]

`py3plex.algorithms.hedwig.`**`run`**(*kwargs: Dict[str, Any]*, *cli: bool = True*, *generator_tag: bool = False*, *num_threads: str | int = 'all'*) → List[Tuple[Any, List[Any]]] `py3plex.algorithms.hedwig.run_learner`(*kwargs: Dict[str, Any]*, *kb:* ExperimentKB, *validator:* Validate, *generator: bool = False*, *num_threads: str | int = 'all'*) → List[Tuple[Any, List[Any]]]

Example-related classes.

@author: anze.vavpetic@ijs.si

class `py3plex.algorithms.hedwig.core.example.Example`(*id*, *label*, *score*, *annotations=None*, *weights=None*)

Bases: `object`

Represents an example with its score, label, id and annotations.

ClassLabeled = 'class'

Ranked = 'ranked'

Predicate-related classes.

@author: anze.vavpetic@ijs.si

class `py3plex.algorithms.hedwig.core.predicate.BinaryPredicate`(*label*, *pairs*, *kb*, *producer_pred=None*)

Bases: *Predicate*

A binary predicate.

class `py3plex.algorithms.hedwig.core.predicate.Predicate`(*label*, *kb*, *producer_pred*)

Bases: `object`

Represents a predicate as a member of a certain rule.

i = -1

class `py3plex.algorithms.hedwig.core.predicate.UnaryPredicate`(*label*, *members*, *kb*, *producer_pred=None*, *custom_var_name=None*, *negated=False*)

Bases: *Predicate*

A unary predicate.

The rule class.

@author: anze.vavpetic@ijs.si

class `py3plex.algorithms.hedwig.core.rule.Rule`(*kb*, *predicates=None*, *target=None*)

Bases: `object`

Represents a rule, along with its description, examples and statistics.

clone()

Returns a clone of this rule. The predicates themselves are NOT cloned.

clone_append(*predicate_label*, *producer_pred*, *bin=False*)

Returns a copy of this rule where 'predicate_label' is appended to the rule.

clone_negate(*target_pred*)

Returns a copy of this rule where 'taget_pred' is negated.

clone_swap_with_subclass(*target_pred*, *child_pred_label*)

Returns a copy of this rule where 'target_pred' is swapped for 'child_pred_label'.

examples(*positive_only=False*)

Returns the covered examples.

property positives

precision()

rule_report(*show_uris=False*, *latex=False*)

Rule as string with some statistics.

static ruleset_examples_json(*rules_per_target*, *show_uris=False*)

static ruleset_report(*rules*, *show_uris=False*, *latex=False*, *human=<function Rule.<lambda>>*)

similarity(*rule*)

Calculates the similarity between this rule and 'rule'.

size()

Returns the number of conjunts.

static to_json(*rules_per_target*, *show_uris=False*)

Force Atlas 2 Visualization

class py3plex.visualization.fa2.forceatlas2.ForceAtlas2(*outboundAttractionDistribution=False*, *linLogMode=False*, *adjustSizes=False*, *edgeWeightInfluence=1.0*, *jitterTolerance=1.0*, *barnesHutOptimize=True*, *barnesHutTheta=1.2*, *multiThreaded=False*, *scalingRatio=2.0*, *strongGravityMode=False*, *gravity=1.0*, *verbose=True*)

Bases: object

forceatlas2(*G*, *pos=None*, *iterations=30*)

forceatlas2_igraph_layout(*G*, *pos=None*, *iterations=100*, *weight_attr=None*)

forceatlas2_networkx_layout(*G*, *pos=None*, *iterations=100*)

init(*G*, *pos=None*)

class py3plex.visualization.fa2.forceatlas2.Timer(*name='Timer'*)

Bases: object

display()

start()

stop()

class py3plex.visualization.fa2.fa2util.**Edge**

Bases: object

class py3plex.visualization.fa2.fa2util.**Node**

Bases: object

class py3plex.visualization.fa2.fa2util.**Region**(*nodes*)

Bases: object

applyForce(*n*, *theta*, *coefficient=0*)

applyForceOnNodes(*nodes*, *theta*, *coefficient=0*)

buildSubRegions()

updateMassAndGeometry()

py3plex.visualization.fa2.fa2util.**adjustSpeedAndApplyForces**(*nodes*, *speed*, *speedEfficiency*, *jitterTolerance*)

py3plex.visualization.fa2.fa2util.**apply_attraction**(*nodes*, *edges*, *distributedAttraction*, *coefficient*, *edgeWeightInfluence*)

py3plex.visualization.fa2.fa2util.**apply_gravity**(*nodes*, *gravity*, *useStrongGravity=False*)

py3plex.visualization.fa2.fa2util.**apply_repulsion**(*nodes*, *coefficient*)

py3plex.visualization.fa2.fa2util.**linAttraction**(*n1*, *n2*, *e*, *distributedAttraction*, *coefficient=0*)

py3plex.visualization.fa2.fa2util.**linGravity**(*n*, *g*)

py3plex.visualization.fa2.fa2util.**linRepulsion**(*n1*, *n2*, *coefficient=0*)

py3plex.visualization.fa2.fa2util.**linRepulsion_region**(*n*, *r*, *coefficient=0*)

py3plex.visualization.fa2.fa2util.**strongGravity**(*n*, *g*, *coefficient=0*)

Embedding Visualization

py3plex.visualization.embedding_visualization.embedding_visualization.**visualize_embedding**(*m*
la
be
ve
bo

Network Generation and Benchmarking

Additional Statistics and Analysis

`py3plex.algorithms.statistics.bayesiantests.correlated_ttest(x, rope, runs=1, verbose=False, names=('C1', 'C2'))`

`py3plex.algorithms.statistics.bayesiantests.correlated_ttest_MC(x, rope, runs=1, nsamples=50000)`

See correlated_ttest module for explanations

`py3plex.algorithms.statistics.bayesiantests.heaviside(X)`

Compute the Heaviside step function.

The Heaviside function returns 1 for positive values, 0.5 for zero, and 0 for negative values. This is used in signed-rank tests for Bayesian comparisons.

Parameters

X – Input array or scalar

Returns

Array with same shape as X containing:

- 1.0 where X > 0
- 0.5 where X == 0
- 0.0 where X < 0

Return type

np.ndarray

Examples

```
>>> heaviside(np.array([-1, 0, 1, 2]))
array([0. , 0.5, 1. , 1. ])
```

`py3plex.algorithms.statistics.bayesiantests.hierarchical(diff, rope, rho, upperAlpha=2, lowerAlpha=1, lowerBeta=0.01, upperBeta=0.1, std_upper_bound=1000, verbose=False, names=('C1', 'C2'))`

Perform hierarchical Bayesian test for comparing algorithms across multiple datasets.

This test accounts for the hierarchical structure of the data (multiple datasets, each with multiple folds) and correlations due to overlapping training sets.

Parameters

- **diff** – Array of differences between classifier scores
- **rope** – Width of the region of practical equivalence (ROPE)
- **rho** – Correlation between folds (typically around 1/n_folds)
- **upperAlpha** – Upper bound for alpha parameter of Gamma prior (default: 2)

- **lowerAlpha** – Lower bound for alpha parameter of Gamma prior (default: 1)
- **lowerBeta** – Lower bound for beta parameter of Gamma prior (default: 0.01)
- **upperBeta** – Upper bound for beta parameter of Gamma prior (default: 0.1)
- **std_upper_bound** – Upper bound multiplier for standard deviation prior (default: 1000) Posterior is insensitive to this if large enough (>100)
- **verbose** – Whether to print probability results (default: False)
- **names** – Tuple of classifier names for verbose output (default: ("C1", "C2"))

Returns

(p_left, p_rope, p_right)

- p_left: Probability that first classifier is worse
- p_rope: Probability that classifiers are practically equivalent
- p_right: Probability that first classifier is better

Return type

Tuple[float, float, float]

Notes

- The Gamma distribution parameters control the prior on degrees of freedom
- The hierarchical structure models between-dataset and within-dataset variance
- Use when comparing algorithms across multiple datasets with cross-validation

References

- Benavoli, A., Corani, G., & Mangili, F. (2016). Should we really use post-hoc tests based on mean-ranks? The Journal of Machine Learning Research.

See also

hierarchical_MC: Monte Carlo sampling version correlated_ttest: Simpler test for single dataset comparisons

`py3plex.algorithms.statistics.bayesiantests.`**`hierarchical_MC`**(*diff*, *rope*, *rho*, *upperAlpha=2*, *lowerAlpha=1*, *lowerBeta=0.01*, *upperBeta=0.1*, *std_upper_bound=1000*, *names=('C1', 'C2')*)

Monte Carlo sampling for hierarchical Bayesian test.

Generates Monte Carlo samples from the posterior distribution for hierarchical comparison of algorithms across multiple datasets with cross-validation.

Parameters

- **diff** – Array of differences between classifier scores (shape: n_datasets x n_folds)
- **rope** – Width of the region of practical equivalence (ROPE)
- **rho** – Correlation between folds due to overlapping training sets
- **upperAlpha** – Upper bound for alpha parameter of Gamma prior (default: 2)
- **lowerAlpha** – Lower bound for alpha parameter of Gamma prior (default: 1)
- **lowerBeta** – Lower bound for beta parameter of Gamma prior (default: 0.01)
- **upperBeta** – Upper bound for beta parameter of Gamma prior (default: 0.1)
- **std_upper_bound** – Upper bound multiplier for standard deviation prior (default: 1000)
- **names** – Tuple of classifier names for identification (default: (“C1”, “C2”))

Returns

Monte Carlo samples with shape (n_samples, 3)
Each row contains [p_left, p_rope, p_right] for one MC sample

Return type

np.ndarray

Notes

- Uses PyStan for Bayesian inference with hierarchical model
- Data is rescaled by mean standard deviation for numerical stability
- Hierarchical structure captures both within-dataset and between-dataset variance
- Requires pystan package to be installed

Implementation:

- Uses Stan’s NUTS sampler with 4 chains
- Each chain runs 100 iterations (including warmup)
- Total posterior samples: ~200 after warmup

See also

hierarchical: Main function that processes MC samples correlated_ttest_MC: Simpler MC version for single dataset

Raises

`ImportError` – If pystan is not installed `py3plex.algorithms.statistics.bayesiantests.plot_posterior(samples, names=('C1', 'C2'), proba_triplet=None)`

Parameters

- **x** (*array*) – a vector of differences or a 2d array with pairs of scores.
- **names** (*pair of str*) – the names of the two classifiers

Returns

matplotlib.pyplot.figure

`py3plex.algorithms.statistics.bayesiantests.plot_simplex(points, names=('C1', 'C2'), proba_triplet=None)`

`py3plex.algorithms.statistics.bayesiantests.signrank(x, rope, prior_strength=0.6, prior_place=1, nsamples=50000, verbose=False, names=('C1', 'C2'))`

Parameters

- **x** (*array*) – a vector of differences or a 2d array with pairs of scores.
- **rope** (*float*) – the width of the rope
- **prior_strength** (*float*) – prior strength (default: 0.6)
- **prior_place** (*LEFT, ROPE or RIGHT*) – the region to which the prior is assigned (default: ROPE)
- **nsamples** (*int*) – the number of Monte Carlo samples
- **verbose** (*bool*) – report the computed probabilities
- **names** (*pair of str*) – the names of the two classifiers

Returns

p_left, p_rope, p_right

`py3plex.algorithms.statistics.bayesiantests.signrank_MC(x, rope, prior_strength=0.6, prior_place=1, nsamples=50000)`

Parameters

- **x** (*array*) – a vector of differences or a 2d array with pairs of scores.
- **rope** (*float*) – the width of the rope
- **prior_strength** (*float*) – prior strength (default: 0.6)
- **prior_place** (*LEFT, ROPE or RIGHT*) – the region to which the prior is assigned (default: ROPE)
- **nsamples** (*int*) – the number of Monte Carlo samples

Returns

2-d array with rows corresponding to samples and columns to probabilities *[p_left, p_rope, p_right]* `py3plex.algorithms.statistics.bayesiantests.`**`signtest`**(*x*, *rope*, *prior_strength=1*, *prior_place=1*, *nsamples=50000*, *verbose=False*, *names=('C1', 'C2')*)

Parameters

- **x** (*array*) – a vector of differences or a 2d array with pairs of scores.
- **rope** (*float*) – the width of the rope
- **prior_strength** (*float*) – prior strength (default: 1)
- **prior_place** (*LEFT, ROPE or RIGHT*) – the region to which the prior is assigned (default: ROPE)
- **nsamples** (*int*) – the number of Monte Carlo samples
- **verbose** (*bool*) – report the computed probabilities
- **names** (*pair of str*) – the names of the two classifiers

Returns

p_left, p_rope, p_right

`py3plex.algorithms.statistics.bayesiantests.`**`signtest_MC`**(*x*, *rope*, *prior_strength=1*, *prior_place=1*, *nsamples=50000*)

Parameters

- **x** (*array*) – a vector of differences or a 2d array with pairs of scores.
- **rope** (*float*) – the width of the rope
- **prior_strength** (*float*) – prior strength (default: 1)
- **prior_place** (*LEFT, ROPE or RIGHT*) – the region to which the prior is assigned (default: ROPE)
- **nsamples** (*int*) – the number of Monte Carlo samples

Returns

2-d array with rows corresponding to samples and columns to probabilities *[p_left, p_rope, p_right]*

Community Detection Advanced

Node Ranking and Clustering (NoRC) module for community detection.

This module implements algorithms for node ranking and hierarchical clustering in networks, including parallel PageRank computation and hierarchical merging.

py3plex.algorithms.community_detection.NoRC.**NoRC_communities_main**(*input_graph*, *clustering_scheme='hierarchical'*, *max_com_num=100*, *verbose=False*, *parallel_step=None*, *prob_threshold=0.0005*, *community_range=None*, *fine_range=3*, *lag_threshold=10*)

py3plex.algorithms.community_detection.NoRC.**page_rank_kernel**(*index_row*)

Compute normalized PageRank vector for a given node.

Parameters

index_row – Node index for which to compute PageRank

Returns

Tuple of (node_index, normalized_pagerank_vector)

Community-based node ranking framework.

This module implements a framework for ranking nodes within and across communities using PageRank-based metrics and hierarchical clustering.

py3plex.algorithms.community_detection.community_ranking.**create_tree**(*centers: ndarray*) → Dict[int, Dict[str, List]]

py3plex.algorithms.community_detection.community_ranking.**page_rank_kernel**(*index_row: int*) → Tuple[int, ndarray]

py3plex.algorithms.community_detection.community_ranking.**return_infomap_communities**(*network: Any*) → List[List[

Node Ranking and Classification

Node ranking algorithms for multilayer networks.

This module provides various node ranking algorithms including PageRank variants, HITS (Hubs and Authorities), and personalized PageRank (PPR) for network analysis.

Key Functions:

- sparse_page_rank: Compute PageRank scores using sparse matrix operations
- run_PPR: Run Personalized PageRank in parallel across multiple cores

- hubs_and_authorities: Compute HITS scores for nodes
- stochastic_normalization: Normalize adjacency matrix to stochastic form

Notes

The PageRank implementations use sparse matrices for memory efficiency and support parallel computation for large-scale networks.

`py3plex.algorithms.node_ranking.authority_matrix(graph: Graph) → ndarray`

Get the authority matrix representation of a graph.

Computes the matrix A = A.T @ A where A is the adjacency matrix. The authority matrix is used in HITS algorithm computation.

Parameters
graph – NetworkX graph

Returns
Authority matrix (N x N) where N is number of nodes

Return type
np.ndarray

Notes

- For directed graphs: A[i,j] = number of nodes that point to both i and j
- Used internally by HITS algorithm to compute authority scores

See also

hub_matrix: Complementary hub matrix hubs_and_authorities: Compute actual hub/authority scores

`py3plex.algorithms.node_ranking.damping_hyper: float`

`py3plex.algorithms.node_ranking.hub_matrix(graph: Graph) → ndarray`

Get the hub matrix representation of a graph.

Computes the matrix H = A @ A.T where A is the adjacency matrix. The hub matrix is used in HITS algorithm computation.

Parameters
graph – NetworkX graph

Returns
Hub matrix (N x N) where N is number of nodes

Return type
np.ndarray

Notes

- For directed graphs: H[i,j] = number of nodes pointed to by both i and j
- Used internally by HITS algorithm to compute hub scores

See also

authority_matrix: Complementary authority matrix hubs_and_authorities: Compute actual hub/authority scores

`py3plex.algorithms.node_ranking.hubs_and_authorities(graph: Graph) → Tuple[dict, dict]`

Compute HITS (Hubs and Authorities) scores for all nodes in a graph.

Implements the Hyperlink-Induced Topic Search (HITS) algorithm to identify hub nodes (nodes that point to many authorities) and authority nodes (nodes pointed to by many hubs) in a network.

Parameters
: **graph** – NetworkX graph (directed or undirected)

Returns

(hub_scores, authority_scores)

- hub_scores: Dictionary mapping node -> hub score
- authority_scores: Dictionary mapping node -> authority score

Return type
: Tuple[dict, dict]

Notes

- Uses scipy-based implementation from NetworkX (nx.hits_scipy)
- Scores are normalized so that the sum of squares equals 1
- For undirected graphs, hub and authority scores are identical
- Converges using power iteration method

Examples

```
>>> import networkx as nx
>>> G = nx.DiGraph([(0, 1), (0, 2), (1, 2)])
>>> hubs, authorities = hubs_and_authorities(G)
>>> # Node 0 has high hub score (points to others)
>>> # Node 2 has high authority score (pointed to by others)
```

See also

hub_matrix: Get the hub matrix representation authority_matrix: Get the authority matrix representation

`py3plex.algorithms.node_ranking.page_rank_kernel(index_row: int) → Tuple[int, ndarray]`

Compute PageRank vector for a single starting node (multiprocessing kernel).

This function is designed to be called in parallel via multiprocessing.Pool.map(). It computes the personalized PageRank vector starting from a single node.

Parameters
`index_row` – Index of the starting node for personalized PageRank

Returns

(node_index, normalized_pagerank_vector)

- node_index: The input index (for tracking results)
- pagerank_vector: L2-normalized PageRank scores for all nodes

Return type
Tuple[int, np.ndarray]

Notes

- Accesses global variables: ___graph_matrix, damping_hyper, spread_step_hyper, spread_percent_hyper (set by run_PPR before parallel execution)
- Returns zero vector if normalization fails
- L2 normalization ensures comparable magnitudes across different starting nodes

See also

run_PPR: Main function that sets up parallel execution sparse_page_rank: Core PageRank computation

`py3plex.algorithms.node_ranking.run_PPR`(*network: spmatrix*, *cores: int | None = None*, *jobs: List[range] | None = None*, *damping: float = 0.85*, *spread_step: int = 10*, *spread_percent: float = 0.3*, *targets: List[int] | None = None*, *parallel: bool = True*) → Generator[Tuple[int, ndarray] | List[Tuple[int, ndarray]], None, None]

Run Personalized PageRank (PPR) in parallel for multiple starting nodes.

Computes personalized PageRank vectors for multiple nodes using parallel processing. This is useful for creating node embeddings or analyzing node importance from different perspectives in the network.

Parameters

- `network` – Sparse adjacency matrix (will be automatically normalized to stochastic form)
- `cores` – Number of CPU cores to use (default: all available cores)
- `jobs` – Custom job batches as list of ranges (default: auto-generated)
- `damping` – Damping factor for PageRank (default: 0.85) Higher values (0.85-0.99) emphasize network structure
- `spread_step` – Steps to check spread pattern for optimization (default: 10)
- `spread_percent` – Max node fraction for shrinkage optimization (default: 0.3)
- `targets` – Specific node indices to compute PPR for (default: all nodes)

- **parallel** – Enable parallel processing (default: True) Set to False for debugging or single-core execution

Yields

Union[Tuple[int, np.ndarray], List[Tuple[int, np.ndarray]]] – - If parallel=True: Lists of (node_index, pagerank_vector) tuples (batched) - If parallel=False: Individual (node_index, pagerank_vector) tuples

Notes

- Automatically normalizes input matrix to column-stochastic form
- Uses multiprocessing.Pool for parallel execution
- Global variables are used to share the graph matrix across processes
- Results are yielded incrementally (generator pattern) to save memory
- Each pagerank_vector is L2-normalized for comparability

Examples

```
>>> import scipy.sparse as sp
>>> # Create a small network
>>> adj = sp.csr_matrix([[0, 1, 1], [1, 0, 1], [1, 1, 0]])
>>>
>>> # Compute PPR for all nodes in parallel
>>> for batch in run_PPR(adj, cores=2, parallel=True):
...     for node_idx, pr_vector in batch:
...         print(f"Node {node_idx}: {pr_vector}")
>>>
>>> # Compute PPR for specific nodes without parallelism
>>> for node_idx, pr_vector in run_PPR(adj, targets=[0, 1],␣
↪parallel=False):
...     print(f"Node {node_idx}: {pr_vector}")
```

Performance:

- Parallel speedup scales with number of cores (up to ~0.8 * cores efficiency)
- Memory usage: O(N * N_targets) for storing results
- For large networks (>100K nodes), consider processing targets in batches

See also

sparse_page_rank: Core PageRank computation page_rank_kernel: Worker function for parallel execution stochastic_normalization: Matrix normalization (called internally) `py3plex.algorithms.node_ranking.sparse_page_rank(matrix: spmatrix, start_nodes: List[int] | range, epsilon: float = 1e-06, max_steps: int = 100000, damping: float = 0.5, spread_step: int = 10, spread_percent: float = 0.3, try_shrink: bool = True) → ndarray`

Compute personalized PageRank using sparse matrix operations.

Implements an efficient personalized PageRank algorithm with adaptive sparsification to reduce memory usage and computation time. The algorithm uses a power iteration method with early stopping based on convergence criteria.

Parameters

- **matrix** – Column-stochastic sparse adjacency matrix (use stochastic_normalization first)
- **start_nodes** – List or range of starting nodes for personalized PageRank
- **epsilon** – Convergence threshold for L1 norm difference (default: 1e-6)
- **max_steps** – Maximum number of iterations (default: 100000)
- **damping** – Damping factor / teleportation probability (default: 0.5) Higher values (e.g., 0.85) favor network structure over random jumps
- **spread_step** – Number of steps to check for sparsity pattern (default: 10)
- **spread_percent** – Maximum fraction of nodes to consider for shrinkage (default: 0.3)
- **try_shrink** – Enable adaptive shrinkage to reduce computation (default: True)

Returns

PageRank scores for all nodes, with start_nodes set to 0

Return type

np.ndarray

Notes

- Assumes matrix is column-stochastic (use stochastic_normalization first)
- Adaptive shrinkage identifies nodes unreachable from start_nodes and excludes them from computation for efficiency
- Convergence is measured by L1 norm of rank vector difference
- Start nodes are zeroed out in the final result to avoid self-importance

Complexity:

- Time: O(k * E) where k is iterations and E is edges
- Space: O(N) for rank vectors, plus matrix storage

Examples

```
>>> import scipy.sparse as sp
>>> # Create and normalize adjacency matrix
>>> adj = sp.csr_matrix([[0, 1, 1], [1, 0, 1], [1, 1, 0]])
>>> adj_norm = stochastic_normalization(adj)
>>> # Compute PageRank from node 0
>>> pr = sparse_page_rank(adj_norm, [0], damping=0.85)
>>> pr  # PageRank scores (node 0 will be 0)
array([0.  , 0.5, 0.5])
```

Raises
`AssertionError` – If start_nodes is empty

See also

stochastic_normalization: Required preprocessing step run_PPR: Parallel wrapper for computing multiple PageRank vectors

`py3plex.algorithms.node_ranking.`**`spread_percent_hyper:`** **`float`**

`py3plex.algorithms.node_ranking.`**`spread_step_hyper:`** **`int`**

`py3plex.algorithms.node_ranking.`**`stochastic_normalization`**(*matrix: spmatrix*) → spmatrix

Normalize a sparse matrix to stochastic form (column-stochastic).

Converts an adjacency matrix to a stochastic matrix where each column sums to 1. This normalization is required for PageRank-style random walk algorithms.

Parameters
`matrix` – Sparse adjacency matrix to normalize

Returns
Column-stochastic sparse matrix where each column sums to 1

Return type
sp.spmatrix

Notes

- Removes self-loops (sets diagonal to 0) before normalization
- Handles zero-degree nodes by leaving corresponding columns as zeros
- Preserves sparsity structure for memory efficiency

Examples

```
>>> import scipy.sparse as sp
>>> adj = sp.csr_matrix([[0, 1, 1], [1, 0, 1], [1, 1, 0]])
>>> stoch_adj = stochastic_normalization(adj)
>>> stoch_adj.sum(axis=1).A1  # Column sums should be 1
array([1., 1., 1.])
```

`py3plex.algorithms.network_classification.`**`benchmark_classification`**(*matrix: spmatrix*, *labels: ndarray*, *alpha_value: float = 0.85*, *iterations: int = 30*, *normalization_scheme: str = 'freq'*, *dataset_name: str = 'example'*, *verbose: bool = False*, *test_size: float | None = None*) → DataFrame

`py3plex.algorithms.network_classification.`**`label_propagation`**(*graph_matrix: spmatrix*, *class_matrix: ndarray*, *alpha: float = 0.001*, *epsilon: float = 1e-12*, *max_steps: int = 100000*, *normalization: str | List[str] = 'freq'*) → ndarray

Propagate labels through a graph.

Parameters

- **`graph_matrix`** – Sparse graph adjacency matrix
- **`class_matrix`** – Initial class label matrix
- **`alpha`** – Propagation weight parameter
- **`epsilon`** – Convergence threshold
- **`max_steps`** – Maximum number of iterations
- **`normalization`** – Normalization scheme(s) to apply

Returns

Propagated label matrix

`py3plex.algorithms.network_classification.`**`label_propagation_normalization`**(*matrix: spmatrix*) → spmatrix

Normalize a matrix for label propagation.

Parameters
matrix – Sparse matrix to normalize

Returns
Normalized sparse matrix

`py3plex.algorithms.network_classification.`**`label_propagation_tf()`** → None

`py3plex.algorithms.network_classification.`**`normalize_amplify_freq`**(*mat: ndarray*) → ndarray

Normalize and amplify matrix by frequency.

Parameters
mat – Matrix to normalize

Returns
Normalized and amplified matrix

`py3plex.algorithms.network_classification.`**`normalize_exp`**(*mat: ndarray*) → ndarray

Apply exponential normalization.

Parameters
mat – Matrix to normalize

Returns
Exponentially normalized matrix

`py3plex.algorithms.network_classification.`**`normalize_initial_matrix_freq`**(*mat: ndarray*) → ndarray

Normalize matrix by frequency.

Parameters
mat – Matrix to normalize

Returns
Normalized matrix

`py3plex.algorithms.network_classification.validate_label_propagation`(*core_network: spmatrix, labels: ndarray | spmatrix, dataset_name: str = 'test', repetitions: int = 5, normalization_scheme: str | List[str] = 'basic', alpha_value: float = 0.001, random_seed: int = 123, verbose: bool = False*) → DataFrame

Validate label propagation with cross-validation.

Parameters

- **core_network** – Sparse network adjacency matrix
- **labels** – Label matrix
- **dataset_name** – Name of the dataset
- **repetitions** – Number of repetitions
- **normalization_scheme** – Normalization scheme to use
- **alpha_value** – Alpha parameter for propagation
- **random_seed** – Random seed for reproducibility
- **verbose** – Whether to print progress

Returns

DataFrame with validation results

Embeddings and Wrappers

py3plex.wrappers.train_node2vec_embedding.**call_node2vec_binary**(*input_graph: str*, *output_graph: str*, *p: float = 1*, *q: float = 1*, *dimension: int = 128*, *directed: bool = False*, *weighted: bool = True*, *binary: str | None = None*, *timeout: int = 300*) → None

Call the Node2Vec C++ binary with specified parameters.

Parameters

- **input_graph** – Path to input graph file
- **output_graph** – Path to output embedding file
- **p** – Return parameter
- **q** – In-out parameter
- **dimension** – Embedding dimension
- **directed** – Whether graph is directed
- **weighted** – Whether graph is weighted
- **binary** – Path to node2vec binary (defaults to PY3PLEX_NODE2VEC_BINARY env var or "./node2vec")
- **timeout** – Maximum execution time in seconds

Raises

ExternalToolError – If binary is not found or execution fails

py3plex.wrappers.train_node2vec_embedding.**learn_embedding**(*core_network: Any*, *labels: List[Any] | None = None*, *ssize: float = 0.5*, *embedding_outfile: str = 'out.emb'*, *p: float = 0.1*, *q: float = 0.1*, *binary_path: str | None = None*, *parameter_range: str = '[0.25,0.50,1,2,4]'*, *timeout: int = 300*) → Tuple[str, float]

Learn node embeddings for a network.

Parameters

- **core_network** – NetworkX graph
- **labels** – Node labels
- **ssize** – Sample size
- **embedding_outfile** – Output file for embeddings

- **p** – Return parameter
- **q** – In-out parameter
- **binary_path** – Path to node2vec binary (defaults to PY3PLEX_NODE2VEC_BINARY env var or "./node2vec")
- **parameter_range** – String representation of parameter range list
- **timeout** – Maximum execution time in seconds per call

Returns
Tuple of (method_name, elapsed_time)

`py3plex.wrappers.train_node2vec_embedding.n2v_embedding`(*G: Any*, *targets: Any*, *verbose: bool = False*, *sample_size: float = 0.5*, *outfile_name: str = 'test.emb'*, *p: float | None = None*, *q: float | None = None*, *binary_path: str | None = None*, *parameter_range: List[float] | None = None*, *embedding_dimension: int = 128*, *timeout: int = 300*) → None

Train Node2Vec embeddings with parameter optimization.

Parameters

- **G** – NetworkX graph
- **targets** – Target labels for nodes
- **verbose** – Whether to print verbose output
- **sample_size** – Sample size for training
- **outfile_name** – Output embedding file name
- **p** – Return parameter (None triggers grid search)
- **q** – In-out parameter (None triggers grid search)
- **binary_path** – Path to node2vec binary (defaults to PY3PLEX_NODE2VEC_BINARY env var or "./node2vec")
- **parameter_range** – Range of parameters to search
- **embedding_dimension** – Dimension of embeddings
- **timeout** – Maximum execution time in seconds per call

Network Motifs and Patterns

HINMINE Data Structures

class `py3plex.core.HINMINE.dataStructures.Class`(*lab_id*, *name*, *members*)

Bases: `object`

class `py3plex.core.HINMINE.dataStructures.`**`HeterogeneousInformationNetwork`**(*network*, *label_delimiter*, *weight_tag=False*, *target_tag=True*)

Bases: `object`

`add_label`(*node*, *label_id*, *label_name=None*)

`calculate_decomposition_candidates`(*max_decomposition_length=10*)

`calculate_schema`()

`create_label_matrix`(*weights=None*)

`decompose_from_iterator`(*name*, *weighing*, *summing*, *generator=None*, *degrees=None*, *parallel=False*, *pool=None*)

`midpoint_generator`(*node_sequence*, *edge_sequence*)

`process_network`(*label_delimiter*)

`split_to_indices`(*train_indices=()*, *validate_indices=()*, *test_indices=()*)

`split_to_parts`(*lst*, *n*)

Command-Line Interface

Command-line interface for py3plex.

This module provides a comprehensive CLI tool for multilayer network analysis with full coverage of main algorithms.

`py3plex.cli.`**`cmd_aggregate`**(*args: Namespace*) → int

Aggregate multilayer network into single layer.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.`**`cmd_centrality`**(*args: Namespace*) → int

Compute node centrality measures.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success)

`py3plex.cli.`**`cmd_check`**(*args: Namespace*) → int

Lint and validate a graph data file.

Parameters

args – Parsed command-line arguments

Returns

Exit code (0 for success, non-zero for errors) `py3plex.cli.cmd_community(args: Namespace) → int`

Detect communities in the network.

Parameters
args – Parsed command-line arguments

Returns
Exit code (0 for success)

`py3plex.cli.cmd_convert(args: Namespace) → int`

Convert network between different formats.

Parameters
args – Parsed command-line arguments

Returns
Exit code (0 for success)

`py3plex.cli.cmd_create(args: Namespace) → int`

Create a new multilayer network.

Parameters
args – Parsed command-line arguments

Returns
Exit code (0 for success)

`py3plex.cli.cmd_dsl_lint(args: Namespace) → int`

Lint and analyze DSL queries.

Parameters
args – Parsed command-line arguments

Returns
Exit code (0 for no errors, 1 for errors found, 2 for command error)

`py3plex.cli.cmd_help(args: Namespace) → int`

Show detailed help information.

Parameters
args – Parsed command-line arguments

Returns
Exit code (0 for success)

`py3plex.cli.cmd_load(args: Namespace) → int`

Load and inspect a network.

Parameters
args – Parsed command-line arguments

Returns
Exit code (0 for success)

`py3plex.cli.cmd_query(args: Namespace) → int`

Execute DSL queries on networks with Unix piping support.

Parameters
args – Parsed command-line arguments

Returns
Exit code (0 for success)

py3plex.cli.**cmd_quickstart**(*args: Namespace*) → int

Run quickstart demo - creates a tiny demo graph and demonstrates basic operations.

Parameters
args – Parsed command-line arguments

Returns
Exit code (0 for success)

py3plex.cli.**cmd_run_config**(*args: Namespace*) → int

Run workflow from configuration file.

Parameters
args – Parsed command-line arguments

Returns
Exit code (0 for success)

py3plex.cli.**cmd_selftest**(*args: Namespace*) → int

Run self-test to verify installation and core functionality.

Parameters
args – Parsed command-line arguments

Returns
Exit code (0 for success)

py3plex.cli.**cmd_stats**(*args: Namespace*) → int

Compute multilayer network statistics.

Parameters
args – Parsed command-line arguments

Returns
Exit code (0 for success)

py3plex.cli.**cmd_tutorial**(*args: Namespace*) → int

Run interactive tutorial mode to learn py3plex step by step.

Parameters
args – Parsed command-line arguments

Returns
Exit code (0 for success)

py3plex.cli.**cmd_visualize**(*args: Namespace*) → int

Visualize the multilayer network.

Parameters
args – Parsed command-line arguments

Returns
Exit code (0 for success)

py3plex.cli.**create_parser**() → ArgumentParser

Create and configure the argument parser for py3plex CLI.

Returns
Configured ArgumentParser instance

py3plex.cli.**main**(*argv: List[str] | None = None*) → int

Main entry point for the CLI.

Parameters
`argv` – Command-line arguments (defaults to sys.argv)

Returns
Exit code (0 for success, non-zero for errors)

Validation Utilities

Input validation utilities for Py3plex.

This module provides pre-validation functions to catch common input errors early and provide clear, actionable error messages to users.

`py3plex.validation.validate_csv_columns`(*file_path: str*, *required_columns: List[str]*, *optional_columns: List[str] | None = None*) → None

Validate that a CSV file has required columns.

Parameters

- `file_path` – Path to CSV file
- `required_columns` – List of column names that must be present
- `optional_columns` – List of column names that are optional

Raises
ParsingError – If required columns are missing

Contracts:

- Precondition: file_path must be a non-empty string
- Precondition: required_columns must be a non-empty list of strings

`py3plex.validation.validate_edgelist_format`(*file_path: str*, *delimiter: str | None = None*) → None

Validate simple edgelist file format (source target weight).

Parameters

- `file_path` – Path to edgelist file
- `delimiter` – Optional delimiter (default: whitespace)

Raises
ParsingError – If file format is invalid

Contracts:

- Precondition: file_path must be a non-empty string
- Precondition: delimiter must be None or a string

`py3plex.validation.validate_file_exists`(*file_path: str*) → None

Validate that a file exists and is readable.

Parameters
`file_path` – Path to file to validate

Raises
ParsingError – If file doesn't exist or isn't readable

Contracts:

- Precondition: file_path must be a non-empty string

`py3plex.validation.validate_input_type`(*input_type: str*, *valid_types: Set[str] | None = None*) → None

Validate that input_type is recognized.

Parameters

- **input_type** – The input type string to validate
- **valid_types** – Optional set of valid types (uses default if None)

Raises

ParsingError – If input_type is not valid

Contracts:

- Precondition: input_type must be a non-empty string
- Precondition: valid_types must be None or a set

`py3plex.validation.validate_multiedgelist_format`(*file_path: str*, *delimiter: str | None = None*) → None

Validate multiedgelist file format (source target layer weight).

Parameters

- **file_path** – Path to multiedgelist file
- **delimiter** – Optional delimiter (default: whitespace)

Raises

ParsingError – If file format is invalid

Contracts:

- Precondition: file_path must be a non-empty string
- Precondition: delimiter must be None or a string

`py3plex.validation.validate_network_data`(*file_path: str*, *input_type: str*) → None

Validate network data before parsing.

This is the main entry point for validation. It performs appropriate validation based on the input type.

Parameters

- **file_path** – Path to network file
- **input_type** – Type of input file

Raises

ParsingError – If validation fails

Contracts:

- Precondition: file_path must be a string
- Precondition: input_type must be a non-empty string

Network Comparison and Testing

Hedwig Learning Algorithms

Main learner class.

@author: anze.vavpetic@ijs.si

class py3plex.algorithms.hedwig.learners.learner.Learner(*kb*, *n=None*, *min_sup=1*, *sim=1*, *depth=4*, *target=None*, *use_negations=False*, *optimal_subclass=False*)

Bases: `object`

Learner class, supporting various types of induction from the knowledge base.

Default = 'default'

Improvement = 'improvement'

Similarity = 'similarity'

can_specialize(*rule*)

Is the rule good enough to be further refined?

extend(*rules*, *specializations*)

Extends the ruleset in the given way.

extend_replace_worst(*rules*, *specializations*)

Extends the list by replacing the worst rules.

extend_with_similarity(*rules*, *specializations*)

Extends the list based on how similar is 'new_rule' to the rules contained in 'rules'.

get_subclasses(*pred*)

get_superclasses(*pred*)

group_score(*rules*)

Calculates the score of the whole list of rules.

induce()

Induces rules for the given knowledge base.

is_implicit_root(*pred*)

non_redundant(*rule*, *new_rule*)

Is the rule non-redundant compared to its immediate generalization?

specialize(*rule*)

Returns a list of all specializations of 'rule'.

specialize_add_relation(*rule*)

Specialize with new binary relation.

Main learner class.

@author: anze.vavpetic@ijs.si

class py3plex.algorithms.hedwig.learners.bottomup.**BottomUpLearner**(*kb*, *n=None*, *min_sup=1*, *sim=1*, *depth=4*, *target=None*, *use_negations=False*)

Bases: object

Bottom-up learner.

Default = 'default'

Improvement = 'improvement'

Similarity = 'similarity'

bottom_clause()

get_subclasses(*pred*)

get_superclasses(*pred*)

induce()

Induces rules for the given knowledge base.

is_implicit_root(*pred*)

Main learner class.

@author: anze.vavpetic@ijs.si

class py3plex.algorithms.hedwig.learners.optimal.**OptimalLearner**(*kb*, *n=None*, *min_sup=1*, *sim=1*, *depth=4*, *target=None*, *use_negations=False*, *optimal_subclass=True*)

Bases: *Learner*

Finds the optimal top-k rules.

induce()

Induces rules for the given knowledge base.

Hedwig Statistics and Scoring

Score function definitions.

@author: anze.vavpetic@ijs.si

py3plex.algorithms.hedwig.stats.scorefunctions.**chisq**(*rule*)

py3plex.algorithms.hedwig.stats.scorefunctions.**enrichment_score**(*rule*)

py3plex.algorithms.hedwig.stats.scorefunctions.**interesting**(*rule*)

Checks if a given rule is interesting for the given score function

py3plex.algorithms.hedwig.stats.scorefunctions.**kaplan_meier_AUC**(*rule*) py3plex.algorithms.hedwig.stats.scorefunctions.**leverage**(*rule*)

py3plex.algorithms.hedwig.stats.scorefunctions.**lift**(*rule*)

py3plex.algorithms.hedwig.stats.scorefunctions.**precision**(*rule*)

py3plex.algorithms.hedwig.stats.scorefunctions.**t_score**(*rule*)

py3plex.algorithms.hedwig.stats.scorefunctions.**wracc**(*rule*)

py3plex.algorithms.hedwig.stats.scorefunctions.**z_score**(*rule*)

Module for ruleset validation.

@author: anze.vavpetic@ijs.si

class py3plex.algorithms.hedwig.stats.validate.**Validate**(*kb*, *significance_test=<function apply_fisher>*, *adjustment=<function fdr>*)

Bases: object

test(*ruleset*, *alpha=0.05*, *q=0.01*)

Tests the given ruleset and returns the significant rules.

Hedwig Core Components

py3plex.algorithms.hedwig.core.converters.**convert_mapping_to_rdf**(*input_mapping_file*, *extract_subnode_info=False*, *split_node_by=':'*, *keep_index=1*, *layer_type='uniprotkb'*, *annotation_mapping_file='test.gaf'*, *go_identifier='GO:'*, *prepend_string=None*)

py3plex.algorithms.hedwig.core.converters.**obo2n3**(*obofile*, *n3out*, *gaf_file*)

Knowledge-base class.

@author: anze.vavpetic@ijs.si

class py3plex.algorithms.hedwig.core.kb.**ExperimentKB**(*triplets*, *score_fun*, *instances_as_leaves=True*)

Bases: object

The knowledge base for one specific experiment.

add_sub_class(*sub*, *obj*)

Adds the resource 'sub' as a subclass of 'obj'.

bits_to_indices(*bits*)

Converts the bitset to a set of indices.

get_domains(*predicate*)

Returns the bitsets for input and outputexamples of the binary predicate 'predicate'.

get_empty_domain()
Returns a bitset covering no examples.

get_examples()
Returns all examples for this experiment.

get_full_domain()
Returns a bitset covering all examples.

get_members(*predicate*, *bit=True*)
Returns the examples for this predicate, either as a bitset or a set of ids.

get_reverse_members(*predicate*, *bit=True*)
Returns the examples for this predicate, either as a bitset or a set of ids.

get_root()
Root predicate, which covers all examples.

get_score(*ex_idx*)
Returns the score for example id 'ex_idx'.

get_subclasses(*predicate*, *producer_pred=None*)
Returns a list of subclasses (as predicate objects) for 'predicate'.

indices_to_bits(*indices*)
Converts the indices to a bitset.

is_discrete_target()

n_examples()
Returns the number of examples.

n_members(*predicate*)

super_classes(*pred*)
Returns all super classes of pred (with transitivity).

Global settings file.

@author: anze.vavpetic@ijs.si

class py3plex.algorithms.hedwig.core.settings.**Defaults**
Bases: object

ADJUST = 'fwer'

ALPHA = 0.05

BEAM_SIZE = 20

COVERED = None

DEPTH = 5

FDR_Q = 0.05

FORMAT = 'n3'

LEARNER = 'heuristic'

LEAVES = False

`MODE = 'subgroups'`

`NEGATIONS = False`

`NO_CACHE = False`

`OPTIMAL_SUBCLASS = False`

`OUTPUT = None`

`SCORE = 'lift'`

`SUPPORT = 0.1`

`TARGET = None`

`URIS = False`

`VERBOSE = False`

Time Series and Temporal Analysis

Advanced Visualization

Sankey diagram visualization for multilayer networks.

This module provides inter-layer flow visualization to show connection strength between layers in multilayer networks. The visualization displays flows as arrows with widths proportional to the number of inter-layer connections.

Note: This uses a simplified flow diagram approach rather than matplotlib's Sankey class, as the Sankey class is designed for more complex flow networks and doesn't map directly to multilayer network inter-layer connections.

`py3plex.visualization.sankey.`**`draw_multilayer_sankey`**`(`*graphs: List[Graph]*, *multilinks: Dict[str, List[Tuple]]*, *labels: List[str] | None = None*, *ax: Any | None = None*, *display: bool = False*, ***kwargs*`)` → Any

Draw inter-layer flow diagram showing connection strength in multilayer networks.

Creates a flow visualization where: - Each layer is represented in the diagram - Flows between layers show the strength (number) of inter-layer connections - Flow width/text indicates the number of inter-layer edges

Parameters

- **graphs** – List of NetworkX graphs, one per layer
- **multilinks** – Dictionary mapping edge_type -> list of multi-layer edges
- **labels** – Optional list of layer labels. If None, uses layer indices
- **ax** – Matplotlib axes to draw on. If None, creates new figure
- **display** – If True, calls plt.show() after drawing. Default is False to let the caller control rendering.
- ****kwargs** – Reserved for future extensions

Returns

Matplotlib axes object

Examples

```
>>> import matplotlib.pyplot as plt
>>> from py3plex.visualization import draw_multilayer_sankey
>>> network = multi_layer_network()
>>> network.load_network("data.txt", input_type="multiedgelist")
>>> labels, graphs, multilinks = network.get_layers()
>>> fig, ax = plt.subplots(figsize=(12, 8))
>>> ax = draw_multilayer_sankey(graphs, multilinks, labels=labels, ax=ax)
>>> plt.savefig("sankey.png")
```

Note

This visualization is most effective for networks with 2-5 layers. For networks with many layers, the diagram may become cluttered. The implementation uses a simplified flow visualization approach.

Link Prediction

3.5.7 Configuration & Environment

This page documents configuration options, environment variables, and file formats for py3plex.

Configuration Files

py3plex supports configuration files for reproducible workflows. Configuration can be provided in YAML or JSON format.

Network Configuration (YAML):

```yaml
# network_config.yaml
network:
  input_file: "data/multilayer.edges"
  input_type: "multiedgelist"
  directed: false
  weighted: true

# Load with:
# network.load_network(**config['network'])
```

Algorithm Parameters:

```yaml
# algorithm_config.yaml
community_detection:
  algorithm: "louvain_multilayer"
  params:
    gamma: 1.0
    omega: 0.5
    random_state: 42

node2vec:
  dimensions: 128
```

(continues on next page)

(continued from previous page)

```
  walk_length: 80
  num_walks: 10
  p: 1.0
  q: 1.0

dynamics:
  model: "SIR"
  beta: 0.3
  gamma: 0.1
  initial_infected: 0.05
  steps: 100
```

Visualization Settings:

```
# viz_config.yaml
visualization:
  layout: "force_directed"
  node_size: 50
  edge_width: 2
  dpi: 300
  output_format: "png"
  color_by_layer: true
```

Loading configuration:

```
import yaml
from py3plex.core import multinet

# Load config
with open('config.yaml', 'r') as f:
    config = yaml.safe_load(f)

# Use config
network = multinet.multi_layer_network(
    directed=config['network']['directed']
)
network.load_network(
    config['network']['input_file'],
    input_type=config['network']['input_type']
)
```

Environment Variables

py3plex respects the following environment variables for customization:

Data Directories:

- `PY3PLEX_DATA_DIR` — Override default data directory location
  - Default: `~/.py3plex/data`
  - Used for: Cached datasets, downloaded networks
- `PY3PLEX_CACHE_DIR` — Cache directory for computed results

- Default: `~/.py3plex/cache`
- Used for: Embedding caches, computation results

Performance Tuning:

- `PY3PLEX_NUM_WORKERS` — Number of parallel workers for algorithms
  - Default: `os.cpu_count()`
  - Used by: Node2Vec, community detection, some centrality computations
- `PY3PLEX_MEMORY_LIMIT` — Memory limit for large-scale operations (in GB)
  - Default: System memory / 2
  - Used by: Tensor operations, large network processing

Setting environment variables:

```
# Linux/macOS
export PY3PLEX_DATA_DIR="/path/to/data"
export PY3PLEX_NUM_WORKERS=8

# Or in Python
import os
os.environ['PY3PLEX_DATA_DIR'] = '/path/to/data'
```

Debugging and Development:

- `PY3PLEX_DEBUG` — Enable debug mode (verbose output)
  - Values: `0` (off), `1` (on)
  - Default: `0`
- `PY3PLEX_PROFILE` — Enable profiling for performance analysis
  - Values: `0` (off), `1` (on)
  - Default: `0`

Input Formats

Supported file formats:

- `multiedgelist` — Multilayer edge list format
- `edgelist` — Standard edge list
- `json` — JSON network format
- `graphml` — GraphML format
- `parquet` — Apache Parquet (high performance)

See *How to Load and Build Networks* for format details.

Output Formats

Networks can be exported to:

- Pickle (Python serialization)
- JSON
- CSV/TSV

- GraphML
- Parquet

See *How to Export and Serialize Networks* for export details.

Algorithm Configuration

Many algorithms accept configuration parameters:

Community Detection

```
from py3plex.algorithms.community_detection import multilayer_louvain

communities = multilayer_louvain(
    network,
    resolution=1.0,  # Resolution parameter
    omega=0.5        # Inter-layer coupling
)
```

Node2Vec

```
from py3plex.wrappers import train_node2vec

embeddings = train_node2vec(
    network,
    dimensions=128,      # Embedding size
    walk_length=80,      # Walk length
    num_walks=10,        # Walks per node
    p=1.0,              # Return parameter
    q=1.0,              # In-out parameter
    workers=4           # Parallel workers
)
```

Dynamics

```
from py3plex.dynamics import SIRDynamics

sir = SIRDynamics(
    network,
    beta=0.3,           # Infection rate
    gamma=0.1,          # Recovery rate
    initial_infected=5  # Initial infected count
)
```

See *Algorithm Roadmap* for complete parameter documentation.

Visualization Configuration

Customize visualization:

```
network.visualize_network(
    output_file='network.png',
    layout_algorithm='force_directed',
    node_size=50,
    edge_width=2,
    dpi=300,
    show=False
)
```

See *How to Visualize Multilayer Networks* for visualization options.

Logging Configuration

py3plex uses Python's standard `logging` module. Configure logging for debugging and monitoring:

Basic Logging Setup:

```
import logging

# Set log level
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)

# Now py3plex operations will log to console
from py3plex.core import multinet
network = multinet.multi_layer_network()
# ... logs will appear
```

Log Levels:

- `DEBUG` — Detailed information, typically of interest only when diagnosing problems
- `INFO` — Confirmation that things are working as expected (default)
- `WARNING` — An indication that something unexpected happened
- `ERROR` — A more serious problem, the software has not been able to perform some function
- `CRITICAL` — A serious error, the program may be unable to continue running

Log to File:

```
import logging

# Configure file handler
logging.basicConfig(
    level=logging.DEBUG,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
```

(continues on next page)

(continued from previous page)

```
        logging.FileHandler('py3plex.log'),
        logging.StreamHandler()  # Also log to console
    ]
)
```

Log File Location:

By default, logs are written to:

- Console (stdout) when using `StreamHandler`
- Custom file when using `FileHandler`
- Environment variable `PY3PLEX_LOG_FILE` can override default location

Component-Specific Logging:

```
import logging

# Enable debug logging only for specific components
logging.getLogger('py3plex.algorithms').setLevel(logging.DEBUG)
logging.getLogger('py3plex.dsl').setLevel(logging.INFO)
logging.getLogger('py3plex.dynamics').setLevel(logging.WARNING)
```

Custom Logging Handlers:

For advanced use cases (e.g., sending logs to external systems):

```
import logging
from logging.handlers import RotatingFileHandler

# Rotating file handler (10MB max, keep 5 backups)
handler = RotatingFileHandler(
    'py3plex.log',
    maxBytes=10*1024*1024,
    backupCount=5
)
handler.setLevel(logging.DEBUG)
handler.setFormatter(
    logging.Formatter('%(asctime)s - %(name)s - %(levelname)s - %(message)s')
)

# Add to py3plex logger
logger = logging.getLogger('py3plex')
logger.addHandler(handler)
logger.setLevel(logging.DEBUG)
```

Performance Tuning

See *Benchmarking & Performance* for performance optimization strategies.

Next Steps

- **Load networks:** *How to Load and Build Networks*
- **Export networks:** *How to Export and Serialize Networks*
- **Algorithm reference:** *Algorithm Roadmap*

3.6 Part VI: Examples & Recipes

Complete working examples and analysis recipes.

3.6.1 Examples & Recipes

Complete working examples demonstrating py3plex capabilities. All examples are available in the examples/ directory[9] of the repository.

What's here:

- **Runnable Examples** — Copy-paste code for common tasks
- **Analysis Recipes** — Reusable patterns (*Analysis Recipes & Workflows*)
- **Case Studies** — In-depth applications (*Use Cases & Case Studies*)

Quick Start:

- **DSL examples:** `example_dsl_builder_api.py` — Best starting point
- **Visualization:** `example_multilayer_visualization.py`
- **Community detection:** `example_community_detection.py`
- **10-minute tutorial:** `tutorial_10min.py`

Featured: DSL Examples

The DSL examples showcase py3plex's SQL-like query language:

```
from py3plex.dsl import Q, L

# From example_dsl_builder_api.py - Comprehensive DSL v2
result = (
    Q.nodes()
      .from_layers(L["social"] + L["work"])
      .where(degree__gt=5)
      .compute("betweenness_centrality", "pagerank")
      .order_by("-betweenness_centrality")
      .limit(20)
      .export_csv("top_influencers.csv")
      .execute(network)
)
```

[9] https://github.com/SkBlaz/py3plex/tree/master/examples

Recommended starting points:

- `example_dsl_builder_api.py` — Complete DSL v2 tutorial
- `example_dsl_queries.py` — String DSL syntax
- `example_dsl_advanced.py` — Advanced patterns

See ../user_guide/dsl for full DSL documentation!

Getting Started Examples

Basic Network Creation

- `example_load_network.py` - Loading networks from files
- `example_create_network.py` - Creating networks from scratch
- `example_basic_stats.py` - Computing basic statistics

Quick Tutorials

- `tutorial_10min.py` - Executable version of the 10-minute tutorial
- `example_quick_start.py` - Quick start examples

Visualization Examples

Basic Visualization

- `example_multilayer_visualization.py` - Basic multilayer plots
- `example_simple_viz.py` - Simple visualization examples
- `example_hairball.py` - Hairball plots

Advanced Visualization

- `example_custom_layouts.py` - Custom layout algorithms
- `example_diagonal_plot.py` - Diagonal projection for multilayer networks
- `example_interactive_plots.py` - Interactive visualizations with Plotly
- `example_layer_visualization.py` - Layer-by-layer visualization

Community Visualization

- `example_community_viz.py` - Visualizing detected communities
- `example_colored_networks.py` - Custom node/edge coloring

Community Detection Examples

Single-Layer Community Detection

- `example_community_detection.py` - Louvain and Infomap
- `example_louvain.py` - Louvain algorithm examples
- `example_label_propagation.py` - Semi-supervised community detection

Multilayer Community Detection

- `example_multilayer_communities.py` - Multilayer Louvain
- `example_modularity.py` - Modularity optimization

- `example_overlapping_communities.py` - Overlapping community detection

DSL and Query Examples

DSL v2 (Recommended)

- `example_dsl_builder_api.py` - Comprehensive Python builder API examples (Q, L, Param)
- `example_dsl_queries.py` - String DSL syntax examples
- `example_dsl_advanced.py` - Advanced queries and transportation network analysis
- `example_dsl_community_detection.py` - Community detection with DSL

Features: - SQL-like syntax for network queries - Python builder API with type hints (`Q.nodes()`, `L["layer"]`, `Param`) - Layer algebra (union, difference, intersection) - Django-style WHERE conditions (`degree__gt=5`) - COMPUTE measures with aliases - ORDER BY, LIMIT, EXPLAIN mode - Export to pandas, NetworkX, Arrow

Network Statistics Examples

Basic Statistics

- `example_multilayer_statistics.py` - Multilayer-specific statistics
- `example_layer_comparison.py` - Comparing layers
- `example_node_metrics.py` - Node-level metrics

Centrality Measures

- `example_centrality.py` - Degree, betweenness, PageRank
- `example_multilayer_centrality.py` - Multilayer centrality measures
- `example_versatility.py` - Versatility centrality

Network Comparison

- `example_network_comparison.py` - Comparing multiple networks
- `example_statistical_tests.py` - Statistical comparison

I/O and Data Format Examples

Loading Networks

- `example_IO.py` - Various input formats
- `example_edgelist_loading.py` - Edge list formats
- `example_graphml.py` - GraphML format
- `example_csv_loading.py` - CSV with sidecars

Modern I/O (Arrow/Parquet)

- `example_arrow_io.py` - Apache Arrow format
- `example_parquet.py` - Parquet format
- `example_schema_graph.py` - Schema-based I/O

Format Conversion

- `example_format_conversion.py` - Converting between formats

- `example_export.py` - Exporting networks

Random Walks and Embeddings

Random Walks

- `example_random_walks.py` - Basic and Node2Vec walks
- `example_walkers.py` - Different walk strategies
- `example_metapath_walks.py` - Meta-path based walks

Node Embeddings

- `example_n2v_embedding.py` - Node2Vec embeddings
- `example_deepwalk.py` - DeepWalk embeddings
- `example_embeddings.py` - Various embedding methods

Embedding Applications

- `example_link_prediction.py` - Link prediction with embeddings
- `example_node_classification.py` - Node classification
- `example_clustering_embeddings.py` - Clustering with embeddings

Network Decomposition

- `example_network_decomposition.py` - Meta-path feature extraction
- `example_feature_extraction.py` - Network feature engineering
- `example_tensor_decomposition.py` - Tensor decomposition methods

Network Manipulation

- `example_manipulation.py` - Network operations (add, remove, filter)
- `example_subnetworks.py` - Extracting subnetworks
- `example_layer_operations.py` - Layer-specific operations
- `example_aggregation.py` - Aggregating layers

Algorithms and Analysis

Network Dynamics

- `example_spreading.py` - Network traversal and spreading processes
- `example_sir_epidemic.py` - SIR epidemic simulation
- `example_diffusion.py` - Diffusion processes

Specialized Algorithms

- `example_ricci_curvature.py` - Ricci curvature computation
- `example_motifs.py` - Network motif discovery
- `example_path_analysis.py` - Path-based analysis

Machine Learning

- `example_ml_features.py` - Feature extraction for ML

- `example_graph_kernels.py` - Graph kernel methods
- `example_supervised_learning.py` - Supervised learning on networks

NetworkX Integration

- `example_networkx_wrapper.py` - Using NetworkX functions
- `example_networkx_interop.py` - NetworkX interoperability
- `example_convert_networkx.py` - Converting to/from NetworkX

Benchmarking

- `example_benchmarking.py` - Performance benchmarking
- `example_scalability.py` - Scalability testing
- `example_memory_profiling.py` - Memory usage analysis

GUI and API

- `example_api_usage.py` - Using the REST API
- `example_gui_integration.py` - GUI integration examples
- `example_batch_processing.py` - Batch processing with CLI

Case Studies: End-to-End Workflows

Complete domain-specific analysis pipelines with interpretation. Each case study follows a standard workflow: Data Import → Stats → Analysis → Visualization → Interpretation.

All case studies are in the `examples/case_studies/` directory. See the Case Studies README[10] for complete documentation.

Biological Networks — Intermediate

- `biological_networks.py` - Protein-gene-disease multilayer network
  - Analyzes how protein interactions, gene regulation, and diseases interconnect
  - Identifies hub proteins (e.g., TP53) as potential drug targets
  - Detects functional biological modules through community detection
  - **Layers:** protein (PPI), gene (regulation), disease (associations)
  - **Key techniques:** Multi-layer centrality, cross-layer communities

Social Networks — Beginner

- `social_networks.py` - Multi-platform social media analysis
  - Identifies cross-platform influencers
  - Compares behavior across Facebook, Twitter, LinkedIn
  - Detects social communities spanning multiple platforms
  - **Layers:** facebook (friends), twitter (followers), linkedin (professional)
  - **Key techniques:** Influence metrics, platform comparison

Transportation Networks — Intermediate

[10] https://github.com/SkBlaz/py3plex/tree/master/examples/case_studies/README.md

- `transportation_networks.py` - Multi-modal urban transport
  - Identifies critical transfer hubs
  - Computes accessibility metrics for urban planning
  - Detects service zones and coverage gaps
  - **Layers:** bus (coverage), metro (backbone), bike (short trips)
  - **Key techniques:** Accessibility analysis, hub identification

Using Case Studies:

Each case study is: * **Self-contained** — Generates synthetic data, no external files needed * **Structured** — Follows 4-step pipeline (Import → Stats → Pipeline → Viz) * **Interpretable** — Extensive domain-specific commentary on results * **Adaptable** — Designed as templates for your own data

Run a case study:

```
python examples/case_studies/biological_networks.py
python examples/case_studies/social_networks.py
python examples/case_studies/transportation_networks.py
```

See also: * **Book Chapter 12-14** — Extended case studies with theoretical background * **Notebooks** — Interactive versions (coming soon)

Real-World Datasets

Biological Networks

- `example_protein_interaction.py` - Protein-protein interaction networks
- `example_gene_regulation.py` - Gene regulatory networks
- `example_metabolic_networks.py` - Metabolic pathways

Social Networks

- `example_social_multiplex.py` - Multiplex social networks
- `example_citation_network.py` - Citation networks
- `example_collaboration.py` - Collaboration networks

Transportation

- `example_multimodal_transport.py` - Multi-modal transportation
- `example_flight_network.py` - Airline networks

Running Examples

All examples can be run directly with Python from the repository root:

```
# Run DSL builder API examples (recommended starting point)
python examples/network_analysis/example_dsl_builder_api.py

# Run string DSL examples
python examples/network_analysis/example_dsl_queries.py

# Run advanced DSL examples
```

(continues on next page)

(continued from previous page)

```
python examples/network_analysis/example_dsl_advanced.py

# Run a visualization example
python examples/visualization/example_multilayer_visualization.py

# Run a community detection example
python examples/communities/example_community_detection.py

# Run a getting started example
python examples/getting_started/tutorial_10min.py
```

Many examples accept command-line arguments:

```
python examples/communities/example_community_detection.py --algorithm louvain
```

Example Template

Use this template when creating new example scripts. The pattern includes a docstring, a `main()` function, and an `if __name__ == "__main__":` guard:

```
"""
Example: Your Feature
=====================

Description of what this example demonstrates.

Usage:
    python examples/<category>/example_your_feature.py
"""

from py3plex.core import multinet
from py3plex.visualization.multilayer import draw_multilayer_default

def main():
    # Load or create network
    network = multinet.multi_layer_network()
    network.add_edges([
        ['A', 'layer1', 'B', 'layer1', 1]
    ], input_type="list")

    # Your analysis
    network.basic_stats()

    # Visualization
    draw_multilayer_default([network], display=True)

if __name__ == "__main__":
    main()
```

Contributing Examples

To contribute an example:

1. Create a well-documented script in the appropriate `examples/<category>/` directory
2. Use the template above with docstring + `main()` + `if __name__ == "__main__":`
3. Follow the naming convention: `example_<feature>.py`
4. Test that it runs without errors
5. Add it to this index with a brief description
6. Submit a pull request

See ../dev/contributing for detailed guidelines.

Related Documentation

- *10-Minute Tutorial* - 10-minute tutorial
- ../user_guide/networks - Working with networks
- ../user_guide/visualization - Visualization guide

Repository:

- Examples directory: https://github.com/SkBlaz/py3plex/tree/master/examples
- Submit examples: https://github.com/SkBlaz/py3plex/pulls

3.6.2 Analysis Recipes & Workflows

This guide provides **practical recipes** for **common analysis workflows** in py3plex. Each recipe is a **complete, ready-to-use solution** for a real-world task.

DSL for Rapid Analysis

Many recipes can be expressed concisely using the DSL:

```python
from py3plex.dsl import Q, L

# Quick hub identification
hubs = (
    Q.nodes()
     .where(degree__gt=5)
     .compute("betweenness_centrality")
     .order_by("-betweenness_centrality")
     .limit(10)
     .execute(network)
)

# Multi-layer comparison
for layer_name in ["social", "work", "family"]:
    result = (
        Q.nodes()
         .from_layers(L[layer_name])
         .compute("degree", "clustering")
         .execute(network)
```

```
    )
    print(f"{layer_name}: {result.count} nodes")
```

See the DSL recipes section below for more patterns!

Related Example Scripts:

Many recipes have corresponding runnable example scripts in the repository:

- I/O examples: `examples/io_and_data/example_IO.py`, `examples/io_and_data/example_new_io.py`
- Community detection: `examples/communities/example_community_detection.py`
- Network statistics: `examples/network_analysis/example_multilayer_statistics.py`
- Visualization: `examples/visualization/example_multilayer_visualization.py`
- DSL queries: `examples/network_analysis/example_dsl_queries.py`
- Complete pipelines: `examples/pipelines/example_6_complex_pipeline.py`

Recipe Index

Data Import & Setup

Recipe 1: Load Network from Multiple File Formats

Task: Load **multilayer networks** from various **file formats** (edgelist, GraphML, CSV).

Script: `examples/io_and_data/example_IO.py`

```python
from py3plex.core import multinet

# From multiedgelist (node1, layer1, node2, layer2, weight)
network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist",
    directed=False
)

# From GraphML
network = multinet.multi_layer_network().load_network(
    "data/network.graphml",
```

(continues on next page)

(continued from previous page)

```
    input_type="graphml"
)

# From edges list programmatically
network = multinet.multi_layer_network()
network.add_edges([
    ['A', 'layer1', 'B', 'layer1', 1.0],
    ['B', 'layer1', 'C', 'layer1', 1.0],
    ['A', 'layer2', 'C', 'layer2', 0.5],
], input_type="list")

# Verify the network
network.basic_stats()
```

Expected Output:

```
Number of nodes: 3
Number of edges: 3
Number of unique nodes (as node-layer tuples): 5
Number of unique node IDs (across all layers): 3
Nodes per layer:
  Layer 'layer1': 3 nodes
  Layer 'layer2': 2 nodes
```

When to use: Starting any **analysis project** with external data.

Recipe 2: Create Synthetic Benchmark Networks

Task: Generate **synthetic multilayer networks** for **testing algorithms**.

Script: `examples/getting_started/example_random_generator.py`

```
from py3plex.core.random_generators import random_multilayer_ER
import numpy as np

# Set random seed for reproducibility
np.random.seed(42)

# Create a random Erdős-Rényi multilayer network
# Parameters: n_nodes, n_layers, edge_probability
network = random_multilayer_ER(
    n=100,          # 100 nodes
    L=3,            # 3 layers
    p=0.1,          # 10% edge probability
    directed=False
)

network.basic_stats()
```

When to use: **Algorithm validation**, **testing**, **benchmarking**, or when real data is unavailable.

Recipe 3: Convert Between Network Formats

Task: Export networks to **different formats** for use in **other tools**.

```
from py3plex.core import multinet

# Load network
network = multinet.multi_layer_network().load_network(
    "input.txt",
    input_type="multiedgelist"
)

# Save as GraphML (standard format, compatible with Gephi, Cytoscape)
network.save_network("output.graphml", output_type="graphml")

# Save as edge list with node mapping
inverse_map = network.serialize_to_edgelist("output_edges.txt")

# Save as pickle (preserves all Python objects)
import pickle
with open("network.pkl", "wb") as f:
    pickle.dump(network, f)
```

When to use: Sharing data with collaborators, importing to visualization tools, or archiving results.

Network Exploration & Analysis

Recipe 4: Quick Network Summary

Task: Get comprehensive overview of network structure.

```
from py3plex.core import multinet

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Basic statistics
print("=== Basic Statistics ===")
network.basic_stats()

# Layer information
layers = network.get_layers()
print(f"\nLayers: {layers}")
print(f"Number of layers: {len(layers)}")

# Node and edge counts
nodes = list(network.get_nodes())
edges = list(network.get_edges())
print(f"Total nodes: {len(nodes)}")
```

(continues on next page)

(continued from previous page)

```
print(f"Total edges: {len(edges)}")

# Get unique nodes (nodes may appear in multiple layers)
unique_nodes = set([n[0] for n in nodes])
print(f"Unique nodes: {len(unique_nodes)}")
```

Expected Output:

```
=== Basic Statistics ===
Number of nodes: 250
Number of edges: 180
Number of unique nodes (as node-layer tuples): 250
Number of unique node IDs (across all layers): 120
Nodes per layer:
  Layer 'collaboration': 85 nodes
  Layer 'citation': 90 nodes
  Layer 'coauthor': 75 nodes

Layers: ['collaboration', 'citation', 'coauthor']
Number of layers: 3
Total nodes: 250
Total edges: 180
Unique nodes: 120
```

When to use: Initial data exploration, quality checks, reporting.

Recipe 5: Layer-by-Layer Analysis

Task: Analyze each layer independently and compare properties.

```
from py3plex.core import multinet
import networkx as nx

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Analyze each layer
layers = network.get_layers()
layer_stats = {}

for layer_name in layers:
    # Extract single layer
    layer = network.subnetwork([layer_name], subset_by="layers")

    # Compute layer statistics
    stats = {}
    stats['nodes'] = len(list(layer.get_nodes()))
    stats['edges'] = len(list(layer.get_edges()))
```

(continues on next page)

(continued from previous page)

```
    # NetworkX metrics on layer
    G = layer.core_network
    if len(G) > 0:  # Check if layer has nodes
        stats['density'] = nx.density(G)
        stats['avg_degree'] = sum(dict(G.degree()).values()) / len(G)

        # Check connectivity
        if not layer.directed:
            stats['connected'] = nx.is_connected(G)
            if stats['connected']:
                stats['diameter'] = nx.diameter(G)

    layer_stats[layer_name] = stats

# Display results
print("\n=== Layer-by-Layer Analysis ===")
for layer, stats in layer_stats.items():
    print(f"\nLayer: {layer}")
    for metric, value in stats.items():
        print(f"  {metric}: {value}")
```

When to use: Understanding layer-specific properties, identifying important layers.

Recipe 6: Compute Multilayer Statistics

Task: Calculate multilayer-specific metrics that account for cross-layer structure.

Script: `examples/network_analysis/example_multilayer_statistics.py`

```
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

layers = network.get_layers()

# === Layer-Level Statistics ===
print("=== Layer Statistics ===")
for layer in layers:
    density = mls.layer_density(network, layer)
    print(f"Layer {layer} density: {density:.4f}")

# Layer diversity
entropy = mls.entropy_of_multiplexity(network)
print(f"\nLayer diversity (entropy): {entropy:.4f} bits")
```

(continues on next page)

(continued from previous page)

```
# === Node-Level Statistics ===
print("\n=== Node Activity ===")
sample_nodes = list(network.get_nodes())[:5]
for node_tuple in sample_nodes:
    node = node_tuple[0]  # Extract node name
    activity = mls.node_activity(network, node)
    print(f"Node {node} active in {activity*100:.1f}% of layers")

# === Cross-Layer Analysis ===
if len(layers) >= 2:
    print("\n=== Cross-Layer Similarity ===")
    layer1, layer2 = str(layers[0]), str(layers[1])

    # Cosine similarity of adjacency matrices
    similarity = mls.layer_similarity(network, layer1, layer2, method='cosine
↪')
    print(f"Cosine similarity ({layer1} vs {layer2}): {similarity:.4f}")

    # Jaccard similarity of edge sets
    overlap = mls.edge_overlap(network, layer1, layer2)
    print(f"Edge overlap (Jaccard): {overlap:.4f}")

# === Versatility Analysis ===
print("\n=== Node Versatility (Cross-Layer Importance) ===")
versatility = mls.versatility_centrality(network, centrality_type='degree')
top_nodes = sorted(versatility.items(), key=lambda x: x[1], reverse=True)[:5]
for node, score in top_nodes:
    print(f"  {node}: {score:.4f}")
```

When to use: Quantifying multilayer structure, comparing layers, identifying versatile nodes.

Centrality & Node Importance

Recipe 7: Single-Layer Centrality Analysis

Task: Identify important nodes within individual layers.

```
from py3plex.core import multinet

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Select a layer
layers = network.get_layers()
target_layer = [str(layers[0])]
layer = network.subnetwork(target_layer, subset_by="layers")

# Compute multiple centrality measures
```

(continues on next page)

(continued from previous page)

```
centrality_measures = {
    'degree': layer.monoplex_nx_wrapper("degree_centrality"),
    'betweenness': layer.monoplex_nx_wrapper("betweenness_centrality"),
    'closeness': layer.monoplex_nx_wrapper("closeness_centrality"),
    'eigenvector': layer.monoplex_nx_wrapper("eigenvector_centrality"),
}

# Display top nodes for each measure
print(f"=== Centrality Analysis for Layer {target_layer[0]} ===\n")
for measure_name, scores in centrality_measures.items():
    print(f"{measure_name.capitalize()} Centrality (Top 5):")
    top_nodes = sorted(scores.items(), key=lambda x: x[1], reverse=True)[:5]
    for node, score in top_nodes:
        print(f"  {node}: {score:.4f}")
    print()
```

When to use: Identifying influential nodes, hub detection, network simplification.

Recipe 8: Multiplex Participation Coefficient

Task: Measure how evenly nodes participate across layers in a multiplex network.

```
from py3plex.core import multinet
from py3plex.algorithms.multicentrality import multiplex_participation_
↪coefficient

# Load or create multiplex network (same nodes across all layers)
network = multinet.multi_layer_network().load_network(
    "data/multiplex_network.txt",
    input_type="multiedgelist"
)

# Compute MPC (requires true multiplex: identical node set per layer)
try:
    mpc = multiplex_participation_coefficient(
        network,
        normalized=True,
        check_multiplex=True
    )

    # Display results
    print("=== Multiplex Participation Coefficient ===")
    print("(0 = active in one layer only, 1 = equally active across all␣
↪layers)\n")

    # Sort by MPC score
    sorted_nodes = sorted(mpc.items(), key=lambda x: x[1], reverse=True)

    print("Top 10 most versatile nodes:")
    for node, score in sorted_nodes[:10]:
```

(continues on next page)

(continued from previous page)

```
        print(f"  {node}: {score:.4f}")

    print("\nTop 10 most specialized nodes:")
    for node, score in sorted_nodes[-10:]:
        print(f"  {node}: {score:.4f}")

except ValueError as e:
    print(f"Error: {e}")
    print("Network is not a true multiplex (layers have different node sets)")
```

When to use: Analyzing multiplex networks, identifying cross-layer hubs, understanding node specialization.

Community Detection

Recipe 9: Multilayer Community Detection

Task: Detect communities that span multiple layers.

Script: examples/communities/example_community_detection.py

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection.multilayer_modularity import␣
↪louvain_multilayer
from collections import Counter

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Run multilayer Louvain algorithm
# gamma: resolution parameter (higher = more communities)
# omega: inter-layer coupling strength (higher = communities span layers)
partition = louvain_multilayer(
    network,
    gamma=1.0,      # Standard resolution
    omega=1.0,      # Standard coupling
    random_state=42 # For reproducibility
)

# Analyze results
num_communities = len(set(partition.values()))
print(f"Detected {num_communities} communities")

# Community size distribution
community_sizes = Counter(partition.values())
print("\nTop 10 communities by size:")
for comm_id, size in community_sizes.most_common(10):
    print(f"  Community {comm_id}: {size} nodes")

```

(continues on next page)

(continued from previous page)

```
# Check which nodes are in largest community
largest_comm = community_sizes.most_common(1)[0][0]
largest_comm_nodes = [node for node, comm in partition.items()
                      if comm == largest_comm]
print(f"\nNodes in largest community: {largest_comm_nodes[:10]}...")
```

When to use: Understanding network organization, identifying functional modules, clustering nodes.

Recipe 10: Compare Community Detection Algorithms

Task: Run multiple community detection methods and compare results.

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection import community_wrapper as cw
from py3plex.algorithms.community_detection.multilayer_modularity import␣
↪louvain_multilayer
from collections import Counter
import numpy as np

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Method 1: Single-layer Louvain on aggregated network
partition_louvain = cw.louvain_communities(network)

# Method 2: Multilayer Louvain
partition_multilayer = louvain_multilayer(
    network,
    gamma=1.0,
    omega=1.0,
    random_state=42
)

# Compare results
print("=== Community Detection Comparison ===\n")
print(f"Single-layer Louvain: {len(set(partition_louvain.values()))}␣
↪communities")
print(f"Multilayer Louvain: {len(set(partition_multilayer.values()))}␣
↪communities")

# Calculate normalized mutual information (if sklearn available)
try:
    from sklearn.metrics import normalized_mutual_info_score

    # Align node sets
    common_nodes = set(partition_louvain.keys()) & set(partition_multilayer.
↪keys())
```

(continues on next page)

(continued from previous page)

```
    labels1 = [partition_louvain[n] for n in common_nodes]
    labels2 = [partition_multilayer[n] for n in common_nodes]

    nmi = normalized_mutual_info_score(labels1, labels2)
    print(f"\nNormalized Mutual Information: {nmi:.4f}")
    print("(1.0 = identical partitions, 0.0 = independent)")
except ImportError:
    print("\nInstall scikit-learn to compute NMI: pip install scikit-learn")
```

When to use: Method validation, algorithm selection, robustness analysis.

Visualization

Recipe 11: Basic Network Visualization

Task: Create publication-ready network visualizations.

Script: `examples/visualization/example_multilayer_visualization.py`

```
from py3plex.core import multinet
from py3plex.visualization.multilayer import hairball_plot
import matplotlib
matplotlib.use('Agg')  # For non-interactive environments
import matplotlib.pyplot as plt

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Get network in visualization format
network_colors, graph = network.get_layers(style="hairball")

# Create visualization
plt.figure(figsize=(12, 12))
hairball_plot(
    graph,
    network_colors,
    layout_algorithm="force",        # Options: "force", "circular", "spring"
    layout_parameters={"iterations": 100},
    node_size=5,
    edge_width=0.5,
    alpha_channel=0.8,
    scale_by_size=True
)
plt.title("Multilayer Network", fontsize=16, fontweight='bold')
plt.axis('off')
plt.tight_layout()
plt.savefig("network_viz.png", dpi=150, bbox_inches='tight',
            facecolor='white', edgecolor='none')
plt.close()
```

(continues on next page)

(continued from previous page)

```
print("Visualization saved to network_viz.png")
```

When to use: Presentations, papers, reports, exploratory analysis.

Recipe 12: Visualize Communities on Network

Task: Color nodes by community membership in network plot.

```
from py3plex.core import multinet
from py3plex.visualization.multilayer import hairball_plot
from py3plex.visualization.colors import colors_default
from py3plex.algorithms.community_detection.multilayer_modularity import␣
↪louvain_multilayer
from collections import Counter
import matplotlib.pyplot as plt
import matplotlib
matplotlib.use('Agg')

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Detect communities
partition = louvain_multilayer(network, gamma=1.0, omega=1.0, random_state=42)

# Select top N communities to color
top_n = 10
community_counts = Counter(partition.values())
top_communities = [c for c, _ in community_counts.most_common(top_n)]

# Create color mapping
color_map = dict(zip(top_communities, colors_default[:top_n]))

# Assign colors to nodes
network_colors = [
    color_map.get(partition.get(node), "lightgray")
    for node in network.get_nodes()
]

# Get network for visualization
_, graph = network.get_layers(style="hairball")

# Create plot
plt.figure(figsize=(14, 14))
hairball_plot(
    graph,
    network_colors,
    layout_algorithm="force",
    layout_parameters={"iterations": 150},
```

(continues on next page)

(continued from previous page)

```
    node_size=8,
    edge_width=0.5,
    alpha_channel=0.7,
    scale_by_size=True
)
plt.title(f"Network with {top_n} Largest Communities",
          fontsize=16, fontweight='bold')
plt.axis('off')
plt.tight_layout()
plt.savefig("network_communities.png", dpi=150,
            bbox_inches='tight', facecolor='white')
plt.close()
print("Community visualization saved to network_communities.png")
```

When to use: Visualizing analysis results, understanding community structure.

Advanced Workflows

Recipe 13: Complete Analysis Pipeline

Task: Full workflow from data loading to results export.

```
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
from py3plex.algorithms.community_detection.multilayer_modularity import␣
↪louvain_multilayer
from collections import Counter
import json
import numpy as np

# Set random seed
np.random.seed(42)

# === 1. Load Data ===
print("=== Loading Network ===")
network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist",
    directed=False
)
network.basic_stats()

# === 2. Compute Statistics ===
print("\n=== Computing Multilayer Statistics ===")
layers = network.get_layers()

stats = {
    'num_layers': len(layers),
    'layer_densities': {},
    'layer_diversity': mls.entropy_of_multiplexity(network),
```

(continues on next page)

(continued from previous page)

```
}

for layer in layers:
    stats['layer_densities'][str(layer)] = mls.layer_density(network, layer)

# Versatility
versatility = mls.versatility_centrality(network, centrality_type='degree')
top_versatile = sorted(versatility.items(), key=lambda x: x[1],␣
↪reverse=True)[:10]
stats['top_versatile_nodes'] = {str(k): float(v) for k, v in top_versatile}

# === 3. Community Detection ===
print("\n=== Detecting Communities ===")
partition = louvain_multilayer(network, gamma=1.0, omega=1.0, random_state=42)
community_sizes = Counter(partition.values())

stats['num_communities'] = len(set(partition.values()))
stats['community_sizes'] = dict(community_sizes.most_common(10))

# === 4. Export Results ===
print("\n=== Exporting Results ===")

# Save statistics as JSON
with open("analysis_results.json", "w") as f:
    json.dump(stats, f, indent=2)
print("Statistics saved to analysis_results.json")

# Save community assignments
with open("communities.txt", "w") as f:
    for node, comm in partition.items():
        f.write(f"{node}\t{comm}\n")
print("Communities saved to communities.txt")

# Save processed network
network.save_network("processed_network.graphml", output_type="graphml")
print("Network saved to processed_network.graphml")

print("\n=== Analysis Complete ===")
```

When to use: Reproducible research workflows, batch processing, automated analysis.

Recipe 14: Network Comparison Study

Task: Compare multiple networks systematically.

```
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
import pandas as pd
import numpy as np

```

(continues on next page)

(continued from previous page)

```python
# List of networks to compare
network_files = [
    ("Network A", "data/network_a.txt"),
    ("Network B", "data/network_b.txt"),
    ("Network C", "data/network_c.txt"),
]

results = []

for name, filepath in network_files:
    print(f"\nAnalyzing {name}...")

    # Load network
    network = multinet.multi_layer_network().load_network(
        filepath,
        input_type="multiedgelist",
        directed=False
    )

    # Compute metrics
    layers = network.get_layers()
    nodes = list(network.get_nodes())
    edges = list(network.get_edges())

    metrics = {
        'Network': name,
        'Layers': len(layers),
        'Total Nodes': len(nodes),
        'Unique Nodes': len(set([n[0] for n in nodes])),
        'Total Edges': len(edges),
        'Layer Diversity': mls.entropy_of_multiplexity(network),
    }

    # Average layer density
    densities = [mls.layer_density(network, layer) for layer in layers]
    metrics['Avg Layer Density'] = np.mean(densities)
    metrics['Std Layer Density'] = np.std(densities)

    results.append(metrics)

# Create comparison table
df = pd.DataFrame(results)
print("\n=== Network Comparison ===")
print(df.to_string(index=False))

# Save to CSV
df.to_csv("network_comparison.csv", index=False)
print("\nComparison saved to network_comparison.csv")
```

When to use: Comparative studies, evaluating datasets, selecting networks for analysis.

Recipe 15: Random Walks for Node Embeddings

Task: Generate node embeddings using random walks (Node2Vec approach).

Script: `examples/advanced/example_random_walks.py`

```
from py3plex.core import multinet
from py3plex.algorithms.general.walkers import generate_walks, node2vec_walk
import numpy as np

# Set random seed
np.random.seed(42)

network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)

# Get core NetworkX graph
G = network.core_network

# Generate random walks
# p: return parameter (higher = less likely to return to previous node)
# q: in-out parameter (higher = more BFS-like, lower = more DFS-like)
walks = generate_walks(
    G,
    num_walks=10,       # Number of walks per node
    walk_length=80,     # Length of each walk
    p=1.0,              # Return parameter
    q=1.0,              # In-out parameter
    seed=42
)

print(f"Generated {len(walks)} random walks")
print(f"First walk (truncated): {walks[0][:10]}...")

# Use walks for downstream tasks:
# 1. Train Word2Vec model (if gensim available)
try:
    from gensim.models import Word2Vec

    # Convert walks to strings for Word2Vec
    walks_str = [[str(node) for node in walk] for walk in walks]

    # Train embedding model
    model = Word2Vec(
        walks_str,
        vector_size=128,    # Embedding dimension
        window=5,           # Context window
        min_count=1,
        sg=1,               # Skip-gram
```

(continues on next page)

(continued from previous page)

```
        workers=4,
        epochs=5,
        seed=42
    )

    print(f"\nTrained embedding model with {len(model.wv)} nodes")

    # Get embedding for a node
    sample_node = str(list(G.nodes())[0])
    if sample_node in model.wv:
        embedding = model.wv[sample_node]
        print(f"Embedding for node {sample_node}: {embedding[:5]}... (dim=
→{len(embedding)})")

        # Find similar nodes
        similar = model.wv.most_similar(sample_node, topn=5)
        print(f"\nMost similar nodes to {sample_node}:")
        for node, similarity in similar:
            print(f"  {node}: {similarity:.4f}")

    # Save embeddings
    model.wv.save_word2vec_format("node_embeddings.txt")
    print("\nEmbeddings saved to node_embeddings.txt")

except ImportError:
    print("\nInstall gensim for embedding generation: pip install gensim")
```

When to use: Node classification, link prediction, network clustering, feature extraction.

Performance & Optimization

Recipe 16: Handle Large Networks Efficiently

Task: Process large multilayer networks without running out of memory.

```
from py3plex.core import multinet
import gc

# === 1. Load network efficiently ===
# Use generators instead of loading all at once
network = multinet.multi_layer_network()

# For very large files, load in chunks or stream
# (This is a conceptual example - actual implementation may vary)

# === 2. Process layers independently ===
layers = network.get_layers()

layer_results = {}
for layer_name in layers:
```

(continues on next page)

(continued from previous page)

```
    print(f"Processing layer {layer_name}...")

    # Extract and process one layer at a time
    layer = network.subnetwork([layer_name], subset_by="layers")

    # Compute statistics for this layer
    nodes = len(list(layer.get_nodes()))
    edges = len(list(layer.get_edges()))

    layer_results[str(layer_name)] = {'nodes': nodes, 'edges': edges}

    # Clean up to free memory
    del layer
    gc.collect()

print("\n=== Results ===")
for layer, stats in layer_results.items():
    print(f"{layer}: {stats['nodes']} nodes, {stats['edges']} edges")

# === 3. Use sparse representations ===
# When computing supra-adjacency matrix
supra_adj = network.get_supra_adjacency_matrix(sparse=True)
print(f"\nSupra-adjacency matrix: {supra_adj.shape} (sparse)")

# === 4. Sample for exploratory analysis ===
# For visualization or initial exploration, work with a sample
all_nodes = list(network.get_nodes())
import random
random.seed(42)
sample_size = min(1000, len(all_nodes))
sampled_nodes = random.sample(all_nodes, sample_size)

# Create subnetwork with sampled nodes
# (Implementation depends on network structure)
print(f"\nSampled {sample_size} nodes for exploratory analysis")
```

When to use: Networks with >10,000 nodes, limited RAM, large-scale studies.

Tips: - Process layers sequentially rather than all at once - Use sparse matrices for large supra-adjacency representations - Sample networks for visualization and initial exploration - Save intermediate results to disk - Use generators and iterators instead of lists

Recipe 17: Benchmark and Profile Your Analysis

Task: Measure performance and identify bottlenecks.

```
from py3plex.core import multinet
from py3plex.algorithms.community_detection.multilayer_modularity import␣
↪louvain_multilayer
import time
```

(continues on next page)

(continued from previous page)

```
import psutil
import os

def get_memory_usage():
    """Get current memory usage in MB"""
    process = psutil.Process(os.getpid())
    return process.memory_info().rss / 1024 / 1024

# Load network
print("=== Performance Benchmarking ===\n")
start_mem = get_memory_usage()

start = time.time()
network = multinet.multi_layer_network().load_network(
    "data/network.txt",
    input_type="multiedgelist"
)
load_time = time.time() - start
load_mem = get_memory_usage() - start_mem

print(f"Network loading:")
print(f"  Time: {load_time:.3f} seconds")
print(f"  Memory: {load_mem:.2f} MB")

# Benchmark statistics computation
start = time.time()
network.basic_stats()
stats_time = time.time() - start
print(f"\nBasic statistics:")
print(f"  Time: {stats_time:.3f} seconds")

# Benchmark community detection
start_mem = get_memory_usage()
start = time.time()
partition = louvain_multilayer(network, gamma=1.0, omega=1.0, random_state=42)
comm_time = time.time() - start
comm_mem = get_memory_usage() - start_mem

print(f"\nCommunity detection:")
print(f"  Time: {comm_time:.3f} seconds")
print(f"  Memory: {comm_mem:.2f} MB")
print(f"  Communities found: {len(set(partition.values()))}")

# Total resource usage
print(f"\n=== Total Performance ===")
print(f"Total time: {load_time + stats_time + comm_time:.3f} seconds")
print(f"Peak memory: {get_memory_usage():.2f} MB")
```

When to use: Optimizing workflows, comparing algorithms, capacity planning.

Note: Requires `psutil` package: `pip install psutil`

Tips & Best Practices

General Tips

1. **Always set random seeds** for reproducibility:

```
import numpy as np
np.random.seed(42)
```

2. **Verify network structure** after loading:

```
network.basic_stats()  # Quick sanity check
```

3. **Handle errors gracefully**:

```
try:
    result = network.some_operation()
except Exception as e:
    print(f"Operation failed: {e}")
```

4. **Save intermediate results** for long analyses:

```
import pickle
with open("checkpoint.pkl", "wb") as f:
    pickle.dump(results, f)
```

5. **Use appropriate file formats**:
   - GraphML: Standard, compatible with other tools
   - Pickle: Fast, preserves Python objects, not portable
   - Edgelist: Simple, human-readable, large file size

Performance Tips

1. **For large networks**, process layers independently
2. **Use sparse matrices** when working with supra-adjacency
3. **Sample networks** for visualization (>1000 nodes gets crowded)
4. **Profile your code** to find bottlenecks
5. **Close plots** after saving to free memory: `plt.close()`

Visualization Tips

1. **Choose layout carefully**:
   - Force-directed: General purpose, slow for >5000 nodes
   - Circular: Fast, good for small networks
   - Spring: Balance between quality and speed
2. **Adjust node size** based on network size:

```
node_size = max(1, 100 / np.sqrt(num_nodes))
```

3. **Use alpha channel** for edge-dense networks:

```
alpha_channel=0.3  # More transparent
```

4. **Save high-resolution** for publications:

```
plt.savefig("figure.png", dpi=300)
```

Community Detection Tips

1. **Tune resolution parameter** (gamma):
   - Higher gamma → more, smaller communities
   - Lower gamma → fewer, larger communities
2. **Tune coupling parameter** (omega) for multilayer:
   - Higher omega → communities span layers
   - Lower omega → layer-specific communities
3. **Run multiple times** with different random seeds to check stability
4. **Compare multiple algorithms** to validate findings

Common Issues & Solutions

Issue: "Network appears to be empty"

Cause: Incorrect file format or delimiter.

Solution::

```
# Check file format
with open("data.txt", "r") as f:
    print(f.readlines()[:5])  # Inspect first lines

# Try different input_type
network = multinet.multi_layer_network().load_network(
    "data.txt",
    input_type="multiedgelist"  # or "edgelist", "graphml"
)
```

Issue: "No module named 'community'"

Cause: Optional dependency not installed.

Solution::

```
pip install python-louvain
```

Issue: "Visualization doesn't appear"

Cause: Using non-interactive backend or missing display.

Solution::

```
import matplotlib
matplotlib.use('Agg')  # For saving files
import matplotlib.pyplot as plt

# ... create plot ...
plt.savefig("output.png")  # Save instead of show
```

Issue: "Memory error with large network"

Cause: Loading entire network into RAM at once.

Solution: See Recipe 16 for memory-efficient processing.

Issue: "Community detection is slow"

Cause: Large network or complex structure.

Solution::

```
# Use simpler algorithm for quick results
from py3plex.algorithms.community_detection import community_wrapper as cw
partition = cw.louvain_communities(network)  # Faster than multilayer
```

Further Reading

Documentation:

- Installation guide - See main documentation
- Quick start tutorial - See main documentation
- Core concepts - See main documentation
- Algorithm selection guide - See main documentation

Examples:

See `examples/` directory for 50+ complete, working examples covering all major functionality.

Research:

- Kivelä et al. (2014) - Multilayer networks theory
- De Domenico et al. (2013) - Mathematical framework
- See Citations section in main documentation for complete reference list

DSL Query Recipes

The py3plex DSL provides powerful, concise syntax for network queries. These recipes showcase common patterns.

Recipe 18: Hub Identification with DSL

Task: Find and rank influential nodes across layers using the DSL.

DSL Example: Hub Identification

```
from py3plex.core import multinet
from py3plex.dsl import Q, L

# Load network
network = multinet.multi_layer_network().load_network(
    "data/network.txt", input_type="multiedgelist"
)

# Find top 20 hubs by betweenness centrality
hubs = (
    Q.nodes()
     .where(degree__gt=3)  # Pre-filter by degree for efficiency
     .compute("betweenness_centrality", "degree", "clustering")
     .order_by("-betweenness_centrality")
     .limit(20)
     .export_csv("top_hubs.csv")
     .execute(network)
)

# Display results
df = hubs.to_pandas()
print(df)
```

When to use: Identifying key players, finding bridge nodes, influence analysis.

Recipe 19: Multi-Layer Comparative Analysis

Task: Compare network structure across different layers using DSL.

DSL Example: Layer Comparison

```
from py3plex.dsl import Q, L
import pandas as pd

# Analyze each layer separately
layers = ["social", "work", "family"]
results = []

for layer_name in layers:
    result = (
        Q.nodes()
         .from_layers(L[layer_name])
         .compute("degree", "betweenness_centrality", "clustering")
         .execute(network)
    )

    # Calculate layer-level statistics
    df = result.to_pandas()
```

```
    layer_stats = {
        'layer': layer_name,
        'nodes': result.count,
        'avg_degree': df['degree'].mean(),
        'max_centrality': df['betweenness_centrality'].max(),
        'avg_clustering': df['clustering'].mean(),
    }
    results.append(layer_stats)

# Create comparison table
comparison_df = pd.DataFrame(results)
print(comparison_df)
```

When to use: Understanding layer-specific properties, detecting structural differences.

Recipe 20: Advanced Filtering and Export

Task: Complex multi-criteria filtering with automated export.

DSL Example: Advanced Filtering

```
from py3plex.dsl import Q, L

# Find nodes that meet multiple criteria
result = (
    Q.nodes()
     # Combine layers using set operations
     .from_layers(L["social"] + L["work"] - L["bots"])
     # Multiple filter conditions
     .where(degree__gte=5, clustering__gt=0.3)
     # Compute multiple measures
     .compute("betweenness_centrality", "pagerank", "degree")
     # Sort and limit
     .order_by("-pagerank")
     .limit(50)
     # Export in multiple formats
     .export_json("filtered_nodes.json", orient="records")
     .execute(network)
)

# Also available as pandas DataFrame
df = result.to_pandas()

# Or as NetworkX subgraph
subgraph = result.to_networkx(network)
```

When to use: Targeted analysis, automated reporting, filtering complex networks.

Recipe 21: Parameterized Query Templates

Task: Create reusable query templates for systematic analysis.

DSL Example: Query Templates

```
from py3plex.dsl import Q, L, Param

# Define a reusable query template
def find_top_nodes(layer_name, degree_threshold, top_n):
    """Reusable template for finding top nodes in a layer"""
    return (
        Q.nodes()
         .from_layers(L[layer_name])
         .where(degree__gt=degree_threshold)
         .compute("betweenness_centrality", "pagerank")
         .order_by("-betweenness_centrality")
         .limit(top_n)
    )

# Use the template with different parameters
social_hubs = find_top_nodes("social", 5, 10).execute(network)
work_hubs = find_top_nodes("work", 3, 20).execute(network)

# Alternative: Use Param for dynamic parameter binding
query = (
    Q.nodes()
     .from_layers(L[Param.str("layer")])
     .where(degree__gt=Param.int("min_degree"))
     .compute("betweenness_centrality")
     .limit(Param.int("top_n"))
)

# Execute with different parameters
result1 = query.execute(network, layer="social", min_degree=5, top_n=10)
result2 = query.execute(network, layer="work", min_degree=3, top_n=20)
```

When to use: Batch analysis, systematic parameter sweeps, reproducible workflows.

Recipe 22: DSL with EXPLAIN Mode

Task: Optimize queries before execution on large networks.

DSL Example: Query Optimization

```
from py3plex.dsl import Q, L

# Build a potentially expensive query
query = (
    Q.nodes()
```

```
      .compute("betweenness_centrality")  # O(n³) operation!
      .order_by("-betweenness_centrality")
      .limit(10)
)

# Check the execution plan first
plan = query.explain().execute(network)

print("Query Execution Plan:")
for step in plan.steps:
    print(f"- {step.description}")
    print(f"  Complexity: {step.estimated_complexity}")

# Check for warnings
if plan.warnings:
    print("\nWarnings:")
    for warning in plan.warnings:
        print(f"  {warning}")

# Optimized version: filter first to reduce computation
optimized_query = (
    Q.nodes()
      .where(degree__gt=5)  # Pre-filter to reduce nodes
      .compute("betweenness_centrality")
      .order_by("-betweenness_centrality")
      .limit(10)
)

# Execute the optimized query
result = optimized_query.execute(network)
```

When to use: Large networks, expensive centrality computations, performance optimization.

3.6.3 Use Cases & Case Studies

"In theory there is no difference between theory and practice. In practice there is." — Yogi Berra

DSL in Case Studies

Throughout these case studies, notice how DSL simplifies analysis:

```
from py3plex.dsl import Q, L

# Identify candidate proteins (biological networks)
candidates = (
    Q.nodes()
      .where(degree__gt=10)
      .compute("betweenness_centrality", "clustering")
      .order_by("-betweenness_centrality")
      .limit(50)
```

```
    .execute(ppi_network)
)

# Compare social media presence (social networks)
for platform in ["twitter", "facebook", "instagram"]:
    influencers = (
        Q.nodes()
         .from_layers(L[platform])
         .where(degree__gt=100)
         .execute(social_network)
    )
    print(f"{platform}: {influencers.count} influencers")
```

DSL enables rapid iteration in exploratory analysis!

This chapter provides complete, end-to-end case studies demonstrating py3plex in different research domains. Each case study includes:

1. **Problem context** — What real-world question are we answering?
2. **Data modeling decisions** — How do we represent this problem as a multilayer network?
3. **Complete code** — Working examples you can adapt
4. **Interpretation** — What do the results mean?
5. **Adaptation guide** — How to apply this template to your own data

Why Case Studies Matter

Reading about algorithms is one thing. Applying them to real problems is another entirely.

The case studies in this chapter come from real research domains where multilayer network analysis has proven valuable. They're not toy examples—they represent the kinds of questions that researchers and practitioners actually ask:

- How do we identify proteins that are likely to be functionally important?
- Which social media influencers are genuinely cross-platform celebrities versus one-hit wonders?
- What happens to urban mobility when a major transit hub fails?
- Which academic researchers are likely to collaborate based on their publication patterns?

Each case study walks through the complete analysis workflow, from problem formulation through data modeling, analysis, and interpretation. Pay attention not just to the code, but to the reasoning behind each decision.

Lessons from the Field

Before diving into specific cases, here are some lessons learned from applying multilayer network analysis to real problems:

Modeling decisions matter more than algorithm choice. Whether you use Louvain or Infomap for community detection is less important than whether you've correctly identified what should be a layer, what should be a node, and how layers should be coupled. Spend your time on modeling.

Start simple, add complexity as needed. It's tempting to immediately build a network with ten layers, weighted edges, temporal dynamics, and inter-layer dependencies. Resist this urge. Start with the simplest model that might answer your question, and add complexity only when you can demonstrate it improves your analysis.

Validate with domain knowledge. Network analysis can reveal surprising patterns—but it can also reveal artifacts of your modeling choices. Always sanity-check results against what domain experts know. If your community detection puts obviously unrelated entities in the same cluster, you have a modeling problem.

Document your decisions. When you return to an analysis six months later, you won't remember why you chose certain parameters. Write down your reasoning. Future-you will thank present-you.

Case Study 1: Biological Network Analysis

Identifying high-confidence protein interactions and functional modules

Real-World Impact

Protein-protein interaction (PPI) networks are foundational to understanding cellular function. The proteins in your body don't work alone—they form complexes, signaling cascades, and metabolic pathways. Knowing which proteins interact helps researchers:

- **Understand disease mechanisms:** Many diseases result from disrupted protein interactions
- **Identify drug targets:** Proteins central to disease networks are potential therapeutic targets
- **Predict protein function:** A protein's interactors suggest its biological role

The challenge? PPI data comes from multiple experimental methods, each with different biases and error rates. High-throughput screens find many interactions quickly but include false positives. Literature curation is accurate but incomplete. How do you combine these sources intelligently?

This is exactly what multilayer network analysis is for.

Multilayer Protein-Protein Interaction Network

Problem Context:

You're a computational biologist studying protein interactions in yeast. You have PPI data from three experimental sources with different reliability levels:

- **Yeast two-hybrid (Y2H):** High-throughput but many false positives
- **Affinity purification mass spectrometry (AP-MS):** More reliable but biased toward stable complexes
- **Literature curated:** Highly reliable but incomplete

You want to:

1. Find functional modules (groups of proteins that work together)
2. Identify proteins that are central across evidence types (likely real interactions)
3. Discover proteins that bridge different functional modules

Data Modeling Decisions:

- **Node type:** Proteins (same set across layers for multiplex analysis)
- **Layers:** One per evidence type (Y2H, AP-MS, Literature)
- **Edges:** Physical protein-protein interactions
- **Network type:** Multiplex (same proteins, different interaction evidence)

Complete Code:

```
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
from py3plex.algorithms.community_detection.multilayer_modularity import␣
↪louvain_multilayer
from collections import Counter
import numpy as np

# === 1. Create the network ===
# In practice, you'd load from files; here we create a toy example
network = multinet.multi_layer_network(network_type='multiplex')

# Simulated PPI data (protein pairs by evidence type)
# Replace with your actual data loading
ppi_data = {
    'Y2H': [
        ('CDC28', 'CLB2'), ('CDC28', 'CLB5'), ('CLB2', 'SIC1'),
        ('ACT1', 'MYO2'), ('ACT1', 'TPM1'), ('MYO2', 'TPM1'),
        ('CDC28', 'CDC6'), ('CDC6', 'ORC1'),  # cell cycle proteins
    ],
    'APMS': [
        ('CDC28', 'CLB2'), ('CDC28', 'CKS1'), ('CLB2', 'CKS1'),
        ('ACT1', 'ABP1'), ('ACT1', 'TPM1'),
        ('CDC6', 'ORC1'), ('ORC1', 'ORC2'), ('ORC2', 'ORC3'),  # origin␣
↪recognition complex
    ],
    'Literature': [
        ('CDC28', 'CLB2'), ('CDC28', 'CLB5'), ('CDC28', 'CKS1'),
        ('ACT1', 'MYO2'), ('MYO2', 'TPM1'),
        ('CDC6', 'ORC1'),  # well-established interaction
    ],
}

# Add edges to network
for layer, interactions in ppi_data.items():
    for protein1, protein2 in interactions:
        network.add_edges([
            [protein1, layer, protein2, layer, 1.0]
        ], input_type="list")

# === 2. Basic exploration ===
print("=== Network Overview ===")
network.basic_stats()
```

(continues on next page)

(continued from previous page)

```
# === 3. Identify high-confidence interactions ===
# Interactions that appear in multiple evidence types are more reliable
print("\n=== Evidence Overlap Analysis ===")

# Get edges from each layer
edges_by_layer = {}
for layer in network.get_layers():
    layer_subnet = network.subnetwork([layer], subset_by="layers")
    # Extract edge pairs (ignoring layer info)
    edges = set()
    for edge in layer_subnet.get_edges():
        # Normalize edge representation (sorted tuple)
        pair = tuple(sorted([edge[0][0], edge[1][0]]))
        edges.add(pair)
    edges_by_layer[layer] = edges

# Find edges in multiple layers
all_edges = set()
for edges in edges_by_layer.values():
    all_edges.update(edges)

edge_evidence_counts = {}
for edge in all_edges:
    count = sum(1 for layer_edges in edges_by_layer.values() if edge in layer_
↪edges)
    edge_evidence_counts[edge] = count

print("High-confidence interactions (in 3/3 evidence types):")
for edge, count in sorted(edge_evidence_counts.items(), key=lambda x: x[1],␣
↪reverse=True):
    if count == 3:
        print(f"  {edge[0]} -- {edge[1]}: {count} evidence types")

print("\nMedium-confidence interactions (in 2/3 evidence types):")
for edge, count in sorted(edge_evidence_counts.items(), key=lambda x: x[1],␣
↪reverse=True):
    if count == 2:
        print(f"  {edge[0]} -- {edge[1]}: {count} evidence types")

# === 4. Node activity analysis ===
print("\n=== Protein Activity Across Evidence Types ===")
unique_proteins = set()
for node in network.get_nodes():
    unique_proteins.add(node[0])  # Extract protein name

protein_activity = {}
for protein in unique_proteins:
    activity = mls.node_activity(network, protein)
```

(continues on next page)

(continued from previous page)

```
    protein_activity[protein] = activity

print("Proteins present in all evidence types (activity = 1.0):")
for protein, activity in sorted(protein_activity.items(), key=lambda x: x[1],␣
↪reverse=True):
    if activity == 1.0:
        print(f"  {protein}")

# === 5. Community detection (functional modules) ===
print("\n=== Functional Module Detection ===")

# Lower coupling (omega=0.5) because different evidence types
# may reveal different aspects of the interactome
partition = louvain_multilayer(
    network,
    gamma=1.0,      # Standard resolution
    omega=0.5,      # Moderate coupling
    random_state=42
)

# Analyze communities
num_communities = len(set(partition.values()))
print(f"Detected {num_communities} functional modules")

# Group proteins by community
communities = {}
for node, comm in partition.items():
    protein = node[0]  # Extract protein name
    if comm not in communities:
        communities[comm] = set()
    communities[comm].add(protein)

print("\nFunctional modules found:")
for comm_id, proteins in sorted(communities.items(), key=lambda x: len(x[1]),␣
↪reverse=True):
    print(f"  Module {comm_id}: {proteins}")

# === 6. Versatility analysis (cross-module bridging) ===
print("\n=== Bridge Proteins (High Versatility) ===")
versatility = mls.versatility_centrality(network, centrality_type='degree')
top_versatile = sorted(versatility.items(), key=lambda x: x[1],␣
↪reverse=True)[:5]
for node, score in top_versatile:
    print(f"  {node}: versatility = {score:.4f}")
```

Interpretation:

The analysis reveals several insights:

1. **High-confidence interactions** (CDC28-CLB2, ACT1-TPM1) appear in all three evidence types, making them likely true interactions suitable for downstream analysis.

2. **Functional modules** correspond to known protein complexes:
   - Cell cycle regulators (CDC28, CLB2, CLB5, CKS1, SIC1)
   - Actin cytoskeleton (ACT1, MYO2, TPM1, ABP1)
   - DNA replication origin complex (CDC6, ORC1, ORC2, ORC3)
3. **Bridge proteins** with high versatility may connect different functional modules and could be interesting drug targets or key regulators.

Adapting to Your Data:

1. Replace the `ppi_data` dictionary with your actual data loading code
2. Adjust `omega` based on your domain: lower for diverse evidence types, higher if you expect consistency
3. Add Gene Ontology enrichment analysis to validate that communities match known functional annotations
4. Consider edge weights based on evidence quality (e.g., literature = 3.0, APMS = 2.0, Y2H = 1.0)

Case Study 2: Multi-Platform Social Network Analysis

Cross-Platform Influence Analysis

Problem Context:

You're a social media researcher studying how influencers operate across platforms. You have data from three platforms:

- **Twitter/X:** Public follower/following relationships
- **YouTube:** Subscription relationships
- **Instagram:** Follow relationships

You want to:

1. Identify influencers who are successful across platforms (vs. platform-specific)
2. Understand how different platforms are related (do the same people follow each other?)
3. Find communities that span platforms

Data Modeling Decisions:

- **Node type:** Users (mapped across platforms using verified identities)
- **Layers:** One per platform (Twitter, YouTube, Instagram)
- **Edges:** Follow/subscribe relationships (directed in reality, undirected for this analysis)
- **Network type:** Multiplex (same users, different platforms)

Complete Code:

```
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
from py3plex.algorithms.community_detection.multilayer_modularity import␣
↪louvain_multilayer
from collections import Counter
import networkx as nx
```

(continues on next page)

(continued from previous page)

```
# === 1. Create the network ===
network = multinet.multi_layer_network(network_type='multiplex')

# Simulated social network data
# In practice: load from APIs or datasets with user ID mapping
social_data = {
    'Twitter': [
        ('influencer_A', 'user_1'), ('influencer_A', 'user_2'),
        ('influencer_A', 'user_3'), ('influencer_B', 'user_2'),
        ('influencer_B', 'user_4'), ('user_1', 'user_2'),
        ('user_3', 'user_5'), ('influencer_C', 'user_6'),
    ],
    'YouTube': [
        ('influencer_A', 'user_1'), ('influencer_A', 'user_3'),
        ('influencer_B', 'user_2'), ('influencer_B', 'user_3'),
        ('influencer_B', 'user_4'), ('influencer_C', 'user_6'),
        ('influencer_C', 'user_7'),
    ],
    'Instagram': [
        ('influencer_A', 'user_1'), ('influencer_A', 'user_2'),
        ('influencer_A', 'user_4'), ('influencer_B', 'user_5'),
        ('user_1', 'user_2'), ('user_2', 'user_3'),
        ('influencer_C', 'user_6'), ('influencer_C', 'user_8'),
    ],
}

for layer, connections in social_data.items():
    for user1, user2 in connections:
        network.add_edges([
            [user1, layer, user2, layer, 1.0]
        ], input_type="list")

# === 2. Basic statistics ===
print("=== Network Overview ===")
network.basic_stats()

# === 3. Cross-platform presence ===
print("\n=== Cross-Platform Presence ===")
unique_users = set()
for node in network.get_nodes():
    unique_users.add(node[0])

for user in sorted(unique_users):
    activity = mls.node_activity(network, user)
    platforms = activity * len(network.get_layers())
    if activity == 1.0:
        print(f"  {user}: present on ALL platforms")
    elif activity > 0.5:
```

(continues on next page)

(continued from previous page)

```
        print(f"  {user}: present on {platforms:.0f}/3 platforms")

# === 4. Platform-specific centrality ===
print("\n=== Centrality by Platform ===")

for platform in network.get_layers():
    layer_subnet = network.subnetwork([platform], subset_by="layers")
    G = layer_subnet.core_network

    # Compute degree centrality
    degree_cent = nx.degree_centrality(G)
    top_users = sorted(degree_cent.items(), key=lambda x: x[1],␣
↪reverse=True)[:3]

    print(f"\n{platform} - Top 3 by degree:")
    for node, score in top_users:
        print(f"  {node[0]}: {score:.3f}")

# === 5. Cross-platform influence (versatility) ===
print("\n=== Cross-Platform Influencers (High Versatility) ===")
versatility = mls.versatility_centrality(network, centrality_type='degree')
top_versatile = sorted(versatility.items(), key=lambda x: x[1],␣
↪reverse=True)[:5]

for node, score in top_versatile:
    print(f"  {node}: {score:.4f}")

# === 6. Layer similarity ===
print("\n=== Platform Similarity ===")
layers = list(network.get_layers())
for i in range(len(layers)):
    for j in range(i+1, len(layers)):
        layer1, layer2 = str(layers[i]), str(layers[j])

        # Edge overlap
        overlap = mls.edge_overlap(network, layer1, layer2)

        # Jaccard similarity
        similarity = mls.layer_similarity(network, layer1, layer2, method=
↪'jaccard')

        print(f"  {layer1} vs {layer2}:")
        print(f"    Edge overlap: {overlap:.3f}")
        print(f"    Jaccard similarity: {similarity:.3f}")

# === 7. Cross-platform community detection ===
print("\n=== Cross-Platform Communities ===")

# High coupling because we expect influencers to attract similar audiences
```

(continues on next page)

(continued from previous page)

```
partition = louvain_multilayer(
    network,
    gamma=1.0,
    omega=1.5,      # High coupling for cross-platform consistency
    random_state=42
)

num_communities = len(set(partition.values()))
print(f"Detected {num_communities} cross-platform communities")

# Analyze which users are in which community
communities = {}
for node, comm in partition.items():
    user = node[0]
    if comm not in communities:
        communities[comm] = set()
    communities[comm].add(user)

print("\nCommunities (by unique users):")
for comm_id, users in sorted(communities.items(), key=lambda x: len(x[1]),␣
↪reverse=True):
    influencers = [u for u in users if 'influencer' in u]
    regular = [u for u in users if 'influencer' not in u]
    print(f"  Community {comm_id}: {len(influencers)} influencers,
↪{len(regular)} regular users")
    print(f"    Influencers: {influencers}")
    print(f"    Sample users: {regular[:3]}")
```

Interpretation:

1. **Cross-platform influencers** (influencer_A, influencer_B) have high versatility, indicating they successfully maintain audiences across platforms. Platform-specific influencers (influencer_C) may have niche appeal.
2. **Platform similarity** shows how audiences overlap:
   - High Twitter-Instagram similarity suggests similar user bases
   - Lower YouTube overlap might indicate different content consumption patterns
3. **Communities** reveal audience segments:
   - Some communities cluster around specific influencers
   - Cross-platform communities suggest genuine fandom that follows across platforms

Adapting to Your Data:

1. Map user identities across platforms (use verified accounts, linked profiles, or username matching)
2. Consider using directed edges for asymmetric follow relationships
3. Add edge weights based on interaction strength (likes, comments, shares)
4. Include temporal layers to track audience evolution over time

Case Study 3: Multi-Modal Transportation Analysis

Urban Mobility and Resilience

Problem Context:

You're an urban transportation analyst studying a city's public transit system. You have network data for:

- **Metro/Subway:** High-capacity, fixed routes
- **Bus:** Lower capacity, more flexible routes
- **Bike-share:** Individual, first/last mile connections

You want to:

1. Identify multimodal hubs (stations serving multiple transport modes)
2. Analyze network resilience (what happens if a metro station closes?)
3. Find communities ("travel basins") that span transport modes

Data Modeling Decisions:

- **Node type:** Stations/stops (with geographic coordinates)
- **Layers:** One per transport mode (Metro, Bus, Bikeshare)
- **Edges:** Direct connections between stops (can walk/transfer)
- **Network type:** Multiplex with geographic coupling (same location = same node)

Complete Code:

```
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
from py3plex.algorithms.community_detection.multilayer_modularity import␣
↪louvain_multilayer
import networkx as nx
from collections import Counter

# === 1. Create the transportation network ===
network = multinet.multi_layer_network(network_type='multiplex')

# Simulated city transport data
# Stations are named by neighborhood/landmark
transport_data = {
    'Metro': [
        ('Central', 'Financial'), ('Financial', 'Tech_Park'),
        ('Central', 'University'), ('University', 'Hospital'),
        ('Central', 'Shopping_Mall'), ('Shopping_Mall', 'Residential_North'),
    ],
    'Bus': [
        ('Central', 'Financial'), ('Financial', 'Residential_East'),
        ('Central', 'University'), ('University', 'Residential_South'),
        ('Central', 'Shopping_Mall'), ('Shopping_Mall', 'Residential_North'),
        ('Tech_Park', 'Residential_East'),  # Bus connects where metro doesn't
        ('Hospital', 'Residential_South'),
        ('Airport', 'Central'),  # Bus to airport
```

(continues on next page)

(continued from previous page)

```
    ],
    'Bikeshare': [
        ('Central', 'Financial'), ('Financial', 'Tech_Park'),
        ('Central', 'University'), ('University', 'Residential_South'),
        ('Shopping_Mall', 'Residential_North'),
        ('Tech_Park', 'Coffee_District'),  # Bike-only area
        ('University', 'Student_Housing'),  # Bike-only connection
    ],
}

for mode, connections in transport_data.items():
    for stop1, stop2 in connections:
        network.add_edges([
            [stop1, mode, stop2, mode, 1.0]
        ], input_type="list")

# === 2. Network overview ===
print("=== Transportation Network Overview ===")
network.basic_stats()

# === 3. Multimodal hub identification ===
print("\n=== Multimodal Hubs ===")
unique_stations = set()
for node in network.get_nodes():
    unique_stations.add(node[0])

station_modes = {}
for station in unique_stations:
    activity = mls.node_activity(network, station)
    num_modes = activity * len(network.get_layers())
    station_modes[station] = num_modes

print("Stations by number of transport modes:")
for station, modes in sorted(station_modes.items(), key=lambda x: x[1],␣
↪reverse=True):
    mode_names = []
    for mode in network.get_layers():
        layer_subnet = network.subnetwork([mode], subset_by="layers")
        layer_nodes = [n[0] for n in layer_subnet.get_nodes()]
        if station in layer_nodes:
            mode_names.append(mode)

    if modes >= 2:
        print(f"  {station}: {modes:.0f} modes ({', '.join(mode_names)})")

# === 4. Mode-specific analysis ===
print("\n=== Transport Mode Characteristics ===")

for mode in network.get_layers():
```

(continues on next page)

(continued from previous page)

```
    layer_subnet = network.subnetwork([mode], subset_by="layers")
    G = layer_subnet.core_network

    num_stops = G.number_of_nodes()
    num_connections = G.number_of_edges()
    density = mls.layer_density(network, mode)

    # Find most connected stops
    degree_cent = nx.degree_centrality(G)
    top_stop = max(degree_cent.items(), key=lambda x: x[1])

    print(f"\n{mode}:")
    print(f"  Stops: {num_stops}")
    print(f"  Connections: {num_connections}")
    print(f"  Network density: {density:.3f}")
    print(f"  Most connected: {top_stop[0][0]} (degree={top_stop[1]:.2f})")

# === 5. Resilience analysis: What if Central closes? ===
print("\n=== Resilience Analysis: Central Station Failure ===")

# Current connectivity
G_full = network.core_network
num_components_before = nx.number_connected_components(G_full.to_undirected())
print(f"Before failure: {num_components_before} connected component(s)")

# Simulate failure by removing Central from all layers
G_after = G_full.copy()
central_nodes = [n for n in G_after.nodes() if n[0] == 'Central']
G_after.remove_nodes_from(central_nodes)

num_components_after = nx.number_connected_components(G_after.to_undirected())
print(f"After removing Central: {num_components_after} connected component(s)
↪")

# Which stations become disconnected?
if num_components_after > 1:
    components = list(nx.connected_components(G_after.to_undirected()))
    print("\nResulting network fragments:")
    for i, comp in enumerate(sorted(components, key=len, reverse=True)):
        stations = set(n[0] for n in comp)
        print(f"  Fragment {i+1} ({len(stations)} stations): {stations}")

# === 6. Travel basin detection ===
print("\n=== Travel Basins (Communities) ===")

# Moderate coupling: travel patterns may differ by mode
partition = louvain_multilayer(
    network,
    gamma=1.0,
```

(continues on next page)

(continued from previous page)

```
    omega=1.0,
    random_state=42
)

num_basins = len(set(partition.values()))
print(f"Detected {num_basins} travel basins")

# Group stations by basin
basins = {}
for node, basin in partition.items():
    station = node[0]
    if basin not in basins:
        basins[basin] = set()
    basins[basin].add(station)

print("\nTravel basins:")
for basin_id, stations in sorted(basins.items(), key=lambda x: len(x[1]),␣
↪reverse=True):
    print(f"  Basin {basin_id}: {stations}")
```

Interpretation:

1. **Multimodal hubs** (Central, Financial, University) are critical infrastructure points where passengers transfer between modes. These should be prioritized for maintenance and security.
2. **Resilience analysis** reveals that Central is a critical node—its failure fragments the network. This suggests the need for redundant connections or alternative routes.
3. **Travel basins** correspond to geographic areas where people typically travel:
   - Downtown basin: Central, Financial, Shopping_Mall
   - University/Hospital basin: University, Hospital, Student_Housing
   - Residential basins: various residential areas with their nearest transit

Adapting to Your Data:

1. Include geographic distance as edge weight (travel time between stops)
2. Add passenger flow data to weight edges by usage
3. Include temporal layers (peak hours, off-peak, weekend)
4. Add inter-layer edges representing walking transfers between nearby stations

Case Study 4: Heterogeneous Academic Network

Research Collaboration and Impact Analysis

Problem Context:

You're analyzing academic research patterns using data from publications. Your network includes:

- **Authors:** Researchers who write papers
- **Papers:** Research publications

- **Venues:** Conferences and journals where papers appear
- **Institutions:** Organizations where authors work

You want to:

1. Find influential authors (not just by citation count)
2. Identify research communities that span institutions
3. Discover emerging collaboration patterns

Data Modeling Decisions:

- **Node types:** Authors, Papers, Venues, Institutions (heterogeneous)
- **Edge types:** * Author → Paper (authorship) * Paper → Venue (publication) * Author → Institution (affiliation)
- **Network type:** Heterogeneous Information Network (HIN)

Complete Code:

```
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls
import networkx as nx
from collections import Counter, defaultdict

# === 1. Create the academic network ===
network = multinet.multi_layer_network()

# Academic publication data
# In practice: load from DBLP, Semantic Scholar, or similar

# Authors write papers
authorship_data = [
    ('Dr_Smith', 'Paper_1'), ('Dr_Smith', 'Paper_2'),
    ('Dr_Jones', 'Paper_1'), ('Dr_Jones', 'Paper_3'),
    ('Dr_Chen', 'Paper_2'), ('Dr_Chen', 'Paper_4'),
    ('Dr_Kim', 'Paper_3'), ('Dr_Kim', 'Paper_4'),
    ('Dr_Garcia', 'Paper_5'), ('Dr_Lee', 'Paper_5'),
    ('Dr_Smith', 'Paper_6'), ('Dr_Lee', 'Paper_6'),  # Cross-institution␣
↪collab
]

# Papers published in venues
publication_data = [
    ('Paper_1', 'ICML'), ('Paper_2', 'NeurIPS'),
    ('Paper_3', 'ICML'), ('Paper_4', 'NeurIPS'),
    ('Paper_5', 'AAAI'), ('Paper_6', 'ICML'),
]

# Authors affiliated with institutions
affiliation_data = [
    ('Dr_Smith', 'MIT'), ('Dr_Jones', 'MIT'),
    ('Dr_Chen', 'Stanford'), ('Dr_Kim', 'Stanford'),
    ('Dr_Garcia', 'Berkeley'), ('Dr_Lee', 'Berkeley'),
```

(continues on next page)

(continued from previous page)

```
]

# Add to network with explicit layer/type information
for author, paper in authorship_data:
    network.add_edges([
        [author, 'authors', paper, 'papers', 1.0]
    ], input_type="list")

for paper, venue in publication_data:
    network.add_edges([
        [paper, 'papers', venue, 'venues', 1.0]
    ], input_type="list")

for author, institution in affiliation_data:
    network.add_edges([
        [author, 'authors', institution, 'institutions', 1.0]
    ], input_type="list")

# === 2. Network overview ===
print("=== Academic Network Overview ===")
network.basic_stats()

# === 3. Meta-path based analysis ===
# Meta-path: Author → Paper → Venue → Paper → Author
# This finds authors who publish in the same venues

print("\n=== Meta-Path Analysis: Authors Publishing in Same Venues ===")

G = network.core_network

# Find which authors publish where
author_venues = defaultdict(set)
for author, paper in authorship_data:
    for p, venue in publication_data:
        if paper == p:
            author_venues[author].add(venue)

print("Authors by venue:")
venue_authors = defaultdict(list)
for author, venues in author_venues.items():
    for venue in venues:
        venue_authors[venue].append(author)

for venue, authors in sorted(venue_authors.items()):
    print(f"  {venue}: {authors}")

# Find co-venue patterns (authors who could collaborate based on venue)
print("\nPotential collaborators (same venues):")
for venue, authors in venue_authors.items():
```

(continues on next page)

(continued from previous page)

```
    if len(authors) >= 2:
        for i in range(len(authors)):
            for j in range(i+1, len(authors)):
                print(f"  {authors[i]} -- {authors[j]} (both publish at
↪{venue})")

# === 4. Cross-institution collaboration analysis ===
print("\n=== Cross-Institution Collaborations ===")

# Find author affiliations
author_institution = dict(affiliation_data)

# Find collaborations (authors on same paper)
collaborations = defaultdict(set)
paper_authors = defaultdict(list)
for author, paper in authorship_data:
    paper_authors[paper].append(author)

cross_institution_collabs = []
for paper, authors in paper_authors.items():
    if len(authors) >= 2:
        for i in range(len(authors)):
            for j in range(i+1, len(authors)):
                a1, a2 = authors[i], authors[j]
                inst1 = author_institution.get(a1, 'Unknown')
                inst2 = author_institution.get(a2, 'Unknown')
                if inst1 != inst2:
                    cross_institution_collabs.append((a1, a2, paper, inst1,␣
↪inst2))

print("Cross-institution collaborations:")
for a1, a2, paper, inst1, inst2 in cross_institution_collabs:
    print(f"  {a1} ({inst1}) -- {a2} ({inst2}) on {paper}")

# === 5. Author influence metrics ===
print("\n=== Author Influence ===")

# Count papers per author
author_paper_count = Counter(a for a, p in authorship_data)

# Count unique venues per author
author_venue_count = {a: len(v) for a, v in author_venues.items()}

# Count co-authors
author_coauthors = defaultdict(set)
for paper, authors in paper_authors.items():
    for author in authors:
        for coauthor in authors:
            if author != coauthor:
```

(continues on next page)

(continued from previous page)

```
                author_coauthors[author].add(coauthor)

print("Author metrics:")
print(f"{'Author':<15} {'Papers':<8} {'Venues':<8} {'Coauthors':<10}")
print("-" * 45)

all_authors = set(a for a, p in authorship_data)
for author in sorted(all_authors):
    papers = author_paper_count.get(author, 0)
    venues = author_venue_count.get(author, 0)
    coauthors = len(author_coauthors.get(author, set()))
    print(f"{author:<15} {papers:<8} {venues:<8} {coauthors:<10}")

# === 6. Institution collaboration network ===
print("\n=== Institution Collaboration Network ===")

# Build institution-level collaboration graph
inst_collabs = Counter()
for a1, a2, paper, inst1, inst2 in cross_institution_collabs:
    pair = tuple(sorted([inst1, inst2]))
    inst_collabs[pair] += 1

print("Institution pairs by collaboration count:")
for pair, count in inst_collabs.most_common():
    print(f"  {pair[0]} -- {pair[1]}: {count} joint papers")
```

Interpretation:

1. **Meta-path analysis** reveals implicit relationships—authors who publish at the same venues but haven't collaborated yet are potential future collaborators.
2. **Cross-institution collaborations** highlight knowledge transfer patterns. The Smith-Lee collaboration bridges MIT and Berkeley.
3. **Author influence** considers multiple factors beyond raw paper count:
   - Venue diversity (publishing at multiple top venues)
   - Collaboration breadth (working with many different people)
   - Cross-institution reach

Adapting to Your Data:

1. Load from academic databases (DBLP XML, Semantic Scholar API)
2. Add citation edges between papers for impact analysis
3. Include temporal information for trend analysis
4. Weight edges by author position (first author, corresponding author)

Adapting Case Studies to Your Domain

General Adaptation Process

1. **Identify your entities** → These become node types or layers
2. **Identify your relationships** → These become edges
3. **Decide multiplex vs. heterogeneous:**
   - Same entities, different relationship types → Multiplex
   - Different entity types → Heterogeneous
4. **Map identifiers** → Ensure same entity has same ID across layers
5. **Choose appropriate metrics** based on your research questions
6. **Validate with domain knowledge** → Do results match expectations?

Data Loading Templates

From CSV with explicit layers:

```
import pandas as pd
from py3plex.core import multinet

network = multinet.multi_layer_network()

df = pd.read_csv('your_data.csv')
# Expected columns: source, target, layer, weight

for _, row in df.iterrows():
    network.add_edges([
        [row['source'], row['layer'], row['target'], row['layer'], row['weight
↪']]
    ], input_type="list")
```

From multiple files (one per layer):

```
layer_files = {
    'layer1': 'layer1_edges.csv',
    'layer2': 'layer2_edges.csv',
    'layer3': 'layer3_edges.csv',
}

network = multinet.multi_layer_network()

for layer_name, filepath in layer_files.items():
    df = pd.read_csv(filepath)
    for _, row in df.iterrows():
        network.add_edges([
            [row['source'], layer_name, row['target'], layer_name, 1.0]
        ], input_type="list")
```

From database:

```
import sqlite3
from py3plex.core import multinet

network = multinet.multi_layer_network()

conn = sqlite3.connect('network.db')
cursor = conn.cursor()

cursor.execute('SELECT source, target, layer, weight FROM edges')
for source, target, layer, weight in cursor.fetchall():
    network.add_edges([
        [source, layer, target, layer, weight]
    ], input_type="list")

conn.close()
```

Conclusion: Patterns Across Domains

Looking across these case studies, several patterns emerge:

Multilayer structure reveals what single-layer analysis misses. In the PPI network, interactions confirmed by multiple evidence types are more reliable—information you lose if you flatten into a single network. In the social network, cross-platform influencers are fundamentally different from single-platform celebrities. In transportation, failure cascades depend on inter-modal connections.

The coupling parameter matters. Lower coupling (omega < 1) finds layer-specific communities; higher coupling (omega > 1) finds cross-layer communities. The right choice depends on your question. If you expect the same underlying structure across layers, use higher coupling. If layers represent truly different phenomena, use lower coupling.

Validation requires domain expertise. Network analysis can find structure, but whether that structure is meaningful requires interpretation. Do the detected communities correspond to known functional categories? Do the identified hubs match known influential entities? Ground your analysis in domain knowledge.

Start simple. Each case study could be made more complex—weighted edges, temporal dynamics, more layers, heterogeneous node types. But starting simple lets you understand what's happening and catch errors before the analysis becomes opaque.

The Meta-Lesson

These case studies demonstrate a workflow pattern:

1. **Formulate your question** in terms of network structure
2. **Model your data** as nodes, edges, and layers
3. **Apply appropriate algorithms** based on your question
4. **Interpret results** using domain knowledge
5. **Iterate** as you learn what works

This pattern applies regardless of domain. The specific proteins, users, or stations change; the analytical approach remains similar.

Further Reading

- *Analysis Recipes & Workflows* — More ready-to-use recipes
- *Multilayer Networks 101* — Conceptual foundations
- community_detection — Detailed community detection guide
- statistics — Complete statistics reference
- *Algorithm Landscape* — Algorithm selection guide

Academic References:

- Kivelä et al. (2014). "Multilayer networks." *Journal of Complex Networks*.
- De Domenico et al. (2015). "Ranking in interconnected multilayer networks." *Nature Communications*.
- Battiston et al. (2017). "The physics of higher-order interactions in complex systems." *Nature Physics*.

3.7 Part VII: Project Info

Project information, contributing guidelines, and citations.

3.7.1 Project Info

Information about the py3plex project, including how to contribute, cite, and stay up-to-date with development.

This section covers:

- *Changelog* — Release history and version notes
- *Roadmap & Vision* — Future development plans
- *Contributing to py3plex* — How to contribute to py3plex
- *Benchmarking & Performance* — Performance characteristics and optimization
- *Citation and References* — How to cite py3plex in publications

Quick Links:

- **GitHub Repository:** https://github.com/SkBlaz/py3plex
- **Issue Tracker:** https://github.com/SkBlaz/py3plex/issues
- **PyPI Package:** https://pypi.org/project/py3plex/
- **Documentation:** https://py3plex.readthedocs.io/

Getting Involved

Report Issues:

Found a bug? Have a feature request? Open an issue on GitHub.

Contribute Code:

See *Contributing to py3plex* for guidelines on submitting pull requests.

Join Discussions:

Participate in GitHub Discussions to ask questions and share ideas.

Stay Updated:

- Watch the repository for release announcements
- Follow the *Changelog* for recent changes
- Check the *Roadmap & Vision* for planned features

License

py3plex is released under the MIT License. See LICENSE file in the repository.

Acknowledgements

See *Citation and References* for acknowledgements and how to cite py3plex.

3.7.2 Changelog

This page documents the release history of py3plex. For the most up-to-date information, see GitHub Releases[11].

Release History

Version 1.0.0 (Current)

Release Date: 2024

Major Features:

- **DSL Query Language:** SQL-like domain-specific language for querying multilayer networks
  - String syntax (`execute_query`) for simple queries
  - Builder API (`Q.nodes()`, `Q.edges()`) for complex, chainable queries
  - Layer algebra (`L[layer]`, `L.intersection()`, `L.union()`)
  - Comprehensive filter, compute, and aggregation operations
- **First-Class Uncertainty Quantification:**
  - `StatSeries` and `StatMatrix` types for statistics with uncertainty
  - Resampling methods (SEED, PERTURBATION) for error propagation
  - Backward-compatible with existing code expecting scalars
  - Integrated across centrality, community detection, and dynamics modules
- **Temporal Network Support:**
  - Time-stamped edges and temporal queries
  - Snapshot-based analysis
  - Temporal dynamics simulation
- **Enhanced Dynamics Framework:**
  - Refactored dynamics API with consistent interface
  - SIR, SIS, and custom dynamics models

[11] https://github.com/SkBlaz/py3plex/releases

  - Layer-specific and time-varying parameters
  - Comprehensive result objects with measure extraction
- **Community Detection Improvements:**
  - Multilayer Louvain with inter-layer coupling
  - Infomap integration (with external binary)
  - Layer-specific community detection workflows
  - Community comparison and stability analysis
- **Web GUI:**
  - Interactive network exploration (FastAPI + SvelteKit)
  - Real-time visualization and query interface
  - REST API for programmatic access
- **CLI Tools:**
  - `py3plex` command-line interface
  - Community detection, statistics, DSL linting
  - Network conversion and export utilities
- **Documentation Overhaul:**
  - Comprehensive how-to guides
  - Conceptual documentation
  - API reference with examples
  - Tutorial notebooks
- **Testing & Quality:**
  - 500+ unit and integration tests
  - Continuous integration (CI/CD)
  - Type hints throughout codebase
  - Code coverage tracking

API Changes:

- Standardized function signatures across modules
- Consistent parameter naming (`gamma` for resolution, `omega` for coupling)
- Deprecated legacy APIs (warnings emitted, will be removed in 2.0)

Performance Improvements:

- Optimized tensor operations
- Sparse matrix support for large networks
- Caching for expensive computations

Bug Fixes:

- Numerous edge case fixes in community detection
- Corrected centrality calculations for directed multilayer networks
- Fixed memory leaks in dynamics simulations

- Resolved NetworkX compatibility issues

Previous Versions (0.x Series)

Version 0.30 - 0.35 (2020-2023)

- Incremental feature additions
- Experimental DSL prototype
- Initial dynamics framework
- Community detection algorithms

Version 0.20 - 0.29 (2018-2020)

- Multilayer centrality measures
- Visualization improvements
- NetworkX integration

Version 0.10 - 0.19 (2016-2018)

- Initial public release
- Core multilayer network data structures
- Basic algorithms (degree, clustering)
- Example notebooks

Version 0.1 - 0.9 (2014-2016)

- Research prototype
- Proof-of-concept implementations
- Academic collaborations

How to Check Your Version

```
import py3plex
print(py3plex.__version__)
```

Expected output:

```
1.0.0
```

Upgrade Instructions

Upgrade to latest stable release:

```
pip install --upgrade py3plex
```

Install from specific version:

```
pip install py3plex==1.0.0
```

Install development version:

```
pip install git+https://github.com/SkBlaz/py3plex.git@main
```

Migration Guides

Migrating from 0.x to 1.0

Breaking Changes:

1. **Network construction API:**
   - Old: `network.add_nodes(node_list)`
   - New: `network.add_nodes([('node', 'layer'), ...])` (explicit tuples)
2. **Community detection returns:**
   - Old: `louvain_communities()` returned dict with node IDs as keys
   - New: Returns dict with `(node, layer)` tuples as keys
3. **Centrality functions:**
   - Old: Scalar returns
   - New: Can return `StatSeries` when `uncertainty=True`
   - Backward compatible: Use `result.mean` or `np.array(result)`

Deprecated APIs (still work but emit warnings):

- `network.get_nodes_by_layer(layer)` → Use DSL: `Q.nodes().from_layers(L[layer])`
- `network.get_layer_edges(layer)` → Use DSL: `Q.edges().from_layers(L[layer])`
- `compute_centrality(network, measure)` → Use DSL: `Q.nodes().compute(measure)`

New Recommended Patterns:

```
# Old way (still works)
nodes = network.get_nodes_by_layer('layer1')
for node in nodes:
    degree = network.core_network.degree(node)

# New DSL way (recommended)
from py3plex.dsl import Q, L
result = (
    Q.nodes()
     .from_layers(L['layer1'])
     .compute("degree")
     .execute(network)
)
```

Migration Steps:

1. Update py3plex: `pip install --upgrade py3plex`
2. Run your existing code - warnings will indicate deprecated usage
3. Gradually migrate to DSL for queries
4. Update community detection code to expect tuple keys

5. Handle uncertainty in centrality results if using `uncertainty=True`

API Changes and Deprecations

Removed in 1.0:

- `old_louvain_function()` - replaced by `louvain_communities()`
- `legacy_centrality_wrapper()` - use centrality functions directly

Deprecated (warnings only, will be removed in 2.0):

- Direct core_network manipulation - use DSL or network methods
- Legacy node/edge addition formats - use dict or tuple formats

Renamed:

- `multinet.multi_layer_network()` - unchanged (primary API)
- `execute_dsl_query()` → `execute_query()` (alias still works)

Contributing to Changelog

When submitting PRs, please update the "Unreleased" section in CHANGELOG.md in the repository root with:

- Brief description of changes
- Link to issue (if applicable)
- Breaking changes clearly marked
- Migration notes for API changes

See *Contributing to py3plex* for details.

Staying Updated

- **GitHub Releases:** https://github.com/SkBlaz/py3plex/releases
- **PyPI:** https://pypi.org/project/py3plex/
- **Discussions:** https://github.com/SkBlaz/py3plex/discussions

Next Steps

- **Upgrade guide:** See "Migration Guides" above
- **What's new:** *Roadmap & Vision* for planned features
- **Report issues:** https://github.com/SkBlaz/py3plex/issues

3.7.3 Roadmap & Vision

This page outlines the vision and planned development for py3plex. For the latest updates, see GitHub Releases[12] and GitHub Issues[13].

[12] https://github.com/SkBlaz/py3plex/releases
[13] https://github.com/SkBlaz/py3plex/issues

Long-Term Vision

py3plex aims to be the go-to Python library for multilayer network analysis with a focus on:

Research Goals:

- Implementing state-of-the-art multilayer network algorithms from research literature
- Providing a flexible framework for developing new multilayer methods
- Supporting reproducible research with comprehensive documentation and examples

Production Use Cases:

- Enabling analysis of real-world multilayer networks at scale
- Providing robust APIs for integration into production systems
- Supporting diverse data formats and interoperability with existing tools

Community Building:

- Growing an active community of researchers and practitioners
- Facilitating collaboration through open-source development
- Providing educational resources and workshops

Current Focus (v1.0+)

Established Features:

- Core multilayer network data structures and operations
- DSL (Domain-Specific Language) for declarative network queries
- Community detection algorithms (Louvain, Infomap, multilayer variants)
- Dynamics simulation (SIR, SIS, random walks, custom models)
- Network embeddings (Node2Vec, DeepWalk)
- Centrality measures (degree, betweenness, PageRank, multilayer variants)
- First-class uncertainty quantification
- Visualization tools (matplotlib, plotly, interactive)
- CLI tools for common operations
- Web GUI for interactive exploration

Planned Enhancements

Priority areas for future development:

Near Term (Next Release)

- Performance optimizations for large-scale networks (>1M nodes)
- Additional temporal network analysis tools
- Extended DSL capabilities (aggregation, window functions)
- More community detection algorithms
- Enhanced visualization options (layer-specific layouts)

Medium Term (1-2 Releases)

- GPU acceleration for select algorithms
- Streaming network analysis capabilities
- Advanced temporal dynamics models
- Integration with graph database systems
- Additional export formats (Neo4j, GraphML extensions)
- More comprehensive benchmarking suite

Longer Term (Future Releases)

- Distributed computing support (Dask, Spark integration)
- Advanced machine learning on multilayer networks
- Causal inference tools for multilayer systems
- Time-varying parameter estimation
- Additional domain-specific toolkits (bioinformatics, social, transport)

Note: Feature priorities may shift based on community feedback and research developments. Track specific feature requests and vote on priorities through GitHub Issues[14].

Research Directions

py3plex development is informed by ongoing research in multilayer network science:

Current Research Areas:

- **Multilayer centrality:** Novel measures that account for layer dependencies
- **Temporal dynamics:** Time-varying multilayer processes and intervention modeling
- **Uncertainty quantification:** Principled handling of measurement error and structural uncertainty
- **Scalability:** Algorithms that scale to networks with millions of nodes and layers
- **Domain applications:** Collaborations in biology, social science, transportation, finance

Contributing Research:

If you're developing new multilayer network methods, we welcome contributions! See *Contributing to py3plex* for guidelines on:

- Implementing new algorithms
- Adding test cases and benchmarks
- Contributing examples and documentation
- Citing your research in the codebase

[14] https://github.com/SkBlaz/py3plex/issues

Community Requests

The following features have been requested by the community. For the full list and to vote on priorities, see GitHub Issues[15].

High-Priority Requests:

- Support for very large networks (>10M nodes) - **In progress**
- Additional temporal network analysis tools - **Planned**
- More extensive documentation and tutorials - **Ongoing**
- Enhanced performance for centrality computations - **Planned**
- Integration with popular graph databases - **Under consideration**

Vote on Features

Help prioritize development by:

- Commenting on existing feature request issues with your use case
- Opening new issues for feature requests (use the "enhancement" label)
- Using reactions to vote on priorities
- Contributing implementations for features you need

Contributing to Development

Want to help implement these features? We welcome contributions of all kinds:

- **Code contributions:** Implement new algorithms, fix bugs, improve performance
- **Documentation:** Write tutorials, improve API docs, add examples
- **Testing:** Add test cases, report bugs, validate algorithms
- **Community:** Answer questions, review PRs, organize workshops

See *Contributing to py3plex* for detailed guidelines.

Release Cycle

py3plex follows semantic versioning (MAJOR.MINOR.PATCH):

Version Scheme:

- **Major releases** (x.0.0): Breaking API changes, major new features
- **Minor releases** (1.x.0): New features, backward-compatible changes
- **Patch releases** (1.0.x): Bug fixes, documentation updates

Development Process:

- **Main branch:** Stable, production-ready code
- **Development branches:** Feature development and testing
- **Release candidates:** Beta testing before major releases
- **Documentation:** Updated with each release

Release Timeline:

[15] https://github.com/SkBlaz/py3plex/issues

- Major releases: ~1-2 per year
- Minor releases: As features are completed (typically quarterly)
- Patch releases: As needed for bug fixes

See *Changelog* for release history.

Staying Updated

- **GitHub Releases:** https://github.com/SkBlaz/py3plex/releases
- **Issue Tracker:** https://github.com/SkBlaz/py3plex/issues
- **Discussions:** https://github.com/SkBlaz/py3plex/discussions

Next Steps

- **Current features:** *10-Minute Tutorial*
- **Contribute:** *Contributing to py3plex*
- **Report issues:** https://github.com/SkBlaz/py3plex/issues

3.7.4 Contributing to py3plex

We welcome contributions to py3plex. This guide explains how to contribute effectively.

Ways to Contribute

You can contribute in many ways:

- Report bugs - Open issues for bugs you encounter
- Suggest features - Propose new features or improvements
- Write documentation - Improve or expand documentation
- Fix bugs - Submit pull requests fixing issues
- Add features - Implement new algorithms or capabilities
- Write tests - Improve test coverage
- Review PRs - Help review pull requests from others

Getting Started

Fork and Clone

1. Fork the repository on GitHub
2. Clone your fork:

```
git clone https://github.com/YOUR_USERNAME/py3plex.git
cd py3plex
```

3. Add upstream remote:

```
git remote add upstream https://github.com/SkBlaz/py3plex.git
```

Development Setup

Recommended (Makefile-based):

Use the Makefile for a streamlined setup:

```
# Create virtual environment and install dependencies
make setup

# Install package in editable mode with dev dependencies
make dev-install
```

Alternative (manual venv + pip):

If you prefer manual setup:

```
# Create virtual environment
python3 -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate

# Install in editable mode with dev dependencies
pip install -e ".[dev]"
```

Both methods install:

- Core dependencies
- Testing tools (pytest, coverage)
- Linting tools (black, ruff, isort, mypy)
- Documentation tools (sphinx)

Development Workflow

Create a Branch

Always create a new branch for your work:

```
git checkout -b feature/my-new-feature
# or
git checkout -b fix/issue-123
```

Make Changes

1. Write your code following our coding standards (see below)
2. Add or update tests
3. Update documentation
4. Run linters and tests

Run Tests

```
# Run all tests
make test

# Or directly with pytest
python run_tests.py
```

Run Linters

```
# Run all linters
make lint

# Or individual tools
make format  # Auto-format code (black, isort)
ruff check .
mypy py3plex
```

Commit Changes

Write clear, descriptive commit messages:

```
git add .
git commit -m "Add feature X to support Y

- Implement algorithm for Z
- Add tests for edge cases
- Update documentation"
```

Push and Create PR

```
git push origin feature/my-new-feature
```

Then create a Pull Request on GitHub with:

- Clear title describing the change
- Description of what changed and why
- Reference to related issues (e.g., "Fixes #123")
- Screenshots for UI changes

Coding Standards

Code Style

We follow PEP 8 with these specifics:

- Line length: 100 characters (not 80)
- Indentation: 4 spaces (no tabs)
- String quotes: Single quotes preferred ('text' not "text")

- Imports: Grouped and sorted (using isort)

Auto-format your code:

```
make format
```

This runs:

- `isort` - Sort and organize imports
- `black` - Format code consistently
- `ruff --fix` - Auto-fix linting issues

Naming Conventions

- Functions/variables: `snake_case`
- Classes: `PascalCase`
- Constants: `UPPER_SNAKE_CASE`
- Private members: `_leading_underscore`

```python
# Good
def compute_centrality(network, node_id):
    MAX_ITERATIONS = 100
    centrality_scores = {}
    return centrality_scores

class MultilayerNetwork:
    def __init__(self):
        self._internal_state = {}
```

Docstrings

Use NumPy-style docstrings for all public functions and classes:

```python
def multilayer_degree_centrality(network, layer=None, weighted=False):
    """
    Compute degree centrality for multilayer network.

    Parameters
    ----------
    network : multi_layer_network
        The multilayer network to analyze.
    layer : str, optional
        Specific layer to analyze. If None, aggregates across all layers.
    weighted : bool, default=False
        If True, compute strength (weighted degree) instead of degree.

    Returns
    -------
    dict
        Dictionary mapping node names to centrality scores.
```

(continues on next page)

(continued from previous page)

```
    Examples
    --------
    >>> network = multinet.multi_layer_network()
    >>> network.add_edges([['A', 'L1', 'B', 'L1', 1]])
    >>> centrality = multilayer_degree_centrality(network)
    >>> centrality['A---L1']
    1.0

    See Also
    --------
    multilayer_betweenness_centrality : Betweenness centrality

    Notes
    -----
    For directed networks, this computes out-degree by default.

    References
    ----------
    .. [1] De Domenico, M., et al. (2013). Mathematical formulation of
           multilayer networks. Physical Review X, 3(4), 041022.
    """
    pass
```

Type Hints

Add type hints to improve code clarity:

```
from typing import Dict, List, Optional
from py3plex.core.multinet import multi_layer_network

def compute_statistics(
    network: multi_layer_network,
    layers: Optional[List[str]] = None
) -> Dict[str, float]:
    """Compute network statistics."""
    pass
```

Testing Guidelines

Test Requirements

All new code should include tests:

- Unit tests for individual functions
- Integration tests for workflows
- Edge cases for boundary conditions
- Documentation tests for examples in docstrings

Writing Tests

Use pytest for testing:

```python
# tests/test_my_feature.py
import pytest
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls

def test_layer_density():
    """Test layer density calculation."""
    # Setup
    network = multinet.multi_layer_network()
    network.add_edges([
        ['A', 'L1', 'B', 'L1', 1],
        ['B', 'L1', 'C', 'L1', 1],
    ], input_type='list')

    # Execute
    density = mls.layer_density(network, 'L1')

    # Assert
    assert 0 <= density <= 1
    assert density == pytest.approx(0.333, abs=0.01)

def test_invalid_layer():
    """Test error handling for invalid layer."""
    network = multinet.multi_layer_network()

    with pytest.raises(ValueError):
        mls.layer_density(network, 'nonexistent')
```

Test Coverage

Aim for high test coverage:

```
# Run tests with coverage
pytest --cov=py3plex --cov-report=html

# View coverage report
open htmlcov/index.html  # macOS
# or
xdg-open htmlcov/index.html  # Linux
```

Target: >80% coverage for new code

Documentation

Update Documentation

When adding features, update:

1. **Docstrings** in the code

2. **RST files** in `docfiles/` if adding major features
3. **Examples** in `examples/` directory
4. **README** if changing installation or basic usage

Build Documentation

```
cd docfiles
make html

# View documentation
open _build/html/index.html
```

Documentation Style

- Clear and concise - Use simple language
- Code examples - Include working examples
- Cross-references - Link to related documentation
- Visual aids - Add diagrams where helpful

Pull Request Guidelines

Before Submitting

[OK] Code follows style guide (`make lint` passes)

[OK] All tests pass (`make test` passes)

[OK] New code has tests

[OK] Documentation is updated

[OK] Commit messages are clear

[OK] Branch is up to date with main

PR Description

Include in your PR description:

- What changed
- Why the change is needed
- How you implemented it
- Testing done
- Screenshots for visual changes
- Breaking changes if any

Example PR template:

```
## Description

Implements multilayer PageRank centrality following [citation].
```

(continues on next page)

(continued from previous page)

```
## Motivation

Closes #123 - needed for analyzing directed multilayer networks.

## Changes

- Add `multilayer_pagerank()` function
- Add tests with 90% coverage
- Add example in `example_centrality.py`
- Update documentation in `centrality.rst`

## Testing

- [x] Unit tests pass
- [x] Integration test on real network
- [x] Tested on Python 3.8, 3.10, 3.12
- [x] Linting passes

## Breaking Changes

None.
```

Review Process

1. Maintainers will review your PR
2. Address any feedback or requested changes
3. Once approved, your PR will be merged
4. Your contribution will be in the next release!

Reporting Issues

Bug Reports

When reporting bugs, include:

- Description of the bug
- Steps to reproduce
- Expected behavior
- Actual behavior
- Python version (`python --version`)
- py3plex version
- Operating system
- Minimal code example

Example:

````
## Bug Description

`layer_density()` returns negative values for certain graphs.

## Steps to Reproduce

```python
network = multinet.multi_layer_network()
network.add_edges([['A', 'L1', 'B', 'L1', -1]])
density = mls.layer_density(network, 'L1')
print(density)  # -0.5
```

## Expected

Density should be between 0 and 1, or raise ValueError for negative weights.

## Actual

Returns -0.5

## Environment

- Python 3.10.5
- py3plex 0.95a
- Ubuntu 22.04
````

Feature Requests

For feature requests, describe:

- Use case - What problem does it solve?
- Proposed solution - How should it work?
- Alternatives - Other approaches considered
- Additional context - Examples, papers, etc.

Code of Conduct

- Be respectful and inclusive
- Welcome newcomers
- Focus on what is best for the community
- Show empathy towards other community members

Contributors are expected to follow these principles to foster a welcoming and productive community.

License

By contributing, you agree that your contributions will be licensed under the MIT License.

Recognition

Contributors are recognized in:

- GitHub contributors page
- Release notes
- acknowledgements section in the documentation

Getting Help

- Questions: Open a GitHub Discussion
- Chat: Join our community chat (link in README)
- Email: Contact maintainers at blaz.skrlj@ijs.si

Next Steps

- Read development for development workflow details
- See architecture for system architecture
- Browse existing issues[16]
- Check good first issues[17]

Thank you for contributing to py3plex!

3.7.5 Benchmarking & Performance

Performance characteristics and optimization strategies for py3plex.

Network Scale Guidelines

py3plex is optimized for research-scale networks:

Table 3: Network Scale Performance

Network Size	Performance	Visualization	Recommendations
Small (<100 nodes)	Excellent	Fast, detailed	Use dense visualization mode
Medium (100-1k nodes)	Good	Fast, balanced	Default settings work well
Large (1k-10k nodes)	Good	Slower, minimal	Use sparse matrices, sampling
Very Large (>10k nodes)	Variable	Very slow	Sampling required

[16] https://github.com/SkBlaz/py3plex/issues
[17] https://github.com/SkBlaz/py3plex/issues?q=is%3Aissue+is%3Aopen+label%3A%22good+first+issue%22

Performance Tips

Use Sparse Matrices

For large networks, use sparse matrix representations:

```
from py3plex.core import multinet

network = multinet.multi_layer_network(sparse=True)
```

This reduces memory usage by 10-100x for typical networks.

Batch Operations

Process multiple operations together:

```
from py3plex.dsl import Q

# Compute multiple metrics at once
result = (
    Q.nodes()
     .compute("degree", "betweenness_centrality", "clustering")
     .execute(network)
)
```

Avoid repeated single-metric computations.

Use Arrow/Parquet for I/O

For large datasets:

```
import pyarrow.parquet as pq

# Save
table = pq.write_table(edges_table, 'network.parquet')

# Load (much faster than CSV)
table = pq.read_table('network.parquet')
```

Parallel Processing

For Node2Vec and other CPU-intensive algorithms:

```
from py3plex.wrappers import train_node2vec

embeddings = train_node2vec(
    network,
    workers=8  # Use multiple CPU cores
)
```

Benchmark Results

Performance benchmarks for common operations on synthetic multilayer networks. These results provide guidance for planning analyses and optimizing workflows.

Test Environment:

- CPU: Intel Core i7-9700K @ 3.6GHz (8 cores)
- RAM: 32 GB DDR4
- Python: 3.10
- py3plex: v1.0.0

Algorithm Runtimes vs. Network Size

Community Detection (Louvain)

Table 4: Louvain Algorithm Runtime

Nodes	Edges	Layers	Runtime
100	500	3	0.05s
1,000	5,000	3	0.3s
10,000	50,000	3	4.2s
100,000	500,000	3	58s

Centrality Computation (Betweenness)

Table 5: Betweenness Centrality Runtime

Nodes	Edges	Layers	Runtime
100	500	3	0.12s
1,000	5,000	3	8.5s
10,000	50,000	3	1,240s (21 min)
100,000	500,000	3	N/A (too slow)

Note: Betweenness is $O(n^3)$ - use approximation methods for large networks

Node2Vec Embeddings

Table 6: Node2Vec Runtime (128-dim, 10 walks/node)

Nodes	Edges	Layers	Runtime
100	500	3	2.3s
1,000	5,000	3	18s
10,000	50,000	3	245s (4 min)
100,000	500,000	3	3,200s (53 min)

Dynamics Simulation (SIR)

Table 7: SIR Simulation Runtime (100 steps)

Nodes	Edges	Layers	Runtime
100	500	3	0.8s
1,000	5,000	3	4.5s
10,000	50,000	3	52s
100,000	500,000	3	680s (11 min)

Memory Usage Profiles

Table 8: Peak Memory Usage

Nodes	Edges	Dense Storage	Sparse Storage
100	500	2 MB	0.5 MB
1,000	5,000	24 MB	2 MB
10,000	50,000	2.4 GB	18 MB
100,000	500,000	240 GB	180 MB

Key Insight: Sparse storage reduces memory by 10-1000x for typical networks.

Comparison with Other Tools

Community Detection: py3plex vs. NetworkX

Table 9: Louvain Performance Comparison (1k nodes, 5k edges, single layer)

Tool	Runtime	Notes
py3plex	0.3s	Multilayer-aware
NetworkX + python-louvain	0.2s	Single-layer only
graph-tool	0.08s	C++ backend, faster

Verdict: py3plex is competitive for single-layer, adds multilayer capability others lack.

Node Embeddings: py3plex vs. node2vec

Table 10: Node2Vec Performance (1k nodes, 128-dim, 10 walks)

Tool	Runtime	Notes
py3plex	18s	Python wrapper
node2vec (original)	15s	C++ implementation
Gensim	12s	Optimized Word2Vec

Verdict: py3plex uses established libraries (gensim), performance is comparable.

Benchmarking Notes:

- Results vary based on network structure (density, clustering, layer coupling)
- Runtimes scale differently for different algorithms (linear, quadratic, cubic)
- Use these benchmarks as rough guidelines, not exact predictions
- For the most accurate estimates, run benchmarks on your specific hardware and data

Running Custom Benchmarks:

See the `benchmarks/` directory in the repository for scripts to reproduce these results or run your own benchmarks.

Running Benchmarks

Run benchmarks yourself:

```
cd benchmarks
python run_benchmarks.py
```

See the *benchmarks/* directory in the repository for benchmark scripts.

Profiling Your Code

Use Python profiling tools:

```
import cProfile
import pstats

# Profile your analysis
cProfile.run('your_analysis_function(network)', 'profile_stats')

# View results
stats = pstats.Stats('profile_stats')
stats.sort_stats('cumulative')
stats.print_stats(20)
```

Memory Profiling

```
pip install memory_profiler
python -m memory_profiler your_script.py
```

Optimization Strategies

For Large Networks

1. **Sample the network** for exploratory analysis
2. **Use layer-specific analysis** instead of full multilayer
3. **Compute metrics incrementally** rather than all at once
4. **Cache intermediate results**

For Repeated Analysis

1. **Precompute and save** expensive metrics
2. **Use config-driven workflows** for reproducibility
3. **Batch process** multiple networks

For Production

1. **Use Docker containers** for consistent environments
2. **Implement monitoring** for long-running jobs
3. **Add checkpointing** for crash recovery

See *Docker Usage Guide* for deployment best practices.

Hardware Recommendations

Minimum:

- 4 GB RAM
- 2 CPU cores
- Small networks (<1k nodes)

Recommended:

- 16 GB RAM
- 8 CPU cores
- Networks up to 10k nodes

High-Performance:

- 64+ GB RAM
- 16+ CPU cores
- Large networks (>10k nodes)

Next Steps

- **Optimize I/O:** *How to Export and Serialize Networks*
- **Deploy to production:** *Docker Usage Guide*
- **See deployment guide:** *Performance and Scalability Best Practices* (original full guide)

3.7.6 Citation and References

If you use py3plex in your research, please **cite our work**.

Primary Citation

Recommended citation for py3plex:

```
@Article{Skrlj2019,
  author    = {{\v{S}}krlj, Bla{\v{z}} and Kralj, Jan and Lavra{\v{c}}, Nada},
  title     = {Py3plex toolkit for visualization and analysis of multilayer␣
```

(continues on next page)

(continued from previous page)

```
↪networks},
  journal   = {Applied Network Science},
  year      = {2019},
  volume    = {4},
  number    = {1},
  pages     = {94},
  doi       = {10.1007/s41109-019-0203-7},
  url       = {https://doi.org/10.1007/s41109-019-0203-7}
}
```

Plain text:

Škrlj, B., Kralj, J., & Lavrač, N. (2019). Py3plex toolkit for visualization and analysis of multilayer networks. *Applied Network Science*, 4(1), 94. https://doi.org/10.1007/s41109-019-0203-7

Conference Paper

For the **initial algorithmic work**:

```
@InProceedings{Skrlj2019Conference,
  author    = {{\v{S}}krlj, Bla{\v{z}} and Kralj, Jan and Lavra{\v{c}}, Nada},
  editor    = {Aiello, Luca Maria and Cherifi, Chantal and Cherifi, Hocine
               and Lambiotte, Renaud and Li{\'o}, Pietro and Rocha, Luis M.},
  title     = {Py3plex: A Library for Scalable Multilayer Network Analysis␣
↪and Visualization},
  booktitle = {Complex Networks and Their Applications VII},
  year      = {2019},
  publisher = {Springer International Publishing},
  address   = {Cham},
  pages     = {757--768},
  isbn      = {978-3-030-05411-3},
  doi       = {10.1007/978-3-030-05411-3_60}
}
```

Algorithm-Specific Citations

When using **specific algorithms**, please also cite the **original papers**:

Multilayer Modularity

```
@Article{Mucha2010,
  author  = {Mucha, Peter J. and Richardson, Thomas and Macon, Kevin
             and Porter, Mason A. and Onnela, Jukka-Pekka},
  title   = {Community structure in time-dependent, multiscale, and multiplex␣
↪networks},
  journal = {Science},
  year    = {2010},
  volume  = {328},
  number  = {5980},
```

(continues on next page)

(continued from previous page)

```
  pages    = {876--878},
  doi      = {10.1126/science.1184819}
}
```

Node2Vec

```
@InProceedings{Grover2016,
  author    = {Grover, Aditya and Leskovec, Jure},
  title     = {node2vec: Scalable Feature Learning for Networks},
  booktitle = {Proceedings of the 22nd ACM SIGKDD International Conference
               on Knowledge Discovery and Data Mining},
  year      = {2016},
  pages     = {855--864},
  doi       = {10.1145/2939672.2939754}
}
```

Louvain Algorithm

```
@Article{Blondel2008,
  author  = {Blondel, Vincent D. and Guillaume, Jean-Loup and Lambiotte,␣
↪Renaud
             and Lefebvre, Etienne},
  title   = {Fast unfolding of communities in large networks},
  journal = {Journal of Statistical Mechanics: Theory and Experiment},
  year    = {2008},
  volume  = {2008},
  number  = {10},
  pages   = {P10008},
  doi     = {10.1088/1742-5468/2008/10/P10008}
}
```

Infomap

```
@Article{Rosvall2008,
  author  = {Rosvall, Martin and Bergstrom, Carl T.},
  title   = {Maps of random walks on complex networks reveal community␣
↪structure},
  journal = {Proceedings of the National Academy of Sciences},
  year    = {2008},
  volume  = {105},
  number  = {4},
  pages   = {1118--1123},
  doi     = {10.1073/pnas.0706851105}
}
```

MultiXRank

```
@Article{Baptista2022,
  author  = {Baptista, Anthony and Gonzalez, Aur{\'e}lien and Baudot, Ana{\"\
↪i}s},
  title   = {Universal multilayer network exploration by random walk with␣
↪restart},
  journal = {Communications Physics},
  year    = {2022},
  volume  = {5},
  number  = {1},
  pages   = {170},
  doi     = {10.1038/s42005-022-00937-9}
}
```

DeepWalk

```
@InProceedings{Perozzi2014,
  author    = {Perozzi, Bryan and Al-Rfou, Rami and Skiena, Steven},
  title     = {DeepWalk: Online Learning of Social Representations},
  booktitle = {Proceedings of the 20th ACM SIGKDD International Conference
               on Knowledge Discovery and Data Mining},
  year      = {2014},
  pages     = {701--710},
  doi       = {10.1145/2623330.2623732}
}
```

Multilayer Network Theory

Foundational papers on multilayer networks:

```
@Article{Kivela2014,
  author  = {Kivel{\"a}, Mikko and Arenas, Alex and Barthelemy, Marc
             and Gleeson, James P. and Moreno, Yamir and Porter, Mason A.},
  title   = {Multilayer networks},
  journal = {Journal of Complex Networks},
  year    = {2014},
  volume  = {2},
  number  = {3},
  pages   = {203--271},
  doi     = {10.1093/comnet/cnu016}
}
```

```
@Article{Boccaletti2014,
  author  = {Boccaletti, Stefano and Bianconi, Ginestra and Criado, Regino
             and del Genio, Charo I. and G{\'o}mez-Garde{\~n}es, Jes{\'u}s
             and Romance, Miguel and Sendi{\~n}a-Nadal, Irene and Wang, Zhen
             and Zanin, Massimiliano},
  title   = {The structure and dynamics of multilayer networks},
```

(continues on next page)

(continued from previous page)

```
  journal = {Physics Reports},
  year    = {2014},
  volume  = {544},
  number  = {1},
  pages   = {1--122},
  doi     = {10.1016/j.physrep.2014.07.001}
}
```

```
@Article{DeDomenico2013,
  author  = {De Domenico, Manlio and Sol{\'e}-Ribalta, Albert and Cozzo,␣
↪Emanuele
             and Kivel{\"a}, Mikko and Moreno, Yamir and Porter, Mason A.
             and G{\'o}mez, Sergio and Arenas, Alex},
  title   = {Mathematical formulation of multilayer networks},
  journal = {Physical Review X},
  year    = {2013},
  volume  = {3},
  number  = {4},
  pages   = {041022},
  doi     = {10.1103/PhysRevX.3.041022}
}
```

Complete Citation List

For algorithm citations, see the ../algorithm_guide which includes all major algorithms with their original references and DOIs.

Acknowledgments

Development and Contributors

py3plex is developed and maintained by:

- **Blaž Škrlj** (lead developer) - Jožef Stefan Institute, Ljubljana, Slovenia
- **Jan Kralj** (contributor) - Jožef Stefan Institute
- **Nada Lavrač** (contributor) - Jožef Stefan Institute

Funding and Support

This work has been **supported by**:

- **Jožef Stefan Institute**, Ljubljana, Slovenia
- **Slovenian Research Agency** (research program P2-0103)

External Libraries

py3plex builds upon **excellent open-source libraries**:

- **NetworkX** - Graph data structures and algorithms
- **NumPy** - Numerical computing

- **SciPy** - Scientific computing
- **Matplotlib** - Visualization
- **scikit-learn** - Machine learning utilities

Community Acknowledgments

We thank all contributors, users, and the broader network science community for their support, feedback, and contributions.

Special thanks to contributors who have submitted pull requests, reported issues, or provided feedback.

License Information

py3plex is released under the **MIT License**.

Note: Some bundled algorithms may have **different licenses**:

- Infomap community detection code: **AGPLv3**
- Louvain community detection: **BSD-3-Clause**

See `LICENSE` file in the repository for detailed license compatibility information.

Contact

For questions about citing py3plex or collaboration opportunities:

- **Email:** blaz.skrlj@ijs.si
- **GitHub:** https://github.com/SkBlaz/py3plex
- **Issues:** https://github.com/SkBlaz/py3plex/issues

Related Work

Other tools and libraries for multilayer network analysis:

- **muxViz** - Multilayer network visualization (R)
- **pymnet** - Multilayer networks in Python
- **multinet** - R package for multilayer networks
- **graphistry** - GPU-accelerated graph visualization

For a comprehensive comparison and positioning of py3plex, see the original publication.

Usage in Publications

If you've used py3plex in your research and would like to be listed here, please:

1. Open an issue or pull request
2. Provide citation to your paper
3. Brief description of how py3plex was used

We maintain a list of publications using py3plex to showcase applications and build community.

See Also

- acknowledgements - Full acknowledgments
- ../algorithm_guide - Complete algorithm citations with references
- py3plex GitHub[18] - Source code and issues
- Applied Network Science paper[19] - Primary publication

3.8 Additional Sections

3.8.1 Environments & Deployment

This section covers running py3plex from the command line, in Docker containers, and at scale.

This section covers:

- *Docker Usage Guide* — Command-line interface and containerized deployment
- *Performance and Scalability Best Practices* — Memory management, optimization, large networks

When to Use This Section

Read this section when you need to:

- Automate analyses that currently run interactively
- Process networks too large for a single Python session
- Share reproducible environments with collaborators
- Integrate py3plex into a production pipeline

[18] https://github.com/SkBlaz/py3plex
[19] https://doi.org/10.1007/s41109-019-0203-7

CLI provides a scriptable interface for common operations—no Python required.

Docker ensures identical results everywhere, eliminating environment issues.

Performance chapter covers memory management and optimization for large networks.

Start with *Docker Usage Guide* for automation, then *Performance and Scalability Best Practices* for optimization.

Tip

Deployment checklist:

- Version-pin all dependencies
- Add error handling for edge cases
- Verify results on test data
- Set up logging for debugging
- Test on the target environment

3.8.2 Docker Usage Guide

This guide provides comprehensive instructions for using Py3plex via Docker.

Table of Contents

Quickstart

The fastest way to get started with Py3plex using Docker:

```
# Clone the repository
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Build the Docker image
docker build -t py3plex:latest .

# Run a self-test to verify installation
docker run --rm py3plex:latest selftest

# Display help
```

(continues on next page)

(continued from previous page)

```
docker run --rm py3plex:latest help
```

Building the Image

Using Docker

```
# Build from the repository root
docker build -t py3plex:latest .

# Build with a specific tag
docker build -t py3plex:0.95a .

# Build without cache (if needed)
docker build --no-cache -t py3plex:latest .
```

Using Docker Compose

```
# Build using docker-compose
docker-compose build

# Force rebuild
docker-compose build --no-cache
```

Running Commands

Basic Commands

The Docker container is set up with `py3plex` as the entrypoint, so you can run any py3plex CLI command directly:

```
# Show version
docker run --rm py3plex:latest --version

# Run self-test
docker run --rm py3plex:latest selftest

# Show help
docker run --rm py3plex:latest help

# Show help for a specific command
docker run --rm py3plex:latest create --help
```

Using Docker Compose

With docker-compose, commands are slightly more verbose but easier to manage:

```
# Show version
docker-compose run --rm py3plex --version
```

(continues on next page)

(continued from previous page)

```
# Run self-test
docker-compose run --rm py3plex selftest -v

# Show help
docker-compose run --rm py3plex help
```

Working with Files

To work with network files, you need to mount a local directory to the container's `/data` directory.

Creating a Data Directory

```
# Create a local data directory
mkdir -p data
```

Mounting Volumes

With docker run:

```
# Mount current directory's data folder
docker run --rm -v $(pwd)/data:/data py3plex:latest create --nodes 100 --
↪layers 3 --output /data/network.edgelist

# On Windows (PowerShell)
docker run --rm -v ${PWD}/data:/data py3plex:latest create --nodes 100 --
↪layers 3 --output /data/network.edgelist

# On Windows (CMD)
docker run --rm -v %cd%/data:/data py3plex:latest create --nodes 100 --layers␣
↪3 --output /data/network.edgelist
```

With docker-compose:

The `docker-compose.yml` file already configures volume mounting from `./data` to `/data`, so you can simply:

```
docker-compose run --rm py3plex create --nodes 100 --layers 3 --output /data/
↪network.edgelist
```

Complete Workflow Example

```
# Create data directory
mkdir -p data

# 1. Create a multilayer network
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  create --nodes 100 --layers 3 --type random --probability 0.1 --output /
```

(continues on next page)

(continued from previous page)

```
↪data/network.edgelist

# 2. Load and display network information
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  load /data/network.edgelist --info

# 3. Compute statistics
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  stats /data/network.edgelist --measure all --output /data/stats.json

# 4. Detect communities
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  community /data/network.edgelist --algorithm louvain --output /data/
↪communities.json

# 5. Visualize the network
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  visualize /data/network.edgelist --output /data/network.png --layout␣
↪multilayer

# 6. View the results
ls -lh data/
```

Advanced Usage

Interactive Shell

To explore the container interactively:

```
# Start an interactive shell in the container
docker run --rm -it -v $(pwd)/data:/data --entrypoint /bin/bash py3plex:latest

# Inside the container, you can run:
# py3plex --version
# py3plex selftest
# python -c "import py3plex; print(py3plex.__version__)"
```

Using Python API

You can also use py3plex as a Python library inside the container:

```
# Create a Python script
cat > data/analyze.py << 'EOF'
from py3plex.core import multinet

# Create a network
network = multinet.multi_layer_network()
nodes = [{"source": f"node{i}", "type": "layer1"} for i in range(10)]
network.add_nodes(nodes, input_type="dict")
```

(continues on next page)

(continued from previous page)

```
print(f"Created network with {network.core_network.number_of_nodes()} nodes")
EOF

# Run the script in the container
docker run --rm -v $(pwd)/data:/data --entrypoint python py3plex:latest /data/
↪analyze.py
```

Custom Image Builds

If you need additional packages:

```
# Create a custom Dockerfile
FROM py3plex:latest

# Install additional packages
RUN pip install --no-cache-dir pandas jupyter

# Or system packages
USER root
RUN apt-get update && apt-get install -y vim && rm -rf /var/lib/apt/lists/*
USER py3plex
```

Docker Compose with Multiple Services

You can extend the `docker-compose.yml` to include related services:

```
version: '3.8'

services:
  py3plex:
    build: .
    image: py3plex:latest
    volumes:
      - ./data:/data

  jupyter:
    image: py3plex:latest
    ports:
      - "8888:8888"
    volumes:
      - ./data:/data
      - ./notebooks:/notebooks
    command: jupyter notebook --ip=0.0.0.0 --port=8888 --no-browser --allow-
↪root
    working_dir: /notebooks
```

Helper Scripts

The repository includes convenient helper scripts to simplify Docker usage.

py3plex-docker.sh

A wrapper script that simplifies running py3plex commands via Docker:

```
# The script automatically handles:
# - Docker installation checks
# - Image building (if needed)
# - Data directory creation
# - Volume mounting

# Usage examples:
./py3plex-docker.sh --version
./py3plex-docker.sh selftest
./py3plex-docker.sh create --nodes 100 --layers 3 --output /data/network.
↪edgelist
./py3plex-docker.sh load /data/network.edgelist --info
```

Features:

- Automatically checks if Docker is installed
- Prompts to build the image if it doesn't exist
- Creates the `data/` directory if needed
- Mounts the local `data/` directory to `/data` in the container
- Passes all arguments directly to py3plex

Script location: `py3plex-docker.sh` in the repository root

test-docker-setup.sh

A comprehensive test script to validate your Docker setup:

```
# Run the validation script
./test-docker-setup.sh
```

What it tests:

1. Docker installation verification
2. Dockerfile existence check
3. .dockerignore existence check
4. docker-compose.yml existence check
5. Docker image build process
6. Container version command
7. Container help command
8. Container selftest
9. Volume mounting and file creation

The script provides colored output ([OK] PASS / [X] FAIL) and a summary report.

Script location: `test-docker-setup.sh` in the repository root

Docker Configuration Files

.dockerignore

The `.dockerignore` file controls which files are excluded from the Docker build context:

Key exclusions:

- **Git and version control**: `.git/`, `.github/`, `.gitignore`
- **Python artifacts**: `__pycache__/`, `*.pyc`, `dist/`, `build/`
- **Virtual environments**: `venv/`, `env/`
- **Testing and coverage**: `.pytest_cache/`, `htmlcov/`, `coverage.*`
- **Documentation builds**: `docs/_build/`, `docfiles/`
- **Examples and datasets**: `examples/`, `datasets/`, `multilayer_datasets/`
- **Development files**: `tests/`, `run_tests.py`, `validate_*.py`

Why this matters:

- **Faster builds**: Smaller build context means faster Docker builds
- **Smaller images**: Excludes unnecessary files from the final image
- **Security**: Prevents accidental inclusion of sensitive files
- **Efficiency**: Only includes files needed to run py3plex

The `.dockerignore` file is located in the repository root.

Troubleshooting

Testing the Docker Setup

To verify your Docker setup is working correctly, use the included test script:

```
# Run the test script
./test-docker-setup.sh
```

This script will:

- Check Docker installation
- Verify all Docker-related files exist
- Build the Docker image
- Run basic py3plex commands (–version, help, selftest)
- Test volume mounting and file creation
- Clean up test artifacts

Permission Issues

If you encounter permission issues with mounted volumes:

```
# On Linux, you might need to run with your user ID
docker run --rm -v $(pwd)/data:/data --user $(id -u):$(id -g) py3plex:latest␣
↪create --output /data/network.edgelist
```

Container Not Found

If the image isn't found:

```
# List available images
docker images | grep py3plex

# Rebuild if necessary
docker build -t py3plex:latest .
```

Network Issues During Build

If you encounter network timeouts during build:

```
# Increase timeout
docker build --build-arg PIP_DEFAULT_TIMEOUT=300 -t py3plex:latest .

# Or use a different PyPI mirror
docker build --build-arg PIP_INDEX_URL=https://pypi.tuna.tsinghua.edu.cn/
↪simple -t py3plex:latest .
```

Out of Disk Space

Clean up unused Docker resources:

```
# Remove unused images
docker image prune -a

# Remove all stopped containers, unused networks, and dangling images
docker system prune -a
```

Debugging Build Issues

Build with verbose output:

```
docker build --progress=plain --no-cache -t py3plex:latest .
```

Best Practices

1. **Use Volume Mounts**: Always mount volumes for input/output files
2. **Use –rm Flag**: Remove containers after execution with `--rm` to save space
3. **Tag Images**: Use specific tags (e.g., `py3plex:0.95a`) for reproducibility

4. **Keep Data Separate**: Store network files in the mounted `data` directory
5. **Use Docker Compose**: For repeated operations, docker-compose simplifies commands
6. **Regular Updates**: Rebuild the image periodically to get latest py3plex updates

Practical Examples

Basic Network Analysis

Create and analyze a multilayer network:

```
# Create data directory
mkdir -p data

# Create a network
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  create --nodes 100 --layers 3 --type random --probability 0.05 \
  --output /data/network.edgelist

# Analyze the network
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  load /data/network.edgelist --info --stats

# Visualize
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  visualize /data/network.edgelist --output /data/network.png
```

Community Detection

```
# Detect communities using Louvain
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  community /data/network.edgelist --algorithm louvain \
  --output /data/communities.json

# View community statistics
cat data/communities.json | python -m json.tool | head -20
```

Centrality Analysis

```
# Compute degree centrality
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  centrality /data/network.edgelist --measure degree \
  --top 10 --output /data/centrality.json

# Compute betweenness centrality
docker run --rm -v $(pwd)/data:/data py3plex:latest \
  centrality /data/network.edgelist --measure betweenness \
  --top 10 --output /data/betweenness.json
```

Batch Processing

Process multiple networks (save as `batch-process.sh`):

```
#!/bin/bash
# Process multiple networks in batch
set -e

DATA_DIR=$(pwd)/data
mkdir -p $DATA_DIR

# Create several networks
for i in {1..5}; do
  echo "Creating network $i..."
  docker run --rm -v $DATA_DIR:/data py3plex:latest \
    create --nodes $((50 * i)) --layers 3 \
    --type random --probability 0.1 \
    --output /data/network_$i.edgelist
done

# Analyze each network
for i in {1..5}; do
  echo "Analyzing network $i..."
  docker run --rm -v $DATA_DIR:/data py3plex:latest \
    stats /data/network_$i.edgelist --measure all \
    --output /data/stats_$i.json
done

echo "Done! Results in $DATA_DIR"
```

Custom Analysis Script

Create a Python script (`analysis.py`) and run it in the container:

Python script:

```
# analysis.py
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Load network
network = multinet.multi_layer_network()
network.load_network("/data/network.edgelist", input_type="multiedgelist")

# Compute statistics
layers = ["layer1", "layer2", "layer3"]
for layer in layers:
    density = mls.layer_density(network, layer)
    print(f"{layer} density: {density:.4f}")

print("Analysis complete!")
```

Run it:

```
# Copy script to data directory
cp analysis.py data/

# Run in container
docker run --rm -v $(pwd)/data:/data \
  --entrypoint python py3plex:latest /data/analysis.py
```

Complete Analysis Pipeline

A comprehensive analysis pipeline:

```
#!/bin/bash
# complete-pipeline.sh
set -e

DATA_DIR=$(pwd)/data
mkdir -p $DATA_DIR

echo "Step 1: Creating network..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  create --nodes 200 --layers 3 --type ba --probability 0.05 \
  --output /data/network.edgelist

echo "Step 2: Computing statistics..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  stats /data/network.edgelist --measure all \
  --output /data/stats.json

echo "Step 3: Detecting communities..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  community /data/network.edgelist --algorithm louvain \
  --output /data/communities.json

echo "Step 4: Computing centrality..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  centrality /data/network.edgelist --measure pagerank \
  --top 20 --output /data/centrality.json

echo "Step 5: Visualizing..."
docker run --rm -v $DATA_DIR:/data py3plex:latest \
  visualize /data/network.edgelist --output /data/network.png \
  --layout multilayer --width 16 --height 12

echo "Pipeline complete! Check $DATA_DIR for results."
```

Comparative Network Analysis

Compare different network types:

```bash
#!/bin/bash
# compare-networks.sh
set -e

DATA_DIR=$(pwd)/data
mkdir -p $DATA_DIR

TYPES=("random" "er" "ba" "ws")

for type in "${TYPES[@]}"; do
  echo "Creating $type network..."
  docker run --rm -v $DATA_DIR:/data py3plex:latest \
    create --nodes 100 --layers 3 --type $type --probability 0.1 \
    --output /data/network_${type}.edgelist

  echo "Analyzing $type network..."
  docker run --rm -v $DATA_DIR:/data py3plex:latest \
    stats /data/network_${type}.edgelist --measure all \
    --output /data/stats_${type}.json
done

echo "Comparison complete!"
```

CI/CD Integration

GitHub Actions

Example workflow for network analysis:

```yaml
# .github/workflows/network-analysis.yml
name: Network Analysis with Docker

on: [push, pull_request]

jobs:
  analyze:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Build Docker image
        run: docker build -t py3plex:latest .

      - name: Create test network
        run: |
          mkdir -p data
          docker run --rm -v $PWD/data:/data py3plex:latest \
```

(continues on next page)

(continued from previous page)

```
          create --nodes 50 --layers 2 --output /data/test.edgelist

    - name: Run analysis
      run: |
        docker run --rm -v $PWD/data:/data py3plex:latest \
          stats /data/test.edgelist --measure all --output /data/stats.json

    - name: Upload results
      uses: actions/upload-artifact@v3
      with:
        name: analysis-results
        path: data/
```

GitLab CI

Example GitLab CI configuration:

```
# .gitlab-ci.yml
image: docker:latest

services:
  - docker:dind

variables:
  DOCKER_DRIVER: overlay2

stages:
  - build
  - analyze

build:
  stage: build
  script:
    - docker build -t py3plex:latest .
    - docker save py3plex:latest > py3plex-image.tar
  artifacts:
    paths:
      - py3plex-image.tar

analyze:
  stage: analyze
  script:
    - docker load < py3plex-image.tar
    - mkdir -p data
    - docker run --rm -v $PWD/data:/data py3plex:latest create --nodes 100 --
↪layers 3 --output /data/network.edgelist
    - docker run --rm -v $PWD/data:/data py3plex:latest stats /data/network.
↪edgelist --measure all --output /data/stats.json
  artifacts:
```

(continues on next page)

(continued from previous page)

```
  paths:
    - data/
```

Additional Resources

- Py3plex Documentation[20]
- CLI Tutorial
- Docker Documentation[21]
- Docker Compose Documentation[22]
- DOCKER.md[23] - Main Docker guide in the repository

Support

For issues related to:

- **Py3plex functionality**: Open an issue at https://github.com/SkBlaz/py3plex/issues
- **Docker setup**: Check this guide first, then open an issue if problems persist
- **General questions**: See the main README.md and documentation

3.8.3 Performance and Scalability Best Practices

This guide provides recommendations for optimizing Py3plex performance and handling large multilayer networks.

Table of Contents

[20] https://skblaz.github.io/py3plex/
[21] https://docs.docker.com/
[22] https://docs.docker.com/compose/
[23] https://github.com/SkBlaz/py3plex/blob/master/DOCKER.md

Network Scale Guidelines

Py3plex is optimized for research-scale networks. This table shows expected performance characteristics:

Table 11: Network Scale Performance

Network Size	Performance	Visualization	Recommendations
Small (<100 nodes)	Excellent	Fast, detailed	Use dense visualization mode
Medium (100-1k nodes)	Good	Fast, balanced	Default settings work well
Large (1k-10k nodes)	Good	Slower, minimal	Use sparse matrices, sampling
Very Large (>10k nodes)	Variable	Very slow	Sampling required, use NetworkX/igraph

Sparse Matrix Backend

Why Sparse Matrices?

Most real-world networks are **sparse** (few edges compared to possible edges). Sparse matrices:

- **Reduce memory usage** by 10-100x for typical networks
- **Speed up** matrix operations (multiplication, inversion)
- **Enable** analysis of larger networks

Example:

```
import numpy as np
from scipy.sparse import csr_matrix

# Dense representation (10k × 10k network)
# Memory: 10,000^2 × 8 bytes = 800 MB

# Sparse representation (with 1% density)
# Memory: ~8 MB (100x reduction!)
```

Automatic Sparse Matrix Usage

Py3plex **automatically uses sparse matrices** for:

- Supra-adjacency matrix operations
- Large network storage (>1000 nodes)
- Matrix-based algorithms (PageRank, spectral methods)

Verify sparse usage:

```
from py3plex.core import multinet

network = multinet.multi_layer_network()
network.load_network("large_network.csv", input_type="multiedgelist")

# Get sparse adjacency matrix
adj_sparse = network.get_sparse_adjacency_matrix()

print(f"Matrix size: {adj_sparse.shape}")
print(f"Non-zero entries: {adj_sparse.nnz}")
print(f"Sparsity: {1 - adj_sparse.nnz / (adj_sparse.shape[0]**2):.2%}")
```

Force Sparse Operations

For custom algorithms, explicitly use sparse operations:

```
from scipy.sparse import csr_matrix, lil_matrix
import numpy as np

# Create sparse adjacency matrix
adj = lil_matrix((n_nodes, n_nodes))
```

(continues on next page)

(continued from previous page)

```python
for u, v in network.core_network.edges():
    i, j = node_to_idx[u], node_to_idx[v]
    adj[i, j] = 1
    adj[j, i] = 1  # Undirected

# Convert to efficient format for operations
adj_csr = adj.tocsr()

# Sparse matrix operations are MUCH faster
result = adj_csr @ adj_csr  # Matrix multiplication
eigvals = scipy.sparse.linalg.eigs(adj_csr, k=10)  # Top eigenvalues
```

Network Sampling

When to Sample

Sampling is necessary when:

- Network is too large to visualize (>5k nodes)
- Algorithms take too long (>10 minutes)
- Memory usage is excessive (>8GB RAM)
- You need quick exploratory analysis

Random Node Sampling

```python
import random
from py3plex.core import multinet

# Load full network
network = multinet.multi_layer_network()
network.load_network("large_network.csv", input_type="multiedgelist")

# Sample 1000 random nodes
all_nodes = list(network.get_nodes())
sample_nodes = random.sample(all_nodes, min(1000, len(all_nodes)))

# Create subnetwork
subnetwork = network.get_subnetwork(sample_nodes)

print(f"Original: {len(all_nodes)} nodes")
print(f"Sample: {len(sample_nodes)} nodes")
print(f"Sampling ratio: {len(sample_nodes)/len(all_nodes):.1%}")
```

Stratified Sampling (Preserve Layer Distribution)

```
# Sample proportionally from each layer
layers = network.get_layer_names()
sample_nodes_per_layer = {}

for layer in layers:
    layer_nodes = [n for n in network.get_nodes() if n[1] == layer]
    sample_size = len(layer_nodes) // 10  # 10% sample
    sample_nodes_per_layer[layer] = random.sample(layer_nodes, sample_size)

# Combine samples
all_samples = [node for nodes in sample_nodes_per_layer.values() for node in␣
↪nodes]
subnetwork = network.get_subnetwork(all_samples)
```

Hub-Based Sampling (Keep Important Nodes)

```
import networkx as nx

# Sample high-degree nodes (hubs)
degrees = dict(network.core_network.degree())

# Sort by degree and take top 1000
sorted_nodes = sorted(degrees.items(), key=lambda x: x[1], reverse=True)
hub_nodes = [node for node, deg in sorted_nodes[:1000]]

subnetwork = network.get_subnetwork(hub_nodes)
print(f"Sampled top 1000 hubs")
```

Algorithm Optimization

Choose Efficient Algorithms

Some algorithms scale better than others:

Table 12: Algorithm Complexity

Algorithm	Complexity	Recommendations
Degree centrality	O(n + m)	Fast, use freely
Betweenness centrality	O(nm)	Slow for large networks, sample first
PageRank	O(iterations × m)	Fast if sparse, limit iterations
Community detection (Louvain)	O(m log n)	Fast, recommended
Shortest paths (all pairs)	$O(n^2m)$	Very slow, use sampling or approximate
Force-directed layout	$O(n^2)$	Slow for >5k nodes, use alternatives

Example - Fast centrality:

```
import networkx as nx

G = network.core_network

# FAST: Degree centrality
degree_cent = nx.degree_centrality(G)  # O(n+m) - instant

# SLOW: Betweenness centrality
# For large networks, sample first or use approximate algorithm
if G.number_of_nodes() < 1000:
    between_cent = nx.betweenness_centrality(G)
else:
    # Use approximate algorithm
    between_cent = nx.betweenness_centrality(G, k=100)  # Sample 100 nodes
```

Limit Algorithm Iterations

```
import networkx as nx

# PageRank with iteration limit
pagerank = nx.pagerank(
    network.core_network,
    max_iter=50,        # Limit iterations
    tol=1e-4            # Tolerance for convergence
)

# Community detection with resolution limit
from py3plex.algorithms.community_detection import community_louvain
communities = community_louvain.best_partition(
    network.core_network,
    resolution=1.0      # Adjust resolution parameter
)
```

Parallel Processing

Multi-Core Processing

Use joblib for parallel node/edge operations:

```
from joblib import Parallel, delayed
import networkx as nx

def compute_node_centrality(node, graph):
    """Compute centrality for a single node."""
    # Custom centrality computation
    neighbors = list(graph.neighbors(node))
    return node, len(neighbors)

# Parallel processing
```

(continues on next page)

(continued from previous page)

```
nodes = list(network.core_network.nodes())
results = Parallel(n_jobs=-1)(  # Use all cores
    delayed(compute_node_centrality)(node, network.core_network)
    for node in nodes
)

centralities = dict(results)
```

GPU Acceleration (Advanced)

For very large networks, use GPU acceleration:

```
# Install CuPy for GPU NumPy operations
pip install cupy-cuda11x  # Replace 11x with CUDA version
```

```
# Requires NVIDIA GPU with CUDA
try:
    import cupy as cp

    # Convert to GPU array
    adj_matrix = network.get_sparse_adjacency_matrix().toarray()
    gpu_adj = cp.array(adj_matrix)

    # GPU-accelerated matrix operations
    gpu_result = cp.dot(gpu_adj, gpu_adj)

    # Transfer back to CPU
    result = cp.asnumpy(gpu_result)

except ImportError:
    print("CuPy not available, using CPU")
```

Visualization Optimization

Reduce Visual Complexity

```
from py3plex.visualization.multilayer import draw_multilayer_default

# For large networks (>1000 nodes)
draw_multilayer_default(
    network.get_layers(),
    node_size=3,              # Tiny nodes
    labels=False,             # No labels
    edge_size=0.3,            # Thin edges
    alphalevel=0.2,           # Very transparent
    remove_isolated_nodes=True  # Remove disconnected
)
```

Save to File Instead of Display

```
import matplotlib.pyplot as plt

# Don't show interactively (faster)
fig, ax = plt.subplots(1, 1, figsize=(10, 8))
draw_multilayer_default(
    network.get_layers(),
    display=False,  # Don't show
    axis=ax
)

# Save directly to file
plt.savefig('network.png', dpi=150, bbox_inches='tight')
plt.close()  # Free memory
```

Use Lower Resolution

```
# For quick exploration, use low DPI
plt.figure(figsize=(8, 6), dpi=72)  # Low resolution

# For publications, use high DPI
plt.figure(figsize=(10, 8), dpi=300)  # High resolution
```

Memory Management

Monitor Memory Usage

```
import psutil
import os

def get_memory_usage():
    """Get current memory usage in MB."""
    process = psutil.Process(os.getpid())
    return process.memory_info().rss / 1024 / 1024

print(f"Memory before loading: {get_memory_usage():.1f} MB")

network = multinet.multi_layer_network()
network.load_network("large_network.csv", input_type="multiedgelist")

print(f"Memory after loading: {get_memory_usage():.1f} MB")
```

Free Memory When Done

```
# Delete network when no longer needed
del network
```

(continues on next page)

(continued from previous page)

```
# Force garbage collection
import gc
gc.collect()
```

Use Generators Instead of Lists

```
# BAD: Loads all nodes into memory
all_nodes = list(network.get_nodes())
for node in all_nodes:
    process(node)

# GOOD: Processes nodes one at a time
for node in network.get_nodes():
    process(node)
```

Batch Processing

Process Networks in Batches

For multiple networks:

```
import os
import gc

network_files = ["net1.csv", "net2.csv", "net3.csv", ...]

results = []
for i, file in enumerate(network_files):
    print(f"Processing {i+1}/{len(network_files)}: {file}")

    # Load network
    network = multinet.multi_layer_network()
    network.load_network(file, input_type="multiedgelist")

    # Compute statistics
    result = analyze_network(network)
    results.append(result)

    # Free memory
    del network
    gc.collect()

    # Save intermediate results every 10 networks
    if (i + 1) % 10 == 0:
        save_results(results, f"results_batch_{i+1}.json")
```

Benchmark Results

Performance Benchmarks (2025)

Tested on: Intel i7-10700K, 32GB RAM, Python 3.10

Table 13: Operation Performance

Operation	100 nodes	1k nodes	10k nodes	100k nodes
Load from CSV	<1s	<1s	2s	20s
Basic statistics	<1s	<1s	<1s	3s
Degree centrality	<1s	<1s	1s	10s
PageRank	<1s	<1s	2s	25s
Louvain communities	<1s	1s	5s	60s
Visualization (sparse)	<1s	2s	15s	N/A*

* Visualization not recommended for >10k nodes without sampling

Scaling Recommendations

Based on network size:

<1k nodes:

- Use any algorithms
- Full visualization
- No sampling needed

1k-10k nodes:

- Use sparse matrices
- Minimal visualization
- Sample for some algorithms

10k-100k nodes:

- Sparse matrices required
- Sample for visualization
- Use approximate algorithms
- Consider igraph for speed

>100k nodes:

- Use specialized tools (igraph, graph-tool, NetworKit)
- Sample heavily for Py3plex operations
- Focus on specific analyses

Alternative Tools for Scale

When Py3plex Isn't Enough

For networks >100k nodes, consider:

igraph (C-based, very fast):

```
import igraph as ig

# 10-100x faster for large networks
g = ig.Graph.Read_GraphML("large_network.graphml")
communities = g.community_multilevel()

# Export back to Py3plex if needed
# (via GraphML or edge list)
```

graph-tool (C++, fastest):

```
import graph_tool.all as gt

# Fastest for >1M edges
g = gt.load_graph("large_network.graphml")
communities = gt.community_structure.minimize_blockmodel_dl(g)
```

NetworKit (C++, parallel):

```
import networkit as nk

# Excellent for parallel algorithms
G = nk.readGraph("large_network.edgelist", nk.Format.EdgeList)
communities = nk.community.detectCommunities(G)
```

Quick Performance Checklist

Before Running on Large Network

```
[ ] Enable sparse matrices (automatic for most ops)
[ ] Sample network if >10k nodes
[ ] Choose efficient algorithms (degree > betweenness)
[ ] Limit visualization detail
[ ] Monitor memory usage
[ ] Use batch processing for multiple networks
[ ] Consider alternative tools if >100k nodes
```

Optimization Order

1. **Use sparse matrices** (biggest impact, usually automatic)
2. **Sample network** (if >10k nodes)
3. **Choose efficient algorithms** (avoid O(n^3) operations)
4. **Parallelize** (if multi-core available)
5. **GPU acceleration** (only if CUDA GPU available)

Next Steps

- ../user_guide/io_and_formats - Efficient data loading
- ../user_guide/visualization - Optimize visualizations
- *Docker Usage Guide* - Docker deployment for production

For performance issues, open an issue on GitHub Issues[24].

3.8.4 GUI: Web Interface for Network Exploration

> **Warning**
>
> **Development Mode Only:** The GUI is intended for local development and exploration. Do not expose it to the public internet without proper authentication and security hardening. See gui_deployment for production configuration.

The py3plex GUI provides a web-based interface for interactive multilayer network exploration.

Features:

- Load networks from various file formats without code
- Interactive, zoomable visualizations
- Compute statistics via point-and-click menus
- Detect communities and visualize them
- Export results for further analysis

This section covers:

- *Py3plex GUI* — Loading data, exploring networks, running analyses
- gui_deployment — Running locally, Docker, production setup
- gui_api_reference — Backend API documentation
- gui_architecture — How the GUI is built
- gui_testing — Testing infrastructure

Running the GUI Locally

Quick start (Docker - Recommended):

```
# Clone the repository
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex/gui

# Copy environment configuration
cp .env.example .env

# Start all services
make up

# Open browser to http://localhost:8080
```

[24] https://github.com/SkBlaz/py3plex/issues

Alternative: Install from source:

```
# Install with GUI extras
pip install py3plex[gui]

# Or install from source
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
pip install -e ".[gui]"

# Navigate to GUI directory and start services
cd gui
make up
```

The GUI will be available at `http://localhost:8080`. See gui_deployment for detailed setup instructions and troubleshooting.

GUI Actions vs py3plex APIs

The GUI provides a point-and-click interface for common py3plex operations. Here's how GUI actions map to Python API calls:

GUI Action	Equivalent Python API
Load Network (upload file)	`network = multinet.multi_layer_network().load_network(file_path, input_type="...")`
Compute Statistics → Basic Stats	`network.basic_stats()`
Compute Statistics → Layer Density	`from py3plex.algorithms.statistics import multilayer_statistics as mls mls.layer_density(network, layer_name)`
Detect Communities → Louvain	`from py3plex.algorithms.community_detection import community_louvain partition = community_louvain.best_partition(network.get_layers()[layer])`
Compute Centrality → Degree	`from py3plex.algorithms.multilayer_algorithms.centrality import MultilayerCentrality calc = MultilayerCentrality(network) degree = calc.overlapping_degree_centrality()`
Visualize Network	`from py3plex.visualization.multilayer import draw_multilayer_default draw_multilayer_default([network], display=True)`
Export → JSON	`network.save_network("output.json", output_type="json")`

This mapping helps you transition from GUI exploration to programmatic analysis.

When to Use the GUI

Use GUI for:

- Domain experts who prefer not to write Python
- Quick exploratory analysis of new datasets

- Demonstrations and teaching
- Collaborative work with mixed technical backgrounds

Use Python library for:

- Production pipelines and automation
- Reproducible analysis workflows
- Advanced customization
- Large-scale processing

Start with gui_deployment to run the GUI, then *Py3plex GUI* for usage.

3.8.5 Py3plex GUI

A production-ready web-based GUI for **py3plex** multilayer network analysis, running locally via Docker Compose.

Visual Overview

The Py3plex GUI provides an intuitive web interface for multilayer network analysis. Below are screenshots of the main interface pages:

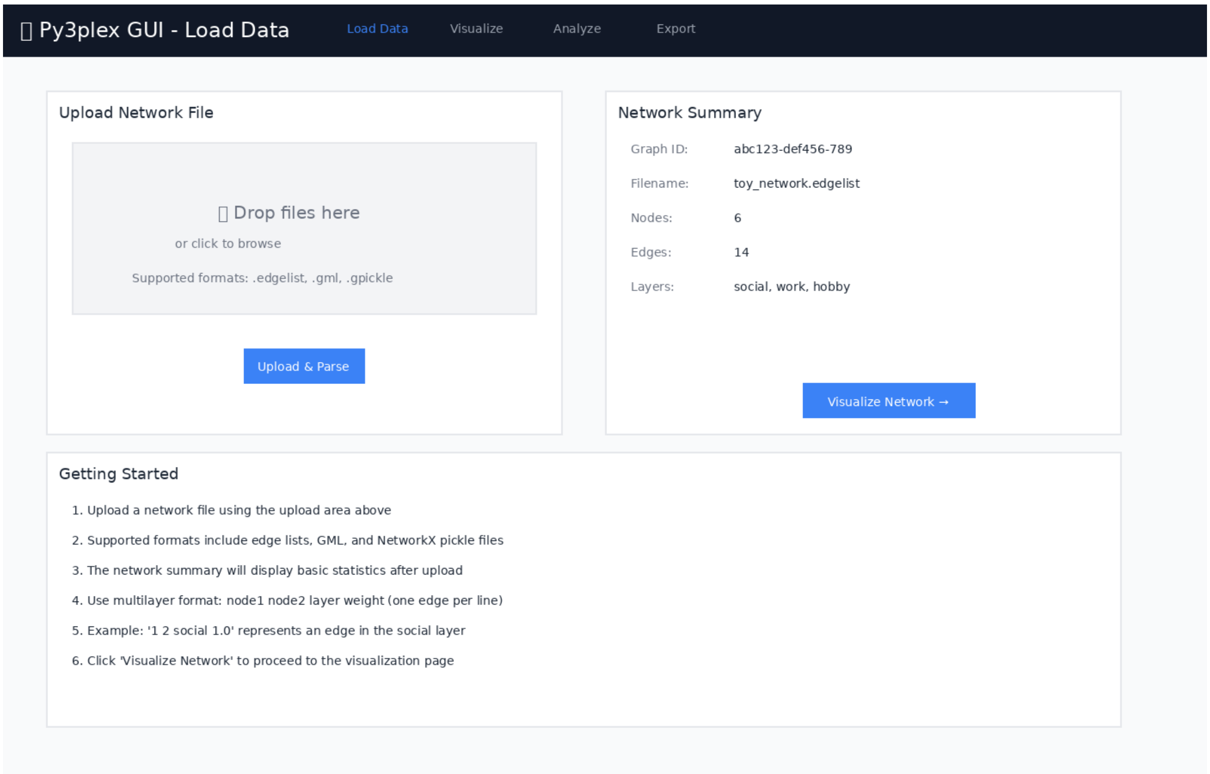


Figure 1: Load Data page for uploading network files in various formats

Quickstart

```
# Navigate to the gui directory
cd gui

# Copy environment configuration
cp .env.example .env

# Start all services (builds automatically)
make up

# Open in browser
#  Application: http://localhost:8080
# Data Job Monitor (Flower): http://localhost:5555
```

First time? The build takes ~3-5 minutes. Subsequent starts are instant.

That's it! The application will be running with:

- React + TypeScript frontend with hot reload
- FastAPI backend with py3plex integration
- Celery workers for async jobs
- Redis broker
- Nginx reverse proxy

Prerequisites

- Docker (>= 20.10)
- Docker Compose (>= 2.0)
- 4GB RAM minimum
- Ports available: 8080 (GUI), 5555 (Flower), 8000 (API), 6379 (Redis)

Architecture

```
┌─────────────────────────────────────────────────────────────┐
│                     Nginx (Port 8080)                        │
│  Reverse Proxy + Static Serving                             │
└─────────────────────────────────────────────────────────────┘
        │                         │
┌─────────────┐          ┌──────────────────────┐
│  Frontend   │          │      FastAPI         │
│React + Vite │          │  (Port 8000)         │
│  (Dev HMR)  │          │  py3plex wrapper     │
└─────────────┘         └──────────────────────┘
                                  │
                     ┌──────────────────────────────┐

            ┌─────────────┐              ┌─────────────┐
```

(continues on next page)

(continued from previous page)

```
      │   Redis     │                  │   Celery     │
      │ (Port 6379) │─────────────────│  Worker     │
      │   Broker    │                  │  + Flower    │
      └─────────────┘                 └─────────────┘
              │

      ──────────────────────────────────
      │  Shared Volume: /data             │
      │  - uploads/                       │
      │  - artifacts/                     │
      │  - workspaces/                    │
      └───────────────────────────────────┘
```

Key Components:

- **Frontend**: React 18 + Vite + TypeScript + Tailwind CSS
- **API**: FastAPI (Python 3.12) with py3plex integration
- **Worker**: Celery for long-running analysis jobs
- **Broker**: Redis for job queue
- **Proxy**: Nginx for unified access
- **Monitoring**: Flower dashboard for job inspection

Features

Data Loading

- Upload network files (.edgelist, .txt, .gml, .gpickle)
- Automatic format detection
- Multilayer network support
- Real-time file preview

Visualization

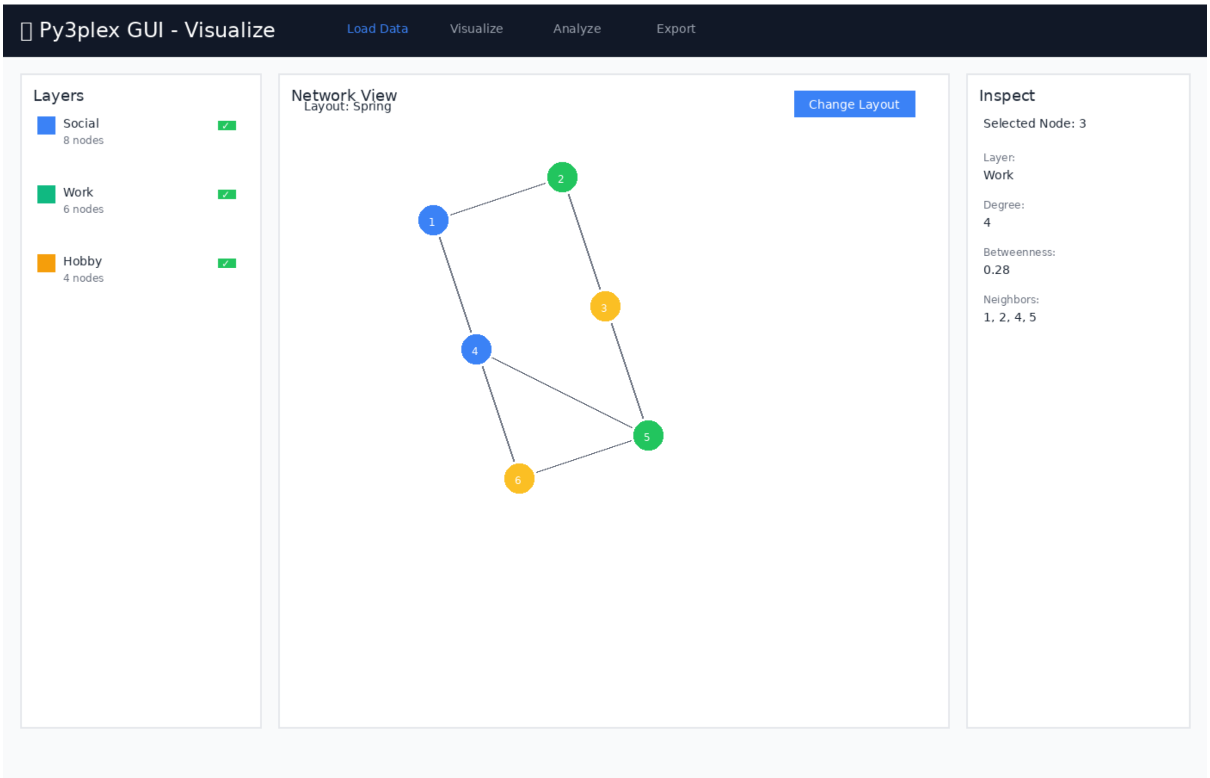


Figure 2: Network visualization with layer controls and node inspection panel

Features:

- Layer-centric view
- Interactive node/edge inspection
- Configurable layouts
- Position caching

Analysis (Async via Celery)

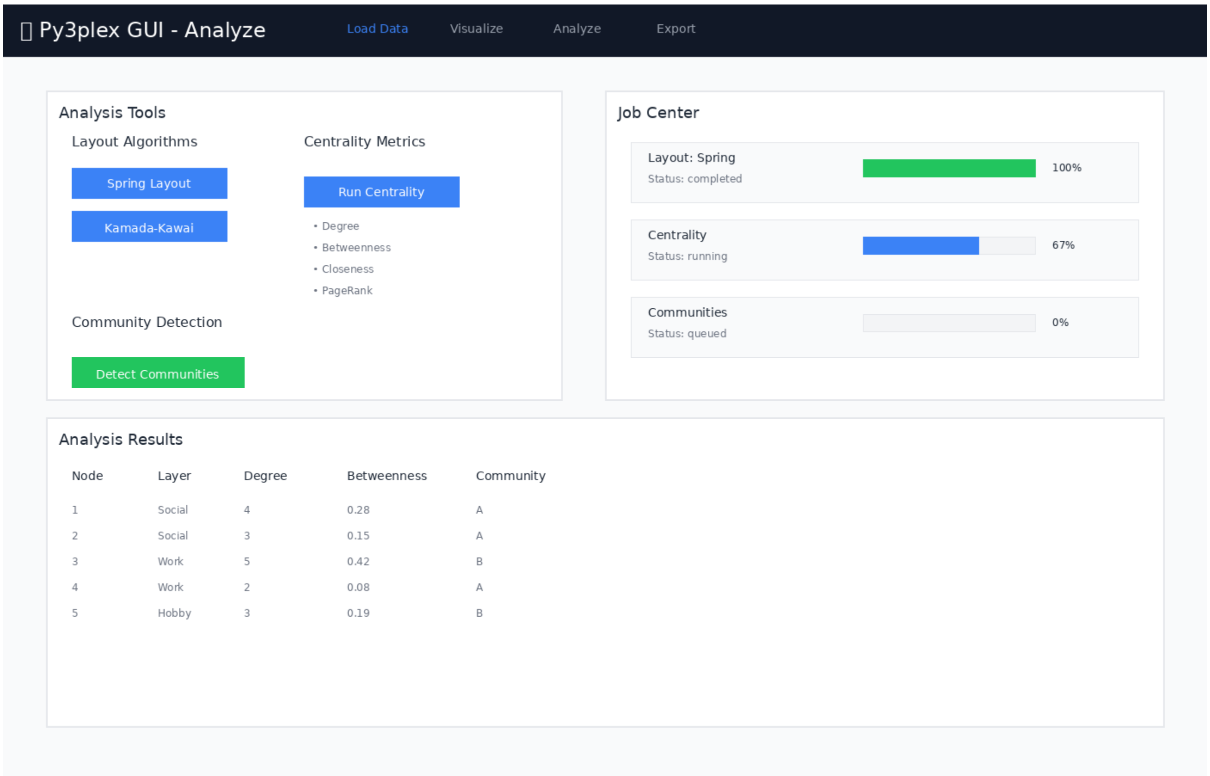


Figure 3: Analysis page with job controls and real-time progress tracking

Available Analysis Tools:

- **Layouts**: Spring, Kamada-Kawai, Circular, Random
- **Centrality**: Degree, Betweenness, Closeness, Eigenvector, PageRank
- **Community Detection**: Louvain, Label Propagation, Greedy Modularity
- Real-time progress tracking
- Result caching

Export

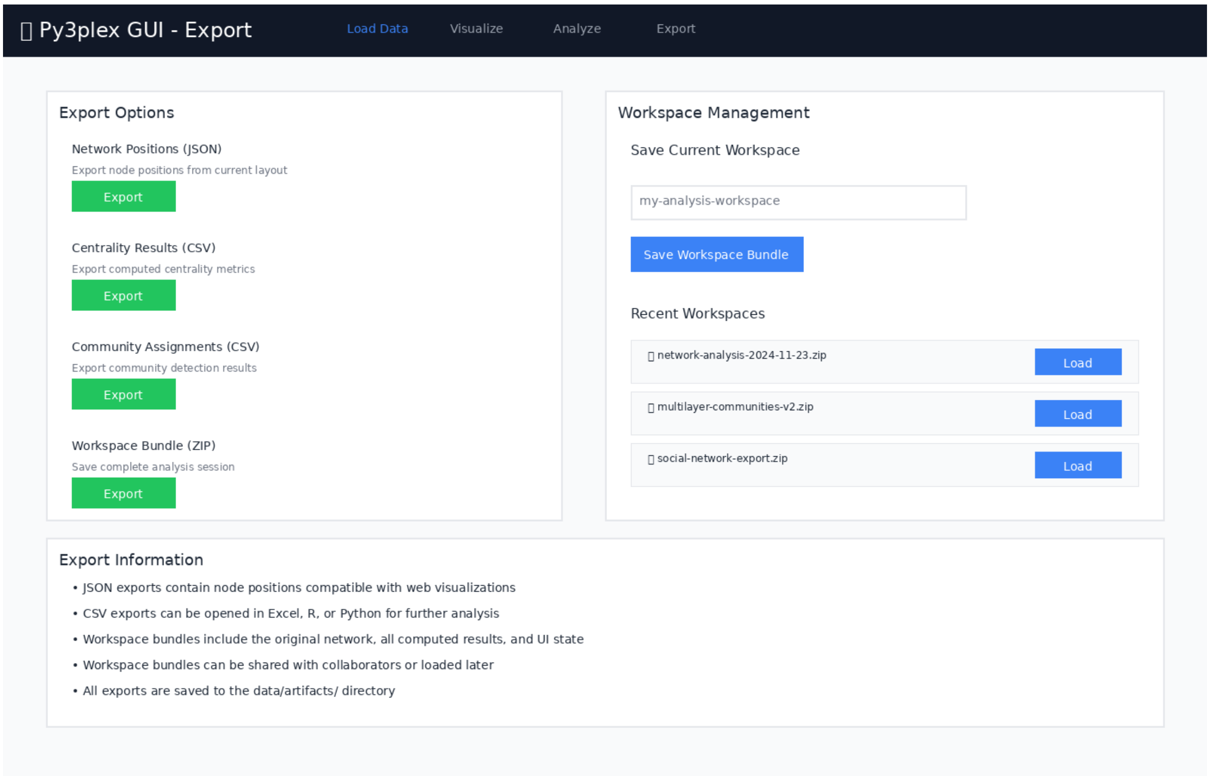


Figure 4: Export page for saving analysis results and workspace bundles

Export Options:

- CSV summaries (centrality, communities)
- JSON position data
- PNG snapshots (planned)
- Workspace bundles (data + state + results)

Workspace Bundles

Save and restore complete analysis sessions:

```
# Bundle includes:
# - Original network file
# - Computed layouts
# - Centrality results
# - Community assignments
# - UI view state
```

Job Monitoring

The GUI includes Flower dashboard for real-time monitoring of background jobs:

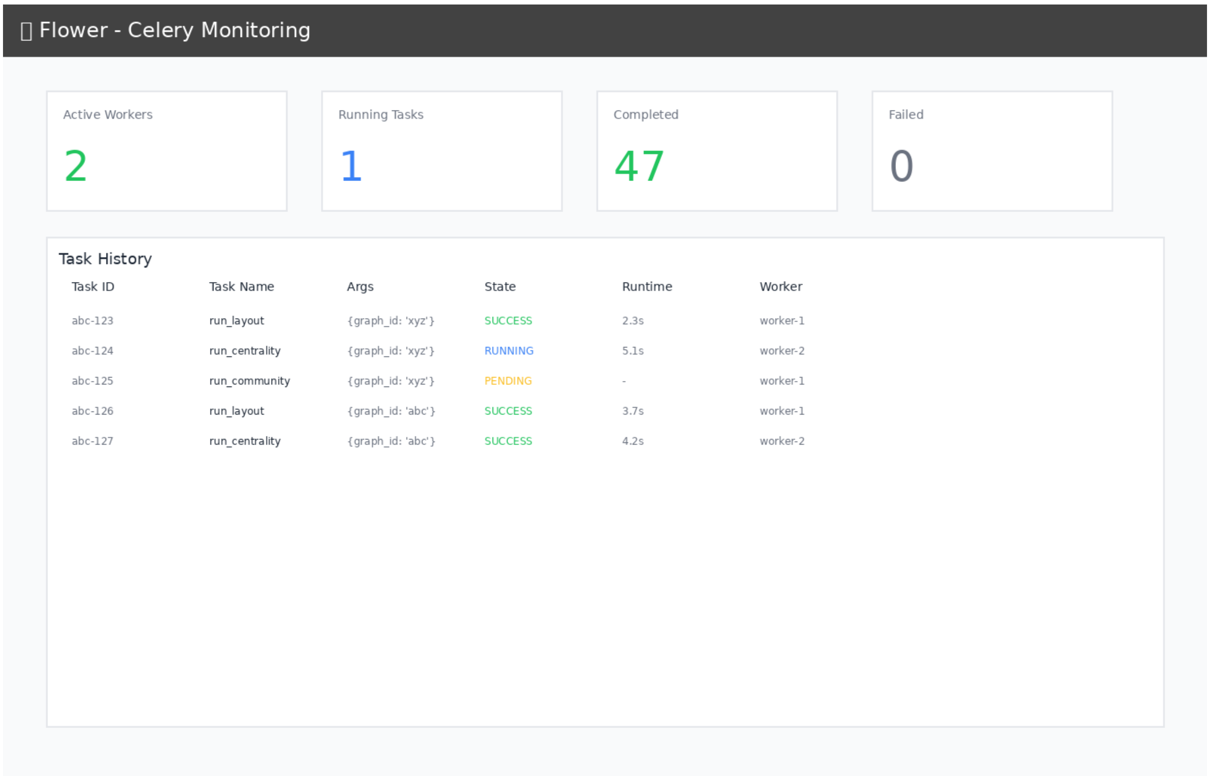


Figure 5: Flower dashboard showing task history and worker status

Monitoring Features:

- Track task execution in real-time
- View worker status and performance
- Inspect task history and results
- Monitor task queue and completion rates
- Access at http://localhost:5555 when running

Development Guide

Project Structure

```
gui/
├── docker-compose.yml          # Main orchestration
├── compose.gpu.yml             # GPU override (optional)
├── Makefile                    # Convenience commands
├── .env.example                # Configuration template
├── nginx/
│   └── nginx.conf              # Reverse proxy config
```

(continues on next page)

(continued from previous page)

```
├── api/
│   ├── Dockerfile.api          # API container
│   ├── pyproject.toml          # Python dependencies
│   └── app/
│       ├── main.py             # FastAPI app
│       ├── schemas.py          # Pydantic models
│       ├── deps.py             # DI & config
│       ├── routes/             # API endpoints
│       ├── services/           # Business logic (py3plex wrappers)
│       ├── workers/            # Celery tasks
│       └── utils/              # Helpers
├── worker/
│   └── Dockerfile              # Worker container
├── frontend/
│   ├── Dockerfile.frontend     # Frontend container
│   ├── package.json            # Node dependencies
│   ├── vite.config.ts          # Vite config
│   └── src/
│       ├── App.tsx             # Root component
│       ├── lib/api.ts          # API client
│       ├── pages/              # Page components
│       └── components/         # Reusable components
└── data/                       # Shared volume (gitignored)
    ├── uploads/
    ├── artifacts/
    └── workspaces/
```

Makefile Commands

```
make setup          # Copy .env.example to .env
make up             # Start all services (build if needed)
make down           # Stop and remove containers + volumes
make restart        # Restart all services
make build          # Rebuild Docker images
make logs           # Tail logs from all services

# Development helpers
make bash-api       # Shell into API container
make bash-worker    # Shell into worker container
make bash-frontend  # Shell into frontend container

# Testing
make test-api       # Run API tests
make e2e            # Run end-to-end tests (WIP)

# Cleanup
make clean          # Remove containers, volumes, and data
```

Local Development Against py3plex

The py3plex repository root is bind-mounted read-only into containers at `/workspace`:

```
# In Dockerfile.api
ARG PY3PLEX_PATH=/workspace
RUN pip install -e ${PY3PLEX_PATH}
```

Benefits:

- Changes to py3plex core reflect immediately (no rebuild)
- Editable install for development
- Isolated GUI code under `gui/`

Note: The mount is read-only to prevent accidental writes from containers.

Configuration

Environment Variables (.env)

```
# API
API_WORKERS=2              # Uvicorn workers
MAX_UPLOAD_MB=512          # Max file size

# Celery
CELERY_CONCURRENCY=2       # Worker threads
REDIS_URL=redis://redis:6379/0

# Frontend
VITE_API_URL=http://localhost:8080/api
```

GPU Support (Optional)

Enable NVIDIA GPU acceleration:

```
docker compose -f docker-compose.yml -f compose.gpu.yml up
```

Requirements:

- NVIDIA Docker runtime installed
- CUDA-compatible GPU

Data Formats

Accepted Input Formats

1. Edge List (.edgelist, .txt)

```
# Multilayer format: node1 node2 layer weight
1 2 social 1.0
1 3 work 1.0
2 3 hobby 0.5
```

(continues on next page)

(continued from previous page)

```
# Simple format: node1 node2
A B
B C
C D
```

2. **GML (.gml)**
   - Graph Modeling Language
   - Preserves attributes and metadata
3. **NetworkX Pickle (.gpickle)**
   - Native NetworkX serialization
   - Fastest for large graphs

Example Dataset

Try the included toy network:

```
# From host
curl -F "file=@gui/toy_network.edgelist" http://localhost:8080/api/upload

# Or use the Web UI at http://localhost:8080
```

Testing

Continuous Integration

GUI tests run automatically on GitHub Actions for any changes to the `gui/` directory:

```
# Workflow: .github/workflows/gui-tests.yml
- API unit tests (pytest)
- Integration tests (Docker Compose)
- Frontend build verification
```

Status:

[25] Tests include:

- Health endpoint validation
- File upload with toy network
- Layout job execution and completion
- Centrality computation
- Service health checks

API Tests

```
# Run inside API container
make bash-api
```

(continues on next page)

[25] https://github.com/SkBlaz/py3plex/actions/workflows/gui-tests.yml

(continued from previous page)

```
pytest ci/api-tests/ -v

# Or directly
docker compose exec api pytest ci/api-tests/ -v
```

Integration Tests

The CI workflow runs full integration tests:

```
# Upload and analyze a network
cd gui
curl -F "file=@toy_network.edgelist" http://localhost:8080/api/upload
# Run layout, centrality, and community detection jobs
```

Frontend Tests (WIP)

```
# Playwright smoke tests
make e2e
```

Manual Testing Workflow

1. **Upload** `toy_network.edgelist` via Web UI
2. **Visualize** the network (6 nodes, 14 edges, 3 layers)
3. **Analyze**:
   - Run Spring Layout
   - Compute Centrality (degree + betweenness)
   - Detect Communities (Louvain)
4. **Monitor** jobs at http://localhost:5555 (Flower)
5. **Export** workspace bundle

Production Deployment

Static Build (Recommended)

For production, serve pre-built frontend assets:

```
# Modify Dockerfile.frontend to use multi-stage build
FROM node:20-alpine AS builder
WORKDIR /app
COPY frontend/ .
RUN npm install && npm run build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx/nginx.conf /etc/nginx/nginx.conf
```

Update `docker-compose.yml`:

```
frontend:
  build:
    context: .
    dockerfile: Dockerfile.frontend.prod
  # Remove dev server volume mounts
```

Security Considerations

- Set CORS `allow_origins` to specific domains (not `*`)
- Add authentication/authorization middleware
- Use HTTPS (add TLS termination in Nginx)
- Validate file uploads strictly
- Set resource limits per user/job
- Enable Celery task rate limiting

Current Status: Development mode - suitable for local use only. Do not expose to public internet without proper security hardening.

Troubleshooting

Issue: Port already in use

```
# Find process using port 8080
lsof -ti:8080 | xargs kill -9

# Or change port in docker-compose.yml
ports:
  - "8081:80"  # Use 8081 instead
```

Issue: Permission denied on /data

```
# Fix volume permissions
sudo chown -R $(whoami):$(whoami) gui/data/
```

Issue: py3plex import error in containers

```
# Verify mount
docker compose exec api ls -la /workspace

# Reinstall if needed
docker compose exec api pip install -e /workspace
```

Issue: Frontend can't reach API

Check Nginx proxy config:

```
docker compose exec nginx cat /etc/nginx/nginx.conf

# Test API directly
curl http://localhost:8000/api/health
```

API Documentation

Interactive docs available at:

- **Swagger UI**: http://localhost:8080/api/docs
- **ReDoc**: http://localhost:8080/api/redoc

Key Endpoints

```
POST   /api/upload                              # Upload network file
GET    /api/graphs/{id}/summary                 # Graph statistics
POST   /api/graphs/{id}/layout                  # Compute layout (async)
POST   /api/graphs/{id}/analysis/centrality # Compute centrality (async)
POST   /api/graphs/{id}/analysis/community  # Detect communities (async)
GET    /api/jobs/{id}                           # Poll job status
POST   /api/workspaces/save                     # Save workspace bundle
```

Example: Run Analysis

```
# 1. Upload
GRAPH_ID=$(curl -F "file=@toy_network.edgelist" \
  http://localhost:8080/api/upload | jq -r .graph_id)

# 2. Start layout job
JOB_ID=$(curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"algorithm":"spring","seed":42,"dimensions":2}' \
  http://localhost:8080/api/graphs/$GRAPH_ID/layout | jq -r .job_id)

# 3. Poll job
curl http://localhost:8080/api/jobs/$JOB_ID

# 4. Get positions
curl http://localhost:8080/api/graphs/$GRAPH_ID/positions
```

Contributing

This GUI is part of the py3plex[26] project.

Guidelines:

- Keep all GUI code under `gui/`
- Do not modify py3plex core
- Follow existing code style (Black, Ruff for Python; ESLint for TypeScript)

[26] https://github.com/SkBlaz/py3plex

- Add tests for new features
- Update documentation with new features

License

This GUI follows the py3plex repository license:

- **GUI code** (under `gui/`): BSD-3-Clause (same as py3plex)
- **Dependencies**: See individual package licenses

Acknowledgments

Built with:

- py3plex[27] - Multilayer network analysis
- FastAPI[28] - Modern Python web framework
- Celery[29] - Distributed task queue
- React[30] - UI library
- Vite[31] - Frontend build tool
- Tailwind CSS[32] - Utility-first CSS

Support

- **Issues**: https://github.com/SkBlaz/py3plex/issues
- **Docs**: https://skblaz.github.io/py3plex/
- **py3plex Paper**: Applied Network Science 2019[33]

3.8.6 Developer & Contributor Docs

This section covers development setup, code architecture, and contribution guidelines.

This section covers:

- *Development Guide* — Development environment, running tests, submitting changes
- *Architecture and Design* — How py3plex is organized internally
- repo_layout — Directory structure and file locations
- contributing — Bug reports, code style, review process

Types of Contributions

- **Bug reports** — Clear issue reports help fix problems
- **Documentation** — Improvements to explanations and examples
- **Code fixes** — Bug fixes and performance improvements

[27] https://github.com/SkBlaz/py3plex
[28] https://fastapi.tiangolo.com/
[29] https://docs.celeryproject.org/
[30] https://react.dev/
[31] https://vitejs.dev/
[32] https://tailwindcss.com/
[33] https://doi.org/10.1007/s41109-019-0203-7

- **New features** — Algorithms, visualizations, I/O formats
- **Examples** — Tutorials for specific use cases
- **Community support** — Helping others in issues and discussions

Getting Started

1. Read *Development Guide* to set up your environment
2. Browse GitHub issues labeled "good first issue"
3. Start small—a documentation fix or simple bug is fine
4. Ask questions when stuck

See contributing for the complete process.

3.8.7 Development Guide

This guide covers **development workflows**, **Makefile commands**, **testing**, and **contributing** to py3plex.

Getting Started

Clone the repository and install in **development mode**:

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Setup development environment
make setup

# Install package in editable mode with dev dependencies
make dev-install
```

Makefile Commands

The project includes a **production-grade Makefile** for streamlined development:

```
# View all available commands
make help

# Environment setup
make setup          # Create virtual environment
make dev-install    # Install in editable mode

# Code quality
make format         # Auto-format code
make lint           # Run linters

# Testing
make test           # Run tests with coverage
make coverage       # Open coverage report

```

(continues on next page)

(continued from previous page)

```
# Documentation
make docs            # Build Sphinx documentation

# Build & publish
make clean           # Remove build artifacts
make build           # Build distributions
make publish         # Publish to PyPI

# CI
make ci              # Run full CI checks
```

Key Features:

- **Smart tool detection**: Uses `.venv/bin/` tools if available, otherwise falls back to **global tools**
- **Colorized output**: Clear **success/warning/error** messages
- **Cross-platform**: Works on **Linux** and **macOS**

Testing

Quick testing:

```
python run_tests.py
```

Development testing with pytest:

```
# Run all tests
pytest

# Run with coverage
pytest --cov=py3plex --cov-report=html

# Run specific test file
pytest tests/test_core.py

# Using Makefile (recommended)
make test
```

Code Quality

Tools configured in `pyproject.toml`:

- **Black**: Code formatting
- **Ruff**: Fast linting
- **isort**: Import sorting
- **Mypy**: Type checking
- **Pytest**: Testing with coverage

Before committing:

```
make format  # Auto-format code
make lint    # Check code quality
make test    # Run tests
make ci      # Run all checks
```

Contributing

We welcome contributions! Here's how:

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Run tests and linting: `make ci`
5. Commit with clear messages
6. Push to your fork
7. Open a Pull Request

Code Guidelines:

- Follow existing code style (enforced by Black and Ruff)
- Add tests for new features
- Update documentation as needed
- Keep changes focused and atomic
- Write clear commit messages

Documentation

Build Sphinx documentation:

```
# Using Makefile (recommended)
make docs

# Manual build
cd docfiles
sphinx-build -b html . _build/html
```

The built documentation will be in `docfiles/_build/html/`.

Documentation Style Guide

When contributing to documentation, follow these conventions for consistency:

Naming Conventions:

- Use "Quickstart" (one word) instead of "Quick Start" or "Quickstart Guide"
- Use "multilayer" (no hyphen) consistently, not "multi-layer" or "multi layer"
- Use short, direct section titles without boilerplate

Tone and Structure:

- Write first paragraphs that directly explain why the reader is on this page

- Avoid boilerplate like "This chapter will introduce…" or "In this section, you will learn…"
- Start with concrete examples rather than abstract explanations
- Provide 2-3 explicit reading paths for different audiences (new users, experienced users, advanced users)

Code Examples:

All code examples must include:

1. **Imports** — Always show required imports
2. **Network creation** — Build a tiny example network from scratch or via helper
3. **Core operation** — Run one focused operation
4. **Expected output** — Show what the user should see

Example structure:

```
# 1. Imports
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls

# 2. Create network
network = multinet.multi_layer_network(directed=False)
network.add_nodes([{'source': 'A', 'type': 'layer1'}])
network.add_edges([
    {'source': 'A', 'target': 'B', 'source_type': 'layer1', 'target_type':
↪'layer1'}
])

# 3. Run operation
density = mls.layer_density(network, 'layer1')
print(f"Density: {density:.3f}")
```

Expected output:

```
Density: 1.000
```

Marking Experimental APIs:

Use warning directives for experimental or unstable features:

```
.. warning::

   **Experimental Feature:** This API is under active development and may␣
↪change in future versions.
```

Cross-references:

- Link to related sections using `:doc:` for documentation pages
- Link to algorithm details using `:ref:` for labeled sections
- When documenting algorithms, link to the Algorithm Roadmap as the conceptual entry point

Version Labels:

- Use consistent version strings throughout (e.g., "py3plex 1.0.0 documentation")
- Update version in `conf.py` and `pyproject.toml` together

Resources

- **Repository**: https://github.com/SkBlaz/py3plex
- **Documentation**: https://skblaz.github.io/py3plex/
- **Issues**: https://github.com/SkBlaz/py3plex/issues
- **Examples**: https://github.com/SkBlaz/Py3Plex/tree/master/examples

3.8.8 Architecture and Design

This document describes the **system architecture**, **design patterns**, and **extension points** of py3plex.

System Overview

py3plex is built as a **modular, layered architecture** with clear separation of concerns:

```
┌─────────────────────────────────────────────────────┐
│         High-Level Interfaces (Wrappers)            │
│  node2vec_embedding, benchmark_nodes                │
└─────────────────────────────────────────────────────┘
                         ↓
┌─────────────────────────────────────────────────────┐
│              Algorithms Layer                       │
│  ┌──────────────┬────────────────┬──────────────┐   │
│  │ Community    │ Statistics     │ Multilayer   │   │
│  │ Detection    │                │ Algorithms   │   │
│  └──────────────┴────────────────┴──────────────┘   │
└─────────────────────────────────────────────────────┘
                         ↓
┌─────────────────────────────────────────────────────┐
│            Visualization Layer                      │
│  ┌──────────────┬────────────────┬──────────────┐   │
│  │ Multilayer   │ Drawing        │ Layout       │   │
│  │ Plots        │ Machinery      │ Algorithms   │   │
│  └──────────────┴────────────────┴──────────────┘   │
└─────────────────────────────────────────────────────┘
                         ↓
┌─────────────────────────────────────────────────────┐
│                Core Layer                           │
│  ┌──────────────┬────────────────┬──────────────┐   │
│  │ multinet     │ Parsers        │ Converters   │   │
│  │ (MultiLayer  │                │              │   │
│  │  Network)    │                │              │   │
│  └──────────────┴────────────────┴──────────────┘   │
└─────────────────────────────────────────────────────┘
                         ↓
┌─────────────────────────────────────────────────────┐
```

(continues on next page)

(continued from previous page)

```
│          NetworkX Foundation                              │
│  MultiDiGraph, MultiGraph, Algorithms                     │
└──────────────────────────────────────────────────────────┘
```

Architectural Layers

Core Layer

Purpose: Fundamental data structures and I/O operations

Key Components:

- `multinet.py` - The `multi_layer_network` **class**
- `parsers.py` - **Input/output** for various formats
- `converters.py` - **Format conversion** utilities
- `random_generators.py` - **Random network** generators
- `HINMINE/` - **Heterogeneous network decomposition**

Responsibilities:

- **Network construction** and manipulation
- **File I/O** (GraphML, GML, GEXF, edge lists, etc.)
- **Layer management**
- **Matrix representations** (adjacency, supra-adjacency)
- **NetworkX integration**

Design Pattern: Facade Pattern - `multi_layer_network` provides a unified interface to complex NetworkX operations

Algorithms Layer

Purpose: Network analysis algorithms optimized for multilayer networks

Key Components:

- `community_detection/` - **Community detection** algorithms
- `statistics/` - **Network statistics** and metrics
- `multilayer_algorithms/` - **Multilayer-specific** algorithms
- `node_ranking/` - **Centrality** and ranking measures
- `general/` - **General-purpose** algorithms (random walks, etc.)

Responsibilities:

- **Community detection** (Louvain, Infomap, Label Propagation)
- **Statistical analysis** (17+ multilayer metrics)
- **Centrality computation** (degree, betweenness, PageRank, etc.)
- **Random walks** and embeddings
- **Network decomposition**

Design Pattern: Strategy Pattern - Different algorithms implement common interfaces

Visualization Layer

Purpose: Network plotting and rendering

Key Components:

- `multilayer.py` - High-level multilayer plotting
- `drawing_machinery.py` - Core drawing primitives
- `layout_algorithms.py` - Layout computation
- `colors.py` - Color scheme generators
- `fa2/` - ForceAtlas2 layout

Responsibilities:

- Diagonal projection plots
- Force-directed layouts
- Matrix visualizations
- Color mapping and legends
- Interactive plots (via Plotly)

Design Pattern: Template Method Pattern - Layout algorithms follow a common template

Wrappers Layer

Purpose: High-level interfaces for common workflows

Key Components:

- `node2vec_embedding.py` - Node2Vec embedding generation
- `benchmark_nodes.py` - Node classification benchmarking

Responsibilities:

- Simplified interfaces for complex workflows
- Integration with external tools
- Benchmarking and evaluation

Design Pattern: Facade Pattern - Simplify complex multi-step operations

Core Data Structure

The multi_layer_network Class

Central to py3plex, this class manages multilayer network state:

```
class multi_layer_network:
    def __init__(self, directed=True, label_delimiter="---", coupling_
↪weight=1.0):
        self.core_network = nx.MultiDiGraph() if directed else nx.MultiGraph()
        self.layer_name_map = {}  # Bidirectional mapping
        self.label_delimiter = label_delimiter
        self.coupling_weight = coupling_weight
        self.embedding = None
        self.labels = None
```

Key Attributes:

- `core_network` - Underlying NetworkX graph
- `layer_name_map` - Maps layer names to integer IDs
- `label_delimiter` - Separator for node-layer encoding (default: "—")
- `coupling_weight` - Default weight for inter-layer edges
- `embedding` - Cached node embedding matrix
- `labels` - Node classification labels

Encoding Scheme:

Nodes are encoded as `"{node_id}{delimiter}{layer_id}"`

Example: Node 'A' in layer 'social' becomes `"A---social"`

This allows NetworkX to handle multilayer structure transparently.

Design Patterns

Facade Pattern

Used in: `multi_layer_network`, wrappers

Purpose: Provide simplified interface to complex subsystems

```
# Complex underlying operations hidden behind simple interface
network = multinet.multi_layer_network()
network.add_edges(edges, input_type='list')  # Handles parsing, encoding,␣
↪validation
network.basic_stats()  # Aggregates multiple NetworkX calls
```

Strategy Pattern

Used in: Algorithms, layout computation

Purpose: Interchangeable algorithms following common interface

```
# Different community detection strategies
def detect_communities(network, method='louvain'):
    strategies = {
        'louvain': community_louvain.best_partition,
        'infomap': community_wrapper.infomap_communities,
        'label_prop': label_propagation.propagate
    }
    return strategies[method](network.core_network)
```

Template Method Pattern

Used in: Visualization, layout algorithms

Purpose: Define algorithm skeleton, allow customization in subclasses

```
class LayoutAlgorithm:
    def compute(self, graph):
```

(continues on next page)

(continued from previous page)

```
        self.initialize(graph)
        self.iterate()
        return self.finalize()

    def initialize(self, graph):
        raise NotImplementedError

    def iterate(self):
        raise NotImplementedError

    def finalize(self):
        raise NotImplementedError
```

Dependency Injection

Used in: Configuration, algorithm parameters

Purpose: Inject dependencies rather than hard-coding

```
# Configuration injected rather than hard-coded
from py3plex.config import DEFAULT_COLORS, LAYOUT_PARAMS

def draw_network(network, colors=None, layout_params=None):
    colors = colors or DEFAULT_COLORS
    layout_params = layout_params or LAYOUT_PARAMS
    # Use injected configuration
```

Data Flow

Typical Workflow

1. **Input:** Load or create network

```
network = multinet.multi_layer_network()
network.load_network("data.graphml", input_type="graphml")
```

2. **Processing:** Apply algorithms

```
communities = community_louvain.best_partition(network.core_network)
centrality = calc.multilayer_degree_centrality(network)
```

3. **Analysis:** Compute statistics

```
density = mls.layer_density(network, 'layer1')
correlation = mls.inter_layer_degree_correlation(network, 'L1', 'L2')
```

4. **Visualization:** Render results

```
draw_multilayer_default([network], display=True)
```

5. **Output:** Export results

```
network.save_network("output.graphml", output_type="graphml")
```

State Management

Immutable Operations: Most algorithms don't modify the network

```
# These don't modify the network
centrality = calc.multilayer_degree_centrality(network)
communities = community_louvain.best_partition(network.core_network)
```

Mutable Operations: Some operations modify network state

```
# These modify the network
network.add_edges(new_edges, input_type='list')
network.aggregate_layers(['L1', 'L2'], 'combined')
```

Extension Points

Custom Algorithms

Add new algorithms by following existing patterns:

```
# py3plex/algorithms/my_module/my_algorithm.py
from py3plex.core.multinet import multi_layer_network

def my_centrality(network: multi_layer_network) -> dict:
    """
    Custom centrality measure.

    Parameters
    ----------
    network : multi_layer_network
        Input network

    Returns
    -------
    dict
        Node centrality scores
    """
    G = network.core_network
    centrality = {}

    for node in G.nodes():
        # Implement custom logic
        centrality[node] = compute_score(node, G)

    return centrality
```

Custom Visualizations

Create custom plots using drawing machinery:

```
from py3plex.visualization import drawing_machinery as dm
import matplotlib.pyplot as plt

def my_custom_plot(network):
    """Custom visualization."""
    fig, ax = plt.subplots(figsize=(10, 8))

    # Compute layout
    pos = dm.compute_layout(network.core_network, 'force')

    # Draw elements
    dm.draw_nodes(ax, network.core_network, pos, node_size=50)
    dm.draw_edges(ax, network.core_network, pos, edge_width=1)
    dm.draw_labels(ax, pos, labels=network.get_node_labels())

    plt.show()
```

Custom Parsers

Add support for new file formats:

```
# py3plex/core/parsers.py
def parse_my_format(input_file, **kwargs):
    """
    Parse custom file format.

    Parameters
    ----------
    input_file : str
        Path to input file

    Returns
    -------
    multi_layer_network
        Parsed network
    """
    network = multi_layer_network()

    with open(input_file, 'r') as f:
        for line in f:
            # Parse line and add to network
            pass

    return network
```

Configuration System

Centralized Configuration

`py3plex/config.py` provides centralized configuration:

```
# Default color palettes (8 options including colorblind-safe)
DEFAULT_COLORS = 'Set1'
COLORBLIND_SAFE = 'colorblind'

# Visualization defaults
DEFAULT_NODE_SIZE = 20
DEFAULT_EDGE_WIDTH = 1.0
DEFAULT_ALPHA = 0.7

# Layout parameters
LAYOUT_PARAMS = {
    'force': {'iterations': 500, 'optimal_distance': 1.0},
    'fa2': {'iterations': 1000, 'gravity': 1.0}
}

# Performance settings
SPARSE_THRESHOLD = 1000  # Use sparse matrices above this node count
MEMORY_WARNING_THRESHOLD = 10000  # Warn for large dense matrices
```

Usage:

```
from py3plex.config import DEFAULT_COLORS, LAYOUT_PARAMS

colors = DEFAULT_COLORS
iterations = LAYOUT_PARAMS['force']['iterations']
```

Testing Architecture

Test Organization

```
tests/
├── test_core_functionality.py      # Core data structure tests
├── test_multilayer_*.py            # Multilayer algorithm tests
├── test_random_walks.py            # Random walk tests
├── test_io_*.py                    # I/O and parsing tests
├── test_config_api.py              # Configuration tests
└── test_utils.py                   # Utility function tests
```

Test Patterns

Unit Tests: Test individual functions in isolation

```
def test_layer_density():
    network = create_test_network()
    density = mls.layer_density(network, 'layer1')
    assert 0 <= density <= 1
```

Integration Tests: Test workflows across modules

```
def test_community_detection_workflow():
    network = load_network("test_data.graphml")
    communities = community_louvain.best_partition(network.core_network)
    assert len(communities) > 0
```

Property-Based Tests: Test invariants

```
def test_centrality_normalization():
    network = create_random_network()
    centrality = calc.multilayer_degree_centrality(network)
    # Centrality values should be normalized
    assert all(0 <= v <= 1 for v in centrality.values())
```

Performance Considerations

Lazy Evaluation

Expensive operations are computed on-demand:

```
class multi_layer_network:
    @property
    def supra_adjacency(self):
        if self._supra_adj_cache is None:
            self._supra_adj_cache = self._compute_supra_adjacency()
        return self._supra_adj_cache
```

Sparse Matrices

Use sparse representations for large networks:

```
def get_supra_adjacency_matrix(self, sparse=True):
    if sparse or len(self.get_nodes()) > SPARSE_THRESHOLD:
        return scipy.sparse.csr_matrix(adj)
    return np.array(adj)
```

Vectorization

Prefer NumPy vectorized operations:

```
# Bad: Python loop
degrees = [sum(1 for _ in G.neighbors(node)) for node in nodes]

# Good: Vectorized
degrees = np.array(list(dict(G.degree()).values()))
```

Logging Infrastructure

Centralized Logging

`py3plex/logging_config.py` provides structured logging:

```
import logging
from py3plex.logging_config import get_logger

logger = get_logger(__name__)

logger.info("Processing network with %d nodes", num_nodes)
logger.warning("Large network detected, using sparse matrices")
logger.error("Invalid layer: %s", layer_name)
```

Log Levels

- **DEBUG:** Detailed diagnostic information
- **INFO:** General informational messages
- **WARNING:** Warning messages (e.g., performance concerns)
- **ERROR:** Error messages
- **CRITICAL:** Critical errors

Error Handling

Custom Exceptions

`py3plex/exceptions.py` defines domain-specific exceptions:

```
class NetworkError(Exception):
    """Base exception for network errors."""
    pass

class LayerNotFoundError(NetworkError):
    """Raised when layer doesn't exist."""
    pass

class InvalidFormatError(NetworkError):
    """Raised when file format is invalid."""
    pass
```

Usage:

```
from py3plex.exceptions import LayerNotFoundError

def get_layer(self, layer_name):
    if layer_name not in self.layer_name_map:
        raise LayerNotFoundError(f"Layer '{layer_name}' not found")
    return self.layer_name_map[layer_name]
```

Future Architecture

Planned Improvements

1. **Backend Registry:** Support for igraph, cugraph backends
2. **Streaming API:** Process networks larger than memory
3. **Distributed Computing:** Dask/Ray integration for large-scale analysis
4. **Plugin System:** Easy addition of third-party algorithms
5. **Type System:** Full type hints coverage (currently 65%)

See Also

- contributing - Contributing guidelines
- development - Development workflow
- core - Core API documentation

CHAPTER

FOUR

CITATION

If you use py3plex in your research, please cite:

```
@Article{Skrlj2019,
  author={Skrlj, Blaz and Kralj, Jan and Lavrac, Nada},
  title={Py3plex toolkit for visualization and analysis of multilayer networks},
  journal={Applied Network Science},
  year={2019},
  volume={4},
  number={1},
  pages={94},
  doi={10.1007/s41109-019-0203-7}
}
```

CHAPTER

FIVE

INDICES AND TABLES

- genindex
- modindex
- search

PYTHON MODULE INDEX

p

INDEX

Symbols

A

B

D

E

H

J

K

M

N

O

P

S

U